云数据库 GaussDB
8.102
特性指南

文档版本 01

发布日期 2024-04-30



华为云计算技术有限公司

版权所有 © 华为云计算技术有限公司 2024。 保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明


和其他华为商标均为华为技术有限公司的商标。
本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为云计算技术有限公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为云计算技术有限公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

华为云计算技术有限公司

地址： 贵州省贵安新区黔中大道交兴功路华为云数据中心 邮编：550029

网址： https://www.huaweicloud.com/

目 录

1 分布式版 1
1.1 物化视图 1
1.1.1 全量物化视图 1
1.1.1.1 概述 1
1.1.1.2 支持和约束 1
1.1.1.3 使用 2
1.1.2 增量物化视图 3
1.1.2.1 概述 3
1.1.2.2 支持和约束 3
1.1.2.3 使用 4
1.2 设置密态等值查询 5
1.2.1 密态等值查询概述 5
1.2.2 使用 gsql 操作密态数据库 8
1.2.3 使用 JDBC 操作密态数据库 10
1.2.4 前向兼容与安全增强 15
1.2.5 密态支持函数/存储过程 18
1.3 透明数据加密 19
1.4 设置账本数据库 22
1.4.1 账本数据库概述 22
1.4.2 查看账本历史操作记录 24
1.4.3 校验账本数据一致性 26
1.4.4 归档账本数据库 27
1.4.5 修复账本数据库 28
1.5 基于标签的强制访问控制 30
1.5.1 定义安全标签 31
1.5.2 应用安全标签 31
1.5.3 基于标签的强制访问控制检查 32
1.6 逻辑复制 33
1.6.1 逻辑解码 34
1.6.1.1 逻辑解码概述 34
1.6.1.2 逻辑解码选项 37
1.6.1.3 使用 SQL 函数接口进行逻辑解码 44
1.6.1.4 使用流式解码实现数据逻辑复制 45

 版权所有 © 华为云计算技术有限公司

1.7 分区表 45
1.7.1 大容量数据库 45
1.7.1.1 大容量数据库背景介绍 45
1.7.1.2 表分区技术 45
1.7.1.3 数据分区查找优化 46
1.7.1.4 数据分区运维管理 47
1.7.2 分区表介绍 47
1.7.2.1 基本概念 47
1.7.2.1.1 分区表（母表） 47
1.7.2.1.2 分区（分区子表、子分区） 48
1.7.2.1.3 分区键 49
1.7.2.2 分区策略 49
1.7.2.2.1 范围分区 49
1.7.2.2.2 哈希分区 51
1.7.2.2.3 列表分区 52
1.7.2.2.4 分区表对导入操作的性能影响 52
1.7.2.3 分区基本使用 53
1.7.2.3.1 创建分区表 53
1.7.2.3.2 使用和管理分区表 55
1.7.2.3.3 分区表 DQL/DML 55
1.7.3 分区表查询优化 57
1.7.3.1 分区剪枝 57
1.7.3.1.1 分区表静态剪枝 57
1.7.3.1.2 分区表动态剪枝 62
1.7.3.2 分区索引 69
1.7.3.3 分区表统计信息 72
1.7.3.3.1 级联收集统计信息 72
1.7.3.3.2 分区级统计信息 75
1.7.4 分区表运维管理 79
1.7.4.1 新增分区 79
1.7.4.1.1 向范围分区表新增分区 80
1.7.4.1.2 向列表分区表新增分区 80
1.7.4.2 删除分区 80
1.7.4.3 交换分区 81
1.7.4.4 清空分区 82
1.7.4.5 分割分区 83
1.7.4.5.1 对范围分区表分割分区 83
1.7.4.5.2 对列表分区表分割分区 83
1.7.4.6 合并分区 84
1.7.4.7 移动分区 85
1.7.4.8 重命名分区 85
1.7.4.8.1 对分区表重命名分区 85

 版权所有 © 华为云计算技术有限公司

1.7.4.8.2 对 Local 索引重命名索引分区 85
1.7.4.9 分区表行迁移 85
1.7.4.10 分区表索引重建/不可用 86
1.7.4.10.1 索引重建/不可用 86
1.7.4.10.2 Local 索引分区重建/不可用 86
1.7.5 分区表系统视图&DFX 87
1.7.5.1 分区表相关系统视图 87
1.7.5.2 分区表相关内置工具函数 87
1.8 存储引擎 90
1.8.1 存储引擎体系架构 90
1.8.1.1 存储引擎体系架构概述 90
1.8.1.1.1 静态编译架构 90
1.8.1.1.2 通用数据库服务层 91
1.8.1.2 设置存储引擎 92
1.8.1.3 存储引擎更新说明 93
1.8.1.3.1 GaussDB 内核 505 版本 93
1.8.1.3.2 GaussDB 内核 503 版本 93
1.8.1.3.3 GaussDB 内核 R2 版本 93
1.8.2 Astore 存储引擎 94
1.8.2.1 Astore 简介 94
1.8.3 Ustore 存储引擎 94
1.8.3.1 Ustore 简介 94
1.8.3.1.1 Ustore 特性与规格 95
1.8.3.1.2 使用 Ustore 进行测试 95
1.8.3.1.3 Ustore 的最佳实践 96
1.8.3.2 存储格式 99
1.8.3.2.1 RCR Uheap 99
1.8.3.2.2 UBTree 101
1.8.3.2.3 Undo 105
1.8.3.2.4 Enhanced Toast 106
1.8.3.3 Ustore 事务模型 110
1.8.3.3.1 事务提交 110
1.8.3.3.2 事务回滚 111
1.8.3.4 闪回恢复 111
1.8.3.4.1 闪回查询 111
1.8.3.4.2 闪回表 113
1.8.3.4.3 闪回 DROP/TRUNCATE 115
1.8.3.5 常用视图工具 123
1.8.3.6 常见问题及定位手段 126
1.8.3.6.1 snapshot too old 126
1.8.3.6.2 storage test error 128
1.8.3.6.3 备机读业务报错:"UBTreeSearch::read_page has conflict with recovery, please try again later" 128

 版权所有 © 华为云计算技术有限公司

1.8.4 数据生命周期管理-OLTP 表压缩 129
1.8.4.1 特性简介 129
1.8.4.2 特性约束 130
1.8.4.3 特性规格 130
1.8.4.4 使用说明 130
1.8.4.5 维护窗口参数配置 134
1.8.4.6 运维 TIPS 135
1.9 内存管理 141
1.10 DBMind: 数据库自治运维 146
1.10.1 DBMind 模式说明 148
1.10.1.1 service 子命令 149
1.10.1.2 component 子命令 155
1.10.1.3 set 子命令 156
1.10.1.4 upgrade 子命令 157
1.10.2 DBMind 的支持组件 158
1.10.2.1 Prometheus Exporter 组件 159
1.10.2.1.1 概述 159
1.10.2.1.2 环境部署 159
1.10.2.1.3 使用指导 160
1.10.2.1.4 获取帮助 164
1.10.2.1.5 命令参考 164
1.10.2.1.6 常见问题处理 171
1.10.3 DBMind 的 AI 子功能 171
1.10.3.1 X-Tuner 223
1.10.3.1.1 概述 223
1.10.3.1.2 使用准备 223
1.10.3.1.3 使用示例 227
1.10.3.1.4 获取帮助 231
1.10.3.1.5 命令参考 232
1.10.3.1.6 常见问题处理 234
1.10.3.2 Index-advisor 234
1.10.3.2.1 单 query 索引推荐 234
1.10.3.2.2 虚拟索引 236
1.10.3.2.3 workload 级别索引推荐 239
1.10.3.3 Slow Query Diagnosis 243
1.10.3.3.1 概述 243
1.10.3.3.2 环境部署 243
1.10.3.3.3 使用指导 243
1.10.3.3.4 获取帮助 245
1.10.3.3.5 命令参考 245
1.10.3.3.6 注意事项说明 246
1.10.3.4 SQLdiag 247

 版权所有 © 华为云计算技术有限公司

1.10.3.4.1 概述 247
1.10.3.4.2 使用指导 248
1.10.3.4.3 获取帮助 249
1.10.3.4.4 命令参考 250
1.10.3.4.5 常见问题处理 250
1.10.3.5 SQL Rewriter 250
1.10.3.5.1 概述 250
1.10.3.5.2 使用指导 251
1.10.3.5.3 获取帮助 251
1.10.3.5.4 命令参考 251
1.10.3.5.5 常见问题处理 252
1.10.3.6 Anomaly detection 252
1.10.3.6.1 概述 252
1.10.3.6.2 使用指导 252
1.10.3.6.3 获取帮助 253
1.10.3.6.4 命令参考 254
1.10.3.6.5 常见问题处理 254
1.10.3.7 Distributed-key-advisor 254
1.10.3.7.1 外部工具 254
1.10.3.7.2 SQL 内置接口 258
1.10.3.8 Cluster Diagnosis 259
1.10.3.8.1 概述 259
1.10.3.8.2 使用指导 260
1.10.3.8.3 获取帮助 261
1.10.3.8.4 命令参考 262
1.10.3.8.5 常见问题处理 262
1.10.3.9 智能巡检 262
1.10.3.9.1 概述 262
1.10.3.9.2 使用指导 269
1.10.3.9.3 命令参考 275
1.10.3.9.4 常见问题处理 275
1.11 ABO 优化器 276
1.11.1 智能基数估计 276
1.11.1.1 概述 276
1.11.1.2 前置条件 276
1.11.1.3 使用指导 276
1.11.1.4 最佳实践 276
1.11.1.5 常见问题处理 277
1.12 Foreign Data Wrapper 277
1.12.1 file_fdw 278
1.13 GTM 模式 279
1.14 敏感数据发现 280

 版权所有 © 华为云计算技术有限公司

2 主备版 282
2.1 物化视图 282
2.1.1 全量物化视图 282
2.1.1.1 概述 282
2.1.1.2 支持和约束 282
2.1.1.3 使用 283
2.1.2 增量物化视图 284
2.1.2.1 概述 284
2.1.2.2 支持和约束 284
2.1.2.3 使用 284
2.2 设置密态等值查询 286
2.2.1 密态等值查询概述 286
2.2.2 使用 gsql 操作密态数据库 289
2.2.3 使用 JDBC 操作密态数据库 291
2.2.4 使用 Go 驱动操作密态数据库 296
2.2.5 前向兼容与安全增强 298
2.2.6 密态支持函数/存储过程 301
2.3 内存解密逃生通道 303
2.3.1 内存解密逃生通道概述 303
2.3.2 使用 gsql 操作内存解密逃生通道 304
2.3.3 使用 JDBC 驱动操作内存解密逃生通道 307
2.3.4 使用 Go 驱动操作内存解密逃生通道 308
2.3.5 使用 libpq_ce 操作内存解密逃生通道 309
2.4 透明数据加密 309
2.5 设置账本数据库 312
2.5.1 账本数据库概述 312
2.5.2 查看账本历史操作记录 315
2.5.3 校验账本数据一致性 316
2.5.4 归档账本数据库 317
2.5.5 修复账本数据库 318
2.6 基于标签的强制访问控制 320
2.6.1 定义安全标签 320
2.6.2 应用安全标签 321
2.6.3 基于标签的强制访问控制检查 322
2.7 逻辑复制 323
2.7.1 逻辑解码 323
2.7.1.1 逻辑解码概述 324
2.7.1.2 逻辑解码选项 327
2.7.1.3 使用 SQL 函数接口进行逻辑解码 334
2.7.1.4 使用流式解码实现数据逻辑复制 335
2.7.1.5 逻辑解码支持 DDL 335
2.8 分区表 343

 版权所有 © 华为云计算技术有限公司

2.8.1 大容量数据库 343
2.8.1.1 大容量数据库背景介绍 343
2.8.1.2 表分区技术 343
2.8.1.3 数据分区查找优化 344
2.8.1.4 数据分区运维管理 345
2.8.2 分区表介绍 345
2.8.2.1 基本概念 345
2.8.2.1.1 分区表（母表） 346
2.8.2.1.2 分区（分区子表、子分区） 347
2.8.2.1.3 分区键 348
2.8.2.2 分区策略 348
2.8.2.2.1 范围分区 348
2.8.2.2.2 间隔分区 350
2.8.2.2.3 哈希分区 351
2.8.2.2.4 列表分区 352
2.8.2.2.5 二级分区 352
2.8.2.2.6 分区表对导入操作的性能影响 355
2.8.2.3 分区基本使用 357
2.8.2.3.1 创建分区表 357
2.8.2.3.2 使用和管理分区表 360
2.8.2.3.3 分区表 DQL/DML 360
2.8.3 分区表查询优化 362
2.8.3.1 分区剪枝 362
2.8.3.1.1 分区表静态剪枝 363
2.8.3.1.2 分区表动态剪枝 365
2.8.3.2 分区算子执行优化 373
2.8.3.2.1 Partition Iterator 算子消除 373
2.8.3.2.2 Merge Append 374
2.8.3.2.3 Max/Min 375
2.8.3.2.4 分区导入数据性能优化 376
2.8.3.3 分区索引 377
2.8.3.4 分区表统计信息 380
2.8.3.4.1 级联收集统计信息 381
2.8.3.4.2 分区级统计信息 383
2.8.4 分区自动扩展 387
2.8.4.1 范围分区自动扩展 387
2.8.4.2 列表分区自动扩展 387
2.8.4.2.1 一级分区表自动扩展 387
2.8.4.2.2 二级分区表自动扩展 388
2.8.4.3 开启/关闭分区自动扩展 389
2.8.4.3.1 开启/关闭范围分区自动扩展 389
2.8.4.3.2 开启/关闭一级列表分区自动扩展 390

 版权所有 © 华为云计算技术有限公司

2.8.4.3.3 开启/关闭二级列表分区自动扩展 391
2.8.4.4 自动扩展分区的创建策略 391
2.8.5 分区表运维管理 392
2.8.5.1 新增分区 393
2.8.5.1.1 向范围分区表新增分区 393
2.8.5.1.2 向间隔分区表新增分区 393
2.8.5.1.3 向列表分区表新增分区 393
2.8.5.1.4 向二级分区表新增一级分区 394
2.8.5.1.5 向二级分区表新增二级分区 394
2.8.5.2 删除分区 394
2.8.5.2.1 对一级分区表删除分区 395
2.8.5.2.2 对二级分区表删除一级分区 395
2.8.5.2.3 对二级分区表删除二级分区 396
2.8.5.3 交换分区 396
2.8.5.3.1 对一级分区表交换分区 397
2.8.5.3.2 对二级分区表交换二级分区 397
2.8.5.4 清空分区 398
2.8.5.4.1 对一级分区表清空分区 398
2.8.5.4.2 对二级分区表清空一级分区 398
2.8.5.4.3 对二级分区表清空二级分区 398
2.8.5.5 分割分区 399
2.8.5.5.1 对范围分区表分割分区 399
2.8.5.5.2 对间隔分区表分割分区 400
2.8.5.5.3 对列表分区表分割分区 400
2.8.5.5.4 对*-RANGE 二级分区表分割二级分区 401
2.8.5.5.5 对*-LIST 二级分区表分割二级分区 401
2.8.5.6 合并分区 402
2.8.5.6.1 对一级分区表合并分区 402
2.8.5.6.2 对二级分区表合并二级分区 403
2.8.5.7 移动分区 403
2.8.5.7.1 对一级分区表移动分区 403
2.8.5.7.2 对二级分区表移动二级分区 403
2.8.5.8 重命名分区 403
2.8.5.8.1 对一级分区表重命名分区 403
2.8.5.8.2 对二级分区表重命名一级分区 403
2.8.5.8.3 对二级分区表重命名二级分区 404
2.8.5.8.4 对 Local 索引重命名索引分区 404
2.8.5.9 分区表行迁移 404
2.8.5.10 分区表索引重建/不可用 405
2.8.5.10.1 索引重建/不可用 405
2.8.5.10.2 Local 索引分区重建/不可用 405
2.8.6 分区并发控制 406

 版权所有 © 华为云计算技术有限公司

2.8.6.1 常规锁设计.....406
2.8.6.2 DQL/DML-DQL/DML 并发.....408
2.8.6.3 DQL/DML-DDL 并发.....408
2.8.6.4 DDL-DDL 并发.....410
2.8.7 分区表系统视图&DFX.....410
2.8.7.1 分区表相关系统视图.....411
2.8.7.2 分区表相关内置工具函数.....411
2.9 存储引擎.....414
2.9.1 存储引擎体系架构.....414
2.9.1.1 存储引擎体系架构概述.....414
2.9.1.1.1 静态编译架构.....414
2.9.1.1.2 通用数据库服务层.....415
2.9.1.2 设置存储引擎.....415
2.9.1.3 存储引擎更新说明.....417
2.9.1.3.1 GaussDB 内核 505 版本.....417
2.9.1.3.2 GaussDB 内核 503 版本.....417
2.9.1.3.3 GaussDB 内核 R2 版本.....417
2.9.2 Astore 存储引擎.....417
2.9.2.1 Astore 简介.....417
2.9.3 Ustore 存储引擎.....418
2.9.3.1 Ustore 简介.....418
2.9.3.1.1 Ustore 特性与规格.....418
2.9.3.1.2 使用 Ustore 进行测试.....419
2.9.3.1.3 Ustore 的最佳实践.....420
2.9.3.2 存储格式.....423
2.9.3.2.1 RCR Uheap.....423
2.9.3.2.2 UBTree.....424
2.9.3.2.3 Undo.....428
2.9.3.2.4 Enhanced Toast.....429
2.9.3.3 Ustore 事务模型.....433
2.9.3.3.1 事务提交.....434
2.9.3.3.2 事务回滚.....434
2.9.3.4 闪回恢复.....435
2.9.3.4.1 闪回查询.....435
2.9.3.4.2 闪回表.....437
2.9.3.4.3 闪回 DROP/TRUNCATE.....438
2.9.3.5 常用视图工具.....446
2.9.3.6 常见问题及定位手段.....450
2.9.3.6.1 snapshot too old.....450
2.9.3.6.2 storage test error.....451
2.9.3.6.3 备机读业务报错:"UBTreeSearch::read_page has conflict with recovery, please try again later".....452
2.9.4 数据生命周期管理-OLTP 表压缩.....453

 版权所有 © 华为云计算技术有限公司

2.9.4.1 特性简介 453
2.9.4.2 特性约束 454
2.9.4.3 特性规格 454
2.9.4.4 使用说明 454
2.9.4.5 维护窗口参数配置 458
2.9.4.6 运维 TIPS 459
2.10 内存管理 465
2.11 DBMind: 数据库自治运维 470
2.11.1 DBMind 模式说明 472
2.11.1.1 service 子命令 473
2.11.1.2 component 子命令 480
2.11.1.3 set 子命令 481
2.11.1.4 upgrade 子命令 482
2.11.2 DBMind 的支持组件 483
2.11.2.1 Prometheus Exporter 组件 484
2.11.2.1.1 概述 484
2.11.2.1.2 环境部署 484
2.11.2.1.3 使用指导 485
2.11.2.1.4 获取帮助 489
2.11.2.1.5 命令参考 489
2.11.2.1.6 常见问题处理 496
2.11.3 DBMind 的 AI 子功能 496
2.11.3.1 X-Tuner 547
2.11.3.1.1 概述 547
2.11.3.1.2 使用准备 547
2.11.3.1.3 使用示例 551
2.11.3.1.4 获取帮助 555
2.11.3.1.5 命令参考 556
2.11.3.1.6 常见问题处理 558
2.11.3.2 Index-advisor 558
2.11.3.2.1 单 query 索引推荐 559
2.11.3.2.2 虚拟索引 560
2.11.3.2.3 workload 级别索引推荐 563
2.11.3.3 Slow Query Diagnosis 567
2.11.3.3.1 概述 567
2.11.3.3.2 环境部署 567
2.11.3.3.3 使用指导 567
2.11.3.3.4 获取帮助 569
2.11.3.3.5 命令参考 570
2.11.3.3.6 注意事项说明 570
2.11.3.4 SQLdiag 571
2.11.3.4.1 概述 572

 版权所有 © 华为云计算技术有限公司

2.11.3.4.2 使用指导 572
2.11.3.4.3 获取帮助 573
2.11.3.4.4 命令参考 574
2.11.3.4.5 常见问题处理 574
2.11.3.5 SQL Rewriter 574
2.11.3.5.1 概述 574
2.11.3.5.2 使用指导 575
2.11.3.5.3 获取帮助 575
2.11.3.5.4 命令参考 576
2.11.3.5.5 常见问题处理 576
2.11.3.6 Anomaly detection 576
2.11.3.6.1 概述 576
2.11.3.6.2 使用指导 576
2.11.3.6.3 获取帮助 581
2.11.3.6.4 命令参考 581
2.11.3.6.5 常见问题处理 583
2.11.3.7 Cluster Diagnosis 583
2.11.3.7.1 概述 583
2.11.3.7.2 使用指导 584
2.11.3.7.3 获取帮助 585
2.11.3.7.4 命令参考 586
2.11.3.7.5 常见问题处理 586
2.11.3.8 智能巡检 586
2.11.3.8.1 概述 586
2.11.3.8.2 使用指导 593
2.11.3.8.3 命令参考 599
2.11.3.8.4 常见问题处理 599
2.12 DB4AI: 数据库驱动 AI 600
2.12.1 原生 DB4AI 引擎 600
2.12.2 全流程 AI 608
2.12.2.1 DB4AI-Snapshots 数据版本管理 608
2.13 ABO 优化器 612
2.13.1 智能基数估计 612
2.13.1.1 概述 612
2.13.1.2 前置条件 613
2.13.1.3 使用指导 613
2.13.1.4 最佳实践 613
2.13.1.5 常见问题处理 614
2.13.2 自适应计划选择 614
2.13.2.1 概述 614
2.13.2.2 前置条件 614
2.13.2.3 使用指导 615

 版权所有 © 华为云计算技术有限公司

2.13.2.4 最佳实践 615
2.13.2.5 常见问题处理 615
2.13.3 自适应代价估计 615
2.13.3.1 概述 615
2.13.3.2 前置条件 616
2.13.3.3 使用指导 616
2.13.3.4 最佳实践 616
2.13.3.5 常见问题处理 618
2.14 Foreign Data Wrapper 618
2.14.1 file_fdw 618
2.15 SPM 计划管理 620
2.16 敏感数据发现 624
2.17 计划内应用无损透明 625

 版权所有 © 华为云计算技术有限公司

1 分布式版

1.1 物化视图

物化视图是一种特殊的物理表，物化视图是相对普通视图而言的。普通视图是虚拟表，应用的局限性较大，任何对视图的查询实际上都是转换为对SQL语句的查询，性能并没有实际提高。而物化视图实际上就是存储SQL所执行语句的结果，起到缓存的效果。物化视图常用的操作包括创建、查询、删除和刷新。

根据创建规则，物化视图分为全量物化视图和增量物化视图。全量物化视图只支持全量刷新；增量物化视图支持全量刷新和增量刷新两种方式。全量刷新会将基表中的数据全部重新刷入物化视图中，而增量刷新只会将两次刷新间隔期间的基表产生的增量数据刷入物化视图中。

目前Ustore引擎不支持创建、使用物化视图。

1.1.1 全量物化视图

1.1.1.1 概述

全量物化视图仅支持对创建好的物化视图做全量刷新，而不支持做增量刷新。创建全量物化视图语法和CREATE TABLE AS语法一致（详情请参见《开发者指南》中的“SQL参考 > SQL语法 > CREATE TABLE AS”章节），不支持对全量物化视图指定NodeGroup创建。全量物化视图的创建继承GTM-Free的相关约束。

1.1.1.2 支持和约束

支持场景

- 大体上，全量物化视图所支持的查询范围与CREATE TABLE AS语句一致。
- 创建全量物化视图可以指定分布列。
- 可以在全量物化视图上创建索引。
- 支持analyze、explain。

版权所有 © 华为云计算技术有限公司

不支持场景

- 全量物化视图不支持NodeGroup。
- 不可对物化视图做增删改操作，只支持查询语句。
- Ustore引擎不支持全量物化视图的创建和使用。

约束

- 创建全量物化视图所使用的基表必须在所有DN上有定义，基表所属nodegroup必须为installation group。
- 全量物化视图的刷新、删除过程中会给基表加高级别锁，若物化视图的定义涉及多张表，需要注意业务逻辑，避免死锁产生。

1.1.1.3 使用

语法格式

- 创建全量物化视图
  ```
  CREATE MATERIALIZED VIEW view_name AS query;
  ```
- 刷新全量物化视图
  ```
  REFRESH MATERIALIZED VIEW view_name;
  ```
- 删除物化视图
  ```
  DROP MATERIALIZED VIEW view_name;
  ```
- 查询物化视图
  ```
  SELECT * FROM view_name;
  ```

参数说明

- **view_name**

  要创建的物化视图名。

  取值范围：字符串，要符合标识符的命名规范。

- **AS query**

  一个SELECT VALUES命令或者一个运行预备好的SELECT或VALUES查询的EXECUTE命令。

示例

```
-- 修改表的默认类型
gaussdb=#  set enable_default_ustore_table=off;

-- 准备数据
CREATE TABLE t1(c1 int, c2 int);
INSERT INTO t1 VALUES(1, 1);
INSERT INTO t1 VALUES(2, 2);

-- 创建全量物化视图
gaussdb=# CREATE MATERIALIZED VIEW mv AS select count(*) from t1;
CREATE MATERIALIZED VIEW

-- 查询物化视图结果
gaussdb=# SELECT * FROM mv;
 count
-------
     2
(1 row)
```

 版权所有 © 华为云计算技术有限公司

```
-- 再次向物化视图中基表插入数据
gaussdb=# INSERT INTO t1 VALUES(3, 3);

-- 对全量物化视图做全量刷新
gaussdb=# REFRESH MATERIALIZED VIEW mv;
REFRESH MATERIALIZED VIEW

-- 查询物化视图结果
gaussdb=# SELECT * FROM mv;
 count
-------
     3
(1 row)

-- 删除物化视图，删除表
gaussdb=# DROP MATERIALIZED VIEW mv;
DROP MATERIALIZED VIEW
gaussdb=# DROP TABLE t1;
DROP TABLE
```

1.1.2 增量物化视图

1.1.2.1 概述

增量物化视图可以对物化视图增量刷新，需要用户手动执行语句，刷新物化视图在一段时间内的增量数据。与全量创建物化视图的不同在于目前增量物化视图所支持场景较小。目前物化视图创建语句仅支持基表扫描语句或者UNION ALL语句。

1.1.2.2 支持和约束

支持场景

- 单表查询语句。
- 多个单表查询的UNION ALL。
- 在物化视图上创建索引。
- 对物化视图进行ANALYZE操作。
- 增量物化视图会继承基表NodeGroup创建（检查各个基表是否在同一个NodeGroup，并基于这个NodeGroup进行创建）。

不支持场景

- 物化视图中不支持带Stream计划，多表join连接计划以及subquery计划。
- 除少部分ALTER操作外，不支持对物化视图中基表做绝大多数DDL操作。
- 创建物化视图不可指定物化视图分布列。
- 不可对物化视图做增删改操作，只支持查询语句。
- 不支持用临时表/hashbucket/unlog/分区表创建物化视图，只支持hash分布表。
- 不支持物化视图嵌套创建（物化视图上创建物化视图）。
- 不支持UNLOGGED类型的物化视图，不支持WITH语法。
- Ustore引擎不支持增量物化视图的创建和使用。

 版权所有 © 华为云计算技术有限公司

约束

- 物化视图定义如果为UNION ALL，则其中每个子查询需使用不同的基表，且各基表分布列相同。物化视图的分布列会自动推导且与各基表相同。
- 物化视图定义的列必须包含基表的所有分布列。
- 增量物化视图的创建、全量刷新、删除过程中会给基表加高级别锁，若物化视图的定义为UNION ALL，需要注意业务逻辑，避免死锁产生。

1.1.2.3 使用

语法格式

- 创建增量物化视图
  ```
  CREATE INCREMENTAL MATERIALIZED VIEW view_name AS query;
  ```
- 全量刷新物化视图
  ```
  REFRESH MATERIALIZED VIEW view_name;
  ```
- 增量刷新物化视图
  ```
  REFRESH INCREMENTAL MATERIALIZED VIEW view_name;
  ```
- 删除物化视图
  ```
  DROP MATERIALIZED VIEW view_name;
  ```
- 查询物化视图
  ```
  SELECT * FROM view_name;
  ```

参数说明

- **view_name**

  要创建的物化视图名。

  取值范围：字符串，要符合标识符的命名规范。

- **AS query**

  一个SELECT VALUES命令或者一个运行预备好的SELECT或VALUES查询的EXECUTE命令。

示例

```
-- 修改表的默认类型
gaussdb=#  SET enable_default_ustore_table=off;

-- 准备数据
CREATE TABLE t1(c1 int, c2 int);
INSERT INTO t1 VALUES(1, 1);
INSERT INTO t1 VALUES(2, 2);

-- 创建增量物化视图
gaussdb=# CREATE INCREMENTAL MATERIALIZED VIEW mv AS SELECT * FROM t1;
CREATE MATERIALIZED VIEW

-- 插入数据
gaussdb=# INSERT INTO t1 VALUES(3, 3);
INSERT 0 1

-- 增量刷新物化视图
gaussdb=# REFRESH INCREMENTAL MATERIALIZED VIEW mv;
REFRESH MATERIALIZED VIEW

-- 查询物化视图结果
gaussdb=# SELECT * FROM mv;
 c1 | c2
```

 版权所有 © 华为云计算技术有限公司

```
----+----
  1 |  1
  2 |  2
  3 |  3
(3 rows)

-- 插入数据
gaussdb=# INSERT INTO t1 VALUES(4, 4);
INSERT 0 1

-- 全量刷新物化视图
gaussdb=# REFRESH MATERIALIZED VIEW mv;
REFRESH MATERIALIZED VIEW

-- 查询物化视图结果
gaussdb=# select * from mv;
 c1 | c2
----+----
  1 |  1
  2 |  2
  3 |  3
  4 |  4
(4 rows)

-- 删除物化视图，删除表
gaussdb=# DROP MATERIALIZED VIEW mv;
DROP MATERIALIZED VIEW
gaussdb=# DROP TABLE t1;
DROP TABLE
```

1.2 设置密态等值查询

1.2.1 密态等值查询概述

随着企业数据上云，数据的安全隐私保护面临越来越严重的挑战。密态数据库将解决数据整个生命周期中的隐私保护问题，涵盖网络传输、数据存储以及数据运行状态；更进一步，密态数据库可以实现云化场景下的数据隐私权限分离，即实现数据拥有者和实际数据管理者的数据读取能力分离。密态等值查询将优先解决密文数据的等值类查询问题。

加密模型

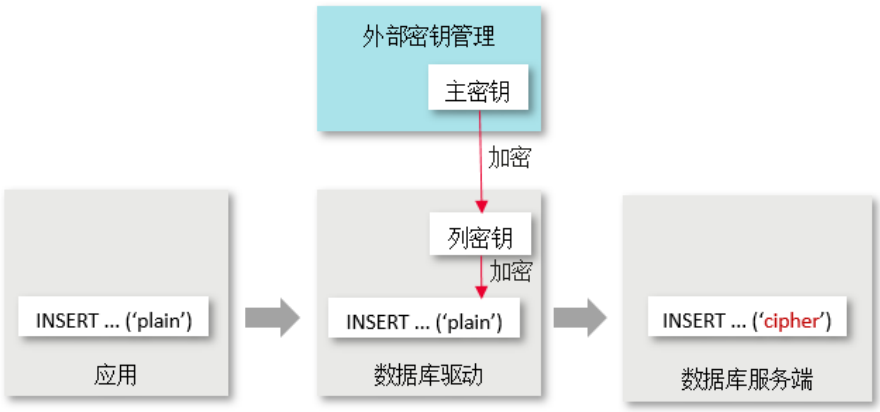
全密态数据库使用多级加密模型，加密模型中涉及3个对象：数据、列密钥和主密钥，以下是对3个对象的介绍：

- **数据**：
  a. SQL语法中包含的数据，比如INSERT... VALUES ('data')语法中包含'data'。
  b. 从数据库服务端返回的查询结果，如执行SELECT语法返回的查询结果。

  说明

  密态数据库会在驱动中，对SQL语法中属于加密列的数据进行加密，对数据库服务端返回的属于加密列的查询结果进行解密。
- **列密钥**：数据由列密钥进行加密，列密钥由数据库驱动生成或由用户手动导入，列密钥密文存储在数据库服务端。
- **主密钥**：列密钥由主密钥加密，主密钥由外部密钥管理者生成并存储。数据库驱动会自动访问外部密钥管理者，以实现对列密钥进行加解密。

 版权所有 © 华为云计算技术有限公司



整体流程

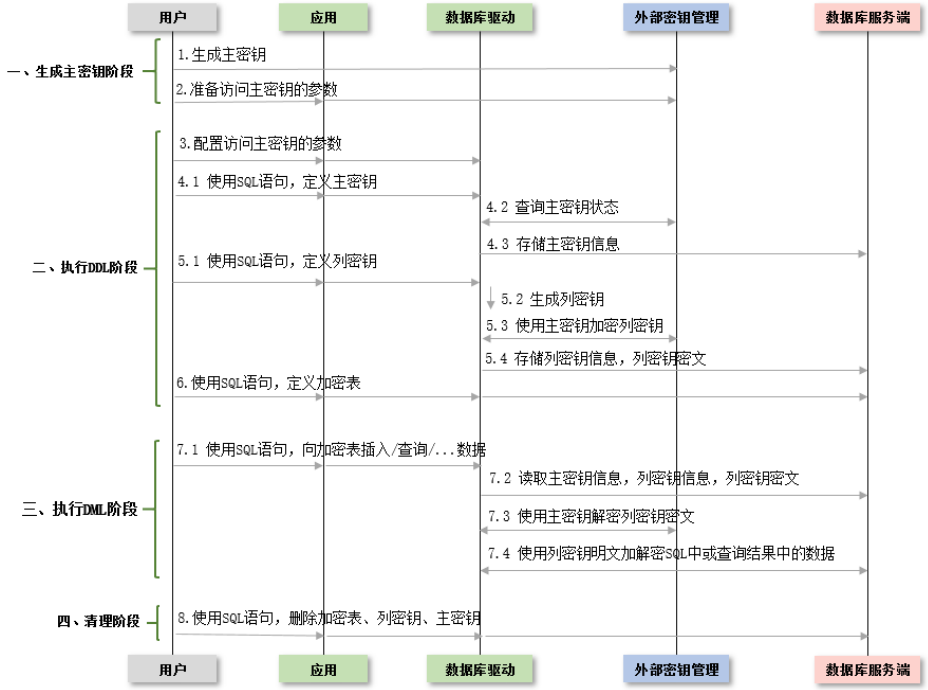
在使用全密态数据库的过程中，主要流程包括如下四个阶段，本节（2.1）介绍整体流程。**使用gsql操作密态数据库**、**使用JDBC操作密态数据库**章节介绍详细使用流程。

一、生成主密钥阶段：首先，用户需在外部密钥管理中生成主密钥。外部密钥管理分为多种，包括：华为云密钥服务、，根据使用场景选择其中一种。生成主密钥后，需在外部密钥管理中，准备访问主密钥的参数，以供数据库使用。

二、执行DDL阶段：在本阶段，用户可使用密态数据库的密钥语法依次定义主密钥和列密钥，然后定义表并指定表中某列为加密列。定义主密钥和列密钥的过程中，需访问上一阶段生成的主密钥。

三、执行DML阶段：在创建加密表后，用户可直接执行包含但不限于INSERT、SELECT、UPDATE、DELETE等语法。数据库驱动会自动根据上一阶段的加密定义自动对加密列中的数据进行加解密。

四、清理阶段：依次删除加密表、列密钥和主密钥。

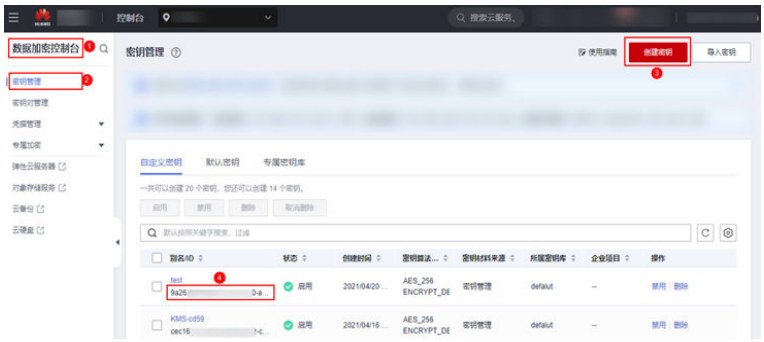


版权所有 © 华为云计算技术有限公司

生成主密钥阶段

首次使用密态数据库时，需使用外部密钥管理服务生成至少一个主密钥，生成方式如下：

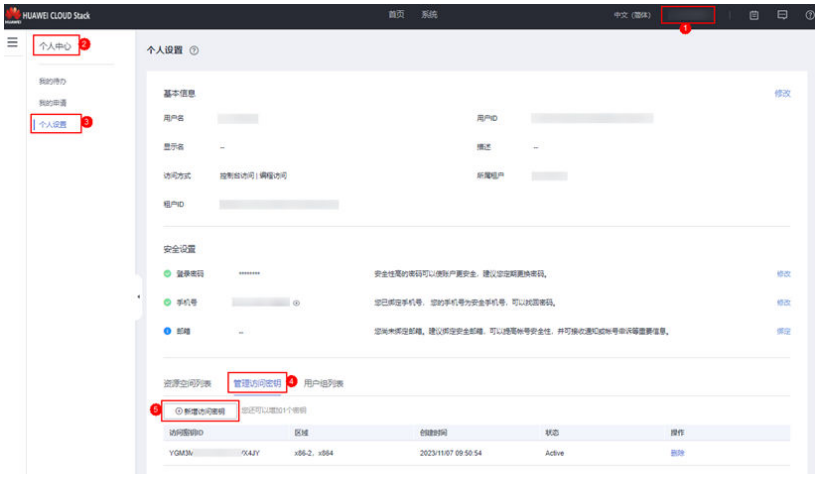
- **华为混合云场景**
  a. 进入密钥管理服务：登录华为云管控，搜索并进入"数据加密服务"。
  b. 创建主密钥：选择"密钥管理"功能，并通过"创建密钥"按钮创建至少1个密钥，即主密钥。



  c. 记住主密钥ID：成功创建主密钥后，每个主密钥都有1个密钥ID。在后续使用密态数据过程中，需配置主密钥ID，数据库驱动会通过Restful接口访问该主密钥。

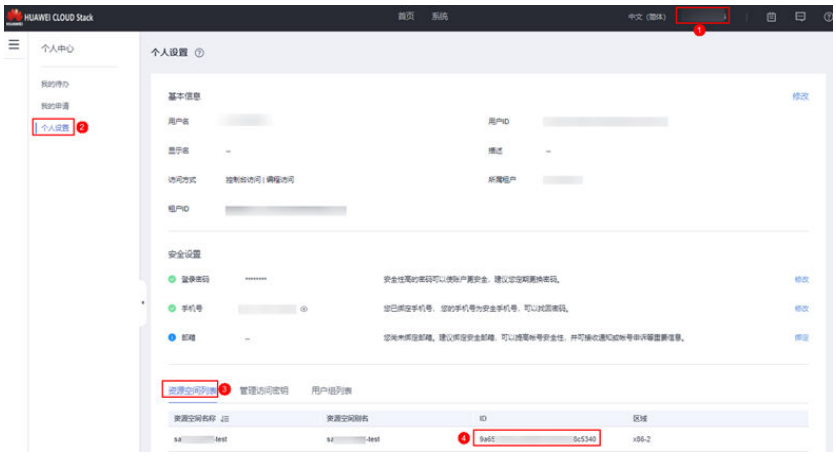
在生成主密钥后，需为数据驱动准备访问主密钥的参数，比如用户名、资源ID等参数。华为云支持两种身份认证方式，两种方式需要的参数个数与参数类型不同，选择其中一种方式即可。下述步骤介绍如何获取这些参数：

- **方式一 aksk认证**
  a. AK、SK：登录华为云管控，单击右上角"用户名"，进入"个人设置"页面。在"管理访问密钥"中，创建新的访问密钥，即AK和SK。初次创建访问密钥时，可下载密钥，下载后可在文件中分别找到AK和SK。



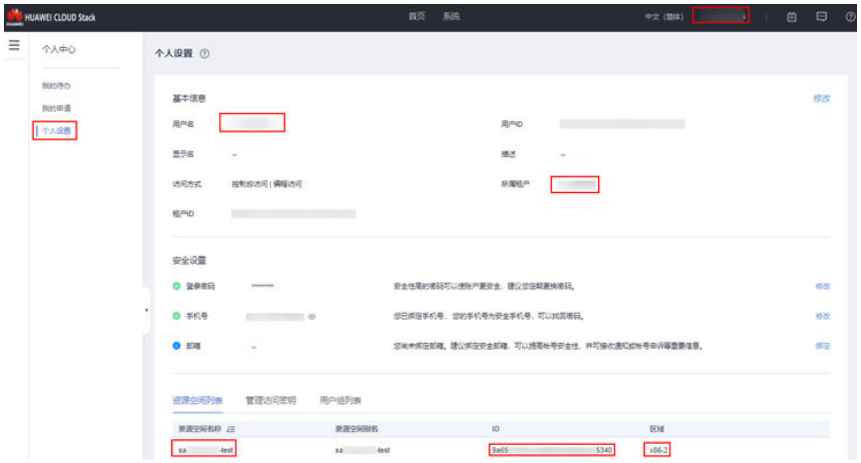
  b. 资源ID：进入"个人设置"页面，在"资源空间列表"中，可获取"资源ID"。

版权所有 © 华为云计算技术有限公司



- **方式二 账号密码认证**

  a. 用户名、所属租户、资源空间名称、资源空间ID、区域：登录华为云管控，单击右上角"用户名"，进入"个人设置"页面。进入"个人设置"页面。可依次获取所需参数。



  b. IAM服务器地址、KMS服务器地址：请自行根据部署方式查询。

  c. 用户密码：即用户名对应的密码。

1.2.2 使用 gsql 操作密态数据库

执行 SQL 语句

执行本节的SQL语句前，请确保已提前生成主密钥，并确认访问主密钥的参数。

本节以完整的执行流程为例，介绍如何使用密态数据库语法，包括三个阶段：使用DDL阶段、使用DML阶段、清理阶段。

步骤1 连接数据库，并通过-C参数开启全密态开关

```
gsql -p PORT -d DATABASE -h HOST -U USER -W PASSWORD -r -C
```

步骤2 通过元命令设置访问主密钥的参数

注意：从keyType字符串开始，不要添加换行，不要添加空格，否则gsql工具无法识别完整参数。

华为云支持两种认证方式，两种认证方式的参数个数与参数类型不同，选择其中一种方式即可。

版权所有 © 华为云计算技术有限公司

- **认证方式一** aksk认证
  ```
  gaussdb=# \key_info keyType=hcs_kms,kmsProjectId={资源ID},ak={AK},sk={SK}
  ```
  参数获取：生成主密钥阶段介绍了如何获取相关参数：项目ID、AK、SK。

  示例：\key_info keyType=hcs_kms,kmsProjectId=0b59929e8100268a2f22c01429802728,ak=XMAUMJY******DFWLQW,sk=ga6rO8lx1Q4uB*********2gf80muIzUX
- **认证方式二** 账号密码认证
  ```
  gaussdb=# \key_info keyType=hcs_kms,iamUrl={IAM服务器地址},iamUser={用户名},iamPassword={用户密码},iamDomain={所属租户},kmsProject={资源空间名称}
  ```
  参数获取：生成主密钥阶段介绍了如何获取上述提到的5个参数。

  示例：\key_info keyType=hcs_kms,iamUrl=https://iam.xxx.com/v3/auth/tokens,iamUser=test_user,iamPassword=*********,iamDomain=test_account,kmsProject=xxx

步骤3 定义主密钥

在生成主密钥阶段，密钥服务已生成并存储主密钥，执行本语法只是将主密钥的相关信息存储在数据库中，方便以后访问。该语法详细格式参考：《开发者指南》中“SQL参考 > SQL语法 > CREATE CLIENT MASTER KEY”章节。

```
gaussdb=# CREATE CLIENT MASTER KEY cmk1 WITH ( KEY_STORE = hcs_kms , KEY_PATH = '{KMS服务器地址}/{密钥ID}', ALGORITHM = AES_256);
CREATE CLIENT MASTER KEY
```

- 参数获取：生成主密钥阶段介绍了如何获取如下参数：KMS服务器地址、密钥ID。

  KEY_PATH示例：https://kms-scc-apig.sa-fb-1.console-arm2.com/v1.0/0b59929e8100268a2f22c01429802728/kms/9a262917-8b31-41af-a1e0-a53235f32de9

步骤4 定义列密钥

列密钥由上一步定义的主密钥加密。详细语法参考：《开发者指南》中“SQL参考 > SQL语法 > CREATE COLUMN ENCRYPTION KEY”章节。

```
gaussdb=# CREATE COLUMN ENCRYPTION KEY cek1 WITH VALUES (CLIENT_MASTER_KEY = cmk1, ALGORITHM  = AES_256_GCM);
```

步骤5 定义加密表

本示例中，通过语法指定表中name和credit_card为加密列。

```
gaussdb=# CREATE TABLE creditcard_info (
  id_number int,
  name text encrypted with (column_encryption_key = cek1, encryption_type = DETERMINISTIC),
  credit_card varchar(19) encrypted with (column_encryption_key = cek1, encryption_type = DETERMINISTIC));
NOTICE:  The 'DISTRIBUTE BY' clause is not specified. Using 'id_number' as the distribution column by default.
HINT:  Please use 'DISTRIBUTE BY' clause to specify suitable data distribution column.
CREATE TABLE
```

步骤6 对加密表进行其他操作

```
-- 向加密表写入数据
gaussdb=# INSERT INTO creditcard_info VALUES (1,'joe','6217986500001288393');
INSERT 0 1
gaussdb=# INSERT INTO creditcard_info VALUES (2, 'joy','6219985678349800033');
INSERT 0 1

-- 从加密表中查询数据
gaussdb=# select * from creditcard_info where name = 'joe';
 id_number | name |     credit_card
-----------+------+---------------------
         1 | joe  | 6217986500001288393
```

 版权所有 © 华为云计算技术有限公司

```
-- 更新加密表中数据
gaussdb=# update creditcard_info set credit_card = '80000000011111111' where name = 'joy';
UPDATE 1

-- 向表中新增一列加密列
gaussdb=# ALTER TABLE creditcard_info ADD COLUMN age int ENCRYPTED WITH
(COLUMN_ENCRYPTION_KEY = cek1, ENCRYPTION_TYPE = DETERMINISTIC);
ALTER TABLE

-- 从表中删除一列加密列
gaussdb=# ALTER TABLE creditcard_info DROP COLUMN age;
ALTER TABLE

-- 从系统表中查询主密钥信息
gaussdb=# SELECT * FROM gs_client_global_keys;
 global_key_name | key_namespace | key_owner | key_acl |        create_date
-----------------+---------------+-----------+---------+---------------------------
 cmk1            |          2200 |        10 |         | 2021-04-21 11:04:00.656617
(1 rows)

-- 从系统表中查询列密钥信息
gaussdb=# SELECT column_key_name,column_key_distributed_id ,global_key_id,key_owner  FROM
gs_column_keys;
 column_key_name | column_key_distributed_id | global_key_id | key_owner
-----------------+---------------------------+---------------+-----------
 cek1            |                760411027 |         16392 |        10
(1 rows)

-- 查看表中列的元信息
gaussdb=# \d creditcard_info
        Table "public.creditcard_info"
   Column    |       Type        | Modifiers
-------------+-------------------+------------
 id_number   | integer           |
 name        | text              |  encrypted
 credit_card | character varying |  encrypted
```

步骤7 清理阶段

```
-- 删除加密表
gaussdb=# DROP TABLE creditcard_info;
DROP TABLE

-- 删除列密钥
gaussdb=# DROP COLUMN ENCRYPTION KEY cek1;
DROP COLUMN ENCRYPTION KEY

-- 删除主密钥
gaussdb=# DROP CLIENT MASTER KEY cmk1;
DROP CLIENT MASTER KEY
```

----结束

1.2.3 使用 JDBC 操作密态数据库

配置 JDBC 驱动

1. 获取JDBC驱动包，JDBC驱动获取及使用可参考《开发者指南》中“应用程序开发教程 > 基于JDBC开发”章节。

   密态数据库支持的JDBC驱动包为gsjdbc4.jar、opengaussjdbc.jar、gscejdbc.jar。

   - gsjdbc4.jar：主类名为“org.postgresql.Driver”，数据库连接的url前缀为“jdbc:postgresql”。
   - opengaussjdbc.jar：主类名为“com.huawei.opengauss.jdbc.Driver”，数据库连接的url前缀为“jdbc:opengauss”。

 版权所有 © 华为云计算技术有限公司

- gscejdbc.jar（目前仅支持EulerOS操作系统）：主类名为“com.huawei.gaussdb.jdbc.Driver”，数据库连接的url前缀为“jdbc:gaussdb”，密态场景推荐使用此驱动包。

2. 配置LD_LIBRARY_PATH

   密态场景使用JDBC驱动包时，需要先设置环境变量LD_LIBRARY_PATH。

   - 使用gscejdbc.jar驱动包时，gscejdbc.jar驱动包中密态数据库需要的依赖库会自动复制到该路径下，并在开启密态功能连接数据库的时候加载。
   - 使用opengaussjdbc.jar或gsjdbc4.jar时，需要同时解压包名为GaussDB-Kernel_数据库版本号_操作系统版本号_64bit_libpq.tar.gz的压缩包解压到指定目录，并将lib文件夹所在目录路径，添加至LD_LIBRARY_PATH环境变量中。

   ⚠ 注意

   全密态场景使用JDBC驱动包时需要有System.loadLibrary权限，以及环境变量LD_LIBRARY_PATH中第一优先路径的文件读写权限，建议使用独立目录作为全密态依赖库的存放路径。若在执行的时候指定java.library.path，需要与LD_LIBRARY_PATH的第一优先路径保持一致。

使用gscejdbc.jar时，jvm加载class文件需要依赖系统的libstdc++库，若开启密态则gscejdbc.jar会自动复制密态数据库依赖的动态库（包括libstdc++库）到用户设定的LD_LIBRARY_PATH路径下。如果依赖库与现有系统库版本不匹配，则首次运行仅部署依赖库，再次调用后即可正常使用。

执行 SQL 语句

执行本节的SQL语句前，请确保已提前生成主密钥，并确认访问主密钥的参数。

本节以完整的执行流程为例，介绍如何使用密态数据库语法，包括三个阶段：使用DDL阶段、使用DML阶段、清理阶段。

JDBC开发中与非密态场景操作一致的部分请参考《开发者指南》中“应用程序开发教程 > 基于JDBC开发”章节。

- 密态数据库连接参数

  enable_ce：String类型。其中enable_ce=0表示不开启全密态开关，enable_ce=1表示支持密态等值查询基本能力，enable_ce=3表示在密态等值查询能力的基础上支持内存解密逃生通道。

```
// 以下用例以gscejdbc.jar驱动为例，如果使用其他驱动包，仅需修改驱动类名和数据库连接的url前缀。
// gsjdbc4.jar: 主类名为“org.postgresql.Driver”，数据库连接的url前缀为“jdbc:postgresql”。
// opengaussjdbc.jar：主类名为“com.huawei.opengauss.jdbc.Driver”，数据库连接的url前缀为“jdbc:opengauss”。
// gscejdbc.jar：主类名为“com.huawei.gaussdb.jdbc.Driver”，数据库连接的url前缀为“jdbc:gaussdb”

public static void main(String[] args) {
    // 驱动类。
    String driver = "com.huawei.gaussdb.jdbc.Driver";
    // 数据库连接描述符。enable_ce=1表示支持密态等值查询基本能力。
    String sourceURL = "jdbc:gaussdb://127.0.0.1:8000/postgres?enable_ce=1";
    // 在环境变量USER、PASSWORD分别配置用户名密码。
    String username = System.getenv("USER");
    String passwd = System.getenv("PASSWORD");
    Connection conn = null;
    try {
        // 加载驱动
```

 版权所有 © 华为云计算技术有限公司

```
        Class.forName(driver);
        // 创建连接
        conn = DriverManager.getConnection(sourceURL, username, passwd);
        System.out.println("Connection succeed!");
        // 创建语句对象
        Statement stmt = conn.createStatement();

        // 设置访问主密钥的参数
        // 此处介绍2种方式，选择其中1种方式即可：
        // 认证方式一 aksk认证（生成主密钥阶段介绍了如何获取相关参数：项目ID、AK、SK）

        conn.setClientInfo("key_info", "keyType=hcs_kms, kmsProjectId={项目ID}, ak={AK}, sk={SK}");
        /* 示例:
            conn.setClientInfo("key_info",
"keyType=hcs_kms,kmsProjectId=0b59929e8100268a2f22c01429802728," +
                "ak=XMAUMJY******DFWLQW, sk=ga6rO8lx1Q4uB*********2gf80muIzUX,");
        */
        // 认证方式二 账号密码认证（生成主密钥阶段介绍了如何获取相关参数：IAM服务器地址、IAM用户名、IAM用户密码、账号名、项目）
        conn.setClientInfo("key_info", "keyType=hcs_kms," +
            "iamUrl={IAM服务器地址}," +
            "iamUser={IAM用户名}," +
            "iamPassword={IAM用户密码}," +
            "iamDomain={账号名}," +
            "kmsProject={项目}");
        /* 示例:
        conn.setClientInfo("key_info", "keyType=hcs_kms," +
                "iamUrl=https://iam.xxx.com/v3/auth/tokens," +
                "iamUser=test," +
                "iamPassword=*********," +
                "iamDomain=test_account," +
                "kmsProject=xxx");
        */
        // 定义主密钥：cmk1为主密钥名字，可自行取名
        // 生成主密钥阶段介绍了如何获取如下参数：KMS服务器地址、密钥ID

        int rc = stmt.executeUpdate("CREATE CLIENT MASTER KEY ImgCMK1 WITH ( KEY_STORE =
hcs_kms , KEY_PATH = '{KMS服务器地址}/{密钥ID}', ALGORITHM = AES_256);");
        /*
            KEY_PATH示例：https://kms-scc-apig.sa-fb-1.console-arm2.com/
v1.0/0b59929e8100268a2f22c01429802728/kms/9a262917-8b31-41af-a1e0-a53235f32de9
            解释：在生成主密钥阶段，密钥服务已生成并存储主密钥，执行本语法只是将主密钥的相关信息存储在数据库中，方便以后访问
            提示：KEY_PATH格式请参考：《开发者指南》中“SQL参考 > SQL语法 > CREATE CLIENT MASTER KEY”章节
        */
        // 定义列加密密钥：列密钥由上一步创建的主密钥加密。详细语法参考：《开发者指南》中“SQL参考 > SQL语法 > CREATE COLUMN ENCRYPTION KEY”章节
        int rc2 = stmt.executeUpdate("CREATE COLUMN ENCRYPTION KEY ImgCEK1 WITH VALUES
(CLIENT_MASTER_KEY = ImgCMK1, ALGORITHM  = AES_256_GCM);");
        // 定义加密表
        int rc3 = stmt.executeUpdate("CREATE TABLE creditcard_info (id_number int, name varchar(50)
encrypted with (column_encryption_key = ImgCEK1, encryption_type = DETERMINISTIC),credit_card
varchar(19) encrypted with (column_encryption_key = ImgCEK1, encryption_type =
DETERMINISTIC));");
        // 插入数据
        int rc4 = stmt.executeUpdate("INSERT INTO creditcard_info VALUES
(1,'joe','6217986500001288393');");
        // 查询加密表
        ResultSet rs = null;
        rs = stmt.executeQuery("select * from creditcard_info where name = 'joe';");
        // 删除加密表
        int rc5 = stmt.executeUpdate("DROP TABLE IF EXISTS creditcard_info;");
        // 删除列加密密钥
        int rc6 = stmt.executeUpdate("DROP COLUMN ENCRYPTION KEY IF EXISTS ImgCEK1;");
        // 删除客户端主密钥
        int rc7 = stmt.executeUpdate("DROP CLIENT MASTER KEY IF EXISTS ImgCMK1;");
        // 关闭语句对象
        stmt.close();
```

 版权所有 © 华为云计算技术有限公司

```
        // 关闭连接
        conn.close();
    } catch (Exception e) {
        e.printStackTrace();
        return;
    }
}
```

说明

- 使用JDBC操作密态数据库时，一个数据库连接对象对应一个线程，否则，不同线程变更可能导致冲突。
- 使用JDBC操作密态数据库时，不同connection对密态配置数据有变更，由客户端调用isvalid方法保证connection能够持有变更后的密态配置数据，此时需要保证参数refreshClientEncryption为1(默认值为1)，在单客户端操作密态数据场景下，refreshClientEncryption参数可以设置为0。

- 调用isValid方法刷新缓存示例

```
// 创建连接conn1
Connection conn1 = DriverManager.getConnection("url","user","password");
// 在另外一个连接conn2中创建客户端主密钥
...
// conn1通过调用isValid刷新缓存，刷新conn1密钥缓存
try {
 if (!conn1.isValid(60)) {
  System.out.println("isValid Failed for connection 1");
 }
} catch (SQLException e) {
 e.printStackTrace();
        return null;
}
```

执行密态等值密文解密

数据库连接接口PgConnection类型新增解密接口，可以对全密态数据库的密态等值密文进行解密。解密后返回其明文值，通过schema.table.column找到解文对应的密文列并返回其原始数据类型。

表 1-1 新增 org.postgresql.jdbc.PgConnection 函数接口

方法名	返回值类型	支持JDBC 4
decryptData(String ciphertext, Integer len, String schema, String table, String column)	ClientLogicDecryptResult	Yes

参数说明：

- **ciphertext**
  需要解密的密文。
- **len**
  密文长度。当取值小于实际密文长度时，解密失败。
- **schema**
  加密列所属schema名称。
- **table**

 版权所有 © 华为云计算技术有限公司

加密列所属table名称。

- **column**
  加密列所属column名称。

  说明

  下列场景可以解密成功，但不推荐：
  - 密文长度入参比实际密文长。
  - schema.table.column指向其他加密列，此时将返回被指向的加密列的原始数据类型。

表 1-2 新增 org.postgresql.jdbc.clientlogic.ClientLogicDecryptResult 函数接口

方法名	返回值类型	描述	支持JDBC4
isFailed()	Boolean	解密是否失败，若失败返回True，否则返回False。	Yes
getErrMsg()	String	获取错误信息。	Yes
getPlaintext()	String	获取解密后的明文。	Yes
getPlaintextSize()	Integer	获取解密后的明文长度。	Yes
getOriginalType()	String	获取加密列的原始数据类型。	Yes

```
// 通过非密态连接、逻辑解码等其他方式获得密文后，可使用该接口对密文进行解密
import org.postgresql.jdbc.PgConnection;
import org.postgresql.jdbc.clientlogic.ClientLogicDecryptResult;

// conn为密态连接
// 调用密态PgConnection的decryptData方法对密文进行解密，通过列名称定位到该密文的所属加密列，并返回其原始数据类型
ClientLogicDecryptResult decrypt_res = null;
decrypt_res = ((PgConnection)conn).decryptData(ciphertext, ciphertext.length(), schemaname_str,
        tablename_str, colname_str);
// 检查返回结果类解密成功与否，失败可获取报错信息，成功可获得明文及长度和原始数据类型
if (decrypt_res.isFailed()) {
    System.out.println(String.format("%s\n", decrypt_res.getErrMsg()));
} else {
    System.out.println(String.format("decrypted plaintext: %s size: %d type: %s\n", decrypt_res.getPlaintext(),
        decrypt_res.getPlaintextSize(), decrypt_res.getOriginalType()));
}
```

执行加密表的预编译 SQL 语句

```
// 调用Connection的prepareStatement方法创建预编译语句对象。
PreparedStatement pstmt = conn.prepareStatement("INSERT INTO creditcard_info VALUES (?, ?, ?);");
// 调用PreparedStatement的setShort设置参数。
pstmt.setInt(1, 2);
pstmt.setString(2, "joy");
pstmt.setString(3, "6219985678349800033");
// 调用PreparedStatement的executeUpdate方法执行预编译SQL语句。
int rowcount = pstmt.executeUpdate();
// 调用PreparedStatement的close方法关闭预编译语句对象。
pstmt.close();
```


版权所有 © 华为云计算技术有限公司

执行加密表的批处理操作

```
// 调用Connection的prepareStatement方法创建预编译语句对象。
Connection conn = DriverManager.getConnection("url","user","password");
PreparedStatement pstmt = conn.prepareStatement("INSERT INTO creditcard_info (id_number, name,
credit_card) VALUES (?,?,?)");
// 针对每条数据都要调用setShort设置参数，以及调用addBatch确认该条设置完毕。
int loopCount = 20;
 for (int i = 1; i < loopCount + 1; ++i) {
      pstmt.setInt(1, i);
      pstmt.setString(2, "Name " + i);
      pstmt.setString(3, "CreditCard " + i);
      // Add row to the batch.
      pstmt.addBatch();
}
// 调用PreparedStatement的executeBatch方法执行批处理。
int[] rowcount = pstmt.executeBatch();
// 调用PreparedStatement的close方法关闭预编译语句对象。
pstmt.close();
```

1.2.4 前向兼容与安全增强

前向兼容

在上文中，支持通过key_info设置访问外部密钥管理的参数：

1. 使用gsql时，通过元命令\key_info xxx设置。
2. 使用JDBC时，通过连接参数conn.setProperty(“key_info”，“xxx”)设置。

为保持前向兼容，还支持通过环境变量等方式设置访问主密钥的参数。

⚠ 注意

第一次配置使用密态数据库时，可忽略下述方法。如果以前使用下述方法配置密态数据库，建议改用’key_info’配置。

使用系统级环境变量配置的方式如下：

```
export HUAWEI_KMS_INFO='iamUrl=https://iam.{项目}.myhuaweicloud.com/v3/auth/tokens,iamUser={IAM用户名},iamPassword={IAM用户密码},iamDomain={账号名},kmsProject={项目}'
# 该方法中操作系统日志可能会记录环境变量中的敏感信息，使用过程中注意及时清理。
```

还可通过标准库接口设置进程级环境变量，不同语言设置方法如下：

1. C/C++
   ```
   setenv("HIS_KMS_INFO", "xxx");
   ```
2. GO
   ```
   os.Setenv("HIS_KMS_INFO", "xxx");
   ```

外部密钥服务的身份验证

当数据库驱动访问华为云密钥管理服务时，为避免攻击者伪装为密钥服务，在数据库驱动与密钥服务建立https连接的过程中，可通过CA证书验证密钥服务器的合法性。为此，需提前配置CA证书，如果未配置，将不会验证密钥服务的身份。本节介绍如何下载与配置CA证书。

配置方法

在key_info参数的中，增加证书相关参数即可。

版权所有 © 华为云计算技术有限公司

- 使用gsql时

```
gaussdb=# \key_info keyType=hcs_kms,iamUrl=https://iam.xxx.com/v3/auth/tokens,iamUser={IAM用户名},iamPassword={IAM用户密码},iamDomain={账号名},kmsProject={项目},iamCaCert=/路径/IAM的CA证书文件,kmsCaCert=/路径/KMS的CA证书文件

gaussdb=# \key_info keyType=hcs_kms,kmsProjectId={项目ID},ak={AK},sk={SK},kmsCaCert=/路径/KMS的CA证书文件
```

- 使用JDBC时

```
conn.setProperty("key_info", "keyType=hcs_kms," +
    "iamUrl=https://iam.{xxx.com/v3/auth/tokens," +
    "iamUser={IAM用户名}," +
    "iamPassword={IAM用户密码}," +
    "iamDomain={账号名}," +
    "kmsProject={项目}," +
    "iamCaCert=/路径/IAM的CA证书文件," +
    "kmsCaCert=/路径/KMS的CA证书文件");

conn.setProperty("key_info", "keyType=hcs_kms, kmsProjectId={项目ID}, ak={AK}, sk={SK},
kmsCaCert=/路径/KMS的CA证书文件");
```

获取证书

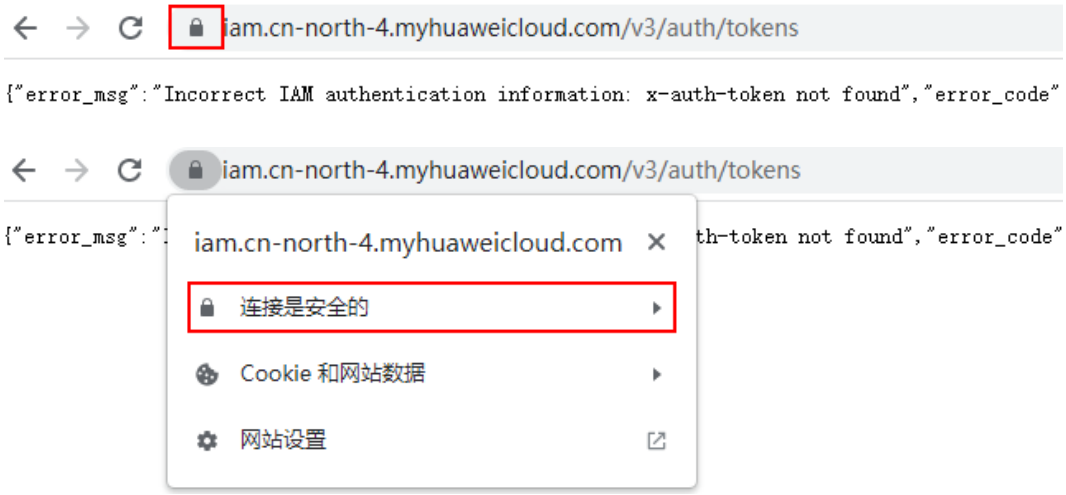
大部分浏览器均会自动下载网站对应的CA证书，并提供证书导出功能。虽然，诸如https://www.ssleye.com/ssltool/certs_down.html等很多网站也提供自动下载CA证书的功能，但可能因本地环境中存在代理或网关，导致CA证书无法正常使用。所以，建议借助浏览器下载CA证书。下载方式如下：

⚠ 注意

由于使用restful接口访问密钥服务，当在浏览器输入接口对应的url时，可忽略下述步骤2中的失败页面，因为即使在失败的情况下，浏览器也早已提前自动下载CA证书。

步骤1 输入域名：打开浏览器，在华为云场景中，分别输入IAM服务器地址和KMS服务器地址，地址获取方式参见：生成主密钥阶段。

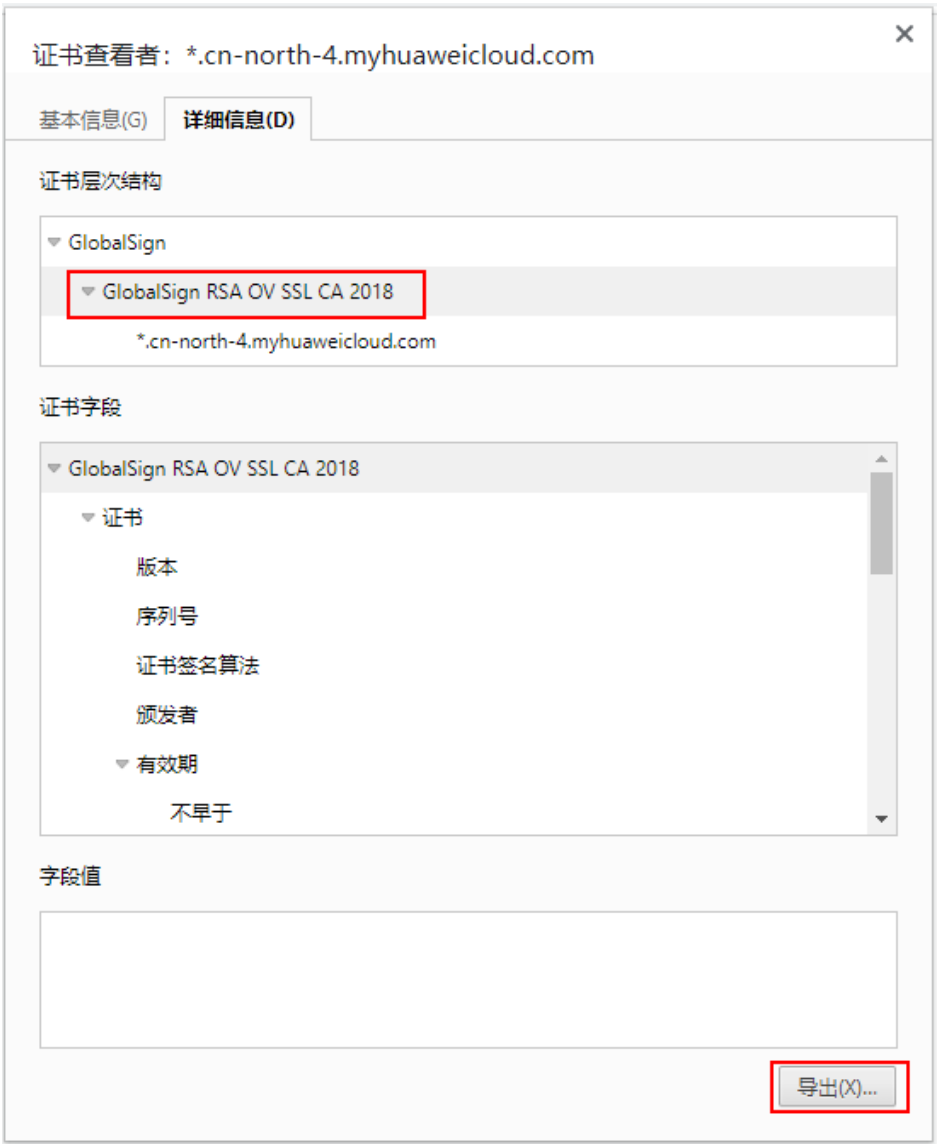
步骤2 查找证书：在每次输入域名后，找到SSL连接相关信息，单击后会发现证书，继续单击可查看证书内容。



版权所有 © 华为云计算技术有限公司



步骤3 导出证书：在证书查看页面，可能会看到证书分为很多级，仅需要域名的上一级证书即可，选择该证书并单击导出，便可直接生成证书文件，即需要的证书文件。



步骤4 上传证书：将导出的证书上传至应用端，并配置到上述参数中即可。

----结束

版权所有 © 华为云计算技术有限公司

1.2.5 密态支持函数/存储过程

密态支持函数/存储过程，当前版本只支持sql和plpgsql两种语言。由于密态支持存储过程中创建和执行函数/存储过程对用户是无感知的，因此语法和非密态无区别。

函数/存储过程语法参考《开发者指南》中“用户自定义函数”章节和“存储过程”章节。

密态等值查询支持函数存储过程新增系统表gs_encrypted_proc，用于存储参数返回的原始数据类型。

系统表具体字段含义可参考《开发者指南》中“系统表和系统视图 > 系统表 > GS_ENCRYPTED_PROC”章节。

创建并执行涉及加密列的函数/存储过程

步骤1 创建密钥，详细步骤请参见**使用gsql操作密态数据库**。

步骤2 创建加密表。

```
gaussdb=# CREATE TABLE creditcard_info (
  id_number int,
  name  text,
  credit_card varchar(19) encrypted with (column_encryption_key = cek1, encryption_type = DETERMINISTIC)
 ) with (orientation=row) distribute by hash(id_number);
CREATE TABLE
```

步骤3 插入数据。

```
gaussdb=# insert into creditcard_info values(1, 'Avi', '1234567890123456');
INSERT 0 1
gaussdb=# insert into creditcard_info values(2, 'Eli', '2345678901234567');
INSERT 0 1
```

步骤4 创建函数支持密态等值查询。

```
gaussdb=# CREATE FUNCTION f_encrypt_in_sql(val1 text, val2 varchar(19)) RETURNS text AS 'SELECT name from creditcard_info where name=$1 or credit_card=$2 LIMIT 1' LANGUAGE SQL;
CREATE FUNCTION
gaussdb=# CREATE FUNCTION f_encrypt_in_plpgsql (val1 text, val2 varchar(19), OUT c text) AS $$
 BEGIN
 SELECT into c name from creditcard_info where name=$1 or credit_card =$2 LIMIT 1;
 END; $$
 LANGUAGE plpgsql;
CREATE FUNCTION
```

步骤5 执行函数。

```
gaussdb=# SELECT f_encrypt_in_sql('Avi','1234567890123456');
 f_encrypt_in_sql
------------------
 Avi
(1 row)

gaussdb=# SELECT f_encrypt_in_plpgsql('Avi', val2=>'1234567890123456');
 f_encrypt_in_plpgsql
----------------------
 Avi
(1 row)
```

----结束

版权所有 © 华为云计算技术有限公司

说明

1. 函数/存储过程中“执行动态查询语句”中的查询是在执行过程中编译，因此函数/存储过程中的表名、列名不能在创建阶段未知，输入参数不能用于表名、列名或以任何方式连接。
2. 函数/存储过程中的“执行动态查询语句”不支持EXECUTE 'query'中带有需要加密的数据值。
3. 在RETURNS、IN和OUT的参数中，不支持混合使用加密和非加密类型参数。虽然参数类型都是原始数据类型，但实际类型不同。
4. 在高级包接口中，如dbe_output.print_line()等在服务端打印输出的接口不会做解密操作，由于加密数据类型在强转成明文原始数据类型时会打印出该数据类型的默认值。
5. 当前版本函数/存储过程的LANGUAGE只支持SQL和plpgsql，不支持C和JAVA等其他过程语言。
6. 不支持在函数/存储过程中执行其他查询加密列的函数/存储过程。
7. 当前版本不支持default、DECLARE中为变量赋予默认值，且不支持对DECLARE中的返回值进行解密，用户可以在执行函数时用输入参数、输出参数来代替使用。
8. 不支持gs_dump对涉及加密列的function进行备份。
9. 不支持在函数/存储过程中创建密钥。
10. 该版本密态函数/存储过程不支持触发器。
11. 密态等值查询函数/存储过程不支持对plpgsql语言对语法进行转义，对于语法主体带有引号的CREATE FUNCTION AS ‘语法主体'，可以用CREATE FUNCTION AS $$语法主体$$代替。
12. 不支持在密态等值查询函数/存储过程中执行修改加密列定义的操作，包括对创建加密表，添加加密列，由于执行函数是在服务端，客户端没法判断是否需要刷新缓存，需断开连接后或触发刷新客户端加密列缓存才可以对该列做加密操作。
13. 不支持使用密态数据类型（byteawithoutorderwithequalcol、byteawithoutordercol、_byteawithoutorderwithequalcol、_byteawithoutordercol）创建函数和存储过程。
14. 密态函数若返回值有加密类型，不支持返回不确定的行类型结果，如RETURN [SETOF] RECORD，可以使用返回可确定的行类型结果替代，如RETURN TABLE(columnname typename[,...])。
15. 密态支持函数在创建加密函数时会在系统表gs_encrypted_proc中添加参数对应的加密列的OID，因此删除表后重建同名表可能会使密态函数失效，需要重新创建密态函数。

1.3 透明数据加密

透明加密提供表级数据加密存储功能。当用户使用本特性提供的语法创建加密表后，数据库向磁盘写入加密表数据前，会自动将其加密；同时，数据库从磁盘读取加密表数据后，会自动将其解密。向加密表中进行数据插入、更新、查询和删除等语法与非加密表一致。

说明

- 透明加密基本原理请参见《特性描述》的“数据库安全 > 透明数据加密”章节。
- 使用透明加密前需联系管理员配置开启该功能。

查看透明加密基本配置

步骤1 查看透明加密功能是否已开启

enable_tde取值为on时表示开启，取值为off是表示关闭。该参数由管理员设置。

```
gaussdb=# SHOW enable_tde;
 enable_tde
------------
 on
(1 row)
```

 版权所有 © 华为云计算技术有限公司

步骤2 查看是否已设置访问密钥管理服务的参数

tde_key_info参数为空时表示未设置，tde_key_info不为空时表示已设置。该参数由管理员设置。

```
gaussdb=# show tde_key_info;
     tde_key_info
-------------------------
 keyType=...
```

----结束

操作加密表

步骤1 创建加密表。

创建表时，通过在WITH子句中设置enable_tde=on参数，即可设置该表为加密表。

数据库默认使用'AES_128_CTR'算法对加密表进行加密，如需使用其他算法，可通过encrypt_algo参数设置。

```
gaussdb=# CREATE TABLE t1 (c1 INT, c2 TEXT) WITH (enable_tde = on);
CREATE TABLE
gaussdb=# CREATE TABLE t2 (c1 INT, c2 TEXT) WITH (enable_tde = on, encrypt_algo = 'SM4_CTR');
CREATE TABLE
```

步骤2 查看加密表基本信息。

加密表基本信息存储在pg_class系统表中的reloptions字段中。其中，dek_cipher为数据密钥密文，由数据库自动生成，并由密钥管理服务加密。每个加密表都有1个独立的数据密钥。

```
gaussdb=# SELECT relname,reloptions FROM pg_class WHERE relname = 't1';
 relname | reloptions
---------
+-------------------------------------------------------------------------------------------------------------------------
 t1      |
{orientation=row,enable_tde=on,encrypt_algo=AES_128_CTR,compression=no,storage_type=USTORE,key_type
=...,dek_cipher=...
```

步骤3 向加密表写入数据。

操作加密表与非加密表的语法一致。数据库将表中数据写入磁盘前，才会自动对加密表的数据进行加密。

```
gaussdb=# INSERT INTO t1 VALUES (1, 'tde plain 123');
INSERT 0 1
```

步骤4 从加密表查询数据。

对于合法用户而言，查询加密表与非加密表的语法一致，加解密操作由数据库自动实现。如果攻击者绕过数据库，直接读取磁盘上加密表对应的数据文件，会发现文件中的数据均已被加密。

```
gaussdb=# SELECT * FROM t1;
 c1 |      c2
----+---------------
  1 | tde plain 123
(1 row)
```

步骤5 轮转加密表的密钥。

为提高安全性，建议定期使用以下语法轮转加密表的数据密钥，即使用新的密钥对数据进行加密。

```
gaussdb=# ALTER TABLE t1 ENCRYPTION KEY ROTATION;
ALTER TABLE
```

轮转密钥后，数据库仍可以正常解密由旧密钥加密的数据。

 版权所有 © 华为云计算技术有限公司

步骤6 加密表与非加密表转换。

透明加密支持将加密表转换为非加密表，以及将非加密表转换为加密表。建议在每次转换后，手动执行VACUUM FULL tablename命令，以强制同步转换表中所有数据。

```
gaussdb=# CREATE TABLE t3 (c1 INT, c2 TEXT);
CREATE TABLE
gaussdb=# ALTER TABLE t3 SET (enable_tde = on);
ALTER TABLE
gaussdb=# VACUUM FULL t3;
VACUUM
gaussdb=# ALTER TABLE t3 SET (enable_tde = off);
ALTER TABLE
gaussdb=# VACUUM FULL t3;
VACUUM
```

步骤7 删除加密表。

```
gaussdb=# DROP TABLE IF EXISTS t1, t2, t3;
DROP TABLE
```

----结束

操作加密索引

步骤1 创建加密表。

创建索引的基表，需确保基表也是加密表。

```
gaussdb=# CREATE TABLE t1 (c1 INT, c2 TEXT) WITH (enable_tde = on);
CREATE TABLE
```

步骤2 创建加密索引。

与创建加密表的方式相同，通过在WITH子句中设置enable_tde=on参数，即将索引设置为加密索引。

索引与基表使用相同的加密算法和密钥，对基表进行密钥轮转时，索引也会使用新密钥。

```
gaussdb=# CREATE INDEX i1 ON t1(c2) WITH (enable_tde = on);
CREATE INDEX
```

步骤3 查看加密索引基本信息。

与加密表一样，索引基本信息也存储在pg_class系统表中的reloptions字段中，索引的dek_cipher、encrypt_algo等参数与基表保持一致。

```
gaussdb=# SELECT relname,reloptions FROM pg_class WHERE relname = 'i1';
 relname | reloptions
---------
+-----------------------------------------------------------------------------------------------------------------------
 i1      |
{orientation=row,enable_tde=on,encrypt_algo=AES_128_CTR,compression=no,storage_type=USTORE,key_type
=...,dek_cipher=...
```

步骤4 加密索引与非加密索引转换。

透明加密支持将非加密索引转换为加密索引，将加密索引转换为非加密索引。

```
gaussdb=# CREATE TABLE t2 (c1 INT, c2 TEXT) WITH (enable_tde = on);
ALTER TABLE
gaussdb=# CREATE INDEX i2 ON t2(c2);
CREATE INDEX
gaussdb=# ALTER INDEX i2 SET (enable_tde = on);
ALTER INDEX
gaussdb=# ALTER INDEX i2 SET (enable_tde = off);
ALTER INDEX
```

 版权所有 © 华为云计算技术有限公司

步骤5 自动对索引进行加密。

默认情况下，主动设置enable_tde参数才可创建加密索引。当设置GUC参数tde_index_default_encrypt=on，且以加密表为基表创建索引时，数据库会自动将索引转换为加密索引。示例如下：

```
gaussdb=# CREATE TABLE t3 (c1 INT, c2 TEXT) WITH (enable_tde = on);
ALTER TABLE
gaussdb=# CREATE INDEX i3 ON t3(c2);
CREATE INDEX
gaussdb=# SELECT relname,reloptions FROM pg_class WHERE relname = 'i3';
 relname | reloptions
---------
+-----------------------------------------------------------------------------------------------------------------------
 i1      |
{orientation=row,enable_tde=on,encrypt_algo=AES_128_CTR,compression=no,storage_type=USTORE,key_type
=...,dek_cipher=...

-- 解释：虽然未指定i3为加密索引，但是开启了tde_index_default_encrypt=on，且基表t3是加密表，数据库自动
将i3转换为加密索引
```

步骤6 删除加密表和索引。

```
gaussdb=# DROP TABLE IF EXISTS t1, t2, t3;
DROP TABLE
```

----结束

1.4 设置账本数据库

1.4.1 账本数据库概述

背景信息

账本数据库融合了区块链思想，将用户操作记录至两种历史表：用户历史表和全局区块表中。当用户创建防篡改用户表时，系统将自动为该表添加一个hash列来保存每行数据的hash摘要信息，同时在blockchain模式下会创建一张用户历史表来记录对应用户表中每条数据的变更行为；而用户对防篡改用户表的每一次修改行为将被记录到全局区块表中。由于历史表具有只可追加不可修改的特点，因此历史表记录串联起来便形成了用户对防篡改用户表的修改历史。

用户历史表命名和结构如下：

表 1-3 用户历史表 blockchain.<schemaname>_<tablename>_hist 所包含的字段

字段名	类型	描述
rec_num	bigint	行级修改操作在历史表中的执行序号。
hash_ins	hash16	INSERT或UPDATE操作插入的数据行的hash值。
hash_del	hash16	DELETE或UPDATE操作删除数据行的hash值。
pre_hash	hash32	当前用户历史表的数据整体摘要。

版权所有 © 华为云计算技术有限公司

表 1-4 hash_ins 与 hash_del 场景对应关系

-	hash_ins	hash_del
INSERT	(√) 插入行的hash值。	空
DELETE	空	(√) 删除行的hash值。
UPDATE	(√) 新插入数据的hash值。	(√) 删除前该行的hash值。

操作步骤

步骤1 创建防篡改模式。

例如，创建防篡改模式ledgernsp。

```
gaussdb=# CREATE SCHEMA ledgernsp WITH BLOCKCHAIN;
```

说明

如果需要创建防篡改模式或更改普通模式为防篡改模式，则需设置enable_ledger参数为on。enable_ledger默认参数为off。

步骤2 在防篡改模式下创建防篡改用户表。

例如，创建防篡改用户表ledgernsp.usertable。

```
gaussdb=# CREATE TABLE ledgernsp.usertable(id int, name text);
```

查看防篡改用户表结构及其对应的用户历史表结构。

```
gaussdb=# \d+ ledgernsp.usertable;
gaussdb=# \d+ blockchain.ledgernsp_usertable_hist;
```

执行结果如下：

```
gaussdb=# \d+ ledgernsp.usertable;
                    Table "ledgernsp.usertable"
 Column     |  Type   | Modifiers | Storage  | Stats target | Description
--------+---------+-----------+----------+--------------+-------------
 id         | integer |           | plain    |              |
 name       | text    |           | extended |              |
 hash_7a0c87   | hash16  |           | plain    |              |
Has OIDs: no
Distribute By: HASH(id)
Location Nodes: ALL DATANODES
Options: orientation=row, compression=no
History table name: ledgernsp_usertable_hist

gaussdb=# \d+ blockchain.ledgernsp_usertable_hist;
             Table "blockchain.ledgernsp_usertable_hist"
  Column  |  Type  | Modifiers | Storage | Stats target | Description
----------+--------+-----------+---------+--------------+-------------
 rec_num  | bigint |           | plain   |              |
 hash_ins | hash16 |           | plain   |              |
 hash_del | hash16 |           | plain   |              |
 pre_hash | hash32 |           | plain   |              |
Indexes:
    "gs_hist_16388_index" PRIMARY KEY, btree (rec_num int4_ops) TABLESPACE pg_default
Has OIDs: no
Distribute By: HASH(rec_num)
Location Nodes: ALL DATANODES
Options: internal_mask=263
```


版权所有 © 华为云计算技术有限公司

说明

1. 防篡改模式下仅行存表为防篡改表，临时表、外表、unlog表及非行存表均无防篡改属性。
2. 防篡改表在创建时会自动增加一个用于校验的系统列，所以防篡改表单表最大列数为1599。

步骤3 修改防篡改用户表数据。

例如，对防篡改用户表执行INSERT、UPDATE、DELETE操作。

```
gaussdb=# INSERT INTO ledgernsp.usertable VALUES(1, 'alex'), (2, 'bob'), (3, 'peter');
INSERT 0 3
gaussdb=# SELECT *, hash_7a0c87 FROM ledgernsp.usertable ORDER BY id;
 id | name  |       hash_7a0c87
----+-------+------------------
  1 | alex  | 1f2e543c580cb8c5
  2 | bob   | 8fcd74a8a6a4b484
  3 | peter | f51b4b1b12d0354b
(3 rows)

gaussdb=# UPDATE ledgernsp.usertable SET name = 'bob2' WHERE id = 2;
UPDATE 1
gaussdb=# SELECT *, hash_7a0c87 FROM ledgernsp.usertable ORDER BY id;
 id | name  |       hash_7a0c87
----+-------+------------------
  1 | alex  | 1f2e543c580cb8c5
  2 | bob2  | 437761affbb7c605
  3 | peter | f51b4b1b12d0354b
(3 rows)

gaussdb=# DELETE FROM ledgernsp.usertable WHERE id = 3;
DELETE 1
gaussdb=# SELECT *, hash_7a0c87 FROM ledgernsp.usertable ORDER BY id;
 id | name |       hash_7a0c87
----+------+------------------
  1 | alex | 1f2e543c580cb8c5
  2 | bob2 | 437761affbb7c605
(2 rows)
```

步骤4 删除表和模式。

若要执行其他账本数据库章节的示例，请在执行完之后再执行当前步骤，否则请直接执行当前步骤。

```
gaussdb=# DROP TABLE ledgernsp.usertable;
DROP TABLE
gaussdb=# DROP SCHEMA ledgernsp;
DROP SCHEMA
```

----结束

1.4.2 查看账本历史操作记录

前提条件

- 系统中需要有审计管理员或者具有审计管理员权限的角色。
- 数据库正常运行，并且对防篡改数据库执行了一系列增、删、改等操作，保证在查询时段内有账本操作记录结果产生。
- 数据库各个CN节点全局区块表记录单独记录，全局区块表只能记录连接到当前CN执行的SQL操作。

 版权所有 © 华为云计算技术有限公司

背景信息

- 只有拥有AUDITADMIN属性的用户才可以查看账本历史操作记录。有关数据库用户及创建用户的办法请参见《开发者指南》中“数据库安全 > 用户及权限 > 用户”章节。
- 查询全局区块表命令是直接查询gs_global_chain表，操作为：

  ```
  SELECT * FROM gs_global_chain;
  ```

  该表有10个字段，每个字段的含义请参见《开发者指南》中“系统表和系统视图 > 系统表 > GS_GLOBAL_CHAIN”章节。
- 查询用户历史表的命令是直接查询BLOCKCHAIN模式下的用户历史表，操作为：

  例如用户表所在的模式为ledgernsp，表名为usertable，则对应的用户历史表名为blockchain.ledgernsp_usertable_hist：

  ```
  SELECT * FROM blockchain.ledgernsp_usertable_hist;
  ```

  用户历史表有4个字段，每个字段的含义请参见**表1-3**。

  说明

  用户历史表的表名一般为blockchain.<schemaname>_<tablename>_hist形式。当防篡改用户表模式名或者表名过长导致前述方式生成的表名超出表名长度限制，则会采用blockchain.<schema_oid>_<table_oid>_hist的方式命名。

操作步骤

步骤1 查询全局区块表记录。

```
gaussdb=# SELECT * FROM gs_global_chain;
```

查询结果如下：

```
 blocknum |  dbname  | username |          starttime          | relid |  relnsp   |  relname  |     relhash
|           globalhash           |
                txcommand
----------+----------+----------+-----------------------------+-------+-----------+-----------+-----------------
+----------------------------------+------------------
-------------------------------------------------------------
        0 | testdb | omm      | 2021-04-14 07:00:46.32757+08  | 16393 | ledgernsp | usertable |
a41714001181a294 | 6b5624e039e8aee36bff3e8295c75b40 | insert into ledge
rnsp.usertable values(1, 'alex'), (2, 'bob'), (3, 'peter');
        1 | testdb | omm      | 2021-04-14 07:01:19.767799+08 | 16393 | ledgernsp | usertable |
b3a9ed0755131181 | 328b48c4370faed930937869783c23e0 | update ledgernsp.
usertable set name = 'bob2' where id = 2;
        2 | testdb | omm      | 2021-04-14 07:01:29.896148+08 | 16393 | ledgernsp | usertable |
0ae4b4e4ed2fcab5 | aa8f0a236357cac4e5bc1648a739f2ef | delete from ledge
rnsp.usertable where id = 3;
```

该结果表明，用户omm连续执行了三条DML命令，包括INSERT、UPDATE和DELETE操作。

步骤2 查询历史表记录。

```
gaussdb=# SELECT * FROM blockchain.ledgernsp_usertable_hist;
```

查询结果如下：

```
 rec_num |     hash_ins     |     hash_del     |             pre_hash
---------+------------------+------------------+----------------------------------
       0 | 1f2e543c580cb8c5 |                  | e1b664970d925d09caa295abd38d9b35
       1 | 8fcd74a8a6a4b484 |                  | dad3ed8939a141bf3682043891776b67
       2 | f51b4b1b12d0354b |                  | 53eb887fc7c4302402343c8914e43c69
       3 | 437761affbb7c605 | 8fcd74a8a6a4b484 | c2868c5b49550801d0dbbbaa77a83a10
       4 |                  | f51b4b1b12d0354b | 9c512619f6ffef38c098477933499fe3
(5 rows)
```

 版权所有 © 华为云计算技术有限公司

查询结果显示，用户omm对ledgernsp.usertable表插入了3条数据，更新了1条数据，随后删除了1行数据，最后剩余2行数据，hash值分别为1f2e543c580cb8c5和437761affbb7c605。

步骤3 查询用户表数据及校验列。

```
gaussdb=# SELECT *, hash_7a0c87 FROM ledgernsp.usertable;
```

查询结果如下：

```
 id | name |       hash_7a0c87
----+------+------------------
  1 | alex | 1f2e543c580cb8c5
  2 | bob2 | 437761affbb7c605
(2 rows)
```

查询结果显示，用户表中剩余2条数据，与2中的记录一致。

----结束

1.4.3 校验账本数据一致性

前提条件

数据库正常运行，并且对防篡改数据库执行了一系列增、删、改等操作，保证在查询时段内有账本操作记录结果产生。

背景信息

- 账本数据库校验功能目前提供两种校验接口，分别为：ledger_hist_check(text, text)和ledger_gchain_check(text, text)。普通用户调用校验接口，仅能校验自己有权限访问的表。
- 校验防篡改用户表和用户历史表的接口为pg_catalog.ledger_hist_check，操作为：

  ```
  SELECT pg_catalog.ledger_hist_check(schema_name text,table_name text);
  ```

  如果校验通过，函数返回t，反之则提示失败原因并返回f。
- 校验防篡改用户表、用户历史表和全局区块表三者是否一致的接口为pg_catalog.ledger_gchain_check，操作为：

  ```
  SELECT pg_catalog.ledger_gchain_check(schema_name text, table_name text);
  ```

  如果校验通过，函数返回t，反之则提示失败原因并返回f。

操作步骤

步骤1 校验防篡改用户表ledgernsp.usertable与其对应的历史表是否一致。

```
gaussdb=# SELECT pg_catalog.ledger_hist_check('ledgernsp', 'usertable');
```

查询结果如下：

```
 ledger_hist_check
-------------------
 t
(1 row)
```

该结果表明防篡改用户表和用户历史表中记录的结果能够一一对应，保持一致。

步骤2 查询防篡改用户表ledgernsp.usertable与其对应的历史表以及全局区块表中关于该表的记录是否一致。

```
gaussdb=# SELECT pg_catalog.ledger_gchain_check('ledgernsp', 'usertable');
```


版权所有 © 华为云计算技术有限公司

查询结果如下：

```
 ledger_gchain_check
---------------------
 t
(1 row)
```

查询结果显示，上述三表中关于ledgernsp.usertable的记录保持一致，未发生篡改行为。

----结束

1.4.4 归档账本数据库

前提条件

- 系统中需要有审计管理员或者具有审计管理员权限的角色。
- 数据库正常运行，并且对防篡改数据库执行了一系列增、删、改等操作，保证在查询时段内有账本操作记录结果产生。
- 数据库已经正确配置审计文件的存储路径audit_directory。

背景信息

- 账本数据库归档功能目前提供两种校验接口，分别为：ledger_hist_archive(text, text)和ledger_gchain_archive(text, text)。账本数据库接口仅审计管理员可以调用。
- 归档用户历史表的接口为pg_catalog.ledger_hist_archive，表示归档当前DN的用户历史表数据。执行操作为：
  ```
  SELECT pg_catalog.ledger_hist_archive(schema_name text,table_name text);
  ```
  如果归档成功，函数返回t，反之则提示失败原因并返回f。
- 归档全局区块表的接口为pg_catalog.ledger_gchain_archive，表示归档当前CN的全局历史表数据。执行操作为：
  ```
  SELECT pg_catalog.ledger_gchain_archive();
  ```
  如果归档成功，函数返回t，反之则提示失败原因并返回f。

操作步骤

步骤1 使用EXECUTE DIRECT对某个DN节点进行归档操作。

```
gaussdb=# EXECUTE DIRECT ON (datanode1) 'select pg_catalog.ledger_hist_archive(''ledgernsp'', ''usertable'');';
```

查询结果如下：

```
 ledger_hist_archive
---------------------
 t
(1 row)
```

用户历史表将归档为一条数据：

```
gaussdb=# EXECUTE DIRECT ON (datanode1) 'SELECT * FROM blockchain.ledgernsp_usertable_hist;';
 rec_num |     hash_ins     |     hash_del     |             pre_hash
---------+------------------+------------------+----------------------------------
       3 | e78e75b00d396899 | 8fcd74a8a6a4b484 | fd61cb772033da297d10c4e658e898d7
(1 row)
```

该结果表明datanode1节点用户历史表导出成功。

步骤2 连接CN执行全局区块表导出操作。

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# SELECT pg_catalog.ledger_gchain_archive();
```

查询结果如下：

```
 ledger_gchain_archive
-----------------------
 t
(1 row)
```

全局历史表将以用户表为单位归档为N（用户表数量）条数据：

```
gaussdb=# SELECT * FROM gs_global_chain;
 blocknum |  dbname  | username |           starttime           | relid |  relnsp   |  relname  |     relhash
|            globalhash            | txcommand
----------+----------+----------+-------------------------------+-------+-----------+-----------+------------------
+----------------------------------+-----------
        1 | testdb | libc     | 2021-05-10 19:59:38.619472+08 | 16388 | ledgernsp | usertable |
57c101076694b415 | be82f98ee68b2bc4e375f69209345406 | Archived.
(1 row)
```

该结果表明，当前coordinator节点全局区块表导出成功。

----结束

1.4.5 修复账本数据库

前提条件

- 系统中需要有审计管理员或者具有审计管理员权限的角色。
- 数据库正常运行，并且对防篡改数据库执行了一系列增、删、改等操作，保证在查询时段内有账本操作记录结果产生。

背景信息

- 当前的账本数据库机制为：全局区块表存储在CN端，各个CN数据独立。用户历史表存储在DN端，历史表记录的数据为所在DN防篡改表的数据变化。因此，在触发数据重分布时，可能导致防篡改表和用户历史表数据不一致，此时需要使用ledger_hist_repair(text, text)接口对指定DN节点的用户历史表进行修复，修复后当前DN节点调用历史表校验接口结果为true。在CN剔除、修复的场景下，可能导致全局区块表数据丢失或者与用户历史表不一致，此时需要使用ledger_gchain_repair(text, text)接口对整个集群范围内的全局区块表进行修复，修复后调用全局区块表校验接口结果为true。
- 修复用户历史表的接口为pg_catalog.ledger_hist_repair，操作为：
  ```
  SELECT pg_catalog.ledger_hist_repair(schema_name text,table_name text);
  ```
  如果修复成功，函数返回修复过程中用户历史表hash的增量。

  注：对用户表执行闪回DROP时，可使用该函数恢复用户表和用户历史表名称，请参见**恢复用户表和用户历史表名称**。
- 修复全局区块表的接口为pg_catalog.ledger_gchain_repair，操作为：
  ```
  SELECT pg_catalog.ledger_gchain_repair(schema_name text,table_name text);
  ```
  如果修复成功，函数返回修复过程中全局区块表中指定表的hash总和。

恢复用户表数据和全局区块表数据

以omm用户为例进行操作，步骤如下。

步骤1 以操作系统用户omm登录数据库主节点。

 版权所有 © 华为云计算技术有限公司

步骤2 使用EXECUTE DIRECT对某个DN节点进行历史表修复操作。

```
gaussdb=# EXECUTE DIRECT ON (datanode1) 'select pg_catalog.ledger_hist_repair(''ledgernsp'', ''usertable'');';
```

查询结果如下：

```
 ledger_hist_repair
--------------------
 84e8bfc3b974e9cf
(1 row)
```

该结果表明datanode1节点用户历史表修复成功，修复造成的用户历史表hash增量为84e8bfc3b974e9cf。

步骤3 连接CN执行全局区块表修复操作。

```
gaussdb=# SELECT pg_catalog.ledger_gchain_repair('ledgernsp', 'usertable');
```

查询结果如下：

```
 ledger_gchain_repair
----------------------
 a41714001181a294
(1 row)
```

该结果表明，当前集群全局区块表修复成功，且向当前CN节点插入一条修复数据，其hash值为a41714001181a294。

----结束

恢复用户表和用户历史表名称

已通过enble_recyclebin参数和recyclebin_retention_time参数开启闪回DROP功能，恢复用户表和用户历史表名称。示例如下：

- DROP用户表，对用户表执行闪回DROP。使用ledger_hist_repair对用户表、用户历史表进行表名恢复。

```
-- 对用户表执行闪回drop，使用ledger_hist_repair对用户历史表进行表名恢复。
gaussdb=# CREATE TABLE ledgernsp.tab2(a int, b text);
NOTICE:  The 'DISTRIBUTE BY' clause is not specified. Using 'a' as the distribution column by default.
HINT:  Please use 'DISTRIBUTE BY' clause to specify suitable data distribution column.
NOTICE:  The 'DISTRIBUTE BY' clause is not specified. Using 'rec_num' as the distribution column by default.
HINT:  Please use 'DISTRIBUTE BY' clause to specify suitable data distribution column.
CREATE TABLE
gaussdb=# DROP TABLE ledgernsp.tab2;
DROP TABLE
gaussdb=# SELECT rcyrelid, rcyname, rcyoriginname FROM gs_recyclebin;
 rcyrelid |           rcyname              |    rcyoriginname
----------+--------------------------------+---------------------
    32838 | BIN$39B523388046$55C8400==$0 | tab2
    32846 | BIN$39B52338804E$55C90E8==$0 | gs_hist_tab2_index
    32843 | BIN$39B52338804B$55C96A0==$0 | ledgernsp_tab2_hist
    32841 | BIN$39B523388049$55C9EE0==$0 | pg_toast_32838
(4 rows)
-- 对用户表执行闪回drop。
gaussdb=# TIMECAPSULE TABLE ledgernsp.tab2 TO BEFORE DROP;
TimeCapsule Table
-- 使用ledger_hist_repair恢复用户历史表表名。
gaussdb=# SELECT ledger_hist_repair('ledgernsp', 'tab2');
 ledger_hist_repair
--------------------
 0000000000000000
(1 row)

gaussdb=# \d+ ledgernsp.tab2;
                        Table "ledgernsp.tab2"
  Column   |  Type   | Modifiers | Storage  | Stats target | Description
```

 版权所有 © 华为云计算技术有限公司

```
-------------+---------+-----------+----------+--------------+-------------
 a           | integer |           | plain    |              |
 b           | text    |           | extended |              |
 hash_1d2d14 | hash16  |           | plain    |              |
Has OIDs: no
Distribute By: HASH(a)
Location Nodes: ALL DATANODES
Options: orientation=row, compression=no, storage_type=USTORE, segment=off,
toast.storage_type=USTORE, toast.toast_storage_type=enhanced_toast
History table name: ledgernsp_tab2_hist

-- 对用户表执行闪回drop，使用ledger_hist_repair对用户表进行表名恢复。
gaussdb=# CREATE TABLE ledgernsp.tab3(a int, b text);
NOTICE:  The 'DISTRIBUTE BY' clause is not specified. Using 'a' as the distribution column by default.
HINT:  Please use 'DISTRIBUTE BY' clause to specify suitable data distribution column.
NOTICE:  The 'DISTRIBUTE BY' clause is not specified. Using 'rec_num' as the distribution column by
default.
HINT:  Please use 'DISTRIBUTE BY' clause to specify suitable data distribution column.
CREATE TABLE
gaussdb=# DROP TABLE ledgernsp.tab3;
DROP TABLE
gaussdb=# SELECT rcyrelid, rcyname, rcyoriginname FROM gs_recyclebin;
 rcyrelid |            rcyname             |    rcyoriginname
----------+--------------------------------+---------------------
    32952 | BIN$80B6233880B8$FECFF98==$0 | tab3
    32960 | BIN$80B6233880C0$FED0C98==$0 | gs_hist_tab3_index
    32957 | BIN$80B6233880BD$FED1250==$0 | ledgernsp_tab3_hist
    32955 | BIN$80B6233880BB$FED1A00==$0 | pg_toast_32952
(4 rows)
-- 对用户历史表执行闪回drop。
gaussdb=# TIMECAPSULE TABLE blockchain.ledgernsp_tab3_hist TO BEFORE DROP;
TimeCapsule Table
-- 拿到回收站中用户表对应的rcyname，使用ledger_hist_repair恢复用户表表名。
gaussdb=# SELECT ledger_hist_repair('ledgernsp', 'BIN$80B6233880B8$FECFF98==$0');
 ledger_hist_repair
--------------------
 0000000000000000
(1 row)

gaussdb=# \d+ ledgernsp.tab3;
                      Table "ledgernsp.tab3"
   Column    |  Type   | Modifiers | Storage  | Stats target | Description
-------------+---------+-----------+----------+--------------+-------------
 a           | integer |           | plain    |              |
 b           | text    |           | extended |              |
 hash_7a0c87 | hash16  |           | plain    |              |
Has OIDs: no
Distribute By: HASH(a)
Location Nodes: ALL DATANODES
Options: orientation=row, compression=no, storage_type=USTORE, segment=off,
toast.storage_type=USTORE, toast.toast_storage_type=enhanced_toast
History table name: ledgernsp_tab3_hist

-- 删除表。
gaussdb=# DROP TABLE ledgernsp.tab2 PURGE;
DROP TABLE
gaussdb=# DROP TABLE ledgernsp.tab3 PURGE;
DROP TABLE
```

1.5 基于标签的强制访问控制

版权所有 © 华为云计算技术有限公司

1.5.1 定义安全标签

前提条件

数据库正常运行，执行操作的用户需要具有SYSADMIN权限或者继承了内置角色gs_role_seclabel的权限。

背景信息

提供系统表gs_security_label来查看系统中已创建好的安全标签：

```
SELECT * FROM gs_security_label;
```

操作步骤

以某个实际业务应用举例说明，比如某公司的信息等级为公开、秘密和绝密，那么就可分别对应成等级L1、L2和L3。信息存储范围为一层到五层各层员工的信息，那么可分别对应成范围G1、G2、G3、G4、G5，以此为基础定义安全标签。

步骤1 创建一个安全标签，标志信息等级为公开，范围为二层和四层。

```
gaussdb=# CREATE SECURITY LABEL label1 'L1:G2,G4';
```

步骤2 创建一个安全标签，标志信息等级为秘密，范围为二层到四层。

```
gaussdb=# CREATE SECURITY LABEL label2 'L2:G2-G4';
```

步骤3 创建一个安全标签，标志信息等级为绝密，范围为一层到五层。

```
gaussdb=# CREATE SECURITY LABEL label3 'L3:G1-G5';
```

步骤4 查询系统表gs_security_label。

```
gaussdb=# SELECT * FROM gs_security_label;
```

查询结果如下：

```
gaussdb=# SELECT * FROM gs_security_label;
 label_name | label_content
------------+---------------
 label1     | L1:G2,G4
 label2     | L2:G2-G4
 label3     | L3:G1-G5
(3 rows)
```

步骤5 删除安全标签。

若要执行其他安全标签章节的示例，请在执行完之后再执行当前步骤，否则请直接执行当前步骤。

```
gaussdb=# DROP SECURITY LABEL label1;
gaussdb=# DROP SECURITY LABEL label2;
gaussdb=# DROP SECURITY LABEL label3;
```

----结束

1.5.2 应用安全标签

前提条件

数据库正常运行，执行操作的用户需要具有SYSADMIN权限或者继承了内置角色gs_role_seclabel的权限。

版权所有 © 华为云计算技术有限公司

背景信息

- 提供系统表pg_seclabel来查看表或表列上的安全标签：
  ```
  SELECT * FROM pg_seclabel;
  ```
- 提供系统表pg_shseclabel来查看用户/角色上的安全标签：
  ```
  SELECT * FROM pg_shseclabel;
  ```
- 提供系统视图pg_seclabels来查看系统中所有的安全标签记录：
  ```
  SELECT * FROM pg_seclabels;
  ```

操作步骤

数据库中已有用户user1、user2和数据表tbl。

步骤1 对用户user1应用安全标签label1，表示用户user1对应的信息等级为公开，范围为二层和四层。

```
gaussdb=# SECURITY LABEL ON USER user1 is 'label1';
```

步骤2 对用户user2应用安全标签label3，表示用户user2对应的信息等级为绝密，范围为一层到五层。

```
gaussdb=# SECURITY LABEL ON USER user2 is 'label3';
```

步骤3 对表tbl应用安全标签label2，表示表tbl对应的信息等级为秘密，范围为二层到四层。

```
gaussdb=# SECURITY LABEL ON TABLE tbl is 'label2';
```

步骤4 查询系统视图pg_seclabels。

```
gaussdb=# SELECT * FROM pg_seclabels;
```

查询结果如下：

```
gaussdb=# SELECT * FROM pg_seclabels;
 objoid | classoid | objsubid | objtype | objnamespace | objname | provider | label
--------+----------+----------+---------+--------------+---------+----------+--------
  16399 |     1259 |        0 | table   |         2200 | tbl     | maclabel | label2
  16391 |     1260 |        0 | role    |              | user1   | maclabel | label1
  16395 |     1260 |        0 | role    |              | user2   | maclabel | label3
(3 rows)
```

----结束

1.5.3 基于标签的强制访问控制检查

前提条件

- 数据库正常运行，强制访问控制检查开关（enable_mac_check=on）打开。
- 执行操作的用户已具有所需要访问表的ACL权限。

背景信息

基于安全标签的强制访问控制策略规则由系统内置设定，用户不能更改：

- 插入（INSERT）策略：只有主体安全标记等级小于等于客体安全标记等级且主体安全标记范围是客体安全标记范围的子集时才允许插入数据。
- 查询（SELECT）策略：只有主体安全标记等级大于等于客体安全标记等级且主体安全标记范围是客体安全标记范围的超集时才允许查询数据。
- 修改（UPDATE）和删除（DELETE）策略：只有主体安全标记等级等于客体安全标记等级且主体安全标记范围等于客体安全标记范围时才允许修改和删除数据。

 版权所有 © 华为云计算技术有限公司

- 若客体未标记，则强制访问控制策略对该客体不生效。
- 若主体未标记，则不能访问任意带有标记的客体。

操作步骤

步骤1 授予用户user1和user2数据表tbl的SELECT和INSERT操作权限。

```
gaussdb=# GRANT SELECT,INSERT ON tbl TO user1,user2;
```

步骤2 用户user1登录数据库执行SELECT和INSERT操作，SELECT失败，INSERT成功，符合强制访问控制策略规则。

```
gaussdb=> SELECT current_user;
 current_user
--------------
 user1
(1 row)
gaussdb=> SELECT * FROM tbl;
ERROR:  permission denied for relation tbl
DETAIL:  N/A
gaussdb=> INSERT INTO tbl VALUES (1);
INSERT 0 1
```

步骤3 用户user2登录数据库执行SELECT和INSERT操作，SELECT成功，INSERT失败，符合强制访问控制策略规则。

```
gaussdb=> SELECT current_user;
 current_user
--------------
 user2
(1 row)
gaussdb=> SELECT * FROM tbl;
 fir
-----
   1
(1 row)
gaussdb=> INSERT INTO tbl VALUES (1);
ERROR:  permission denied for relation tbl
DETAIL:  N/A
```

----结束

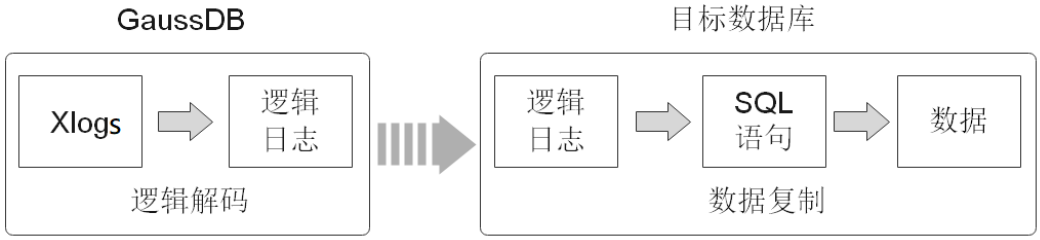
1.6 逻辑复制

GaussDB对数据复制能力的支持情况为：

支持通过数据迁移工具定期向异构数据库（如Oracle等）进行数据同步，不具备实时数据复制能力，因此不足以支撑与异构数据库间并网运行实时数据同步的诉求。

基于上述情况，GaussDB提供了逻辑解码功能，通过反解xlog的方式生成逻辑日志。目标数据库解析逻辑日志以实时进行数据复制。具体如图1-1所示。逻辑复制降低了对目标数据库的形态限制，支持异构数据库、同构异形数据库对数据的同步，支持目标库进行数据同步期间的数据可读写，数据同步时延低。

图 1-1 逻辑复制



版权所有 © 华为云计算技术有限公司

逻辑复制由两部分组成：逻辑解码和数据复制。逻辑解码会输出以事务为单位组织的逻辑日志。业务或数据库中间件将会对逻辑日志进行解析并最终实现数据复制。GaussDB当前只提供逻辑解码功能，因此本章节只涉及逻辑解码的说明。

1.6.1 逻辑解码

1.6.1.1 逻辑解码概述

功能描述

逻辑解码为逻辑复制提供事务解码的基础能力，GaussDB可以使用SQL函数接口进行逻辑解码。此方法调用方便，不需使用工具，对接外部工具接口也比较清晰，不需要额外适配。

由于逻辑日志是以事务为单位的，在事务提交后才能输出，且逻辑解码是由用户驱动的。因此，为了防止事务开始时的xlog被系统回收，或所需的事务信息被VACUUM回收，GaussDB新增了逻辑复制槽，用于阻塞xlog的回收。

一个逻辑复制槽表示一个更改流，这些更改可以在其他数据库上以它们在原数据库上产生的顺序重新执行。每个逻辑复制槽都由其对应逻辑日志的获取者维护。如果处于流式解码中的逻辑复制槽所在库不存在业务，则该复制槽会依照其他库的日志位置来推进。活跃状态的LSN序逻辑复制槽在处理到活跃事务快照日志时可以根据当前日志的LSN推进复制槽；活跃状态的CSN序逻辑复制槽在处理到虚拟事务日志时可以根据当前日志的CSN推进复制槽。

前提条件

- 逻辑日志目前从CN或DN中抽取，如果进行逻辑复制，应使用SSL连接，因此需要保证相应节点上的GUC参数ssl设置为on。

  说明

  为避免安全风险，请保证启用SSL连接。
- 设置GUC参数wal_level为logical。
- 设置GUC参数max_replication_slots>=每个节点所需的（物理流复制槽数+备份槽数+逻辑复制槽数）。

  说明

  关于逻辑复制槽数，请按如下规则考虑：
  - 一个逻辑复制槽只能解码一个Database的修改，如果需要解码多个Database，则需要创建多个逻辑复制槽。
  - 如果需要多路逻辑复制同步给多个目标数据库，在源端数据库需要创建多个逻辑复制槽，每个逻辑复制槽对应一条逻辑复制链路。
  - 同一实例上，最多支持同时开启20个逻辑复制槽进行解码。
- 用户需要通过DN端口连接数据库，才可以直接使用SQL函数接口进行逻辑解码操作，相关操作请参见**使用SQL函数接口进行逻辑解码**。如果使用CN端口连接数据库，则需要通过EXECUTE DIRECT ON (datanode_name) 'statement'语句来执行SQL函数。
- 仅限初始用户或拥有REPLICATION权限的用户进行操作。三权分立关闭时数据库管理员可进行逻辑复制操作，三权分立开启时不允许数据库管理员进行逻辑复制操作。

版权所有 © 华为云计算技术有限公司

注意事项

- 逻辑解码不支持DDL。
- 在执行特定的DDL语句（如普通表truncate或分区表exchange）时，可能造成解码数据丢失。
- 不支持数据页复制的DML解码。
- 当执行DDL语句（如alter table）后，该DDL语句前尚未解码的物理日志可能会丢失。
- 逻辑复制不支持集群在线扩容。在线扩容前，需要删除已存在的逻辑复制槽，扩容完成后重新创建。
- 单条元组大小不超过1GB，考虑解码结果可能大于插入数据，因此建议单条元组大小不超过500MB。
- GaussDB支持解码的数据类型为：INTEGER、BIGINT、SMALLINT、TINYINT、SERIAL、SMALLSERIAL、BIGSERIAL、FLOAT、DOUBLE PRECISION、BOOLEAN、BIT(n)、BIT VARYING(n)、DATE、TIME[WITHOUT TIME ZONE]、TIMESTAMP[WITHOUT TIME ZONE]、CHAR(n)、VARCHAR(n)、TEXT、CLOB（解码成TEXT格式）。
- 逻辑复制槽名称必须小于64个字符，仅支持小写字母、数字以及_?-.字符，且不支持“.”或“..”单独作为复制槽名称。
- 对多库的解码需要分别在库内创建流复制槽并开始解码，每个库的解码都需要单独扫描一遍日志。
- 不支持强切，强切后需要重新全量导出数据。
- 备机解码时，switchover和failover时可能出现解码数据变多，需用户手动过滤。Quorum协议下，switchover和failover选择升主的备机，需要与当前主机日志同步。
- 只支持CN和主DN创建删除复制槽。当删除的复制槽为最后一个复制槽时，删除完成后会产生告警"replicationSlotMinLSN is INVALID_WAL_REC_PTR!!!"和"replicationSlotMaxLSN is INVALID_WAL_REC_PTR!!!"。
- 不允许主备DN，多个备DN同时使用同一个复制槽解码，否则会产生数据不一致的情况。
- 数据库故障重启或逻辑复制进程重启后，解码数据可能存在重复，用户需手动过滤。
- 计算机内核故障后，解码可能存在乱码，需手动或自动过滤。
- 请确保在创建逻辑复制槽过程中未启动长事务，启动长事务会阻塞逻辑复制槽的创建。
- 不支持interval partition表DML复制。
- 不支持全局临时表的DML解码。
- 不支持本地临时表的DML解码。
- M兼容性下，不支持SELECT INTO语句的解码。非M兼容性下，SELECT INTO语句会解码创建目标表的DDL操作，数据插入的DML操作不解码。
- 为解析某个astore表的UPDATE和DELETE语句，需为此表配置REPLICA IDENTITY属性，在此表无主键时需要配置为FULL，具体配置方式请参考《开发者指南》中“SQL参考 > SQL语法 > ALTER TABLE”章节中“REPLICA IDENTITY { DEFAULT | USING INDEX index_name | FULL | NOTHING }”字段。
- 禁止在使用逻辑复制槽时在其他节点对该复制槽进行操作，删除复制槽的操作需在该复制槽停止解码后执行。

版权所有 © 华为云计算技术有限公司

- 基于目标库可能需要源库的系统状态信息考虑，逻辑解码仅自动过滤模式'pg_catalog'和'pg_toast'下OID小于16384的系统表的逻辑日志。若目标库不需要复制其他相关系统表的内容，逻辑日志回放过程中需要对相关系统表进行过滤。
- 在开启逻辑复制的场景下，如需创建包含系统列的主键索引，必须将该表的REPLICA IDENTITY属性设置为FULL或是使用USING INDEX指定不包含系统列的、唯一的、非局部的、不可延迟的、仅包括标记为NOT NULL列的索引。
- 对于缩容或升级前已存在的复制表场景，需要对复制表手动配置logical_repl_node属性或RESET为默认值，配置方式请参考《开发者指南》中“SQL参考 > SQL语法 > ALTER TABLE”章节中“storage_parameter”参数的使用说明，以及“logical_repl_node”属性相关说明。
- 若一个事务的子事务过多导致落盘文件过多，退出解码时需执行SQL函数pg_terminate_backend(*逻辑解码的walsender线程id*)来手动停止解码，而且退出时延增加约为1分钟/30万个子事务。因此在开启逻辑解码时，若一个事务的子事务数量达到5万时，会打印一条WARNING日志。
- 当逻辑复制槽处于非活跃状态，且设置GUC参数enable_xlog_prune=on、enable_logicalrepl_xlog_prune=on、max_size_for_xlog_retention为非零值，且备份槽或逻辑复制槽导致保留日志段数已超过GUC参数wal_keep_segments，同时其他复制槽并未导致更多的保留日志段数时，如果max_size_for_xlog_retention大于0且当前逻辑复制槽导致保留日志的段数（每段日志大小为16MB）超过max_size_for_xlog_retention，或者max_size_for_xlog_retention小于0且磁盘使用率达到(-max_size_for_xlog_retention)/100，当前逻辑复制槽会强制失效，其restart_lsn将被设置为7FFFFFFF/FFFFFFFF。该状态的逻辑复制槽不参与阻塞日志回收或系统表历史版本的回收，但仍占用复制槽的限制数量，需要手动删除。
- 备机解码启动后，向主机发送复制槽推进指令后会占用主机上对应的逻辑复制槽（即标识为活跃状态）。在此之前主机上对应逻辑复制槽为非活跃状态，此状态下如果满足逻辑复制槽强制失效条件则会被标记为失效（即restart_lsn将被设置为7FFFFFFF/FFFFFFFF），备机将无法推进主机复制槽，且备机回放完成复制槽失效日志后当前复制槽的备机解码断开后将无法重连。
- 不活跃的逻辑复制槽将阻塞WAL日志回收和系统表元组历史版本清理，导致磁盘日志堆积和系统表扫描性能下降，因此不再使用的逻辑复制槽请及时清理。需要特别注意，在升级提交之前观察期内使用DN扩展IP连接DN创建的逻辑复制槽，在升级回滚之前务必手动清理，否则随着DN扩展IP特性回滚无法直连DN清理。
- 分布式强一致逻辑解码（连接CN解码）仅支持GTM-Lite分布式部署及流式解码，不支持CN连接备DN进行解码、SQL逻辑解码函数、在线扩容、全局索引。
- 针对分布式强一致逻辑解码（连接CN解码）功能，CN高可用由业务负责切换。
- CN上的CSN序逻辑复制槽仅起到占位作用，不随着逻辑解码的进行而推进，同时也不会阻塞日志回收。
- 通过协议连接CN创建逻辑复制槽仅支持CSN序复制槽，通过协议连接DN创建逻辑复制槽仅支持LSN序复制槽。
- 针对分布式解码，对于故障报错或者手动停止解码客户端等场景，需等待15秒再次重试解码，如有复制槽占用则需通过执行SQL函数pg_terminate_backend(*占用该复制槽线程id*)来手动解除复制槽占用。
- 在CN上创建复制槽失败报错后，需要在CN上进行复制槽删除操作，然后在CN上重新创建复制槽。
- 在CN上删除逻辑复制槽时，若为LSN序逻辑复制槽，则仅删除当前节点复制槽，其他节点同名复制槽不受影响；否则只要其他节点有残留同名CSN序逻辑复制槽，执行删除时不会因为某些节点不存在复制槽而报错，同时所有节点的同名复制槽会被成功删除；如果任何节点均不存在该复制槽，则报错。

版权所有 © 华为云计算技术有限公司

- 在CN上创建CSN序逻辑复制槽时，某些节点如残留同名LSN序逻辑复制槽，需在这些节点执行删除残留复制槽的操作。否则会在除当前CN节点外，其他不存在同名复制槽的CN和主DN节点上创建CSN序逻辑复制槽。
- 如果当前CN节点残留LSN序逻辑复制槽，同时其他某些节点上残留同名CSN序逻辑复制槽，则在当前CN节点上执行删除复制槽操作仅会删除本地LSN序逻辑复制槽，待删除完成再次执行删除操作方可删除其他节点的同名复制槽。
- 解码使用JSON格式输出时不支持数据列包含特殊字符（如'\0'空字符），解码输出列内容将出现被截断现象。
- 不支持无日志表的DML解码。
- 删除复制槽需要在删除数据库之前执行，当逻辑复制槽所在数据库被删除后，这些复制槽变为不可用状态。
- 当同一事务产生大量需要落盘的子事务时，同时打开的文件句柄可能会超限，需将GUC参数max_files_per_process配置成大于子事务数量上限的两倍。
- 不支持全局二级索引，不支持分布列修改的DML解码。
- 容灾集群的备集群不支持通过SQL系统函数或工具进行逻辑解码。
- 不支持账本数据库功能，当前版本如果开启解码任务的数据库中有关于账本数据库的DML操作，则解码结果中会包含hash列，从而导致回放失败。
- 扩容场景下，若集群中创建有逻辑复制槽，会导致扩容失败，因此扩容前需要删除当前集群中已存在的逻辑复制槽。

SQL 函数解码性能

在Benchmarksql-5.0的100warehouse场景下，采用pg_logical_slot_get_changes时：

- 单次解码数据量4K行（对应约5MB～10MB日志），解码性能0.3MB/s～0.5MB/s。
- 单次解码数据量32K行（对应约40MB～80MB日志），解码性能3MB/s～5MB/s。
- 单次解码数据量256K行（对应约320MB～640MB日志），解码性能3MB/s～5MB/s。
- 单次解码数据量再增大，解码性能无明显提升。

如果采用pg_logical_slot_peek_changes + pg_replication_slot_advance方式，解码性能相比采用pg_logical_slot_get_changes时要下降30%～50%。

1.6.1.2 逻辑解码选项

逻辑解码选项可以用来为本次逻辑解码提供限制或额外功能，如“解码结果是否包含事务号”、“解码时是否忽略空事务”等。对于具体配置方法，SQL函数解码请参考《开发者指南》中“SQL参考 > 函数和操作符 > 系统管理函数 > 逻辑复制函数”章节中函数pg_logical_slot_peek_changes的可选入参''options_name'和'options_value'，JDBC流式解码请参考《开发者指南》中“应用程序开发教程 > 基于JDBC开发 > 示例：逻辑复制代码示例”章节示例代码中函数withSlotOption的使用方法。

通用选项（串行解码和并行解码均可配置，但可能无效，请参考相关选项详细说明）

- include-xids：

  解码出的data列是否包含xid信息。

  取值范围：boolean型，默认值为true。

版权所有 © 华为云计算技术有限公司

  - false：设为false时，解码出的data列不包含xid信息。
  - true：设为true时，解码出的data列包含xid信息。
- skip-empty-xacts：

  解码时是否忽略空事务信息。

  取值范围：boolean型，默认值为false。
  - false：设为false时，解码时不忽略空事务信息。
  - true：设为true时，解码时会忽略空事务信息。
- include-timestamp：

  解码信息是否包含commit时间戳。

  取值范围：boolean型，针对并行解码场景默认值为false，针对SQL函数解码和串行解码场景默认值为true。
  - false：设为false时，解码信息不包含commit时间戳。
  - true：设为true时，解码信息包含commit时间戳。
- only-local：

  是否仅解码本地日志。

  取值范围：boolean型，默认值为true。
  - false：设为false时，解码非本地日志和本地日志。
  - true：设为true时，仅解码本地日志。
- white-table-list：

  白名单参数，包含需要进行解码的schema和表名。

  取值范围：包含白名单中表名的字符串，不同的表以','为分隔符进行隔离；使用'*'来模糊匹配所有情况；schema名和表名间以'.'分割，不允许存在任意空白符。例如：

  ```
  select * from pg_logical_slot_peek_changes('slot1', NULL, 4096, 'white-table-list',
  'public.t1,public.t2,*.t3,my_schema.*');
  ```
- max-txn-in-memory：

  内存管控参数，单位为MB，单个事务占用内存大于该值即进行落盘。

  串行解码-取值范围：0~100的整型，默认值为0，即不开启此种管控。

  并行解码-取值范围：0~max_process_memory总量的25%，默认值为max_process_memory/4/1024，其中1024为kB到MB的单位转换，0表示不开启此条内存管控项。
- max-reorderbuffer-in-memory

  内存管控参数，单位为GB，拼接-发送线程中正在拼接的事务总内存（包含缓存）大于该值则对当前解码事务进行落盘。

  串行解码-取值范围：0~100的整型，默认值为0，即不开启此种管控。

  并行解码-取值范围：0~max_process_memory总量的50%，默认值为max_process_memory/2/1048576，其中1048576为kB到GB的单位转换，0表示不开启此条内存管控项。
- include-user：

  事务的BEGIN逻辑日志是否输出事务的用户名。事务的用户名特指授权用户——执行事务对应会话的登录用户，它在事务的整个执行过程中不会发生变化。

  取值范围：boolean型，默认值为false。
  - false：设为false时，事务的BEGIN逻辑日志不输出事务的用户名。

 版权所有 © 华为云计算技术有限公司

  - true：设为true时，事务的BEGIN逻辑日志输出事务的用户名。
- exclude-userids：

  黑名单用户的OID参数。

  取值范围：字符串类型，指定黑名单用户的OID，多个OID通过','分隔，不校验用户OID是否存在。
- exclude-users：

  黑名单用户的名称列表。

  取值范围：字符串类型，指定黑名单用户名，通过','分隔，不校验用户名是否存在。
- dynamic-resolution：

  是否动态解析黑名单用户名。

  取值范围：boolean型，默认值为true。
  - false：设为false时，当解码观测到黑名单exclude-users中用户不存在时将会报错并退出逻辑解码。
  - true：设为true时，当解码观测到黑名单exclude-users中用户不存在时继续解码。
- standby-connection：

  仅流式解码设置，是否仅限制备机解码。

  取值范围：boolean型，默认值为false。
  - true：设为true时，仅允许连接备机解码，连接主机解码时会报错退出。
  - false：设为false时，不做限制，允许连接主机或备机解码。

  说明

  如果主机资源使用率较大且业务对增量数据同步的实时性不敏感，建议进行备机解码；如果业务对增量数据同步的实时性要求高并且主机业务压力较小，建议使用主机解码。
- sender-timeout：

  仅流式解码设置，内核与客户端的心跳超时阈值。如果该时间段内没有收到客户端任何消息，逻辑解码将主动停止，并断开和客户端的连接。单位为毫秒（ms）。

  取值范围：0~2147483647的int型，默认值取决于GUC参数logical_sender_timeout的配置值。
- change-log-max-len:

  逻辑日志缓存长度上限参数，单位为字节。仅连接DN的并行解码有效，分布式强一致解码、串行解码及SQL函数解码无效。如果单条解码结果长度超过上限，则会销毁重新分配大小为1024字节的内存并缓存。过长会增加内存占用，过短会频繁触发内存申请和释放的操作，不建议设置成小于1024的值。

  取值范围：1~65535，默认值为4096。
- max-decode-to-sender-cache-num:

  并行解码日志的缓存条数阈值。仅连接DN的并行解码有效，分布式强一致解码、串行解码及SQL函数解码无效。缓存中的日志条数未超过这个阈值时，使用完毕的解码日志将置入缓存，否则直接释放。

  取值范围：1~65535，默认值为4096。
- enable-heartbeat：

  仅流式解码时设置，代表是否输出心跳日志。

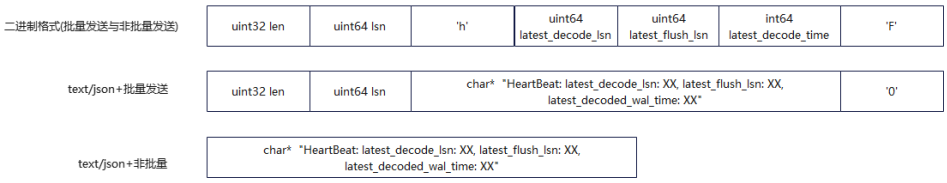
版权所有 © 华为云计算技术有限公司

取值范围：boolean型，默认值为false。

- true：设为true时，输出心跳日志。
- false：设为false时，不输出心跳日志。

说明

若开启心跳日志选项，此处说明并行解码场景心跳日志如何解析：二进制格式首先是字符'h'表示消息是心跳日志，之后是心跳日志内容，分别是8字节uint64，直连DN解码场景代表LSN，表示发送心跳逻辑日志时读取的WAL日志结束位置，而在分布式强一致解码场景为CSN，表示发送心跳逻辑日志时已发送的解码日志事务CSN；8字节uint64，直连DN解码场景代表LSN，表示发送心跳逻辑日志时刻已经落盘的WAL日志的位置，而在分布式强一致解码场景为CSN，表示集群下一个提交事务将获得的CSN；8字节int64代表时间戳（从1970年1月1日开始），表示最新解码到的事务日志或检查点日志的产生时间戳。关于消息结束符：如果是二进制格式则为字符'F'，如果格式为text或者json且为批量发送则结束符为0，否则没有结束符。消息内容采用大端字节序进行数据传输。具体格式见下图（考虑到前向兼容性，相关部分仍保留着LSN的命名方式，实际含义依具体场景而定）：



- parallel-decode-num：

  仅流式解码设置有效，并行解码的Decoder线程数量；系统函数调用场景下此选项无效，仅校验取值范围。

  取值范围：1~20的int型，取1表示按照原有的串行逻辑进行解码，取其余值即为开启并行解码，默认值为1。

须知

当parallel-decode-num不配置（即为默认值1）或显式配置为1时，下述“并行解码”中的选项不可配置。

- output-order：

  仅流式解码设置有效，代表是否使用CSN顺序输出解码结果；系统函数调用场景下此选项无效，仅校验取值范围。

  取值范围：0或1的int型，默认值为0。

  - 0：设为0时，解码结果按照事务的COMMIT LSN排序，当且仅当解码复制槽的confirmed_csn列值为0（即不显示）时可使用该方式，否则报错。
  - 1：设为1时，解码结果按照事务的CSN排序，当且仅当解码复制槽的confirmed_csn列值为非零时可使用该方式，否则报错。

须知

- 当output-order不配置（即为默认值0，按照COMMIT LSN排序）或显式配置为0时，下述“分布式强一致解码”中的选项不可配置。
- 在流式解码场景，DN收到来自CN的逻辑解码连接时，output-order选项失效，默认采用CSN序解码。

版权所有 © 华为云计算技术有限公司

- auto-advance：

  仅流式解码设置有效，代表是否允许自主推进逻辑复制槽。

  取值范围：boolean型，默认值为false。

  - true：设为true时，在已发送日志都被确认推进且没有待发送事务时，推进逻辑复制槽到当前解码位置。
  - false：设为false时，完全交由复制业务调用日志确认接口推进逻辑复制槽。

- skip-generated-columns：

  逻辑解码控制参数，用于跳过生成列的输出。对UPDATE和DELETE的旧元组无效，相应元组始终会输出生成列。分布式版本暂不支持生成列，此配置选项暂无实际影响。

  取值范围：boolean型，默认值为false。

  - true：值为true时，不输出生成列的解码结果。
  - false：设为false时，输出生成列的解码结果。

分布式强一致解码

- logical-receiver-num：

  仅流式解码设置有效，分布式解码启动的logical_receiver数量，系统函数调用场景下此选项无效，仅校验取值范围。

  取值范围：1~20的int型，默认值为1。当该值被设置为比当前集群分片数更大时，将被修改为分片数。

- slice-id：

  仅连接DN解码时设置，指定当前DN所在的分片号，用于复制表解码。

  取值范围：0~8192的int型，默认值为-1，即不指定分片号，但在解码到复制表时会报错。

⚠ 注意

该配置选项在尝试连接DN使用CSN序逻辑复制槽（confirmed_csn为非0值的复制槽）进行解码时使用，用来表示自己的分片号（即第几个分片，第一个分片则输入0），如果不设置该参数（即使用默认值-1）在解码到复制表时将会报错。此参数用于使用连接CN的分布式解码时，CN从DN收集解码结果时使用。不建议在此场景下手动连接DN解码。

- start-position：

  仅连接DN设置，主要功能为过滤掉小于指定CSN对应的事务，以及针对指定的CSN对应的事务，过滤掉小于指定LSN的日志，且指定CSN对应事务的BEGIN日志一定被过滤掉。

  取值范围：字符串类型，可以解析为以'/'分割，左右两侧分别为代表CSN和LSN的两个uint64类型。

  ⚠ 注意

  该配置选项用于CN解码时，CN建立与DN的连接后发送解码请求时使用此配置选项过滤可能已经被接收过的日志。不建议在此场景下手动连接DN解码使用此参数。

版权所有 © 华为云计算技术有限公司

串行解码

- force-binary：

  是否以二进制格式输出解码结果，针对不同场景呈现不同行为。

  - 针对系统函数pg_logical_slot_get_binary_changes和pg_logical_slot_peek_binary_changes：

    取值范围：boolean型，默认值为false。此值无实际意义，均以二进制格式输出解码结果。
  - 针对系统函数pg_logical_slot_get_changes、pg_logical_slot_peek_changes和pg_logical_get_area_changes：

    取值范围：仅取false值的boolean型。以文本格式输出解码结果。
  - 针对流式解码：

    取值范围：boolean型，默认值为false。此值无实际意义，均以文本格式输出解码结果。

并行解码

以下配置选项仅限流式解码设置。

- decode-style：

  指定解码格式。

  取值范围：char型的字符'j'、't'或'b'，分别代表json格式、text格式及二进制格式。默认值为'b'即二进制格式解码。

  对于json格式和text格式解码，开启批量发送选项时的解码结果中，每条解码语句的前4字节组成的uint32代表该条语句总字节数（不包含该uint32类型占用的4字节，0代表本批次解码结束），8字节uint64代表相应lsn（begin对应first_lsn，commit对应end_lsn，其他场景对应该条语句的lsn）。

版权所有 © 华为云计算技术有限公司

说明

二进制格式编码规则如下所示：

1. 前4字节代表接下来到语句级别分隔符字母P（不含）或者该批次结束符F（不含）的解码结果的总字节数，该值如果为0代表本批次解码结束。
2. 接下来8字节uint64代表相应lsn（begin对应first_lsn，commit对应end_lsn，其他场景对应该条语句的lsn）。
3. 接下来1字节的字母有5种B/C/I/U/D，分别代表begin/commit/insert/update/delete。
4. 第3步字母为B时：
    1. 接下来的8字节uint64代表CSN。
    2. 接下来的8字节uint64代表first_lsn。
    3. 【该部分为可选项】接下来的1字节字母如果为T，则代表后面4字节uint32表示该事务commit时间戳长度，再后面等同于该长度的字符为时间戳字符串。
    4. 【该部分为可选项】接下来的1字节字母如果为N，则代表后面4字节uint32表示该事务用户名的长度，再后面等同于该长度的字符为事务的用户名字。
    5. 因为之后仍可能有解码语句，接下来会有1字节字母P或F作为语句间的分隔符，P代表本批次仍有解码的语句，F代表本批次解码完成。
5. 第3步字母为C时：
    1. 【该部分为可选项】接下来1字节字母如果为X，则代表后面的8字节uint64表示xid。
    2. 【该部分为可选项】接下来的1字节字母如果为T，则代表后面4字节uint32表示时间戳长度，再后面等同于该长度的字符为时间戳字符串。
    3. 因为批量发送日志时，一个COMMIT日志解码之后可能仍有其他事务的解码结果，接下来的1字节字母如果为P则表示该批次仍需解码，如果为F则表示该批次解码结束。
6. 第3步字母为I/U/D时：
    1. 接下来的2字节uint16代表schema名的长度。
    2. 按照上述长度读取schema名。
    3. 接下来的2字节uint16代表table名的长度。
    4. 按照上述长度读取table名。
    5. 【该部分为可选项】接下来1字节字母如果为N代表为新元组，如果为O代表为旧元组，这里先发送新元组。
        1. 接下来的2字节uint16代表该元组需要解码的列数，记为attrnum。
        2. 以下流程重复attrnum次。
            1. 接下来2字节uint16代表列名的长度。
            2. 按照上述长度读取列名。
            3. 接下来4字节uint32代表当前列类型的OID。
            4. 接下来4字节uint32代表当前列值（以字符串格式存储）的长度，如果为0xFFFFFFFF则表示NULL，如果为0则表示长度为0的字符串。
            5. 按照上述长度读取列值。
    6. 因为之后仍可能有解码语句，接下来的1字节字母如果为P则表示该批次仍需解码，如果为F则表示该批次解码结束。

- sending-batch：

  指定是否批量发送。

  取值范围：0或1的int型，默认值为0。

  - 0：设为0时，表示逐条发送解码结果。
  - 1：设为1时，表示解码结果累积到达1MB则批量发送解码结果。

  开启批量发送的场景中，当解码格式为'j'或't'时，在原来的每条解码语句之前会附加一个uint32类型，表示本条解码结果长度（长度不包含当前的uint32类型），以及一个uint64类型，表示当前解码结果对应的lsn。

版权所有 © 华为云计算技术有限公司

须知

在CSN序解码（即output-order设置为1）场景下，批量发送仅限于单个事务内（即如果一个事务有多条较小的语句会采用批量发送），即不会使用批量发送功能在同一批次里发送多个事务，且BEGIN和COMMIT语句不会批量发送。

- parallel-queue-size：
  指定并行逻辑解码线程间进行交互的队列长度。
  取值范围：2~1024的int型，且必须为2的整数幂，默认值为128。
  队列长度和解码过程的内存使用量正相关。

1.6.1.3 使用 SQL 函数接口进行逻辑解码

GaussDB可以通过调用SQL函数，进行创建、删除、推进逻辑复制槽，获取解码后的事务日志。

操作步骤

步骤1 以具有REPLICATION权限的用户登录GaussDB集群任一主DN。

步骤2 使用如下命令通过DN端口连接数据库。

```
gsql -U user1 -W password -d gaussdb -p 40000 -r
```

其中，user1为用户名，password为密码，gaussdb为需要连接的数据库名称，40000为数据库DN端口号，用户可根据实际情况替换。复制槽是建立在DN上的，因此需要通过DN端口连接数据库。

步骤3 创建名称为slot1的逻辑复制槽。

```
gaussdb=> SELECT * FROM pg_create_logical_replication_slot('slot1', 'mppdb_decoding');
slotname | xlog_position
----------+---------------
slot1    | 0/601C150
(1 row)
```

步骤4 在数据库中创建表t，并向表t中插入数据。

```
gaussdb=> CREATE TABLE t(a int PRIMARY KEY, b int);
gaussdb=> INSERT INTO t VALUES(3,3);
```

步骤5 读取复制槽slot1解码结果，解码条数为4096。

说明

逻辑解码选项请参见**逻辑解码选项**。

```
gaussdb=> SELECT * FROM pg_logical_slot_peek_changes('slot1', NULL, 4096);
location  |  xid  | data
-----------+-------
+-----------------------------------------------------------------------------------------------------------------------------------------------------------
--------------------
-----------------------------------------------
 0/601C188 | 1010023 | BEGIN 1010023
 0/601ED60 | 1010023 | COMMIT 1010023 (at 2023-09-14 16:03:51.394287+08) CSN 1010022
 0/601ED60 | 1010024 | BEGIN 1010024
 0/601ED60 | 1010024 | {"table_name":"public.t","op_type":"INSERT","columns_name":
["a","b"],"columns_type":["integer","integer"],"columns_val":["3","3"],"old_keys_name":[],"old_keys_type":
[],"old_keys_val":[]}
 0/601EED8 | 1010024 | COMMIT 1010024 (at 2023-09-14 16:03:57.239821+08) CSN 1010023
(5 rows)
```


版权所有 © 华为云计算技术有限公司

步骤6 删除逻辑复制槽slot1，删除业务表t。

```
gaussdb=> SELECT * FROM pg_drop_replication_slot('slot1');
 pg_drop_replication_slot
--------------------------

(1 row)

gaussdb=> DROP TABLE t;
DROP TABLE
```

----结束

1.6.1.4 使用流式解码实现数据逻辑复制

第三方复制工具通过流式逻辑解码从GaussDB抽取逻辑日志后到对端数据库回放。对于使用JDBC连接数据库的复制工具，具体代码请参考《开发者指南》中“应用程序开发教程 > 基于JDBC开发 > 示例：逻辑复制代码示例”章节。

1.7 分区表

本章节围绕分区表在大数据量场景下如何对保存的数据进行“查询优化”和“运维管理”出发，分六个章节对分区表使用进行系统性说明，包含语义、原理、约束限制等方面。

1.7.1 大容量数据库

1.7.1.1 大容量数据库背景介绍

随着处理数据量的日益增长和使用场景的多样化，数据库越来越多地面对容量大、数据多样化的场景。在过去数据库业界发展的20多年时间里，数据量从最初的MB、GB级数据量逐渐发展到现在的TB级数据量，在如此数据大规模、数据多样化的客观背景下，数据库管理系统（DBMS）在数据查询、数据管理方面提出了更高的要求，客观上要求数据库能够支持多种优化查找策略和管理运维方式。

在计算机科学经典的算法中，人们通常使用分治法（Divide and Conquer）解决场景和规模较大的问题。其基本思想就是把一个复杂的问题分成两个或更多的相同或相似的子问题，再把子问题分成更小的子问题直到最后子问题可以简单的直接求解，原问题的解可看成子问题的解的合并。对于大容量数据场景，数据库提供对数据进行“分治处理”的方式即分区，将逻辑数据库或其组成元素划分为不同的独立部分，每一个分区维护逻辑上存在相类似属性的数据，这样就把庞大的数据整体进行了切分，有利于数据的管理、查找和维护。

1.7.1.2 表分区技术

表分区技术（Table-Partitioning）通过将非常大的表或者索引从逻辑上切分为更小、更易管理的逻辑单元（分区），能够让对用户对表查询、变更等语句操作具备更小的影响范围，能够让用户通过分区键（Partition Key）快速定位到数据所在的分区，从而避免在数据库中对大表的全量扫描，能够在不同的分区上并发进行DDL、DML操作。从用户使用的角度来看，表分区技术主要有以下三个方面能力：

1. 提升大容量数据场景查询效率：由于表内数据按照分区键进行逻辑分区，查询结果可以通过访问分区的子集而不是整个表来实现。这种分区剪枝技术可以提供数量级的性能增益。

 版权所有 © 华为云计算技术有限公司

2. 降低运维与查询的并发操作影响：降低DML语句、DDL语句并发场景的相互影响，在对一些大数据量以时间维度进行分区的场景下会明显受益。例如，新数据分区进行入库、实时点查操作，老数据分区进行数据清洗、分区合并等运维性质操作。
3. 提供大容量场景下灵活的数据运维管理方式：由于分区表从物理上对不同分区的数据做了表文件层面的隔离，每个分区可以具有单独的物理属性，如启用或禁用压缩、物理存储设置和表空间。同时它支持数据管理操作，如数据加载、索引创建和重建，以及分区级别的备份和恢复，而不是对整个表进行操作，从而减少了操作时间。

1.7.1.3 数据分区查找优化

分区表对数据查找方面的帮助主要体现在对分区键进行谓词查询场景，例如一张以月份Month作为分区键的表，如图1-2所示。如果以普通表的方式设计表结构则需要访问表全量的数据（Full Table Scan），如果以日期为分区键重新设计该表，那么原有的全表扫描会被优化成为分区扫描。当表内的数据量很大同时具有很长的历史周期时，由于扫描数据缩减所带来的性能提升会有明显的效果，如图1-3所示。

图 1-2 分区表示例图

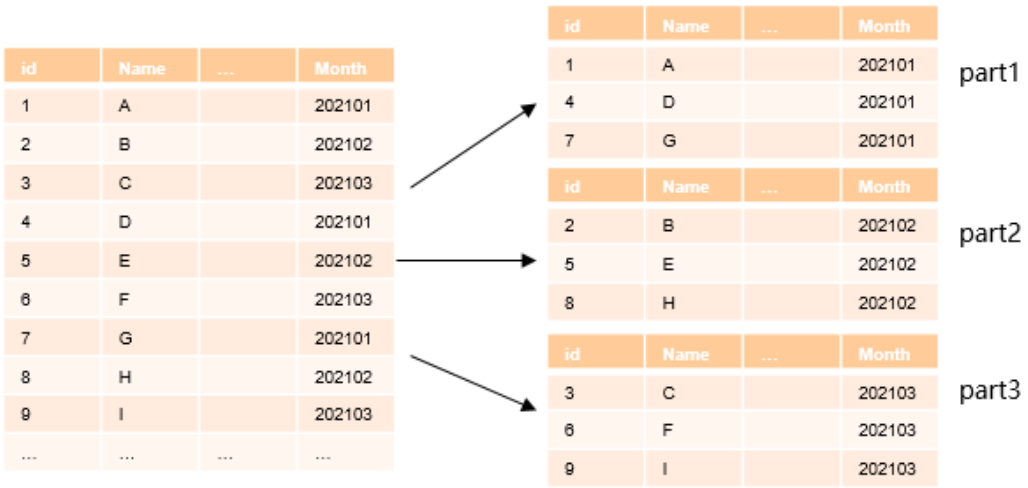
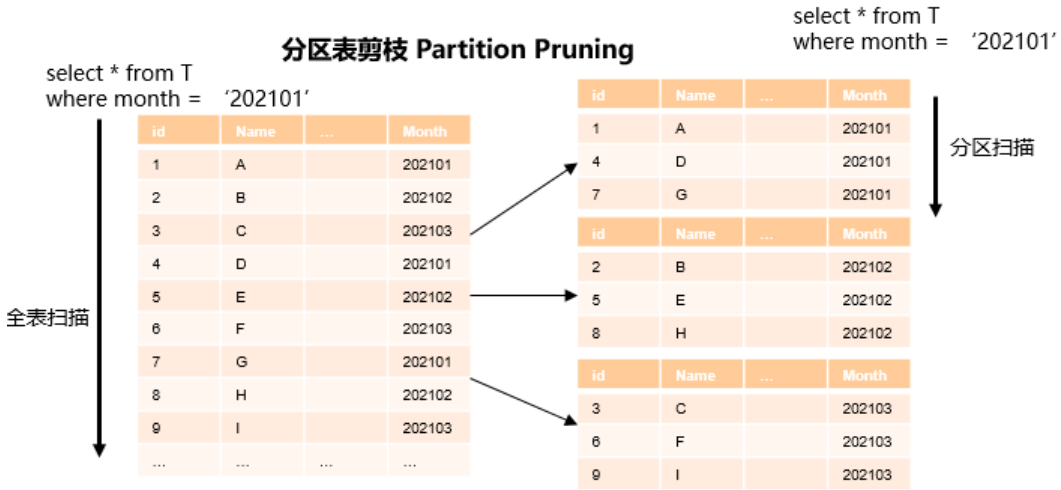


图 1-3 分区表剪枝示例图



版权所有 © 华为云计算技术有限公司

1.7.1.4 数据分区运维管理

分区表技术为数据生命周期管理（Data Life Cycle Management，DLM）提供了灵活性的支持，数据生命周期管理是一组用于在数据的整个使用寿命中管理数据的过程和策略。其中一个重要组成部分是确定在数据生命周期的任何时间点存储数据的最合适和最经济高效的介质：日常操作中使用的较新数据存储在最快、可用性最高的存储层上，而不经常访问的较旧数据可能存储在成本较低、效率较低的存储层。较旧的数据也可能更新的频率较低，因此将数据压缩并存储为只读是有意义的。

分区表为实施DLM解决方案提供了理想的环境，通过不同分区使用不同表空间，最大限度在确保易用性的同时，实现了有效的数据生命周期的成本优化。这部分的设置由数据库运维人员在服务端设置操作完成，实际用户并不感知这一层面的优化设置，对用户而言逻辑上仍然是对同一张表的查询操作。此外不同分区可以分别实施备份、恢复、索引重建等运维性质的操作，能够对单个数据集不同子类进行分治操作，满足用户业务场景的差异化需求。

1.7.2 分区表介绍

分区表（Partitioned Table）指在单节点内对表数据内容按照分区键以及围绕分区键的分区策略对表进行逻辑切分。从数据分区的角度来看是一种水平分区（horizontal partition）策略方式。分区表增强了数据库应用程序的性能、可管理性和可用性，并有助于降低存储大量数据的总体拥有成本。分区允许将表、索引和索引组织的表细分为更小的部分，使这些数据库对象能够在更精细的粒度级别上进行管理和访问。GaussDB提供了丰富的分区策略和扩展，以满足不同业务场景的需求。由于分区策略的实现完全由数据库内部实现，对用户是完全透明的，因此它几乎可以在实施分区表优化策略以后做平滑迁移，无需潜在耗费人力物力的应用程序更改。本章围绕GaussDB分区表的基本概念从以下几个方面展开介绍：

1. 分区表基本概念：从表分区的基本概念出发，介绍分区表的catalog存储方式以及内部对应原理。
2. 分区策略：从分区表所支持的基本类型出发，介绍各种分区模式下对应的特性以及能够达到的优化特点和效果。

1.7.2.1 基本概念

1.7.2.1.1 分区表（母表）

实际对用户体现的表，用户对该表进行常规DML语句的增、删、查、改操作。通常使用在建表DDL语句显式的使用PARTITION BY语句进行定义，创建成功以后在pg_class表中新增一个entry，并且parttype列内容为'p'，表明该entry为分区表的母表。分区母表通常是一个逻辑形态，对应的表文件并不存放数据。

示例：t1_hash为一个分区表，分区类型为hash：

```
gaussdb=# CREATE TABLE t1_hash (c1 INT, c2 INT, c3 INT)
PARTITION BY HASH(c1)
(
   PARTITION p0,
   PARTITION p1,
   PARTITION p2,
   PARTITION p3,
   PARTITION p4,
   PARTITION p5,
   PARTITION p6,
   PARTITION p7,
   PARTITION p8,
   PARTITION p9
);
```

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# \d+ t1_hash
                    Table "public.t1_hash"
Column |  Type   | Modifiers | Storage | Stats target | Description
--------+---------+-----------+---------+--------------+-------------
c1     | integer |           | plain   |              |
c2     | integer |           | plain   |              |
c3     | integer |           | plain   |              |
Partition By HASH(c1)
Number of partitions: 10 (View pg_partition to check each partition range.)
Distribute By: HASH(c1)
Location Nodes: ALL DATANODES
Has OIDs: no
Options: orientation=row, compression=no

--查询t1_hash分区类型。
gaussdb=#  SELECT relname, parttype FROM pg_class WHERE relname = 't1_hash';
relname | parttype
---------+----------
t1_hash | p
(1 row)

--删除t1_hash。
gaussdb=# DROP TABLE t1_hash;
```

1.7.2.1.2 分区（分区子表、子分区）

分区表中实际保存数据的表，对应的entry通常保存在pg_partition中，各个子分区的parentid作为外键关联其分区母表在pg_class表中的OID列。

示例：t1_hash为一个分区表：

```
gaussdb=# CREATE TABLE t1_hash (c1 INT, c2 INT, c3 INT)
PARTITION BY HASH(c1)
(
   PARTITION p0,
   PARTITION p1,
   PARTITION p2,
   PARTITION p3,
   PARTITION p4,
   PARTITION p5,
   PARTITION p6,
   PARTITION p7,
   PARTITION p8,
   PARTITION p9
);

--查询t1_hash分区类型。
gaussdb=# SELECT oid, relname, parttype FROM pg_class WHERE relname = 't1_hash';
oid  | relname | parttype
-------+---------+----------
16685 | t1_hash | p
(1 row)

--查询t1_hash的分区信息。
gaussdb=# SELECT oid, relname, parttype, parentid FROM pg_partition WHERE parentid = 16685;
oid  | relname | parttype | parentid
-------+---------+----------+----------
16688 | t1_hash | r        |    16685
16689 | p0      | p        |    16685
16690 | p1      | p        |    16685
16691 | p2      | p        |    16685
16692 | p3      | p        |    16685
16693 | p4      | p        |    16685
16694 | p5      | p        |    16685
16695 | p6      | p        |    16685
16696 | p7      | p        |    16685
16697 | p8      | p        |    16685
16698 | p9      | p        |    16685
```

 版权所有 © 华为云计算技术有限公司

```
(11 rows)

--删除t1_hash
gaussdb=# DROP TABLE t1_hash;
```

1.7.2.1.3 分区键

分区键由一个或多个列组成，分区键值结合对应分区方法能够唯一确定某一元组所在的分区，通常在建表时通过PARTITION BY语句指定：

```
CREATE TABLE table_name (…) PARTITION BY part_strategy (partition_key) (…)
```

须知

范围分区表和列表分区表支持最多16列分区键，其他分区表只支持1列分区键。

1.7.2.2 分区策略

分区策略在使用DDL语句建表语句时通过PARTITION BY语句的语法指定，分区策略描述了在分区表中数据和分区路由映射规则。常见的分区类型有基于条件的Range分区、基于哈希散列函数的Hash分区、基于数据枚举的List列表分区：

```
CREATE TABLE table_name (…) PARTITION BY partition_strategy (partition_key) (…)
```

1.7.2.2.1 范围分区

范围分区（Range Partition）根据为每个分区建立分区键的值范围将数据映射到分区。范围分区是生产系统中最常见的分区类型，通常在以时间维度（Date、Time Stamp）描述数据场景中使用。范围分区有两种语法格式，示例如下：

1. VALUES LESS THAN的语法格式

   对于从句是VALUE LESS THAN的语法格式，范围分区策略的分区键最多支持16列。

   - 单列分区键示例如下：

     ```
     gaussdb=# CREATE TABLE range_sales
     (
         product_id      INT4 NOT NULL,
         customer_id     INT4 NOT NULL,
         time            DATE,
         channel_id      CHAR(1),
         type_id         INT4,
         quantity_sold   NUMERIC(3),
         amount_sold     NUMERIC(10,2)
     )
     PARTITION BY RANGE (time)
     (
         PARTITION date_202001 VALUES LESS THAN ('2020-02-01'),
         PARTITION date_202002 VALUES LESS THAN ('2020-03-01'),
         PARTITION date_202003 VALUES LESS THAN ('2020-04-01'),
         PARTITION date_202004 VALUES LESS THAN ('2020-05-01')
     );

     gaussdb=# DROP TABLE range_sales;
     ```

     其中date_202002表示2020年2月的分区，将包含分区键值从2020年2月1日到2020年2月29日的数据。

     每个分区都有一个VALUES LESS子句，用于指定分区的非包含上限。大于或等于该分区键的任何值都将添加到下一个分区。除第一个分区外，所有分区都具有由前一个分区的VALUES LESS子句指定的隐式下限。可以为最高分区

版权所有 © 华为云计算技术有限公司

定义MAXVALUE关键字，MAXVALUE表示一个虚拟无限值，其排序高于分区键的任何其他可能值，包括空值。

- 多列分区键示例如下：

```
gaussdb=# CREATE TABLE range_sales_with_multiple_keys
(
   c1      INT4 NOT NULL,
   c2      INT4 NOT NULL,
   c3      CHAR(1)
)
PARTITION BY RANGE (c1,c2)
(
   PARTITION p1 VALUES LESS THAN (10,10),
   PARTITION p2 VALUES LESS THAN (10,20),
   PARTITION p3 VALUES LESS THAN (20,10)
);
gaussdb=# INSERT INTO range_sales_with_multiple_keys VALUES(9,5,'a');
gaussdb=# INSERT INTO range_sales_with_multiple_keys VALUES(9,20,'a');
gaussdb=# INSERT INTO range_sales_with_multiple_keys VALUES(9,21,'a');
gaussdb=# INSERT INTO range_sales_with_multiple_keys VALUES(10,5,'a');
gaussdb=# INSERT INTO range_sales_with_multiple_keys VALUES(10,15,'a');
gaussdb=# INSERT INTO range_sales_with_multiple_keys VALUES(10,20,'a');
gaussdb=# INSERT INTO range_sales_with_multiple_keys VALUES(10,21,'a');
gaussdb=# INSERT INTO range_sales_with_multiple_keys VALUES(11,5,'a');
gaussdb=# INSERT INTO range_sales_with_multiple_keys VALUES(11,20,'a');
gaussdb=# INSERT INTO range_sales_with_multiple_keys VALUES(11,21,'a');

gaussdb=# SELECT * FROM range_sales_with_multiple_keys PARTITION (p1);
 c1 | c2 | c3
----+----+----
  9 |  5 | a
  9 | 20 | a
  9 | 21 | a
 10 |  5 | a
(4 rows)

gaussdb=# SELECT * FROM range_sales_with_multiple_keys PARTITION (p2);
 c1 | c2 | c3
----+----+----
 10 | 15 | a
(1 row)

gaussdb=# SELECT * FROM range_sales_with_multiple_keys PARTITION (p3);
 c1 | c2 | c3
----+----+----
 10 | 20 | a
 10 | 21 | a
 11 |  5 | a
 11 | 20 | a
 11 | 21 | a
(5 rows)

gaussdb=# DROP TABLE range_sales_with_multiple_keys;
```

说明

多列分区的分区规则如下：

1. 从第一列开始比较。
2. 如果插入的当前列小于分区当前列边界值，则直接插入。
3. 如果插入的当前列等于分区当前列的边界值，则比较插入值的下一列与分区下一列边界值的大小。
4. 如果插入的当前列大于分区当前列的边界值，则换下一个分区进行比较。

2. START END语法格式

   对于从句是START END语法格式，范围分区策略的分区键最多支持1列。

   示例如下：

版权所有 © 华为云计算技术有限公司

```
-- 创建表空间。
gaussdb=# CREATE TABLESPACE startend_tbs1 LOCATION '/home/omm/startend_tbs1';
gaussdb=# CREATE TABLESPACE startend_tbs2 LOCATION '/home/omm/startend_tbs2';
gaussdb=# CREATE TABLESPACE startend_tbs3 LOCATION '/home/omm/startend_tbs3';
gaussdb=# CREATE TABLESPACE startend_tbs4 LOCATION '/home/omm/startend_tbs4';
-- 创建临时schema。
gaussdb=# CREATE SCHEMA tpcds;
gaussdb=# SET CURRENT_SCHEMA TO tpcds;
-- 创建分区表，分区键是integer类型。
gaussdb=# CREATE TABLE tpcds.startend_pt (c1 INT, c2 INT)
TABLESPACE startend_tbs1
PARTITION BY RANGE (c2) (
    PARTITION p1 START(1) END(1000) EVERY(200) TABLESPACE startend_tbs2,
    PARTITION p2 END(2000),
    PARTITION p3 START(2000) END(2500) TABLESPACE startend_tbs3,
    PARTITION p4 START(2500),
    PARTITION p5 START(3000) END(5000) EVERY(1000) TABLESPACE startend_tbs4
)
ENABLE ROW MOVEMENT;

-- 查看分区表信息。
gaussdb=# SELECT relname, boundaries, spcname FROM pg_partition p JOIN pg_tablespace t ON
    p.reltablespace=t.oid and p.parentid='tpcds.startend_pt'::regclass ORDER BY 1;
relname | boundaries | spcname
-------------+------------+---------------
p1_0 | {1} | startend_tbs2
p1_1 | {201} | startend_tbs2
p1_2 | {401} | startend_tbs2
p1_3 | {601} | startend_tbs2
p1_4 | {801} | startend_tbs2
p1_5 | {1000} | startend_tbs2
p2 | {2000} | startend_tbs1
p3 | {2500} | startend_tbs3
p4 | {3000} | startend_tbs1
p5_1 | {4000} | startend_tbs4
p5_2 | {5000} | startend_tbs4
startend_pt | | startend_tbs1
(12 rows)
```

1.7.2.2.2 哈希分区

哈希分区（Hash Partition）基于对分区键使用哈希算法将数据映射到分区。使用的哈希算法为GaussDB内置哈希算法，在分区键取值范围不倾斜（no data skew）的场景下，哈希算法在分区之间均匀分布行，使分区大小大致相同。因此哈希分区是实现分区间均匀分布数据的理想方法。哈希分区也是范围分区的一种易于使用的替代方法，尤其是当要分区的数据不是历史数据或没有明显的分区键时，示例如下：

```
gaussdb=# CREATE TABLE bmsql_order_line (
    ol_w_id         INTEGER   NOT NULL,
    ol_d_id         INTEGER   NOT NULL,
    ol_o_id         INTEGER   NOT NULL,
    ol_number       INTEGER   NOT NULL,
    ol_i_id         INTEGER   NOT NULL,
    ol_delivery_d   TIMESTAMP,
    ol_amount       DECIMAL(6,2),
    ol_supply_w_id  INTEGER,
    ol_quantity     INTEGER,
    ol_dist_info    CHAR(24)
)
--预先定义100个分区。
PARTITION BY HASH(ol_d_id)
(
    PARTITION p0,
    PARTITION p1,
    PARTITION p2,
    ...
    PARTITION p99
);
```

 版权所有 © 华为云计算技术有限公司

上述例子中，bmsql_order_line表的ol_d_id进行了分区，ol_d_id列是一个identifier性质的属性列，本身并不带有时间或者某一个特定维度上的区分。使用哈希分区策略来对其进行分表处理则是一个较为理想的选择。相比其他分区类型，除了预先确保分区键没有过多数据倾斜（某一、某几个值重复度高），只需要指定分区键和分区数即可创建分区，同时还能够确保每个分区的数据均匀，提升了分区表的易用性。

1.7.2.2.3 列表分区

列表分区（List Partition）能够通过在每个分区的描述中为分区键指定离散值列表来显式控制行如何映射到分区。列表分区的优势在于可以以枚举分区值方式对数据进行分区，可以对无序和不相关的数据集进行分组和组织。对于未定义在列表中的分区键值，可以使用默认分区（DEFAULT）来进行数据的保存，这样所有未映射到任何其他分区的行都不会生成错误。示例如下：

```
gaussdb=# CREATE TABLE bmsql_order_line (
    ol_w_id         INTEGER   NOT NULL,
    ol_d_id         INTEGER   NOT NULL,
    ol_o_id         INTEGER   NOT NULL,
    ol_number       INTEGER   NOT NULL,
    ol_i_id         INTEGER   NOT NULL,
    ol_delivery_d   TIMESTAMP,
    ol_amount       DECIMAL(6,2),
    ol_supply_w_id  INTEGER,
    ol_quantity     INTEGER,
    ol_dist_info    CHAR(24)
)
PARTITION BY LIST(ol_d_id)
(
    PARTITION p0 VALUES (1,4,7),
    PARTITION p1 VALUES (2,5,8),
    PARTITION p2 VALUES (3,6,9),
    PARTITION p3 VALUES (DEFAULT)
);

--清理示例
gaussdb=# DROP TABLE bmsql_order_line;
```

上述例子和之前给出的哈希分区的例子类似，同样通过ol_d_id列进行分区，但是在List分区中直接通过对ol_d_id的可能取值范围进行限定，不在列表中的数据会进入p3分区（DEFAULT）。相比哈希分区，List列表分区对分区键的可控性更好，往往能够准确的将目标数据保存在预想的分区中，但是如果列表值较多在分区定义时变得麻烦，该情况下推荐使用Hash分区。List、Hash分区往往都是处理无序、不相关的数据集进行分组和组织。

⚠ 注意

列表分区的分区键最多支持16列。如果分区键定义为1列，子分区定义时List列表中的枚举值不允许为NULL值；如果分区键定义为多列，子分区定义时List列表中的枚举值允许有NULL值。

1.7.2.2.4 分区表对导入操作的性能影响

在GaussDB内核实现中，分区表数据插入的处理过程相比非分区表增加分区路由部分的开销，因从整体上分区表场景的数据插入开销主要看成：（1）heap-insert基表插入；（2）partition-routing分区路由两个部分。其中heap基表插入解决tuple入库对应heap表的问题并且该部分普通表和分区表共用，而分区路由部分解决分区路由即tuple元组插入到对应partRel的问题。

因此对数据插入优化的侧重点如下：

版权所有 © 华为云计算技术有限公司

1. 分区表基表Heap表插入：
   a. 算子底噪优化
   b. heap数据插入
   c. 索引插入build优化（带索引）
2. 分区表分区路由：
   a. 路由查找算法逻辑优化
   b. 路由底噪优化，包括分区表partRel句柄开启、新增的函数调用逻辑开销

说明

分区路由的性能主要通过大数据量的单条INSERT语句体现，UPDATE场景内部包含了查找对应要更新的元组进行DELETE操作然后再进行INSERT，因此不如单条INSERT语句场景直接。

不同分区类型的路由算法逻辑如表1-5所示：

表 1-5 路由算法逻辑

分区方式	路由算法复杂度	实现概述说明
范围分区（Range Partition）	O(logN)	基于二分binary-search实现
哈希分区（Hash-Partition）	O(1)	基于key-partOid哈希表实现
列表分区（List-Partition）	O(1)	基于key-partOid哈希表实现

注意

分区路由的主要处理逻辑根据导入数据元组的分区键计算其所在分区的过程，相比非分区表这部分为额外增加的开销，这部分开销在最终数据导入上的具体性能损失和服务器CPU处理能力、表宽度、磁盘/内存的实际容量相关，通常可以粗略认为：

- x86服务器场景下分区表相比普通表的导入性能会略低10%以内。
- ARM服务器场景下为20%，造成x86和ARM指向性能略微差异的主要原因是分区路由为in-memory计算强化场景，主流x86体系CPU在单核指令处理能力上略优于ARM。

1.7.2.3 分区基本使用

1.7.2.3.1 创建分区表

创建分区表

由于SQL语言功能强大和灵活多样性，SQL语法树通常比复杂，分区表同样如此，分区表的创建可以理解成在原有非分区表的基础上新增表分区属性，因此分区表的语法接口可以看成是对原有非分区表CREATE TABLE语句进行扩展PARTITION BY语句部分，同时指定分区相关的三个核元素：

版权所有 © 华为云计算技术有限公司

1. 分区类型（ partType ）： 描述分区表的分区策略，分别有RANGE/LIST/HASH。
2. 分区键（ partKey ）： 描述分区表的分区列，目前RANGE/LIST分区支持多列（ 不超过16列 ）分区键，HASH分区只支持单列分区。
3. 分区表达式（ partExpr ）： 描述分区表的具体分区表方式，即键值与分区的对应映射关系。

这三部分重要元素在建表语句的Partition By Clause子句中体现，*PARTITION BY partType* (*partKey*) (*partExpr[,partExpr]*…) 。示例如下：

```
CREATE TABLE [ IF NOT EXISTS ] partition_table_name
(
   [ /* 该部份继承于普通表的Create Table */
   { column_name data_type [ COLLATE collation ] [ column_constraint [ ... ] ]
   | table_constraint
   | LIKE source_table [ like_option [...] ] }[, ... ]
   ]
)
[ WITH ( {storage_parameter = value} [, ... ] ) ]
[ COMPRESS | NOCOMPRESS ]
[ TABLESPACE tablespace_name ]
/* 范围分区场景 */
PARTITION BY RANGE (partKey) (
   { partition_start_end_item [, ... ] | partition_less_then_item [, ... ] }
)
/* 列表分区场景 */
PARTITION BY LIST (partKey)
(
   PARTITION partition_name VALUES (list_values_clause) [ TABLESPACE tablespace_name [, ... ] ]
...
)
/* 哈希分区场景 */
PARTITION BY HASH (partKey) (
   PARTITION partition_name [ TABLESPACE tablespace_name [, ... ] ]
...
)
/* 开启/关闭分区表行迁移 */
[ { ENABLE | DISABLE } ROW MOVEMENT ];
```

规格约束：

1. Range/List分区最大支持16个分区键，Hash分区只支持1个分区键。
2. 除哈希分区外，分区键不能插入空值，否则DML语句会进行报错处理。唯一例外：Range分区表定义有MAXVALUE分区/List分区表定义有DEFAULT分区。
3. 分区数最大值为1048575个，可以满足大部分业务场景的诉求。但分区数增加会导致系统中文件数增加，影响系统的性能，一般对于单个表而言不建议分区数超过200。

修改分区属性

分区表和分区相关的部分属性可以使用类似非分区表的ALTER-TABLE命令进行分区属性修改，常用的分区属性修改语句包括：

1. 增加分区
2. 删除分区
3. 删除/清空分区数据
4. 切割分区
5. 合并分区
6. 移动分区
7. 交换分区

版权所有 © 华为云计算技术有限公司

8. 重命名分区

以上常用的分区属性变更语句基于对普通表ALTER TABLE语句进行扩展，在使用方式上大部分使用方式类似，分区表属性变更的基本语法框架示例如下：

```
/* 基本alter table语法 */
ALTER TABLE [ IF EXISTS ] { table_name [*] | ONLY table_name | ONLY ( table_name )}
action [, ... ];
```

分区表ALTER TABLE语句使用方法请参见**分区表运维管理**、《开发者指南》中“SQL参考 > SQL语法 > ALTER TABLE PARTITION”章节。

1.7.2.3.2 使用和管理分区表

分区表支持大部分非分区表的相关功能，具体可以参考《开发者指南》中常规表的各类操作语法相关资料。

除此之外，分区表还支持大量的分区级操作命令，包括分区级DQL/DML（如SELECT、INSERT、UPDATE、DELETE、UPSERT、MERGE INTO）、分区级DDL（如ADD、DROP、TRUNCATE、EXCHANGE、SPLIT、MERGE、MOVE、RENAME）、分区VACUUM/ANALYZE、分类分区索引等。相关命令使用方法请参见**分区表DQL/DML**、**分区索引**、**分区表运维管理**、以及《开发者指南》中各个语法命令对应的章节。

分区级操作命令一般通过指定分区名或者分区值的方式进行，比如语法命令可能是如下情形：

```
sql_action [ t_name ] { PARTITION | SUBPARTITION } { p_name | (p_name) };
sql_action [ t_name ] { PARTITION | SUBPARTITION } FOR (p_value);
```

通过指定分区名p_name或指定分区值p_value来定向操作某个特定分区，此时业务只会作用于对象分区，而不会影响其他任何分区。如果通过指定分区名p_name来执行业务，数据库会匹配p_name对应的分区，该分区不存在则业务抛出异常；如果通过指定分区值p_value来执行业务，数据库会匹配p_value值所属分区。

比如定义有如下的分区表：

```
gaussdb=# CREATE TABLE list_01
(
   id   INT,
   role VARCHAR(100),
   data VARCHAR(100)
)
PARTITION BY LIST (id)
(
   PARTITION p_list_1 VALUES(0,1,2,3,4),
   PARTITION p_list_2 VALUES(5,6,7,8,9),
   PARTITION p_list_3 VALUES(DEFAULT)
);

-- 清理示例
gaussdb=# DROP TABLE list_01;
```

指定分区业务中，PARTITION p_list_1与PARTITION FOR (4) 等价，为同一个分区；PARTITION p_list_3与PARTITION FOR (12) 等价，为同一个分区。

1.7.2.3.3 分区表 DQL/DML

由于分区的实现完全体现在数据库内核中，用户对分区表的DQL/DML与非分区表相比，在语法上没有任何区别。

出于分区表的易用性考虑，GaussDB支持指定分区的DQL/DML操作，指定分区可以通过PARTITION (partname)或者PARTITION FOR (partvalue)来进行。对于二级分区，可以通过SUBPARTITION(subpartname)或者SUBPARTITION FOR (subpartvalue)指

版权所有 © 华为云计算技术有限公司

定具体的二级分区。指定分区执行DQL/DML时，若插入的数据不属于目标分区，则业务报错；若查询的数据不属于目标分区，则跳过该数据的处理。

指定分区DQL/DML支持以下几类语法：

1. 查询（SELECT）
2. 插入（INSERT）
3. 更新（UPDATE）
4. 删除（DELETE）
5. 插入或更新（UPSERT）
6. 合并（MERGE INTO）

指定分区做DQL/DML的示例如下：

```
--创建分区表list_02。
gaussdb=# CREATE TABLE IF NOT EXISTS list_02
(
    id   INT,
    role VARCHAR(100),
    data VARCHAR(100)
)
PARTITION BY LIST (id)
(
    PARTITION p_list_2 VALUES(0,1,2,3,4,5,6,7,8,9),
    PARTITION p_list_3 VALUES(10,11,12,13,14,15,16,17,18,19),
    PARTITION p_list_4 VALUES( DEFAULT ),
    PARTITION p_list_5 VALUES(20,21,22,23,24,25,26,27,28,29),
    PARTITION p_list_6 VALUES(30,31,32,33,34,35,36,37,38,39),
    PARTITION p_list_7 VALUES(40,41,42,43,44,45,46,47,48,49)
) ENABLE ROW MOVEMENT;
--导入数据。
INSERT INTO list_02 VALUES(null, 'alice', 'alice data');
INSERT INTO list_02 VALUES(2, null, 'bob data');
INSERT INTO list_02 VALUES(null, null, 'peter data');

--对指定分区进行查询。
-- 查询分区表全部数据。
gaussdb=# SELECT * FROM list_02 ORDER BY data;
 id | role  |    data
----+-------+------------
    | alice | alice data
  2 |       | bob data
    |       | peter data
(3 rows)
--查询分区p_list_2数据。
gaussdb=# SELECT * FROM list_02 PARTITION (p_list_2) ORDER BY data;
 id | role |   data
----+------+----------
  2 |      | bob data
(1 row)
--查询(100)所对应的分区的数据，即分区p_list_4。
gaussdb=# SELECT * FROM list_02 PARTITION FOR (100) ORDER BY data;
 id | role  |    data
----+-------+------------
    | alice | alice data
    |       | peter data
(2 rows)

--对指定分区做IUD。
-- 删除分区p_list_5中的全部数据。
gaussdb=# DELETE FROM list_02 PARTITION (p_list_5);
--指定分区p_list_7插入数据，由于数据不符合该分区约束，插入报错。
gaussdb=# INSERT INTO list_02 PARTITION (p_list_7) VALUES(null, 'cherry', 'cherry data');
ERROR:  inserted partition key does not map to the table partition
--将分区值100所属分区，即分区p_list_4的数据进行更新。
```

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# UPDATE list_02 PARTITION FOR (100) SET data = '';

--UPSERT。
gaussdb=# INSERT INTO list_02 (id, role, data) VALUES (1, 'test', 'testdata') ON DUPLICATE KEY UPDATE
role = VALUES(role), data = VALUES(data);

--MERGE INTO。
gaussdb=# CREATE TABLE IF NOT EXISTS list_tmp
(
    id   INT,
    role VARCHAR(100),
    data VARCHAR(100)
)
PARTITION BY LIST (id)
(
    PARTITION p_list_2 VALUES(0,1,2,3,4,5,6,7,8,9),
    PARTITION p_list_3 VALUES(10,11,12,13,14,15,16,17,18,19),
    PARTITION p_list_4 VALUES( DEFAULT ),
    PARTITION p_list_5 VALUES(20,21,22,23,24,25,26,27,28,29),
    PARTITION p_list_6 VALUES(30,31,32,33,34,35,36,37,38,39),
    PARTITION p_list_7 VALUES(40,41,42,43,44,45,46,47,48,49)) ENABLE ROW MOVEMENT;

gaussdb=# MERGE INTO list_tmp target
USING list_02 source
ON (target.id = source.id)
WHEN MATCHED THEN
  UPDATE SET target.data = source.data,
             target.role = source.role
WHEN NOT MATCHED THEN
  INSERT (id, role, data)
  VALUES (source.id, source.role, source.data);

--删除表。
gaussdb=#
DROP TABLE list_02;
DROP TABLE list_tmp;
```

1.7.3 分区表查询优化

说明

本节示例对应explain_perf_mode参数值为normal。

1.7.3.1 分区剪枝

分区剪枝是GaussDB提供的一种分区表查询优化技术，数据库SQL引擎会根据查询条件，只扫描特定的部分分区。分区剪枝是自动触发的，当分区表查询条件符合剪枝场景时，会自动触发分区剪枝。根据剪枝阶段的不同，分区剪枝分为静态剪枝和动态剪枝，静态剪枝在优化器阶段进行，在生成计划之前，数据库已经知道需要访问的分区信息；动态剪枝在执行器阶段进行（执行开始/执行过程中），在生成计划时，数据库并不知道需要访问的分区信息，只是判断“可以进行分区剪枝”，具体的剪枝信息由执行器决定。

只有分区表页面扫描和Local索引扫描才会触发分区剪枝，Global索引没有分区的概念，不需要进行剪枝。

1.7.3.1.1 分区表静态剪枝

对于检索条件中分区键上带有常数的分区表查询语句，在优化器阶段将对indexscan、bitmap indexscan、indexonlyscan等算子中包含的检索条件作为剪枝条件，完成分区的筛选。算子包含的检索条件中需要至少包含一个分区键字段，对于含有多个分区键的分区表，包含任意分区键子集即可。

 版权所有 © 华为云计算技术有限公司

静态剪枝支持范围如下所示：

1. 支持分区类型：范围分区、哈希分区、列表分区。
2. 支持表达式类型：比较表达式（<，<=，=，>=，>）、逻辑表达式、数组表达式。

⚠ 注意

目前静态剪枝不支持子查询表达式。

为了支持分区表剪枝，在计划生成时会将分区键上的过滤条件强制转换为分区键类型，该操作与隐式类型转换规则存在差异，可能导致相同条件在分区键上转换报错，非分区键无报错的情况。

- 静态剪枝支持的典型场景具体示例如下：
  a. 比较表达式

```
--创建分区表。
gaussdb=# CREATE TABLE t1 (c1 int, c2 int)
PARTITION BY RANGE (c1)
(
    PARTITION p1 VALUES LESS THAN(10),
    PARTITION p2 VALUES LESS THAN(20),
    PARTITION p3 VALUES LESS THAN(MAXVALUE)
);
gaussdb=# SET max_datanode_for_plan = 1;

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 = 1;
                        QUERY PLAN
------------------------------------------------------------
 Data Node Scan
   Output: t1.c1, t1.c2
   Node/s: datanode1
   Remote query: SELECT c1, c2 FROM public.t1 WHERE c1 = 1

 Remote SQL: SELECT c1, c2 FROM public.t1 WHERE c1 = 1
 Datanode Name: datanode1
   Partition Iterator
     Output: c1, c2
     Iterations: 1
     ->  Partitioned Seq Scan on public.t1
           Output: c1, c2
           Filter: (t1.c1 = 1)
           Selected Partitions:  1

(15 rows)

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 < 1;
                        QUERY PLAN
------------------------------------------------------------
 Data Node Scan
   Output: t1.c1, t1.c2
   Node/s: All datanodes
   Remote query: SELECT c1, c2 FROM public.t1 WHERE c1 < 1

 Remote SQL: SELECT c1, c2 FROM public.t1 WHERE c1 < 1
 Datanode Name: datanode1
   Partition Iterator
     Output: c1, c2
     Iterations: 1
     ->  Partitioned Seq Scan on public.t1
           Output: c1, c2
           Filter: (t1.c1 < 1)
           Selected Partitions:  1
```


版权所有 © 华为云计算技术有限公司

```
(15 rows)

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 > 11;
                     QUERY PLAN
-------------------------------------------------------------
 Data Node Scan
   Output: t1.c1, t1.c2
   Node/s: All datanodes
   Remote query: SELECT c1, c2 FROM public.t1 WHERE c1 > 11

 Remote SQL: SELECT c1, c2 FROM public.t1 WHERE c1 > 11
 Datanode Name: datanode1
   Partition Iterator
     Output: c1, c2
     Iterations: 2
     ->  Partitioned Seq Scan on public.t1
           Output: c1, c2
           Filter: (t1.c1 > 11)
           Selected Partitions:  2..3

(15 rows)

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 is NULL;
                       QUERY PLAN
-----------------------------------------------------------------
 Data Node Scan
   Output: t1.c1, t1.c2
   Node/s: datanode1
   Remote query: SELECT c1, c2 FROM public.t1 WHERE c1 IS NULL

 Remote SQL: SELECT c1, c2 FROM public.t1 WHERE c1 IS NULL
 Datanode Name: datanode1
   Partition Iterator
     Output: c1, c2
     Iterations: 1
     ->  Partitioned Seq Scan on public.t1
           Output: c1, c2
           Filter: (t1.c1 IS NULL)
           Selected Partitions:  3

(15 rows)
```

b. 逻辑表达式

```
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 = 1 AND c2 = 2;
                          QUERY PLAN
------------------------------------------------------------------------
 Data Node Scan
   Output: t1.c1, t1.c2
   Node/s: datanode1
   Remote query: SELECT c1, c2 FROM public.t1 WHERE c1 = 1 AND c2 = 2

 Remote SQL: SELECT c1, c2 FROM public.t1 WHERE c1 = 1 AND c2 = 2
 Datanode Name: datanode1
   Partition Iterator
     Output: c1, c2
     Iterations: 1
     ->  Partitioned Seq Scan on public.t1
           Output: c1, c2
           Filter: ((t1.c1 = 1) AND (t1.c2 = 2))
           Selected Partitions:  1

(15 rows)

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 = 1 OR c1 = 2;
                          QUERY PLAN
------------------------------------------------------------------------
 Data Node Scan
   Output: t1.c1, t1.c2
   Node/s: All datanodes
   Remote query: SELECT c1, c2 FROM public.t1 WHERE c1 = 1 OR c1 = 2
```

 版权所有 © 华为云计算技术有限公司

```
 Remote SQL: SELECT c1, c2 FROM public.t1 WHERE c1 = 1 OR c1 = 2
 Datanode Name: datanode1
   Partition Iterator
     Output: c1, c2
     Iterations: 1
     ->  Partitioned Seq Scan on public.t1
           Output: c1, c2
           Filter: ((t1.c1 = 1) OR (t1.c1 = 2))
           Selected Partitions:  1

(15 rows)

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE NOT c1 = 1;
                        QUERY PLAN
-----------------------------------------------------------------
 Data Node Scan
   Output: t1.c1, t1.c2
   Node/s: All datanodes
   Remote query: SELECT c1, c2 FROM public.t1 WHERE NOT c1 = 1

 Remote SQL: SELECT c1, c2 FROM public.t1 WHERE NOT c1 = 1
 Datanode Name: datanode1
   Partition Iterator
     Output: c1, c2
     Iterations: 3
     ->  Partitioned Seq Scan on public.t1
           Output: c1, c2
           Filter: (t1.c1 <> 1)
           Selected Partitions:  1..3

(15 rows)
```

c. 数组表达式

```
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 IN (1, 2, 3);
                              QUERY PLAN
-----------------------------------------------------------------------------
 Data Node Scan
   Output: t1.c1, t1.c2
   Node/s: All datanodes
   Remote query: SELECT c1, c2 FROM public.t1 WHERE c1 = ANY (ARRAY[1, 2, 3])

 Remote SQL: SELECT c1, c2 FROM public.t1 WHERE c1 = ANY (ARRAY[1, 2, 3])
 Datanode Name: datanode1
   Partition Iterator
     Output: c1, c2
     Iterations: 1
     ->  Partitioned Seq Scan on public.t1
           Output: c1, c2
           Filter: (t1.c1 = ANY ('{1,2,3}'::integer[]))
           Selected Partitions:  1

(15 rows)

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 = ALL(ARRAY[1,
2, 3]);
                              QUERY PLAN
-----------------------------------------------------------------------------
 Data Node Scan
   Output: t1.c1, t1.c2
   Node/s: All datanodes
   Remote query: SELECT c1, c2 FROM public.t1 WHERE c1 = ALL (ARRAY[1, 2, 3])

 Remote SQL: SELECT c1, c2 FROM public.t1 WHERE c1 = ALL (ARRAY[1, 2, 3])
 Datanode Name: datanode1
   Partition Iterator
     Output: c1, c2
     Iterations: 0
     ->  Partitioned Seq Scan on public.t1
           Output: c1, c2
```

 版权所有 © 华为云计算技术有限公司

```
          Filter: (t1.c1 = ALL ('{1,2,3}'::integer[]))
          Selected Partitions:  NONE

(15 rows)

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 = ANY(ARRAY[1,
2, 3]);
                              QUERY PLAN
------------------------------------------------------------------------------
 Data Node Scan
   Output: t1.c1, t1.c2
   Node/s: All datanodes
   Remote query: SELECT c1, c2 FROM public.t1 WHERE c1 = ANY (ARRAY[1, 2, 3])

 Remote SQL: SELECT c1, c2 FROM public.t1 WHERE c1 = ANY (ARRAY[1, 2, 3])
 Datanode Name: datanode1
   Partition Iterator
     Output: c1, c2
     Iterations: 1
     ->  Partitioned Seq Scan on public.t1
           Output: c1, c2
           Filter: (t1.c1 = ANY ('{1,2,3}'::integer[]))
           Selected Partitions:  1

(15 rows)

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 =
SOME(ARRAY[1, 2, 3]);
                              QUERY PLAN
------------------------------------------------------------------------------
 Data Node Scan
   Output: t1.c1, t1.c2
   Node/s: All datanodes
   Remote query: SELECT c1, c2 FROM public.t1 WHERE c1 = ANY (ARRAY[1, 2, 3])

 Remote SQL: SELECT c1, c2 FROM public.t1 WHERE c1 = ANY (ARRAY[1, 2, 3])
 Datanode Name: datanode1
   Partition Iterator
     Output: c1, c2
     Iterations: 1
     ->  Partitioned Seq Scan on public.t1
           Output: c1, c2
           Filter: (t1.c1 = ANY ('{1,2,3}'::integer[]))
           Selected Partitions:  1

(15 rows)
```

- 静态剪枝不支持的典型场景具体示例如下：

  子查询表达式

```
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 = ALL(SELECT c2
FROM t1 WHERE c1 > 10);
                            QUERY PLAN
------------------------------------------------------------------------------
 Streaming (type: GATHER)
   Output: public.t1.c1, public.t1.c2
   Node/s: All datanodes
   ->  Partition Iterator
         Output: public.t1.c1, public.t1.c2
         Iterations: 3
         ->  Partitioned Seq Scan on public.t1
               Output: public.t1.c1, public.t1.c2
               Distribute Key: public.t1.c1
               Filter: (SubPlan 1)
               Selected Partitions:  1..3
               SubPlan 1
                 ->  Materialize
                       Output: public.t1.c2
                       ->  Streaming(type: BROADCAST)
                             Output: public.t1.c2
```

 版权所有 © 华为云计算技术有限公司

```
                        Spawn on: All datanodes
                        Consumer Nodes: All datanodes
                        ->  Partition Iterator
                             Output: public.t1.c2
                             Iterations: 2
                             ->  Partitioned Seq Scan on public.t1
                                  Output: public.t1.c2
                                  Distribute Key: public.t1.c1
                                  Filter: (public.t1.c1 > 10)
                                  Selected Partitions:  2..3
(26 rows)

--清理示例环境。
gaussdb=# DROP TABLE t1;
```

1.7.3.1.2 分区表动态剪枝

对于检索条件中存在带有变量的分区表查询语句，由于优化器阶段无法获取用户的绑定参数，因此优化器阶段仅能完成indexscan、bitmapindexscan、indexonlyscan等算子检索条件的解析，后续会在执行器阶段获得绑定参数后，完成分区筛选。算子包含的检索条件中需要至少包含一个分区键字段，对于含有多个分区键的分区表，包含任意分区键子集即可。目前分区表动态剪枝仅支持PBE（Prepare/Bind/Execute）场景和参数化路径场景。

PBE 动态剪枝

PBE动态剪枝支持范围如下所示：

1. 支持分区类型：范围分区、哈希分区、列表分区。
2. 支持表达式类型：比较表达式（<，<=，=，>=，>）、逻辑表达式、数组表达式。
3. 支持隐式类型转换和函数：对于类型可以相互转换的场景和immutable函数可以支持PBE动态剪枝

⚠ 注意

- PBE动态剪枝支持表达式、隐式转换、immutable函数和stable函数，不支持子查询表达式和volatile函数。对于stable函数，如to_timestamp等类型转换函数，可能会受GUC参数变化，影响剪枝结果。为了保持性能优化，此情况可以通过analyze表重新生成gplan解决。
- 由于PBE动态剪枝是基于generic plan的剪枝，所以判断语句是否能PBE动态剪枝时，需要设置参数plan_cache_mode = 'force_generic_plan'，排除custom plan的干扰。

- PBE动态剪枝支持的典型场景具体示例如下：

  a. 比较表达式。

```
gaussdb=#
--创建分区表
CREATE TABLE t1 (c1 int, c2 int)
PARTITION BY RANGE (c1)
(
    PARTITION p1 VALUES LESS THAN(10),
    PARTITION p2 VALUES LESS THAN(20),
    PARTITION p3 VALUES LESS THAN(MAXVALUE)
);

gaussdb=# PREPARE p1(int) AS SELECT * FROM t1 WHERE c1 = $1;
```


版权所有 © 华为云计算技术有限公司

```
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p1(1);
                        QUERY PLAN
-----------------------------------------------------------
 Data Node Scan
   Output: t1.c1, t1.c2
   Node/s: datanode1
   Node expr: $1
   Remote query: SELECT c1, c2 FROM public.t1 WHERE c1 = $1

 Remote SQL: SELECT c1, c2 FROM public.t1 WHERE c1 = $1
 Datanode Name: datanode1
   Partition Iterator
     Output: c1, c2
     Iterations: PART
     ->  Partitioned Seq Scan on public.t1
           Output: c1, c2
           Filter: (t1.c1 = $1)
           Selected Partitions:  1 (pbe-pruning)

(16 rows)
```

b. 逻辑表达式。

```
gaussdb=# PREPARE p2(INT, INT) AS SELECT * FROM t1 WHERE c1 = $1 AND c2 = $2;
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p2(1, 2);
                            QUERY PLAN
-------------------------------------------------------------------------
 Data Node Scan
   Output: t1.c1, t1.c2
   Node/s: datanode1
   Node expr: $1
   Remote query: SELECT c1, c2 FROM public.t1 WHERE c1 = $1 AND c2 = $2

 Remote SQL: SELECT c1, c2 FROM public.t1 WHERE c1 = $1 AND c2 = $2
 Datanode Name: datanode1
   Partition Iterator
     Output: c1, c2
     Iterations: PART
     ->  Partitioned Seq Scan on public.t1
           Output: c1, c2
           Filter: ((t1.c1 = $1) AND (t1.c2 = $2))
           Selected Partitions:  1 (pbe-pruning)

(16 rows)
```

c. 类型转换触发隐式转换。

```
gaussdb=# set plan_cache_mode = 'force_generic_plan';
gaussdb=# PREPARE p3(TEXT) AS SELECT * FROM t1 WHERE c1 = $1;
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p3('12');
                            QUERY PLAN
-------------------------------------------------------------------------
 Data Node Scan
   Output: t1.c1, t1.c2
   Node/s: datanode1
   Node expr: $1
   Remote query: SELECT c1, c2 FROM public.t1 WHERE c1 = $1::bigint

 Remote SQL: SELECT c1, c2 FROM public.t1 WHERE c1 = $1::bigint
 Datanode Name: datanode1
   Partition Iterator
     Output: c1, c2
     Iterations: PART
     ->  Partitioned Seq Scan on public.t1
           Output: c1, c2
           Filter: (t1.c1 = ($1)::bigint)
           Selected Partitions:  2 (pbe-pruning)

(16 rows)
```


版权所有 © 华为云计算技术有限公司

- PBE动态剪枝不支持的典型场景具体示例如下：

  a. 子查询表达式。

```
gaussdb=# PREPARE p4(INT) AS SELECT * FROM t1 WHERE c1 = ALL(SELECT c2 FROM t1
WHERE c1 > $1);
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p4(1);
                            QUERY PLAN
------------------------------------------------------------------------
 Streaming (type: GATHER)
   Output: public.t1.c1, public.t1.c2
   Node/s: All datanodes
   ->  Partition Iterator
         Output: public.t1.c1, public.t1.c2
         Iterations: 3
         ->  Partitioned Seq Scan on public.t1
               Output: public.t1.c1, public.t1.c2
               Distribute Key: public.t1.c1
               Filter: (SubPlan 1)
               Selected Partitions:  1..3
               SubPlan 1
                 ->  Materialize
                       Output: public.t1.c2
                       ->  Streaming(type: BROADCAST)
                             Output: public.t1.c2
                             Spawn on: All datanodes
                             Consumer Nodes: All datanodes
                             ->  Partition Iterator
                                   Output: public.t1.c2
                                   Iterations: 3
                                   ->  Partitioned Seq Scan on public.t1
                                         Output: public.t1.c2
                                         Distribute Key: public.t1.c1
                                         Filter: (public.t1.c1 > 1)
                                         Selected Partitions:  1..3
(26 rows)
```

  b. 类型转换无法直接触发隐式转换。

```
gaussdb=# PREPARE p5(name) AS SELECT * FROM t1 WHERE c1 = $1;
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p5('12');
                            QUERY PLAN
------------------------------------------------------------------------------
 Data Node Scan
   Output: t1.c1, t1.c2
   Node/s: All datanodes
   Remote query: SELECT c1, c2 FROM public.t1 WHERE c1::text = '12'::text

 Remote SQL: SELECT c1, c2 FROM public.t1 WHERE c1::text = '12'::text
 Datanode Name: datanode1
   Partition Iterator
     Output: c1, c2
     Iterations: 3
     ->  Partitioned Seq Scan on public.t1
           Output: c1, c2
           Filter: ((t1.c1)::text = '12'::text)
           Selected Partitions:  1..3

(15 rows)
```

  c. stable/volatile函数。

```
gaussdb=# create sequence seq;
gaussdb=# PREPARE p6(TEXT) AS SELECT * FROM t1 WHERE c1 = currval($1);--volatile函数不支
持剪枝。
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p6('seq');
                             QUERY PLAN
--------------------------------------------------------------------------------
 Data Node Scan
   Output: t1.c1, t1.c2
```

 版权所有 © 华为云计算技术有限公司

```
  Node/s: All datanodes
  Remote query: SELECT c1, c2 FROM ONLY public.t1 WHERE true
  Coordinator quals: ((t1.c1)::numeric = currval(('seq'::text)::regclass))

 Remote SQL: SELECT c1, c2 FROM ONLY public.t1 WHERE true
 Datanode Name: datanode1
  Partition Iterator
    Output: c1, c2
    Iterations: 3
    ->  Partitioned Seq Scan on public.t1
          Output: c1, c2
          Selected Partitions:  1..3

(15 rows)

--清理示例环境。
gaussdb=# DROP TABLE t1;
```

参数化路径动态剪枝

参数化路径动态剪枝支持范围如下所示：

1. 支持分区类型：范围分区、哈希分区、列表分区。
2. 支持算子类型：indexscan、indexonlyscan、bitmapscan。
3. 支持表达式类型：比较表达式（<，<=，=，>=，>）、逻辑表达式。

⚠ 注意

参数化路径动态剪枝不支持子查询表达式，不支持stable和volatile函数，不支持跨QueryBlock参数化路径，不支持BitmapOr、BitmapAnd算子。

- 参数化路径动态剪枝支持的典型场景具体示例如下：
  a. 比较表达式

```
--创建分区表和索引
gaussdb=# CREATE TABLE t1 (c1 INT, c2 INT)
PARTITION BY RANGE (c1)
(
    PARTITION p1 VALUES LESS THAN(10),
    PARTITION p2 VALUES LESS THAN(20),
    PARTITION p3 VALUES LESS THAN(MAXVALUE)
);
gaussdb=# CREATE TABLE t2 (c1 INT, c2 INT)
PARTITION BY RANGE (c1)
(
    PARTITION p1 VALUES LESS THAN(10),
    PARTITION p2 VALUES LESS THAN(20),
    PARTITION p3 VALUES LESS THAN(MAXVALUE)
);
gaussdb=# CREATE INDEX t1_c1 ON t1(c1) LOCAL;
gaussdb=# CREATE INDEX t2_c1 ON t2(c1) LOCAL;
gaussdb=# CREATE INDEX t1_c2 ON t1(c2) LOCAL;
gaussdb=# CREATE INDEX t2_c2 ON t2(c2) LOCAL;

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT /*+ nestloop(t1 t2) indexscan(t1)
indexscan(t2) */ * FROM t2 JOIN t1 ON t1.c1 = t2.c1;
                                                  QUERY
PLAN
------------------------------------------------------------------------------------------------------------------------
-----------------------------------
 Data Node Scan
   Output: t2.c1, t2.c2, t1.c1, t1.c2
   Node/s: All datanodes
   Remote query: SELECT/*+ NestLoop(t1 t2) IndexScan(t1) IndexScan(t2)*/ t2.c1, t2.c2, t1.c1,
```


版权所有 © 华为云计算技术有限公司

```
t1.c2 FROM public.t2 JOIN public.t1 ON t1.c1 = t2.c1

 Remote SQL: SELECT/*+ NestLoop(t1 t2) IndexScan(t1) IndexScan(t2)*/ t2.c1, t2.c2, t1.c1, t1.c2
FROM public.t2 JOIN public.t1 ON t1.c1 = t2.c1
 Datanode Name: datanode1
  Nested Loop
    Output: t2.c1, t2.c2, t1.c1, t1.c2
    ->  Partition Iterator
          Output: t2.c1, t2.c2
          Iterations: 3
          ->  Partitioned Index Scan using t2_c1 on public.t2
                Output: t2.c1, t2.c2
                Selected Partitions:  1..3
    ->  Partition Iterator
          Output: t1.c1, t1.c2
          Iterations: PART
          ->  Partitioned Index Scan using t1_c1 on public.t1
                Output: t1.c1, t1.c2
                Index Cond: (t1.c1 = t2.c1)
                Selected Partitions:  1..3 (ppi-pruning)

(23 rows)

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT /*+ nestloop(t1 t2) indexscan(t1)
indexscan(t2) */ * FROM t2 JOIN t1 ON t1.c1 < t2.c1;
                          QUERY PLAN
------------------------------------------------------------------------
 Streaming (type: GATHER)
  Output: t2.c1, t2.c2, t1.c1, t1.c2
  Node/s: All datanodes
  ->  Nested Loop
        Output: t2.c1, t2.c2, t1.c1, t1.c2
        ->  Streaming(type: BROADCAST)
              Output: t2.c1, t2.c2
              Spawn on: All datanodes
              Consumer Nodes: All datanodes
              ->  Partition Iterator
                    Output: t2.c1, t2.c2
                    Iterations: 3
                    ->  Partitioned Seq Scan on public.t2
                          Output: t2.c1, t2.c2
                          Distribute Key: t2.c1
                          Selected Partitions:  1..3
        ->  Partition Iterator
              Output: t1.c1, t1.c2
              Iterations: PART
              ->  Partitioned Index Scan using t1_c1 on public.t1
                    Output: t1.c1, t1.c2
                    Distribute Key: t1.c1
                    Index Cond: (t1.c1 < t2.c1)
                    Selected Partitions:  1..3 (ppi-pruning)
(24 rows)

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT /*+ nestloop(t1 t2) indexscan(t1)
indexscan(t2) */ * FROM t2 JOIN t1 ON t1.c1 < t2.c1;
                          QUERY PLAN
------------------------------------------------------------------------
 Streaming (type: GATHER)
  Output: t2.c1, t2.c2, t1.c1, t1.c2
  Node/s: All datanodes
  ->  Nested Loop
        Output: t2.c1, t2.c2, t1.c1, t1.c2
        ->  Streaming(type: BROADCAST)
              Output: t2.c1, t2.c2
              Spawn on: All datanodes
              Consumer Nodes: All datanodes
              ->  Partition Iterator
                    Output: t2.c1, t2.c2
                    Iterations: 3
```


版权所有 © 华为云计算技术有限公司

```
                  ->  Partitioned Seq Scan on public.t2
                        Output: t2.c1, t2.c2
                        Distribute Key: t2.c1
                        Selected Partitions:  1..3
        ->  Partition Iterator
              Output: t1.c1, t1.c2
              Iterations: PART
              ->  Partitioned Index Scan using t1_c1 on public.t1
                    Output: t1.c1, t1.c2
                    Distribute Key: t1.c1
                    Index Cond: (t1.c1 > t2.c1)
                    Selected Partitions:  1..3 (ppi-pruning)
(24 rows)
```

b. 逻辑表达式

```
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT /*+ nestloop(t1 t2) indexscan(t1)
indexscan(t2) */ * FROM t2 JOIN t1 ON t1.c1 = t2.c1 AND t1.c2 = 2;
                                                
```

```
                  Iterations: 3
                  ->  Partitioned Seq Scan on public.t2
                        Output: t2.c1, t2.c2
                        Distribute Key: t2.c1
                        Selected Partitions:  1..3
        ->  Partition Iterator
              Output: t1.c1, t1.c2
              Iterations: 3
              ->  Partitioned Bitmap Heap Scan on public.t1
                    Output: t1.c1, t1.c2
                    Distribute Key: t1.c1
                    Recheck Cond: ((t1.c1 = t2.c1) OR (t1.c2 = 2))
                    Selected Partitions:  1..3
                    ->  BitmapOr
                          ->  Partitioned Bitmap Index Scan on t1_c1
                                Index Cond: (t1.c1 = t2.c1)
                          ->  Partitioned Bitmap Index Scan on t1_c2
                                Index Cond: (t1.c2 = 2)
(29 rows)
```

b. 隐式转换

```
gaussdb=# CREATE TABLE t3(c1 TEXT, c2 INT);
CREATE TABLE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 JOIN t3 ON t1.c1 = t3.c1;
WARNING:  Statistics in some tables or columns(public.t1.c1, public.t3.c1) are not collected.
HINT:  Do analyze for them in order to generate optimized plan.
                          QUERY PLAN
------------------------------------------------------------------------
 Streaming (type: GATHER)
   Output: t1.c1, t1.c2, t3.c1, t3.c2
   Node/s: All datanodes
   ->  Nested Loop
         Output: t1.c1, t1.c2, t3.c1, t3.c2
         Join Filter: (t1.c1 = (lengthb(t3.c1)))
         ->  Partition Iterator
               Output: t1.c1, t1.c2
               Iterations: 3
               ->  Partitioned Index Scan using t1_c1 on public.t1
                     Output: t1.c1, t1.c2
                     Distribute Key: t1.c1
                     Selected Partitions:  1..3
         ->  Materialize
               Output: t3.c1, t3.c2, (lengthb(t3.c1))
               ->  Streaming(type: REDISTRIBUTE)
                     Output: t3.c1, t3.c2, (lengthb(t3.c1))
                     Distribute Key: (lengthb(t3.c1))
                     Spawn on: All datanodes
                     Consumer Nodes: All datanodes
                     ->  Seq Scan on public.t3
                           Output: t3.c1, t3.c2, lengthb(t3.c1)
                           Distribute Key: t3.c1
(23 rows)
```

c. 函数

```
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 JOIN t3 ON t1.c1 =
LENGTHB(t3.c1);
                       QUERY PLAN
------------------------------------------------------------
 Nested Loop
   Output: t1.c1, t1.c2, t3.c1, t3.c2
   ->  Seq Scan on public.t3
         Output: t3.c1, t3.c2
   ->  Partition Iterator
         Output: t1.c1, t1.c2
         Iterations: 3
         ->  Partitioned Index Scan using t1_c1 on public.t1
               Output: t1.c1, t1.c2
               Index Cond: (t1.c1 = lengthb(t3.c1))
               Selected Partitions:  1..3
(11 rows)
```


版权所有 © 华为云计算技术有限公司

```
--清理示例环境。
gaussdb=# DROP TABLE t1;
gaussdb=# DROP TABLE t2;
gaussdb=# DROP TABLE t3;
```

1.7.3.2 分区索引

分区表上的索引共有三种类型：

1. Global Non-Partitioned Index
2. Global Partitioned Index
3. Local Partitioned Index

目前GaussDB支持Global Non-Partitioned Index和Local Partitioned Index类型索引。

图 1-4 Global Non-Partitioned Index

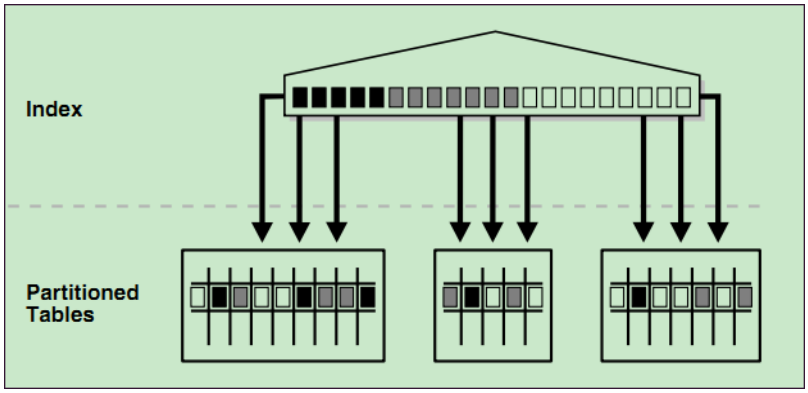
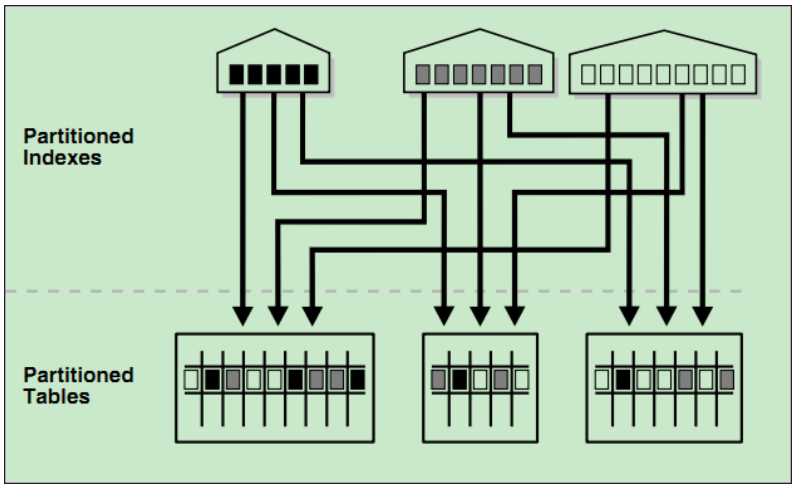
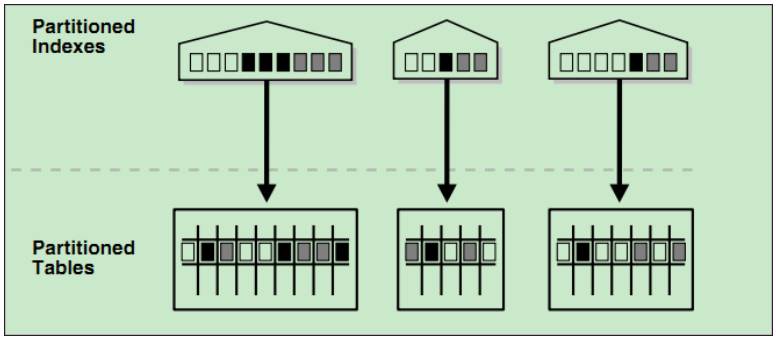


图 1-5 Global Partitioned Index



版权所有 © 华为云计算技术有限公司

图 1-6 Local Partitioned Index



约束

- 分区表索引分为LOCAL索引与GLOBAL索引：LOCAL索引与某个具体分区绑定，而GLOBAL索引则对应整个分区表。
- 唯一约束和主键约束的约束键包含所有分区键则创建LOCAL索引，否则创建GLOBAL索引。
- 在创建LOCAL索引时，可以通过FOR { partition_name | (partition_value [, ...]) }子句，指定在单个分区上创建LOCAL索引，此类索引在其他分区上不生效，后续新增的分区也不会自动创建该索引。需要注意的是，当前仅静态剪枝到单个分区的计划支持生成分类索引的查询路径。

说明

当查询语句在查询数据涉及多个分区时，建议使用GLOBAL索引，反之建议使用LOCAL索引。但需要注意GLOBAL索引在分区维护语法中存在额外的开销。

示例

- 创建表

```
gaussdb=# CREATE TABLE web_returns_p2
(
    ca_address_sk INTEGER NOT NULL ,
    ca_address_id CHARACTER(16) NOT NULL ,
    ca_street_number CHARACTER(10) ,
    ca_street_name CHARACTER VARYING(60) ,
    ca_street_type CHARACTER(15) ,
    ca_suite_number CHARACTER(10) ,
    ca_city CHARACTER VARYING(60) ,
    ca_county CHARACTER VARYING(30) ,
    ca_state CHARACTER(2) ,
    ca_zip CHARACTER(10) ,
    ca_country CHARACTER VARYING(20) ,
    ca_gmt_offset NUMERIC(5,2) ,
    ca_location_type CHARACTER(20)
)
PARTITION BY RANGE (ca_address_sk)
(
    PARTITION P1 VALUES LESS THAN(5000),
    PARTITION P2 VALUES LESS THAN(10000),
    PARTITION P3 VALUES LESS THAN(15000),
    PARTITION P4 VALUES LESS THAN(20000),
    PARTITION P5 VALUES LESS THAN(25000),
    PARTITION P6 VALUES LESS THAN(30000),
    PARTITION P7 VALUES LESS THAN(40000),
    PARTITION P8 VALUES LESS THAN(MAXVALUE)
)
ENABLE ROW MOVEMENT;
```


版权所有 © 华为云计算技术有限公司

- 创建索引
  - 创建分区表LOCAL索引tpcds_web_returns_p2_index1，不指定索引分区的名称。
    ```
    gaussdb=# CREATE INDEX tpcds_web_returns_p2_index1 ON web_returns_p2 (ca_address_id)
    LOCAL;
    ```
    当结果显示为如下信息，则表示创建成功。
    ```
    CREATE INDEX
    ```
  - 创建分区表LOCAL索引tpcds_web_returns_p2_index2，并指定索引分区的名称。
    ```
    gaussdb=# CREATE TABLESPACE example2 LOCATION '/home/omm/example2';
    gaussdb=# CREATE TABLESPACE example3 LOCATION '/home/omm/example3';
    gaussdb=# CREATE TABLESPACE example4 LOCATION '/home/omm/example4';

    gaussdb=# CREATE INDEX tpcds_web_returns_p2_index2 ON web_returns_p2 (ca_address_sk)
    LOCAL
    (
        PARTITION web_returns_p2_P1_index,
        PARTITION web_returns_p2_P2_index TABLESPACE example3,
        PARTITION web_returns_p2_P3_index TABLESPACE example4,
        PARTITION web_returns_p2_P4_index,
        PARTITION web_returns_p2_P5_index,
        PARTITION web_returns_p2_P6_index,
        PARTITION web_returns_p2_P7_index,
        PARTITION web_returns_p2_P8_index
    ) TABLESPACE example2;
    ```
    当结果显示为如下信息，则表示创建成功。
    ```
    CREATE INDEX
    ```
  - 创建分区表GLOBAL索引tpcds_web_returns_p2_global_index。
    ```
    gaussdb=# CREATE INDEX tpcds_web_returns_p2_global_index ON web_returns_p2
    (ca_street_number) GLOBAL;
    ```
    当结果显示为如下信息，则表示创建成功。
    ```
    CREATE INDEX
    ```
  - 创建分类分区索引

    指定分区名：
    ```
    gaussdb=# CREATE INDEX tpcds_web_returns_for_p1 ON web_returns_p2 (ca_address_id)
    LOCAL(partition ind_part for p1);
    ```
    指定分区键的值：
    ```
    gaussdb=# CREATE INDEX tpcds_web_returns_for_p2 ON web_returns_p2 (ca_address_id)
    LOCAL(partition ind_part for (5000));
    ```
    当结果显示为如下信息，则表示创建成功。
    ```
    CREATE INDEX
    ```
- 修改索引分区的表空间
  - 修改索引分区web_returns_p2_P2_index的表空间为example1。
    ```
    gaussdb=# ALTER INDEX tpcds_web_returns_p2_index2 MOVE PARTITION
    web_returns_p2_P2_index TABLESPACE example1;
    ```
    当结果显示为如下信息，则表示修改成功。
    ```
    ALTER INDEX
    ```
  - 修改索引分区web_returns_p2_P3_index的表空间为example2。
    ```
    gaussdb=# ALTER INDEX tpcds_web_returns_p2_index2 MOVE PARTITION
    web_returns_p2_P3_index TABLESPACE example2;
    ```
    当结果显示为如下信息，则表示修改成功。
    ```
    ALTER INDEX
    ```
- 重命名索引分区
  - 执行如下命令对索引分区web_returns_p2_P8_index重命名web_returns_p2_P8_index_new。

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# ALTER INDEX tpcds_web_returns_p2_index2 RENAME PARTITION
web_returns_p2_P8_index TO web_returns_p2_P8_index_new;
```

当结果显示为如下信息，则表示重命名成功。

```
ALTER INDEX
```

- 查询索引
  - 执行如下命令查询系统和用户定义的所有索引。

    ```
    gaussdb=# SELECT RELNAME FROM PG_CLASS WHERE RELKIND='i' or RELKIND='I';
    ```

  - 执行如下命令查询指定索引的信息。

    ```
    gaussdb=# \di+ tpcds_web_returns_p2_index2
    ```

- 删除索引

  ```
  gaussdb=# DROP INDEX tpcds_web_returns_p2_index1;
  ```

  当结果显示为如下信息，则表示删除成功。

  ```
  DROP INDEX
  ```

- 清理示例

  ```
  gaussdb=# DROP TABLE web_returns_p2;
  ```

1.7.3.3 分区表统计信息

对于分区表，支持收集分区级统计信息，相关统计信息可以在pg_partition和pg_statistic系统表以及pg_stats和pg_ext_stats视图中查询。分区级统计信息适用于分区表进行静态剪枝后，分区表的扫描范围剪枝到单分区的场景下。分区级统计信息的支持范围为：分区级的page数和tuple数、单列统计信息、多列统计信息、表达式索引统计信息。

分区表统计信息有以下收集方式：

- 级联收集统计信息
- 指定具体单个分区收集统计信息

1.7.3.3.1 级联收集统计信息

在ANALYZE | ANALYSE分区表时，系统会根据用户指定的或默认的PARTITION_MODE，自动收集分区表中所有符合语义的分区级统计信息，PARTITION_MODE的相关信息请参见《开发者指南》中“SQL参考 > SQL语法 > ANALYZE | ANALYSE”中的PARTITION_MODE参数。

⚠ 注意

- 当级联收集复制表、hashbucket表类型的分区表的统计信息，且PARTITION_MODE为ALL时，其行为将转换为ALL COMPLETE模式。
- 分区级统计信息级联收集不支持default_statistics_target为负数的场景。

示例

- 创建分区表并插入数据

  ```
  gaussdb=# CREATE TABLE t1_range_int
  (
     c1 INT,
     c2 INT,
     c3 INT,
     c4 INT
  )
  PARTITION BY RANGE(c1)
  ```


版权所有 © 华为云计算技术有限公司

```
(
    PARTITION range_p00 VALUES LESS THAN(10),
    PARTITION range_p01 VALUES LESS THAN(20),
    PARTITION range_p02 VALUES LESS THAN(30),
    PARTITION range_p03 VALUES LESS THAN(40),
    PARTITION range_p04 VALUES LESS THAN(50)
);
gaussdb=# INSERT INTO t1_range_int SELECT v,v,v,v FROM generate_series(0, 49) AS v;
```

- 级联收集统计信息

```
gaussdb=# ANALYZE t1_range_int WITH ALL;
```

- 查看分区级统计信息

```
gaussdb=# SELECT relname, parttype, relpages, reltuples FROM pg_partition WHERE
parentid=(SELECT oid FROM pg_class WHERE relname='t1_range_int') ORDER BY relname;
   relname    | parttype | relpages | reltuples
--------------+----------+----------+-----------
 range_p00    | p        |        4 |         9
 range_p01    | p        |        7 |        17
 range_p02    | p        |        6 |        13
 range_p03    | p        |        2 |         5
 range_p04    | p        |        4 |         9
 t1_range_int | r        |        0 |         0
(6 rows)

gaussdb=# SELECT
schemaname,tablename,partitionname,subpartitionname,attname,inherited,null_frac,avg_width,n_disti
nct,n_dndistinct,most_common_vals,most_common_freqs,histogram_bounds FROM pg_stats WHERE
tablename='t1_range_int' ORDER BY tablename, partitionname, attname;
 schemaname |   tablename    | partitionname | subpartitionname | attname | inherited | null_frac |
avg_width | n_distinct | n_dndistinct | most_common_vals | most_common_freqs
|                                                histogram_bounds

------------+--------------+---------------+------------------+---------+-----------+-----------+-----------
+------------+--------------+------------------+-------------------
+-----------------------------------------------------------------------------------------------
----------------------------------------------------
 public     | t1_range_int | range_p00     |                  | c1      | f         |         0 |         4 |        -1 |
-1 |              |                  | {0,1,2,3,4,5,6,7,8,9}
 public     | t1_range_int | range_p00     |                  | c2      | f         |         0 |         4 |        -1 |
-1 |              |                  | {0,1,2,3,4,5,6,7,8,9}
 public     | t1_range_int | range_p00     |                  | c3      | f         |         0 |         4 |        -1 |
-1 |              |                  | {0,1,2,3,4,5,6,7,8,9}
 public     | t1_range_int | range_p00     |                  | c4      | f         |         0 |         4 |        -1 |
-1 |              |                  | {0,1,2,3,4,5,6,7,8,9}
 public     | t1_range_int | range_p01     |                  | c1      | f         |         0 |         4 |        -1 |
-1 |              |                  | {10,11,12,13,14,15,16,17,18,19}
 public     | t1_range_int | range_p01     |                  | c2      | f         |         0 |         4 |        -1 |
-1 |              |                  | {10,11,12,13,14,15,16,17,18,19}
 public     | t1_range_int | range_p01     |                  | c3      | f         |         0 |         4 |        -1 |
-1 |              |                  | {10,11,12,13,14,15,16,17,18,19}
 public     | t1_range_int | range_p01     |                  | c4      | f         |         0 |         4 |        -1 |
-1 |              |                  | {10,11,12,13,14,15,16,17,18,19}
 public     | t1_range_int | range_p02     |                  | c1      | f         |         0 |         4 |        -1 |
-1 |              |                  | {20,21,22,23,24,25,26,27,28,29}
 public     | t1_range_int | range_p02     |                  | c2      | f         |         0 |         4 |        -1 |
-1 |              |                  | {20,21,22,23,24,25,26,27,28,29}
 public     | t1_range_int | range_p02     |                  | c3      | f         |         0 |         4 |        -1 |
-1 |              |                  | {20,21,22,23,24,25,26,27,28,29}
 public     | t1_range_int | range_p02     |                  | c4      | f         |         0 |         4 |        -1 |
-1 |              |                  | {20,21,22,23,24,25,26,27,28,29}
 public     | t1_range_int | range_p03     |                  | c1      | f         |         0 |         4 |        -1 |
-1 |              |                  | {30,31,32,33,34,35,36,37,38,39}
 public     | t1_range_int | range_p03     |                  | c2      | f         |         0 |         4 |        -1 |
-1 |              |                  | {30,31,32,33,34,35,36,37,38,39}
 public     | t1_range_int | range_p03     |                  | c3      | f         |         0 |         4 |        -1 |
-1 |              |                  | {30,31,32,33,34,35,36,37,38,39}
 public     | t1_range_int | range_p03     |                  | c4      | f         |         0 |         4 |        -1 |
-1 |              |                  | {30,31,32,33,34,35,36,37,38,39}
 public     | t1_range_int | range_p04     |                  | c1      | f         |         0 |         4 |        -1 |
```

 版权所有 © 华为云计算技术有限公司

```
-1 |               |                | {40,41,42,43,44,45,46,47,48,49}
 public    | t1_range_int | range_p04    |               | c2     | f        |       0 |       4 |       -1 |
-1 |               |                | {40,41,42,43,44,45,46,47,48,49}
 public    | t1_range_int | range_p04    |               | c3     | f        |       0 |       4 |       -1 |
-1 |               |                | {40,41,42,43,44,45,46,47,48,49}
 public    | t1_range_int | range_p04    |               | c4     | f        |       0 |       4 |       -1 |
-1 |               |                | {40,41,42,43,44,45,46,47,48,49}
 public    | t1_range_int |              |               | c1     | f        |       0 |       4 |       -1 |         -1
|               |              |
{0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,
34,35,36,37,38,39,40,41,42,43,44,45,46,47,48,49}
 public    | t1_range_int |              |               | c2     | f        |       0 |       4 |       -1 |         -1
|               |              |
{0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,
34,35,36,37,38,39,40,41,42,43,44,45,46,47,48,49}
 public    | t1_range_int |              |               | c3     | f        |       0 |       4 |       -1 |         -1
|               |              |
{0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,
34,35,36,37,38,39,40,41,42,43,44,45,46,47,48,49}
 public    | t1_range_int |              |               | c4     | f        |       0 |       4 |       -1 |         -1
|               |              |
{0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,
34,35,36,37,38,39,40,41,42,43,44,45,46,47,48,49}
(24 rows)
```

- 生成多列数据的分区级统计信息

```
gaussdb=# ALTER TABLE t1_range_int ADD STATISTICS ((c2, c3));
gaussdb=# ANALYZE t1_range_int WITH ALL;
```

- 查看多列数据的分区级统计信息

```
gaussdb=# SELECT
schemaname,tablename,partitionname,subpartitionname,attname,inherited,null_frac,avg_width,n_disti
nct,n_dndistinct,most_common_vals,most_common_freqs,histogram_bounds FROM pg_ext_stats
WHERE tablename='t1_range_int' ORDER BY tablename,partitionname,attname;
 schemaname |   tablename    | partitionname | subpartitionname | attname | inherited | null_frac |
avg_width | n_distinct | n_dndistinct | most_common_vals | most_common_freqs | histogram_bounds
------------+--------------+---------------+------------------+---------+-----------+-----------+-----------
+------------+--------------+------------------+------------------+------------------
 public    | t1_range_int | range_p00    |               | 2 3    | f        |       0 |       8 |       -1 |
-1 |               |                |
 public    | t1_range_int | range_p01    |               | 2 3    | f        |       0 |       8 |       -1 |
-1 |               |                |
 public    | t1_range_int | range_p02    |               | 2 3    | f        |       0 |       8 |       -1 |
-1 |               |                |
 public    | t1_range_int | range_p03    |               | 2 3    | f        |       0 |       8 |       -1 |
-1 |               |                |
 public    | t1_range_int | range_p04    |               | 2 3    | f        |       0 |       8 |       -1 |
-1 |               |                |
 public    | t1_range_int |              |               | 2 3    | f        |       0 |       8 |       -1 |         -1
|               |              |
(6 rows)
```

- 创建表达式索引并生成对应的分区级统计信息

```
gaussdb=# CREATE INDEX t1_range_int_index ON t1_range_int(text(c1)) LOCAL;
gaussdb=# ANALYZE t1_range_int WITH ALL;
```

- 查看表达式索引的分区级统计信息

```
gaussdb=# SELECT
schemaname,tablename,partitionname,subpartitionname,attname,inherited,null_frac,avg_width,n_disti
nct,n_dndistinct,most_common_vals,most_common_freqs,histogram_bounds FROM pg_stats WHERE
tablename='t1_range_int_index' ORDER BY tablename,partitionname,attname;
 schemaname |      tablename      |   partitionname    | subpartitionname | attname | inherited |
null_frac | avg_width | n_distinct | n_dndistinct | most_common_vals | most_common_freqs
|                                                  histogram_bounds

------------+---------------------+--------------------+------------------+---------+-----------+-----------
+-----------+------------+--------------+------------------+------------------
+-------------------------------------------------------------------------------
---------------------------------------------------------------
 public    | t1_range_int_index | range_p00_text_idx |               | text   | f        |       0 |       5
|        -1 |           0 |                |                  | {0,1,2,3,4,5,6,7,8,9}
```

 版权所有 © 华为云计算技术有限公司

```
 public     | t1_range_int_index | range_p01_text_idx |                  | text    | f         |        0 |        6
|         -1 |           0 |                  |                  | {10,11,12,13,14,15,16,17,18,19}
 public     | t1_range_int_index | range_p02_text_idx |                  | text    | f         |        0 |        6
|         -1 |           0 |                  |                  | {20,21,22,23,24,25,26,27,28,29}
 public     | t1_range_int_index | range_p03_text_idx |                  | text    | f         |        0 |        6
|         -1 |           0 |                  |                  | {30,31,32,33,34,35,36,37,38,39}
 public     | t1_range_int_index | range_p04_text_idx |                  | text    | f         |        0 |        6
|         -1 |           0 |                  |                  | {40,41,42,43,44,45,46,47,48,49}
 public     | t1_range_int_index |                  |                  | text    | f         |        0 |        5 |       -1
|           0 |                  |                  |
{0,1,10,11,12,13,14,15,16,17,18,19,2,20,21,22,23,24,25,26,27,28,29,3,30,31,32,33,3
4,35,36,37,38,39,4,40,41,42,43,44,45,46,47,48,49,5,6,7,8,9}
(6 rows)
```

- 删除分区表

```
gaussdb=# DROP TABLE t1_range_int;
```

1.7.3.3.2 分区级统计信息

指定单分区统计信息收集

当前分区表支持指定单分区统计信息收集，已收集统计信息的分区会在再次收集时自动更新维护。该功能适用于列表分区、哈希分区和范围分区。

```
gaussdb=# CREATE TABLE only_fisrt_part(id int,name varchar)PARTITION BY RANGE (id)
 (PARTITION id11 VALUES LESS THAN (1000000),
PARTITION id22 VALUES LESS THAN (2000000),
PARTITION max_id1 VALUES LESS THAN (MAXVALUE));

gaussdb=# INSERT INTO only_fisrt_part SELECT generate_series(1,5000),'test';


gaussdb=# ANALYZE only_fisrt_part PARTITION (id11);
gaussdb=# ANALYZE only_fisrt_part PARTITION (id22);
gaussdb=# ANALYZE only_fisrt_part PARTITION (max_id1);

gaussdb=# SELECT relname, relpages, reltuples FROM pg_partition WHERE relname IN ('id11', 'id22',
'max_id1');
 relname | relpages | reltuples
---------+----------+-----------
 id11    |     3400 |      5000
 id22    |        0 |         0
 max_id1 |        0 |         0
(3 rows)

gaussdb=# \x
gaussdb=# SELECT * FROM pg_stats WHERE tablename ='only_fisrt_part' AND partitionname ='id11';
-[ RECORD 1 ]----------
+-----------------------
```

```
subpartitionname       |
-[ RECORD 2 ]----------
+-----------------------
```

```
  ->  Partition Iterator  (cost=0.00..1.18 rows=1 width=12)
        Iterations: 1
        ->  Partitioned Seq Scan on only_first_part_two  (cost=0.00..1.18 rows=1 width=12)
              Filter: (c2 = 0)
              Selected Partitions:  1
(7 rows)

gaussdb=# DROP TABLE only_fisrt_part_two;
```

通过改写分区子句剪枝优化

当没有分区级统计信息时，在优化器行数估算模块，通过在逻辑上对分区子句进行伪谓词的改写，利用改写后的伪谓词影响选择率的计算和整表的统计信息获取一个比较准确的行数估算值。

说明

- 特性只作用于选择率的计算。
- 特性不支持二级分区。
- 特性只支持范围分区（range partition）、列表分区（list partition）。
- 对于范围分区，只支持单列分区键的改写，不支持多列分区键的改写。
- 对于列表分区，出于性能考虑，设置列表指定分区的枚举值个数的阈值为40个。
  - 当指定分区的列表枚举值个数超过40时，本特性不再适用。
  - 对于default分区，其列表枚举值个数是所有非default分区的枚举值个数的总和。

示例1：对于范围分区的改写

```
gaussdb=# CREATE TABLE test_int4_maxvalue(id INT, name VARCHAR)
PARTITION BY RANGE(id)
(
    PARTITION id1 VALUES LESS THAN(1000),
    PARTITION id2 VALUES LESS THAN(2000),
    PARTITION max_id VALUES LESS THAN(MAXVALUE)
);
gaussdb=# INSERT INTO test_int4_maxvalue SELECT GENERATE_SERIES(1,5000),'test';
gaussdb=# ANALYZE test_int4_maxvalue with global;

-- 查询指定分区id1
gaussdb=# EXPLAIN SELECT * FROM test_int4_maxvalue PARTITION(id1);
                                  QUERY PLAN
-------------------------------------------------------------------------------------------
 Data Node Scan  (cost=0.00..0.00 rows=0 width=0)
   Node/s: All datanodes

 Remote SQL: SELECT id, name FROM public.test_int4_maxvalue PARTITION (id1)
 Datanode Name: d1_datanode1
   Partition Iterator  (cost=0.00..7.91 rows=491 width=9)
     Iterations: 1
     ->  Partitioned Seq Scan on test_int4_maxvalue  (cost=0.00..7.91 rows=491 width=9)
           Selected Partitions:  1

 Datanode Name: d1_datanode2
   Partition Iterator  (cost=0.00..8.08 rows=508 width=9)
     Iterations: 1
     ->  Partitioned Seq Scan on test_int4_maxvalue  (cost=0.00..8.08 rows=508 width=9)
           Selected Partitions:  1

(16 rows)

-- 查询指定分区max_id
gaussdb=# EXPLAIN SELECT * FROM test_int4_maxvalue PARTITION(max_id);
                                  QUERY PLAN
-------------------------------------------------------------------------------------------
 Data Node Scan  (cost=0.00..0.00 rows=0 width=0)
```

 版权所有 © 华为云计算技术有限公司

```
  Node/s: All datanodes

 Remote SQL: SELECT id, name FROM public.test_int4_maxvalue PARTITION (max_id)
 Datanode Name: d1_datanode1
   Partition Iterator  (cost=0.00..24.46 rows=1546 width=9)
     Iterations: 1
     ->  Partitioned Seq Scan on test_int4_maxvalue  (cost=0.00..24.46 rows=1546 width=9)
           Selected Partitions:  3

 Datanode Name: d1_datanode2
   Partition Iterator  (cost=0.00..23.55 rows=1455 width=9)
     Iterations: 1
     ->  Partitioned Seq Scan on test_int4_maxvalue  (cost=0.00..23.55 rows=1455 width=9)
           Selected Partitions:  3

(16 rows)

-- drop partition table
gaussdb=# DROP TABLE test_int4_maxvalue;
```

示例2：对于列表分区的改写

```
gaussdb=# CREATE TABLE test_default
(
    c1 INT,
    c2 BIGINT
)
PARTITION BY LIST(c2)
(
    PARTITION p_1 VALUES (10000, 20000),
    PARTITION p_2 VALUES (300000, 400000, 500000),
    PARTITION p_3 VALUES (DEFAULT)
);
gaussdb=# INSERT INTO test_default SELECT GENERATE_SERIES(1, 1000), 10000;
gaussdb=# INSERT INTO test_default SELECT GENERATE_SERIES(1001, 2000), 600000;
gaussdb=# ANALYZE test_default with global;

-- 查询指定分区p_1
gaussdb=# EXPLAIN SELECT * FROM test_default PARTITION(p_1);
                                QUERY PLAN
-------------------------------------------------------------------------------
 Data Node Scan  (cost=0.00..0.00 rows=0 width=0)
   Node/s: All datanodes

 Remote SQL: SELECT c1, c2 FROM public.test_default PARTITION (p_1)
 Datanode Name: d1_datanode1
   Partition Iterator  (cost=0.00..7.92 rows=492 width=12)
     Iterations: 1
     ->  Partitioned Seq Scan on test_default  (cost=0.00..7.92 rows=492 width=12)
           Selected Partitions:  1

 Datanode Name: d1_datanode2
   Partition Iterator  (cost=0.00..8.08 rows=508 width=12)
     Iterations: 1
     ->  Partitioned Seq Scan on test_default  (cost=0.00..8.08 rows=508 width=12)
           Selected Partitions:  1

(16 rows)

-- 查询指定分区p_3
gaussdb=# EXPLAIN SELECT * FROM test_default PARTITION(p_3);
                                QUERY PLAN
-------------------------------------------------------------------------------
 Data Node Scan  (cost=0.00..0.00 rows=0 width=0)
   Node/s: All datanodes

 Remote SQL: SELECT c1, c2 FROM public.test_default PARTITION (p_3)
 Datanode Name: d1_datanode1
   Partition Iterator  (cost=0.00..8.24 rows=524 width=12)
     Iterations: 1
     ->  Partitioned Seq Scan on test_default  (cost=0.00..8.24 rows=524 width=12)
```

 版权所有 © 华为云计算技术有限公司

```
        Selected Partitions:  3

 Datanode Name: d1_datanode2
   Partition Iterator  (cost=0.00..7.76 rows=476 width=12)
     Iterations: 1
     ->  Partitioned Seq Scan on test_default  (cost=0.00..7.76 rows=476 width=12)
         Selected Partitions:  3

(16 rows)

-- drop partition table
gaussdb=# DROP TABLE test_default;
```

1.7.4 分区表运维管理

分区表运维管理包括分区管理、分区表管理、分区索引管理和分区表业务并发支持等。

- 分区管理：也称分区级DDL，包括新增（Add）、删除（Drop）、交换（Exchange）、清空（Truncate）、分割（Split）、合并（Merge）、移动（Move）、重命名（Rename）共8种。

⚠ 注意

- 对于哈希分区，涉及分区数的变更会导致数据re-shuffling，故当前GaussDB不支持导致Hash分区数变更的操作，包括新增（Add）、删除（Drop）、分割（Split）、合并（Merge）这4种。
- 涉及分区数据变更的操作会使得Global索引失效，可以通过UPDATE GLOBAL INDEX子句来同步更新Global索引，包括删除（Drop）、交换（Exchange）、清空（Truncate）、分割（Split）、合并（Merge）这5种。

说明

- 大部分分区DDL支持partition和partition for指定分区两种写法，前者需要指定分区名，后者需要指定分区定义范围内的任一分区值。比如假设分区part1的范围定义为[100, 200），那么partition part1和partition for(150)这两种写法是等价的。
- 不同分区DDL的执行代价各不相同，由于在执行分区DDL过程中目标分区会被锁住，用户需要评估其代价以及对业务的影响。一般而言，分割（Split）、合并（Merge）的执行代价远大于其他分区DDL，与源分区的大小正相关；交换（Exchange）的代价主要源于Global索引的重建和validation校验；移动（Move）的代价限制于磁盘I/O；其余分区DDL的执行代价都很低。

- 分区表管理：除了继承普通表的功能外，还支持开启/关闭分区表行迁移的功能。
- 分区索引管理：支持用户设置索引/索引分区不可用，或者重建不可用的索引/索引分区，比如由于分区管理操作导致的Global索引失效场景。
- 分区表业务并发支持：分布式分区表的DDL操作会锁全表，不支持跨分区DDL-DQL/DML并发。

1.7.4.1 新增分区

用户可以在已建立的分区表中新增分区，来维护新业务的进行。当前各种分区表支持的分区上限为1048575，如果达到了上限则不能继续添加分区。同时需要考虑分区占用内存的开销，分区表使用内存大致为（分区数 * 3 / 1024）MB，分区占用内存不允许大于local_syscache_threshold的值，同时还需要预留部分空间以供其他功能使用。

版权所有 © 华为云计算技术有限公司

⚠ 注意

- 新增分区不能作用于HASH分区上。
- 新增分区不继承表上的分类索引属性。

1.7.4.1.1 向范围分区表新增分区

使用ALTER TABLE ADD PARTITION可以将分区添加到现有分区表的最后面，新增分区的上界值必须大于当前最后一个分区的上界值。

例如，对范围分区表range_sales新增一个分区。

```
ALTER TABLE range_sales ADD PARTITION date_202005 VALUES LESS THAN ('2020-06-01') TABLESPACE tb1;
```

须知

当范围分区表有MAXVALUE分区时，无法新增分区。可以使用ALTER TABLE SPLIT PARTITION命令分割分区。分割分区同样适用于需要在现有分区表的前面/中间添加分区的情形，请参见**对范围分区表分割分区**。

1.7.4.1.2 向列表分区表新增分区

使用ALTER TABLE ADD PARTITION可以在列表分区表中新增分区，新增分区的枚举值不能与已有的任一个分区的枚举值重复。

例如，对列表分区表list_sales新增一个分区。

```
ALTER TABLE list_sales ADD PARTITION channel5 VALUES ('X') TABLESPACE tb1;
```

须知

当列表分区表有DEFAULT分区时，无法新增分区。可以使用ALTER TABLE SPLIT PARTITION命令分割分区。

1.7.4.2 删除分区

用户可以使用删除分区的命令来移除不需要的分区。删除分区可以通过指定分区名或者分区值来进行。

⚠ 注意

- 删除分区不能作用于HASH分区上。
- 执行删除分区命令会使得Global索引失效，可以通过UPDATE GLOBAL INDEX子句来同步更新Global索引，或者用户自行重建Global索引。
- 删除分区时，如果该分区上带有仅属于当前分区的分类索引时，则会级联删除分类索引。

使用ALTER TABLE DROP PARTITION可以删除指定分区表的任何一个分区，这个行为可以作用在范围分区表、列表分区表上。

版权所有 © 华为云计算技术有限公司

例如，通过指定分区名删除范围分区表range_sales的分区date_202005，并更新Global索引。

```
ALTER TABLE range_sales DROP PARTITION date_202005 UPDATE GLOBAL INDEX;
```

或者，通过指定分区值来删除范围分区表range_sales中'2020-05-08'所对应的分区。由于不带UPDATE GLOBAL INDEX子句，执行该命令后Global索引会失效。

```
ALTER TABLE range_sales DROP PARTITION FOR ('2020-05-08');
```

须知

- 当分区表只有一个分区时，不支持通过ALTER TABLE DROP PARTITION命令删除分区。
- 当分区表为哈希分区表时，不支持通过ALTER TABLE DROP PARTITION命令删除分区。

1.7.4.3 交换分区

用户可以使用交换分区的命令来将分区与普通表的数据进行交换。交换分区可以快速将数据导入/导出分区表，实现数据高效加载的目的。在业务迁移的场景，使用交换分区比常规导入会快很多。交换分区可以通过指定分区名或者分区值来进行。

须知

- 执行交换分区命令会使得Global索引失效，可以通过UPDATE GLOBAL INDEX子句来同步更新Global索引，或者用户自行重建Global索引。
- 执行交换分区时，可以申明WITH/WITHOUT VALIDATION，表明是否校验普通表数据满足目标分区的分区键约束规则（默认校验）。数据校验活动开销较大，如果能确保交换的数据属于目标分区，可以申明WITHOUT VALIDATION来提高交换性能。
- 可以申明WITH VALIDATION VERBOSE，此时数据库会校验普通表的每一行，将不满足目标分区的分区键约束规则的数据，插入到分区表的其他分区中，最后再进行普通表与目标分区的交换。

例如，给出如下分区定义和普通表exchange_sales的数据分布，并将分区DATE_202001和普通表exchange_sales做交换，则根据申明子句的不同，存在以下三种行为：

- 申明WITHOUT VALIDATION，数据全部交换到分区DATE_202001中，由于'2020-02-03', '2020-04-08'不满足分区DATE_202001的范围约束，后续业务可能会出现异常。
- 申明WITH VALIDATION，由于'2020-02-03', '2020-04-08'不满足分区DATE_202001的范围约束，数据库给出相应的报错。
- 申明WITH VALIDATION VERBOSE，数据库会将'2020-02-03'插入分区DATE_202002，将'2020-04-08'插入分区DATE_202004，再将剩下的数据交换到分区DATE_202001中。

```
--分区定义
PARTITION DATE_202001 VALUES LESS THAN ('2020-02-01'),
PARTITION DATE_202002 VALUES LESS THAN ('2020-03-01'),
PARTITION DATE_202003 VALUES LESS THAN ('2020-04-01'),
PARTITION DATE_202004 VALUES LESS THAN ('2020-05-01')
```


版权所有 © 华为云计算技术有限公司

```
-- exchange_sales的数据分布
('2020-01-15', '2020-01-17', '2020-01-23', '2020-02-03', '2020-04-08')
```

⚠ 警告

如果交换的数据不完全属于目标分区，请不要申明WITHOUT VALIDATION交换分区，否则会破坏分区约束规则，导致分区表后续DML业务结果异常。

进行交换的普通表和分区必须满足如下条件：

- 普通表和分区的列数目相同，对应列的信息严格一致。
- 普通表和分区的表压缩信息严格一致。
- 普通表索引和分区Local索引个数相同，且对应索引的信息严格一致。
- 普通表和分区的表约束个数相同，且对应表约束的信息严格一致。
- 普通表不可以是临时表。
- 普通表和分区表上不可以有动态数据脱敏，行访问控制约束。

使用ALTER TABLE EXCHANGE PARTITION可以对分区表交换分区。

例如，通过指定分区名将范围分区表range_sales的分区date_202001和普通表exchange_sales进行交换，不进行分区键校验，并更新Global索引。

```
ALTER TABLE range_sales EXCHANGE PARTITION (date_202001) WITH TABLE exchange_sales WITHOUT VALIDATION UPDATE GLOBAL INDEX;
```

或者，通过指定分区值将范围分区表range_sales中'2020-01-08'所对应的分区和普通表exchange_sales进行交换，进行分区校验并将不满足目标分区约束的数据插入到分区表的其他分区中。由于不带UPDATE GLOBAL INDEX子句，执行该命令后Global索引会失效。

```
ALTER TABLE range_sales EXCHANGE PARTITION FOR ('2020-01-08') WITH TABLE exchange_sales WITH VALIDATION VERBOSE;
```

1.7.4.4 清空分区

用户可以使用清空分区的命令来快速清空分区的数据。与删除分区功能类似，区别在于清空分区只会删除分区中的数据，分区的定义和物理文件都会保留。清空分区可以通过指定分区名或者分区值来进行。

⚠ 注意

执行清空分区命令会使得Global索引失效，可以通过UPDATE GLOBAL INDEX子句来同步更新Global索引，或者用户自行重建Global索引。

使用ALTER TABLE TRUNCATE PARTITION可以清空指定分区表的任何一个分区。

例如，通过指定分区名清空范围分区表range_sales的分区date_202005，并更新Global索引。

```
ALTER TABLE range_sales TRUNCATE PARTITION date_202005 UPDATE GLOBAL INDEX;
```

或者，通过指定分区值来清空范围分区表range_sales中'2020-05-08'所对应的分区。由于不带UPDATE GLOBAL INDEX子句，执行该命令后Global索引会失效。

```
ALTER TABLE range_sales TRUNCATE PARTITION FOR ('2020-05-08');
```


版权所有 © 华为云计算技术有限公司

1.7.4.5 分割分区

用户可以使用分割分区的命令来将一个分区分割为两个或多个新分区。当分区数据太大，或者需要对有MAXVALUE的范围分区/DEFAULT的列表分区新增分区时，可以考虑执行该操作。分割分区可以指定分割点将一个分区分割为两个新分区，也可以不指定分割点将一个分区分割为多个新分区。分割分区可以通过指定分区名或者分区值来进行。

⚠ 注意

- 分割分区不能作用于哈希分区上。
- 执行分割分区命令会使得Global索引失效，可以通过UPDATE GLOBAL INDEX子句来同步更新Global索引，或者用户自行重建Global索引。
- 分割的目标分区如果包含分类索引时，该分区不支持分割。
- 分割后的新分区，可以与源分区名字相同，比如将分区p1分割为p1,p2。但数据库不会将分割前后相同名的分区视为同一个分区。

1.7.4.5.1 对范围分区表分割分区

使用ALTER TABLE SPLIT PARTITION可以对范围分区表分割分区。

例如，假设范围分区表range_sales的分区date_202001定义范围为['2020-01-01', '2020-02-01')。可以指定分割点'2020-01-16'将分区date_202001分割为两个分区，并更新Global索引。

```
ALTER TABLE range_sales SPLIT PARTITION date_202001 AT ('2020-01-16') INTO
(
    PARTITION date_202001_p1, --第一个分区上界是'2020-01-16'
    PARTITION date_202001_p2  --第二个分区上界是'2020-02-01'
) UPDATE GLOBAL INDEX;
```

或者，不指定分割点，将分区date_202001分割为多个分区，并更新Global索引。

```
ALTER TABLE range_sales SPLIT PARTITION date_202001 INTO
(
    PARTITION date_202001_p1 VALUES LESS THAN ('2020-01-11'),
    PARTITION date_202001_p2 VALUES LESS THAN ('2020-01-21'),
    PARTITION date_202001_p3 --第三个分区上界是'2020-02-01'
)UPDATE GLOBAL INDEX;
```

又或者，通过指定分区值而不是指定分区名来分割分区。

```
ALTER TABLE range_sales SPLIT PARTITION FOR ('2020-01-15') AT ('2020-01-16') INTO
(
    PARTITION date_202001_p1, --第一个分区上界是'2020-01-16'
    PARTITION date_202001_p2  --第二个分区上界是'2020-02-01'
) UPDATE GLOBAL INDEX;
```

须知

若对MAXVALUE分区进行分割，前面几个分区不能申明MAXVALUE范围，最后一个分区会继承MAXVALUE分区范围。

1.7.4.5.2 对列表分区表分割分区

使用ALTER TABLE SPLIT PARTITION可以对列表分区表分割分区。

版权所有 © 华为云计算技术有限公司

例如，假设列表分区表list_sales的分区channel2定义范围为('6', '7', '8', '9')。可以指定分割点('6', '7') 将分区channel2分割为两个分区，并更新Global索引。

```
ALTER TABLE list_sales SPLIT PARTITION channel2 VALUES ('6', '7') INTO
(
   PARTITION channel2_1, --第一个分区范围是('6', '7' )
   PARTITION channel2_2  --第二个分区范围是('8', '9' )
) UPDATE GLOBAL INDEX;
```

或者，不指定分割点，将分区channel2分割为多个分区，并更新Global索引。

```
ALTER TABLE list_sales SPLIT PARTITION channel2 INTO
(
   PARTITION channel2_1 VALUES ('6'),
   PARTITION channel2_2 VALUES ('8'),
   PARTITION channel2_3 --第三个分区范围是('7', '9' )
)UPDATE GLOBAL INDEX;
```

又或者，通过指定分区值而不是指定分区名来分割分区。

```
ALTER TABLE list_sales SPLIT PARTITION FOR ('6') VALUES ('6', '7') INTO
(
   PARTITION channel2_1, --第一个分区范围是('6', '7' )
   PARTITION channel2_2  --第二个分区范围是('8', '9' )
) UPDATE GLOBAL INDEX;
```

⚠ 注意

若对DEFAULT分区进行分割，前面几个分区不能申明DEFAULT范围，最后一个分区会继承DEFAULT分区范围。

1.7.4.6 合并分区

用户可以使用合并分区的命令来将多个分区合并为一个分区。合并分区只能通过指定分区名来进行，不支持指定分区值的写法。

⚠ 注意

- 合并分区不能作用于哈希分区上。
- 执行合并分区命令会使得Global索引失效，可以通过UPDATE GLOBAL INDEX子句来同步更新Global索引，或者用户自行重建Global索引。
- 合并前的分区如果包含分类索引则不支持合并。

须知

合并后的新分区，对于范围分区，可以与最后一个源分区名字相同，比如将p1,p2合并为p2；对于列表分区，可以与任一源分区名字相同，比如将p1,p2合并为p1。

如果新分区与源分区名字相同，数据库会将新分区视为对源分区的继承。

使用ALTER TABLE MERGE PARTITIONS可以将多个分区合并为一个分区。

例如，将范围分区表range_sales的分区date_202001和date_202002合并为一个新的分区，并更新Global索引。

```
ALTER TABLE range_sales MERGE PARTITIONS date_202001, date_202002 INTO
   PARTITION date_2020_old UPDATE GLOBAL INDEX;
```


版权所有 © 华为云计算技术有限公司

1.7.4.7 移动分区

用户可以使用移动分区的命令来将一个分区移动到新的表空间中。移动分区可以通过指定分区名或者分区值来进行。

使用ALTER TABLE MOVE PARTITION可以对分区表移动分区。

例如，通过指定分区名将范围分区表range_sales的分区date_202001移动到表空间tb1中。

```
ALTER TABLE range_sales MOVE PARTITION date_202001 TABLESPACE tb1;
```

或者，通过指定分区值将列表分区表list_sales中'0'所对应的分区移动到表空间tb1中。

```
ALTER TABLE list_sales MOVE PARTITION FOR ('0') TABLESPACE tb1;
```

1.7.4.8 重命名分区

用户可以使用重命名分区的命令来将一个分区命名为新的名称。重命名分区可以通过指定分区名或者分区值来进行。

1.7.4.8.1 对分区表重命名分区

使用ALTER TABLE RENAME PARTITION可以对分区表重命名分区。

例如，通过指定分区名将范围分区表range_sales的分区date_202001重命名。

```
ALTER TABLE range_sales RENAME PARTITION date_202001 TO date_202001_new;
```

或者，通过指定分区值将列表分区表list_sales中'0'所对应的分区重命名。

```
ALTER TABLE list_sales RENAME PARTITION FOR ('0') TO channel_new;
```

1.7.4.8.2 对 Local 索引重命名索引分区

使用ALTER INDEX RENAME PARTITION可以对Local索引重命名索引分区。具体方法与一级分区表重命名分区相同。

1.7.4.9 分区表行迁移

用户可以使用ALTER TABLE ENABLE/DISABLE ROW MOVEMENT来开启/关闭分区表行迁移。

开启行迁移时，允许通过更新操作将一个分区中的数据迁移到另一个分区中；关闭行迁移时，如果出现这种更新行为，则业务报错。

须知

如果业务明确不允许对分区键所在列进行更新操作，建议关闭分区表行迁移。

例如，创建列表分区表，并开启分区表行迁移，此时可以跨分区更新分区键所在列；关闭分区表行迁移后，对分区键所在列进行跨分区更新会业务报错。

```
CREATE TABLE list_sales
(
    product_id      INT4 NOT NULL,
    customer_id     INT4 PRIMARY KEY,
    time_id         DATE,
    channel_id      CHAR(1),
    type_id         INT4,
    quantity_sold   NUMERIC(3),
    amount_sold     NUMERIC(10,2)
)
```


版权所有 © 华为云计算技术有限公司

```
PARTITION BY LIST (channel_id)
(
   PARTITION channel1 VALUES ('0', '1', '2'),
   PARTITION channel2 VALUES ('3', '4', '5'),
   PARTITION channel3 VALUES ('6', '7'),
   PARTITION channel4 VALUES ('8', '9')
) ENABLE ROW MOVEMENT;
INSERT INTO list_sales VALUES (153241,65143129,'2021-05-07','0',864134,89,34);
--跨分区更新成功，数据从分区channel1迁移到分区channel2
UPDATE list_sales SET channel_id = '3' WHERE channel_id = '0';
--关闭分区表行迁移
ALTER TABLE list_sales DISABLE ROW MOVEMENT;
--跨分区更新失败，报错fail to update partitioned table "list_sales"
UPDATE list_sales SET channel_id = '0' WHERE channel_id = '3';
--分区内更新依然成功
UPDATE list_sales SET channel_id = '4' WHERE channel_id = '3';
```

1.7.4.10 分区表索引重建/不可用

用户可以通过命令使得一个分区表索引或者一个索引分区不可用，此时该索引/索引分区不再维护。使用重建索引命令可以重建分区表索引，恢复索引的正常功能。

此外，部分分区级DDL操作也会使得Global索引失效，包括删除drop、交换exchange、清空truncate、分割split、合并merge。如果在DDL操作中带UPDATE GLOBAL INDEX子句，则会同步更新Global索引，否则需要用户自行重建索引。

1.7.4.10.1 索引重建/不可用

使用ALTER INDEX可以设置索引是否可用。

例如，假设分区表range_ sales上存在索引range_sales_idx，可以通过如下命令设置其不可用。

```
ALTER INDEX range_sales_idx UNUSABLE;
```

可以通过如下命令重建索引range_sales_idx。

```
ALTER INDEX range_sales_idx REBUILD;
```

1.7.4.10.2 Local 索引分区重建/不可用

- 使用ALTER INDEX PARTITION可以设置Local索引分区是否可用。
- 使用ALTER TABLE MODIFY PARTITION可以设置分区表上指定分区的所有索引分区是否可用。

例如，假设分区表range_sales上存在两张Local索引range_sales_idx1和range_sales_idx2，假设其在分区date_202001上对应的索引分区名分别为range_sales_idx1_part1和range_sales_idx2_part1。

下面给出了维护分区表分区索引的语法：

- 可以通过如下命令设置分区date_202001上的所有索引分区均不可用。
  ```
  ALTER TABLE range_sales MODIFY PARTITION date_202001 UNUSABLE LOCAL INDEXES;
  ```
- 或者通过如下命令单独设置分区date_202001上的索引分区range_sales_idx1_part1不可用。
  ```
  ALTER INDEX range_sales_idx1 MODIFY PARTITION range_sales_idx1_part1 UNUSABLE;
  ```
- 可以通过如下命令重建分区date_202001上的所有索引分区。
  ```
  ALTER TABLE range_sales MODIFY PARTITION date_202001 REBUILD UNUSABLE LOCAL INDEXES;
  ```
- 或者通过如下命令单独重建分区date_202001上的索引分区range_sales_idx1_part1。
  ```
  ALTER INDEX range_sales_idx1 REBUILD PARTITION range_sales_idx1_part1;
  ```

 版权所有 © 华为云计算技术有限公司

1.7.5 分区表系统视图&DFX

1.7.5.1 分区表相关系统视图

分区表系统视图根据权限分为3类，具体字段信息请参考《 开发者指南 》中“ 系统表和系统视图 > 系统视图”章节。

1. 所有分区视图：
   - ADM_PART_TABLES：所有分区表信息。
   - ADM_TAB_PARTITIONS：所有分区信息。
   - ADM_PART_INDEXES：所有Local索引信息。
   - ADM_IND_PARTITIONS：所有索引分区信息。
2. 当前用户可访问的视图：
   - DB_PART_TABLES：当前用户可访问的分区表信息。
   - DB_TAB_PARTITIONS：当前用户可访问的分区信息。
   - DB_PART_INDEXES：当前用户可访问的Local索引信息。
   - DB_IND_PARTITIONS：当前用户可访问的索引分区信息。
3. 当前用户拥有的视图：
   - MY_PART_TABLES：当前用户拥有的分区表信息。
   - MY_TAB_PARTITIONS：当前用户拥有的分区信息。
   - MY_PART_INDEXES：当前用户拥有的Local索引信息。
   - MY_IND_PARTITIONS：当前用户拥有的索引分区信息。

1.7.5.2 分区表相关内置工具函数

前置建表相关信息

- 前置建表：

```
CREATE TABLE test_range_pt (a INT, b INT, c INT)
PARTITION BY RANGE (a)
(
   PARTITION p1 VALUES LESS THAN (2000),
   PARTITION p2 VALUES LESS THAN (3000),
   partition p3 VALUES LESS THAN (4000),
   partition p4 VALUES LESS THAN (5000),
   partition p5 VALUES LESS THAN (MAXVALUE)
)ENABLE ROW MOVEMENT;
```

- 查看分区表OID：

```
SELECT oid FROM pg_class WHERE relname = 'test_range_pt';
oid
-------
49290
(1 row)
```

- 查看分区信息：

```
SELECT oid,relname,parttype,parentid,boundaries FROM pg_partition WHERE parentid = 49290;
oid |    relname    | parttype | parentid | boundaries
-------+---------------+----------+----------+------------
49293 | test_range_pt   | r        |    49290 |
49294 | p1            | p        |    49290 | {2000}
49295 | p2            | p        |    49290 | {3000}
49296 | p3            | p        |    49290 | {4000}
49297 | p4            | p        |    49290 | {5000}
```


版权所有 © 华为云计算技术有限公司

```
49298 | p5            | p       |    49290 | {NULL}
(6 rows)
```

- 创建索引：

```
CREATE INDEX idx_range_a ON test_range_pt(a) LOCAL;
CREATE INDEX
--查看分区索引oid
SELECT oid FROM pg_class WHERE relname = 'idx_range_a';
oid
-------
90250
(1 row)
```

- 查看索引分区信息：

```
SELECT oid,relname,parttype,parentid,boundaries,indextblid FROM pg_partition WHERE parentid =
90250;
oid  | relname  | parttype | parentid | boundaries | indextblid
-------+----------+----------+----------+------------+------------
90255 | p5_a_idx | x       |    90250 |            |      49298
90254 | p4_a_idx | x       |    90250 |            |      49297
90253 | p3_a_idx | x       |    90250 |            |      49296
90252 | p2_a_idx | x       |    90250 |            |      49295
90251 | p1_a_idx | x       |    90250 |            |      49294
(5 rows)
```

工具函数示例

- pg_get_tabledef获取分区表的定义，入参可以为表的OID或者表名。

```
SELECT pg_get_tabledef('test_range_pt');

pg_get_tabledef
----------------------------------------------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------
 SET search_path =
public;
                                                +
 CREATE TABLE test_range_pt
(
                                              +
    a
integer,
                                                        +
    b
integer,
                                                        +
    c
integer
                                                        +
 )
                                                           +
 WITH (orientation=row, compression=no, storage_type=USTORE,
segment=off)
          +
 PARTITION BY RANGE
(a)
                                            +

(
                                                          +
    PARTITION p1 VALUES LESS THAN (2000) TABLESPACE
pg_default,
              +
    PARTITION p2 VALUES LESS THAN (3000) TABLESPACE
pg_default,
              +
    PARTITION p3 VALUES LESS THAN (4000) TABLESPACE
pg_default,
              +
    PARTITION p4 VALUES LESS THAN (5000) TABLESPACE
```

 版权所有 © 华为云计算技术有限公司

```
pg_default,
                +
    PARTITION p5 VALUES LESS THAN (MAXVALUE) TABLESPACE
pg_default
              +
)
                                                    +
 ENABLE ROW
MOVEMENT;
                                                  +
 CREATE INDEX idx_range_a ON test_range_pt USING ubtree (a) LOCAL(PARTITION p1_a_idx,
PARTITION p2_a_idx, PARTITION p3_a_idx, PARTITION p4_a_idx, PARTITION p5_a_idx)  WITH
(storage_type=USTORE) TABLESPACE pg_default;
(1 row)
```

- pg_stat_get_partition_tuples_hot_updated返回给定分区id的分区热更新元组数的统计。

  在分区p1中插入10条数据并更新，统计分区p1的热更新元组数。

  ```
  INSERT INTO test_range_pt VALUES(generate_series(1,10),1,1);
  INSERT 0 10
  SELECT pg_stat_get_partition_tuples_hot_updated(49294);
  pg_stat_get_partition_tuples_hot_updated
  ------------------------------------------
  0
  (1 row)
  UPDATE test_range_pt SET b = 2;
  UPDATE 10
  SELECT pg_stat_get_partition_tuples_hot_updated(49294);
  pg_stat_get_partition_tuples_hot_updated
  ------------------------------------------
  10
  (1 row)
  ```

- pg_partition_size(oid,oid)指定OID代表的分区使用的磁盘空间。其中，第一个oid为表的OID，第二个oid为分区的OID。

  查看分区p1的磁盘空间。

  ```
  SELECT pg_partition_size(49290, 49294);
  pg_partition_size
  -------------------
  90112
  (1 row)
  ```

- pg_partition_size(text, text)指定名称的分区使用的磁盘空间。其中，第一个text为表名，第二个text为分区名。

  查看分区p1的磁盘空间。

  ```
  SELECT pg_partition_size('test_range_pt', 'p1');
  pg_partition_size
  -------------------
  90112
  (1 row)
  ```

- pg_partition_indexes_size(oid,oid)指定OID代表的分区索引使用的磁盘空间。其中，第一个oid为表的OID，第二个oid为分区的OID。

  查看分区p1的索引分区磁盘空间。

  ```
  SELECT pg_partition_indexes_size(49290, 49294);
  pg_partition_indexes_size
  ---------------------------
  204800
  (1 row)
  ```

- pg_partition_indexes_size(text,text)指定名称的分区索引使用的磁盘空间。其中，第一个text为表名，第二个text为分区名。

  查看分区p1的索引分区磁盘空间。

 版权所有 © 华为云计算技术有限公司

```
SELECT pg_partition_indexes_size('test_range_pt', 'p1');
pg_partition_indexes_size
---------------------------
204800
(1 row)
```

- pg_partition_filenode(partition_oid)获取到指定分区表的OID所对应的filenode。
  查看分区p1的filenode。

```
SELECT pg_partition_filenode(49294);
pg_partition_filenode
-----------------------
49294
(1 row)
```

- pg_partition_filepath(partition_oid)指定分区的文件路径名。
  查看分区p1的文件路径。

```
SELECT pg_partition_filepath(49294);
pg_partition_filepath
-----------------------
base/16521/49294
(1 row)
```

1.8 存储引擎

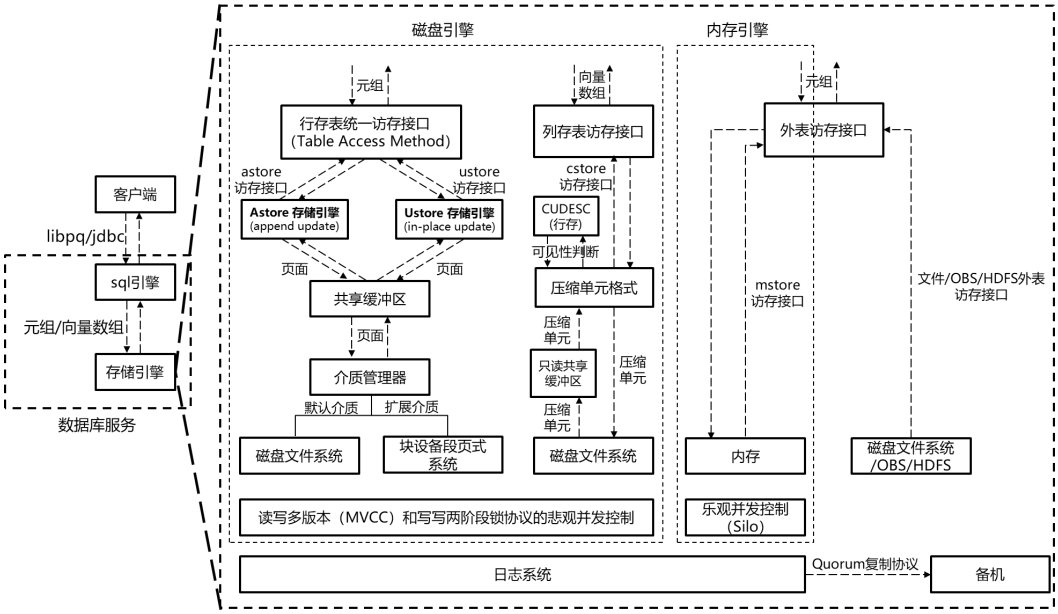
1.8.1 存储引擎体系架构

1.8.1.1 存储引擎体系架构概述

1.8.1.1.1 静态编译架构

从整个数据库服务的组成构架来看，存储引擎向上对接SQL引擎，为SQL引擎提供或接收标准化的数据格式（元组或向量数组）；存储引擎向下对接存储介质，按照特定的数据组织方式，以页面、压缩单元（Compress Unit）或其他形式为单位，通过存储介质提供的特定接口，对存储介质中的数据完成读写操作。GaussDB通过静态编译使数据库专业人员可以为特定的应用程序需求选择专用的存储引擎。为了减少对执行引擎的干扰，提供行存访问接口层TableAM，用来屏蔽底层行存引擎带来的差异，使得不同行存引擎可以分别独立演进。如下图所示。

版权所有 © 华为云计算技术有限公司



在此基础之上，存储引擎通过日志系统提供数据的持久化和可靠性能力。通过并发控制（事务）系统保证同时执行的、多个读写操作之间的原子性、一致性和隔离性，通过索引系统提供对特定数据的加速寻址和查询能力，通过主备复制系统提供整个数据库服务的高可用能力。

行存引擎主要面向OLTP（OnLine Transaction Processing）类业务应用场景，适合高并发、小数据量的单点或小范围数据读写操作。行存引擎向上为SQL引擎提供元组形式的读写接口，向下以页面为单位通过可扩展的介质管理器对存储介质进行读写操作，并通过页面粒度的共享缓冲区来优化读写操作的效率。对于读写并发操作，采用多版本并发控制（MVCC，Multi-Version Concurrency Control）；对于写写并发操作，采用基于两阶段锁协议（2PL，Two-Phase Locking）的悲观并发控制（PCC，Pessimistic Concurrency Control）。当前，行存引擎默认的介质管理器采用磁盘文件系统接口，后续可扩展支持块设备等其他类型的存储介质。GaussDB行存引擎可以选择基于Append update 的Astore或基于In-place update的Ustore。

1.8.1.1.2 通用数据库服务层

从技术角度来看，存储引擎需要一些基础架构组件，主要包括：

并发：不同存储引擎选择正确的锁可以减少开销，从而提高整体性能。此外提供多版本并发控制或“快照”读取等功能。

事务：均需满足ACID的要求，提供事务状态查询等功能。

内存缓存：不同存储引擎在访问索引和数据时一般会对其进行缓存。缓存池允许直接从内存中处理经常使用的数据，从而加快了处理速度。

检查点：不同存储引擎一般都支持增量checkpoint/double write或全量checkpoint/full page write模式。应用可以根据不同条件进行选择增量或者全量，这个对存储引擎是透明的。

日志：GaussDB采用的是物理日志，其写入/传输/回放对存储引擎透明。

版权所有 © 华为云计算技术有限公司

1.8.1.2 设置存储引擎

存储引擎会对数据库整体效率和性能具有巨大影响，请根据实际需求选择适当的存储引擎。用户可使用WITH ([ORIENTATION | STORAGE_TYPE] [= value] [, ...])为表或索引指定一个可选的存储参数。参数的详细描述如下所示：

ORIENTATION	STORAGE_TYPE
ROW（缺省值）：表的数据将以行式存储。	[USTORE(缺省值)|ASTORE|空]

如果ORIENTATION指定为ROW，且STORAGE_TYPE为空的情况下创建出的表类型取决于GUC参数enable_default_ustore_table（取值为on/off，默认情况为on，参数详情请参见《管理员指南》中“配置运行参数 > GUC参数说明”章节）：如果参数设置为on，创建出的表为Ustore类型；如果为off，创建出的表为Astore类型。

具体示例如下：

```
gaussdb=# CREATE TABLE TEST(a int);
gaussdb=# \d+ test
                    Table "public.test"
 Column |  Type   | Modifiers | Storage | Stats target | Description
--------+---------+-----------+---------+--------------+-------------
 a      | integer |           | plain   |              |
Has OIDs: no
Options: orientation=row, compression=no, storage_type=USTORE, segment=off

gaussdb=# CREATE TABLE TEST1(a int) with(orientation=row, storage_type=ustore);
gaussdb=# \d+ test1
Table "public.test1"
 Column |  Type   | Modifiers | Storage | Stats target | Description
--------+---------+-----------+---------+--------------+-------------
 a      | integer |           | plain   |              |
Has OIDs: no
Options: orientation=row, storage_type=ustore, compression=no, segment=off

gaussdb=# CREATE TABLE TEST2(a int) with(orientation=row, storage_type=astore);
gaussdb=# \d+ test2
Table "public.test2"
 Column |  Type   | Modifiers | Storage | Stats target | Description
--------+---------+-----------+---------+--------------+-------------
 a      | integer |           | plain   |              |
Has OIDs: no
Options: orientation=row, storage_type=astore, compression=no
gaussdb=# CREATE TABLE test4(a int) with(orientation=row);
gaussdb=# \d+
                                        List of relations
 Schema | Name  | Type |   Owner   | Size  |                          Storage                          | Description
--------+-------+-------+-----------+---------+-----------------------------------------------------------------------
+-------------
 public | test  | table | z7ee88f3a | 0 bytes |
{orientation=row,compression=no,storage_type=USTORE,segment=off} |
 public | test1 | table | z7ee88f3a | 0 bytes |
{orientation=row,storage_type=ustore,compression=no,segment=off} |
 public | test2 | table | z7ee88f3a | 0 bytes | {orientation=row,storage_type=astore,compression=no}            |
 public | test3 | table | z7ee88f3a | 16 kB   |
{orientation=column,storage_type=astore,compression=low}         |
 public | test4 | table | z7ee88f3a | 0 bytes |
{orientation=row,compression=no,storage_type=USTORE,segment=off} |
(5 rows)

gaussdb=# show enable_default_ustore_table;
 enable_default_ustore_table
```


版权所有 © 华为云计算技术有限公司

```
----------------------------
 on
(1 row)

gaussdb=#  DROP TABLE test;
gaussdb=#  DROP TABLE test1;
gaussdb=#  DROP TABLE test2;
gaussdb=#  DROP TABLE test4;
```

1.8.1.3 存储引擎更新说明

1.8.1.3.1 GaussDB 内核 505 版本

- Ustore支持柔性字段高效存储。

- Ustore支持Toast规模商用。

- Ustore增加页面恢复与逃生技术。

- Ustore支持SMP技术。

1.8.1.3.2 GaussDB 内核 503 版本

- Ustore适配分布式/并行查询/Global Temp Table/Vacuum full/列约束DEFERRABLE以及INITIALLY DEFERRED。

- Ustore增加在线重建索引。

- Ustore增加增强版本B-tree空页面估算，提升优化器代价估算准确度。

- Ustore增加存储引擎可靠性验证框架，Dignose Page/Page Verify。

- Ustore增强存储引擎相关的解析/检测/修复视图。

- Ustore增强基于WAL日志的定位能力，新增gs_redo_upage系统视图，支持对单页面的不断重放，获取并打印该页面的任何一个历史版本，加速页面损坏类问题的定位。

- Ustore扩展事务槽TD物理格式，为事务内空间复用做好铺垫。

- Ustore增加在线创建索引。

- Ustore适配闪回功能（for Ustore）/极致RTO。

1.8.1.3.3 GaussDB 内核 R2 版本

- Ustore增加新的基于原位更新的行存储引擎Ustore，首次实现新旧版本的记录的分离存储。

- Ustore增加回滚段模块。

- Ustore增加回滚过程，支持同步/异步/页内模式。

- Ustore增加支持事务的增强版本B-tree。

- Astore增加闪回功能，支持闪回表/闪回查询/闪回Drop/闪回Truncate。

- Ustore不支持的特性包括：分布式/并行查询/Table Sampling/Global Temp Table/在线创建/重建索引/极致RTO/Vacuum Full/列约束DEFERRABLE以及INITIALLY DEFERRED。

 版权所有 © 华为云计算技术有限公司

1.8.2 Astore 存储引擎

1.8.2.1 Astore 简介

Astore与Ustore的多版本实现最大的区别在于最新版本和历史版本是否分离存储。Astore不进行分离存储，而Ustore当前也只是分离了数据，索引本身没有分开。

使用 Astore 的优势

1. Astore没有回滚段，而Ustore有回滚段。对于Ustore来说，回滚段是非常重要的，回滚段损坏会导致数据丢失甚至数据库无法启动的严重问题，且Ustore恢复时同步需要Redo和Undo。由于Astore没有回滚段，旧数据都是记录在原先的文件中；所以，当数据库异常crash后恢复时，不会像Ustore数据库那样进行复杂的恢复。
2. 由于旧的数据是直接记录在数据文件中，而不是回滚段中，所以不会经常报Snapshot Too Old错误。
3. 回滚可以很快完成，因为回滚并不删除数据。

   ⚠ 注意

   回滚时很复杂，在事务回滚时必须清理该事务所进行的修改，插入的记录要删除，更新的记录要更新回来，同时回滚的过程也会再次产生大量的Redo日志。

4. WAL日志要简单一些，仅需要记录数据文件的变化，不需要记录回滚段的变化。
5. 支持回收站（闪回DROP、闪回Truncate）功能。

1.8.3 Ustore 存储引擎

1.8.3.1 Ustore 简介

Ustore（Unified Storage）是GaussDB推出的一款原位更新的存储引擎，其多版本的实现较Astore最大的区别在于最新版本和历史版本的数据是分离存储的，而索引当前还没有分离。

使用 Ustore 的优势

- 最新版本和历史版本分离存储，相比Astore扫描范围小。去除Astore的HOT chain，非索引列/索引列更新，Heap均可原位更新，ROWID可保持不变。历史版本可批量回收，空间膨胀可控。
- B-tree索引增加了事务信息，能够独立进行MVCC，增加了IndexOnlyScan的比例，大大减少回表次数。
- 不依赖Vacuum进行旧版本清理。独立的空间回收能力，索引与堆表解耦，可独立清理，IO平稳度更优。
- 大并发更新同一行的场景，相对于Astore的ROWID会偏移，Ustore的原位更新机制保证了元组ROWID稳定，先到先得，更新时延相对稳定。
- 支持闪回功能。

版权所有 © 华为云计算技术有限公司

> **⚠ 注意**
>
> Ustore DML在修改数据页面时，也需要同步生成Undo，因此更新操作开销会稍大一些。此外单条Tuple扫描开销由于需要复制（Astore返回指针）也会大一些。

1.8.3.1.1 Ustore 特性与规格

特性约束

类别	特性	是否支持
事务	Serializable	×
	在事务块中对分区表执行DDL操作	×
可扩展性	Hashbucket	×
SQL	Table sampling/物化视图/键值锁	×

存储规格

1. 数据表最大列数不能超过1600列。
2. init_td（TD（Transaction Directory，事务目录）是Ustore表独有的用于存储页面事务信息的结构，TD的数量决定该页面支持的最大并发数。在创建表或索引时可以指定初始的TD大小init_td）取值范围[2, 128]，默认值4。单页面支持的最大并发不超过128个。
3. Ustore表（不含toast情况）最大Tuple长度不能超过（8192 - MAXALIGN(56 + init_td * 26 + 4)），其中MAXALIGN表示8字节对齐。当插入数据长度超过阈值时，用户会收到元组长度过长无法插入的报错。其中init_td对于Tuple长度的影响如下：
   - 表init_td数量为最小值2时，Tuple长度不能超过8192 - MAXALIGN(56+2*26+4) = 8080B。
   - 表init_td数量为默认值4时，Tuple长度不能超过8192 - MAXALIGN(56+4*26+4) = 8024B。
   - 表init_td数量为最大值128时，Tuple长度不能超过8192 - MAXALIGN(56+128*26+4) = 4800B。
4. 索引最大列数不能超过32列。全局分区索引最大列数不能超过31列。
5. 索引元组长度不能超过(8192 - MAXALIGN(28 + 3 * 4 + 3 * 10) - MAXALIGN(42))/3, 其中MAXALIGN表示8字节对齐。当插入数据长度超过阈值时，用户会收到索引元组长度过长无法插入的报错，其中索引页头为28B，行指针为4B，元组CTID+INFO标记位为10B，页尾为42B。
6. 回滚段容量最大支持16TB。

1.8.3.1.2 使用 Ustore 进行测试

创建Ustore表

使用CREATE TABLE语句创建Ustore表。

版权所有 © 华为云计算技术有限公司

```
gaussdb=# CREATE TABLE ustore_table(a INT PRIMARY KEY, b CHAR (20)) WITH (STORAGE_TYPE=USTORE);
NOTICE:  CREATE TABLE / PRIMARY KEY will create implicit index "ustore_table_pkey" for table
"ustore_table"
CREATE TABLE
gaussdb=# \d+ ustore_table
Table "public.ustore_table"
Column |     Type      | Modifiers | Storage  | Stats target | Description
--------+---------------+-----------+----------+--------------+-------------
a      | integer       | not null  | plain    |              |
b      | character(20) |           | extended |              |
Indexes:
"ustore_table_pkey" PRIMARY KEY, ubtree (a) WITH (storage_type=USTORE) TABLESPACE pg_default
Has OIDs: no
Distribute By: HASH(a)
Location Nodes: ALL DATANODES
Options: orientation=row, storage_type=ustore, compression=no, segment=off
```

删除Ustore表

```
gaussdb=# DROP TABLE ustore_table;
 DROP TABLE
```

为Ustore表创建索引

Ustore当前仅支持BTree类型的多版本索引。在一些场景中，为了区别于Astore的BTree索引，也会将Ustore表的多版本BTree索引称为UBTree（Ustore BTree，UBTree介绍详见**UBTree**章节）。用户可以参照以下方式使用CREATE INDEX语句为Ustore表的“a”属性创建一个UBTree索引。

Ustore表不指定创建索引类型，默认创建的是UBTree索引。

⚠ 注意

UBTree索引分为RCR版本和PCR版本，默认创建RCR版本的UBTree。若在创建索引时with选项指定(index_txntype=pcr)或者指定GUC的index_txntype=pcr，则创建的是PCR版本的UBTree。

```
gaussdb=# CREATE TABLE test(a int);
CREATE TABLE
gaussdb=# CREATE INDEX UB_tree_index ON test(a);
CREATE INDEX
gaussdb=# \d+ test
Table "public.test"
 Column |  Type   | Modifiers | Storage | Stats target | Description
--------+---------+-----------+---------+--------------+-------------
 a      | integer |           | plain   |              |
Indexes:
    "ub_tree_index" ubtree (a) WITH (storage_type=USTORE) TABLESPACE pg_default
Has OIDs: no
Distribute By: HASH(a)
Location Nodes: ALL DATANODES
Options: orientation=row, compression=no, storage_type=USTORE, segment=off

--删除Ustore表索引。
gaussdb=# DROP TABLE test;
DROP TABLE
```

1.8.3.1.3 Ustore 的最佳实践

版权所有 © 华为云计算技术有限公司

怎么配置 init_td 大小

TD（Transaction Directory，事务目录）是Ustore表独有的用于存储页面事务信息的结构，TD的数量决定该页面支持的最大并发数。在创建表或索引时可以指定初始的TD大小init_td，默认值为4，即同时支持4个并发事务修改该页面，最大值为128。

用户需要结合业务并发度分析是否需要手动配置init_td。另外也可以结合业务运行过程中“wait available td”等待事件出现的频率来分析是否需要调整，一般“wait available td”等于0。如果“wait available td”一直不为0，就存在等待TD的事件，此时建议增大init_td再进行观察，反复几次，如果大于0的情况属于偶发，不建议调整，多余的TD槽位会占用更多的空间。推荐的增大的方法可以按照倍数进行测试，建议可从小到大尝试8、16、32、48、...、128，并观测对应的等待事件是否有明显减少，尽量取等待事件较少中init_td数量最小的值作为默认值以节省空间。wait available td是wait_status的值之一，wait_status表示当前线程的等待状态，包含等待状态详细信息。通过PG_THREAD_WAIT_STATUS视图可以查询wait_status的值（none表示没在等待任意事件，如果有等待事件即可看到对应wait available td的值），示例如下。init_td的配置和详细描述参见《开发者指南》的“SQL参考 > SQL语法 > CREATE TABLE”章节。init_td查看和修改方法的具体实例如下：

```
gaussdb=# CREATE TABLE test1(name varchar) WITH(storage_type = ustore, init_td=2);
gaussdb=# \d+ test1
                        Table "public.test1"
 Column |       Type        | Modifiers | Storage  | Stats target | Description
--------+-------------------+-----------+----------+--------------+-------------
 name   | character varying |           | extended |              |
Has OIDs: no
Distribute By: HASH(a)
Location Nodes: ALL DATANODES
Options: orientation=row, storage_type=ustore, init_td=2, compression=no, segment=off,
toast.storage_type=ustore, toast.toast_storage_type=enhanced_toast

gaussdb=# ALTER TABLE test1 SET(init_td=8);
gaussdb=# \d+ test1
                        Table "public.test1"
 Column |       Type        | Modifiers | Storage  | Stats target | Description
--------+-------------------+-----------+----------+--------------+-------------
 name   | character varying |           | extended |              |
Has OIDs: no
Distribute By: HASH(a)
Location Nodes: ALL DATANODES
Options: orientation=row, storage_type=ustore, compression=no, segment=off, init_td=8,
toast.storage_type=ustore, toast.toast_storage_type=enhanced_toast

gaussdb=# SELECT * FROM pg_thread_wait_status;
 node_name | db_name  |          thread_name          |    query_id     |       tid       |    sessionid    | lwtid |
psessionid | tlevel | smpid | wait_status | wait_event | locktag | lo
ckmode | block_sessionid | global_sessionid
-----------+----------+-------------------------------+-----------------+-----------------+-----------------+-------
+------------+--------+-------+-------------+------------+---------+---
-------+-----------------+-----------------
 sgnode    |          | PageWriter                    |               0 | 139769678919424 | 139769678919424 | 16915
|          |     0 |     0 | none        | none       |         |
      |               | 0:0#0
 sgnode    |          | PageWriter                    |               0 | 139769736066816 | 139769736066816 | 16913
|          |     0 |     0 | none        | none       |         |
      |               | 0:0#0
 sgnode    |          | PageWriter                    |               0 | 139769707755264 | 139769707755264 | 16914
|          |     0 |     0 | none        | none       |         |
      |               | 0:0#0
 sgnode    |          | PageWriter                    |               0 | 139769761756928 | 139769761756928 | 16912
|          |     0 |     0 | none        | none       |         |
      |               | 0:0#0
 sgnode    |          | PageWriter                    |               0 | 139769783772928 | 139769783772928 | 16911
|          |     0 |     0 | none        | none       |         |
      |               | 0:0#0
```

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# DROP TABLE test1;
DROP TABLE
```

怎么配置 fillfactor 大小

fillfactor是用于描述页面填充率的参数，该参数与页面能存放的元组数量、大小以及表的物理空间直接相关。Ustore表的默认页面填充率为92%，预留的8%空间用于更新的扩展，也可以用于TD列表的扩展空间。fillfactor的配置和详细描述参见《开发者指南》的“SQL参考 > SQL语法 > CREATE TABLE”章节。

用户需要结合业务分析是否需要手动配置fillfactor。如果表数据导入后只有查询或定长更新操作，可将页面填充率调整为100%。如果数据导入后存在大量非定长更新操作，建议为不调整页面填充率或者将页面填充率数值调整更小，以减少跨页更新带来的性能损耗。fillfactor查看和修改方法具体实例如下：

```
gaussdb=# CREATE TABLE test(a int) with(fillfactor=100);
gaussdb=# \d+ test
                    Table "public.test"
 Column |  Type   | Modifiers | Storage | Stats target | Description
--------+---------+-----------+---------+--------------+-------------
 a      | integer |           | plain   |              |
Has OIDs: no
Distribute By: HASH(a)
Location Nodes: ALL DATANODES
Options: orientation=row, fillfactor=100, compression=no, storage_type=USTORE, segment=off

gaussdb=# ALTER TABLE test set(fillfactor=92);
gaussdb=# \d+ test
                    Table "public.test"
 Column |  Type   | Modifiers | Storage | Stats target | Description
--------+---------+-----------+---------+--------------+-------------
 a      | integer |           | plain   |              |
Has OIDs: no
Options: orientation=row, compression=no, storage_type=USTORE, segment=off， fillfactor=92

gaussdb=# DROP TABLE test;
DROP TABLE
```

在线校验功能

在线校验是Ustore特有的，在运行过程中可以有效预防页面因编码逻辑错误导致的逻辑损坏，默认开启UPAGE:UBTREE:UNDO三个模块校验。业务现网请保持开启，性能场景除外。

关闭：

```
gs_guc reload -Z coordinator -Z datanode -N all -I all -c "ustore_attr=''"
```

打开：

```
gs_guc reload -Z coordinator -Z datanode -N all -I all -c
"ustore_attr='ustore_verify_level=fast;ustore_verify_module=upage:ubtree:undo'"
```

怎么配置回滚段大小

一般情况下回滚段大小的参数使用默认值即可。为了达到最佳性能，部分场景下可调整回滚段大小的相关参数。具体场景与设置方法如下：

1. 保留给定时间内的历史版本数据。

   当使用闪回或者支撑问题定位时，通常希望保留更多历史版本数据，此时需要修改undo_retention_time。undo_retention_time默认值是0，取值范围为 0~3天，

 版权所有 © 华为云计算技术有限公司

输入有效单位为s,min,h,d。调整的推荐值为900s，需要注意的是，undo_retention_time的取值越大，对业务的影响除了Undo空间占用增多，也会造成数据空间膨胀，进一步影响数据扫描更新性能。当不使用闪回或者希望减少历史旧版本的磁盘空间占用时，需要将undo_retention_time调小来达到最佳性能。可以通过如下方法选择更适合自己业务模型的取值：

使用Undo统计信息的系统函数gs_stat_undo，如果入参为false，输出undo_space_limit_size、undo_limit_size_per_transaction 、undo_retention_time 参数的合理化建议。具体详细参数值参见《 开发者指南 》的“SQL参考 > 函数和操作符 > undo系统函数”章节。

2. 保留给定空间大小的历史版本数据。

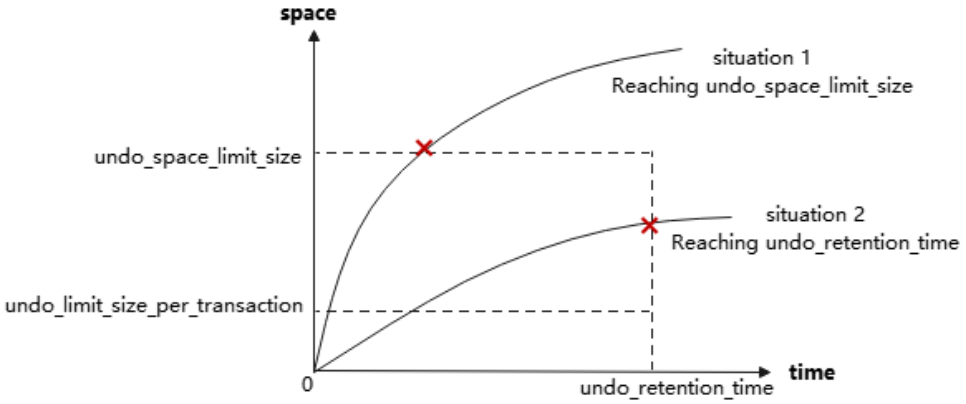
   如果业务中存在长事务或大事务可能导致Undo空间膨胀时，需要将undo_space_limit_size调大，undo_space_limit_size默认值为256GB，取值范围为800MB~16TB。

   在磁盘空间允许的条件下，推荐undo_space_limit_size设置翻倍。同时undo_space_limit_size的取值越大则占用磁盘空间越大，可能降低性能。如果查询视图系统函数gs_stat_undo的curr_used_undo_size发现不存在Undo空间膨胀，可以恢复为原值。

   调整undo_space_limit_size后可相应提高单事务平均占用undo空间undo_limit_size_per_transaction的取值，undo_limit_size_per_transaction取值范围为2MB~16TB，默认值为32GB。设置时建议undo_limit_size_per_transaction不超过undo_space_limit_size，即单事务Undo分配空间阈值不大于Undo总空间阈值。

3. 历史版本的保留参数的调整优先级。

   在undo_retention_time、undo_space_limit_size、undo_limit_size_per_transaction中，先触发的空间阈值会先进行约束限制。



例如：Undo强制回收阈值参数undo_space_limit_size设置为1GB，Undo旧版本保留时间undo_retention_time为900s。如果900s内产生的历史版本数据不足1GB*0.8，则按照900s进行回收限制；否则按照1GB*0.8进行回收限制。遇到该情况时，如果磁盘空闲空间充足，则上调undo_space_limit_size，如果磁盘空闲空间紧缺，则下调undo_retention_time。

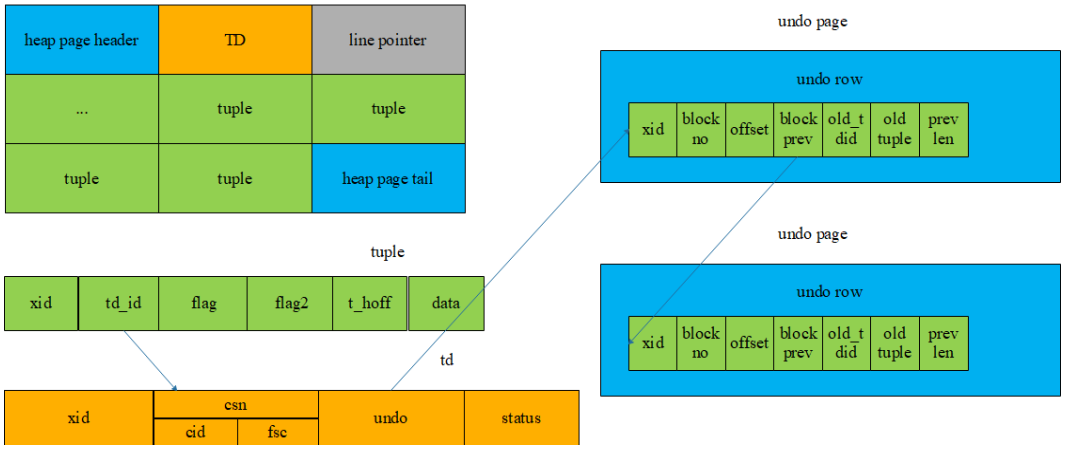
1.8.3.2 存储格式

1.8.3.2.1 RCR Uheap

版权所有 © 华为云计算技术有限公司

RCR Uheap 多版本管理

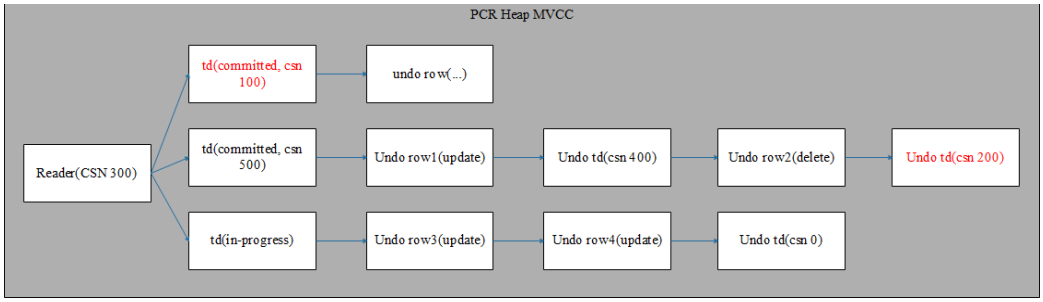
Ustore对其使用的heap做了如下重要的增强，简称Uheap。



Ustore RCR(Row Consistency Read)的多版本管理是基于数据行的行级多版本管理。不过Ustore将XID记录在了页面的TD(Transaction Directory)区域区别于常见的将XID存储在数据行上，节省了页面空间。事务修改记录时，会将历史数据记录到Undo Row中，在Tuple中的td_id指向的TD槽上记录产生的Undo Row地址(zone_id, block no, page offset)，并将新的数据覆盖写入页面。访问元组时，沿着版本链还原该元组，直到找到自己对应的版本。

RCR Uheap 可见性机制

Ustore可见性判断是通过构建数据行的一致性版本获得的，老快照可通过Undo记录获取历史版本。



RCR Uheap 空闲空间管理

Ustore使用Free Space Map（FSM）文件记录了每个数据页的潜在空闲空间，并且以树的结构组织起来。每当用户想要对某个表执行插入操作或者是非原位更新操作时，就会从该表对应的FSM中进行快速查找，查看当前FSM上记录的最大空闲空间是否可以满足插入所需的空间要求；如果满足则返回对应的blocknum用于执行插入操作，否则执行拓展页面逻辑。

每一个表或者分区对应的FSM结构存放在一个独立的FSM文件中，该FSM文件与表数据放在相同的目录下。例如，假设表t1对应的数据文件为32181，则其对应的FSM文件为32181_fsm。FSM内部同样是以数据块的格式存储，这里称为FSM block，FSM block之间的逻辑结构组成了一棵有三层节点的树，树的节点在逻辑上是大顶堆关系。每次在FSM上查找时从根节点进行，一直查找到叶子节点，然后在叶子节点内搜索到一个可用的页面并返回给业务用于执行后续操作。

版权所有 © 华为云计算技术有限公司

该结构不保证和数据页实际可用空间保持实时一致，会在DML的执行过程中进行维护。Ustore会在Auto Vacuum的过程中概率性对该FSM进行修复重建。当用户执行插入类型的DML语句，类似Insert/Non-Inplace Update(新页面)/Multi Insert时，会查询FSM结构，寻找到一个可以插入当前记录的空间。用户执行完DML操作后会根据当前页面的潜在空闲空间与实际空闲空间的差值来决定是否将该页面的空闲空间刷新到FSM上。该差值越大，即潜在空间大于实际空间越多，则该页面被更新至FSM的几率越大。FSM上会记录数据页的潜在空闲空间 ，在用户执行插入操作找到一个页面时，如果该页面上的空闲空间较大则直接插入，否则如果潜在空间较大则对页面执行清理后插入。最后如果空间不够则重新搜索FSM结构或者拓展总页面数量。更新FSM结构主要有以下几个位置，DML、页面清理、vacuum、拓展页面、分区合并、页面扫描等。

1.8.3.2.2 UBTree

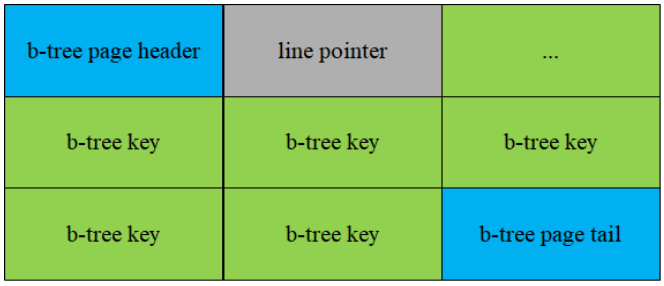
其使用的btree做了如下重要的增强，简称Ubtree。

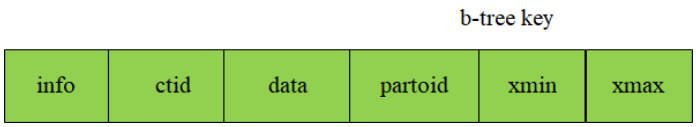
- Ubtree索引增加了事务信息，能够独立进行MVCC；增加了IndexOnlyScan的比例，大大减少回表次数。
- 不依赖Vacuum进行旧版本清理。独立的空间回收能力，索引与堆表解耦，可独立清理，IO平稳度更优。

RCR UBTree

?.1. RCR UBTree 多版本管理

RCR(Row Consistency Read) btree 的多版本管理是基于数据行的行级多版本管理。将XID记录在了数据行上，会增加Key的大小，索引会有5-20%左右的膨胀。最新版本和历史版本均在btree上，索引没有记录Undo信息。插入或者删除key时按照key + TID的顺序排列，索引列相同的元组按照对应元组的TID作为第二关键字进行排序，会将xmin、xmax追加到key的后面。索引分裂时，多版本信息随着key的迁移而迁移。

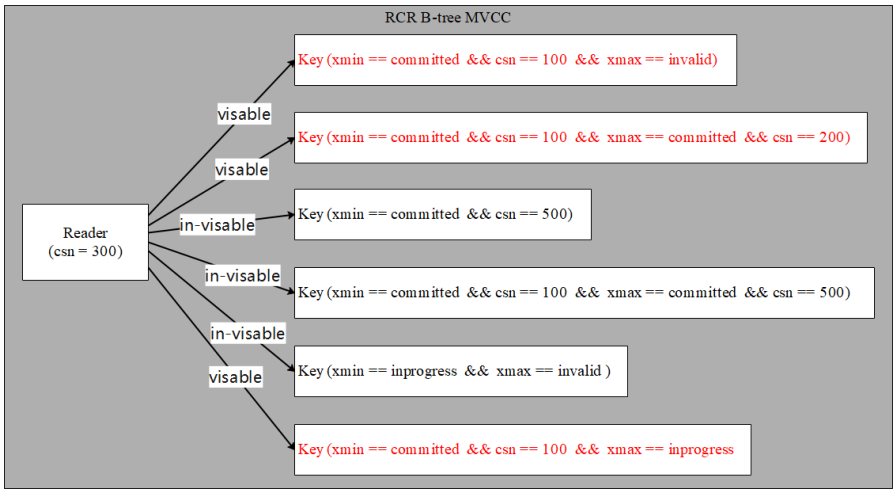




?.2. RCR UBTree 可见性机制

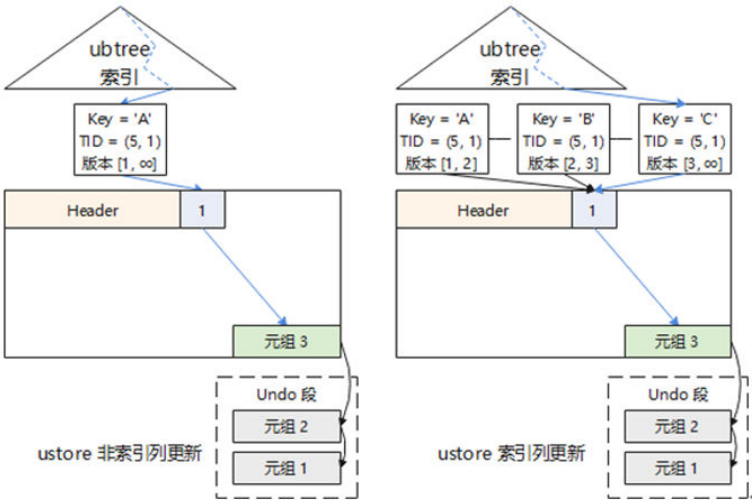
RCR UBTree可见性判断是通过判断Key上xmin/xmax来确定的，与Astore堆表数据行上xmin/xmax作用类似。

版权所有 © 华为云计算技术有限公司



?.3. RCR UBTree 增删改查

- **Insert操作**：Ubtree的插入逻辑基本不变，只需增加索引插入时直接获取事务信息填写xmin字段。
- **Delete操作**：Ubtree额外增加了索引删除流程。索引删除主要步骤与插入相似，获取事务信息填写xmax字段（B-tree索引不维护版本信息，不需要删除操作）,同时更新页面上的active_tuple_count。若active_tuple_count被减为0，则尝试页面回收。
- **Update操作**：对于Ustore而言，数据更新对Ubtree索引列的操作也与Astore有所不同。数据更新包含两种情况：索引列和非索引列更新，下图给出了Ubtree在数据发生更新时的处理。



上图展示Ubtree在索引列和非索引列更新的差异：

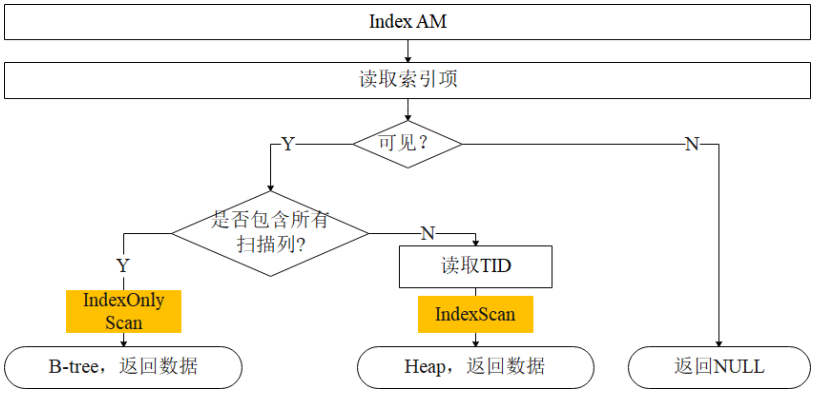
- 在非索引列更新的情况下，索引不发生任何变化。index tuple仍指向第一次插入的data tuple，Uheap不会插入新的data tuple，而是修改当下data tuple并将历史数据存入Undo中。
- 在索引列更新的情况下，Ubtree也会插入新的index tuple，但是会指向同一个data linepointer和同一个data tuple。扫描旧版本的数据则需要从Undo中读取。

- **Scan操作**：用户在读取数据时，可通过使用索引扫描加速，Ubtree支持索引数据的多版本管理及可见性检查，索引层的可见性检查使得索引扫描（Index Scan）及仅索引扫描（IndexOnly Scan）性能有所提升。

版权所有 © 华为云计算技术有限公司

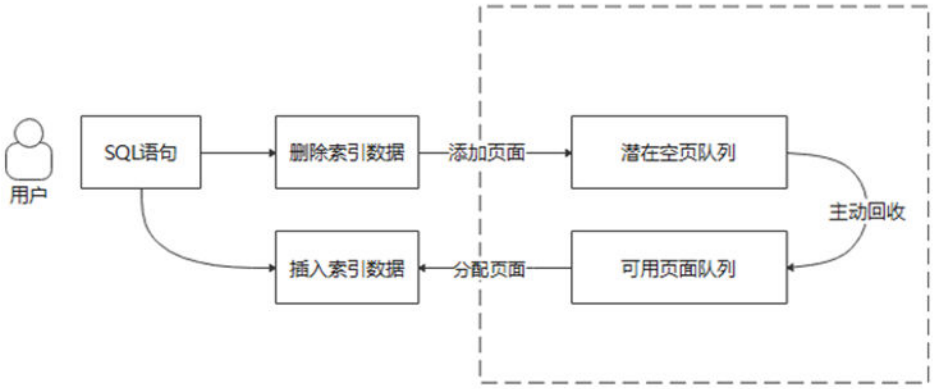
对于索引扫描：

- 若索引列包含所有扫描列（IndexOnly Scan），则通过扫描条件在索引上进行二分查找，找到符合条件元组即可返回数据。
- 若索引列不包含所有扫描列（Index Scan），则通过扫描条件在索引上进行二分查找，找到符合条件元组的TID，再通过TID到数据表上查找对应的数据元组。如下图所示。



?.4. RCR UBTree 空间管理

当前Astore的索引依赖AutoVacuum和Free Space Map（FSM）进行空间管理，存在回收不及时的问题，而Ustore的索引使用其特有的URQ（UBTree Recycle Queue，一种基于循环队列的数据结构，即双循环队列）对索引空闲空间进行管理。双循环队列是指有两个循环队列，一个潜在空页队列，另一个可用空页队列。在DML过程中完成索引的空间管理，能有效地缓解DML过程中造成的空间急剧膨胀问题。 索引回收队列单独储存在B-tree索引对应的FSM文件中。



如上图所示，索引页面在双循环队列间流动如下：

1. **索引空页流动到潜在队列**

   索引页尾字段中记录了页面上活跃元组个数（activeTupleCount）。在DML过程中，删空一个页面的所有元组，即activeTupleCount为零时会将索引页放入潜在队列中。

2. **潜在队列流动到可用队列**

   潜在队列到可用队列的转化主要是达到一个潜在队列收支平衡以及可用队列在拿页时有页可拿的目的。即当从可用队列拿出一个索引空页用完后，建议从潜在队列转化至少一个索引页面到可用队列中，以及每当潜在队列新加入一个索引页面

版权所有 © 华为云计算技术有限公司

时，能从潜在队列中移除至少一个索引页插入可用队列中，达到潜在队列的收支平衡，以及可用队列有页可用的目的。

3. **可用队列流动到索引空页**

   索引在分裂等获取一个索引空页面时，会先从可用队列中进行查找是否有可以复用的索引页，如果找到则直接进行复用，没有可复用页面则进行物理扩页。

PCR UBTree

相比于RCR版本的UBTree，PCR版本的UBTree有以下特点。

- 索引元组的事务信息统一由TD槽进行管理。
- 增加了Undo操作，插入和删除前需要先写入Undo，事务abort时需要进行回滚操作。
- 支持闪回。

PCR UBTree通过在创建索引时with选项设置“index_txntype=pcr”或者设置GUC参数“index_txntype=pcr”进行创建。若没有显示指定with选项或者GUC，则默认创建RCR版本的UBTree。当前PCR版本的UBTree不支持在线进行创建、极致RTO回放和备机读的功能。

注意，当前版本PCR索引在大数据量的回滚上耗时可能较长（回滚时间随数据量增长可能呈指数型增长，数据量太大可能会导致回滚未完成），回滚时间会在下个版本进行优化。以下是当前版本回滚时间的具体规格：

表 1-6 PCR 索引回滚时间的规格

类型/数据量	100	1000	1万	10万	100万
带PCR索引的回滚时间	0.692 ms	9.610 ms	544.678 ms	52,963.754 ms	89,440,029.048 ms
不带PCR索引的回滚时间	0.226 ms	0.916 ms	8.974 ms	94.903 ms	1206.177 ms
两者比值	3.06	10.49	60.70	558.08	74,151.66

?.1. PCR UBTree 多版本管理

与RCR UBTree的区别是，PCR(Page Consistency Read) 的多版本管理是基于页面的多版本管理，所有元组的事务信息统一由TD槽进行管理。

?.2. PCR UBTree 可见性机制

PCR UBTree可见性判断是通过把页面回滚到快照可见的时刻得到元组全部可见的页面完成的。

?.3. PCR UBTree 增删改查

- **Insert操作**：操作与RCR UBTree基本一致，区别是：插入前需要先申请TD和写入Undo。

版权所有 © 华为云计算技术有限公司

- **Delete操作**：操作与RCR UBTree基本一致，区别是：删除前需要先申请TD和写入Undo。
- **Update操作**：操作与RCR UBTree无区别，均转换为一条Delete操作和和一条Insert操作。
- **Scan操作**：操作与RCR UBTree基本一致，区别是：查询操作需要将页面复制一个CR页面出来，将CR页面回滚到扫描快照可见的状态，从而整个页面的元组对于快照都是可见版本。

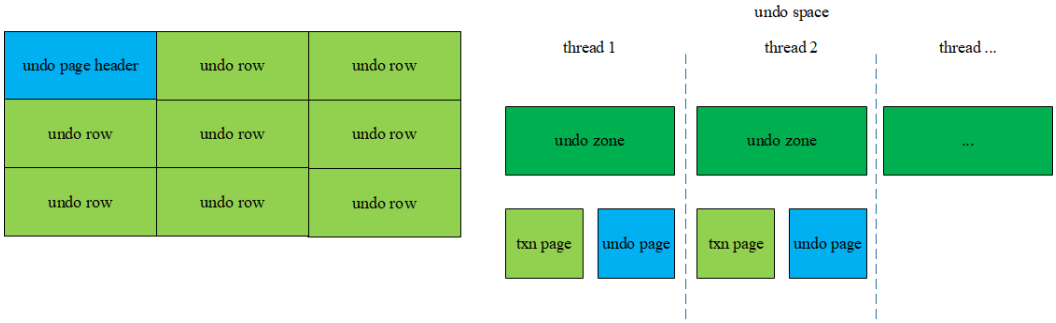
?.4. PCR UBTree 空间管理

空间管理操作与RCR UBTree基本一致，区别是：PCR UBTree支持闪回，所以页面可回收的时间点由OldestXmin改成了GlobalRecycleXid。

1.8.3.2.3 Undo

历史版本数据集中存放在$node_dir/undo目录中，其中$node_dir为数据库节点路径，回滚段日志是与单个写事务关联的所有撤销日志的集合。支持permanent/unlogged/temp三种表类型。

回滚段管理



1. 每个undo zone除了管理部分transaction page（用于存储事务回滚的元数据）外，还管理undo page。
2. Undo页面中存储undo row，对数据的修改会将历史版本记录到Undo中。
3. Undo记录也是数据，因此对Undo页面的修改同样会记录Redo。

文件组织结构

如需查询当前回滚段使用的存储方式是页式或段页式，可以查询系统表。当前仅支持页式。

示例：

```
gaussdb=# SELECT * FROM gs_global_config where name like '%undostoragetype%';
      name       | value
-----------------+---------
 undostoragetype | page
(1 row)
```

- 当回滚段使用的存储方式为页式：
  - txn page所在文件组织结构：
    ```
    $node_dir/undo/{permanent|unlogged|temp}/$undo_zone_id.meta.$segno
    ```
  - undo row所在文件组织结构：
    ```
    $node_dir/undo/{permanent|unlogged|temp}/$undo_zone_id.$segno
    ```


版权所有 © 华为云计算技术有限公司

空间管理

Undo子系统依赖后台回收线程进行空闲空间回收。负责主机上Undo模块的空间回收，备机通过回放xLog进行回收。回收线程遍历使用中的undo zone，对该zone中的txn page扫描，依据xid从小到大的顺序进行遍历。回收已提交或者已回滚完成的事务，且该事务的提交时间应早于$(current_time-undo_retention_time)。对于遍历过程中需要回滚的事务，后台回收线程会为该事务添加异步回滚任务。

当数据库中存在运行时间长、修改数据量大的事务，或者开启闪回时间较长的时候，可能出现undo空间持续膨胀的情况。当undo占用空间接近undo_space_limit_size时，就会触发强制回收。只要事务已提交或者已回滚完成，即使事务提交时间晚于$(current_time-undo_retention_time)，在这种情况下也可能被回收掉。

1.8.3.2.4 Enhanced Toast

概述

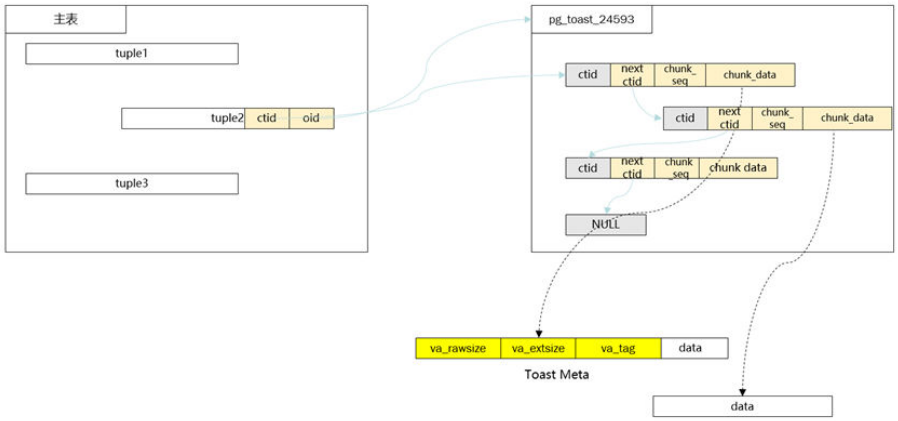
Enhanced Toast是一种用于处理超大字段的技术。首先，减少了Toast Pointer中的冗余信息，存储支持单表超长字段列数超过500列。其次，优化了主表与线外存储表之间的映射关系，无需通过pg_toast_index来存储主表数据与线外存储表数据的关系，降低了用户存储空间。最后，Enhanced Toast技术通过让分割数据自链接，消除了Oid分配的依赖，极大地加快了写入效率。

说明

- Astore存储引擎不支持Enhanced Toast。
- 不支持对Enhanced Toast类型的线外存储表单独进行Vacuum Full操作。

Enhanced Toast 存储结构

Enhanced Toast技术使用自链接的方式来处理元组间的依赖关系。线外存储表把超长数据按照2K分割成链表块，主表的Toast Pointer指向线外存储表的对应数据链表头。这样极大简化了主表与线外存储表间的映射关系，有效的提升了数据写入与查询的性能。



?.1. Enhanced Toast 使用

新增的GUC参数enable_enhance_toast_table用于控制线外存储结构。
“enable_enhance_toast_table=on”表示使用Enhanced Toast线外存储表，
“enable_enhance_toast_table=off”表示使用Toast线外存储表。

```
gs_guc reload -Z coordinator -Z datanode -N all -I all -c "enable_enhance_toast_table=on"
```


版权所有 © 华为云计算技术有限公司

Enhanced Toast 增删改查

Insert操作：触发Enhanced Toast的写入条件保持与原有Toast一致，除了数据写入时增加了数据间的链接信息之外，插入基本逻辑保持不变。

Delete操作：Enhanced Toast的数据删除流程不再依赖Toast数据索引，仅依靠数据间的链接信息将对应的数据进行遍历删除。

Update操作：Enhanced Toast的更新流程与原有Toast保持一致。

Enhanced Toast 相关 DDL 操作

Enhanced Toast表的创建

建表时指定Toast表的存储类型为Enhanced Toast或者Toast：

```
gaussdb=#  CREATE TABLE test_toast (id int, content text) with(toast.toast_storage_type=toast);
CREATE TABLE
gaussdb=#  \d+ test_toast
                    Table "public.test_toast"
 Column  |  Type   | Modifiers | Storage  | Stats target | Description
---------+---------+-----------+----------+--------------+-------------
 id      | integer |           | plain    |              |
 content | text    |           | extended |              |
Has OIDs: no
Distribute By: HASH(a)
Location Nodes: ALL DATANODES
Options: orientation=row, compression=no, storage_type=USTORE, segment=off,
toast.storage_type=USTORE, toast.toast_storage_type=toast
gaussdb=#  DROP TABLE test_toast;
DROP TABLE
gaussdb=#  CREATE TABLE test_toast (id int, content text) with(toast.toast_storage_type=enhanced_toast);
CREATE TABLE
gaussdb=#  \d+ test_toast
                    Table "public.test_toast"
 Column  |  Type   | Modifiers | Storage  | Stats target | Description
---------+---------+-----------+----------+--------------+-------------
 id      | integer |           | plain    |              |
 content | text    |           | extended |              |
Has OIDs: no
Distribute By: HASH(a)
Location Nodes: ALL DATANODES
Options: orientation=row, compression=no, storage_type=USTORE, segment=off,
toast.storage_type=USTORE, toast.toast_storage_type=enhanced_toast
gaussdb=#  DROP TABLE test_toast;
DROP TABLE
```

建表时不指定线外存储表的类型，则创建线外存储表类型依赖于GUC参数enable_enhance_toast_table：

```
-- 根据“Enhanced Toast使用”章节打开GUC
gaussdb=#  show enable_enhance_toast_table;
 enable_enhance_toast_table
----------------------------
 on
(1 row)
gaussdb=#  CREATE TABLE test_toast (id int, content text);
CREATE TABLE
gaussdb=#   \d+ test_toast
                    Table "public.test_toast"
 Column  |  Type   | Modifiers | Storage  | Stats target | Description
---------+---------+-----------+----------+--------------+-------------
 id      | integer |           | plain    |              |
 content | text    |           | extended |              |
Has OIDs: no
Distribute By: HASH(a)
Location Nodes: ALL DATANODES
```

 版权所有 © 华为云计算技术有限公司

```
Options: orientation=row, compression=no, storage_type=USTORE, segment=off,
toast.storage_type=USTORE, toast.toast_storage_type=enhanced_toast

gaussdb=#   DROP TABLE test_toast;
DROP TABLE
gaussdb=#  SET enable_enhance_toast_table = off;
SET
gaussdb=#  show enable_enhance_toast_table;
 enable_enhance_toast_table
----------------------------
 off
(1 row)
gaussdb=#   CREATE TABLE test_toast (id int, content text);
CREATE TABLE
gaussdb=#  \d+ test_toast
                    Table "public.test_toast"
 Column  |  Type   | Modifiers | Storage  | Stats target | Description
---------+---------+-----------+----------+--------------+-------------
 id      | integer |           | plain    |              |
 content | text    |           | extended |              |
Has OIDs: no
Distribute By: HASH(a)
Location Nodes: ALL DATANODES
Options: orientation=row, compression=no, storage_type=USTORE, segment=off,
toast.storage_type=USTORE, toast.toast_storage_type=toast
gaussdb=#  DROP TABLE test_toast;
DROP TABLE
```

线外存储表结构的升级

当GUC参数“enable_enhance_toast_table=on”时，线外存储表支持通过Vacuum Full操作将Toast升级为Enhanced Toast结构。

```
gaussdb=#   CREATE TABLE test_toast (id int, content text);
CREATE TABLE
gaussdb=#  \d+ test_toast
                    Table "public.test_toast"
 Column  |  Type   | Modifiers | Storage  | Stats target | Description
---------+---------+-----------+----------+--------------+-------------
 id      | integer |           | plain    |              |
 content | text    |           | extended |              |
Has OIDs: no
Distribute By: HASH(a)
Location Nodes: ALL DATANODES
Options: orientation=row, compression=no, storage_type=USTORE, segment=off,
toast.storage_type=USTORE, toast.toast_storage_type=toast
gaussdb=#  VACUUM FULL test_toast;
VACUUM
gaussdb=#  \d+ test_toast
                    Table "public.test_toast"
 Column  |  Type   | Modifiers | Storage  | Stats target | Description
---------+---------+-----------+----------+--------------+-------------
 id      | integer |           | plain    |              |
 content | text    |           | extended |              |
Has OIDs: no
Distribute By: HASH(a)
Location Nodes: ALL DATANODES
Options: orientation=row, compression=no, storage_type=USTORE, segment=off,
toast.storage_type=USTORE, toast.toast_storage_type=enhanced_toast
gaussdb=#  DROP TABLE test_toast;
DROP TABLE
```

分区表merge操作

支持将分区表的分区间不同的线外存储表类型进行合并操作。

 版权所有 © 华为云计算技术有限公司

说明

- 对于相同类型的线外存储分区，合并与原有逻辑保持一致，进行物理合并。
- 对于不同类型的线外存储分区，合并后的分区线外存储表为Enhanced Toast表，需要进行逻辑合并，性能劣于物理合并。

```
gaussdb=#  CREATE TABLE test_partition_table(a int, b text)PARTITION BY range(a)(partition p1 values less than (2000),partition p2 values less than (3000));
gaussdb=# SELECT relfilenode FROM pg_partition WHERE relname='p1';
 relfilenode
-------------
     17529
(1 row)

gaussdb=#  \d+ pg_toast.pg_toast_part_17529
TOAST table "pg_toast.pg_toast_part_17529"
   Column   |  Type   | Storage
------------+---------+---------
 chunk_id   | oid     | plain
 chunk_seq  | integer | plain
 chunk_data | bytea   | plain
Options: storage_type=ustore, toast_storage_type=toast

gaussdb=# SELECT relfilenode FROM pg_partition WHERE relname='p2';
 relfilenode
-------------
     17528
(1 row)

gaussdb=#  \d+ pg_toast.pg_toast_part_17528
TOAST table "pg_toast.pg_toast_part_17528"
   Column   |  Type   | Storage
------------+---------+---------
 chunk_seq  | integer | plain
 next_chunk | tid     | plain
 chunk_data | bytea   | plain
Options: storage_type=ustore, toast_storage_type=enhanced_toast
gaussdb=#  ALTER TABLE test_partition_table MERGE PARTITIONS p1,p2 INTO partition p1_p2;
ALTER TABLE
gaussdb=#  SELECT reltoastrelid::regclass FROM pg_partition where relname='p1_p2';
        reltoastrelid
------------------------------
 pg_toast.pg_toast_part_17559
(1 row)
gaussdb=#  \d+ pg_toast.pg_toast_part_17559
TOAST table "pg_toast.pg_toast_part_17559"
   Column   |  Type   | Storage
------------+---------+---------
 chunk_seq  | integer | plain
 next_chunk | tid     | plain
 chunk_data | bytea   | plain
Options: storage_type=ustore, toast_storage_type=enhanced_toast
gaussdb=# DROP TABLE test_partition_table;
DROP TABLE
```

Enhanced Toast 运维管理

通过gs_parse_page_bypath解析主表中的ToastPointer信息。

```
gaussdb=#  SELECT ctid,next_chunk,chunk_seq FROM pg_toast.pg_toast_part_17559;
 ctid  | next_chunk | chunk_seq
-------+------------+-----------
 (0,1) | (0,0)      |         1
 (0,2) | (0,1)      |         0
 (0,3) | (0,0)      |         1
 (0,4) | (0,3)      |         0
(4 rows)
gaussdb=#  SELECT gs_parse_page_bypath((SELECT * FROM
```

 版权所有 © 华为云计算技术有限公司

```
pg_relation_filepath('test_toast')),0,'uheap',false);
                        gs_parse_page_bypath
-------------------------------------------------------------------
 ${data_dir}/gs_log/dump/1663_13113_17603_0.page
(1 row)
```

解析文件1663_13113_17603_0.page中存储了ToastPointer的相关信息，具体如下：

```
Toast_Pointer:
    column_index: 1
        toast_relation_oid: 17608     --线外存储表OID信息
        ctid: (4, 1)                  --线外存储数据链，头指针
        bucket id: -1                  --bucket id信息
    column_index: 2
        toast_relation_oid: 17608
        ctid: (2, 1)
        bucket id: -1
```

Enhanced Toast数据查询，通过直接查询到的Enhanced Toast表数据可以判断其链式结构的完整性。

```
gaussdb=#  SELECT ctid,next_chunk,chunk_seq FROM pg_toast.pg_toast_part_17559;
 ctid  | next_chunk | chunk_seq
-------+------------+-----------
 (0,1) | (0,0)      |         1
 (0,2) | (0,1)      |         0
 (0,3) | (0,0)      |         1
 (0,4) | (0,3)      |         0
(4 rows)
```

1.8.3.3 Ustore 事务模型

GaussDB事务基础：

1. 事务启动时不会自动分配XID，该事务中的第一条DML/DDL语句运行时才会真正为该事务分配XID。
2. 事务结束时，会产生代表事务提交状态的CLOG（Commit Log），CLOG共有四种状态：事务运行中、事务提交、事务同步回滚、子事务提交。每个事务的 CLOG 状态位为2 bits，CLOG页面上每个字节可以表示四个事务的提交状态。
3. 事务结束时，还会产生代表事务提交顺序的CSN（Commit sequence number）。CSN为实例级变量，每个XID都有自己对应的唯一CSN。CSN可以标记事务的以下状态：事务提交中、事务提交、事务回滚、事务已冻结等。

1.8.3.3.1 事务提交

针对隐式事务和显式事务，其提交策略如下所示：

1. 隐式事务。单条DML/DDL语句自动触发隐式事务，这种事务没有显式的事务块控制语句（START TRANSACTION/BEGIN/COMMIT/END），DML语句结束后自动提交。
2. 显式事务。显式事务由显式的START TRANSACTION/BEGIN语句控制事务的开始，由COMMIT/END语句控制事务的提交。

   子事务必须存在于显式事务或存储过程中，由SAVEPOINT语句控制子事务开始，由RELEASE SAVEPOINT语句控制子事务结束。如果一个事务在提交时还存在未释放的子事务，该事务提交前会先执行子事务的提交，所有子事务提交完毕后才会进行父事务的提交。

   Ustore支持读已提交隔离级别。语句在执行开始时，获取当前系统的CSN作为当前语句的查询CSN。整个语句的可见结果由语句开始那一刻决定，不受后续其他事务修改影响。Ustore中read committed默认是保持一致性读的。Ustore也支持标准的2PC事务。

 版权所有 © 华为云计算技术有限公司

1.8.3.3.2 事务回滚

回滚是在事务运行的过程中发生了故障等异常情形下，事务不能继续执行，系统需要将事务中已完成的修改操作进行撤销。Astore、Ubtree没有回滚段，自然没有这个专门的回滚动作。Ustore为了性能考虑，它的回滚流程结合了同步、异步和页面级回滚等3种形式。

- **同步回滚**

  有三种情况会触发事务的同步回滚：

  - 事务块中的ROLLBACK关键字会触发同步回滚。
  - 事务运行过程中如果发生ERROR级别报错，此时的COMMIT关键字与ROLLBACK功能相同，也会触发同步回滚。
  - 事务运行过程中如果发生FATAL/PANIC级别报错，在线程退出前会尝试将该线程绑定的事务进行一次同步回滚。

- **异步回滚**

  同步回滚失败或者在系统宕机后再次重启时，会由Undo回收线程为未回滚完成的事务发起异步回滚任务，立即对外提供服务。由异步回滚任务发起线程undo launch负责拉起异步回滚工作线程undo worker，再由异步回滚工作线程实际执行回滚任务。undo launch线程最多可以同时拉起5个undo worker线程。

- **页面级回滚**

  当事务需要回滚但还未回滚到本页面时，如果其他事务需要复用该事务所占用的TD，就会在复用前对该事务在本页面的所有修改执行页面级回滚。页面级回滚只负责回滚事务在本页面的修改，不涉及其他页面。

  Ustore子事务的回滚由ROLLBACK TO SAVEPOINT语句控制，子事务回滚后父事务可以继续运行，子事务的回滚不影响父事务的事务状态。如果一个事务在回滚时还存在未释放的子事务，该事务回滚前会先执行子事务的回滚，所有子事务回滚完毕后才会进行父事务的回滚。

1.8.3.4 闪回恢复

闪回恢复功能是数据库恢复技术的一环，可以有选择性地撤销一个已提交事务的影响，将数据从人为不正确的操作中进行恢复。在采用闪回技术之前，只能通过备份恢复、PITR等手段找回已提交的数据库修改，恢复时长需要数分钟甚至数小时。采用闪回技术后，通过闪回Drop和闪回Truncate恢复已提交的数据库Drop/Truncate的数据，只需要秒级，而且恢复时间和数据库大小无关。

说明

- ASTORE引擎只支持闪回DROP/TRUNCATE功能。
- 备机不支持闪回操作。
- 用户可以根据需要开启闪回功能，开启后会带来一定的性能劣化。

1.8.3.4.1 闪回查询

背景信息

闪回查询可以查询过去某个时间点表的某个snapshot数据，这一特性可用于查看和逻辑重建意外删除或更改的受损数据。闪回查询基于MVCC多版本机制，通过检索查询旧版本，获取指定老版本数据。

版权所有 © 华为云计算技术有限公司

前提条件

整体方案分为三部分：旧版本保留、快照的维护和旧版本检索。旧版本保留：新增undo_retention_time配置参数，用来设置旧版本保留的时间，超过该时间的旧版本将被回收清理，若使用闪回查询则需将该参数设置为大于0的值，请联系管理员修改。

语法

```
{[ ONLY ] table_name [ * ] [ partition_clause ] [ [ AS ] alias [ ( column_alias [, ...] ) ] ]
[ TABLESAMPLE sampling_method ( argument [, ...] ) [ REPEATABLE ( seed ) ] ]
[TIMECAPSULE { TIMESTAMP | CSN } expression ]
|( select ) [ AS ] alias [ ( column_alias [, ...] ) ]
|with_query_name [ [ AS ] alias [ ( column_alias [, ...] ) ] ]
|function_name ( [ argument [, ...] ] ) [ AS ] alias [ ( column_alias [, ...] | column_definition [, ...] ) ]
|function_name ( [ argument [, ...] ] ) AS ( column_definition [, ...] )
|from_item [ NATURAL ] join_type from_item [ ON join_condition | USING ( join_column [, ...] ) ]}
```

语法树中“TIMECAPSULE {TIMESTAMP | CSN} expression”为闪回功能新增表达方式，其中TIMECAPSULE表示使用闪回功能，TIMESTAMP以及CSN表示闪回功能使用具体时间点信息或使用CSN（commit sequence number）信息。

参数说明

- TIMESTAMP
  - 指要查询某个表在TIMESTAMP这个时间点上的数据，TIMESTAMP指一个具体的历史时间。
- CSN
  - 指要查询整个数据库逻辑提交序下某个CSN点的数据，CSN指一个具体逻辑提交时间点，数据库中的CSN为写一致性点，每个CSN代表整个数据库的一个一致性点，查询某个CSN下的数据代表SQL查询数据库在该一致性点的相关数据。

备注：使用时间点进行闪回时，可能会有3s的误差。想要闪回到精确的操作点，需要使用CSN进行闪回。GTM-Free模式下没有全局一致性csn点，暂时不支持以csn的方式进行闪回。

使用示例

- 示例（需将undo_retention_time参数设置为大于0的值）：

```
gaussdb=# DROP TABLE IF EXISTS "public".flashtest;
NOTICE:  table "flashtest" does not exist, skipping
DROP TABLE
--创建表flashtest。
gaussdb=# CREATE TABLE "public".flashtest (col1 INT,col2 TEXT) with(storage_type=ustore);
NOTICE:  The 'DISTRIBUTE BY' clause is not specified. Using 'col1' as the distribution column by
default.
HINT:  Please use 'DISTRIBUTE BY' clause to specify suitable data distribution column.
CREATE TABLE
--查询csn。
gaussdb=# SELECT int8in(xidout(next_csn)) FROM gs_get_next_xid_csn();
  int8in
----------
 79351682
 79351682
 79351682
 79351682
 79351682
 79351682
(6 rows)
--查询当前时间戳。
gaussdb=# SELECT now();
```

 版权所有 © 华为云计算技术有限公司

```
              now
-------------------------------
 2023-09-13 19:35:26.011986+08
(1 row)
--插入数据。
gaussdb=# INSERT INTO flashtest VALUES(1,'INSERT1'),(2,'INSERT2'),(3,'INSERT3'),(4,'INSERT4'),
(5,'INSERT5'),(6,'INSERT6');
INSERT 0 6
gaussdb=# SELECT * FROM flashtest;
 col1 |  col2
------+---------
    3 | INSERT3
    1 | INSERT1
    2 | INSERT2
    4 | INSERT4
    5 | INSERT5
    6 | INSERT6
(6 rows)
--闪回查询某个csn处的表。
gaussdb=# SELECT * FROM flashtest TIMECAPSULE CSN 79351682;
 col1 | col2
------+------
(0 rows)
gaussdb=# SELECT * FROM flashtest;
 col1 |  col2
------+---------
    1 | INSERT1
    2 | INSERT2
    4 | INSERT4
    5 | INSERT5
    3 | INSERT3
    6 | INSERT6
(6 rows)
--闪回查询某个时间戳处的表。
gaussdb=# SELECT * FROM flashtest TIMECAPSULE TIMESTAMP '2023-09-13 19:35:26.011986';
 col1 | col2
------+------
(0 rows)
gaussdb=# SELECT * FROM flashtest;
 col1 |  col2
------+---------
    1 | INSERT1
    2 | INSERT2
    4 | INSERT4
    5 | INSERT5
    3 | INSERT3
    6 | INSERT6
(6 rows)
--闪回查询某个时间戳处的表。
gaussdb=# SELECT * FROM flashtest TIMECAPSULE TIMESTAMP to_timestamp ('2023-09-13
19:35:26.011986', 'YYYY-MM-DD HH24:MI:SS.FF');
 col1 | col2
------+------
(0 rows)
--闪回查询某个csn处的表，并对表进行重命名。
gaussdb=# SELECT * FROM flashtest AS ft TIMECAPSULE CSN 79351682;
 col1 | col2
------+------
(0 rows)
gaussdb=# DROP TABLE IF EXISTS "public".flashtest;
DROP TABLE
```

1.8.3.4.2 闪回表

背景信息

闪回表可以将表恢复至特定时间点，当逻辑损坏仅限于一个或一组表，而不是整个数据库时，此特性可以快速恢复表的数据。闪回表基于MVCC多版本机制，通过删除指定

 版权所有 © 华为云计算技术有限公司

时间点和该时间点之后的增量数据，并找回指定时间点和当前时间点删除的数据，实现表级数据还原。

前提条件

整体方案分为三部分：旧版本保留、快照的维护和旧版本检索。旧版本保留：新增undo_retention_time配置参数，用来设置旧版本保留的时间，超过该时间的旧版本将被回收清理，请联系管理员修改。

语法

```
TIMECAPSULE TABLE table_name TO { TIMESTAMP | CSN } expression
```

使用示例

```
gaussdb=# DROP TABLE IF EXISTS "public".flashtest;
NOTICE:  table "flashtest" does not exist, skipping
DROP TABLE
--创建表
gaussdb=# CREATE TABLE "public".flashtest (col1 INT,col2 TEXT) with(storage_type=ustore);
NOTICE:  The 'DISTRIBUTE BY' clause is not specified. Using 'col1' as the distribution column by default.
HINT:  Please use 'DISTRIBUTE BY' clause to specify suitable data distribution column.
CREATE TABLE
--查询csn
gaussdb=# SELECT int8in(xidout(next_csn)) FROM gs_get_next_xid_csn();
  int8in
----------
 79352065
 79352065
 79352065
 79352065
 79352065
 79352065
(6 rows)
--查询当前的时间戳
gaussdb=# SELECT now();
              now
-------------------------------
 2023-09-13 19:46:34.102863+08
(1 row)
--查看表flashtest
gaussdb=# SELECT * FROM flashtest;
 col1 | col2
------+------
(0 rows)
--插入数据
gaussdb=# INSERT INTO flashtest VALUES(1,'INSERT1'),(2,'INSERT2'),(3,'INSERT3'),(4,'INSERT4'),
(5,'INSERT5'),(6,'INSERT6');
INSERT 0 6
gaussdb=# SELECT * FROM flashtest;
 col1 |  col2
------+---------
    3 | INSERT3
    1 | INSERT1
    2 | INSERT2
    4 | INSERT4
    5 | INSERT5
    6 | INSERT6
(6 rows)
--闪回表至特定csn
gaussdb=# TIMECAPSULE TABLE flashtest TO CSN 79352065;
TimeCapsule Table
gaussdb=# SELECT * FROM flashtest;
 col1 | col2
------+------
(0 rows)
```

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# SELECT now();
              now
-------------------------------
 2023-09-13 19:52:21.551028+08
(1 row)
--插入数据
gaussdb=# INSERT INTO flashtest VALUES(1,'INSERT1'),(2,'INSERT2'),(3,'INSERT3'),(4,'INSERT4'),
(5,'INSERT5'),(6,'INSERT6');
INSERT 0 6
gaussdb=# SELECT * FROM flashtest;
 col1 |  col2
------+---------
    3 | INSERT3
    6 | INSERT6
    1 | INSERT1
    2 | INSERT2
    4 | INSERT4
    5 | INSERT5
(6 rows)
--闪回表至特定的时间戳
gaussdb=# TIMECAPSULE TABLE flashtest TO TIMESTAMP to_timestamp ('2023-09-13 19:52:21.551028',
'YYYY-MM-DD HH24:MI:SS.FF');
TimeCapsule Table
gaussdb=# SELECT * FROM flashtest;
 col1 | col2
------+------
(0 rows)
gaussdb=# select now();
              now
-------------------------------
 2023-09-13 19:54:00.641506+08
(1 row)
--插入数据
gaussdb=# INSERT INTO flashtest VALUES(1,'INSERT1'),(2,'INSERT2'),(3,'INSERT3'),(4,'INSERT4'),
(5,'INSERT5'),(6,'INSERT6');
INSERT 0 6
gaussdb=# SELECT * FROM flashtest;
 col1 |  col2
------+---------
    3 | INSERT3
    6 | INSERT6
    1 | INSERT1
    2 | INSERT2
    4 | INSERT4
    5 | INSERT5
(6 rows)
--闪回表至特定的时间戳
gaussdb=# TIMECAPSULE TABLE flashtest TO TIMESTAMP '2023-09-13 19:54:00.641506';
TimeCapsule Table
gaussdb=# SELECT * FROM flashtest;
 col1 | col2
------+------
(0 rows)
gaussdb=# DROP TABLE IF EXISTS "public".flashtest;
DROP TABLE
```

1.8.3.4.3 闪回 DROP/TRUNCATE

背景信息

- 闪回DROP：可以恢复意外删除的表，从回收站（recyclebin）中恢复被删除的表及其附属结构如索引、表约束等。闪回drop是基于回收站机制，通过还原回收站中记录的表的物理文件，实现已drop表的恢复。
- 闪回TRUNCATE：可以恢复误操作或意外被进行truncate的表，从回收站中恢复被truncate的表及索引的物理数据。闪回truncate基于回收站机制，通过还原回收站中记录的表的物理文件，实现已truncate表的恢复。

 版权所有 © 华为云计算技术有限公司

前提条件

- 开启enable_recyclebin参数（GUC参数在gaussdb.conf文件修改），启用回收站，请联系管理员修改。
- recyclebin_retention_time参数用于设置回收站对象保留时间，超过该时间的回收站对象将被自动清理，请联系管理员修改。

相关语法

- 删除表
```
DROP TABLE table_name [PURGE]
```
- 清理回收站对象
```
PURGE { TABLE { table_name }
| INDEX { index_name }
| RECYCLEBIN
}
```
- 闪回被删除的表
```
TIMECAPSULE TABLE { table_name } TO BEFORE DROP [RENAME TO new_tablename]
```
- 截断表
```
TRUNCATE TABLE { table_name } [ PURGE ]
```
- 闪回截断的表
```
TIMECAPSULE TABLE { table_name } TO BEFORE TRUNCATE
```

参数说明

- **DROP/TRUNCATE TABLE table_name PURGE**

  默认将表数据放入回收站中，PURGE直接清理。
- **PURGE RECYCLEBIN**

  表示清理回收站对象。
- **TO BEFORE DROP**

  使用这个子句检索回收站中已删除的表及其子对象。

  可以指定原始用户指定的表的名称，或对象删除时数据库分配的系统生成名称。
  - 回收站中系统生成的对象名称是唯一的。因此，如果指定系统生成名称，那么数据库检索指定的对象。使用“select * from gs_recyclebin;”语句查看回收站中的内容。
  - 在指定了用户指定的名称且回收站中包含多个该名称的对象的情况下，数据库检索回收站中最近移动的对象，如果想要检索更早版本的表，可以执行如下操作：
    - 指定你想要检索的表的系统生成名称。
    - 执行TIMECAPSULE TABLE ... TO BEFORE DROP语句，直到你要检索的表。
  - 恢复DROP表时，只恢复基表名，其他子对象名均保持回收站对象名。用户可根据需要，执行DDL命令手工调整子对象名。
  - 回收站对象不支持DML、DCL、DDL等写操作，不支持DQL查询操作（后续支持）。
  - 闪回点和当前点之间，执行过修改表结构或影响物理结构的语句，闪回失败。执行过DDL的表进行闪回操作报错：“ERROR：The table definition of %s has been changed.”。涉及namespace、表名改变等操作的DDL执行闪回操作报错： ERROR: recycle object %s desired does not exis；

 版权所有 © 华为云计算技术有限公司

  - 如果基表有truncate trigger、truncate表时，无法触发trigger，不能同时truncate目标表。用户想要truncate目标表，需要手动操作。

- **RENAME TO**

  为从回收站中检索的表指定一个新名称。

- **TO BEFORE TRUNCATE**

  闪回到TRUNCATE之前。

语法示例

```
-- PURGE TABLE table_name; --
--查看回收站
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid | rcyname | rcyoriginname | rcyoperation | rcytype | rcyrecyclecsn |
rcyrecycletime | rcycreatecsn | rcychangecsn | rcynamespace | rcyowner | rcytablespace
 | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+---------+---------------+--------------+---------+---------------+----------------
+--------------+--------------+--------------+----------+--------------
-+----------------+---------------+-------------+--------------+----------------+-----------
(0 rows)

gaussdb=# DROP TABLE IF EXISTS flashtest;
NOTICE:  table "flashtest" does not exist, skipping
DROP TABLE
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid | rcyname | rcyoriginname | rcyoperation | rcytype | rcyrecyclecsn |
rcyrecycletime | rcycreatecsn | rcychangecsn | rcynamespace | rcyowner | rcytablespace
 | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+---------+---------------+--------------+---------+---------------+----------------
+--------------+--------------+--------------+----------+--------------
-+----------------+---------------+-------------+--------------+----------------+-----------
(0 rows)
--创建表flashtest
gaussdb=# CREATE TABLE IF NOT EXISTS flashtest(id int, name text) with (storage_type = ustore);
NOTICE:  The 'DISTRIBUTE BY' clause is not specified. Using 'id' as the distribution column by default.
HINT:  Please use 'DISTRIBUTE BY' clause to specify suitable data distribution column.
CREATE TABLE
--插入数据
gaussdb=# INSERT INTO flashtest VALUES(1, 'A');
INSERT 0 1
gaussdb=# SELECT * FROM flashtest;
 id | name
----+------
  1 | A
(1 row)
--DROP表flashtest
gaussdb=# DROP TABLE IF EXISTS flashtest;
DROP TABLE
--查看回收站，删除的表被放入回收站
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid |          rcyname           |    rcyoriginname     | rcyoperation | rcytype |
rcyrecyclecsn |        rcyrecycletime         | rcycreatecsn | rcychangecs
n | rcynamespace | rcyowner | rcytablespace | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid |
rcyfrozenxid64 | rcybucket
-----------+---------+----------+----------------------------+----------------------+--------------+---------
+---------------+--------------------------------+--------------+------------
--+--------------+----------+---------------+----------------+---------------+-------------+--------------
+----------------+-----------
    18591 |   12737 |    18585 | BIN$31C14EB4899$9737$0==$0   | flashtest            | d            |       0 |
79352606 | 2023-09-13 20:01:28.640664+08 |     79352595 |     7935259
5 |         2200 |       10 |            0 |          18585 | t            | t           | 225492       |        225492 |
    18591 |   12737 |    18588 | BIN$31C14EB489C$12D1BF60==$0 | pg_toast_18585       | d            |       2
|     79352606 | 2023-09-13 20:01:28.641018+08 |            0 |
0 |          99 |       10 |            0 |          18588 | f            | f           | 225492       |        225492 |
(2 rows)
--查看表flashtest，表不存在
gaussdb=# SELECT * FROM flashtest;
```

 版权所有 © 华为云计算技术有限公司

```
ERROR:  relation "flashtest" does not exist
LINE 1: SELECT * FROM flashtest;
                      ^
--PURGE表，将回收站中的表删除
gaussdb=# PURGE TABLE flashtest;
PURGE TABLE
--查看回收站，回收站中的表被删除
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid | rcyname | rcyoriginname | rcyoperation | rcytype | rcyrecyclecsn |
rcyrecycletime | rcycreatecsn | rcychangecsn | rcynamespace | rcyowner | rcytablespace
 | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+---------+---------------+--------------+---------+---------------+----------------
+--------------+--------------+--------------+----------+--------------
-+----------------+--------------+-------------+--------------+----------------+-----------
(0 rows)


-- PURGE INDEX index_name; --
gaussdb=# DROP TABLE IF EXISTS flashtest;
NOTICE:  table "flashtest" does not exist, skipping
DROP TABLE
--创建表flashtest
gaussdb=# CREATE TABLE IF NOT EXISTS flashtest(id int, name text) WITH (storage_type = ustore);
NOTICE:  The 'DISTRIBUTE BY' clause is not specified. Using 'id' as the distribution column by default.
HINT:  Please use 'DISTRIBUTE BY' clause to specify suitable data distribution column.
CREATE TABLE
--为表flashtest创建索引flashtest_index
gaussdb=# CREATE INDEX flashtest_index ON flashtest(id);
CREATE INDEX
--查看flashtest表的基本信息
gaussdb=# \d+ flashtest
                    Table "public.flashtest"
 Column |  Type   | Modifiers | Storage  | Stats target | Description
--------+---------+-----------+----------+--------------+-------------
 id     | integer |           | plain    |              |
 name   | text    |           | extended |              |
Indexes:
    "flashtest_index" ubtree (id) WITH (storage_type=USTORE) TABLESPACE pg_default
Has OIDs: no
Distribute By: HASH(id)
Location Nodes: ALL DATANODES
Options: orientation=row, storage_type=ustore, compression=no, segment=off,toast.storage_type=ustore,
toast.toast_storage_type=enhanced_toast

--DROP表
gaussdb=# DROP TABLE IF EXISTS flashtest;
DROP TABLE
--查看回收站
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid |           rcyname            |     rcyoriginname      | rcyoperation | rcytype |
rcyrecyclecsn |         rcyrecycletime          | rcycreatecsn | rcychangecs
n | rcynamespace | rcyowner | rcytablespace | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid |
rcyfrozenxid64 | rcybucket
-----------+---------+----------+------------------------------+----------------------+--------------+---------
+--------------+------------------------------+--------------+------------
--+--------------+----------+---------------+----------------+---------------+-------------+--------------
+----------------+-----------
     18648 |   12737 |    18641 | BIN$31C14EB48D1$9A85$0==$0   | flashtest            | d            |       0 |
79354509 | 2023-09-13 20:40:11.360638+08 |     79354506 |     7935450
8 |         2200 |       10 |             0 |          18641 | t            | t           | 226642       |         226642 |
     18648 |   12737 |    18644 | BIN$31C14EB48D4$12E236A0==$0 | pg_toast_18641       | d            |       2
|      79354509 | 2023-09-13 20:40:11.36112+08  |             0 |
0 |          99 |       10 |             0 |          18644 | f            | f           | 226642       |         226642 |
     18648 |   12737 |    18647 | BIN$31C14EB48D7$9A85$0==$0   | flashtest_index      | d            |       1 |
79354509 | 2023-09-13 20:40:11.361246+08 |     79354508 |     7935450
8 |         2200 |       10 |             0 |          18647 | f            | t           | 0            |             0 |
(3 rows)

--PURGE索引flashtest_index
```

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# PURGE INDEX flashtest_index;
PURGE INDEX
--查看回收站，回收站中的索引flashtest_index被删除
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid |           rcyname            |     rcyoriginname     | rcyoperation | rcytype |
rcyrecyclecsn |        rcyrecycletime         | rcycreatecsn | rcychangecs
n | rcynamespace | rcyowner | rcytablespace | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid |
rcyfrozenxid64 | rcybucket
-----------+---------+----------+------------------------------+----------------------+--------------+---------
+---------------+-------------------------------+--------------+------------
--+--------------+----------+---------------+----------------+---------------+-------------+--------------
+----------------+-----------
    18648 |   12737 |    18641 | BIN$31C14EB48D1$9A85$0==$0   | flashtest            | d            |       0 |
79354509 | 2023-09-13 20:40:11.360638+08 |     79354506 |     7935450
8 |         2200 |       10 |             0 |          18641 | t            | t           | 226642       |        226642 |
    18648 |   12737 |    18644 | BIN$31C14EB48D4$12E236A0==$0 | pg_toast_18641       | d            |       2
|     79354509 | 2023-09-13 20:40:11.36112+08  |             0 |
0 |          99 |       10 |             0 |          18644 | f            | f           | 226642       |        226642 |
(2 rows)

-- PURGE RECYCLEBIN --
--PURGE回收站
gaussdb=# PURGE RECYCLEBIN;
PURGE RECYCLEBIN
--查看回收站，回收站被清空
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid | rcyname | rcyoriginname | rcyoperation | rcytype | rcyrecyclecsn |
rcyrecycletime | rcycreatecsn | rcychangecsn | rcynamespace | rcyowner | rcytablespace
 | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+---------+---------------+--------------+---------+---------------+----------------
+--------------+--------------+--------------+----------+--------------
-+----------------+---------------+-------------+--------------+----------------+-----------
(0 rows)

-- TIMECAPSULE TABLE { table_name } TO BEFORE DROP [RENAME TO new_tablename] --
gaussdb=# DROP TABLE IF EXISTS flashtest;
NOTICE:  table "flashtest" does not exist, skipping
DROP TABLE
--创建表flashtest
gaussdb=# CREATE TABLE IF NOT EXISTS flashtest(id int, name text) with (storage_type = ustore);
NOTICE:  The 'DISTRIBUTE BY' clause is not specified. Using 'id' as the distribution column by default.
HINT:  Please use 'DISTRIBUTE BY' clause to specify suitable data distribution column.
CREATE TABLE
--插入数据
gaussdb=# INSERT INTO flashtest VALUES(1, 'A');
INSERT 0 1
gaussdb=# SELECT * FROM flashtest;
 id | name
----+------
  1 | A
(1 row)

--DROP表
gaussdb=# DROP TABLE IF EXISTS flashtest;
DROP TABLE
--查看回收站，表被放入回收站
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid |           rcyname            |     rcyoriginname     | rcyoperation | rcytype |
rcyrecyclecsn |        rcyrecycletime         | rcycreatecsn | rcychangecs
n | rcynamespace | rcyowner | rcytablespace | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid |
rcyfrozenxid64 | rcybucket
-----------+---------+----------+------------------------------+----------------------+--------------+---------
+---------------+-------------------------------+--------------+------------
--+--------------+----------+---------------+----------------+---------------+-------------+--------------
+----------------+-----------
    18658 |   12737 |    18652 | BIN$31C14EB48DC$9B2B$0==$0   | flashtest            | d            |       0 |
79354760 | 2023-09-13 20:47:57.075907+08 |     79354753 |     7935475
3 |         2200 |       10 |             0 |          18652 | t            | t           | 226824       |        226824 |
    18658 |   12737 |    18655 | BIN$31C14EB48DF$12E46400==$0 | pg_toast_18652       | d            |       2
```

 版权所有 © 华为云计算技术有限公司

```
|      79354760 | 2023-09-13 20:47:57.07621+08  |             0 |
0 |           99 |       10 |             0 |          18655 | f             | f           | 226824       |         226824 |
(2 rows)

--查看表，表不存在
gaussdb=# SELECT * FROM flashtest;
ERROR:  relation "flashtest" does not exist
LINE 1: select * from flashtest;
                      ^
--闪回drop表
gaussdb=# TIMECAPSULE TABLE flashtest to before drop;
TimeCapsule Table
--查看表，表被恢复到drop之前
gaussdb=# SELECT * FROM flashtest;
 id | name
----+------
  1 | A
(1 row)

--查看回收站，回收站中的表被删除
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid | rcyname | rcyoriginname | rcyoperation | rcytype | rcyrecyclecsn |
rcyrecycletime | rcycreatecsn | rcychangecsn | rcynamespace | rcyowner | rcytablespace
 | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+---------+---------------+--------------+---------+---------------+----------------
+--------------+--------------+--------------+----------+--------------
-+----------------+---------------+-------------+--------------+----------------+-----------
(0 rows)

--DROP表
gaussdb=# DROP TABLE IF EXISTS flashtest;
DROP TABLE
gaussdb=# SELECT * FROM flashtest;
ERROR:  relation "flashtest" does not exist
LINE 1: SELECT * FROM flashtest;
                      ^
--查看回收站，表被放入回收站
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid |           rcyname            |       rcyoriginname         | rcyoperation | rcytype |
rcyrecyclecsn |        rcyrecycletime         | rcycreatecsn | rcy
changecsn | rcynamespace | rcyowner | rcytablespace | rcyrelfilenode | rcycanrestore | rcycanpurge |
rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+------------------------------+-----------------------------+--------------+---------
+---------------+------------------------------+--------------+----
----------+--------------+----------+---------------+----------------+---------------+-------------+--------------
+----------------+-----------
    18664 |   12737 |    18652 | BIN$31C14EB48DC$9B4E$0==$0   | flashtest                   | d            |       0
|      79354845 | 2023-09-13 20:49:17.762977+08 |      79354753 |
 79354753 |         2200 |       10 |             0 |          18652 | t            | t           | 226824       |         226824 |
    18664 |   12737 |    18657 | BIN$31C14EB48E1$12E680A8==$0 | BIN$31C14EB48E1$12E45E00==$0 |
d            |        3 |      79354845 | 2023-09-13 20:49:17.763271+08 |      79354753 |
 79354753 |           99 |       10 |             0 |          18657 | f            | f           | 0            |              0 |
    18664 |   12737 |    18655 | BIN$31C14EB48DF$12E68698==$0 | BIN$31C14EB48DF$12E46400==$0 |
d            |        2 |      79354845 | 2023-09-13 20:49:17.763343+08 |             0 |
       0 |           99 |       10 |             0 |          18655 | f            | f           | 226824       |         226824 |
(3 rows)

--闪回drop表，表名用回收站中的rcyname
gaussdb=# TIMECAPSULE TABLE "BIN$31C14EB48DC$9B4E$0==$0" to before drop;
TimeCapsule Table
----查看回收站，回收站中的表被删除
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid | rcyname | rcyoriginname | rcyoperation | rcytype | rcyrecyclecsn |
rcyrecycletime | rcycreatecsn | rcychangecsn | rcynamespace | rcyowner | rcytablespace
 | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+---------+---------------+--------------+---------+---------------+----------------
+--------------+--------------+--------------+----------+--------------
-+----------------+---------------+-------------+--------------+----------------+-----------
(0 rows)
```

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# SELECT * FROM flashtest;
 id | name
----+------
  1 | A
(1 row)

--DROP表
gaussdb=# DROP TABLE IF EXISTS flashtest;
DROP TABLE
----查看回收站，表被放入回收站
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid |           rcyname            |       rcyoriginname         | rcyoperation | rcytype |
rcyrecyclecsn |        rcyrecycletime         | rcycreatecsn | rcy
changecsn | rcynamespace | rcyowner | rcytablespace | rcyrelfilenode | rcycanrestore | rcycanpurge |
rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+------------------------------+-----------------------------+--------------+---------
+---------------+-------------------------------+--------------+----
----------+--------------+----------+---------------+---------------+--------------+-------------+--------------
+----------------+-----------
    18667 |   12737 |    18652 | BIN$31C14EB48DC$9B8D$0==$0   | flashtest                   | d            |       0
|      79354943 | 2023-09-13 20:52:14.525946+08 |     79354753 |
 79354753 |         2200 |       10 |             0 |          18652 | t            | t           | 226824       |         226824 |
    18667 |   12737 |    18657 | BIN$31C14EB48E1$1320B4F0==$0 | BIN$31C14EB48E1$12E680A8==$0 |
d            |       3 |      79354943 | 2023-09-13 20:52:14.526319+08 |     79354753 |
 79354753 |          99 |       10 |             0 |          18657 | f            | f           | 0            |             0 |
    18667 |   12737 |    18655 | BIN$31C14EB48DF$1320BAE0==$0 | BIN$31C14EB48DF$12E68698==$0 |
d            |       2 |      79354943 | 2023-09-13 20:52:14.526423+08 |            0 |
       0 |          99 |       10 |             0 |          18655 | f            | f           | 226824       |         226824 |
(3 rows)

--查看表，表不存在
gaussdb=# SELECT * FROM flashtest;
ERROR:  relation "flashtest" does not exist
LINE 1: SELECT * FROM flashtest;
                      ^
--闪回drop表，并重命名表
gaussdb=# TIMECAPSULE TABLE flashtest to before drop rename to flashtest_rename;
TimeCapsule Table
--查看原表，表不存在
gaussdb=# SELECT * FROM flashtest;
ERROR:  relation "flashtest" does not exist
LINE 1: SELECT * FROM flashtest;
                      ^
--查看重命名后的表，表存在
gaussdb=# SELECT * FROM flashtest_rename;
 id | name
----+------
  1 | A
(1 row)

--查看回收站，回收站中的表被删除
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid | rcyname | rcyoriginname | rcyoperation | rcytype | rcyrecyclecsn |
rcyrecycletime | rcycreatecsn | rcychangecsn | rcynamespace | rcyowner | rcytablespace
 | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+---------+---------------+--------------+---------+---------------+----------------
+--------------+--------------+--------------+----------+--------------
-+----------------+---------------+-------------+--------------+----------------+-----------
(0 rows)
--drop表
gaussdb=# DROP TABLE IF EXISTS flashtest_rename;
DROP TABLE
--清空回收站
gaussdb=# PURGE RECYCLEBIN;
PURGE RECYCLEBIN
--查看回收站，回收站被清空
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid | rcyname | rcyoriginname | rcyoperation | rcytype | rcyrecyclecsn |
```

 版权所有 © 华为云计算技术有限公司

```
rcyrecycletime | rcycreatecsn | rcychangecsn | rcynamespace | rcyowner | rcytablespace
 | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+---------+--------------+-------------+---------+--------------+----------------
+-------------+-------------+-------------+----------+-------------
-+----------------+--------------+-------------+-------------+----------------+-----------
(0 rows)


-- TIMECAPSULE TABLE { table_name } TO BEFORE TRUNCATE --
gaussdb=# DROP TABLE IF EXISTS flashtest;
NOTICE:  table "flashtest" does not exist, skipping
DROP TABLE
--创建表flashtest
gaussdb=# CREATE TABLE IF NOT EXISTS flashtest(id int, name text) WITH (storage_type = ustore);
NOTICE:  The 'DISTRIBUTE BY' clause is not specified. Using 'id' as the distribution column by default.
HINT:  Please use 'DISTRIBUTE BY' clause to specify suitable data distribution column.
CREATE TABLE
--插入数据
gaussdb=# INSERT INTO flashtest VALUES(1, 'A');
INSERT 0 1
gaussdb=# SELECT * FROM flashtest;
 id | name
----+------
  1 | A
(1 row)

--truncate表
gaussdb=# TRUNCATE TABLE flashtest;
TRUNCATE TABLE
--查看回收站，表的数据被放入回收站
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid |           rcyname            |     rcyoriginname     | rcyoperation | rcytype |
rcyrecyclecsn |        rcyrecycletime         | rcycreatecsn | rcychangecs
n | rcynamespace | rcyowner | rcytablespace | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid |
rcyfrozenxid64 | rcybucket
-----------+---------+----------+------------------------------+----------------------+--------------+---------
+---------------+------------------------------+--------------+------------
--+--------------+----------+---------------+----------------+---------------+-------------+--------------
+----------------+-----------
    18703 |   12737 |    18697 | BIN$31C14EB4909$9E4C$0==$0   | flashtest            | t            |       0 |
79356608 | 2023-09-13 21:24:42.819863+08 |     79356606 |     7935660
6 |         2200 |       10 |             0 |          18697 | t            | t           | 227927       |         227927 |
    18703 |   12737 |    18700 | BIN$31C14EB490C$132FE3F0==$0 | pg_toast_18697       | t            |       2
|      79356608 | 2023-09-13 21:24:42.820358+08 |            0 |
0 |           99 |       10 |             0 |          18700 | f            | f           | 227927       |         227927 |
(2 rows)

--查看表，表中的数据为空
gaussdb=# SELECT * FROM flashtest;
 id | name
----+------
(0 rows)

--闪回truncate表
gaussdb=# TIMECAPSULE TABLE flashtest to before truncate;
TimeCapsule Table
--查看表，表中的数据被恢复
gaussdb=# SELECT * FROM flashtest;
 id | name
----+------
  1 | A
(1 row)

--查看回收站
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid |           rcyname            |     rcyoriginname     | rcyoperation | rcytype |
rcyrecyclecsn |        rcyrecycletime         | rcycreatecsn | rcychangecs
n | rcynamespace | rcyowner | rcytablespace | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid |
rcyfrozenxid64 | rcybucket
```

 版权所有 © 华为云计算技术有限公司

```
-----------+---------+----------+-----------------------------+---------------------+--------------+---------
+---------------+-----------------------------+-------------+------------
--+--------------+----------+--------------+---------------+--------------+-------------+--------------
+---------------+-----------
    18703 |   12737 |    18700 | BIN$31C14EB490C$13300228==$0 | pg_toast_18697       | t           |       2
|     79356610 | 2023-09-13 21:24:42.872732+08 |            0 |
0 |          99 |       10 |            0 |          18706 | f            | f           | 0           |        227928 |
    18703 |   12737 |    18697 | BIN$31C14EB4909$9E4D$0==$0   | flashtest            | t           |       0 |
79356610 | 2023-09-13 21:24:42.872792+08 |     79356606 |     7935660
6 |         2200 |       10 |            0 |          18704 | t            | t           | 0           |        227928 |
(2 rows)

--drop表
gaussdb=# DROP TABLE IF EXISTS flashtest;
DROP TABLE
--清空回收站
gaussdb=# PURGE RECYCLEBIN;
PURGE RECYCLEBIN
--查看回收站，回收站被清空
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid | rcyname | rcyoriginname | rcyoperation | rcytype | rcyrecyclecsn |
rcyrecycletime | rcycreatecsn | rcychangecsn | rcynamespace | rcyowner | rcytablespace
 | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+---------+---------------+--------------+---------+---------------+----------------
+--------------+--------------+--------------+----------+--------------
-+---------------+--------------+-------------+-------------+----------------+-----------
(0 rows)
```

1.8.3.5 常用视图工具

视图类型	类型	功能描述	使用场景	函数名称
解析	全类型	用于解析指定表页面，并返回存放解析内容的路径。	• 查看页面信息。 • 查看元组（非用户数据）信息。 • 页面或者元组损坏。 • 元组可见性问题。 • 校验报错问题。	gs_parse_page_bypath
	索引回收队列（URQ）	用于解析UB-tree索引回收队列关键信息。	• UB-tree索引空间膨胀。 • UB-tree索引空间回收异常。 • 校验报错问题。	gs_urq_dump_stat

版权所有 © 华为云计算技术有限公司

视图类型	类型	功能描述	使用场景	函数名称
	回滚段（Undo）	用于解析指定Undo Record的内容，不包含旧版本元组的数据。	• undo空间膨胀。 • undo回收异常。 • 回滚异常。 • 日常巡检。 • 校验报错。 • 可见性判断异常。 • 修改参数。	gs_undo_dump_record
		用于解析指定事务生成的所有Undo Record，不包含旧版本元组的数据。		gs_undo_dump_xid
		用于解析指定UndoZone中所有Transaction Slot信息。		gs_undo_translot_dump_slot
		用于解析指定事务对应Transaction Slot信息，包括事务XID和该事务生成的Undo Record范围.		gs_undo_translot_dump_xid
		用于解析指定Undo Zone的元信息，显示Undo Record和Transaction Slot指针使用情况。		gs_undo_meta_dump_zone
		用于解析指定Undo Zone对应Undo Space的元信息，显示Undo Record文件使用情况。		gs_undo_meta_dump_spaces
		用于解析指定Undo Zone对应Slot Space的元信息，显示Transaction Slot文件使用情况。		gs_undo_meta_dump_slot
		用于解析数据页和数据页上数据的所有历史版本，并返回存放解析内容的路径。		gs_undo_dump_parsepage_mv
	预写日志（WAL）	用于解析指定LSN范围之内的xLog日志，并返回存放解析内容的路径。可以通过pg_current_xlog_location()获取当前xLog位置。	• WAL日志出错。 • 日志回放出错。 • 页面损坏。	gs_xlogdump_lsn
		用于解析指定XID的xLog日志，并返回存放解析内容的路径。可以通过txid_current()获取当前事务ID。		gs_xlogdump_xid
		用于解析指定表页面对应的日志，并返回存放解析内容的路径。		gs_xlogdump_tablepath
		用于解析指定表页面和表页面对应的日志，并返回存放解析内容的路径。可以看做一次执行gs_parse_page_bypath和gs_xlogdump_tablepath。该函数执行的前置条件是表文件存在。如果想查看已删除的表的相关日志，请直接调用gs_xlogdump_tablepath。		gs_xlogdump_parsepage_tablepath
统计	回滚段（Undo）	用于显示Undo模块的统计信息，包括Undo Zone使用情况、Undo链使用情况、Undo模块文件创建删除情况和Undo模块参数设置推荐值。	• Undo空间膨胀。 • Undo资源监控。	gs_stat_undo

版权所有 © 华为云计算技术有限公司

视图类型	类型	功能描述	使用场景	函数名称
	预写日志（WAL）	用于统计预写日志（WAL）写盘时的内存状态表内容。	• WAL写/刷盘监控。 • WAL写/刷盘hang住。	gs_stat_wal_entrytable
		用于统计预写日志（WAL）刷盘状态、位置统计信息。		gs_walwriter_flush_position
		用于统计预写日志（WAL）写刷盘次数频率、数据量以及刷盘文件统计信息。		gs_walwriter_flush_stat
校验	堆表/索引	用于离线校验表或者索引文件磁盘页面数据是否异常。	• 页面损坏或者元组损坏。 • 可见性问题。 • 日志回放出错问题。	ANALYZE VERIFY
		用于校验当前实例当前库物理文件是否存在丢失。	文件丢失。	gs_verify_data_file
	索引回收队列（URQ）	用于校验UB-tree索引回收队列（潜在队列/可用队列/单页面）数据是否异常。	• UB-tree索引空间膨胀。 • UB-tree索引空间回收异常。	gs_verify_urq
	回滚段（Undo）	用于离线校验Undo Record数据是否存在异常。	• Undo Record异常或者损坏。 • 可见性问题。 • 回滚出错或者异常。	gs_verify_undo_record
		用于离线校验Transaction Slot数据是否存在异常。	• Undo Record异常或者损坏。 • 可见性问题。 • 回滚出错或者异常。	gs_verify_undo_slot

版权所有 © 华为云计算技术有限公司

视图类型	类型	功能描述	使用场景	函数名称
		用于离线校验Undo元信息数据是否存在异常。	• 因Undo meta引起的节点无法启动问题。 • Undo空间回收异常。 • Snapshot too old问题。	gs_verify_undo_meta
修复	堆表/索引/Undo文件	用于基于备机修复主机丢失的物理文件。	堆表/索引/Undo文件丢失。	gs_repair_file
	堆表/索引/Undo页面	用于校验并基于备机修复主机受损页面。	堆表/索引/Undo页面损坏。	gs_verify_and_tryrepair_page
		用于基于备机页面直接修复主机页面。		gs_repair_page
		用于基于偏移量对页面的备份进行字节修改。		gs_edit_page_bypath
		用于将修改后的页面覆盖写入到目标页面。		gs_repair_page_bypath
	回滚段（Undo）	用于重建Undo元信息，如果校验发现Undo元信息没有问题则不重建。	Undo元信息异常或者损坏。	gs_repair_undo_byzone
	索引回收队列（URQ）	用于重建UB-tree索引回收队列。	索引回收队列异常或者损坏。	gs_repair_urq

1.8.3.6 常见问题及定位手段

1.8.3.6.1 snapshot too old

查询SQL执行时间过长或者其他一些原因，Undo无法保存太久的历史数据就可能因为历史版本被强制回收报错。一般情况下需要扩容回滚段空间，但具体问题需要具体分析。

长事务阻塞 Undo 空间回收

问题现象

1. gs_log中打印如下错误：
   ```
   snapshot too old! the undo record has been forcibly discarded
   ```


版权所有 © 华为云计算技术有限公司

```
xid xxx, the undo size xxx of the transaction exceeds the threshold xxx. trans_undo_threshold_size xxx,undo_space_limit_size xxx.
```

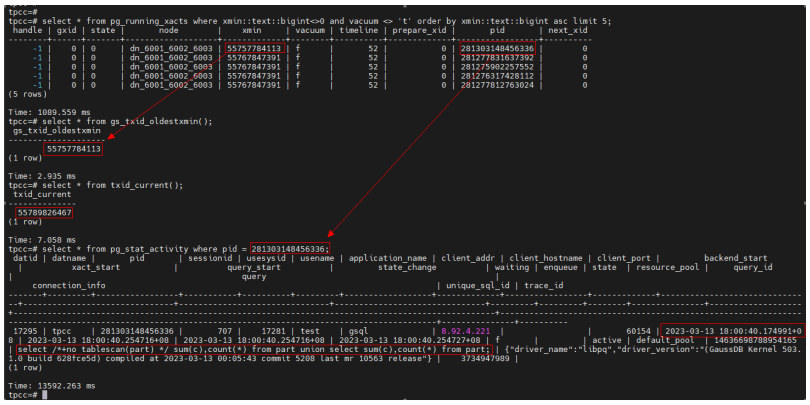
在真实报错信息中，上文中的xxx为实际数据。

2. global_recycle_xid（Undo子系统的全局回收事务XID）长时间不发生变化。

```
gaussdb=# select * from gs_undo_meta_dump_slot(1,-1);
 zone_id | allocate | recycle | frozen_xid | global_frozen_xid | recycle_xid | global_recycle_xid
---------+----------+---------+------------+-------------------+-------------+--------------------
       1 | 280      | 248     | 17028      | 17028             | 17025       | 17028
(1 row)
```

3. pg_running_xacts与pg_stat_activity视图查询存在长事务，阻塞oldestxmin和global_recycle_xid推进。如果pg_running_xacts中查询活跃事务的xmin和gs_txid_oldestxmin相等，且通过pid查询pg_stat_activity查询线程执行语句时间过长，则表明有长事务卡主了回收。

```
SELECT * FROM pg_running_xacts where xmin::text::bigint<>0 and vacuum <> 't' order by xmin::text::bigint asc limit 5;
SELECT * FROM gs_txid_oldestxmin();
SELECT * FROM pg_stat_activity WHERE pid = 长事务所在线程PID;
```



处理方法

通过pg_terminate_session(pid, sessionid)终止长事务所在的会话（提醒：长事务无固定快速恢复手段，强制结束SQL语句为其中一种常用操作，属于高危操作，执行需谨慎，执行前需与业务及华为技术确认，避免造成业务失败或报错）。

大量回滚事务拖慢 Undo 空间回收

问题现象

使用gs_async_rollback_xact_status视图查看有大量的待回滚事务，且待回滚的事务数量维持不变或者持续增高。

```
SELECT * FROM gs_async_rollback_xact_status();
```

处理方法

调大异步回滚线程数量，调整方式有以下两种：

方式1：在gaussdb.conf中配置max_undo_workers，然后重启节点。

方式2：gs_guc reload -Z NODE-TYPE [-N NODE-NAME] [-I INSTANCE-NAME | -D DATADIR] -c max_undo_workers=100 重启实例。

版权所有 © 华为云计算技术有限公司

1.8.3.6.2 storage test error

业务执行过程中，数据页、索引或者Undo页面发生变更后，该页面放锁之前会主动进行逻辑损坏检测，发现页面损坏问题后会输出包含“storage test error”关键字的日志信息到数据库运行日志（gs_log文件），执行事务回滚，页面会恢复到修改前的状态。

问题现象

gs_log中打印“storage test error”关键字。

处理方法

请联系华为技术支持解决。

1.8.3.6.3 备机读业务报错:"UBTreeSearch::read_page has conflict with recovery, please try again later"

问题现象

业务在使用备机读时，出现报错（错误码43244），错误信息中包含“UBTreeSearch::read_page has conflict with recovery, please try again later”关键字。

问题分析

在开启并行回放或串行回放的情况下（查询GUC参数recovery_parse_workers和recovery_max_workers均是1为串行回放；recovery_parse_workers是1，recovery_max_workers大于1为并行回放），备机的查询线程在做索引扫描时，会先对索引页面加读锁，每当扫到一个元组时会去判可见性。如果该元组对应的事务处于committing状态，需要等待该事务提交后再判断。而备机上的事务提交是依赖日志回放线程推进的，这个过程中会对索引页面进行修改，因此需要加锁。查询线程在等待过程中会释放索引页面的锁，否则会出现查询线程等待回放线程进行事务提交，而回放线程在等待查询线程释放锁。

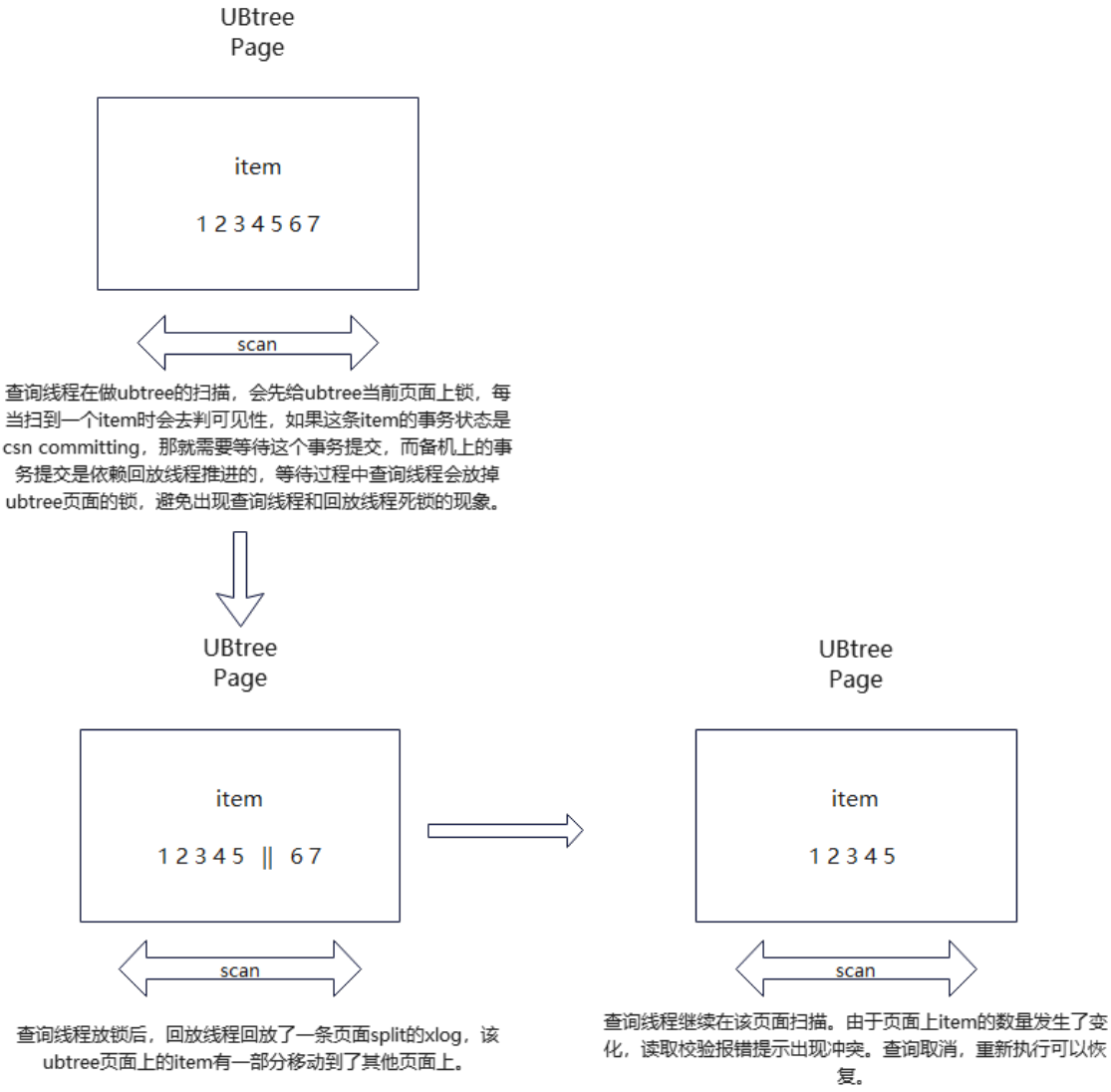
该报错仅出现在查询与回放都需要访问同一个索引页面的场景下，查询线程在释放锁并等待事务结束过程中，访问的页面出现被修改的情况。具体流程图如下图1所示：

说明

- 备机查询在扫到committing状态的元组时，需要等待事务提交是因为事务提交的顺序与产生日志的顺序可能是乱序的。例如主机上tx_1的事务比tx_2先提交，而备机上tx_1的commit日志在tx_2的commit日志之后回放，按照事务提交顺序来看tx_1对tx_2应当是可见的，所以需要等待事务提交。
- 备机查询在扫描索引页面时，发现页面元组数量（包含死元组）发生变化后不可重试，是因为在扫描时可能为正向或反向扫描，而举例来说页面发生分裂后一部分元组移动到右页面，在反向扫描的情况下即使重试只能向左扫描读取，无法再保证结果的正确性，并且由于无法分辨发生分裂或者插入，所以不可重试。

版权所有 © 华为云计算技术有限公司

图 1-7 问题分析



处理方法

出现报错时，建议重试查询。另外建议选择非频繁更新的索引字段、采用软删除的方式（物理删除操作在业务低谷期执行），可以降低出现该报错的概率。

1.8.4 数据生命周期管理-OLTP 表压缩

1.8.4.1 特性简介

OLTP表压缩是GaussDB高级压缩中的一个特性。基于全新的压缩算法、细粒度的自动冷热判定和支持块内压缩等技术创新，可以在提供合理压缩率的同时大幅度降低对业务的影响、增加后台调度、增加查询Job执行状态以及节约空间，能够在支持关键在线业务的容量控制中发挥重要价值。

版权所有 © 华为云计算技术有限公司

1.8.4.2 特性约束

- 不支持系统表、内存表、全局临时表、本地临时表和序列表。
- 仅在ORA兼容模式与PG模式下有效。
- Ustore不支持编解码，压缩率小于2:1。
- 普通表开启压缩时，扩容空间预留需按照解压后的大小评估。
- HashBucket表不支持DBE_HEAT_MAP.ROW_HEAT_MAP和DBE_COMPRESSION.GET_COMPRESSION_TYPE。
- 扩容期间不支持压缩调度。
- 扩容前请确认当前是否有正在执行的压缩任务，如果有的话，要么等待压缩任务结束，要么执行DBE_ILM.STOP_ILM或DBE_ILM_ADMIN.DISABLE_ILM停掉，扩容完成后再执行

  DBE_ILM_ADMIN.ENABLE_ILM开启。

1.8.4.3 特性规格

- TPCC只开启策略、不开调度对原有业务无影响。
- TPCC不开启压缩策略对原有业务无影响。
- TPCC.bmsql_order_line设置ILM策略（只识别完成派送的订单为冷行）不调度，TPmC劣化不高于2%（56核CPU370GB内存+3TB SSD硬盘，350GB SharedBuffer）。
- TPCC.bmsql_order_line设置ILM策略（只识别完成派送的订单为冷行）后台默认参数调度时，TPmC劣化不高于5%（56核CPU370GB内存+3TB SSD硬盘，350GB SharedBuffer）。
- 单线程ILM Job带宽约100MB/秒（56核CPU370GB内存+3TB SSD硬盘，350GB SharedBuffer）。

  度量方式：根据执行压缩的开始时间和结束时间以及压缩的页面个数计算带宽。
- get查询访问压缩数据比非压缩数据性能劣化，驱动侧不高于10%，plsql侧不高于15%（32MB SharedBuffer，6万页面数据）。
- multi-get查询访问压缩数据比非压缩数据性能劣化，驱动侧不高于30%，plsql侧不高于40%（32MB SharedBuffer，6万页面数据）。
- table-scan查询访问压缩数据比非压缩数据性能劣化，驱动侧不高于30%，plsql侧不高于40%（32MB SharedBuffer，6万页面数据）。
- TPCH.lineitem表压缩比（全冷行）不小于2:1。
- 对于TPC-C的Orderline表，以及TPC-H的Lineitem、Orders、Customer、Part表的测试表明，数值型字段较多时，压缩率高于LZ4和ZLIB；而文本型字段较多时，压缩率介于LZ类和LZ+Huffman组合类的压缩算法之间。

1.8.4.4 使用说明

使用高级压缩的功能，用户必须购买License才能使用。具体情况请联系华为工程师。

步骤1 执行如下命令开启压缩功能：

```
gaussdb=# ALTER DATABASE SET ilm = on;
```

检查当前数据库的public schema中是否存在gsilmpolicy_seq和gsilmtask_seq。

```
gaussdb=# \d
                        List of relations
```


版权所有 © 华为云计算技术有限公司

```
 Schema |      Name       |   Type   |   Owner    |                      Storage
--------+-----------------+----------+------------+-----------------------------------------------------
 public | gsilmpolicy_seq | sequence | omm |
 public | gsilmtask_seq   | sequence | omm |
```

或者：

```
gaussdb=# SELECT a.oid, a.relname FROM pg_class a inner join pg_namespace b on a.relnamespace = b.oid
    WHERE (a.relname = 'gsilmpolicy_seq' OR a.relname = 'gsilmtask_seq') AND b.nspname = 'public';

  oid  |     relname
-------+-----------------
 17002 | gsilmpolicy_seq
 17004 | gsilmtask_seq
(2 rows)
```

生成异常会报warning：

```
WARNING:  ILM sequences are already existed while initializing
```

步骤2 为表添加压缩策略。

- 新建带策略的表：
  ```
  gaussdb=# CREATE TABLE ilm_table_1 (col1 int, col2 text)
      ilm add policy row store compress advanced row
      after 3 days of no modification on (col1 < 1000);
  ```
- 为存量表添加策略：
  ```
  gaussdb=# CREATE TABLE ilm_table_2 (col1 int, col2 text);
  gaussdb=# ALTER TABLE ilm_table_2 ilm add policy row store
      compress advanced row after 3 days of no modification;
  ```
- 检查策略视图中是否新增数据：
  ```
  gaussdb=# SELECT * FROM gs_my_ilmpolicies;

   policy_name |  policy_type  | tablespace | enabled | deleted
  -------------+---------------+------------+---------+---------
   p1          | DATA MOVEMENT |            | YES     | NO
   p2          | DATA MOVEMENT |            | YES     | NO
  (2 rows)
  ```
- 检查策略详细信息视图中是否新增了符合刚刚设置的策略：
  ```
  gaussdb=# SELECT * FROM gs_my_ilmdatamovementpolicies;

   policy_name | action_type | scope | compression_level | tier_tablespace | tier_status |
  condition_type     | condition_days | custom_function | policy_subtype | action_clause | tier_to
  -------------+-------------+-------+-------------------+-----------------+-------------+-------------------------
  +---------------+----------------+---------------+--------------+---------
   p1          | COMPRESSION | ROW   | ADVANCED          |                 |             | LAST MODIFICATION
  TIME |             3 |                |               |              |
   p2          | COMPRESSION | ROW   | ADVANCED          |                 |             | LAST MODIFICATION
  TIME |             3 |                |               |              |
  (2 rows)
  ```
- 检查策略与目标表是否对应：
  ```
  gaussdb=# SELECT * FROM gs_my_ilmobjects;
   policy_name | object_owner | object_name | subobject_name | object_type |    inherited_from    |
  tbs_inherited_from | enabled | deleted
  -------------+--------------+-------------+----------------+-------------+----------------------
  +--------------------+---------+---------
   p1          | public       | ilm_table_1 |                | TABLE       | POLICY NOT INHERITED |                    |
  YES     | NO
   p2          | public       | ilm_table_2 |                | TABLE       | POLICY NOT INHERITED |                    |
  YES     | NO
  (2 rows)
  ```

步骤3 执行压缩评估。

- 手动执行压缩评估。

 版权所有 © 华为云计算技术有限公司

⚠ 注意

为方便测试，本功能环境参数中提供POLICY_TIME属性，决定时间条件以天为单位还是以秒为单位。通过下面语句调整：

```
gaussdb=# CALL DBE_ILM_ADMIN.CUSTOMIZE_ILM(11, 1);
```

插入随机数据用于测试：

```
gaussdb=# INSERT INTO ilm_table_1 select *, 'test_data' FROM generate_series(1, 10000);

gaussdb=# DECLARE
    v_taskid number;
gaussdb=# BEGIN
    DBE_ILM.EXECUTE_ILM(OWNER           => 'public',
                        OBJECT_NAME     => 'ilm_table_1',
                        TASK_ID         => v_taskid,
                        SUBOBJECT_NAME  => NULL,
                        POLICY_NAME     => 'ALL POLICIES',
                        EXECUTION_MODE  => 2);
    RAISE INFO 'Task ID is:%', v_taskid;
gaussdb=# END;
/
```

如入参有误，会报对应的错误信息。无误则无输出（上述代码段添加了RAISE INFO语句打印当前task的id）。

```
INFO:  Task ID is:1
```

检查task信息：

```
gaussdb=# SELECT * FROM gs_my_ilmtasks;

task_id | task_owner |   state   |         creation_time         |          start_time           |
completion_time
---------+------------+-----------+-------------------------------+-------------------------------
+-------------------------------
       1 | omm| COMPLETED | 2023-08-29 17:36:38.779555+08 | 2023-08-29 17:36:38.779555+08 |
2023-08-29 17:36:38.879485+08
(1 row)
```

检查评估结果：

```
gaussdb=# SELECT * FROM gs_my_ilmevaluationdetails;

 task_id | policy_name | object_owner | object_name | subobject_name | object_type |
selected_for_execution |      job_name      | comments
---------+-------------+--------------+-------------+----------------+-------------+------------------------
+--------------------+----------
       1 | p1          | public       | ilm_table_1 |                | TABLE       | SELECTED FOR EXECUTION | ilmjob
$_postgres1 |
(1 row)
```

检查压缩job信息：

```
gaussdb=# SELECT * FROM gs_my_ilmresults;

 task_id |     job_name      |       job_state        |          start_time           |       completion_time         |
comments |               statistics
---------+-------------------+------------------------+-------------------------------
+-------------------------------+----------+-----------------------------------------
       1 | ilmjob$_postgres1 | COMPLETED SUCCESSFULLY | 2023-08-29 17:36:38.779555+08 |
2023-08-29 17:36:38.879485+08 |          | SpaceSaving=0,BoundTime=0,LastBlkNum=0
(1 row)
```

- 触发后台自动调度评估。

  使用初始用户登录template1数据库，创建维护窗口：

  ```
  gaussdb=#
  DECLARE
      V_HOUR INT := 22;
      V_MINUTE INT := 0;
      V_SECOND INT := 0;
  ```


版权所有 © 华为云计算技术有限公司

```
    C_ADO_WINDOW_SCHEDULE_NAME TEXT := 'ado_window_schedule';
    C_ADO_WINDOW_PROGRAM_NAME TEXT := 'ado_window_program';
    C_MAINTENANCE_WINDOW_JOB_NAME TEXT := 'maintenance_window_job';
    V_MAINTENCE_WINDOW_REPEAT TEXT;
    V_MAINTENCE_WINDOW_START TIMESTAMPTZ;
    V_BE_SCHEDULE_ENABLE BOOL;
    V_MAINTENANCE_WINDOW_EXIST INT;
BEGIN
    SELECT COUNT(*) INTO V_MAINTENANCE_WINDOW_EXIST FROM PG_CATALOG.PG_JOB WHERE
JOB_NAME = 'maintenance_window_job' AND DBNAME = 'template1';
    IF CURRENT_DATABASE() != 'template1' THEN
        RAISE EXCEPTION 'Create maintenance_window FAILED, current database is not tempalte1';
    END IF;
    IF V_MAINTENANCE_WINDOW_EXIST = 0 AND CURRENT_DATABASE() = 'template1' THEN
        SELECT
            CASE
                WHEN NOW() < CURRENT_DATE + INTERVAL '22 HOUR' THEN CURRENT_DATE +
INTERVAL '22 HOUR'
                ELSE CURRENT_DATE + INTERVAL '1 DAY 22 HOUR'
            END INTO V_MAINTENCE_WINDOW_START;
        --1. prepare for maintence window schedule
        SELECT 'freq=daily;interval=1;byhour='||V_HOUR||';byminute='||V_MINUTE||';bysecond='||
V_SECOND INTO V_MAINTENCE_WINDOW_REPEAT;
        BEGIN
            SELECT
                CASE
                    WHEN VALUE = 1 THEN TRUE -- DBE_ILM_ADMIN.ILM_ENABLED
                    ELSE FALSE
                END INTO V_BE_SCHEDULE_ENABLE
                FROM PG_CATALOG.GS_ILM_PARAM WHERE IDX = 7; -- DBE_ILM_ADMIN.ENABLED
        EXCEPTION
            WHEN OTHERS THEN
                V_BE_SCHEDULE_ENABLE := FALSE;
        END;
        --2. Create ado window schedule
        DBE_SCHEDULER.CREATE_SCHEDULE(
            SCHEDULE_NAME => C_ADO_WINDOW_SCHEDULE_NAME,
            START_DATE => '9999-01-01 00:00:01',
            REPEAT_INTERVAL => NULL,
            END_DATE => NULL,
            COMMENTS => 'ado window schedule');
        --3. Create ado window program
        DBE_SCHEDULER.CREATE_PROGRAM(
            PROGRAM_NAME => C_ADO_WINDOW_PROGRAM_NAME,
            PROGRAM_TYPE => 'plsql_block',
            PROGRAM_ACTION => 'call prvt_ilm.be_execute_ilm(0);',
            NUMBER_OF_ARGUMENTS => 0,
            ENABLED => TRUE,
            COMMENTS => NULL);
        --4. Create maintenance window master job
        DBE_SCHEDULER.CREATE_JOB(
            JOB_NAME => C_MAINTENANCE_WINDOW_JOB_NAME,
            START_DATE => V_MAINTENCE_WINDOW_START,
            REPEAT_INTERVAL => V_MAINTENCE_WINDOW_REPEAT,
            END_DATE => NULL,
            JOB_TYPE => 'STORED_PROCEDURE'::TEXT,
            JOB_ACTION => 'prvt_ilm.be_active_ado_window'::TEXT,
            NUMBER_OF_ARGUMENTS => 0,
            ENABLED => V_BE_SCHEDULE_ENABLE,
            AUTO_DROP => FALSE,
            COMMENTS => 'maintenance window job',
            destination_name=> 'CCN');
    ELSE
        RAISE EXCEPTION 'CREATE ILM MAINTENANCE WINDOW FAILED';
    END IF;
END;
```

自动调度提供若干参数用于调整：

```
gaussdb=# SELECT * FROM gs_adm_ilmparameters;
       name        | value
```

 版权所有 © 华为云计算技术有限公司

```
--------------------+-------
 EXECUTION_INTERVAL |    15
 RETENTION_TIME     |    30
 ENABLED            |     1
 POLICY_TIME        |     0
 ABS_JOBLIMIT       |    10
 JOB_SIZELIMIT      |  1024
 WIND_DURATION      |   240
 BLOCK_LIMITS       |    40
(8 rows)
```

- EXECUTION_INTERVAL：自动调度任务执行间隔，默认每15分钟执行一次。
- RETENTION_TIME：历史压缩任务记录清理间隔，默认每30天清理一次。
- ENABLED：当前自动调度启用情况，默认为开启。
- POLICY_TIME：策略评估的时间单位，测试使用。默认以天为单位。
- ABS_JOBLIMIT：单次评估生成压缩任务数量上限，默认为10个。
- JOB_SIZELIMIT：单个压缩任务的IO上限，默认为1GB。
- WIND_DURATION：单次维护窗口的持续时间。
- BLOCK_LIMITS：控制实例级的行存压缩速率上限，默认是40，取值范围是0到10000（0表示不限制），单位是block/ms，表示每毫秒最多压缩多少个block。速率上限计算方法：BLOCK_LIMITS*1000*BLOCKSIZE，以默认值40为例，其速率上限为：40*1000*8KB=320000KB/s。

以上参数均可通过DBE_ILM_ADMIN.CUSTOMIZE_ILM()接口调整。

维护窗口默认每天晚上22：00（北京时间）开启，可通过DBE_SCHEDULER提供的接口SET_ATTRIBUTE进行设置：

```
\c template1
 CALL DBE_ILM_ADMIN.DISABLE_ILM();
 CALL DBE_ILM_ADMIN.ENABLE_ILM();
DECLARE
    newtime timestamptz := CLOCK_TIMESTAMP() + to_interval('2 seconds');
BEGIN
    DBE_SCHEDULER.set_attribute(
        name          =>          'maintenance_window_job',
        attribute     =>          'start_date',
        value         =>          TO_CHAR(newtime, 'YYYY-MM-DD HH24:MI:SS')
    );
END;
/
```

----结束

1.8.4.5 维护窗口参数配置

- RETENTION_TIME：评估与压缩记录的保留时长，单位天，默认值30。用户可根据自己存储容量自行调节。
- EXECUTION_INTERVAL：评估任务的执行频率，单位分钟，默认值15。用户可根据自己维护窗口期间业务与资源情况调节。该参数与ABS_JOBLIMIT相互影响。单日单线程最大可产生的I/O为WIND_DURATION/EXECUTION_INTERVAL*JOB_SIZELIMIT。
- JOB_SIZELIMIT：控制单个压缩Job可以处理的最大字节数，单位兆，默认值1024。压缩带宽约为100MB/秒，每个压缩Job限制I/O为1GB时，最多10秒完成。用户可根据自己业务闲时情况以及需要压缩的数据量自行调节。
- ABS_JOBLIMIT：控制一次评估最多生成多少个压缩Job。用户可根据自己设置策略的分区及表数量自己调节。建议最大不超过10，可以使用“select count(*) from gs_adm_ilmobjects where enabled = true”命令查询。

 版权所有 © 华为云计算技术有限公司

- POLICY_TIME：控制判定冷行的条件单位是天还是秒，秒仅用来做测试用。取值为：ILM_POLICY_IN_SECONDS或ILM_POLICY_IN_DAYS（默认值）。
- WIND_DURATION：维护窗口持续时长，单位分钟，默认240分钟（4小时）。维护窗口默认从北京时间22点开始持续240分钟，用户可根据自己业务闲时情况自行调节。
- BLOCK_LIMITS：控制实例级的行存压缩速率上限，默认是40，取值范围是0到10000(0表示不限制)，单位是block/ms，表示每毫秒最多压缩多少个block。速率上限计算方法：BLOCK_LIMITS*1000*BLOCKSIZE，以默认值40为例，其速率上限为:40*1000*8KB=320000KB/s。

示例分析：

```
EXECUTION_INTERVAL：15
JOB_SIZELIMIT：10240
WIND_DURATION：240
BLOCK_LIMITS：0
```

此配置下单表分区在一个维护窗口期间可完成240/15*10240MB=160GB数据的评估压缩。压缩带宽为100MB/秒，实际压缩仅耗时160GB/(100MB/秒)=27分钟。其他时间对业务无影响。用户可根据自己业务闲时可支配给压缩的时长来调节参数。

1.8.4.6 运维 TIPS

须知

高级压缩特性使用前必须先联系工作人员购买License，否则执行相关命令会报错。

1. 手动触发一次压缩（示例中一次压缩102400MB）。
   a. 给表加上冷热分离策略：

   ```
   gaussdb=#  DROP TABLE IF EXISTS ILM_TABLE;
   gaussdb=#  CREATE TABLE ILM_TABLE(a int);
   gaussdb=# ALTER TABLE ILM_TABLE ILM ADD POLICY ROW STORE COMPRESS ADVANCED
       ROW AFTER 3 MONTHS OF NO MODIFICATION;
   ```

   b. 手动触发压缩：

   ```
   DECLARE
     v_taskid number;
   BEGIN
     DBE_ILM_ADMIN.CUSTOMIZE_ILM(11, 1);
   DBE_ILM_ADMIN.CUSTOMIZE_ILM(13, 102400);
     DBE_ILM.EXECUTE_ILM(OWNER         => '$schema_name',
                         OBJECT_NAME     => 'ilm_table',
                         TASK_ID         => v_taskid,
                         SUBOBJECT_NAME => NULL,
                         POLICY_NAME     => 'ALL POLICIES',
                         EXECUTION_MODE => 2);
     RAISE INFO 'Task ID is:%', v_taskid;
   END;
   /
   ```

   c. 查看压缩JOB是否完成，可以看到具体的执行信息：

   ```
   gaussdb=# SELECT * FROM gs_adm_ilmresults ORDER BY task_id desc;

    task_id |                job_name                |     start_time      |   completion_time
   |                    statistics
   ---------+----------------------------------------+---------------------+---------------------
   +-----------------------------------------------------------
      17267 | ilmjob$_2 | 2023-03-29 08:11:25 | 2023-03-29 08:11:25 |
   SpaceSaving=453048,BoundTime=1680145883,LastBlkNum=128
   ```

2. 手动停止压缩。

版权所有 © 华为云计算技术有限公司

```
gaussdb=# DBE_ILM.STOP_ILM (task_id => V_TASK, p_drop_running_Jobs => FALSE, p_Jobname =>
V_JOBNAME);
```

表 1-7 DBE_ILM.STOP_ILM 输入参数

名称	描述
task_id	指定待停止ADO task的描述符ID。
p_drop_running_Jobs	标识是否停止正在运行的JOB。
p_Jobname	标识待停止的特定JobName，通过GS_MY_ILMEVALUATIONDETAILS视图可以查询。

3. 为表生成策略及后台调度压缩任务。
   a. 给表加上冷热分离策略：
   ```
   gaussdb=# DROP TABLE IF EXISTS ILM_TABLE;
   gaussdb=# CREATE TABLE ILM_TABLE(a int);
   gaussdb=# ALTER TABLE ILM_TABLE ILM ADD POLICY ROW STORE COMPRESS ADVANCED
      ROW AFTER 3 MONTHS OF NO MODIFICATION;
   ```
   b. 设置ILM执行相关参数：
   ```
   BEGIN
     DBE_ILM_ADMIN.CUSTOMIZE_ILM(11, 1);
     DBE_ILM_ADMIN.CUSTOMIZE_ILM(12, 10);
     DBE_ILM_ADMIN.CUSTOMIZE_ILM(1, 1);
     DBE_ILM_ADMIN.CUSTOMIZE_ILM(13, 512);
   END;
   /
   ```
   c. 开启后台的定时调度：
   ```
   gaussdb=# CALL DBE_ILM_ADMIN.DISABLE_ILM();
   gaussdb=# CALL DBE_ILM_ADMIN.ENABLE_ILM();
   ```
   d. 用户可以根据需要，调用DBE_SCHEDULER.set_attribute设置后台维护窗口的开启时间。当前默认22:00开启。
4. 设置ILM执行相关参数。

   控制ADO的条件单位是天还是秒，秒仅用来做测试用。取值为：ILM_POLICY_IN_SECONDS = 1或ILM_POLICY_IN_DAYS = 0（默认值）：

   ```
   gaussdb=# CALL DBE_ILM_ADMIN.CUSTOMIZE_ILM(11, 1);
   ```

   控制一次ADO Task最多生成多少个ADO Job。取值范围大于等于0小于等于2147483647的整数或浮点数，作用时向下取整：

   ```
   gaussdb=# CALL DBE_ILM_ADMIN.CUSTOMIZE_ILM(12, 10);
   ```

   ADO Task的执行频率，单位分钟，默认值15。取值范围大于等于1小于等于2147483647的整数或浮点数，作用时向下取整：

   ```
   gaussdb=# CALL DBE_ILM_ADMIN.CUSTOMIZE_ILM(1, 1);
   ```

   控制单个ADO Job可以处理的最大字节数，单位兆。取值范围大于等于0小于等于2147483647的整数或浮点数，作用时向下取整：

   ```
   gaussdb=# CALL DBE_ILM_ADMIN.CUSTOMIZE_ILM(13, 512);
   ```

5. 评估一张表是否适合压缩及评估压缩后带来多少收益。
   ```
   gaussdb=# DBE_COMPRESSION.GET_COMPRESSION_RATIO (
     scratchtbsname        IN     VARCHAR2,
     ownname               IN     VARCHAR2,
     objname               IN     VARCHAR2,
     subobjname            IN     VARCHAR2,
     comptype              IN     NUMBER,
     blkcnt_cmp            OUT    PLS_INTEGER,
     blkcnt_uncmp          OUT    PLS_INTEGER,
   ```

 版权所有 © 华为云计算技术有限公司

```
row_cmp              OUT    PLS_INTEGER,
row_uncmp            OUT    PLS_INTEGER,
cmp_ratio            OUT    NUMBER,
comptype_str         OUT    VARCHAR2,
sample_ratio    IN  INTEGER DEFAULT 20,
objtype              IN     PLS_INTEGER DEFAULT OBJTYPE_TABLE);
```

表 1-8 DBE_COMPRESSION.GET_COMPRESSION_RATIO 输入参数

名称	描述
scratchtbsname	数据所在空间名称。
ownname	数据对象的拥有者名称。
objname	数据对象名称。
subobjname	数据的分区名称，默认为NULL。
comptype	压缩类型：COMP_NOCOMPRESS和COMP_ADVANCED。
sample_ratio	采样比例，输入为0-100的整数或浮点数，对应为百分之N的采样比例。默认为20，即对20%的行数进行采样。
objtype	对象类型，本期支持的类型为`OBJTYPE_TABLE`。

表 1-9 DBE_COMPRESSION.GET_COMPRESSION_RATIO 输出参数

名称	描述
blkcnt_cmp	样本被压缩后占用的块数。
blkcnt_uncmp	样本未压缩占用的块数。
row_cmp	样本被压缩后单个块内可容纳的行数。
row_uncmp	样本未被压缩时单个数据块可容纳的行数。
cmp_ratio	压缩比，blkcnt_uncmp除以blkcnt_cmp。
comptype_str	描述压缩类型的字符串。

示例：

```
gaussdb=# ALTER DATABASE set ilm = on;
gaussdb=# CREATE user user1 IDENTIFIED BY '********';
gaussdb=# CREATE user user2 IDENTIFIED BY '********';
gaussdb=# SET ROLE user1 PASSWORD '********';
gaussdb=# CREATE TABLE TEST_DATA (ORDER_ID INT, GOODS_NAME TEXT, CREATE_TIME
TIMESTAMP)
  ILM ADD POLICY ROW STORE COMPRESS ADVANCED ROW AFTER 1 DAYS OF NO MODIFICATION;
INSERT INTO TEST_DATA VALUES (1, '零食大礼包A', NOW());

DECLARE
o_blkcnt_cmp      integer;
o_blkcnt_uncmp    integer;
o_row_cmp         integer;
```


版权所有 © 华为云计算技术有限公司

```
o_row_uncmp       integer;
o_cmp_ratio       number;
o_comptype_str    varchar2;
begin
dbe_compression.get_compression_ratio(
    SCRATCHTBSNAME  =>  NULL,
    OWNNAME         =>  'user1',
    OBJNAME         =>  'test_data',
    SUBOBJNAME      =>  NULL,
    COMPTYPE        =>  2,
    BLKCNT_CMP      =>  o_blkcnt_cmp,
    BLKCNT_UNCMP    =>  o_blkcnt_uncmp,
    ROW_CMP         =>  o_row_cmp,
    ROW_UNCMP       =>  o_row_uncmp,
    CMP_RATIO       =>  o_cmp_ratio,
    COMPTYPE_STR    =>  o_comptype_str,
    SAMPLE_RATIO    =>  100,
    OBJTYPE         =>  1);
RAISE INFO 'Number of blocks used by the compressed sample of the object        : %', o_blkcnt_cmp;
RAISE INFO 'Number of blocks used by the uncompressed sample of the object      : %',
o_blkcnt_uncmp;
RAISE INFO 'Number of rows in a block in compressed sample of the object        : %', o_row_cmp;
RAISE INFO 'Number of rows in a block in uncompressed sample of the object      : %', o_row_uncmp;
RAISE INFO 'Estimated Compression Ratio of Sample                               : %', o_cmp_ratio;
RAISE INFO 'Compression Type                                                    : %', o_comptype_str;
end;
/
INFO:  Number of blocks used by the compressed sample of the object        : 0
INFO:  Number of blocks used by the uncompressed sample of the object      : 0
INFO:  Number of rows in a block in compressed sample of the object        : 0
INFO:  Number of rows in a block in uncompressed sample of the object      : 0
INFO:  Estimated Compression Ratio of Sample                               : 1
INFO:  Compression Type                                                    : Compress Advanced
```

6. 查询每一行的最后修改时间。

```
gaussdb=# DBE_HEAT_MAP.ROW_HEAT_MAP(
 owner     IN VARCHAR2,
 segment_name  IN VARCHAR2,
 partition_name  IN VARCHAR2 DEFAULT NULL,
ctid     IN  VARCHAR2,);
```

表 1-10 DBE_HEAT_MAP.ROW_HEAT_MAP 输入参数

名称	描述
owner	数据对象的所有者。
segment_name	数据对象名称。
partition_name	数据对象分区名称，可选参数。
ctid	目标行的ctid，即block_id或row_id。

表 1-11 DBE_HEAT_MAP.ROW_HEAT_MAP 输出参数

名称	描述
owner	数据对象的所有者。
segment_name	数据对象名称。
partition_name	数据对象分区名称，可选参数。

 版权所有 © 华为云计算技术有限公司

名称	描述
tablespace_name	数据所属的表空间名称。
file_id	行所属的绝对文件ID。
relative_fno	行所属的相对文件ID（GaussDB中无此逻辑，因此取值同上）。
ctid	行的ctid，即block_id或row_id。
writetime	行的最后修改时间。

示例：

```
gaussdb=# ALTER DATABASE set ilm = on;
gaussdb=# CREATE Schema HEAT_MAP_DATA;
gaussdb=# SET current_schema=HEAT_MAP_DATA;

gaussdb=# CREATE TABLESPACE example1 RELATIVE LOCATION 'tablespace1';
gaussdb=# CREATE TABLE HEAT_MAP_DATA.heat_map_table(id INT, value TEXT) TABLESPACE
example1;
gaussdb=# INSERT INTO HEAT_MAP_DATA.heat_map_table VALUES (1, 'test_data_row_1');

gaussdb=# SELECT * from DBE_HEAT_MAP.ROW_HEAT_MAP(
    owner          =>  'heat_map_data',
    segment_name  =>  'heat_map_table',
    partition_name  => NULL,
    ctid           =>  '(0,1)');
     owner     |  segment_name  | partition_name | tablespace_name | file_id | relative_fno | ctid  |
writetime
---------------+----------------+----------------+-----------------+---------+--------------+-------+-----------
 heat_map_data | heat_map_table |                | example1        |   17291 |        17291 | (0,1) |
(1 row)
```

7. 查询ILM调度与执行的相关环境参数。

```
gaussdb=# SELECT * FROM GS_ADM_ILMPARAMETERS;
         name         | value
----------------------+-------
 EXECUTION_INTERVAL |    15
 RETENTION_TIME     |    30
 ENABLED            |     1
 POLICY_TIME        |     0
 ABS_JOBLIMIT       |    10
 JOB_SIZELIMIT      |  1024
 WIND_DURATION      |   240
 BLOCK_LIMITS       |    40
(8 rows)
```

8. 查询ILM策略的概要信息，包含策略名称、类型、启用禁用状态、删除状态。

```
gaussdb=# SELECT * FROM GS_ADM_ILMPOLICIES;
policy_name |  policy_type  | tablespace | enabled | deleted
------------+---------------+------------+---------+---------
 p1         | DATA MOVEMENT |            | YES     | NO


gaussdb=# SELECT * FROM GS_MY_ILMPOLICIES;
 policy_name |  policy_type  | tablespace | enabled | deleted
-------------+---------------+------------+---------+---------
 p1          | DATA MOVEMENT |            | YES     | NO
```

9. 查询ILM策略的数据移动概要信息，包含策略名称、动作类型、条件等。

```
gaussdb=# SELECT * FROM GS_ADM_ILMDATAMOVEMENTPOLICIES;
 policy_name | action_type | scope | compression_level | tier_tablespace | tier_status |
condition_type     | condition_days | custom_function | policy_subtype | action_clause | tier_to
-------------+-------------+-------+-------------------+-----------------+-------------+-------------------------
```


版权所有 © 华为云计算技术有限公司

```
+----------------+-----------------+---------------+---------------+---------
 p1          | COMPRESSION | ROW   | ADVANCED          |               |             | LAST MODIFICATION
TIME |             90 |               |               |             |

gaussdb=# SELECT * FROM GS_MY_ILMDATAMOVEMENTPOLICIES;
 policy_name | action_type | scope | compression_level | tier_tablespace | tier_status |
condition_type     | condition_days | custom_function | policy_subtype | action_clause | tier_to
-------------+-------------+-------+-------------------+-----------------+-------------+------------------------
+----------------+-----------------+---------------+---------------+---------
 p1          | COMPRESSION | ROW   | ADVANCED          |               |             | LAST MODIFICATION
TIME |             90 |               |               |             |
(1 row)
```

10. 查询所有存在ILM策略应用的数据对象与相应策略的概要信息，包含策略名称、数据对象名称、策略的来源、策略的启用删除状态。

```
gaussdb=# SELECT * FROM  GS_ADM_ILMOBJECTS;
 policy_name | object_owner | object_name | subobject_name | object_type |    inherited_from    |
tbs_inherited_from | enabled | deleted
-------------+--------------+-------------+----------------+-------------+----------------------
+--------------------+---------+---------
 p1          | public       | lineitem    |                | TABLE       | POLICY NOT INHERITED |                    |
YES     | NO

gaussdb=# SELECT * FROM  GS_MY_ILMOBJECTS;
 policy_name | object_owner | object_name | subobject_name | object_type |    inherited_from    |
tbs_inherited_from | enabled | deleted
-------------+--------------+-------------+----------------+-------------+----------------------
+--------------------+---------+---------
 p1          | public       | lineitem    |                | TABLE       | POLICY NOT INHERITED |                    |
YES     | NO
```

11. 查询ADO Task的概要信息，包含Task ID，Task Owner，状态以及时间信息。

```
gaussdb=# SELECT * FROM GS_ADM_ILMTASKS;
 task_id | task_owner |   state   |         creation_time         |         start_time          |
completion_time
---------+------------+-----------+-------------------------------+-----------------------------
+-------------------------------
       1 | omm        | COMPLETED | 2023-10-16 12:03:55.113296+08 | 2023-10-16 12:03:55.113296+08 |
2023-10-16 12:03:56.326864+08
(1 row)

gaussdb=# SELECT * FROM GS_MY_ILMTASKS;
 task_id | task_owner |   state   |         creation_time         |         start_time          |
completion_time
---------+------------+-----------+-------------------------------+-----------------------------
+-------------------------------
       1 | omm        | COMPLETED | 2023-10-16 12:03:55.113296+08 | 2023-10-16 12:03:55.113296+08 |
2023-10-16 12:03:56.326864+08
(1 row)
```

12. 查询ADO Task的评估详情信息，包含Task ID，策略信息、对象信息、评估结果以及ADO JOB名称。

```
gaussdb=# SELECT * FROM GS_ADM_ILMEVALUATIONDETAILS;
 task_id | policy_name | object_owner | object_name | subobject_name | object_type |
selected_for_execution |      job_name       | comments

---------+-------------+--------------+-------------+----------------+-------------+------------------------
+--------------------+---------
-
       1 | p2          | public       | ilm_table_1 |                | TABLE       | SELECTED FOR EXECUTION | ilmjob
$_postgres1 |
(1 row)

gaussdb=# SELECT * FROM GS_MY_ILMEVALUATIONDETAILS;
 task_id | policy_name | object_owner | object_name | subobject_name | object_type |
selected_for_execution |      job_name       | comments

---------+-------------+--------------+-------------+----------------+-------------+------------------------
+--------------------+---------
-
```

 版权所有 © 华为云计算技术有限公司

```
      1 | p2          | public       | ilm_table_1 |               | TABLE       | SELECTED FOR EXECUTION | ilmjob
$_postgres1 |
(1 row)
```

13. 查询ADO JOB的执行详情信息，包含Task ID，JOB名称、JOB状态、JOB时间信息等。

```
gaussdb=# SELECT * FROM GS_ADM_ILMRESULTS;
 task_id |     job_name      |       job_state        |          start_time           |       completion_time         |
comments |                statistics
---------+-------------------+------------------------+-------------------------------
+-------------------------------+----------+-----------------------------------------------------
      1 | ilmjob$_postgres1 | COMPLETED SUCCESSFULLY | 2023-10-16 12:03:56.290176+08 |
2023-10-16 12:03:56.319829+08 |          | SpaceSaving=0,BoundTime=1697429033,LastBlkNum=40
(1 row)

gaussdb=# SELECT * FROM GS_MY_ILMRESULTS;
 task_id | job_name | job_state | start_time | completion_time | comments | statistics
---------+----------+-----------+------------+-----------------+----------+------------
(0 rows)
 task_id |     job_name      |       job_state        |          start_time           |       completion_time         |
comments |                statistics
---------+-------------------+------------------------+-------------------------------
+-------------------------------+----------+-----------------------------------------------------
      1 | ilmjob$_postgres1 | COMPLETED SUCCESSFULLY | 2023-10-16 12:03:56.290176+08 |
2023-10-16 12:03:56.319829+08 |          | SpaceSaving=0,BoundTime=1697429033,LastBlkNum=40
(1 row)
```

1.9 内存管理

Linux 内存机制

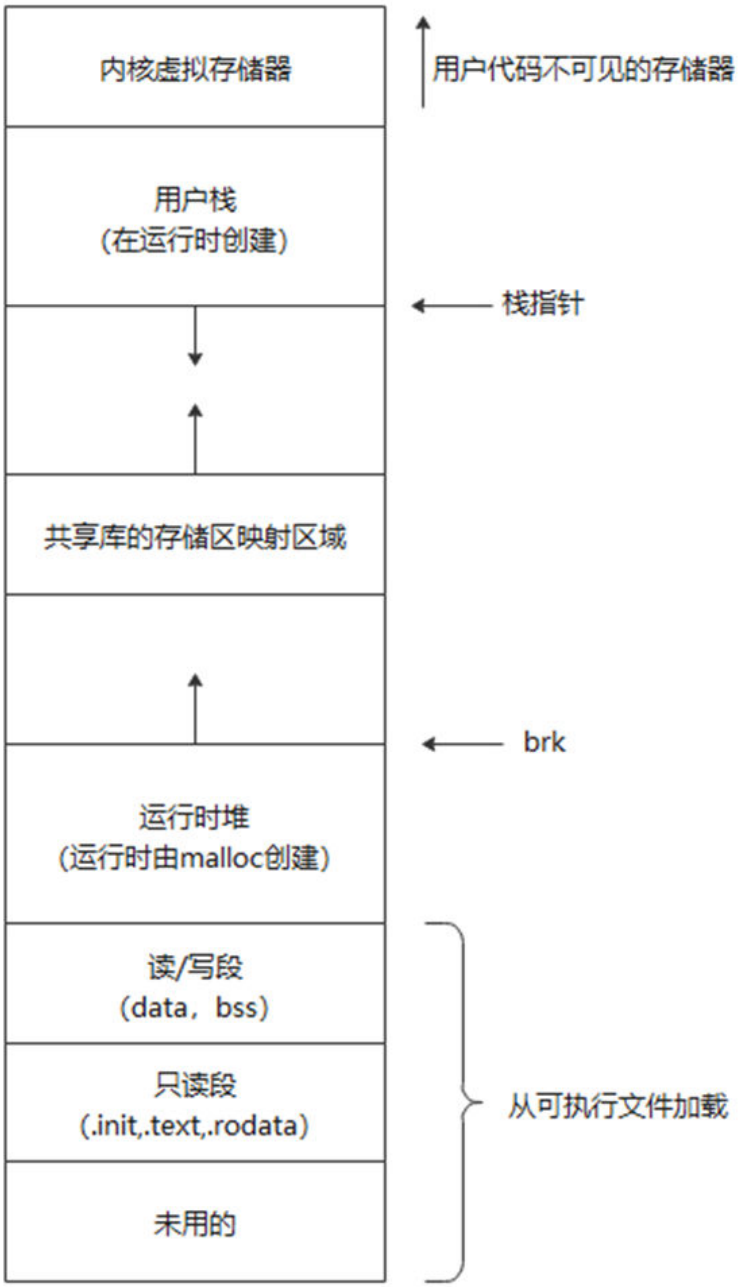
- 物理内存

  物理可见的硬件，即内存条。

- 虚拟内存

  进程运行时所有内存空间的总和，以32位系统为例，每个进程有4GB的内存空间，各进程的内存空间具有类似的结构。

版权所有 © 华为云计算技术有限公司



  - 一个新进程建立的时候，将会建立起自己的内存空间，并将此进程的数据、代码等从磁盘复制到自己的内存空间。进程中每一条数据的地址都由进程控制表中的task_struct记录。task_struct包含一张链表，用来记录内存空间的分配情况，哪些地址有数据、哪些地址无数据、哪些可读、哪些可写都可以通过这个链表记录。
  - 每个进程已经分配的内存空间，都与对应的磁盘空间存在映射关系。

    说明

    建议不要对进程虚拟内存大小进行限制。

- 物理内存地址和虚拟内存地址
  - 内存分页机制

版权所有 © 华为云计算技术有限公司

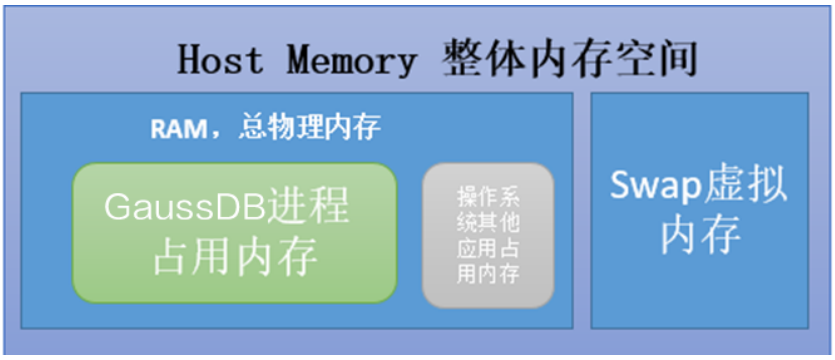
        - 虚拟内存中的页（page）和物理内存页帧（page frame）相同大小。
        - 页表（page table）维护虚拟内存页到物理内存页的映射。
        - 页面失效（page fault）功能：操作系统找到一个最少使用的页帧，让它失效，并把它写入磁盘，随后把需要访问的页放到页帧中，并修改页表中的映射。
      - 虚拟内存地址：页号（与页表中的页号关联）和偏移量（页的大小）组成。
      - 虚存寻址：页表中找到页帧号，若失效则调入页，页帧号及偏移量传给内存管理单元，组成一个物理存在的地址，即可访问物理内存数据。
  - Buffer
      - 根据磁盘的读写设计的，把分散的写操作集中进行，减少磁盘碎片和硬盘的反复寻道，从而提高系统性能。
      - Linux有一个守护进程定期清空缓冲内容（即写入磁盘），也可以通过sync命令手动清空缓冲。
  - Cache
      - 将读取过的数据保存起来，重新读取时若命中（找到需要的数据）就不需要读硬盘了，若没有命中就读硬盘。其中的数据会根据读取频率进行组织，把读取最频繁的内容放在最容易找到的位置，把不再读的内容不断往后排，直至从中删除。
  - OOM
      - 内存溢出（Out Of Memory，简称 OOM）是指应用系统中存在无法回收的内存或使用的内存过多，最终使得程序运行要用到的内存大于能提供的最大内存。此时程序无法运行，系统会提示内存溢出，有时会自动关闭软件，通过重启机器或软件释放掉一部分内存后又可以正常运行该软件，而由系统配置、数据流、用户代码等原因而导致的内存溢出错误，即使用户重新执行任务依然无法避免。

GaussDB 内存资源管理

- 内存管理简介
    - GaussDB的动态内存使用方法均基于内存上下文管理，在内存上下文的机制上，引入了逻辑内存管理机制，同时提供多项视图来追踪内存使用情况。

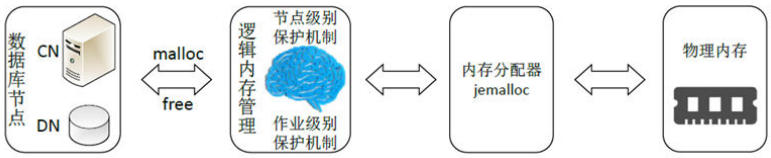
      当前GaussDB的内存管理分为两级，分别是：
        - 内存节点级别控制：通过max_process_memory参数限制CN、DN上可以使用的内存上限。
        - 内存作业级别控制：限制单条query语句最大可使用的内存上限。
- 节点级别内存控制
    - 在Linux操作系统中，内存空间通常由物理内存RAM和虚拟内存Swap组成。其中，物理内存由数据库程序和非数据库程序共同使用；而数据库程序的内存空间由GaussDB进程共同使用。

版权所有 © 华为云计算技术有限公司

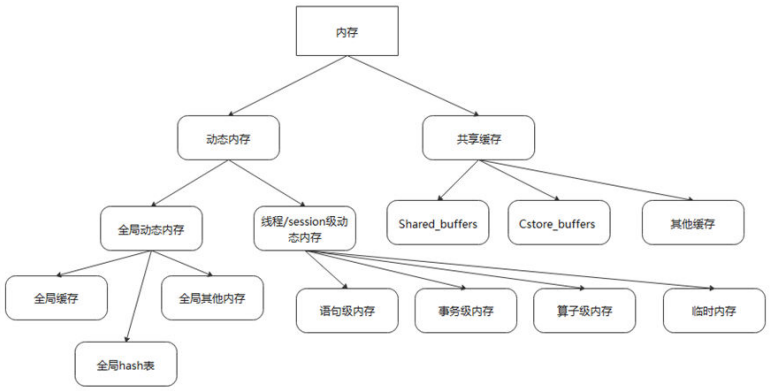


  - 由于非数据库程序也需要使用内存，一般建议GaussDB占系统可用内存的80%。

- 作业级别内存控制
  - 提供query_mem参数，根据能使用的内存量和能同时运行的作业数，指定每个作业执行时能够使用的内存量；提供query_max_mem参数，指定每个作业执行时能够使用的内存上限。
  - 提供work_mem参数，设置内部排序操作和Hash表在开始写入临时磁盘文件之前使用的内存大小；提供maintenance_work_mem参数，设置在维护性操作（比如VACUUM、CREATE INDEX、ALTER TABLE ADD FOREIGN KEY等）中可使用的最大内存。
- 逻辑内存管理机制
  - GaussDB内存管理是通过逻辑内存管理来实现的。其实现原理是在原始的内存分配之上增加一层逻辑内存管理，通过检查已分配内存是否超过规定的内存来决定是否为作业分配内存。逻辑内存管理没有更改原有的内存资源分配机制，仅在分配内存之前增加一个逻辑判断层，查看是否达到允许使用的内存上限，来决定是否分配内存。



- 内存分类



GaussDB的内存使用分类如上所述，其每种类型的说明如下：

  - 动态内存
    - 全局动态内存

 版权所有 © 华为云计算技术有限公司

        - 全局缓存：全局缓存（Cache）用来提高访问效率，Cache中包括一个系统表元组Cache和一个表模式信息Cache（RelCache），SysCache中存放的是最近使用过的系统表的元组，而RelCache中包含所有最近访问过的表的模式信息（包含系统表的信息）。RelCache中存放的不是元组，而是RelationData数据结构，每一个RelationData结构表示一个表的模式信息，这些信息都由系统表元组中的信息构造而来。
        - 全局hash表：保存全局的统计信息或其他数据的内存hash结构。
        - 全局其他内存：保存全局变量等内容。
      - 线程/session级动态内存
        - 语句级内存：生命周期为整个SQL语句执行阶段，语句执行前内存被申请，执行结束后内存被回收释放。
        - 事务级内存：生命周期为整个事务内，事务开始前内存被申请，事务结束后内存被回收释放。
        - 算子级内存：生命周期为单个算子有效期内，算子执行结束后内存被释放回收。
        - 临时内存：一些临时变量申请的内存，使用后及时释放。
    - 共享缓存
      - Shared_buffers

        缓存从磁盘读取的数据所使用的内存。
      - 其他缓存

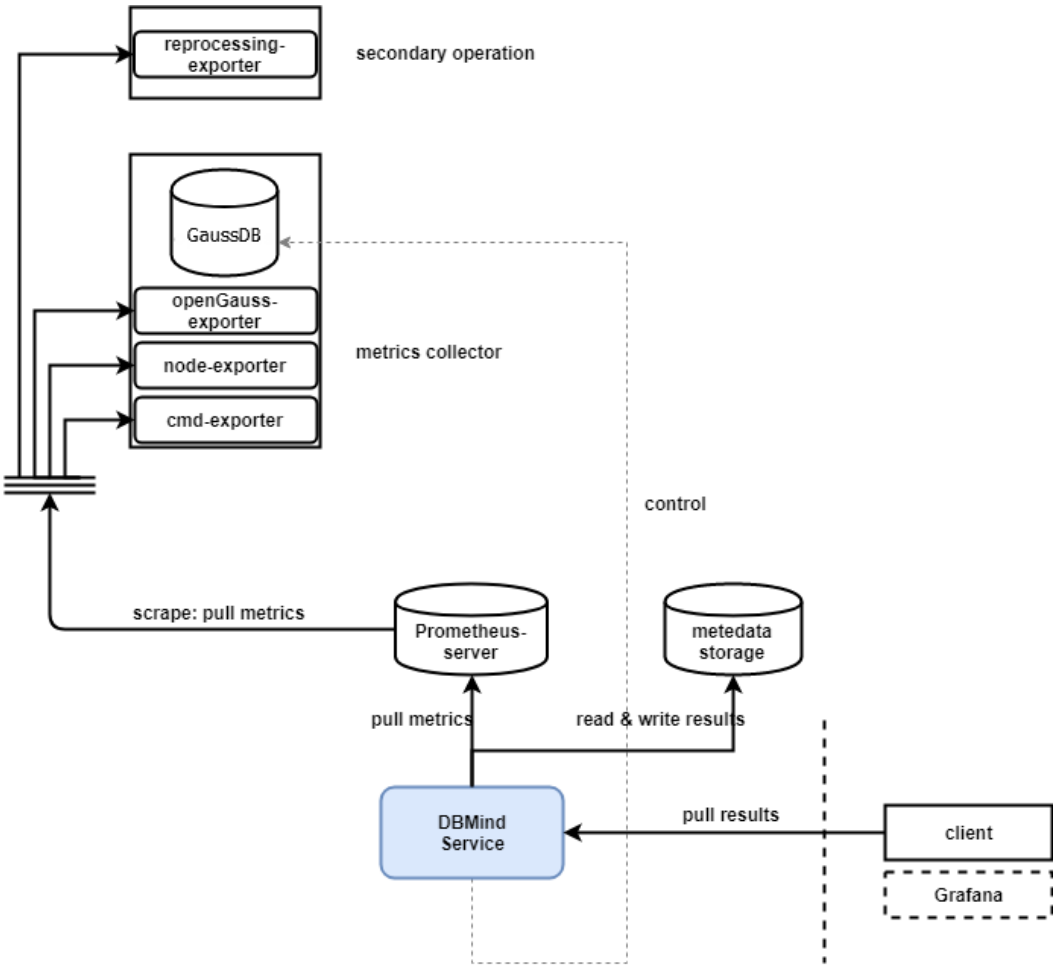
        线程所使用的槽位，锁等资源占用的内存。
- 内存上下文机制
  - 内存上下文介绍
    - 概念：内存上下文用来统一控制内存的申请和释放，内存上下文中记录了其所申请的总内存大小，每一处内存申请的大小，申请内存所在的文件，行号等信息以及与其他内存上下文之间的关联信息。
    - 目的：基于数据库中作业执行逻辑，通过内存上下文管理机制，提升查询内存分配、释放效率。
  - 逻辑内存管理
    - 通过数据库进程内部记账，控制单进程内存使用的上限。
    - 可统计进程总体使用、共享和非共享内存上下文使用及重点模块内存使用（通信库、pooler等）。
    - 逻辑内存的生效机制：enable_memory_limit设置为on，并且max_process_memory减掉缓存等其他预留内存 > 2GB，具体详情请联系管理员。
    - PV_TOTAL_MEMORY_DETAIL视图用于查询数据库进程的内存使用情况，便于分析内存失败后问题，具体字段可参考《开发者指南》中“系统表和系统视图 > 系统视图 > PV_TOTAL_MEMORY_DETAIL”章节。

版权所有 © 华为云计算技术有限公司

1.10 DBMind: 数据库自治运维

DBMind承载了GaussDB的AI for Database特性，主要用于对数据库进行自治运维和管理，从而帮助数据库运维人员减少运维工作量。在实现上，DBMind具有监控和服务化的性质，同时也提供即时AI工具包，提供开箱即用的AI自治功能（如：索引推荐）。DBMind的监控平台以开源Prometheus为主，并提供了数据生产者exporter，可与Prometheus平台完成对接。DBMind服务架构如下图所示：

图 1-8 DBMind 服务架构



图中各关键组件说明：

- DBMind Service：DBMind后台服务，可用于定期离线计算，包括慢SQL根因分析、时序预测等；
- Prometheus-server：存储Prometheus监控指标的服务器；
- metadatabase：DBMind在离线计算结束后，将计算结果存储在此处，支持GaussDB、SQLite等数据库；
- client：用户读取DBMind离线计算结果的客户端，目前该客户端仅支持命令行操作；若采用GaussDB等数据库存储DBMind的计算结果，则用户可以自行配置Grafana等可视化工具，用于对该结果进行可视化；

版权所有 © 华为云计算技术有限公司

- openGauss-exporter：用户从GaussDB数据库节点上采集监控指标，供DBMind服务进行计算；
- node-exporter：Prometheus官方提供的exporter，可用于监控对应节点的系统指标，如CPU和内存使用情况；
- cmd-exporter：在用户安装数据库的环境上执行命令行，并采集该命令行的执行结果，同时，也尝试将数据库日志内容转化为监控指标；例如通过执行cm_ctl命令，查看数据库集群的状态；
- reprocessing-exporter：用于对Prometheus采集到的指标进行二次加工处理，例如计算CPU使用率等。

DBMind 支持的场景规格

- DBMind目前不支持在元数据库变更的情况下保留原始诊断数据；
- DBMind不支持纳管容灾集群中的备节点；
- DBMind支持纳管多集群，但是随着纳管集群节点越多，消耗的资源也对应增多。需要在DBMind消耗的资源过多时，停止纳管更多节点，以避免CPU和内存资源不足。
- DBMind实例纳管建议为1个CPU核对应3个节点（例如3个节点（包含主备）的集中式则需要1个CPU核，9个节点（包含CN，DN等）的分布式需要3个CPU核），如果纳管的实例节点数太多，会导致DBMind服务接口超时或服务异常。
- DBMind只支持文档中写明的场景，不支持文档中没有明确的场景、示例中不包括的部署形态、使用方法等超规格使用形式的情况。
- DBMind服务当前不支持IPv6。
- DBMind在M-Compatibility数据库下仅支持异常检测与根因分析、实例故障诊断功能，不支持其他兼容形态。

说明

云侧网络情况和DBMind云上部署说明：

- 云上的实例节点拥有多层网络结构，分布式节点拥有三层结构：管理层（CM）、数据层（CN）和数据同步层（DN）。
- 对DBMind主程序部署情况的说明如下：
  - 启动prometheus，监听在数据层ip。
  - 启动reprocessing_exporter，监听在数据层ip。
  - 启动DBMind，暴露webservice在管理层ip（管理ip是因为接口调用需要走管理面发起，需要保证管理面网络互通）。
- 对于被纳管的数据库实例，插件部署情况如下：
  - node_exporter、cmd_exporter、opengauss_exporter都监听在数据层ip。
  - 对于仲裁、冷备、offline的节点，只会部署node_exporter、cmd_exporter。
  - 分布式如果节点上无CN组件，只会部署node_exporter、cmd_exporter。
  - 其他情况均会部署node_exporter、cmd_exporter、opengauss_exporter。
- dbmind指标采集主要为本身功能服务，在采集过程中可能存在网络故障、采集超时等异常，另外prometheus本身采集机制为通过标签区分不同指标，因此不能保证采集指标的连续性，如果用户需要将指标用于前端展示，在指标不连续的情况下需要自己补齐。

环境配置

DBMind外置AI功能需要运行在Python 3.7版本以上，当前DBMind独立出包并且包中已包含工具所有三方依赖项，因此用户不用单独安装。如果用户需要自己配置依赖

 版权所有 © 华为云计算技术有限公司

项，DBMind需要的第三方依赖包记录在AI功能根目录的requirements.txt文件中（包括requirements-x86.txt与requirements-arrch64.txt，用户可根据自己平台类型选择）中，可以通过pip install命令安装依赖，命令如下：

```
pip3 install -r requirements-x86.txt
```

如果用户没有安装齐全所需的依赖，则当用户执行gs_dbmind命令时，会再次提醒用户安装第三方依赖。需要注意，该文件提供了DBMind所需的第三方依赖，若用户环境存在第三方包冲突等情况，可由用户根据实际情况进行处理。

如果pip版本过低，可能会出现装包失败的情况，此时可使用以下命令对pip进行升级：

```
pip3 install --upgrade pip
```

1.10.1 DBMind 模式说明

用户可通过gs_dbmind命令调用DBMind下列基本功能：

- 服务功能：service子命令，包括创建并初始化配置目录、启动后台服务、关闭后台服务等。
- 调用组件：component子命令（如索引推荐、参数调优等）可通过该模式进行调用。
- 设置参数：set子命令，通过该命令，可以一键修改配置目录中的配置文件值；用户也可以通过文本编辑器进行手动修改。

用户可以通过 --help 选项获得上述模式的帮助信息，例如：

```
gs_dbmind --help
usage: [-h] [-v] {service,set,component} ...

openGauss DBMind: An autonomous platform for openGauss

optional arguments:
  -h, --help            show this help message and exit
  -v, --version         show program's version number and exit

available subcommands:
  {service,set,component}
                        type '<subcommand> -h' for help on a specific subcommand
    service             send a command to DBMind to change the status of the service
    set                 set a parameter
    component           pass command line arguments to each sub-component.
```

表 1-12 gs_dbmind 选项基本说明

参数	参数说明	取值范围
-h, --help	帮助命令	-
--version	版本号	-
service	服务功能相关的子命令	-
component	调用组件的子命令	-
set	修改配置文件的子命令	-

版权所有 © 华为云计算技术有限公司

1.10.1.1 service 子命令

该子命令可用于对配置目录进行初始化，同时也可以实现启动和停止后台任务。

配置目录初始化

用户可通过“gs_dbmind service setup”子命令进行配置目录的初始化。该配置文件中可包括DBMind的配置文件、日志等内容。该目录中的部分文件说明：

- dbmind.conf：DBMind的参数配置文件，用户可通过“gs_dbmind set”命令进行修改，也可通过文本编辑器进行手动修改。
- dynamic_config.db：DBMind服务保存在本地节点的元信息，主要包括算法的超参数、监控阈值等；该文件为DBMind服务元信息，不可由用户直接配置。
- metric_map.conf：监控指标映射表，可用于适配到不同采集平台中。例如，在DBMind中，监控到的系统cpu使用率名为os_cpu_usage，而用户自行实现的指标采集工具将cpu使用率命名为my_cpu_usage_rate。则在该种情况下，如果想要DBMind中代表cpu使用率的指标名为my_cpu_usage_rate，则需要修改该配置选项。即添加“os_cpu_usage = my_cpu_usage_rate”配置项进行映射。对于普通用户，建议直接使用DBMind配套的采集组件和方案，则不涉及修改该配置文件。
- metric_value_range.conf：指定某些具体监控指标的上下限，以防止对该指标进行预测时，超过合理区间；未在该列表中的指标范围默认为[0, +∞)。
- dbmind.pid：该文件保存正在使用该DBMind配置目录的进程PID。
- dbmind.children.pid：该文件保存DBMind子进程的PID信息，pid之间用逗号隔开。
- VERSION：生成该配置文件目录的DBMind版本，通过验证版本来避免版本不兼容情况下的使用。
- logs：该目录用于存储DBMind服务产生的日志，请不要手动修改该目录下的日志文件，否则可能会导致日志变为不可写状态，无法继续更新DBMind操作记录。
- backtrace.stack：DBMind自身用于定位调试产生的临时文件，包含了当前各个线程的调用栈信息，是通过接收到SIGUSR2信号而产生的。

用户可通过两种方式进行配置目录的初始化，一种为交互式，另一种为非交互式。例如，待初始化的配置目录名为confpath，则分别通过下述方法进行配置：

交互式模式

```
gs_dbmind service setup -c confpath --interactive
```

执行完毕上述命令后，用户可通过命令行终端对配置项进行交互式配置。

非交互式模式

非交互式模式总共分为三个步骤，即创建配置文件，修改配置项，初始化配置。其中第二个步骤需要用户通过文本编辑器手动编辑配置文件。具体步骤如下：

步骤1 创建配置文件，执行下述命令：

```
gs_dbmind service setup -c confpath
```

步骤2 执行完上述命令后，会在confpath目录下生成dbmind.conf配置文件，用户需要利用文本编辑器进行手动修改。相关参数的说明如下：

```
# METADATABASE部分用于指定DBMind生成的分析结果的存储位置。
# 当前支持的数据库类型有SQLite。使用时注意Python驱动psycopg2的兼容性问题，用户可以通过自行编译或修改GUC参数进行适配。
# 其他信息为连接到该数据库的连接信息，注意用户需要有数据库创建权限。
```

 版权所有 © 华为云计算技术有限公司

```
[METADATABASE]
dbtype = sqlite # 元数据库类型，选项：sqlite，postgresql
host = # 元数据库IP地址，如果配置多个元数据库地址，需要用逗号（,）分隔。
port = # 元数据库端口，如果配置多个元数据库地址，且port相同，填写一个port即可。如果port不相同，port数量需要和host数量一致，用逗号（,）分隔。
username = # 元数据库用户名
password = (null) # 元数据库用户密码
database = # 元数据库名

# WORKER用于指定DBMind可以使用的worker子进程数量，如果写0则会进行自适应，即尽可能多地使用CPU资源。
[WORKER]
process_num = 0  # 本地节点上的工作进程数，小于或等于零表示自适应。

# AGENT部分用于指定DBMind连接到openGauss Agent的信息。通过使用该Agent，可以让DBMind获取到被监控实例的即时状态，从而提高分析准确性。同时，也可以向数据库实例下发一些变更动作，如结束某条慢SQL语句（这取决于此处配置的用户是否有足够的权限）。
# 该master_url地址即为Agent的地址，由于openGauss-exporter承担了Agent的角色，故该地址也就是openGauss-exporter 的地址。
# 同时，openGauss-exporter是支持HTTPS协议的，所以，此处也可以根据配置指定SSL证书。
[AGENT]
master_url = # 不配置该参数，即可启动自动发现模式，此时所有注册在TSDB中的代理都会被找到，但是需要配置一套统一的用户名/密码/SSL连接信息。否则，您可以键入主节点的代理URL，例如，https://127.0.0.1:9187，如果您有多组集群，则需要用逗号（,）分隔它们。
username =  # 登录监控数据库的用户名。代理凭证。如果配置监控多套集群，且这些集群的用户名和密码不同，则需要输入多个配置用户名，用逗号（,）分隔，需要和上面的master_url选项一一对应。
password = (null) # 登录监控数据库的密码。代理凭证。如果配置监控多个集群，需要用逗号（,）分隔。具体配置方法同上述参数。
ssl_certfile = (null) # SSL连接的证书文件路径。这是可选的。如果配置监控多个集群，需要用逗号（,）分隔。具体配置方法同上述参数。
ssl_keyfile = (null) # SSL连接的私钥文件路径。这是可选的。如果配置监控多个集群，需要用逗号（,）分隔。具体配置方法同上述参数。
ssl_keyfile_password = (null) # SSL私钥文件的密码。这是可选的。如果配置监控多个集群，需要用逗号（,）分隔。具体配置方法同上述参数。
ssl_ca_file = (null)  # 用于验证请求的CA证书文件路径。这是可选的。如果配置监控多个集群，需要用逗号（,）分隔。具体配置方法同上述参数。

TIMED_TASK表示后台任务配置，通过修改参数启动或停止DBMind运行的后台任务以及任务的运行间隔，单位为秒。
运行间隔不能低于30秒，最大值为python3支持的整型数字上限,2的63次方减1
[TIMED_TASK_LIST]
explanation = Configure scheduled tasks including start tasks and intervals # 配置计划任务，包括启动任务和间隔
current_support_task_list = List of currently supported tasks:  #当前支持的任务列表
task1 = anomaly_detection          # 1. 指标异常检测
task2 = discard_expired_results    # 2. 清理过期数据
task3 = knob_recommend             # 3. 参数推荐
task4 = slow_query_killer          # 4. 慢SQL查杀
task5 = slow_query_diagnosis       # 5. 慢SQL诊断
task6 = cluster_diagnose           # 6. 集群故障故障诊断
task7 = agent_update_detect        # 7. 当检测到代理更新时自动更新异常检测器
task8 = calibrate_security_metrics # 8. 校准security_scenarios.yml提到的安全指标
task9 = check_security_metrics     # 9. 检查security_scenarios.yml提到的安全指标
task10 = update_statistics         # 10. 自动更新指定指标的统计信息

TIMED_TASK表示后台任务配置，通过修改参数启动或停止DBMind运行的后台任务以及任务的运行间隔，单位为秒。
运行间隔不能低于30秒，最大值为python3支持的整型数字上限，2的63次方减1。
[TIMED_TASKle
task = discard_expired_results,anomaly_detection,cluster_diagnose, agent_update_detect,
calibrate_security_metrics, check_security_metrics, update_statistics  # 默认reload启动数据清理、异常检测、集群故障诊断、更新异常检测器、校准security_scenarios.yml指标、检查security_scenarios.yml指标和更新指定指标的统计信息任务
anomaly_detection_interval = 180  # 异常检测任务运行间隔，默认180秒
slow_query_diagnosis_interval = 120  # 慢SQL诊断任务运行间隔，默认120秒
knob_recommend_interval = 3600  # 参数推荐任务运行间隔，默认一小时
slow_query_killer_interval = 30  # 慢SQL查杀任务运行间隔，默认30秒
cluster_diagnose_interval = 30  # 集群故障诊断运行间隔，默认30秒
discard_expired_results_interval = 3600  # 数据清理任务运行间隔，默认一小时
```

 版权所有 © 华为云计算技术有限公司

```
agent_update_detect_interval = 30  # 检测agent更新并自动并自动更新异常检测器间隔，默认30秒
calibrate_security_metrics_interval = 600 # 校准security_scenarios.yml安全指标间隔，默认10分钟
check_security_metrics_interval = 600 # 检查security_scenarios.yml安全指标间隔，默认10分钟
update_statistic_interval = 1800  # 自动更新指定指标统计信息间隔，默认30分钟

# WEB-SERVICE用于DBMind的前台页面展示
[WEB-SERVICE]
ssl = true # 默认使用安全的协议，用户需要提供证书路径，否则DBMind服务不能启动
host = 127.0.0.1  # DBMind-service监听地址
port = 8080  # DBMind-service监听端口
ssl_certfile = (null) # SSL连接的证书文件路径
ssl_keyfile = (null) # SSL连接的私钥文件路径
ssl_keyfile_password = (null) # SSl私钥文件的密码，如果没有密码则忽略
ssl_ca_file = (null)  # 用于验证请求的CA证书文件路径

# LOG表示设置DMBind的日志记录信息。
[LOG]
maxbytes = 10485760 # 默认值为10Mb。单个日志文件的最大大小。如果maxbytes为零，文件将无限增长
backupcount = 1 # 日志文件备份数量
level = INFO  # 日志级别，选项：DEBUG，INFO，WARNING，ERROR
log_directory = logs # 日志文件存放的目录.

# 下列内容表示给用户进行交互配置时的提示信息，用户无需配置。
[COMMENT]
worker = The form of executing compute-intensive tasks. Tasks can be executed locally or distributed to
multiple nodes for execution.
tsdb = Configure the data source for time series data, which come from monitoring the openGauss instance.
metadatabase = Configure the database to record meta-data, which the database can store meta-data for
the forecasting and diagnosis process. The database should be an openGauss instance.
self-monitoring = Set up parameters for monitoring and diagnosing openGauss instance.
self-optimization = Set up parameters for openGauss optimization.

[IP_MAP]
ip_map = (null) # 对于数据ip和管理ip不一致的环境，需要提供对应关系。以下以一主两备集群为例：
“primary_data_ip:primary_management_ip,standby1_data_ip:standby1_management_ip,standby2_data_ip:st
andby2_management_ip”
```

步骤3 待用户手动修改完上述参数后，需要执行下述命令进行配置项的初始化。在该阶段中，DBMind会初步检查配置项的正确性、初始化用于存储结果数据的元数据库表结构和内容，同时也加密配置项中出现的密码。

```
gs_dbmind service setup --initialize -c confpath
```

步骤4 完成配置目录初始化过程，可基于该配置目录启动DBMind后台服务。

----结束

说明

1. 配置文件注释信息用于在交互模式下对用户进行提示，有特殊含义不要手动修改或删除；
2. 需要确保配置项的值与注释信息之间通过空格符分割，否则系统会将注释信息识别为配置项的值；
3. 配置项中的特殊字符，如果需要转义，则通过转义符“百分号”（%）来转义，例如，用户配置的密码为 "password%"，则应通过“百分号”进行转义，即 "password%%";
4. DBMind Service对外以标准RESTfulAPI的方式提供接口，其默认支持HTTPS通信，建议用户使用。
5. DBMind云侧部署主机目前最小规格8U64G，目前云管控支持容灾主集群纳管，不支持容灾备集群纳管。

启动服务

当用户完成配置目录的初始化后，可基于此配置目录启动DBMind后台服务。例如配置目录为confpath，则启动命令如下：

```
gs_dbmind service start -c confpath
```

 版权所有 © 华为云计算技术有限公司

当执行上述命令后，会提示服务已启动。在未指定任何附加参数时，该命令默认会启动所有的后台任务。如果用户只想启动某一个后台任务，需要添加“--only-run”选项。例如，用户只想启动慢SQL根因分析服务，则为：

```
gs_dbmind service start -c confpath --only-run slow_query_diagnosis
```

DBMind运行过程中如果想启或停止定时任务，首先将配置文件TIMED_TASK下task中相关功能删除，在执行以下命令，此时DBMind会刷新后台定时任务：

```
gs_dbmind service reload -c confpath
```

由于DBMind是一个在后台定期执行的服务，故如果用户只是想跑一次DBMind服务进行测试或调试，则可以通过添加“--dry-run”选项，如：

```
gs_dbmind service start -c confpath --dry-run
```

“--dry-run”选项可以与“--only-run”选项合用。

如果当前进程已经在跑，用户希望重启启动该DBMind服务，则可以使用重启命令，即：

```
gs_dbmind service restart -c confpath
```

默认情况下，restart会等待当前DBMind正在执行的任务执行完毕后才会退出并重启服务。如果用户希望强行终止当前正在运行的任务，则可以添加“-f”或“--force”选项，例如：

```
gs_dbmind service restart --force -c confpath
```

关闭服务

关闭服务与启动服务类似，其命令行结构更加简单，只需指定配置目录的地址即可。例如配置目录为confpath，则为：

```
gs_dbmind service stop -c confpath
```

DBMind服务会在后台执行完正在运行的任务后自行退出，如需要强行退出，则使用“--force”或“-f”选项。

DBMind 的高可用

⚠ 注意

DBMind高可用功能需要在未纳管实例的情况下实现，因此接口不需要token认证。应当使用默认的HTTPS协议以进行SSL双向认证，使用HTTP协议则可能引入攻击者直接发起未授权请求的安全风险。

为保证DBMind云侧使用时的高可靠，DBMind对外提供了服务状态查询和部分异常修复的接口，详细请参见**表1-20**。

- DBMind服务状态查询接口示例：

  ```
  curl -X 'POST'"http://127.0.0.1:8080/v1/api/check-status" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"cmd": "./gs_dbmind service start -c dbmindconf "}'
  ```

  如果使用HTTPS协议，则查询示例为：

  ```
  curl -X 'POST'"https://127.0.0.1:8080/v1/api/check-status" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"cmd": "./gs_dbmind service start -c dbmindconf "}' --cacert xx.crt --key xx.key --cert xx.crt
  ```

  返回结果示例：

版权所有 © 华为云计算技术有限公司

```
{"data":{"error_msg":"","result":{},"state":"NORMAL"},"success":true}
```

- DBMind异常修复接口示例：

```
curl -X 'POST'"http://127.0.0.1:8080/v1/api/repair" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"cmd": "./gs_dbmind service start -c dbmindconf "}'
```

如果使用HTTPS协议，则修复示例为：

```
curl -X 'POST'"https://127.0.0.1:8080/v1/api/repair" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"cmd": "./gs_dbmind service start -c dbmindconf "}' --cacert xx.crt --key xx.key --cert xx.crt
```

返回结果示例：

```
{"data":{"error_msg":"","result":{},"state":"SUCCESS"},"success":true}
```

说明

当前DBMind服务支持查询的异常场景如下：

- PID文件丢失或文件内容异常：当用户启动服务/组件后，会自动生成“*.pid”文件记录进程对应的进程标识符，也就是PID。PID文件用于云管控判断进程状态，发现进程级别的异常并进行修复。为防止PID文件的误删除与内容的误修改，需要定期判断PID文件是否存在以及其内容是否正确。
- 日志文件丢失：当用户启动服务/组件后，会自动生成*.log文件来记录进程运行的日志。此日志文件用于快速定位问题的根源、追踪程序执行的过程等。为防止日志文件的误删除，需要定期判断日志文件是否存在。
- 资源占用异常：DBMind服务会执行索引推荐、指标分析、智能巡检等功能，需要占用系统资源；exporter组件会采集服务器上的指标，用于后续分析，也会占用系统资源；当服务/组件的CPU或内存占用超过阈值时，说明资源占用过高，服务/组件出现异常。因此，需要监控服务和组件的资源占用情况。
- 元数据库主备切换：DBMind支持数据库的主备切换功能，在主服务器发生故障等条件下，自动将备份服务器切换为主服务器，确保系统的持续运行。当数据库出现主备切换时，需要DBMind能够正确识别并访问对应的服务器，需要定期判断数据库是否发生了主备切换。如果配置文件中只配置一个元数据库的ip，不支持主备切换。
- 元数据库异常：DBMind需要将配置信息、检测到的异常结果、故障信息等内容进行持久化，存储在元数据库中，因此需要确保元数据库的正常运行；如果元数据库发生异常，则可能会导致依赖元数据库存储结果的任务无法向其插入数据，导致数据遗漏。

资源占用阈值及告警设置如下：

- 内存占用阈值：max(总内存的1%, 200MB)。
- CPU多核占用阈值：cmd_exporter 10%，opengauss_exporter 10%，reprocessing_exporter 10%， DBMind 80%。
- CPU单核占用阈值：cmd_exporter 50%，opengauss_exporter 50%，reprocessing_exporter 50%，DBMind不做限制。
- 调用高可用接口会检查资源占用，连续超过3次阈值才会触发资源占用告警。

风险：当占用资源连续超过阈值三次时，会触发告警，管控侧会重启服务，导致正在运行的任务中断。

当前针对DBMind服务异常的修复性说明如下：

- PID文件丢失：可自动修复；
- 日志文件丢失：可自动修复；
- 组件资源占用异常：不可通过接口修复；
- 元数据库主备切换：可自动修复；
- 元数据库异常：不可通过接口修复；
- 针对Exporter组件不可修复的异常，需要云侧进行处理，如重启进程。

版权所有 © 华为云计算技术有限公司

⚠ 注意

- [METADATABASE]中的元数据库用户需要具有在该数据库下的创表和数据插入更新权限，否则执行时会出现异常。而且，需要用户提前创建好数据库，否则，会提示无法连接到该数据库的错误。
- DBMind尝试为每个配置文件目录启动一个DBMind实例，并通过PID文件记录DBMind实例PID，不支持同一配置文件下启动多个服务。
- DBMind提供了requirement.txt文件，用户可以通过该文件安装所需的第三方依赖。
- DBMind只支持特定场景的异常修复，对于不能修复的异常则需要云侧进行处理。
- DBMind支持元数据库主备切换场景的自动修复，但在修复期间对元数据库的业务操作会执行失败，导致数据无法正常插入、查询或更新。
- 如果用户在配置目录的dbmind.conf中填写了AGENT下的master_url，当调用**DBMind的AI子功能**中/v1/api/agents接口更新agent时，如果参数force=true时则不支持更新，该参数只有mater_url没有填写时支持（会自动查找）。
- DBMind当前使用进程池执行特性功能，因此接口侧的响应时间与进程池参数设置和环境等有关，比如用户在confpath目录下的dbmind.conf配置文件中process_num设置不合理或硬件性能较差等，使进程池负载较重导致**DBMind的AI子功能**中部分接口超时，比如/v1/api/risk-analysis/{metric}。

命令参考

用户可以通过“--help”选项获得该模式的帮助信息，例如：

```
gs_dbmind service --help
usage:  service [-h] -c DIRECTORY
              [--only-run
{discard_expired_results,anomaly_detection,cluster_diagnose,agent_update_detect,knob_recommend,slow_query_killer,slow_query_diagnosis}]
              [--dry-run] [-f] [--interactive | --initialize]
              {setup,start,stop,restart,reload}}

positional arguments:
  {setup,start,stop,restart,reload}}
                        perform an action for service

optional arguments:
  -h, --help            show this help message and exit
  -c DIRECTORY, --conf DIRECTORY
                        set the directory of configuration files
  --only-run
{discard_expired_results,anomaly_detection,cluster_diagnose,agent_update_detect,knob_recommend,slow_query_killer,slow_query_diagnosis}
                        explicitly set a certain task running in the backend
  --dry-run             run the backend task(s) once. the task to run can be
                        specified by the --only-run argument
  -f, --force           force to stop the process and cancel all in-progress
                        tasks
  --interactive         configure and initialize with interactive mode
  --initialize          initialize and check configurations after configuring.
```


版权所有 © 华为云计算技术有限公司

表 1-13 gs_dbmind service 子命令说明

参数	参数说明	取值范围
action	动作参数	• setup：初始化配置目录。 • start：启动服务。 • restart: 重启服务。 • reload: 重新加载配置文件参数。 • stop：停止服务。
-c, --conf	配置文件目录地址	-
--initialize	配置参数初始化	-
--interactive	交互式输入配置参数	-
--only-run	选择只运行的模块	• slow_query_diagnosis：慢SQL根因分析模块。 • anomaly_detection：异常检测模块。 • slow_query_killer：慢SQL查杀。 • cluster_diagnose：集群故障诊断。 • agent_update_detect：代理更新。 • 下列模块也可输入，但是DBMind内部模块使用，无命令行获取接口，对用户透明：knob_recommend、discard_expired_results。
--dry-run	只跑一次DBMind的任务，任务执行完毕后退出	-
-f, --force	强行退出当前正在执行的任务，可用于restart和stop命令中	-
-h, --help	帮助命令	-

1.10.1.2 component 子命令

该子命令可以用于启动DBMind的子组件（或插件），包括可用于监控指标的exporter，以及AI功能等。该命令可以将用户通过命令行传入的命令转发给对应的子组件，故不同的子组件命令需参考其功能的对应说明，详见后文各个子组件对应章节，此处不再赘述。

版权所有 © 华为云计算技术有限公司

命令参考

用户可以通过“--help”选项获得该模式的帮助信息，例如：

```
gs_dbmind component --help
usage:  component [-h] COMPONENT_NAME ...

positional arguments:
  COMPONENT_NAME  choice a component to start. ['anomaly_detection',
                  'cluster_diagnosis', 'cmd_exporter', 'dkr',
                  'extract_log', 'index_advisor', 'opengauss_exporter',
                  'reprocessing_exporter', 'slow_query_diagnosis',
                  'sql_rewriter', 'sqldiag', 'xtuner']
  ARGS            arguments for the component to start

optional arguments:
  -h, --help      show this help message and exit
```

表 1-14 gs_dbmind component 子命令说明

参数	参数说明	取值范围
COMPONENT_NAME	子组件（即插件）名	'anomaly_detection', 'cluster_diagnosis', 'cmd_exporter', 'dkr','extract_log', 'index_advisor', 'opengauss_exporter', 'reprocessing_exporter', 'slow_query_diagnosis', 'sql_rewriter', 'sqldiag', 'xtuner'
ARGS	子组件的参数	参考子组件的命令说明
-h, --help	帮助命令	-

1.10.1.3 set 子命令

该命令用于修改配置文件dbmind.conf中的参数值，与用户手动修改配置文件dbmind.conf一般无差异。例如修改配置目录confpath中的配置文件dbmind.conf中TSDB配置部分，host参数的值，并将其设置为127.0.0.1。则可通过下述命令实现：

```
gs_dbmind set TSDB host 127.0.0.1 -c confpath
```

在修改上述普通参数时，与手动修改配置文件dbmind.conf无差异。但由于DBMind的配置文件中不保存明文密码（如果用户使用明文密码，则DBMind会提示并退出），故当用户想要修改密码项时，有两种方法进行修改，一种是先修改dbmind.conf，并通过以下命令实现配置文件的重新初始化：

```
gs_dbmind service setup --initialize -c confpath
```

另一种方法则是直接通过set子命令进行设置，如：

```
gs_dbmind set METADATABASE password xxxxx -c confpath
```


版权所有 © 华为云计算技术有限公司

说明

1. 该命令对于字符串是大小写敏感的，如果输错则可能出现执行过程错误；
2. 由于set子命令涉及的参数值类型很多，故只会对设置值进行初步检查，用户需要保证输入值的内容正确，如某些值应为正整数而非负数。

命令参考

用户可以通过“--help”选项获得该模式的帮助信息，例如：

```
gs_dbmind set --help
usage:  set [-h] -c DIRECTORY section option target

positional arguments:
  section               which section (case sensitive) to set
  option                which option to set
  target                the parameter target to set

optional arguments:
  -h, --help            show this help message and exit
  -c DIRECTORY, --conf DIRECTORY
                        set the directory of configuration files
```

表 1-15 gs_dbmind set 子命令说明

参数	参数说明	取值范围
-h, --help	帮助命令	-
-c，--conf	配置文件目录confpath	-
section	设置区	-
option	设置项	-
target	设置值	-

1.10.1.4 upgrade 子命令

该子命令用于升降级DBMind配置文件和元数据库，DBMind整体升降级还需配合DBMind主服务与exporter组件重启等操作，具体步骤如下（以配置文件路径confpath为例）：

步骤1 在当前版本DBMind文件路径下，停止DBMind服务。

```
gs_dbmind service stop -c confpath
```

步骤2 在高版本DBMind文件路径下，升级配置文件和元数据库。

```
gs_dbmind upgrade -c confpath --target dbmindpath
```

步骤3 查询reprocessing_exporter、cmd_exporter、opengauss_exporter的进程号并重启。以reprocessing_exporter为例。

1. 查询进程号。
   ```
   ps ux | grep reprocessing_exporter
   ```
2. 终止进程。
   ```
   kill -9 pid
   ```
3. 使用目标版本DBMind启动进程，详细见**component子命令**。
   ```
   gs_dbmind components reprocessing_exporter
   ```


版权所有 © 华为云计算技术有限公司

步骤4 在目标版本DBMind文件路径下，启动DBMind服务。

```
gs_dbmind service start -c confpath && gs_dbmind service reload -c confpath
```

----结束

须知

- 升降级前需确认DBMind服务的API接口正常，否则，DBMind无法升级或升级后API接口不可用。
- 配置文件与元数据库升降级过程中，如果出现元数据无法连接等异常，会导致升级失败，此时，相关文件会自动回滚，如果回滚失败，返回255。
- 升降级前需保证配置文件夹可读写。
- 升降级过程中的硬件故障如磁盘故障，机器宕机等，无法保证回滚成功。
- 升降级中的回滚失败时，可重新执行升降级命令进行升降级。
- 升级时回滚不成功的话，如需回退至原先版本，需先升级再降级；降级时回滚不成功的话，如需回退至原先版本，则需先降级再升级。
- 升降级回滚失败时，会保留${confpath}_backup和${confpath}_new_backup两个文件夹，用于升降级的重试，需保证这两个文件夹与用户文件不冲突。

命令参考

用户可以通过 --help 选项获得该模式的帮助信息，例如：

```
gs_dbmind upgrade --help
usage:  upgrade [-h] -c DIRECTORY [--target DIRECTORY]

optional arguments:
  -h, --help            show this help message and exit
  -c DIRECTORY, --conf DIRECTORY
                        set the directory of configuration files
  --target DIRECTORY    set the directory for dbmind of the target version
```

表 1-16 gs_dbmind upgrade 子命令说明

参数	参数说明	取值范围
-h, --help	帮助命令	-
-c，--conf	配置文件目录confpath	文件夹路径
--target	配置目标版本的DBMind目录	文件夹路径，默认为gs_dbmind文件夹路径

1.10.2 DBMind 的支持组件

支持组件是指DBMind提供的用于支撑整个服务、确保解决方案能够部署和实施的模块。它们本身不是AI功能，却是整个服务体系中非常重要的一环，用于支撑整个自治运维解决方案的快速实施，如用于采集数据库指标的exporter等。

版权所有 © 华为云计算技术有限公司

1.10.2.1 Prometheus Exporter 组件

1.10.2.1.1 概述

Prometheus是业内非常流行的开源监控系统，同时本身也是一款时序数据库。Prometheus的采集端被称为exporter，用来收集被监控模块的指标项。为了与Prometheus平台完成对接，AI工具自带了几款exporter，分别是用来采集数据库指标的opengauss-exporter，用来执行cmd命令并获取返回结果以及采集日志信息的cmd-exporter，以及对采集到的指标进行二次加工的reprocessing-exporter。

须知

- Prometheus和exporter是业内流行的监控和采集平台，部署在内网环境中，不对外部暴露接口，仅供内部监控平台使用。因此，为了增强该平台的安全性，一般需要用户或运维人员配置防火墙等，以便隔离外部访问，从而增强监控平台的安全性。
- Prometheus平台在默认情况下，采用Http协议、并且没有任何安全访问限制。这是因为，该平台一般部署在内网环境中，攻击风险可控。如果用户希望提高安全性，可自行修改Prometheus的TLS配置选项或增加basic_auth_users相关选项，但仍不建议对外部直接暴露访问接口。
- Prometheus 默认绑定的IP地址是0.0.0.0，DBMind无法自动获知用户希望绑定的IP地址，故用户应该显式设置Prometheus的绑定IP地址，以减小安全风险。对于Prometheus的其他开源exporter（如node-exporter）也是同理。
- DBMind的exporter与Prometheus交互时，若使用SSL证书，则默认会进行双向认证。
- Prometheus的CA证书默认只支持SAN（Subject Alternative Name）形式，用户在配置Prometheus的CA证书，以及与Prometheus进行认证的证书时，需要使用SAN格式。同时，Prometheus不支持密码保护的私钥文件，用户在为Prometheus生成私钥文件时，请不要使用密码保护。例如，用户可以参考下述示例，通过openssl工具生成SAN形式的自签发证书和私钥文件：

  ```
  openssl req -new -newkey rsa:2048 -days 365 -nodes -x509 -keyout prometheus.key -out prometheus.crt -subj "/C=CN/ST=openGauss/L=openGauss/O=openGauss-geek/CN=localhost" -addext "subjectAltName = DNS:localhost"
  ```

  通过执行上述命令，可以生成两个文件prometheus.crt和prometheus.key，其中prometheus.crt文件可以同时作为CA证书使用。
- DBMind的exporter默认支持HTTPS协议，需要用户手工指定证书文件路径，建议用户使用。
- DBMind Service的部分功能会从Prometheus中获取时序数据，该通道也可以使用SSL协议进行加固，prometheus默认支持https通信，建议用户使用。

1.10.2.1.2 环境部署

用户可以从Prometheus的官网上下载Prometheus-server和node-exporter，然后根据官方文档中的说明启动它们；

其中，提供的exporter组件默认采用Https通信协议，因此需要用户默认提供SSL证书和密钥文件，并通过“--ssl-keyfile”、“--ssl-certfile”以及“--ssl-ca-file”选项进行配置。若用户不希望使用HTTPS协议，则可以通过“--disable-https”选项禁用该模式。

版权所有 © 华为云计算技术有限公司

说明

由于GaussDB默认模式下的通信加密协议与PostgreSQL不兼容，故导致通过PyPI源安装的基于PostgreSQL编译的Python驱动psycopg2-binary默认无法连接至数据库。

因此，需要用户自行编译psycopg2或修改GUC参数进行适配。也可通过openGauss官方网站下载基于GaussDB编译的psycopg2（官方网站仅提供部分Python版本的编译包，需要用户鉴别是否与当前Python运行时版本一致）。

1.10.2.1.3 使用指导

用户可通过gs_dbmind命令启动对应的exporter。下面为用户演示一个完整的Prometheus监控平台的搭建过程。

步骤1 部署Prometheus主进程，运行如下命令：

```
prometheus --config.file=prometheus.yml
```

步骤2 部署openGauss-exporter：启动openGauss-exporter，采用默认侦听端口号9187，侦听地址为192.168.1.100，采用HTTPS协议，则命令可以为：

```
gs_dbmind component opengauss_exporter --url postgresql://user:password@ip:port/dbname --web.listen-address 192.168.1.100 --ssl-keyfile server.key --ssl-certfile server.crt --ssl-ca-file server.crt
```

步骤3 部署reprocessing-exporter：启动reprocessing-exporter，采用默认侦听端口号8181，侦听地址为192.168.1.101，Prometheus-server IP与端口号为192.168.1.100:9090，采用HTTPS协议，则命令可以为：

```
gs_dbmind component reprocessing_exporter 192.168.1.100 9090 --web.listen-address 192.168.1.101 --ssl-keyfile server.key --ssl-certfile server.crt --ssl-ca-file server.crt
```

步骤4 部署cmd-exporter: 启动cmd-exporter，使用默认参数，设置采集的openGauss日志目录路径，并给定证书信息：

```
gs_dbmind component cmd_exporter --ssl-keyfile server.key --ssl-certfile server.crt --ssl-ca-file server.crt --pg-log-dir /path/to/pglog
```

步骤5 部署node-exporter：一般而言，Prometheus监控平台都需要部署node-exporter用于监控Linux操作系统，后文提到的部分AI功能也需要依赖node-exporter采集Linux系统指标，故也需要用户来部署；使用方法详见：https://prometheus.io/docs/guides/node-exporter/#installing-and-running-the-node-exporter。用户可直接运行该node-exporter进程，其默认端口号为9100，启动命令行为：

```
node_exporter
```

----结束

注意

DBMind高可用功能需要在未纳管实例的情况下实现，因此接口不需要token认证。应当使用默认的HTTPS协议以进行SSL双向认证，使用HTTP协议则可能引入攻击者直接发起未授权请求的安全风险。

为保证DBMind云上使用时高可靠，exporter组件提供了组件状态查询和部分异常修复接口，详细请参见**表1-20**。

- 组件状态查询接口示例：

  ```
  curl -X 'POST'"http://127.0.0.1:8080/v1/api/check-status" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"cmd": "./gs_dbmind component cmd_exporter --web.listen-address 0.0.0.0 --web.listen-port 9181 --log.filepath /xxx/dbmind_cmd_exporter.log --disable-https"}'
  ```

  如果使用HTTPS协议，则查询示例为：

  ```
  curl -X 'POST'"https://127.0.0.1:8080/v1/api/check-status" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"cmd": "./gs_dbmind component cmd_exporter --web.listen-address 0.0.0.0
  ```


版权所有 © 华为云计算技术有限公司

```
--web.listen-port 9181 --log.filepath /xxx/dbmind_cmd_exporter.log --ssl-keyfile xx.key --ssl-certfile xx.crt --ssl-ca-file xx.crt"}' --cacert xx.crt --key xx.key --cert xx.crt
```

返回结果示例：

```
{"data":{"error_msg":"","result":{},"state":"NORMAL"},"success":true}
```

- 组件异常修复接口示例：

```
curl -X 'POST'"http://127.0.0.1:8080/v1/api/repair" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"cmd": "./gs_dbmind component cmd_exporter --web.listen-address 0.0.0.0 --web.listen-port 9181 --log.filepath /xxx/dbmind_cmd_exporter.log --disable-https"}'
```

如果使用HTTPS协议，则修复示例为：

```
curl -X 'POST'"https://127.0.0.1:8080/v1/api/repair" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"cmd": "./gs_dbmind component cmd_exporter --web.listen-address 0.0.0.0 --web.listen-port 9181 --log.filepath /xxx/dbmind_cmd_exporter.log --ssl-keyfile xx.key --ssl-certfile xx.crt --ssl-ca-file xx.crt"}' --cacert xx.crt --key xx.key --cert xx.crt
```

返回结果示例：

```
{"data":{"error_msg":"","result":{},"state":"SUCCESS"},"success":true}
```


版权所有 © 华为云计算技术有限公司

说明

exporter启动后，会根据yamls目录下的配置文件信息采集特定的指标。指标的配置信息如下例所示：

```
indicator_name:
  name: indicator_name
  desc: xxx
  query:
    - name: indicator_name
      sql: xxx
      version: xxx
      timeout: 10
      status: enable
  metrics:
    ...
  status: enable
  ttl: 0
  timeout: 10
```

当status值为enable时，会在指定的超时时间内对该指标进行采集。用户可以对默认开启状态status和超时时间timeout进行修改，修改后需重新启动exporter才能生效。

组件状态查询和修复接口只针对DBMind自研exporter，包括opengauss_exporter、cmd_exporter和reprocessing_exporter，对于开源的node_exporter则不支持。

当前Exporter组件支持查询的异常场景如下：

- PID文件丢失或文件内容异常：当用户启动服务/组件后，会自动生成*.pid文件来记录进程对应的进程标识符，也就是PID。PID文件用于云管控来判断进程状态，发现进程级别的异常并进行修复。为防止PID文件的误删除与内容的误修改，需要定期判断pid文件是否存在以及其内容是否正确。
- 日志文件丢失：当用户启动服务/组件后，会自动生成*.log文件来记录进程运行的日志。此日志文件用于快速定位问题的根源、追踪程序执行的过程等。为防止日志文件的误删除，需要定期判断日志文件是否存在。
- 资源占用异常：DBMind服务会执行索引推荐、指标分析、智能巡检等功能，需要占用系统资源；exporter组件会采集服务器上的指标，用于后续分析，也会占用系统资源；当服务/组件的CPU或内存占用超过阈值时，说明资源占用过高，服务/组件出现异常。因此需要监控服务和组件的资源占用情况。
- 数据库异常：opengauss_exporter需要从数据库中获取数据，用于生成采集结果，所以需要判断数据库是否能够正常连接。

资源占用阈值及告警设置如下：

- 内存占用阈值：max(总内存的1%, 200MB)。
- CPU多核占用阈值：cmd_exporter 10%，opengauss_exporter 10%，reprocessing_exporter 10%， DBMind 80%。
- CPU单核占用阈值：cmd_exporter 50%，opengauss_exporter 50%，reprocessing_exporter 50%，DBMind不做限制。
- 调用高可用接口会检查资源占用，连续超过3次阈值才会触发资源占用告警。

风险：当占用资源连续超过阈值三次时，会触发告警，管控侧会重启服务，导致正在运行的任务中断。

当前Exporter针对组件异常的修复性说明如下：

- PID文件丢失：可自动修复；
- 日志文件丢失：可自动修复；
- 组件资源占用异常：不可通过接口修复；
- 数据库异常：不可通过接口修复；
- 针对Exporter组件不可修复的异常，需要云侧进行处理，如重启进程。

版权所有 © 华为云计算技术有限公司

⚠ 注意

- openGauss-exporter中连接数据库的用户需要monitor admin或以上权限，否则会出现部分指标无法采集的情况。同时openGauss-exporter不支持使用数据库初始用户来进行数据采集。
- openGauss-exporter中连接数据库的用户需要获取dbe_perf模式下的数据，因此需要保证其具有该模式的权限。
- openGauss-exporter会从dbe_perf.statement_history中抽样慢SQL信息，dbe_perf.statement_history视图慢SQL记录与GUC参数log_min_duration_statement和track_stmt_stat_level相关，其中log_min_duration_statement是慢SQL阈值，单位毫秒，具体值由用户设置；track_stmt_stat_level是SQL记录级别，默认为'OFF,L0'，即只记录慢SQL信息，级别为L0，建议用户在详细了解参数意义与作用的情况下谨慎修改。
- openGauss-exporter采集数据库相关信息，主要包括部分系统表和视图中的数据（具体参见代码中opengauss_exporter中的配置文件）。node-exporter采集系统指标信息，主要与系统磁盘、CPU等相关。reprocessing_exporter基于prometheus-server中的某些指标（具体参见代码中reprocessing_exporter中的配置文件）进行二次加工，最终提供加工后的数据供用户使用。
- 当数据库部署在多层网络上时（分布式：管理层-数据层-数据同步层），openGauss-exporter的启动命令中的url参数必须使用数据层的ip地址，否则会影响DBMind的其他功能。
- prometheus-server在拉取exporter数据时有超时机制，超时时间由scrape_timeout（默认10s）控制，因此当exporter采集数据量较大时，用户可根据实际情况增大scrape_timeout以防止超时报错，另外需要注意的是scrape_interval（采集间隔，默认15s）不能比scrape_timeout小，否则会出现异常。
- 如果数据库时区设置和系统不相同，可能会出现时间相关指标时间与系统时间不一致的情况，因此需要将数据库时区与系统保持同步。
- 当使用https通信时，工具会检测证书与密钥文件权限以及证书有效期，如果文件权限大于600则会出现报警，证书有效期小于90天会出现报警。
- 当存在指标重复采集时，openGauss-exporter会出现异常，异常信息会打印到日志中。
- 使用openGauss-exporter的--config、--disable-settings-metrics、--disable-statement-history-metrics三个参数时需要注意，其存在以下几种情况：
  - 用户不指定其中任何参数，则工具会同时对yamls目录下的三个配置文件中的指标进行采集。
  - 用户显式指定--config，则工具不会采集yamls目录下default.yml中的指标，而会采集用户指定配置文件中的指标，同时pg_settings.yml和statements.yml正常采集，此时用户需要注意指定的配置文件中的指标和pg_settings.yml、statements.yml中的指标不能存在重复采集的现象。
  - 用户显式指定--disable-settings-metrics，则工具不会采集yamls目录下pg_settings.yml中的指标；用户显式指定--disable-statement-history-metrics，则工具不会采集yamls目录下statements.yml（慢SQL相关）中的指标。
- exporter启动后，一般情况下不会终止进程并退出（如连接的数据库地址不可用、连接的数据库用户被删除或禁用等），而是会在日志中记录报错信息，并在后台重新尝试。
- cmd_exporter默认Linux系统定义了需要监控的gaussDB的日志路径：$GAUSSLOG，如果没有定义$GAUSSLOG则需要进行定义。

版权所有 © 华为云计算技术有限公司

- cmd_exporter默认匹配的是GUC参数“log_statement='none'”时的日志格式，当log_statement为其他参数时cmd_exporter会无法识别日志格式。
- 由于cmd_exporter需要访问数据库日志和使用cm工具，所以需要由数据库用户部署cmd_exporter来获取相应的状态信息。
- 实例升级过程中，opengauss_exporter无法正常采集数据，cmd_exporter无法采集数据库的cm相关信息。
- cmd_exporter的日志采集功能对于内核日志的日志文件名和日志格式都有要求，在gaussdb.conf文件中，log_filename的格式要求为'gaussdb-%Y-%m-%d_%H%M%S.log'，log_line_prefix的格式要求为'%m %n %u %d %h %p %S %x %a'。
- cmd_exporter采集的日志指标为散点图形式，各个采集点之间标签不同，所以不能合并到同一个集合中，不适合用折线图的方式进行展示，请采用表格或者散点图的方式进行展示。
- 当cmd_exporter在一个采集周期内没有采集到日志指标时（如gaussdb_log_ffic为空，则表示没有生成ffic文件），则该指标数据为空。
- 在某些特殊情况下由于日志文件激增，考虑到cpu的处理能力和内存限制，cmd_exporter为日志的消息队列设置了长度上限以防止内存泄露。当队列满时将停止新的日志事件入列直到队列有空间为止。这一设计可能导致在日志负载高的时候丢失部分日志指标。
- reprocessing_exporter中采集指标os_disk_usage时，只针对EXT和XFS文件系统，其他文件系统不会采集。

1.10.2.1.4 获取帮助

用户可以通过“--help”命令获取Exporter组件启动或停止等操作的帮助信息，例如：

```
gs_dbmind component opengauss_exporter --help
gs_dbmind component reprocessing_exporter --help
gs_dbmind component cmd_exporter --help
```

Exporter组件状态查询和修复的RESETful API列表如下：

API	入参	参数介绍	请求方法	功能描述与预期返回结果
/v1/api/check-status	cmd	组件启动命令，String，必选	POST	获取exporter组件的状态信息并返回状态详情。
/v1/api/repair	cmd	组件启动命令，String，必选	POST	修复exporter组件并返回修复结果。

1.10.2.1.5 命令参考

reprocessing-exporter的使用帮助详情：

```
gs_dbmind component reprocessing_exporter --help
usage:  [-h] [--prometheus-auth-user PROMETHEUS_AUTH_USER]
        [--prometheus-auth-password PROMETHEUS_AUTH_PASSWORD]
        [--disable-https] [--ssl-keyfile SSL_KEYFILE]
        [--ssl-certfile SSL_CERTFILE] [--ssl-ca-file SSL_CA_FILE]
        [--tsdb-ssl-keyfile TSDB_SSL_KEYFILE] [--tsdb-ssl-certfile TSDB_SSL_CERTFILE]
        [--tsdb-ssl-ca-file TSDB_SSL_CA_FILE]
        [--web.listen-address WEB.LISTEN_ADDRESS]
        [--web.listen-port WEB.LISTEN_PORT]
        [--collector.config COLLECTOR.CONFIG] [--log.filepath LOG.FILEPATH]
```


版权所有 © 华为云计算技术有限公司

```
      [--log.level {debug,info,warn,error,fatal}] [-v]
      prometheus_host prometheus_port

Reprocessing Exporter: A re-processing module for metrics stored in the
Prometheus server.

positional arguments:
  prometheus_host       from which host to pull data
  prometheus_port       the port to connect to the Prometheus host

optional arguments:
  -h, --help            show this help message and exit
  --prometheus-auth-user PROMETHEUS_AUTH_USER
                        use this user for basic authorization to connect to
                        the Prometheus server
  --prometheus-auth-password PROMETHEUS_AUTH_PASSWORD
                        use this password for basic authorization to connect
                        to the Prometheus server
  --disable-https       disable Https scheme
  --ssl-keyfile SSL_KEYFILE
                        set the path of ssl key file
  --ssl-certfile SSL_CERTFILE
                        set the path of ssl certificate file
  --ssl-ca-file SSL_CA_FILE
                        set the path of ssl ca file
  --tsdb-ssl-keyfile TSDB_SSL_KEYFILE
                        set the path of tsdb ssl key file
  --tsdb-ssl-certfile TSDB_SSL_CERTFILE
                        set the path of tsdb ssl certificate file
  --tsdb-ssl-ca-file TSDB_SSL_CA_FILE
                        set the path of tsdb ssl ca file
  --web.listen-address WEB.LISTEN_ADDRESS
                        address on which to expose metrics and web interface
  --web.listen-port WEB.LISTEN_PORT
                        listen port to expose metrics and web interface
  --collector.config COLLECTOR.CONFIG, --config COLLECTOR.CONFIG
                        according to the content of the yaml file for metric
                        collection
  --log.filepath LOG.FILEPATH
                        the path to log
  --log.level {debug,info,warn,error,fatal}
                        only log messages with the given severity or above.
                        Valid levels: [debug, info, warn, error, fatal]
  -v, --version         show program's version number and exit
```

表 1-17 reprocessing-exporter 的命令行参数详情表

参数	参数说明	取值范围
prometheus_host	Prometheus-server的IP地址。	-
prometheus_port	Prometheus-server的服务侦听端口号。	1024-65535。
-h, --help	帮助选项。	-
--prometheus-auth-user	prometheus用户名称。	-
--prometheus-auth-password	prometheus用户密码。	-
--disable-https	禁用HTTPS协议。	-

版权所有 © 华为云计算技术有限公司

参数	参数说明	取值范围
--ssl-keyfile	HTTPS协议使用的证书私钥文件路径，如果为密文私钥，需要通过管道传入私钥密码，传输内容为json格式，密码填充在ssl-keyfile-password字段。如：`echo {"ssl-keyfile-password":"password"} | gs_dbmind component reprocessing_exporter ...` 。	-
--ssl-certfile	HTTPS协议使用的证书文件路径。	-
--ssl-ca-file	HTTPS协议使用的CA证书文件路径。	-
--tsdb-ssl-keyfile	TSDB数据库使用的HTTPS协议证书私钥文件路径。	-
--tsdb-ssl-certfile	TSDB数据库使用的HTTPS协议证书文件路径。	-
--tsdb-ssl-ca-file	TSDB数据库使用的HTTPS协议CA证书文件路径。	-
--web.listen-address	该exporter服务的绑定IP。	-
--web.listen-port	该exporter服务的侦听端口。	1024-65535。
--collector.config	显性指定的待采集指标配置文件路径。	-
--log.filepath	日志文件保存路径，默认保存在当前目录下。	-
--log.level	日志文件的打印级别，默认为INFO级别。	DEBUG、INFO、WARN、ERROR、FATAL。
--version	显示版本信息。	-

openGauss-exporter的使用帮助详情：

```
gs_dbmind component opengauss_exporter --help
usage:  [-h] --url URL [--config-file CONFIG_FILE]
        [--include-databases INCLUDE_DATABASES]
        [--exclude-databases EXCLUDE_DATABASES]
        [--constant-labels CONSTANT_LABELS]
        [--scrape-interval-seconds SCRAPE_INTERVAL_SECONDS]
        [--web.listen-address WEB.LISTEN_ADDRESS]
        [--web.listen-port WEB.LISTEN_PORT] [--disable-cache]
```


版权所有 © 华为云计算技术有限公司

```
      [--disable-settings-metrics] [--disable-statement-history-metrics]
      [--disable-https] [--disable-agent] [--ssl-keyfile SSL_KEYFILE]
      [--ssl-certfile SSL_CERTFILE] [--ssl-ca-file SSL_CA_FILE]
      [--parallel PARALLEL] [--connection-pool-size CONNECTION_POOL_SIZE]
      [--log.filepath LOG.FILEPATH]
      [--log.level {debug,info,warn,error,fatal}] [-v]

openGauss Exporter (DBMind): Monitoring or controlling for openGauss.

optional arguments:
  -h, --help            show this help message and exit
  --url URL, --dsn URL  openGauss database target url. It is recommended to
                        connect to the postgres database through this URL, so
                        that the exporter can actively discover and monitor
                        other databases.
  --config-file CONFIG_FILE, --config CONFIG_FILE
                        path to config file.
  --include-databases INCLUDE_DATABASES
                        only scrape metrics from the given database list. a
                        list of database name (format is label=dbname or
                        dbname) separated by comma(,).
  --exclude-databases EXCLUDE_DATABASES
                        scrape metrics from the all auto-discovered databases
                        excluding the list of database. a list of database
                        name (format is label=dbname or dbname) separated by
                        comma(,).
  --constant-labels CONSTANT_LABELS
                        a list of label=value separated by comma(,).
  --scrape-interval-seconds SCRAPE_INTERVAL_SECONDS
                        specify the scrape interval in seconds to reduce
                        redundant results. If set 0, it means automatically
                        calculate.
  --web.listen-address WEB.LISTEN_ADDRESS
                        address on which to expose metrics and web interface
  --web.listen-port WEB.LISTEN_PORT
                        listen port to expose metrics and web interface
  --disable-cache       force not using cache.
  --disable-settings-metrics
                        not collect pg_settings.yml metrics.
  --disable-statement-history-metrics
                        not collect statement-history metrics (including slow
                        queries).
  --disable-https       disable Https scheme
  --disable-agent       by default, this exporter also assumes the role of
                        DBMind-Agent, that is, executing database operation
                        and maintenance actions issued by the DBMind service.
                        With this argument, users can disable the agent
                        functionality, thereby prohibiting the DBMind service
                        from making changes to the database.
  --ssl-keyfile SSL_KEYFILE
                        set the path of ssl key file
  --ssl-certfile SSL_CERTFILE
                        set the path of ssl certificate file
  --ssl-ca-file SSL_CA_FILE
                        set the path of ssl ca file
  --parallel PARALLEL   number of parallels for metrics scrape.
  --connection-pool-size CONNECTION_POOL_SIZE
                        size of connection pool for each database. Set zero to
                        disable connection pool.
  --log.filepath LOG.FILEPATH
                        the path to log
  --log.level {debug,info,warn,error,fatal}
                        only log messages with the given severity or above.
                        Valid levels: [debug, info, warn, error, fatal]
  -v, --version         show program's version number and exit
```

 版权所有 © 华为云计算技术有限公司

表 1-18 openGauss-exporter 的命令行参数详情表

参数	参数说明	取值范围
--url, --dsn	数据库server的连接地址，例如 postgres://user:pwd@host:port/dbname。 密码字段为空时，需要通过管道密码，传输内容为json格式，密码填充在db-password字段。如：`echo {"db-password":"password"} | gs_dbmind component opengauss_exporter ...`。	如果该url涉及到的各字段URL包含特殊字符（如@，/等），则需要通过URL编码进行转义，例如密码中的"@"应转义为%40，"/"应转义为%2F，否则各字段的含义会被错误识别和切分，具体转义规则可以参考URL编码的转义规则，该URL地址规则遵循RFC-1738标准。
--constant-labels	常量列表，k=v格式，用逗号隔开，表明该exporter自带的常量标签。	格式如：“cluster_name=demo,cluster_id=1”。
-h, --help	帮助选项。	-
--disable-https	禁用HTTPS协议。	-
--ssl-keyfile	HTTPS协议使用的证书私钥文件路径，如果为密文私钥，需要通过管道传入私钥密码，传输内容为json格式，密码填充在ssl-keyfile-password字段。如：`echo {"ssl-keyfile-password":"password"} | gs_dbmind component opengauss_exporter ...`。。	-
--ssl-certfile	HTTPS协议使用的证书文件路径。	-
--ssl-ca-file	HTTPS协议使用的CA证书文件路径。	-
--web.listen-address	该exporter服务的绑定IP。	-
--web.listen-port	该exporter服务的侦听端口。	1024-65535。
--config, --config-file	显性指定的待采集指标配置文件路径。	-
--log.filepath	日志文件保存路径，默认保存在当前目录下。	-

版权所有 © 华为云计算技术有限公司

参数	参数说明	取值范围
--log.level	日志文件的打印级别，默认为INFO级别。	DEBUG、INFO、WARN、ERROR、FATAL。
--version	显示版本信息。	-
--disable-cache	禁止使用缓存。	-
--disable-settings-metrics	禁止采集pg_settings表的值。	-
--disable-statement-history-metrics	禁止采集statement_history表中的慢SQL信息。	-
--disable-agent	禁止agent行为。	-
--include-databases	显性表明待采集的数据库名，指定多个数据库时用逗号（,）隔开。	-
--exclude-databases	显性表明不采集的数据库名，指定多个数据库时用逗号（,）隔开。	-
--parallel	指标采集的并行线程数，默认为5。	正整数。
--scrape-interval-seconds	明确抓取间隔避免重复采集指标，默认为0。	>=0的整数，如果为0，则根据上一次采集的时间对采集间隔进行动态调整。
--connection-pool-size	数据库的连接池大小 ，默认为0。	>=0的整数，如果为0，则不采用连接池连接。

cmd-exporter的使用帮助详情：

```
gs_dbmind component cmd_exporter --help
usage:  [-h] [--constant-labels CONSTANT_LABELS] [--web.listen-address WEB.LISTEN_ADDRESS] [--
web.listen-port WEB.LISTEN_PORT] [--disable-https]
       [--config CONFIG] [--ssl-keyfile SSL_KEYFILE] [--ssl-certfile SSL_CERTFILE] [--ssl-ca-file SSL_CA_FILE]
[--parallel PARALLEL]
       [--pg-log-dir PG_LOG_DIR] [--disable-log-exporter] [--log.filepath LOG.FILEPATH] [--log.level
{debug,info,warn,error,fatal}] [-v]

Command Exporter (DBMind): scrape metrics by performing shell commands.

optional arguments:
  -h, --help            show this help message and exit
  --constant-labels CONSTANT_LABELS
                        a list of label=value separated by comma(,).
  --web.listen-address WEB.LISTEN_ADDRESS
                        address on which to expose metrics and web interface
  --web.listen-port WEB.LISTEN_PORT
                        listen port to expose metrics and web interface
  --disable-https       disable Https scheme
  --config CONFIG       path to config dir or file.
  --ssl-keyfile SSL_KEYFILE
                        set the path of ssl key file
```


版权所有 © 华为云计算技术有限公司

```
 --ssl-certfile SSL_CERTFILE
                    set the path of ssl certificate file
 --ssl-ca-file SSL_CA_FILE
                    set the path of ssl ca file
 --parallel PARALLEL   performing shell command in parallel.
 --pg-log-dir PG_LOG_DIR
                    set the directory path of PGLOG, default value is $GAUSSLOG.
 --disable-log-exporter
                    disable log analysis
 --log.filepath LOG.FILEPATH
                    the path to log
 --log.level {debug,info,warn,error,fatal}
                    only log messages with the given severity or above. Valid levels: [debug, info, warn, error,
fatal]
 -v, --version         show program's version number and exit
```

表 1-19 cmd-exporter 的命令行参数详情表

参数	参数说明	取值范围
-h, --help	帮助选项。	-
--disable-https	禁用HTTPS协议。	-
--ssl-keyfile	HTTPS协议使用的证书私钥文件路径，如果为密文私钥，需要通过管道传入私钥密码，为json格式，ssl-keyfile-password字段。如：`echo {"ssl-keyfile-password":"password"} | gs_dbmind component cmd_exporter ...`。	-
--ssl-certfile	HTTPS协议使用的证书文件路径。	-
--ssl-ca-file	HTTPS协议使用的CA证书文件路径。	-
--web.listen-address	该exporter服务的绑定IP。	-
--web.listen-port	该exporter服务的侦听端口。	1024-65535。
--config	显性指定的待采集指标配置文件路径。	默认是该功能yamls目录下的default.yml文件，可以参考该配置文件格式，错误配置会报错。
--log.filepath	日志文件保存路径，默认保存在当前目录下。	-
--log.level	日志文件的打印级别，默认为INFO级别。	DEBUG、INFO、WARN、ERROR、FATAL。
--parallel	并行执行shell命令的并发度。	正整数。

版权所有 © 华为云计算技术有限公司

参数	参数说明	取值范围
--gs-log-dir	日志侦听路径，默认路径为“${GAUSSLOG}/gs_log”。	-
--disable-log-exporter	禁用日志采集功能。	-
--constant-labels	常量列表，k=v格式，用逗号隔开，表明该exporter自带的常量标签。	格式如“cluster_name=demo,cluster_id=1”。
--version	显示版本信息。	-

说明

当启用日志监测功能时，当监控的日志路径发生变动时，请重启cmd_exporter进程。

cmd_exporter进程对容器化环境部分支持。

1.10.2.1.6 常见问题处理

1. 提示需要用户提供--ssl-keyfile与--ssl-certfile选项：

   上述exporter默认采用Https模式通信，因此需要用户指定证书及其私钥文件的路径。相反，如果用户只想采用Http模式，则需要显性指定--disable-https选项，从而禁用HTTPS协议。

2. 提示用户需要输入PEM密码（Enter PEM pass phrase）：

   如果用户采用Https模式，并给定了证书及其密钥文件的路径，且该密钥文件是经过加密的，则需要用户输入该加密私钥证书文件的密码。该密码也可以通过标准输入流传递。

   说明

   prometheus不支持私钥加密，若涉及prometheus通信的私钥加密时部分功能将无法实现。

   prometheus的query_range功能支持的最大序列长度是11000个数据，在使用时需要注意。

   prometheus的query_range功能可能会出现单点数据同时出现在连续两次查询结果中，此为正常现象。

1.10.3 DBMind 的 AI 子功能

用户可以通过gs_dbmind的component子命令启动对应的AI子功能，下述章节展示不同AI功能的具体内容和使用详情。

DBMind进程启动后，其提供的Http(s)接口信息如下：

版权所有 © 华为云计算技术有限公司

表 1-20 DBMind Restful API 列表

API	请求方法	功能描述	参数	返回值	示例
/v1/api/token	POST	获取数据库访问令牌	param username：必选，用户名 param grant_type：可选，授权类型 param password：必选，密码 param scope：必选，数据库实例 param client_id：可选，客户端id param client_secret：可选，客户端密钥	返回token	{"access_token":"qmWWgt28VSu0YAH5","token_type":"bearer","expires_in":600}
/v1/api/agents	GET	获取所有代理列表	无	返回所有代理列表	{"data":{"ip1":["ip1","ip2","ip3"]}
/v1/api/agents	PUT	更新代理信息（包含强制更新），由于集群状态的更改需要一段时间，因此强制更新默认延迟15秒执行。	param force：可选，强制更新	返回更新代理结果	{"data":true,"success":true}
/v1/api/configs/dynamic-config	PUT	设置DBMind配置	param config：必选，配置项分类 param name：必选，配置项名称 param value：必选，配置项取值	返回更新配置结果	{"data":"success","success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/configs/dynamic-config	GET	获取DBMind配置	param config：必选，配置项分类 param name：必选，配置项名称	返回配置信息	{"data":"50","success":true}
/v1/api/configs/dynamic-config/list	GET	获取DBMind配置列表	param is_default：可选，默认为false。取值为false时从元数据库中获取配置信息；否则从本地配置文件中获取配置信息	返回配置信息列表	{"data":{"dynamic":{"detection_params":[["esd_test_alpha","0.05","The Significance level."]...]}...},"success":true}
/v1/api/configs/anomaly-detection/default-settings	GET	获取异常检测默认设置	无	返回异常检测默认设置信息	{"data":{"AlarmInfo":{"alarm_cause":null,"alarm_content":null,...,"window":10}},"success":true}
/v1/api/status/schedulers	GET	获取定时任务状态更新	无	返回定时任务的状态	{"data":{"header":["name","current_status","running_interval"],"rows":[["anomaly_detection","Running",180]...]},"success":true}
/v1/api/status/collection-system	GET	获取流水采集状态	param exporter_type：必选，exporter类型，取值范围：('opengauss_exporter','cmd_exporter','reprocessing_exporter') param ssl_context：必选，SSL上下文信息	返回采集器状态	{"data":{"header":["component","listen_address","is_alive"],"rows":[["opengauss_exporter","ip",true]...],"suggestions":[]},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/status/data-directory	GET	获取数据库数据目录状态	param instance：可选，实例节点的地址 param latest_minutes：可选，接口会统计从请求时间往前推latest_minutes分钟的数据，默认为5	返回数据库目录状态	{"data":{"free_space":2161.57,"tilt_rate":0.03,"total_space":3298.17...},"success":true}
/v1/api/status/overview	GET	获取数据库概览信息	param latest_minutes：可选，接口会统计从请求时间往前推latest_minutes分钟的数据，默认为3	返回数据库概览信息	{"data":{"deployment_mode":"centralized",...,"strength_version":"openGauss 2.1.0..."},"success":true}
/v1/api/status/instances	GET	获取数据库实例状态	无	返回数据库实例状态	{"data":{"header":["instance","role","state"],"rows":[["ip:port,","primary",fawu lse]...]},"success":true}
/v1/api/status/agents	GET	获取数据库代理状态	无	返回数据库代理状态	{"data":{"agent_address":"ip","status":true},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/app/slow-sql-rca	GET	获取慢SQL的根因分析结果	param query：必选，SQL查询语句，字符串长度需小于10240 param db_name：必选，数据库名称 param schema_name：可选，schema名称，默认为public param start_time：可选，建议传递，开始时间 param finish_time：可选，建议传递，结束时间 param template_id：可选，建议传递，归一化SQL ID param debug_query_id：可选，建议传递，唯一SQL ID param n_soft_parse：可选，软解析次数 param n_hard_parse：可选，硬解析次数 param query_plan：可选，执行计划 param n_returned_rows：可选，结果中元组个数	返回慢SQL的根因分析结果	{"data":[true,"public","Seq Scan on t2 (cost=0.00..3.25 rows=1 width=70)\n Filter: (c1 = 'dadsadasds'::text)\n",[[["1. MISSING_INDEXES: (0.53) Missing required index.","2. HEAVY_SCAN_OPERATOR: (0.47) Existing expensive seq scans. Detail: (name: Seq Scan on t2, parent: None, rows:t2(100), cost rate: 100.0%)"]],[["1. Recommended index: (schema: public, index: t2(c1))."," 2. According to business adjustments, try to avoid it"]]]],"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
			param n_tuples_fetched：可选，随机扫描行数 param n_tuples_returned：可选，顺序扫描行数 param n_tuples_inserted：可选，插入行数 param n_tuples_updated：可选，更新行数 param n_tuples_deleted：可选，删除行数 param n_blocks_fetched：可选，buffer的块访问次数 param n_blocks_hit：可选，buffer的块命中次数 param db_time：可选，有效的DB时间花费，单位：微秒 param cpu_time：可选，CPU时间，单位：微秒 param parse_time：可选，解析时间，单位：微秒 param plan_time：可		**说明** 详细参见**注意事项说明**。 • 当用户传入归一化QUERY语句时，内部会使用PBE方式获取执行计划，如果获取失败则不会返回执行计划内容。 • 如果用户执行诊断的用户权限不足，则无法正常返回结果。 • 接口响应时长受数据库负载和资源影响。 • 建议传入query_plan参数，避免内部获取。 • 如果传入的QUERY存在截断且没有传入query_plan参数，则无法进行诊断。 • 接口只支持一条QUERY语句，如果传入多条则只会诊断第一条QUERY。 • 如果用户不传schema信息，则schema默认为public。 • 当前只支持DML语句。

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
			选，执行时间，单位：微秒 param data_io_time：可选，IO时间，单位：微秒 param hash_spill_count：可选，hash过程中，若发生落盘，写文件的次数 param sort_spill_count：可选，sort过程中，若发生落盘，写文件的次数 param n_calls：可选，调用次数 param lock_wait_time：可选，加锁等待耗时 param lwlock_wait_time：可选，轻量级加锁时间 param tz: 可选，若存在时区差异则建议传递，慢SQL时区信息，只支持UTC标准		

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/app/correlation	GET	获取性能指标相关性分析结果	param metric_name：可选，metric名称 param instance：可选，实例地址 param start_time：可选，开始时间的时间戳 param end_time：可选，结束时间的时间戳 注：此处start_time和end_time是两个独立参数，并非为选取时间窗口，所以不强制要求start_time早于end_time。 param metric_filter: 可选，metric的标签。 支持如下两种格式： ● “k1=v1,k2=v2”的kv pairs格式 ● “{'k1':'v1','k2':'v2'}”的json字符串格式	返回异常关联指标及其时序数据	{"data": {"os_mem_usage from ip": [["gs_total_memory_detail_mbytes{\"node name\": \"dn_6001_6002\", \"type\": \"dynamic_used_memory\"} from ip",1.0,0,[0.32950820234487...]]...]},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/app/memory-check	GET	获取内存占用情况	param latest_hours：可选，默认为0	返回当前数据库实例内存使用情况及其可能的风险	{"data": {"timestamps": [1684838536049],"values": [4618.0]},"remark":"too little data...","status":"unknown"},...,"topk_session_memory_sql": [{"application_name": "JobScheduler"...}...]}, "success":true}
/v1/api/app/risk-analysis/{metric}	GET	获取风险分析结果	param metric：必选，metric名称 param instance：可选，实例地址 param warning_hours：可选，风险分析时长 param upper：可选，metric的上限 param lower：可选，metric的下限 param labels：可选，metric的标签。支持如下两种格式： • “k1=v1,k2=v2”的kv pairs格式 • “{'k1':'v1','k2':'v2'}”的json字符串格式 • param tz：可选，时区参数，只支持UTC标准	返回当前指标未来风险分析结果	{"data": {},"success":true} **说明** 1. 该接口会基于历史数据对指标未来变化趋势进行预测，预测时会取三倍于warning_hours的历史数据作为训练数据，以预测未来metric的变化趋势，因此如果warning_hours取值过小导致历史数据不足，可能会导致训练数据过少而无法进行训练，最终无正常结果输出，另外大于等于48小时的预测结果可能会不准确。 2. 该接口适合预测具有趋势和周期变化的数据。 3. 当该接口中没有传递labels参数进行过滤时，如果满足条件的指标过多可能会导致接口超时。 4. 该预测功能基于AI模型，因此每次预测结果可能存在部分差异。

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例

/v1/api/app/anomaly-detection/detectors/{name}	PUT	添加新的异常检测检测器或修改当前检测器	param name：必选，检测器的名称
			需要传入json结构体:
			{
			"duration": 10,
			"forecasting_seconds": 0,
			"alarm_info": {
			"alarm_type": "SYSTEM",
			"alarm_level": "ERROR"
			},
			"detector_info": [
			{
			"detector_name": "EsdTestDetector",
			"metric_name": "gaussdb_blks_hit_ratio",
			"detector_kwargs": {
			"alpha": 0.05
			}
			}
			]
			}
			detector_info字段下检测算法的可选范围及对应检测参数，见表格下方说明
				返回添加状态
				成功返回Success: add {检测器名称} for {集群IP}
				失败返回Failed: add {检测器名称} for {集群IP}
					{"data":"Success: add detector for IP","success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/app/anomaly-detection/detectors/{name}	DELETE	删除指定的异常检测检测器	param name：必选，检测器的名称	返回删除状态	{"data":"Success: delete detector for IP","success":true}
/v1/api/app/anomaly-detection/detectors/{name}/pause	PUT	暂停指定的异常检测检测器	param name：必选，检测器的名称	返回暂停状态	{"data":"Success: pause detector for IP","success":true}
/v1/api/app/anomaly-detection/detectors/{name}/resume	PUT	继续指定的异常检测检测器	param name：必选，检测器的名称	返回继续状态	{"data":"Success: resume detector for IP","success":true}
/v1/api/app/anomaly-detection/detectors/{name}	GET	查看指定的异常检测检测器信息	param name：必选，检测器的名称	返回异常检测器信息	{"data":{"alarm_info":{"alarm_cause":null,"alarm_content":null,"alarm_level":"ERROR","alarm_type":"SYSTEM","extra":null},"detector_info":[...],"duration":10,"forecasting_seconds":0,"running":1},"success":true}
/v1/api/app/anomaly-detection/detectors/all/rebuild	PUT	根据后端落盘信息重建所有异常检测检测器	-	返回重建状态	{"data":"Success: rebuild detectors for IP","success":true}
/v1/api/app/anomaly-detection/detectors	DELETE	清空所有异常检测检测器	-	返回删除状态	{"data":"Success: clear detectors for IP","success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/app/workload-collection	GET	获取数据库工作负载数据	param data_source：必选，数据源，取值范围：('asp', 'statement_history', 'pg_stat_activity') param databases：可选，数据库列表 param schemas：可选，schema列表 param start_time：可选，开始时间 param end_time：可选，结束时间，仅适用于asp和statement_history param db_users：可选，数据库用户列表 param sql_types：可选，SQL语句类型，取值范围：('SELECT', 'UPDATE', 'DELETE', 'INSERT') param template_id：可选，SQL模板id param duration：可选，SQL语句的执行时间，单位	返回工作负载信息	{"data":{"header":["user_name","db_name","schema_name","application_name","unique_query_id","start_time","finish_time","duration","n_returned_rows","n_tuples_fetched","n_tuples_returned","n_tuples_inserted","n_tuples_updated","n_tuples_deleted","n_blocks_fetched","n_blocks_hit","n_soft_parse","n_hard_parse","db_time","cpu_time","parse_time","plan_time","data_io_time","lock_wait_time","lwlock_wait_time","query"],"rows":[["user1","db1","public"...]...]},"success":true} **说明** ● data_source参数数据源解释：asp为采样数据，statement_history为历史慢SQL数据，慢SQL阈值由用户指定，pg_stat_activity为实时SQL数据。 ● 为避免接口超时，返回数据量上限1000条。

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
			ms，仅适用于statement_history和pg_stat_activity		
/v1/api/app/cluster-diagnosis	GET	获取故障诊断结果	param instance：可选，被诊断的实例地址 param role：可选，实例的角色，取值范围：('cn', 'dn') param method：可选，诊断模型使用的方法，取值范围：('logical', 'tree') param timestamp：可选，诊断时间的时间戳，单位：时间戳（13位）	返回故障诊断结果	{"data":[{"bind_ip_failed":0,"cms_phonydead_restart":0,"cms_restart_pending":0,"dn_disk_damage":0,"dn_manual_stop":0,"dn_nic_down":0,"dn_ping_standby":0,"dn_port_conflict":0,"dn_read_only":0,"dn_status":0,"dn_writable":0,"ffic_updated":0,"ping":1},"DN down/disconnection"],"success":true}
/v1/api/summary/metrics	GET	获取所有指标列表	无	返回TSDB上的所有指标列表	{"data":["gaussdb_blks_hit_ratio","gaussdb_blks_read_rate"...],"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/metrics/{name}	GET	获取最新的指定名称序列的指标	param name：必选，指定metric的名称 param instance：可选，metric所属的实例地址 param latest_minutes：可选，指定的时间范围 param from_timestamp：可选，开始时间时间戳，适用于latest_minutes为空的情况 param to_timestamp：可选，结束时间时间戳，适用于latest_minutes为空的情况 param step：可选，获取数据的时间间隔 param fetch_all：可选，是否获取所有序列或只获取一次 param regex：可选，当instance非空，用正则表达式过滤instance param labels：可选，用指定标签过滤序列。支持如下两种格式： ● “k1=v1,k2=v2”的kv pairs格式	返回指定序列的详情。 如果prometheus短期内存入了大量数据，或者查询的区间过大，有可能导致查询超时，或由于内存不足导致查询失败。	{"data":[{"labels":{"device":"device","from_instance":"ip","from_job":"exporter","fstype":"ext4","instance":"ip:port","job":"exporter","mountpoint":"mounpoint"},"name":"os_disk_usage","timestamps":[1684900929939,1684900944939,1684900959939,1684900974939],"values":[0.7115459280083043,0.7115460512887748,0.7115461745692453,0.7115462978497158]}],"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
			• “{'k1':'v1','k2':'v2'}”的json字符串格式 param regex_labels：可选，使用正则表达式过滤序列。支持如下两种格式： • “k1=v1,k2=v2”的kv pairs格式 • “{'k1':'v1','k2':'v2'}”的json字符串格式 param limit：可选，限制返回结果行数 param tz：可选，时区参数，只支持UTC标准		

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/metrics/{name}	DELETE	删除指定范围的指标	param name：必选，指定metric的名称 param instance：可选，metric所属的实例地址 param from_timestamp：可选，开始时间时间戳 param to_timestamp：可选，结束时间时间戳 param regex：可选，当instance非空，用正则表达式过滤instance param labels：可选，用指定标签过滤序列。支持如下两种格式： ● “k1=v1,k2=v2”的kv pairs格式 ● “{'k1':'v1','k2':'v2'}”的json字符串格式 param regex_labels：可选，使用正则表达式过滤序列。支持如下两种格式： ● “k1=v1,k2=v2”的kv pairs格式 ● “{'k1':'v1','k2':'v2'}”的	返回删除状态（from_timestamp和to_timestamp虽然支持毫秒粒度，但是实际删除时仅支持到秒粒度，所以会出现一些临近边界时间范围的值无法删除的情况）	{"data":null,"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
			json字符串格式 param flush：可选，删除指标数据后是否刷新磁盘 param tz：可选，时区参数，只支持UTC标准		

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/alarms	GET	获取历史告警列表	param pagesize：可选，每页展示个数，建议传递 param current：可选，当前页，建议传递 param instance：可选，告警所属实例，如果不传递则返回当前集群内的告警 param alarm_type：可选，告警类型，参数可选范围为['SYSTEM', 'SLOW_QUERY', 'ALARM_LOG', 'ALARM', 'SECURITY', 'PERFORMANCE'] param alarm_level：可选，告警级别，参数可选范围为['CRITICAL', 'FATAL', 'ERROR', 'WARNING', 'WARN', 'INFO', 'NOTICE', 'DEBUG', 'NOTSET'] param metric_name：可选，指标名称 param start_at：可选，查询开始时间戳，建议传递 param end_at：可选，	返回历史告警列表	{"data":{"header":["history_alarm_id","instance","metric_name","metric_filter","alarm_type","alarm_level","start_at","end_at","alarm_content","extra_info","anomaly_type","alarm_cause"],"rows":[[65,"ip","os_mem_usage",null,"SYSTEM",30,1684762097001,1684762547001,"mem_usage_spike_detector:Find obvious spikes in memory usage.",null,"Spike"]]},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
			查询结束时间戳，建议传递 param anomaly_type：可选，异常类型 param group：可选，是否根据告警类型和内容划分告警		

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/alarms/count	GET	获取历史告警数量	param instance：可选，告警所属实例，如果不传递则返回当前集群内的告警 param alarm_type：可选，告警类型，参数可选范围为['SYSTEM', 'SLOW_QUERY', 'ALARM_LOG', 'ALARM', 'SECURITY', 'PERFORMANCE'] param alarm_level：可选，告警级别，参数可选范围为['CRITICAL', 'FATAL', 'ERROR', 'WARNING', 'WARN', 'INFO', 'NOTICE', 'DEBUG', 'NOTSET'] param metric_name：可选，指标名称 param start_at：可选，查询开始时间戳，不得晚于end_at，建议传递 param end_at：可选，查询结束时间戳，不得早于start_at，建议传递 param anomaly_type	返回历史告警数量	{"data":30,"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
			：可选，异常类型 param group：可选，是否根据告警类型和内容划分告警		

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/cluster-diagnosis	GET	获取历史集群诊断列表，集群诊断定时任务结果存储于元数据库中，若元数据库发生故障，结果无法插入表中，会导致部分数据遗漏	param pagesize：可选，每页展示个数，建议传递 param current：可选，当前页，建议传递 param instance：可选，实例IP，如果不传递则返回当前集群内的诊断结果 param instance_like：可选，实例IP模糊匹配，特性预埋字段，未实装 param start_at：可选，查询开始时间戳，仅返回timestamp大于等于该时间戳的结果，建议传递 param end_at：可选，查询结束时间戳，仅返回timestamp小于等于该时间戳的结果，建议传递（若start_at和end_at都不传递，则不对timestamp列进行任何值的筛选，返回全量时间戳的结果） param cluster_role：可选，节点类型 param diagnosis_meth	返回历史集群诊断列表	{"data":{"header":["diagnosis_id","instance","timestamp","cluster_role","diagnosis_method","cluster_feature","diagnosis_result","status_code","alarm_type","alarm_level"],"rows":[[1,ip,1695110107087,"cn","logical","{\"ping\": 0, \"cn_status\": 0, \"bind_ip_failed\": 0, \"panic\": 0, \"ffic_updated\": 0, \"cms_heartbeat_restart\": 0, \"cms_phonydead_restart\": 0, \"cn_dn_disconnected\": 0, \"cn_down_to_delete\": 0, \"cn_restart_time_exceed\": 0, \"cn_read_only\": 0, \"cn_restart\": 0, \"cn_start\": 0, \"cn_manual_stop\": 0, \"cn_disk_damage\": 0, \"cn_nic_down\": 0, \"cn_port_conflict\": 0}","Normal",-1,"ALARM",0]]},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
			od：可选，诊断算法 param status_code：可选，状态码，特性预埋字段，未实装 param alarm_type：可选，告警类型，参数可选范围为['SYSTEM', 'SLOW_QUERY', 'ALARM_LOG', 'ALARM', 'SECURITY', 'PERFORMANCE'] param alarm_level：可选，告警级别，参数可选范围为['CRITICAL', 'FATAL', 'ERROR', 'WARNING', 'WARN', 'INFO', 'NOTICE', 'DEBUG', 'NOTSET'] param is_normal：可选，是否包含诊断结果为Normal的数据		**说明** ● 返回结果中timestamp列所记录的时间戳以DBMind部署的服务器时间为准，如果元数据库所在服务器时间与DBMind部署服务器时间不一致，会导致元数据库中显示的时间戳结果与元数据库服务器实际时间不一致。 ● 返回结果按照以下两个规则排序： ● 按照timestamp从小到大排列； ● 当timestamp一致时按照diagnosis_id从小到大排列。

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/cluster-diagnosis/count	GET	获取历史集群诊断数量，集群诊断定时任务结果存储于元数据库中，若元数据库发生故障，结果无法插入表中，会导致部分数据遗漏	param instance：可选，实例IP，如果不传递则返回当前集群内的诊断结果 param instance_like：可选，实例IP模糊匹配，特性预埋字段，未实装 param start_at：可选，建议传递，查询开始时间戳，仅返回timestamp大于等于该时间戳的结果，建议传递 param end_at：可选，建议传递，查询结束时间戳，仅返回timestamp小于等于该时间戳的结果，建议传递 （若start_at和end_at都不传递，则不对timestamp列进行任何值的筛选，返回全量时间戳的结果） param cluster_role：可选，节点类型 param diagnosis_method：可选，诊断算法 param status_code：可选，状态码，	返回历史集群诊断数量	{"data":30,"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
			特性预埋字段，未实装 param alarm_type：可选，告警类型，参数可选范围为['SYSTEM', 'SLOW_QUERY', 'ALARM_LOG', 'ALARM', 'SECURITY', 'PERFORMANCE'] param alarm_level：可选，告警级别，参数可选范围为['CRITICAL', 'FATAL', 'ERROR', 'WARNING', 'WARN', 'INFO', 'NOTICE', 'DEBUG', 'NOTSET'] param is_normal：可选，是否包含诊断结果为Normal的数据		

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/sql/killed	GET	获取被终止的查询列表	param pagesize：可选，每页展示个数 param current：可选，当前页 param instance：可选，慢SQL所属实例 param query：可选，慢SQL文本 param start_time：可选，获取慢SQL的开始时间，单位：时间戳（13位） param end_time：可选，获取慢SQL的结束时间，单位：时间戳（13位）	返回被终止的查询列表	{"data":{"header":["instance","schema_name","db_name","query","template_id","hit_rate","fetch_rate","cpu_time","data_io_time","parse_time","plan_time","db_time","root_cause","suggestion","start_at","duration_time"],"rows":[(ip1, 'public', 'db1', 'select 1', t_id, ...), (...), ...]},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/sql/killed/count	GET	获取被终止的查询数量	param instance：可选，慢SQL所属实例 param query：可选，慢SQL文本 param start_time：可选，获取慢SQL的开始时间，单位：时间戳（13位） param end_time：可选，获取慢SQL的结束时间，单位：时间戳（13位）	返回被终止的查询数量	{"data":0,"success":true}
/v1/api/summary/sql/top	GET	获取执行最频繁的前十个SQL模板	无	返回执行最频繁的前十个SQL模板	{"data":{"header":["user_name","unique_sql_id","query","n_calls","min_elapse_time","max_elapse_time","avg_elapse_time","n_returned_rows","db_time","cpu_time","execution_time","parse_time","last_updated","sort_spill_count","hash_spill_count"],"rows":[["user",1961954918,"vacuum;",1,38958701,38958701,38958701,"3891978 1.218781218781",0,38959021...]]},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/sql/active	GET	获取当前活跃的SQL列表	无	返回当前活跃的SQL列表	{"data":{"header":["datname","usename","application_name","client_addr","query_start","waiting","state","query","connection_info"], "rows":[["data1","db1","app1","ip:port","time"...]]}, "success":true}
/v1/api/summary/sql/locking	GET	获取当前锁等待的SQL列表	无	返回当前锁等待的SQL列表	{"data":{"header":null,"rows":[]},"success":true}
/v1/api/app/index-recommendation	POST	获取建议的索引列表	param instance：必选，数据库实例 param database：可选，数据库名称 param max_index_num：可选，推荐索引个数的最大值 param max_index_storage：可选，推荐索引的内存使用的最大值 param sqls：可选，SQL语句列表	返回建议的索引列表	{"data":[{"advise_indexes":[],"redundant_indexes":[],"total":0,"useless_indexes":[{"columns":"c1","schemaName":"public","statement":"DROP INDEX t1_c1_idx;","tbName":"t1","type":3},{"columns":"c2","schemaName":"public","statement":"DROP INDEX t2_c2_idx;","tbName":"t2","type":3}]},{}],"success":true}
/v1/api/summary/knob-recommendation/snapshots	GET	获取指标快照	param pagesize：可选，每页展示个数 param current：可选，当前页	返回指标快照	{"data":{"header":["instance","metric","value"],"rows":[]},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/knob-recommendation/snapshots/count	GET	获取快照数量	无	返回快照数量	{"data":0,"success":true}
/v1/api/summary/knob-recommendation/warnings	GET	获取指标报警值	param pagesize：可选，每页展示个数 param current：可选，当前页	返回指标报警值	{"data":{"header":["instance","level","comment"],"rows":[]},"success":true}
/v1/api/summary/knob-recommendation/warnings/count	GET	获取指标报警值数量	无	返回指标报警值数量	{"data":0,"success":true}
/v1/api/summary/knob-recommendation/details	GET	获取参数推荐列表	param pagesize：可选，每页展示个数 param current：可选，当前页	返回参数推荐列表	{"data":{"header":["instance","name","current","recommend","min","max"],"rows":[]},"success":true}
/v1/api/summary/knob-recommendation/details/count	GET	获取参数推荐数量	无	返回参数推荐数量	{"data":0,"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/sql/slow	GET	获取慢查询诊断统计信息	无	返回慢查询诊断统计信息	{"data":{"distribution":{"delete":0,"insert":0,"select":0,"update":0},"main_slow_queries":0,"mean_buffer_hit_rate":-1,"mean_cpu_time":-1,"mean_fetch_rate":-1,"mean_io_time":-1,"nb_unique_slow_queries":0,"slow_query_count":{"timestamps":[],"values":[]},"slow_query_template":{"header":["template_id","count","query"],"rows":[]},"slow_query_threshold":2.0,"statistics_for_database":{},"statistics_for_schema":{},"systable":{"business_table":0,"system_table":0}},"success":true}
/v1/api/summary/regular-inspection	GET	获取定期巡检结果列表	param inspection_type：必选，检测类型，取值范围：('daily_check', 'weekly_check', 'monthly_check')	返回定期巡检结果列表	{"data":{"header":["instance","report","start","end"],"rows":[["ip:port",{"connection":{"active_connection":{"avg":3.0,"max":4.0,"min":1.0,"the_95th":4.0},"total_connection":{"avg":3.2321,"max":5.0,"min":1.0,"the_95th":4.0}}...},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/regular-inspection/count	GET	获取定期巡检结果数量	param inspection_type：必选，检测类型，取值范围：('daily_check', 'weekly_check', 'monthly_check')	返回定期巡检结果数量	{"data":9,"success":true}
/v1/api/summary/database-list	GET	获取数据库列表	无	返回数据库列表	{"data":["db1","db2","db3","db4","db5"...],"success":true}
/v1/api/summary/sql/slow/latest	GET	获取最近的慢查询列表	param instance：可选，慢SQL所属的实例地址 param query：可选，query文本 param start_time：可选，慢SQL开始运行时间，单位：时间戳（13位） param end_time：可选，慢SQL结束运行时间，单位：时间戳（13位） param group：可选，查询时是否使用group逻辑	返回最近的慢查询列表	{"data":{"header":["instance","schema_name","db_name","query","template_id","hit_rate","fetch_rate","cpu_time","data_io_time","parse_time","plan_time","db_time","root_cause","suggestion","start_at","duration_time"],"rows":[(ip1, 'public', 'db1', 'select 1', t_id, ...), (...), ...]},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/sql/slow/latest/count	GET	获取最近的慢查询数量	param pagesize：可选，查询页返回数量 param current：可选，当前页数 param instance：可选，慢SQL所属的实例地址 param distinct：可选，查询时是否使用distinct逻辑 param query：可选，query文本 param start_time：可选，慢SQL开始运行时间，单位：时间戳（13位） param end_time：可选，慢SQL结束运行时间，单位：时间戳（13位） param group：可选，查询时是否使用group逻辑	返回最近的慢查询数量	{"data":0,"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/app/metric-diagnosis	GET	执行指标异常诊断，当前版本不可用。	param metric_name：必选，指标名称。取值范围：os_cpu_user_usage，pg_thread_pool_rate，os_mem_usage，os_disk_usage，os_disk_io_read_delay param metric_filter：必选，筛选指标。支持如下两种格式： ● “k1=v1,k2=v2”的kv pairs格式 ● “{'k1':'v1','k2':'v2'}”的json字符串格式 param alarm_cause：必选，选择分析方法。取值范围：high_cpu_usage，high_thread_pool_rate, high_dynamic_mem_usage, high_shared_mem_usage, high_disk_usage, high_io_delay param start：可选，分析指标开始时间戳，单位毫秒	返回指标异常根因	{"data":{[{'reason1': 0.0, 'reason2': 1.0}, 'conclusion', 'advice']},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
			param end：可选，分析指标结束时间戳，单位毫秒		

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例

/v1/api/app/real-time-inspection	POST	执行智能巡检功能	param inspection_type：必选，巡检类型
			param start_time：可选，起始时间
			param end_time：可选，终止时间
			param instance：必选，实例IP
			param inspection_items：必选，巡检项
			param tz：可选，时区参数，只支持UTC标准
				返回巡检结果	{
					"system_resource": {
					"os_mem_usage": {
					"192.168.0.1": {
					"statistic": {
					"max": 0.4154,
					"min": 0.4128,
					"avg": 0.4149,
					"the_95th": 0.4153
					},
					"warnings": {
					"increase_warning": false,
					"threshold_warning": [],
					"forecast_warning": {
					"occur_time": "",
					"remaining_hours": 0.0,
					"risk": "",
					"timestamps": [],
					"values": []
					}
					},
					"timestamps": [1689210000000, 1689210360000],
					"data": [0.41450417776587234, 0.41450179472976234]
					},
					"192.168.0.2": {
					xxx
					}
					}
					},

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
					"instance_status": {}, "database_resource": {}, "database_performance": {}, "diagnosis_optimization": {}, "conclusion": {} }
/v1/api/app/real-time-inspection/list	GET	展示巡检任务的基础信息	param instance：必选，实例IP	返回巡检任务的基础信息	{"data":{"header":["instance","start","end","id","state","cost_time","inspection_type"],"rows":[["192.168.0.1:8080",1689210000000,1689296400000,5,"success",0.033701,"real_time_check"]]},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例

/v1/api/summary/real-time-inspection	GET	获取指定巡检任务的巡检结果	param instance：必选，实例IP
			param spec_id：必选，巡检任务ID
				返回指定巡检任务的巡检结果	{
					"system_resource": {
					"os_mem_usage": {
					"192.168.0.1": {
					"statistic": {
					"max": 0.4154,
					"min": 0.4128,
					"avg": 0.4149,
					"the_95th": 0.4153
					},
					"warnings": {
					"increase_warning": false,
					"threshold_warning": [],
					"forecast_warning": {
					"occur_time": "",
					"remaining_hours": 0.0,
					"risk": "",
					"timestamps": [],
					"values": []
					}
					},
					"timestamps": [1689210000000, 1689210360000],
					"data": [0.41450417776587234, 0.41450179472976234]
					},
					"192.168.0.2": {
					xxx
					}
					}
					},

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
					"instance_status": {}, "database_resource": {}, "database_performance": {}, "diagnosis_optimization": {}, "conclusion": {} }
/v1/api/app/real-time-inspection	DELETE	删除指定的巡检任务	param instance：必选，实例IP param spec_id：必选，巡检任务ID	返回删除状态	{"data": {"success":true},"success":true}
/v1/api/check-status	POST	获取DBMind服务的状态信息并返回状态详情	param cmd：可选，服务启动命令	返回状态详情	{"data": {"error_msg":"","result": {},"state":"NORMAL"},"success":true}
/v1/api/repair	POST	修复DBMind服务并返回修复结果	param cmd：可选，服务启动命令	返回修复结果	{"data": {"error_msg":"","result": {},"state":"SUCCESS"},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例

/v1/api/summary/sql-trace	GET	SQL全链路查询	param instance_id：必选，实例id
			param labels：必选，过滤使用的指标标签
			param is_online：必选，数据来源是否是在线的
			param from_timestamp：可选，查询开始时间
			param to_timestamp：可选，查询结束时间
			param tz：可选，时区参数，仅支持UTC标准
				返回SQL全链路信息	{
					"data": [
					{
					"all_time": 3477357,
					"application_name": "xxx",
					"client_addr": "",
					"client_port": 0,
					"component_id": "xxx",
					"db_name": "xxx",
					"execution_time_details": {
					"kernel_time": {
					"all_time": 3477357,
					"kernel_time_details": {
					"execution_time": 3466386,
					"other_time": 4958,
					"parse_time": 453,
					"plan_time": 5244,
					"rewrite_time": 316
					}
					},
					"resource_time": {
					"all_time": 3477357,
					"resource_time_details": {
					"cpu_time": 98756,
					"data_io_time": 0,
					"other_time": 3378601
					}
					},
					"wait_event_time": {
					"code_wait_event_time": {

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例

					"all_time": 3477357,
					"code_wait_event_time_details": {
					"events": [
					{
					"event_name": "xxx",
					"event_time": 16810
					},
					{
					"event_name": "xxx",
					"event_time": 5330
					},
					{
					"event_name": "xxx",
					"event_time": 5195
					},
					{
					"event_name": "xxx",
					"event_time": 197
					}
					],
					"left_time": 0,
					"other_time": 3449825
					}
					},
					"resource_wait_event_time": {
					"all_time": 3477357,
					"other_time": 3477357,
					"resource_wait_event_time_details": {
					"data_io_time": {
					"all_time": 0,
					"data_io_time_details": {
					"events": [

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例

					{
					"event_name": "xxx",
					"event_time": 742
					}
					],
					"left_time": 0,
					"other_time": -27532
					}
					},
					"lock_time": {
					"all_time": 0,
					"lock_time_details": {
					"events": [],
					"left_time": 0,
					"other_time": 0
					}
					},
					"lwlock_time": {
					"all_time": 0,
					"lwlock_time_details":
					{
					"events": [],
					"left_time": 0,
					"other_time": 0
					}
					}
					}
					}
					}
					},
					"finish_time":
					"2023-08-01
					17:50:04.923701+08",
					"node_id": "0",
					"schema_name":
					"user,public",

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
					"session_id": 1658590, "sql_exec_id": 72339069024193250, "sql_id": 1951225884, "start_time": "2023-08-01 17:50:01.446446+08", "trace_id": "0", "transaction_id": "0", "user_name": "user" }], "success": true }
/v1/api/summary/metric-unit/{metric}	GET	指标单位查询	param metric：必选，指标名	返回**指标单位**	{"data":"{'en': 'rate', 'cn': '比率'}","success":true}
/v1/api/security/scenarios	GET	自安全，当前版本不可用	无	返回所有安全异常类型	{"data": ["scanning_attack", "brute_force_login_attack","user_violation_rate"],"success":true}
/v1/api/security/scenarios/{name}	GET	自安全，当前版本不可用	param name：安全指标名称	返回安全异常包含的指标的校准状态	{"data": [{"metric":"gaussdb_log_errors_rate","status":"CALIBRATED"}, {"metric":"gaussdb_user_violation_rate","status":"CALIBRATED"}], "success":true}

版权所有 © 华为云计算技术有限公司

说明

对异常检测器检测算法的说明如下，也可以通过调用/v1/api/configs/anomaly-detection/default-settings获得：

1. 学生化残差检测器-EsdTestDetector：{"alpha":0.05}。
2. 梯度检测器-GradientDetector：{"max_coef":1,"side":["positive",["positive","negative","both"]]}。
3. 单调趋势检测器-IncreaseDetector：{"alpha":null,"side":["positive",["positive","negative","both"]]}。
4. 四分位间距检测器-InterQuartileRangeDetector：{"outliers":[3,3]}。
5. 均值漂移检测器-LevelShiftDetector：{"agg":["median",["median","mean","std"]],"outliers":[null,6], "side":["both",["positive","negative","both"]],"window":5}。
6. 分位数检测器-QuantileDetector：{"high":1,"low":0}。
7. 周期数据检测器-SeasonalDetector：{"high_ac_threshold":0.1,"min_seasonal_freq":2,"outliers":[null,3], "period":null,"side":["positive,["positive","negative","both"]],"window":10}"。
8. 尖峰检测器-SpikeDetector：{"agg":["median",["median","mean","std"]],"outliers":[null,3], "side":["both",["positive","negative","both"]],"window":1}。
9. 阈值检测器-ThresholdDetector：{"high":null,"low":null,"percentage":[null,[0,1]]}。
10. 波动率漂移检测器-VolatilityShiftDetector：{"agg":["std",["median","mean","std"]],"outliers":[null,6], "side":["both",["positive","negative","both"]],"window":10}。

说明

- alarm_level参数的所有级别及其对应的权重：CRITICAL、FATAL：50，ERROR：40，WARNING、WARN：30，INFO、NOTICE：20，DEBUG：10，NOTSET：0。
- alarm_type参数的所有类型：SYSTEM、SLOW_QUERY、ALARM_LOG、ALARM、SECURITY、PERFORMANCE。
- anomaly_type参数的所有类型：high_disk_usage_detector、high_mem_usage_detector、high_cpu_usage_detector、high_thread_pool_rate_detector、high_io_delay_detector、deadlock_detector、slow_disk_detector、mem_leak_detector、core_detector。

指标单位查询接口

/v1/api/summary/metric-unit/{metric} 支持的指标列表如下表所示：

tps	gs_redo_stat_writetim	process_start_time_seconds	node_netstat_Udp6_SndbufErrors
qps	gs_index_idx_blks_hit	gs_audit_user_locked_total	node_netstat_UdpLite6_InErrors
node_load1	gaussdb_log_node_start	gs_connections_enqueue_sql	node_rapl_package_joules_total
node_load5	gaussdb_log_gtm_status	gs_session_memory_usedsize	node_timex_pps_frequency_hertz
node_load15	pg_checkpoint_redo_lsn	gaussdb_progress_leaked_fds	node_timex_pps_stability_hertz

版权所有 © 华为云计算技术有限公司

os_disk_iops	pg_checkpoint_next_oid	gaussdb_log_dn_ping_standby	gs_long_xact_wait_events_count
os_mem_usage	pg_database_size_bytes	pg_connections_idle_session	gs_checkpoint_full_page_writes
pg_time_value	pg_db_confl_tablespace	pg_global_ckpt_status_value	gs_lock_time_info_holding_time
os_cpu_iowait	pg_redo_stat_phyblkwrt	pg_memory_context_totalsize	gs_recovery_status_current_rto
os_disk_usage	pg_sql_count_mergeinto	pg_session_memory_totalsize	gs_replication_slots_delay_lsn
load_average1	pg_stat_activity_count	pg_stat_get_wal_senders_pid	gs_stat_bgwriter_total_seconds
os_disk_await	pg_index_idx_blks_read	pg_sql_statement_full_count	gs_stat_replication_flush_diff
gs_time_value	pg_tables_size_relsize	gaussdb_invalid_logins_rate	gs_stat_replication_replay_lag
pg_db_blks_hit	os_disk_io_write_bytes	gaussdb_user_violation_rate	gs_stat_replication_replay_lsn
gs_db_blks_hit	os_disk_io_write_delay	gaussdb_confl_snapshot_rate	gs_stat_replication_write_diff
pg_db_blks_read	gaussdb_blks_read_rate	gaussdb_confl_deadlock_rate	gs_summary_user_logout_counter
pg_db_conflicts	gaussdb_blks_hit_ratio	gaussdb_table_increase_rate	gs_wait_events_total_wait_time
pg_db_deadlocks	gaussdb_conflicts_rate	gaussdb_index_increase_rate	gs_tables_structure_n_tup_read
pg_locker_count	gaussdb_deadlocks_rate	node_context_switches_total	gs_tables_structure_n_live_tup
pg_class_relage	node_boot_time_seconds	node_entropy_available_bits	gs_tables_structure_n_dead_tup
pg_index_mbytes	node_cpu_seconds_total	node_entropy_pool_size_bits	gaussdb_log_cms_restart_pending
os_disk_ioutils	node_memory_Slab_bytes	node_filesystem_avail_bytes	pg_checkpoint_next_multixact_id
node_intr_total	node_network_mtu_bytes	node_hwmon_temp_max_celsius	pg_checkpoint_next_multi_offset
node_network_up	node_timex_sync_status	node_memory_AnonPages_bytes	pg_checkpoint_oldest_active_xid

版权所有 © 华为云计算技术有限公司

node_udp_queues	node_vmstat_pgmajfault	node_memory_HugePages_Total	pg_stat_replication_replay_diff
process_max_fds	gs_checkpoint_redo_lsn	node_memory_SwapTotal_bytes	pg_tables_structure_last_vacuum
gs_db_blks_read	gs_checkpoint_next_oid	node_memory_Writeback_bytes	node_disk_io_time_seconds_total
gs_db_conflicts	gs_database_size_bytes	node_netstat_Icmp6_InErrors	node_disk_reads_completed_total
gs_db_deadlocks	gs_db_confl_tablespace	node_netstat_IpExt_InOctets	node_memory_AnonHugePages_bytes
gs_locker_count	gs_redo_stat_phyblkwrt	node_rapl_dram_joules_total	node_memory_Inactive_anon_bytes
gs_class_relage	gs_sql_count_mergeinto	node_sockstat_TCP_mem_bytes	node_memory_Inactive_file_bytes
gs_index_mbytes	gs_stat_activity_count	node_sockstat_UDPLITE_inuse	node_netstat_TcpExt_ListenDrops
gaussdb_ping_lag	gs_index_idx_blks_read	node_sockstat_UDP_mem_bytes	node_netstat_TcpExt_TCPTimeouts
gaussdb_log_ffic	gs_tables_size_relsize	node_timex_maxerror_seconds	node_network_receive_drop_total
pg_db_confl_lock	gaussdb_log_cms_cn_down	node_timex_pps_jitter_total	node_network_receive_errs_total
pg_db_temp_bytes	pg_audit_ddl_user_total	gs_connections_idle_session	node_network_receive_fifo_total
pg_db_temp_files	pg_connections_max_conn	gs_global_ckpt_status_value	node_nf_conntrack_entries_limit
pg_sql_count_dcl	pg_memory_context_count	gs_memory_context_totalsize	node_schedstat_timeslices_total
pg_sql_count_ddl	pg_prepared_xacts_count	gs_session_memory_totalsize	gs_checkpoint_next_multixact_id
pg_sql_count_dml	pg_stat_replication_lsn	gs_stat_get_wal_senders_pid	gs_checkpoint_next_multi_offset
pg_class_relsize	os_disk_io_queue_length	gs_sql_statement_full_count	gs_checkpoint_oldest_active_xid
gaussdb_cpu_time	os_cpu_processor_number	pg_pagewriter_dirty_page_num	gs_stat_replication_replay_diff
node_arp_entries	gaussdb_qps_by_instance	pg_audit_user_unlocked_total	gs_tables_structure_last_vacuum

版权所有 © 华为云计算技术有限公司

node_disk_io_now	gaussdb_log_errors_rate	pg_checkpoint_checkpoint_lsn	pg_audit_successful_logins_total
node_forks_total	gaussdb_confl_lock_rate	pg_sesstype_memory_totalsize	pg_replication_slots_restart_lsn
process_open_fds	os_network_receive_drop	pg_stat_replication_sent_lsn	pg_never_used_indexes_index_size
gs_db_confl_lock	gaussdb_corrosion_index	pg_tables_structure_tbl_scan	pg_tables_structure_last_analyze
gs_db_temp_bytes	gaussdb_statement_calls	gaussdb_confl_bufferpin_rate	pg_tables_structure_vacuum_count
gs_db_temp_files	gaussdb_idle_connection	gaussdb_statement_sort_spill	gaussdb_table_increase_amplitude
gs_sql_count_dcl	node_hwmon_temp_celsius	gaussdb_statement_hash_spill	node_cpu_package_throttles_total
gs_sql_count_ddl	node_memory_Dirty_bytes	node_cpu_frequency_max_hertz	node_cpu_scaling_frequency_hertz
gs_sql_count_dml	node_memory_Shmem_bytes	node_cpu_frequency_min_hertz	node_disk_writes_completed_total
gs_class_relsize	node_netstat_Tcp_InErrs	node_cpu_guest_seconds_total	node_hwmon_power_is_battery_watt
gaussdb_nic_state	node_netstat_Tcp_InSegs	node_disk_reads_merged_total	node_network_address_assign_type
gaussdb_log_panic	node_sockstat_RAW_inuse	node_filesystem_device_error	node_network_receive_bytes_total
pg_checkpoint_tli	node_sockstat_TCP_alloc	node_hwmon_temp_crit_celsius	node_network_receive_frame_total
pg_db_blks_access	node_sockstat_TCP_inuse	node_memory_PageTables_bytes	node_network_transmit_drop_total
pg_db_reset_delay	node_sockstat_UDP_inuse	node_memory_SUnreclaim_bytes	node_network_transmit_errs_total
pg_db_tup_deleted	node_timex_tick_seconds	node_memory_SwapCached_bytes	node_network_transmit_fifo_total
pg_db_tup_fetched	scrape_duration_seconds	node_netstat_IpExt_OutOctets	node_timex_pps_calibration_total
pg_db_tup_updated	gs_audit_ddl_user_total	node_netstat_Tcp_ActiveOpens	process_virtual_memory_max_bytes
pg_db_xact_commit	gs_connections_max_conn	node_netstat_Tcp_RetransSegs	gs_audit_successful_logins_total

版权所有 © 华为云计算技术有限公司

pg_class_relpages	gs_memory_context_count	node_netstat_Udp_InDatagrams	gs_replication_slots_restart_lsn
pg_index_idx_scan	gs_prepared_xacts_count	node_network_iface_link_mode	gs_never_used_indexes_index_size
os_cpu_user_usage	gs_stat_replication_lsn	node_sockstat_UDPLITE6_inuse	gs_tables_structure_last_analyze
os_cpu_idle_usage	gaussdb_ping_packet_rate	node_softnet_processed_total	gs_tables_structure_vacuum_count
gaussdb_cpu_usage	gaussdb_log_login_denied	node_timex_pps_shift_seconds	gaussdb_log_user_cancel_statement
node_time_seconds	gaussdb_log_errors_total	process_virtual_memory_bytes	gaussdb_log_cms_phonydead_restart
node_timex_status	gaussdb_log_flow_control	gs_pagewriter_dirty_page_num	gaussdb_log_cms_heartbeat_timeout
gs_checkpoint_tli	gaussdb_log_node_restart	gs_audit_user_unlocked_total	pg_shared_memory_detail_totalsize
gs_db_blks_access	pg_checkpoint_oldest_xid	gs_checkpoint_checkpoint_lsn	pg_stat_replication_sync_priority
gs_db_reset_delay	pg_connections_used_conn	gs_sesstype_memory_totalsize	pg_tables_structure_n_tup_hot_upd
gs_db_tup_deleted	pg_connections_used_rate	gs_stat_replication_sent_lsn	pg_tables_structure_analyze_count
gs_db_tup_fetched	pg_lock_sql_locked_times	gs_tables_structure_tbl_scan	pg_sql_statement_history_exc_time
gs_db_tup_updated	pg_sesstype_memory_count	gaussdb_log_lock_wait_timeout	node_disk_read_time_seconds_total
gs_db_xact_commit	pg_tables_size_totalsize	gaussdb_log_undo_invalidation	node_hwmon_temp_crit_alar_celsius
gs_class_relpages	pg_tables_size_indexsize	gaussdb_log_undo_impermission	node_netstat_TcpExt_TCPSynRetrans
gs_index_idx_scan	pg_tables_size_toastsize	pg_session_memory_detail_size	node_network_transmit_bytes_total
gaussdb_ping_state	gaussdb_user_locked_rate	pg_autovacuum_worker_duration	node_network_transmit_colls_total
gaussdb_xlog_count	gaussdb_tup_fetched_rate	pg_audit_invalid_logins_total	node_softnet_times_squeezed_total

版权所有 © 华为云计算技术有限公司

pg_txid_oldestxmin	gaussdb_tup_deleted_rate	pg_audit_user_violation_total	gs_shared_memory_detail_totalsize
pg_checkpoint_time	gaussdb_tup_updated_rate	pg_checkpoint_oldest_xid_dbid	gs_stat_replication_sync_priority
pg_db_tup_inserted	os_network_receive_error	pg_stat_replication_flush_lsn	gs_tables_structure_n_tup_hot_upd
pg_db_tup_returned	os_network_transmit_drop	pg_stat_replication_sent_diff	gs_tables_structure_analyze_count
pg_class_reltuples	os_network_receive_bytes	pg_stat_replication_write_lsn	gs_sql_statement_history_exc_time
os_disk_iocapacity	gaussdb_wait_event_spike	pg_summary_user_login_counter	gaussdb_log_cn_restart_time_exceed
gaussdb_state_time	gaussdb_total_connection	pg_total_memory_detail_mbytes	pg_logical_replication_slots_count
gaussdb_user_login	node_filesystem_readonly	pg_tables_structure_n_tup_mod	pg_lock_time_info_p95_holding_time
os_mem_total_bytes	node_memory_Active_bytes	pg_tables_structure_n_tup_ins	pg_session_memory_detail_totalsize
node_network_flags	node_memory_Bounce_bytes	pg_tables_structure_n_tup_upd	pg_stat_replication_backend_uptime
node_procs_blocked	node_memory_Cached_bytes	pg_tables_structure_n_tup_del	node_disk_write_time_seconds_total
node_procs_running	node_memory_Mapped_bytes	pg_tables_structure_dead_rate	node_edac_correctable_errors_total
node_vmstat_pgpgin	node_netstat_Icmp_InMsgs	gaussdb_confl_temp_files_rate	node_hwmon_temp_crit_alarm_celsius
node_vmstat_pswpin	node_netstat_Tcp_OutRsts	gaussdb_confl_temp_bytes_rate	node_netstat_TcpExt_SyncookiesRecv
gs_txid_oldestxmin	node_netstat_Tcp_OutSegs	node_cooling_device_cur_state	node_netstat_TcpExt_SyncookiesSent
gs_checkpoint_time	node_netstat_Udp_NoPorts	node_cooling_device_max_state	node_network_carrier_changes_total
gs_db_tup_inserted	node_network_speed_bytes	node_cpu_core_throttles_total	node_network_receive_packets_total
gs_db_tup_returned	node_sockstat_FRAG_inuse	node_disk_writes_merged_total	node_network_transmit_queue_length

版权所有 © 华为云计算技术有限公司

gs_class_reltuples	node_sockstat_RAW6_inuse	node_disk_written_bytes_total	node_timex_estimated_error_seconds
gaussdb_mount_state	node_sockstat_TCP6_inuse	node_hwmon_power_average_watt	gs_logical_replication_slots_count
pg_temp_files_count	node_sockstat_TCP_orphan	node_memory_Active_anon_bytes	gs_lock_time_info_p95_holding_time
pg_downstream_count	node_sockstat_UDP6_inuse	node_memory_Active_file_bytes	gs_session_memory_detail_totalsize
pg_db_blk_read_time	gs_checkpoint_oldest_xid	node_memory_CommitLimit_bytes	gs_stat_replication_backend_uptime
pg_db_xact_rollback	gs_connections_used_conn	node_memory_DirectMap1G_bytes	pg_global_double_write_status_value
pg_node_info_uptime	gs_connections_used_rate	node_memory_DirectMap2M_bytes	pg_tables_structure_last_autovacuum
pg_sql_count_select	gs_lock_sql_locked_times	node_memory_DirectMap4k_bytes	pg_sql_statement_statistics_n_calls
pg_sql_count_insert	gs_sesstype_memory_count	node_memory_KernelStack_bytes	pg_sql_statement_statistics_db_time
pg_sql_count_delete	gs_tables_size_totalsize	node_memory_Unevictable_bytes	gaussdb_database_increase_amplitude
pg_sql_count_update	gs_tables_size_indexsize	node_memory_VmallocUsed_bytes	node_memory_HardwareCorrupted_bytes
pg_thread_pool_rate	gs_tables_size_toastsize	node_netstat_Tcp_PassiveOpens	node_netstat_TcpExt_ListenOverflows
pg_settings_setting	gaussdb_log_cms_read_only	node_netstat_Udp6_InDatagrams	node_network_transmit_carrier_total
oldestxmin_increase	pg_long_transaction_count	node_netstat_UdpLite_InErrors	node_network_transmit_packets_total
os_cpu_iowait_usage	pg_long_xact_mem_ctx_size	node_netstat_Udp_OutDatagrams	gs_global_double_write_status_value

版权所有 © 华为云计算技术有限公司

os_cpu_system_usage	pg_state_memory_totalsize	node_netstat_Udp_RcvbufErrors	gs_tables_structure_last_autovacuum
os_process_fds_rate	gaussdb_tup_inserted_rate	node_netstat_Udp_SndbufErrors	gs_sql_statement_statistics_n_calls
gaussdb_user_logout	os_network_transmit_error	node_scrape_collector_success	gs_sql_statement_statistics_db_time
node_filefd_maximum	os_network_transmit_bytes	node_time_zone_offset_seconds	pg_tables_structure_last_autoanalyze
node_vmstat_pgfault	gaussdb_statement_db_time	node_timex_loop_time_constant	node_cpu_scaling_frequency_max_hertz
node_vmstat_pgpgout	gaussdb_active_connection	node_timex_pps_jitter_seconds	node_cpu_scaling_frequency_min_hertz
node_vmstat_pswpout	node_memory_Buffers_bytes	node_timex_tai_offset_seconds	node_edac_uncorrectable_errors_total
gs_temp_files_count	node_memory_CmaFree_bytes	process_resident_memory_bytes	node_netstat_TcpExt_SyncookiesFailed
gs_downstream_count	node_memory_MemFree_bytes	gs_session_memory_detail_size	node_network_receive_multicast_total
gs_db_blk_read_time	node_memory_Mlocked_bytes	gs_autovacuum_worker_duration	node_schedstat_running_seconds_total
gs_db_xact_rollback	node_netstat_Icmp6_InMsgs	gs_audit_invalid_logins_total	node_schedstat_waiting_seconds_total
gs_node_info_uptime	node_netstat_Icmp_OutMsgs	gs_audit_user_violation_total	promhttp_metric_handler_errors_total
gs_sql_count_select	node_netstat_Ip6_InOctets	gs_checkpoint_oldest_xid_dbid	gs_tables_structure_last_autoanalyze
gs_sql_count_insert	node_netstat_Udp6_NoPorts	gs_stat_replication_flush_lsn	pg_summary_file_iostat_total_phyblkrd
gs_sql_count_delete	node_netstat_Udp_InErrors	gs_stat_replication_sent_diff	statement_responsetime_percentile_p80
gs_sql_count_update	node_nf_conntrack_entries	gs_stat_replication_write_lsn	statement_responsetime_percentile_p95

版权所有 © 华为云计算技术有限公司

gs_thread_pool_rate	node_sockstat_FRAG6_inuse	gs_summary_user_login_counter	node_network_receive_compressed_total
gs_settings_setting	node_sockstat_FRAG_memory	gs_total_memory_detail_mbytes	node_timex_frequency_adjustment_ratio
pg_audit_grant_total	node_timex_offset_seconds	gs_tables_structure_n_tup_mod	gs_summary_file_iostat_total_phyblkrd
pg_checkpoint_elapse	process_cpu_seconds_total	gs_tables_structure_n_tup_ins	pg_summary_file_iostat_total_phyblkwrt
pg_db_blk_write_time	gs_long_transaction_count	gs_tables_structure_n_tup_upd	net_conntrack_dialer_conn_closed_total
pg_db_confl_deadlock	gs_long_xact_mem_ctx_size	gs_tables_structure_n_tup_del	net_conntrack_dialer_conn_failed_total
pg_db_confl_snapshot	gs_state_memory_totalsize	gs_tables_structure_dead_rate	node_network_transmit_compressed_total
gaussdb_state_memory	gaussdb_progress_cpu_usage	gaussdb_log_dn_writable_failed	node_scrape_collector_duration_seconds
pg_database_all_size	gaussdb_progress_mem_usage	pg_long_xact_wait_events_count	promhttp_metric_handler_requests_total
node_network_carrier	gaussdb_log_deadlock_count	pg_checkpoint_full_page_writes	gs_summary_file_iostat_total_phyblkwrt
node_sockstat_TCP_tw	gaussdb_log_bind_ip_failed	pg_lock_time_info_holding_time	gaussdb_log_gtm_disconnected_to_primary
gs_database_all_size	pg_audit_user_locked_total	pg_recovery_status_current_rto	pg_session_connection_total_session_num
gs_audit_grant_total	pg_connections_enqueue_sql	pg_replication_slots_delay_lsn	node_timex_pps_stability_exceeded_total
gs_checkpoint_elapse	pg_session_memory_usedsize	pg_stat_bgwriter_total_seconds	gs_session_connection_total_session_num
gs_db_blk_write_time	node_disk_read_bytes_total	pg_stat_replication_flush_diff	net_conntrack_listener_conn_closed_total
gs_db_confl_deadlock	node_filesystem_files_free	pg_stat_replication_replay_lag	node_disk_io_time_weighted_seconds_total

版权所有 © 华为云计算技术有限公司

gs_db_confl_snapshot	node_filesystem_free_bytes	pg_stat_replication_replay_lsn	node_edac_csrow_correctable_errors_total
gaussdb_cluster_state	node_filesystem_size_bytes	pg_stat_replication_write_diff	gaussdb_log_cms_heartbeat_timeout_restart
gaussdb_log_cn_status	node_memory_CmaTotal_bytes	pg_summary_user_logout_counter	pg_stat_bgwriter_checkpoint_avg_sync_time
gaussdb_log_dn_status	node_memory_HugePages_Free	pg_wait_events_total_wait_time	net_conntrack_dialer_conn_attempted_total
gaussdb_log_gtm_panic	node_memory_HugePages_Rsvd	pg_tables_structure_n_tup_read	node_hwmon_power_average_interval_seconds
pg_audit_logout_total	node_memory_HugePages_Surp	pg_tables_structure_n_live_tup	gs_stat_bgwriter_checkpoint_avg_sync_time
pg_audit_revoke_total	node_memory_Inactive_bytes	pg_tables_structure_n_dead_tup	net_conntrack_listener_conn_accepted_total
pg_cpu_load_total_cpu	node_memory_MemTotal_bytes	gaussdb_successful_logins_rate	node_edac_csrow_uncorrectable_errors_total
pg_db_confl_bufferpin	node_memory_SwapFree_bytes	gaussdb_connections_used_ratio	promhttp_metric_handler_requests_in_flight
pg_redo_stat_writetim	node_netstat_Icmp6_OutMsgs	gaussdb_dead_tup_increase_rate	pg_tables_structure_last_data_changed_delay
pg_index_idx_blks_hit	node_netstat_Icmp_InErrors	gaussdb_database_increase_rate	net_conntrack_dialer_conn_established_total
os_disk_io_read_bytes	node_netstat_Ip6_OutOctets	node_memory_Committed_AS_bytes	gs_tables_structure_last_data_changed_delay
os_disk_io_read_delay	node_netstat_Ip_Forwarding	node_memory_Hugepagesize_bytes	pg_sql_statement_statistics_sort_spill_count
node_filefd_allocated	node_netstat_Tcp_CurrEstab	node_memory_MemAvailable_bytes	pg_sql_statement_statistics_hash_spill_count
node_filesystem_files	node_netstat_Udp6_InErrors	node_memory_NFS_Unstable_bytes	gs_sql_statement_statistics_sort_spill_count

版权所有 © 华为云计算技术有限公司

node_sockstat_TCP_mem	node_network_protocol_type	node_memory_SReclaimable_bytes	gs_sql_statement_statistics_hash_spill_count
node_sockstat_UDP_mem	node_sockstat_FRAG6_memory	node_memory_VmallocChunk_bytes	node_hwmon_power_average_interval_max_seconds
gs_audit_logout_total	node_sockstat_sockets_used	node_memory_VmallocTotal_bytes	node_hwmon_power_average_interval_min_seconds
gs_audit_revoke_total	node_softnet_dropped_total	node_memory_WritebackTmp_bytes	pg_stat_bgwriter_checkpoint_proactive_triggering_ratio
gs_cpu_load_total_cpu	node_textfile_scrape_error	node_netstat_Udp6_OutDatagrams	gs_stat_bgwriter_checkpoint_proactive_triggering_ratio
gs_db_confl_bufferpin	node_timex_pps_error_total	node_netstat_Udp6_RcvbufErrors	-

1.10.3.1 X-Tuner

1.10.3.1.1 概述

X-Tuner是一款数据库集成的参数调优工具，通过结合深度强化学习和全局搜索算法等AI技术，实现在无需人工干预的情况下，获取最佳数据库参数配置。本功能不强制与数据库环境部署到一起，支持独立部署，脱离数据库安装环境独立运行。

说明

本功能主要适配集中式部署的数据库，分布式数据库不建议使用。使用X-Tuner功能会尝试建立数据库连接，从而获取数据库规格。如果是分布式数据库，则会提示不支持。

1.10.3.1.2 使用准备

前提条件与使用事项

- 数据库状态正常、客户端能够正常连接、且要求数据库内导入数据，以便调优程序可以执行benchmark测试调优效果。
- 使用本工具需要指定登录到数据库的用户身份，要求该登录到数据库上的用户具有足够的权限，以便可以获得充足的数据库状态信息。
- 使用登录到数据库宿主机上的Linux用户，需要将**$GAUSSHOME/bin**添加到PATH环境变量中，即能够直接运行gsql、gs_guc、gs_ctl等数据库运维工具。
- 本工具支持以三种模式运行，其中tune和train模式要求用户配置好benchmark运行环境，并导入数据，本工具将会通过迭代运行benchmark来判断修改后的参数是否有性能提升。
- recommend模式建议在数据库正在执行workload的过程中执行，以便获得更准确的实时workload信息。

版权所有 © 华为云计算技术有限公司

- 本工具默认带有TPC-C、TPC-H、TPC-DS以及sysbench的benchmark运行脚本样例，如果用户使用上述benchmark对数据库系统进行压力测试，则可以对上述配置文件进行适度修改或配置。如果需要适配用户自己的业务场景，需要您参照benchmark目录中的template.py文件编写驱动您自定义benchmark的脚本文件。

原理简介

调优程序是一个独立于数据库内核之外的工具，需要提供数据库及其所在实例的用户名和登录密码信息，以便控制数据库执行benchmark进行性能测试；在启动调优程序前，要求用户测试环境交互正常，能够正常跑通benchmark测试脚本、能够正常连接数据库。

说明

如果需要调优的参数中，包含重启数据库后才能使修改生效的参数，那么在调优过程中数据库将会重启多次。如果用户的数据库正在执行作业，请慎用train与tune模式。

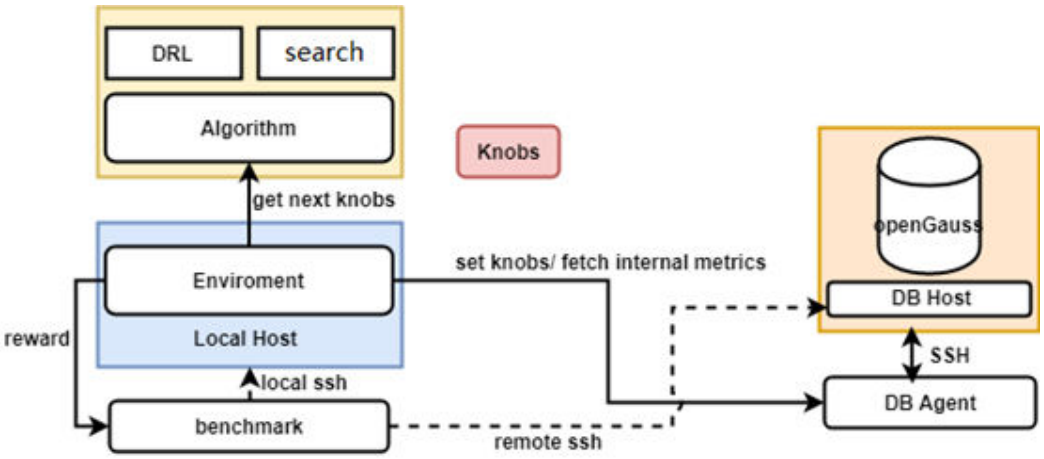
调优程序X-Tuner包含三种运行模式，分别是:

- recommend：通过用户指定的用户名等信息登录到数据库环境中，获取当前正在运行的workload特征信息，根据上述特征信息生成参数推荐报告。报告当前数据库中不合理的参数配置和潜在风险等；输出根据当前正在运行的workload行为和特征；输出推荐的参数配置。**该模式是秒级的，不涉及数据库的重启操作，其他模式可能需要反复重启数据库**。
- train：通过用户提供的benchmark信息，不断地进行参数修改和benchmark的执行。通过反复的迭代过程，训练强化学习模型，以便用户在后面通过tune模式加载该模型进行调优。
- tune：使用优化算法进行数据库参数的调优，当前支持两大类算法，一种是深度强化学习，另一种是全局搜索算法（全局优化算法）。深度强化学习模式要求先运行train模式，生成训练后的调优模型，而使用全局搜索算法则不需要提前进行训练，可以直接进行搜索调优。

须知

如果在tune模式下，使用深度强化学习算法，要求必须有一个训练好的模型，且要求**训练该模型时的参数与进行调优时的参数列表（包括max与min）必须一致**。

图 1-9 X-Tuner 结构图



版权所有 © 华为云计算技术有限公司

X-Tuner的整体架构如上方X-Tuner 结构图所示，系统可以分为：

- DB侧：通过DB_Agent模块对数据库实例进行抽象，通过该模块可以获取数据库内部的状态信息、当前数据库参数、以及设置数据库参数等。DB侧包括登录数据库环境使用的SSH连接。
- 算法侧：用于调优的算法包，包括全局搜索算法（如贝叶斯优化、粒子群算法等）和深度强化学习（如DDPG）。
- X-Tuner主体逻辑模块：通过Environment模块进行封装，每一个step就是一次调优过程。整个调优过程通过多个step进行迭代。
- benchmark：由用户指定的benchmark性能测试脚本，用于运行benchmark作业，通过跑分结果反映数据库系统性能优劣。

说明

应确保benchmark脚本跑分结果越大表示性能越好。

例如TPCH这种衡量SQL语句整体执行时长的benchmark，可以通过取总体执行时间的相反数作为benchmark的输出分数。

X-Tuner 的运行和安装方法

执行下述命令即可获取xtuner功能帮助

```
gs_dbmind component xtuner --help
```

用户可据此给定不同的命令行执行相应的功能。

X-Tuner 的配置文件说明

X-Tuner在运行前需要加载配置文件，用户可以通过 **--help** 命令查看默认加载的配置文件绝对路径：

```
-x TUNER_CONFIG_FILE, --tuner-config-file TUNER_CONFIG_FILE
                      This is the path of the core configuration file of the
                      X-Tuner. You can specify the path of the new
                      configuration file. The default path is DBMINDPATH/dbmind/components/xtuner/tuner/
xtuner.conf.
                      You can modify the configuration file to control the
                      tuning process.
```

修改配置文件的配置项可以指引X-Tuner执行不同的动作，用户可以根据自己的不同需求来修改配置文件的内容，配置文件的配置项说明详见表1-22。如果需要修改配置文件的加载路径，则可以通过选项**-x**命令行选项来指定。

Benchmark 的选择与配置

Benchmark的驱动脚本存放路径为X-Tuner目录的子目录benchmark中。X-Tuner自带常用的benchmark驱动脚本，例如基于时间周期的探测脚本（**默认**）、TPC-C、TPC-H等。X-Tuner通过调用benchmark/__init__.py文件中 **get_benchmark_instance()**命令来加载不同的benchmark驱动脚本，获取benchmark驱动实例。其中，benchmark驱动脚本的格式说明如下：

- 驱动脚本文件名：表示benchmark的名字，该名字用于表示驱动脚本的唯一性，可通过在X-Tuner的配置文件中的配置项**benchmark_script**来指定选择加载benchmark驱动脚本。
- 驱动脚本内容三要素：path变量、cmd变量以及run函数。

下面分别介绍驱动脚本的内容三要素：

版权所有 © 华为云计算技术有限公司

1. path变量：表示benchmark脚本的存放地址，可以直接在驱动脚本中修改，也可以通过配置文件的benchmark_path配置项来指定。
2. cmd变量：表示执行benchmark脚本需要运行的命令，可以直接在驱动脚本中修改，也可以通过配置文件的benchmark_cmd配置项来指定。cmd中的文本允许使用占位符，用于获取某些运行cmd命令时的必要信息，使用示例参见TPC-H驱动脚本示例。这些占位符包括：
   - {host}：数据库宿主机的IP地址
   - {port}：数据库实例的侦听端口号
   - {user}：登录数据库系统上的用户名
   - {password}：与登录数据库系统上的用户相匹配的密码
   - {db}：正在进行调优的数据库名
3. run函数：该函数的函数签名为：

   ```
   def run(remote_server, local_host) -> float:
   ```

   其中，返回数据类型为float，表示benchmark执行后的评估分数值，要求该值越大表示性能越好，例如使用TPC-C跑分结果tpmC即可作为返回值，TPC-H的全部SQL语句执行总时间的相反数（取相反数后可保证返回值越大则性能越好）也可作为返回值。

   remote_server变量是X-Tuner程序传递给脚本使用的远端主机（数据库宿主机）的shell命令接口，local_host变量是X-Tuner程序传递给脚本使用的本地主机（运行X-Tuner脚本的主机）的shell命令接口。上述shell命令接口提供的方法包括：

   ```
   exec_command_sync(command, timeout)
   功能：该方法用于在主机上执行shell命令。
   参数列表：
   command 必选，数据类型可以是str, 以及元素为str类型的list或tuple;
   timeout 可选，表示命令执行的超时时长，单位是秒。
   返回值：
   返回二元组 (stdout, stderr)，stdout表示标准输出流结果，stderr表示标准错误流结果，数据类型均为str.
   exit_status
   功能：该属性表示最近一条shell命令执行后的退出状态码(exit status code)。
   说明：一般情况，退出状态码为0表示执行正常，非0表示存在错误。
   ```

Benchmark驱动脚本示例说明

1. TPC-C 驱动脚本

   ```
   from tuner.exceptions import ExecutionError

   # WARN: You need to download the benchmark-sql test tool to the system,
   # replace the PostgreSQL JDBC driver with the openGauss driver,
   # and configure the benchmark-sql configuration file.
   # The program starts the test by running the following command:
   path = '/path/to/benchmarksql/run'  # TPC-C测试脚本benchmark-sql 的存放路径
   cmd = "./runBenchmark.sh props.gs"  # 自定义一个名为 props.gs 的benchmark-sql测试配置文件


   def run(remote_server, local_host):
       # 切换到 TPC-C 脚本目录下，清除历史错误日志，然后运行测试命令。
       # 此处建议等待几秒钟，因为benchmark-sql 测试脚本生成最终测试报告是通过一个shell脚本实现的，整个过程会有延迟，
       # 为了保证能够获取到最终的tpmC数值报告，这里选择等待3秒钟。
       stdout, stderr = remote_server.exec_command_sync(['cd %s' % path, 'rm -rf benchmarksql-error.log', cmd, 'sleep 3'])
       # 如果标准错误流中有数据，则报异常退出。
       if len(stderr) > 0:
           raise ExecutionError(stderr)

       # 寻找最终tpmC结果
       tpmC = None
       split_string = stdout.split()  # 对标准输出流结果进行分词。
       for i, st in enumerate(split_string):
           # 在5.0版本的benchmark-sql中，tpmC最终测试结果数值在 ‘(NewOrders)’关键字的后两位，正
   ```

 版权所有 © 华为云计算技术有限公司

```
常情况下，找到该字段后直接返回即可。
        if "(NewOrders)" in st:
            tpmC = split_string[i + 2]
            break
    stdout, stderr = remote_server.exec_command_sync(
        "cat %s/benchmarksql-error.log" % path)
    nb_err = stdout.count("ERROR:")  # 判断整个benchmark运行过程中，是否有报错，记录报错的错误数
    return float(tpmC) - 10 * nb_err  # 这里将报错的错误数作为一个惩罚项，惩罚系数为10，越高的惩罚
系数表示越看中报错的数量.
```

2. TPC-H驱动脚本

```
import time

from tuner.exceptions import ExecutionError

# WARN: You need to import data into the database and SQL statements in the following path will
be executed.
# The program automatically collects the total execution duration of these SQL statements.
path = '/path/to/tpch/queries'  # 存放TPC-H测试用的SQL脚本目录
cmd = "gsql -U {user} -W {password} -d {db} -p {port} -f {file}"  # 需要运行TPC-H测试脚本的命令，一
般使用'gsql -f 脚本文件' 来运行


def run(remote_server, local_host):
    # 遍历当前目录下所有的测试用例文件名
    find_file_cmd = "find . -type f -name '*.sql'"
    stdout, stderr = remote_server.exec_command_sync(['cd %s' % path, find_file_cmd])
    if len(stderr) > 0:
        raise ExecutionError(stderr)
    files = stdout.strip().split('\n')
    time_start = time.time()
    for file in files:
        # 使用 file 变量替换 {file}，然后执行该命令行。
        perform_cmd = cmd.format(file=file)
        stdout, stderr = remote_server.exec_command_sync(['cd %s' % path, perform_cmd])
        if len(stderr) > 0:
            print(stderr)
    # 代价为全部测试用例的执行总时长
    cost = time.time() - time_start
    # 取相反数，适配run 函数的定义：返回结果越大表示性能越好。
    return - cost
```

1.10.3.1.3 使用示例

X-Tuner支持三种模式，分别是获取参数诊断报告的recommend模式、训练强化学习模型的train模式、以及使用算法进行调优的tune模式。上述三种模式可以通过命令行参数来区别，通过配置文件来指定具体的细节。

配置数据库连接信息

三种模式连接数据库的配置项是相同的，有两种方式：一种是直接通过命令行输入详细的连接信息，另一种是通过JSON格式的配置文件输入，下面分别对两种指定数据库连接信息的方法进行说明。

1. 通过命令行指定：

   分别传递 --db-name --db-user --port --host --host-user 参数，可选 --host-ssh-port 参数，例如：

   ```
   gs_dbmind component xtuner recommend --db-name testdb --db-user omm --port 5678 --host
   192.168.1.100 --host-user omm
   ```

2. 通过JSON格式的连接信息配置文件指定：

   JSON配置文件的示例如下，并假设文件名为 connection.json：

   ```
   {
     "db_name": "testdb",  # 数据库名
   ```

 版权所有 © 华为云计算技术有限公司

```
  "db_user": "dba",       # 登录到数据库上的用户名
  "host": "127.0.0.1",    # 数据库宿主机的IP地址
  "host_user": "dba",     # 登录到数据库宿主机的用户名
  "port": 5432,           # 数据库的侦听端口号
  "ssh_port": 22          # 数据库宿主机的SSH侦听端口号
}
```

则可通过 -f connection.json 传递。

说明

为了防止密码泄露，配置文件和命令行参数中默认都不包含密码信息，用户在输入上述连接信息后，程序会采用交互式的方式要求用户输数据库密码以及操作系统登录用户的密码。

recommend 模式使用示例

对recommend模式生效的配置项为scenario，若为auto，则自动检测workload类型。

执行下述命令，获取诊断结果：

```
gs_dbmind component xtuner recommend -f connection.json
```

则可以生成诊断报告如下：

版权所有 © 华为云计算技术有限公司

图 1-10 recommend 模式生成的报告示意图

```
Start to recommend knobs. Just a moment, please.
************************* Knob Recommendation Report *************************
INFO:
+--------------------------------------+----------------------+
|                Metric                |        Value         |
+--------------------------------------+----------------------+
|             workload_type            |          tp          |
|         average_connection_age       |          0           |
|         dirty_background_bytes       |          0           |
|             temp_file_size           |          0           |
|          current_connections         |         0.0          |
|          current_locks_count         |         0.0          |
|      current_prepared_xacts_count    |         0.0          |
|          rollback_commit_ratio       | 0.09168372786632421  |
|                 uptime               |  0.122942879722222   |
| checkpoint_proactive_triggering_ratio |  0.488598416181662   |
|         fetched_returned_ratio       |  0.9915911452033203  |
|             cache_hit_rate           |  0.9979742937232552  |
|            read_write_ratio          |  123.86665549830312  |
|          all_database_size           |  134154882.48046875  |
|          search_modify_ratio         |  187.59523392981777  |
|                ap_index              |  2.3759498376861847  |
|            current_free_mem          |       31161892       |
|             os_mem_total             |       32779460       |
|        checkpoint_avg_sync_time      |   381.359603091308   |
|  checkpoint_dirty_writing_time_window |        450.0         |
|             max_processes            |          46          |
|          track_activity_size         |         46.0         |
|            write_tup_speed           |   6810.36309197048   |
|                used_mem              |     73988850.25      |
|              os_cpu_count            |          8           |
|               block_size             |         8.0          |
|             read_tup_speed           |   845237.440716804   |
|      shared_buffer_toast_hit_rate    |  98.16007359705611   |
|      shared_buffer_tidx_hit_rate     |  99.11667280088332   |
|       shared_buffer_idx_hit_rate     |  99.74473859023848   |
|      shared_buffer_heap_hit_rate     |  99.81099543813004   |
|           enable_autovacuum          |         True         |
|                is_64bit              |         True         |
|                 is_hdd               |         True         |
|              load_average            | [1.89, 3.175, 3.005] |
+--------------------------------------+----------------------+
p.s: The unit of storage is kB.
WARN:
[0]. The number of CPU cores is a little small. Please do not run too high concu
rrency. You are recommended to set max_connections based on the number of CPU co
res. If your job does not consume much CPU, you can also increase it.
[1]. The value of wal_buffers is a bit high. Generally, an excessively large val
ue does not bring better performance. You can also set this parameter to -1. The
 database automatically performs adaptation.
BAD:
[0]. The database runs for a short period of time, and the database description
may not be accumulated. The recommendation result may be inaccurate.
************************* Recommended Knob Settings **************************
+---------------------------+-----------+---------+----------+---------+
|            name           | recommend |   min   |   max    | restart |
+---------------------------+-----------+---------+----------+---------+
|       shared_buffers      |  1638973  |  614614 | 1884818  |   True  |
|      max_connections      |     43    |    24   |   500    |   True  |
|    effective_cache_size   |  1638973  | 1638973 | 24584595 |  False  |
|        wal_buffers        |   51217   |   2048  |   51217  |   True  |
|      random_page_cost     |    3.0    |    2.0  |    3.0   |  False  |
| default_statistics_target |    100    |    10   |    150   |  False  |
+---------------------------+-----------+---------+----------+---------+
```

在上述报告中，推荐了该环境上的数据库参数配置，并进行了风险提示。报告同时生成了当前workload的特征信息，其中有几个特征是比较有参考意义的：

- temp_file_size：产生的临时文件数量，如果该结果大于0，则表明系统使用了临时文件。使用过多的临时文件会导致性能不佳，如果可能的话，需要提高work_mem参数的配置。
- cache_hit_rate：shared_buffer的缓存命中率，表明当前workload使用缓存的效率。

版权所有 © 华为云计算技术有限公司

- read_write_ratio：数据库作业的读写比例。
- search_modify_ratio：数据库作业的查询与修改数据的比例。
- ap_index：表明当前workload的AP指数，取值范围是0到10，该数值越大，表明越偏向于数据分析与检索。
- workload_type：根据数据库统计信息，推测当前负载类型，分为AP、TP以及HTAP三种类型。
- checkpoint_avg_sync_time：数据库在checkpoint时，平均每次同步刷新数据到磁盘的时长，单位是毫秒。
- load_average：平均**每个CPU核心**在1分钟、5分钟以及15分钟内的负载。一般地，该数值在1左右表明当前硬件比较匹配workload、在3左右表明运行当前作业压力比较大，大于5则表示当前硬件环境运行该workload压力过大（此时一般建议减少负载或升级硬件）。

说明

- recommend模式会读取数据库中的pg_stat_database以及pg_stat_bgwriter等系统表中的信息，需要登录到数据库上的用户具有足够的权限（建议为**管理员权限**，可通过alter user username sysadmin；授予username相应的权限）。
- 由于某些系统表会一直记录统计信息，这可能会对负载特征识别造成干扰，因此建议先清空某些系统表的统计信息，运行一段时间的workload后再使用recommend模式进行诊断，以便获得更准确的结果。清除统计信息的方法为：

```
select pg_stat_reset_shared('bgwriter');
select pg_stat_reset();
```

train 模式使用示例

该模式是用来训练深度强化学习模型的，与该模式有关的配置项为：

- rl_algorithm：用于训练强化学习模型的算法，当前支持设置为ddpg.
- rl_model_path：训练后生成的强化学习模型保存路径。
- rl_steps：训练过程的最大迭代步数。
- max_episode_steps：每个回合的最大步数。
- scenario：明确指定的workload类型，如果为auto则为自动判断。在不同模式下，推荐的调优参数列表也不一样。
- tuning_list：**用户指定需要调哪些参数，如果不指定，则根据workload类型自动推荐应该调的参数列表。**如需指定，则tuning_list表示调优列表文件的路径。一个调优列表配置文件的文件内容示例如下：

```
{
  "work_mem": {
    "default": 65536,
    "min": 65536,
    "max": 655360,
    "type": "int",
    "restart": false
  },
  "shared_buffers": {
    "default": 32000,
    "min": 16000,
    "max": 64000,
    "type": "int",
    "restart": true
  },
  "random_page_cost": {
    "default": 4.0,
    "min": 1.0,
    "max": 4.0,
```


版权所有 © 华为云计算技术有限公司

```
  "type": "float",
  "restart": false
 },
 "enable_nestloop": {
  "default": true,
  "type": "bool",
  "restart": false
 }
}
```

待上述配置项配置完成后，可以通过下述命令启动训练：

```
gs_dbmind component xtuner train -f connection.json
```

训练完成后，会在配置项rl_model_path指定的目录中生成模型文件。

tune 模式使用示例

tune模式支持多种算法，包括基于强化学习（Reinforcement Learning, RL）的DDPG算法、基于全局搜索（Global OPtimization algorithm, GOP）算法的贝叶斯优化算法（Bayesian Optimization）以及粒子群算法（Particle Swarm Optimization, PSO）。

与tune模式相关的配置项为：

- tune_strategy：指定选择哪种算法进行调优，支持rl（使用强化学习模型进行调优）、gop（使用全局搜索算法）以及auto（自动选择）。若该参数设置为rl，则rl相关的配置项生效。除前文提到过的train模式下生效的配置项外，test_episode配置项也生效，该配置项表明调优过程的最大回合数，该参数直接影响了调优过程的执行时间（一般地，数值越大越耗时）。
- gop_algorithm：选择何种全局搜索算法，支持bayes以及pso。
- max_iterations：最大迭代轮次，数值越高搜索时间越长，效果往往越好。
- particle_nums：在PSO算法上生效，表示粒子数。
- scenario与tuning_list见上文train模式中的描述。

待上述配置项配置完成后，可以通过下述命令启动调优：

```
gs_dbmind component xtuner tune -f connection.json
```

⚠ 注意

在使用tune和train模式前，用户需要先导入benchmark所需数据并检查benchmark能否正常跑通。调优过程结束后，调优程序会自动恢复调优前的数据库参数配置。

1.10.3.1.4 获取帮助

启动调优程序之前，可以通过如下命令获取帮助信息：

```
gs_dbmind component xtuner --help
```

输出帮助信息结果如下：

```
usage:  [-h] [--database DATABASE] [--db-user DB_USER] [--db-port DB_PORT] [--db-host DB_HOST] [--host-user HOST_USER] [--host-ssh-port HOST_SSH_PORT] [-f DB_CONFIG_FILE] [-x TUNER_CONFIG_FILE] [-v]
       {train,tune,recommend}

X-Tuner: a self-tuning tool integrated by openGauss.

positional arguments:
```


版权所有 © 华为云计算技术有限公司

```
 {train,tune,recommend}
                    Train a reinforcement learning model or tune database by model. And also can recommend
best_knobs according to your workload.

optional arguments:
  -h, --help            show this help message and exit
  -f DB_CONFIG_FILE, --db-config-file DB_CONFIG_FILE
                    You can pass a path of configuration file otherwise you should enter database information
by command arguments manually. Please see the template file share/server.json.template.
  -x TUNER_CONFIG_FILE, --tuner-config-file TUNER_CONFIG_FILE
                    This is the path of the core configuration file of the X-Tuner. You can specify the path of
the new configuration file. The default path is /path/to/config/file. You can modify the configuration file to
control the tuning process.
  -v, --version         show program's version number and exit

Database Connection Information:
  --database DATABASE, --db-name DATABASE
                    The name of database where your workload running on.
  --db-user DB_USER     Use this user to login your database. Note that the user must have sufficient
permissions.
  --db-port DB_PORT, --port DB_PORT
                    Use this port to connect with the database.
  --db-host DB_HOST, --host DB_HOST
                    The IP address of your database installation host.
  --host-user HOST_USER
                    The login user of your database installation host.
  --host-ssh-port HOST_SSH_PORT
                    The SSH port of your database installation host.
```

1.10.3.1.5 命令参考

表 1-21 命令行参数

参数	参数说明	取值范围
mode	指定调优程序运行的模式	train，tune，recommend
--tuner-config-file, -x	X-Tuner的核心参数配置文件路径，默认路径为安装目录下的xtuner.conf	-
--db-config-file, -f	调优程序的用于登录到数据库宿主机上的连接信息配置文件路径，若通过该文件配置数据库连接信息，则下述数据库连接信息可省略	-
--db-name	指定需要调优的数据库名	-
--db-user	指定以何用户身份登录到调优的数据库上	-
--port, --db-port	数据库的侦听端口	1024-65535
--host, --db-host	数据库实例的宿主机IP	-
--host-user	指定以何用户身份登录到数据库实例的宿主机上，要求改用户名的环境变量中可以找到gsql、gs_ctl等数据库运维工具。	-
--host-ssh-port	数据库实例所在宿主机的SSH端口号，可选，默认为22	0-65535

版权所有 © 华为云计算技术有限公司

参数	参数说明	取值范围
--help, -h	返回帮助信息	-
--version, -v	返回当前工具版本号	-

表 1-22 配置文件中的参数详解

参数名	参数说明	取值范围
logfile	生成的日志存放路径	-
output_tuning_result	可选，调优结果的保存路径	-
verbose	是否打印详情	on, off
recorder_file	调优中间信息的记录日志存放路径	-
tune_strategy	调优模式下采取哪种策略	rl, gop, auto
drop_cache	是否在每一个迭代轮次中进行drop cache，drop cache可以使benchmark跑分结果更加稳定。若启动该参数，则需要将登录的系统用户加入到 /etc/sudoers 列表中，同时为其增加 **NOPASSWD** 权限（由于该权限可能过高，建议临时启用该权限，调优结束后关闭）。	on, off
used_mem_penalty_term	数据库使用总内存的惩罚系数，用于防止通过无限量占用内存而换取的性能表现。该数值越大，惩罚力度越大。	建议0 ~ 1
rl_algorithm	选择何种RL算法	ddpg
rl_model_path	RL模型保存或读取路径，包括保存目录名与文件名前缀。在train 模式下该路径用于保存模型，在tune模式下则用于读取模型文件	-
rl_steps	深度强化学习算法迭代的步数	-
max_episode_steps	每个回合的最大迭代步数	-
test_episode	使用RL算法进行调优模式的回合数	-
gop_algorithm	采取何种全局搜索算法	bayes, pso
max_iterations	全局搜索算法的最大迭代轮次（并非确定数值，可能会根据实际情况多跑若干轮）	-
particle_nums	PSO算法下的粒子数量	-

版权所有 © 华为云计算技术有限公司

参数名	参数说明	取值范围
benchmark_script	使用何种benchmark驱动脚本，该选项指定加载benchmark路径下同名文件，默认支持TPC-C、TPC-H等典型benchmark	tpcc, tpch, tpcds, sysbench ...
benchmark_path	benchmark 脚本的存储路径，若没有配置该选项，则使用benchmark驱动脚本中的配置	-
benchmark_cmd	启动benchmark 脚本的命令，若没有配置该选项，则使用benchmark驱动脚本中的配置	-
benchmark_period	仅对 period benchmark有效，表明整个benchmark的测试周期是多少，单位是秒	-
scenario	用户指定的当前workload所属的类型	tp, ap, htap
tuning_list	准备调优的参数列表文件，可参考 share/knobs.json.template 文件	-

1.10.3.1.6 常见问题处理

- 数据库实例连接失败：请检查数据库实例的情况，是否数据库实例出现了问题或安全权限配置（gs_hba.conf文件中的配置项）不正确。
- 重启失败：请检查数据库实例健康情况，确保数据库实例工作正常。
- 跑TPC-C作业时发现性能越来越慢：TPC-C等高并发场景下的压力测试，往往伴随着大量的数据修改。由于每一次测试并非是幂等的（TPC-C数据库数据量的增加、没有进行vacuum 清理掉失效元组、数据库没有触发checkpoint、没有进行drop cache等），因此一般建议TPC-C等伴随着较多数据写入的benchmark应该每隔一段时间（视具体并发量、执行时长的不同而异）重新导入一次数据。
- TPC-C 跑作业时，TPC-C驱动脚本报异常 “TypeError: float() argument must be a string or a number, not 'NoneType'”（不能将None转换为float类型）：这是因为没有获取到TPC-C的压测返回结果，造成该问题的原因比较多，请首先手动检测是否能够跑通TPC-C并能够获取返回结果。若无上述问题，则建议将 TPC-C 驱动脚本中的命令列表中的 "sleep" 命令延迟时间设得更大一些。

1.10.3.2 Index-advisor

1.10.3.2.1 单 query 索引推荐

单query索引推荐功能支持用户在数据库中直接进行操作，本功能基于查询语句的语义信息和数据库的统计信息，对用户输入的单条查询语句生成推荐的索引。本功能涉及的函数接口如下。

版权所有 © 华为云计算技术有限公司

表 1-23 单 query 索引推荐功能的接口

函数名	参数	功能
gs_index_advise	SQL语句字符串	针对单条查询语句生成推荐索引。

说明

- 本功能仅支持单条SELECT、INSERT、DELETE、UPDATE类型的语句，不支持其他类型的SQL语句。
- 本功能使用优化器采样结果，用户需要保证最近analyze过，否则优化器结果不准确。
- 本功能暂不支持段页式表、普通视图、物化视图、全局临时表、二级分区表以及密态数据库。
- 如果对ustore表相关语句进行索引推荐，本功能可能无法保证结果的准确性。

使用方法

使用上述函数，获取针对该query生成的推荐索引，推荐结果由索引的表名和列名组成。

例如：

```
gaussdb=# select "table", "column" from gs_index_advise('SELECT c_discount from bmsql_customer where c_w_id = 10');
     table      |  column
----------------+----------
 bmsql_customer | c_w_id
(1 row)
```

上述结果表明：应当在 bmsql_customer 的 c_w_id 列上创建索引，例如可以通过下述SQL语句创建索引：

```
CREATE INDEX idx on bmsql_customer(c_w_id);
```

某些SQL语句，也可能被推荐创建联合索引，例如：

```
gaussdb=# select "table", "column" from gs_index_advise('select name, age, sex from t1 where age >= 18 and age < 35 and sex = ''f'';');
 table | column
-------+------------
 t1    | age, sex
(1 row)
```

则上述语句表明应该在表 t1 上创建一个联合索引 (age, sex)， 则可以通过下述命令创建：

```
CREATE INDEX idx1 on t1(age, sex);
```

针对分区表可推荐具体索引类型，例如：

```
gaussdb=# select "table", "column", "indextype" from gs_index_advise('select name, age, sex from range_table where age = 20;');
 table | column | indextype
-------+--------+-----------
 t1    | age    | global
(1 row)
```


版权所有 © 华为云计算技术有限公司

说明

系统函数gs_index_advise()的参数是文本型，如果参数中存在如单引号（'）等特殊字符，可以使用单引号（'）进行转义，可参考上述示例。

1.10.3.2.2 虚拟索引

虚拟索引功能支持用户在数据库中直接进行操作，本功能将模拟真实索引的建立，避免真实索引创建所需的时间和空间开销，用户基于虚拟索引，可通过优化器评估该索引对指定查询语句的代价影响。

本功能涉及的系统函数接口如下表所示：

表 1-24 虚拟索引功能的接口

函数名	参数	功能
hypopg_create_index	参数1：创建索引语句的字符串 参数2：虚拟索引级别，可选参数，可指定global或session，默认为global	创建虚拟索引。
hypopg_display_index	虚拟索引级别，可选参数，可通过参数指定global或session，默认为global	显示所有创建的虚拟索引信息。
hypopg_drop_index	索引的oid	删除指定的虚拟索引。
hypopg_reset_index	虚拟索引级别，可选参数，可通过参数指定global或session，默认为global	清除所有虚拟索引。
hypopg_estimate_size	索引的oid	估计指定索引创建所需的空间大小。

本功能涉及的GUC参数如下：

表 1-25 虚拟索引功能的 GUC 参数

参数名	功能	默认值
enable_hypo_index	是否开启虚拟索引功能	off

使用步骤

案例一：使用虚拟索引，调优等值查询

版权所有 © 华为云计算技术有限公司

在此案例中，存在表bmsql_customer，该表是TPC-C benchmark中的一张表，此处演示在该表的c_w_id列上创建一个索引，是否可以提升某个等值查询的性能，如果该索引被使用了，则预估执行代价(cost)是多少。

步骤1 使用函数hypopg_create_index创建虚拟索引。例如：

```
gaussdb=# select * from hypopg_create_index('create index on bmsql_customer(c_w_id)');
 indexrelid |              indexname
------------+---------------------------------------
     329726 | <329726>btree_bmsql_customer_c_w_id
(1 row)
```

步骤2 开启GUC参数enable_hypo_index，该参数控制数据库的优化器进行EXPLAIN时是否考虑创建的虚拟索引。通过对特定的查询语句执行explain，用户可根据优化器给出的执行计划评估该索引是否能够提升该查询语句的执行效率。例如：

```
gaussdb=# set enable_hypo_index = on;
SET
```

开启GUC参数前，执行EXPLAIN + 查询语句：

```
gaussdb=# explain SELECT c_discount from bmsql_customer where c_w_id = 10;
                           QUERY PLAN
-----------------------------------------------------------------------
 Seq Scan on bmsql_customer  (cost=0.00..52963.06 rows=31224 width=4)
   Filter: (c_w_id = 10)
(2 rows)
```

开启GUC参数后，执行EXPLAIN + 查询语句：

```
gaussdb=# explain SELECT c_discount from bmsql_customer where c_w_id = 10;
                                          QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
 [Bypass]
 Index Scan using <329726>btree_bmsql_customer_c_w_id on bmsql_customer  (cost=0.00..39678.69
rows=31224 width=4)
   Index Cond: (c_w_id = 10)
(3 rows)
```

通过对比两个执行计划可以观察到，该索引预计会降低指定查询语句的执行代价，用户可考虑创建对应的真实索引。

步骤3 （可选）使用函数hypopg_display_index展示所有创建过的虚拟索引。例如：

```
gaussdb=# select * from hypopg_display_index();
                indexname                 | indexrelid |     table      |      column      |                    indexdef
------------------------------------------+------------+----------------+------------------+-----------------------------------------------------------
 <329726>btree_bmsql_customer_c_w_id      |     329726 | bmsql_customer | (c_w_id)         |CREATE INDEX ON bmsql_customer USING btree (c_w_id)
 <329729>btree_bmsql_customer_c_d_id_c_w_id |     329729 | bmsql_customer | (c_d_id, c_w_id) |CREATE INDEX ON bmsql_customer USING btree (c_d_id, c_w_id)
(2 rows)
```

步骤4 （可选）使用函数hypopg_estimate_size估计虚拟索引创建所需的空间大小（单位：字节）。例如：

```
gaussdb=# select * from hypopg_estimate_size(329729);
 hypopg_estimate_size
----------------------
             15687680
(1 row)
```

步骤5 删除虚拟索引。

使用函数hypopg_drop_index删除指定oid的虚拟索引。例如：

```
gaussdb=# select * from hypopg_drop_index(329726);
 hypopg_drop_index
-------------------
```

 版权所有 © 华为云计算技术有限公司

```
 t
(1 row)
```

使用函数hypopg_reset_index一次性清除所有创建的虚拟索引。例如：

```
gaussdb=# select * from hypopg_reset_index();
 hypopg_reset_index
--------------------


(1 row)
```

----结束

案例二：虚拟索引联合Hint，预测调优效果

Hint可以手动要求数据库优化器使用某种方式生成执行计划，因此，对于某些数据库优化器难以生成最优执行计划的场景，可以手动指定执行计划。例如对某张表中的数据进行扫描操作（Scan），可以采用tablescan, indexscan, indexonlyscan, 其分别对应了表扫描、索引扫描、覆盖索引扫描。对于后两种扫描形式，必须要求先在数据库表上存在索引才可以操作。而虚拟索引则可以实现在不创建索引的情况下，测试某个索引扫描的效果。

步骤1 创建一张表t1，并生成一定量数据，供后续测试。

```
create table t1 (id int, name text);
insert into t1 select generate_series(0, 100000), 'test';
analyze t1;
```

步骤2 测试当前优化器默认的范围检索执行计划，并获取其总代价；由于没有创建索引，该SQL语句使用的是全表扫描（SeqScan）。

```
explain select * from t1 where id > 1;
```

步骤3 在t1表的id列上新建虚拟索引。

```
-- 开启参数，以便后续执行explain时能够采用虚拟索引
set enable_hypo_index = on;
-- 创建session级别虚拟索引，该session退出后，这个虚拟索引信息也会被自动清理掉
select hypopg_create_index('create index on t1(id)','session');
```

步骤4 通过explain语句，查看该SQL语句是否能够采用该索引；由于该列的distinct值很大，且涉及回表，优化器默认不会采用该索引，该语句执行计划与步骤2无变化，仍是全表扫描（SeqScan）。

```
explain select * from t1 where id > 1;
```

步骤5 通过hint操作，手动要求走索引扫描，查看能否成功；由于指定了hint, 且存在该索引（尽管是虚拟的），仍然可以通过explain看到优化器使用了索引扫描 IndexScan。

```
-- 其中<57762>btree_t1_id是自动生成的虚拟索引名，实际操作中以创建虚拟索引时的返回值为准
explain select /*+ indexscan(t1 "<57762>btree_t1_id") */ * from t1 where id > 1;
```

----结束

 版权所有 © 华为云计算技术有限公司

说明

- 执行EXPLAIN ANALYZE不会涉及虚拟索引功能。
- 开启虚拟索引功能并执行EXPLAIN语句时，可以生成创建虚拟索引之后的执行计划；同时，indexscan/indexonlyscan hint支持虚拟索引。
- 会话级别虚拟索引在各个会话间的设置互不影响，关闭会话后将被清空。
- 与真实索引不同，虚拟索引的相关操作不可回滚。
- 虚拟索引相关函数，不支持dblink远程调用。
- 本功能暂不支持视图、物化视图。
- 虚拟索引支持分布式全局二级索引（GSI），但存在如下约束：
  - 虚拟索引GSI不支持段页式、bucket表、全局临时表、物化视图。
  - 虚拟索引GSI不支持创建部分索引、表达式索引。
  - 虚拟索引GSI不支持hint使用。

1.10.3.2.3 workload 级别索引推荐

对于workload级别的索引推荐，用户可通过运行数据库外的脚本使用此功能，本功能将包含有多条DML语句的workload作为输入，最终生成一批可对整体workload的执行表现进行优化的索引。同时，本功能提供从日志中抽取业务数据SQL流水的功能。

前提条件

- 数据库状态正常、客户端能够正常连接。
- 当前执行用户下安装有gsql工具，该工具路径已被加入到PATH环境变量中。
- 具备Python3.7的环境。
- 若使用本功能提供的业务数据抽取功能，需提前将要收集的节点的GUC参数按如下设置：
  - log_min_duration_statement = 0
  - log_statement= 'all'

  说明

  业务数据抽取完毕建议将上述GUC参数复原，否则容易导致日志文件膨胀。

业务数据抽取脚本使用步骤

步骤1 按前提条件中要求设置相关GUC参数。

步骤2 运行命令如下：

```
gs_dbmind component extract_log [-h] [-d DATABASE] [-U DB_USER] [--start-time START_TIME] [--sql-amount SQL_AMOUNT] [--statement] [--max-reserved-period MAX_RESERVED_PERIOD] [--max-template-num MAX_TEMPLATE_NUM] [--json] log_dir file line_prefix
```

其中的输入参数依次为：

- DATABASE：（可选）数据库名称，不指定默认所有数据库。
- DB_USER：（可选）用户名称，不指定默认所有用户。
- START_TIME：（可选）日志收集的开始时间，不指定默认所有文件。
- SQL_AMOUNT：（可选）收集SQL数量的最大值，不指定默认收集所有sql。
- statement：（可选）表示收集gs_log日志中statement标识开头的SQL，不指定默认不收集。

 版权所有 © 华为云计算技术有限公司

- MAX_RESERVED_PERIOD：（可选）指定json模式下，增量收集日志中保留的模板的最大的更新时长，不指定默认都保留，单位：天。
- MAX_TEMPLATE_NUM：（可选）指定json模式下保留的最大模板数量，不指定默认都保留。
- json：（可选）指定收集日志的文件存储格式为SQL归一化后的json，不指定默认格式每条SQL占一行。
- log_dir：gs_log的存放目录。
- file：输出SQL流水文件的保存路径，即抽取出的业务数据存放的文件路径。
- line_prefix：指定每条日志信息的前缀格式，可通过show log_line_prefix查询。

使用示例：

```
gs_dbmind component extract_log $GAUSSLOG/gs_log/dn_6001 sql_log.txt '%m %c %d %p %a %x %n %e' -d testdb -U omm --start_time '2021-07-06 00:00:00' --statement
```

说明

若指定-d/-U参数，日志打印每条日志信息的前缀格式需包含%d、%u，若需要抽取事务，必须指定%p，详见log_line_prefix参数。max_template_num参数设置建议不超5000条，避免workload索引推荐执行时间过长。

步骤3 将步骤1中设置的GUC参数还原为设置前的值。

----结束

索引推荐脚本使用步骤

步骤1 准备好包含有多条DML语句的文件作为输入的workload，文件中每条语句占据一行。用户可从数据库的离线日志中获得历史的业务语句。

步骤2 运行命令如下：

```
gs_dbmind component index_advisor [-h] [--db-host DB_HOST] [-U DB_USER, --db-user DB_USER] [--schema SCHEMA] [--min-improved-rate MIN_IMPROVED_RATE] [--max-index-columns MAX_INDEX_COLUMNS] [--min-reltuples MIN_RELTUPLES] [--max-n-distinct MAX_N_DISTINCT] [--max-index-num MAX_INDEX_NUM] [--max-index-storage MAX_INDEX_STORAGE] [--multi-iter-mode] [--multi-node] [--json] [--driver] [--show-detail] [--show-benefits] [--advise_gsi] [multi_thread_num MULTI_THREAD_NUM] db_port database file
```

密码通过管道输入或交互式输入，对于免密用户，任意输入都可通过检验。

表 1-26 命令行参数

参数	参数说明	取值范围
-h	返回帮助信息。	-
--db-host DB_HOST	（可选）连接数据库的主机号。	-
-U DB_USER	（可选）连接数据库的用户名。	-
--schema SCHEMA	模式名称。	-
--max-index-num MAX_INDEX_NUM	（可选）最大的索引推荐数目。	>=1
--max-index-storage MAX_INDEX_STORAGE	（可选）最大的索引集合空间大小。	>=1

版权所有 © 华为云计算技术有限公司

参数	参数说明	取值范围
--multi-iter-mode	（可选）算法模式，可通过是否设置该参数来切换算法。	-
--max-n-distinct MAX_N_DISTINCT	（1/distinct数）的最大值，默认为0.01。	0-1
--min-improved-rate MIN_IMPROVED_RATE	索引的最大提升幅度，默认为0.1，即提升10%。	0-1
--max-index-columns MAX_INDEX_COLUMNS	联合索引的最大列数（默认为4）。	>=1
--min-reltuples MIN_RELTUPLES	表的最小行数，默认为10000。	>0
--multi-node	（可选）指定是否为分布式数据库。	-
--json	（可选）指定workload语句的文件路径格式为SQL归一化后的json。	-
--driver	（可选）指定是否使用python驱动器连接数据库，默认gsql连接。	-
--show-detail	（可选）是否显示当前推荐索引集合的详细优化信息。	-
--show-benefits	（可选） 是否显示收益信息。	-
db_port	连接数据库的端口号。	-
database	连接数据库的名字。	-
file	包含workload语句的文件路径。	-
advise_gsi	（可选）推荐分布式全局二级索引（GSI）。	-
multi_thread_num	（可选）以多线程运行脚本，指定线程数。	[1,64]

例如：

```
gs_dbmind component index_advisor 6001 testdb tpcds_log.txt --schema public --max_index_num 10
```

结果在屏幕输出，包含候选索引、推荐索引、已创建索引、无用索引（该给定的workload里面没有用到系统中的索引列表）、冗余索引（当前系统中重复创建的索引）以及历史有效索引，如下：

```
################################ Generate candidate indexes
################################
table:  public.catalog_returns columns:  cr_return_amount
table:  public.catalog_sales columns:  cs_item_sk
table:  public.catalog_sales columns:  cs_sold_date_sk
table:  public.customer_address columns:  ca_city type:  global
table:  public.customer_address columns:  ca_state, ca_county type:  global
table:  public.customer_demographics columns:  cd_demo_sk type:  local
table:  public.date_dim columns:  d_month_seq type:  global
table:  public.date_dim columns:  d_year type:  global
table:  public.date_dim columns:  d_date_sk type:  local
```


版权所有 © 华为云计算技术有限公司

```
table:  public.date_dim columns:  d_month_seq type:  local
table:  public.date_dim columns:  d_year type:  local
table:  public.item columns:  i_class type:  global
table:  public.item columns:  i_manager_id, i_brand_id type:  global
table:  public.item columns:  i_manager_id, i_category_id type:  global
table:  public.item columns:  i_manufact_id type:  global
table:  public.item columns:  i_product_name type:  global
table:  public.store_returns columns:  sr_cdemo_sk
table:  public.store_returns columns:  sr_reason_sk
table:  public.store_returns columns:  sr_return_amt
table:  public.store_sales columns:  ss_item_sk, ss_sold_date_sk
table:  public.store_sales columns:  ss_store_sk
table:  public.time_dim columns:  t_time_sk type:  local
table:  public.web_returns columns:  wr_return_amt
table:  public.web_sales columns:  ws_item_sk
table:  public.web_sales columns:  ws_web_page_sk, ws_ship_hdemo_sk, ws_sold_time_sk
################################## Determine optimal indexes
##################################
CREATE INDEX idx_catalog_sales_cs_item_sk ON public.catalog_sales(cs_item_sk);
CREATE INDEX idx_catalog_sales_cs_sold_date_sk ON public.catalog_sales(cs_sold_date_sk);
CREATE INDEX idx_customer_demographics_local_cd_demo_sk ON
public.customer_demographics(cd_demo_sk) local;
CREATE INDEX idx_item_global_i_manufact_id ON public.item(i_manufact_id) global;
CREATE INDEX idx_store_returns_sr_cdemo_sk ON public.store_returns(sr_cdemo_sk);
CREATE INDEX idx_store_sales_ss_item_sk_ss_sold_date_sk ON public.store_sales(ss_item_sk,
ss_sold_date_sk);
CREATE INDEX idx_store_sales_ss_store_sk ON public.store_sales(ss_store_sk);
CREATE INDEX idx_web_sales_ws_web_page_sk_ws_ship_hdemo_sk_ws_sold_time_sk ON
public.web_sales(ws_web_page_sk, ws_ship_hdemo_sk, ws_sold_time_sk);
################################## Created indexes
##################################
public: CREATE UNIQUE INDEX ship_mode_pkey ON ship_mode USING btree (sm_ship_mode_sk)
LOCAL(PARTITION p_list_15_sm_ship_mode_sk_idx, PARTITION p_list_14_sm_ship_mode_sk_idx, PARTITION
p_list_13_sm_ship_mode_sk_idx, PARTITION p_list_12_sm_ship_mode_sk_idx, PARTITION
p_list_11_sm_ship_mode_sk_idx, PARTITION p_list_10_sm_ship_mode_sk_idx, PARTITION
p_list_9_sm_ship_mode_sk_idx, PARTITION p_list_8_sm_ship_mode_sk_idx, PARTITION
p_list_7_sm_ship_mode_sk_idx, PARTITION p_list_6_sm_ship_mode_sk_idx, PARTITION
p_list_5_sm_ship_mode_sk_idx, PARTITION p_list_4_sm_ship_mode_sk_idx, PARTITION
p_list_3_sm_ship_mode_sk_idx, PARTITION p_list_2_sm_ship_mode_sk_idx, PARTITION
p_list_1_sm_ship_mode_sk_idx)  TABLESPACE pg_default;
public: CREATE INDEX temptable_int2_int3_int4_idx ON temptable USING btree (int2, int3, int4)
TABLESPACE pg_default;
public: CREATE INDEX temptable_int2_int3_idx ON temptable USING btree (int2, int3) TABLESPACE
pg_default;
public: CREATE INDEX temptable_int1_int2_int3_idx ON temptable USING btree (int1, int2, int3)
TABLESPACE pg_default;
public: CREATE INDEX temptable_int1_int2_idx ON temptable USING btree (int1, int2) TABLESPACE
pg_default;
public: CREATE INDEX temptable_int1_idx ON temptable USING btree (int1) TABLESPACE pg_default;
################################## Current workload useless indexes
##################################
DROP INDEX temptable_int2_int3_int4_idx;
DROP INDEX temptable_int2_int3_idx;
DROP INDEX temptable_int1_int2_int3_idx;
DROP INDEX temptable_int1_int2_idx;
DROP INDEX temptable_int1_idx;
################################## Redundant indexes
##################################
DROP INDEX public.test1_age_idx;(CREATE INDEX test1_age_idx ON test1 USING btree (age) TABLESPACE
pg_default)
Related indexes:
        CREATE INDEX test1_age_id_idx ON test1 USING btree (age, id) TABLESPACE pg_default

DROP INDEX public.test1_id_idx;(CREATE INDEX test1_id_idx ON test1 USING btree (id) TABLESPACE
pg_default)
Related indexes:
        CREATE INDEX test1_id_age_idx ON test1 USING btree (id, age) TABLESPACE pg_default
################################## Historical effective indexes
##################################
```

 版权所有 © 华为云计算技术有限公司

```
CREATE INDEX idx_temptable_int2 ON ztt_test.temptable(int2);
CREATE INDEX idx_item_i_manufact_id ON public.item(i_manufact_id);
CREATE INDEX idx_item_i_color ON public.item(i_color);
```

----结束

⚠ 注意

如前文所述，"Current workload useless indexes", "Redundant indexes" 分别表示无用索引和冗余索引，判断依据是给定的workload. 由于workload中的SQL语句可能涉及不全（例如日志报错、没有捕获到等原因导致的），故该结论仅供提示，用户需要根据自己的业务逻辑进行排查，防止错误删除。

说明

与单query索引推荐相同，本功能暂不支持段页式表、普通视图、物化视图、全局临时表、二级分区表以及密态数据库。

1.10.3.3 Slow Query Diagnosis

1.10.3.3.1 概述

慢SQL一直是数据运维中的痛点问题，如何有效诊断慢SQL根因是当前一大难题，工具结合GaussDB自身特点融合了现网DBA慢SQL诊断经验，支持慢SQL根因分析，能同时按照可能性大小输出多个根因并提供针对性的建议。

1.10.3.3.2 环境部署

- 数据库运行正常。
- 指标采集系统运行正常。

1.10.3.3.3 使用指导

目前分布式只支持如下慢SQL根因

序号	根因	根因解释	补充说明
1	LOCK_CONTENTION	锁竞争	语句执行期间是否被阻塞，导致单SQL执行较慢。
2	HEAVY_SCAN_OPERATOR	扫描算子代价较大	执行计划中扫描算子代价较大，导致单SQL执行较慢。
3	UNUSED_AND_REDUNDANT_INDEX	无用/冗余索引	表中存在无用/冗余索引，影响插入更新语句性能。
4	WORKLOAD_CONTENTION	数据库负载集中	数据库负载集中导致实例整体执行性能较差。
5	CPU_RESOURCE_CONTENTION	系统CPU负载集中	由于外部进程等其他原因导致CPU资源紧张，实例整体SQL执行性能较差。

版权所有 © 华为云计算技术有限公司

序号	根因	根因解释	补充说明
6	IO_RESOURCE_CONTENTION	系统IO资源集中	由于外部进程等其他原因导致I/O资源紧张，实例整体SQL执行性能较差。
7	MEMORY_RESOURCE_CONTENTION	系统内存资源集中	由于外部进程等其他原因导致内存资源紧张，实例整体SQL执行性能较差。
8	ABNORMAL_NETWORK_STATUS	异常网络状态	网络异常，导致SQL执行性能较差。
9	WAIT_EVENT	等待事件	SQL执行期间的等待事件信息。
10	MISSING_INDEXES	缺少索引	缺失索引导致单SQL执行性能较差。
11	POOR_JOIN_PERFORMANCE	join代价较大	join算子代价较大，影响SQL语句执行性能。
12	REMOTE_QUERY	SQL不下推	SQL语句不下推，在CN上执行，导致执行效率慢。
13	LARGE_BROADCAST	大表broadcast	大表广播到其他DN节点，导致执行效率慢。
14	NO_ROOT_CAUSE_FOUND	没有发现根因	没有发现当前慢SQL的根因。

假设用户已经初始化配置文件目录confpath，则可以通过下述命令实现本特性的功能：

- 仅启动慢SQL诊断功能（慢SQL诊断根因数量由算法运行结果决定，数量不固定），启动命令如下（更多用法参考对service子命令的说明）：
  ```
  gs_dbmind service start -c confpath --only-run slow_sql_diagnosis
  ```
- 用户查询慢SQL诊断历史，命令如下：
  ```
  gs_dbmind component slow_query_diagnosis show -c confpath --instance instance --query SQL --start-time timestamps0 --end-time timestamps1
  ```
- 用户交互式诊断慢SQL，命令如下：
  ```
  gs_dbmind component slow_query_diagnosis diagnosis -c confpath --database dbname --schema schema_name --query SQL
  ```
- 启用慢SQL诊断后台任务，首先将opengauss_exporter下的pg_sql_statement_history开启，具体步骤如下：
  ```
  1、停止opengauss_exporter进程；
  2、进入dbmind/components/opengauss_exporter/yamls/statements.yml中，将该指标的status设置为enable；
  3、将slow_query_diagnosis加入到配置文件dbmind.conf下TIMED_TASK的task中，任务之间用逗号隔离；
  4、重启opengauss_exporter进程；
  5、运行 gs_dbmind service reload -c confpath 命令，启动慢SQL诊断后台任务；
  ```
- 用户手动清理历史预测结果，命令如下：
  ```
  gs_dbmind component slow_query_diagnosis clean -c confpath --retention-days DAYS
  ```
- 停止已启动的服务，命令如下：
  ```
  gs_dbmind service stop -c confpath
  ```


版权所有 © 华为云计算技术有限公司

1.10.3.3.4 获取帮助

模块命令行说明：

```
gs_dbmind component slow_query_diagnosis --help
usage:  [-h] -c DIRECTORY [--instance INSTANCE] [--database DATABASE] [--schema SCHEMA] [--query
SLOW_QUERY]
       [--start-time TIMESTAMP_IN_MICROSECONDS] [--end-time TIMESTAMP_IN_MICROSECONDS]
       [--retention-days DAYS]
       {show,clean,diagnosis}

Slow Query Diagnosis: Analyse the root cause of slow query

positional arguments:
  {show,clean,diagnosis}
                        choose a functionality to perform

optional arguments:
  -h, --help            show this help message and exit
  -c DIRECTORY, --conf DIRECTORY
                        Set the directory of configuration files
  --instance INSTANCE   Set the instance of slow query. Using in show.
  --database DATABASE   Set the name of database
  --schema SCHEMA       Set the schema of database
  --query SLOW_QUERY    Set a slow query you want to retrieve
  --start-time TIMESTAMP_IN_MICROSECONDS
                        Set the start time of a slow SQL diagnosis result to be retrieved
  --end-time TIMESTAMP_IN_MICROSECONDS
                        Set the end time of a slow SQL diagnosis result to be retrieved
  --retention-days DAYS
                        clear historical diagnosis results and set the maximum number of days to retain data
```

1.10.3.3.5 命令参考

表 1-27 gs_dbmind component slow_query_diagnosis 命令行说明

参数	参数说明	取值范围
-h, --help	帮助命令	-
action	动作参数	● show：结果展示 ● clean：清理结果 ● diagnosis：交互诊断
-c，--conf	配置目录	-
--instance	慢SQL实例地址，action为show时使用	数据库实例地址，样例：127.0.0.1:5432
--query	慢SQL文本	*
（可选）--start-time	显示开始时间的时间戳，单位毫秒；或日期时间格式为 %Y-%m-%d %H:%M:%S. action为diagnosis或show时使用	正整数或日期时间格式

版权所有 © 华为云计算技术有限公司

参数	参数说明	取值范围
（可选）--end-time	显示结束时间的时间戳，单位毫秒；或日期时间格式为 %Y-%m-%d %H:%M:%S. action为diagnosis或show时使用	正整数或日期时间格式
--retention-days	清理天数及结果，action为clean时使用	实数（当其小于等于0时，会将结果全部删除）
--database	慢SQL关联的数据库	-
(可选)--schema	慢SQL关联的schema，默认为public	-

1.10.3.3.6 注意事项说明

DBMind当前支持三种模式的慢SQL诊断能力，分别是：后台定时任务、命令行交互和API调用，下面对这三种方式的注意事项进行说明。

说明

如果没有特别指出具体使用模式，则表示该项三种模式都适用。

- 当前慢SQL诊断定时任务依赖opengauss_exporter指标采集慢SQL信息，该指标默认关闭，如果该指标采集不启用则慢SQL诊断功能无法正常运行。启动慢SQL采集的方法是手动将opengauss_exporter/yamls/statements.yaml中pg_sql_statement_history指标status设置成enable，再重启opengauss_exporter进程。
- 慢SQL诊断定时任务间隔支持用户配置，用户需要修改配置文件dbmind.conf中TIMED_TASK的slow_query_diagnosis_interval，然后执行reload操作才能生效，命令参考**service子命令**。
- 慢SQL诊断定时任务依赖dbe_perf.statement_history视图，当其数据量较大时可能出现查询超时，此时慢SQL后台诊断任务不会生成新的诊断结果。用户可以根据数据库状况设置该指标的超时时间timeout，以免超时时间太小不能采集到指标，具体修改的方法可以参见**Prometheus Exporter组件**。
- 在慢SQL诊断定时任务中，由于慢SQL流水数据源dbe_perf.statement_history视图中的query字段可能会出现截断导致query不完整，此时如果没有提供执行计划则不能进行诊断。
- 慢SQL诊断定时任务的采集基于opengauss_exporter实现，服务运行时不能保证全量采集，可能会遗漏部分慢SQL数据。
- 使用慢SQL命令行交互诊断功能时，工具基于RPC和数据采集服务获取必要的数据，因此如果RPC和数据采集服务没启动则无法诊断。
- 在使用慢SQL交互诊断功能时，工具会对传入的数据库名和schema进行检测，如果数据库或schema不存在则会进行提示，不能正常进行诊断。
- 在使用慢SQL诊断功能时，工具会对传入的数据库名和schema进行检测，如果其中包含非法字符则会进行提示并拒绝诊断。特殊情况说明：为适配云侧使用场景（将dbe_perf.statement_history作为SQL采集数据源时，其schema可能带'$'字

版权所有 © 华为云计算技术有限公司

符，例如$user,public ），在调用慢SQL诊断API接口时会对该场景进行特殊处理以支持诊断。

- 慢SQL诊断过程需要获取诊断语句的执行计划，在调用慢SQL诊断API接口时建议传递执行计划（对应query_plan参数），如果前端没有传递执行计划则工具会主动获取执行计划内容，此时需要确保执行诊断的用户具有获取query执行计划的权限。另外获取执行计划过程中如果SQL属于归一化SQL，则工具会基于PBE方式获取执行计划，获取过程中如果由于语法结构不支持、SQL截断等原因导致获取失败则会诊断失败，当SQL不属于归一化SQL时如果SQL截断则也不能正常诊断。
- 在使用慢SQL诊断API接口时，如果在用户传入的db_name不存在则无法诊断，另外当工具在传入的schema_name下获取执行计划失败时会自动其他schema下尝试获取执行计划直到成功获取为止，否则诊断失败。
- 当DBMind纳管多个实例时，使用命令行交互式慢SQL诊断功能时会提示用户选择哪一个实例，然后再进行诊断动作。
- 当前慢SQL诊断只支持DML语句。
- 慢SQL诊断每次只能诊断一条语句，如果输入多条则只会对第一条进行诊断。
- 当前不对系统视图根因进行诊断，根因统一为DATABASE_VIEW。
- 慢SQL诊断过程中如果用户不传schema信息，则schema默认为public。
- 由于reprocessing_exporter在采集磁盘占用率时只支持EXT和XFS文件系统，因此如果非上述文件系统磁盘超过阈值时不会出现磁盘空间不足的根因。
- 如果需要诊断LOCK_CONTENTION或WAIT_EVENT根因，需要使用API调用的方式，并传入debug_query_id参数。在分布式情况下，是使用dbe_perf.get_global_slow_sql_by_timestamp(start_time, end_time)函数查询全局慢SQL的等待事件，为了避免该操作过于耗时，查询时间范围为8小时前至当前时间。

1.10.3.4 SQLdiag

SQLdiag是GaussDB中SQL语句执行时长预测工具。现有的预测技术主要基于执行计划的预测方法，但这些预测方案仅适用于分析型场景且可以获取执行计划的任务，对于OLTP或者HTAP这样的快速、简单查询是没有太多使用价值的。与上述方案不同，SQLdiag着眼于数据库的历史SQL语句，通过对历史SQL语句的执行表现进行总结归纳，将之再用于推断新的未知业务上。由于短时间内数据库SQL语句执行时长不会有太大的差距，SQLdiag可以从历史数据中检测出与已执行SQL语句相似的语句结果集，并基于SQL向量化技术通过SQL模板化和深度学习这两种方法来预测SQL语句执行时长。本工具有如下优点：

1. 不需要SQL语句的执行计划，对数据库性能不会有任何的影响。
2. 使用场景广泛，目前业内的很多算法局限性比较高，比如只适用于OLTP或者其他场景，而SQLdiag使用场景广泛。
3. 该工具容易理解，只需要简单的操作，就可以训练出自己的预测模型。

本工具的典型应用场景是对一批即将上线的SQL语句进行透视，提前识别风险。

1.10.3.4.1 概述

SQLdiag是一个SQL语句执行时间预测工具，通过模板化方法或者深度学习方法，实现在不获取SQL语句执行计划的前提下，依据语句逻辑相似度与历史执行记录，预测SQL语句的执行时间并以此发现异常SQL。

 版权所有 © 华为云计算技术有限公司

说明

本功能主要适配集中式部署的数据库，对于分布式数据库可能不能做到全量支持。

1.10.3.4.2 使用指导

前提条件

- 需要保证用户提供训练数据。
- 如果用户通过提供的工具收集训练数据，则需要启用WDR功能，涉及到的参数为track_stmt_stat_level和log_min_duration_statement，具体情况见下面小结。
- 为保证预测准确率，用户提供的历史语句日志应尽可能全面并具有代表性。

SQL 流水采集方法

说明

SQL流水采集方法暂不支持分布式。

本工具需要用户提前准备数据，训练数据格式如下，每个样本通过换行符分隔：

```
SQL,EXECUTION_TIME
```

预测数据格式如下：

```
SQL
```

其中SQL表示**SQL语句的文本**，EXECUTION_TIME表示**SQL语句的执行时间**，样例数据见sample_data中的train.csv和predict.csv。

用户可以按照要求格式自己收集训练数据，工具也提供了脚本自动采集（load_sql_from_rd），该脚本基于WDR报告获取SQL信息，涉及到的参数有log_min_duration_statement和track_stmt_stat_level：

- 其中log_min_duration_statement表示慢SQL阈值，如果为0则全量收集，时间单位为毫秒；
- track_stmt_stat_level表示信息捕获的级别，建议设置为track_stmt_stat_level='L0,L0'

参数开启后，可能占用一定的系统资源，但一般不大。持续的高并发场景可能产生5%以内的损耗，数据库并发较低的场景，性能损耗可忽略。下述脚本存在于sqldiag根目录（DBMINDPATH/dbmind/components/sqldiag）中。

```
使用脚本获取训练集方式：
load_sql_from_wdr.py [-h] --port PORT --start-time START_TIME
                         --finish-time FINISH_TIME [--save-path SAVE_PATH]
例如：
    python load_sql_from_wdr.py --start-time "2021-04-25 00:00:00" --finish-time "2021-04-26 14:00:00" --
port 5432  --save-path ./data.csv
```

操作步骤

步骤1 提供历史日志以供模型训练

步骤2 进行训练与预测操作。

```
基于模板法的训练与预测：
   gs_dbmind component sqldiag [train, predict] -f FILE --model template --model-path
template_model_path --config-file config_path
基于DNN的训练与预测：
```

 版权所有 © 华为云计算技术有限公司

```
  gs_dbmind component sqldiag [train, predict] -f FILE --model dnn --model-path dnn_model_path --
config-file config_path
```

----结束

使用方法示例

使用提供的测试数据进行模板化训练：

```
gs_dbmind component sqldiag train -f DBMINDPATH/dbmind/components/sqldiag/sample_data/train.csv --
model template --model-path ./template --config-file config_path
```

使用提供的测试数据进行模板化预测：

```
gs_dbmind component sqldiag predict -f DBMINDPATH/dbmind/components/sqldiag/sample_data/
predict.csv --model template --model-path ./template --predicted-file ./result/t_result --config-file
config_path
```

使用提供的测试数据进行模板化模型更新：

```
gs_dbmind component sqldiag finetune -f DBMINDPATH/dbmind/components/sqldiag/sample_data/
train.csv --model template --model-path ./template --config-file config_path
```

使用提供的测试数据进行DNN训练：

```
gs_dbmind component sqldiag train -f DBMINDPATH/dbmind/components/sqldiag/sample_data/train.csv --
model dnn --model-path ./dnn_model --config-file config_path
```

使用提供的测试数据进行DNN预测：

```
gs_dbmind component sqldiag predict -f DBMINDPATH/dbmind/components/sqldiag/sample_data/
predict.csv --model dnn --model-path ./dnn_model --predicted-file --config-file config_path
```

使用提供的测试数据进行DNN模型更新：

```
gs_dbmind component sqldiag finetune -f DBMINDPATH/dbmind/components/sqldiag/sample_data/
train.csv --model dnn --model-path ./dnn_model --config-file config_path
```

1.10.3.4.3 获取帮助

使用SQLdiag工具前，您可以通过以下指令获取帮助。

```
gs_dbmind component sqldiag --help
```

显示如下帮助信息：

```
usage:   [-h] [-f CSV_FILE] [--predicted-file PREDICTED_FILE]
             [--model {template,dnn}] [--query QUERY] [--threshold THRESHOLD] --model-file
MODEL_FILEPATH
             [--config-file CONFIG_FILE]
             {train,predict,finetune}

SQLdiag integrated by openGauss.

positional arguments:
  {train,predict,finetune}
                        The training mode is to perform feature extraction and
                        model training based on historical SQL statements. The
                        prediction mode is to predict the execution time of a
                        new SQL statement through the trained model.

optional arguments:
  -h, --help            show this help message and exit
  -f CSV_FILE, --csv-file CSV_FILE
                        The data set for training or prediction. The file
                        format is CSV. If it is two columns, the format is
                        (SQL statement, duration time). If it is three
                        columns, the format is (timestamp of SQL statement
```

 版权所有 © 华为云计算技术有限公司

```
                execution time, SQL statement, duration time).
--predicted-file PREDICTED_FILE
                The file path to save the predicted result.
--model {template,dnn}
                Choose the model model to use.
--query QUERY       Input the queries to predict.
--threshold THRESHOLD
                Slow SQL threshold.
--model-file MODEL_FILE, --model-path MODEL_FILE
                The storage path of the model file, used to read or
                save the model file.
--config-file CONFIG_FILE, --config CONFIG_FILE
```

1.10.3.4.4 命令参考

表 1-28 命令行参数说明

参数	参数说明	取值范围
-f	训练或预测文件位置	-
--predicted-file	预测结果存储位置	-
--model	模型选择	template, dnn
--model-path	训练模型存储位置	-

1.10.3.4.5 常见问题处理

- 训练场景失败：请检查历史日志文件路径是否正确，且文件格式符合上文规定。
- 预测场景失败：请检查模型路径是否正确。确保待预测负载文件格式正确。

1.10.3.5 SQL Rewriter

1.10.3.5.1 概述

SQL Rewriter是一个SQL改写工具，根据预先设定的规则，将查询语句转换为更为高效或更为规范的形式，使得查询效率得以提升。

说明

- 本功能不适用包含子查询的语句。
- 本功能只支持SELECT语句和DELETE对整个表格删除的语句。
- 本功能包含12个改写规则，对不符合改写规则的语句，不会进行处理。
- 本功能会对原始查询语句和改写后语句进行屏幕输出，不建议对包含涉敏感信息的SQL语句进行改写。
- union转union all规则避免了去重，从而提升了查询性能，所得结果有可能存在冗余。
- 语句中如包含‘order by’+ 指定列名或‘group by’+ 指定列名，无法适用SelfJoin规则。
- 工具不保证查询语句等价转换，其目的是提升查询语句效率，一些推荐结果，需要结合业务实践进行优化，例如显性要求指定select的字段名。
- 本功能主要适配集中式部署的数据库，对于分布式数据库可能不能做到全量支持。

 版权所有 © 华为云计算技术有限公司

1.10.3.5.2 使用指导

前提条件

数据库状态正常、连接正常。

使用方法示例

以tpcc数据库为例：

```
gs_dbmind component sql_rewriter 5030 tpcc queries.sql --db-host 127.0.0.1 --db-user myname --schema public
```

queries.sql为需要改写的SQL，内容如下：

```
delete from bmsql_config;
delete from bmsql_config where cfg_name='1';
```

结果为多个改写后的查询语句，显示在屏幕（无法改写的语句，显示为空），如下：

```
+-------------------------------------------------------------------------+-----------------------------+
| Raw SQL                                                                 | Rewritten SQL               |
+-------------------------------------------------------------------------+-----------------------------+
| delete from bmsql_config;                                               | TRUNCATE TABLE bmsql_config; |
| delete from bmsql_config where cfg_name='1';                            |                             |
+-------------------------------------------------------------------------+-----------------------------+
```

1.10.3.5.3 获取帮助

使用SQL Rewriter前，您可以通过以下指令获取帮助。

```
gs_dbmind component sql_rewriter --help
```

显示如下帮助信息：

```
usage:  [-h] [--db-host DB_HOST] [--db-user DB_USER] [--schema SCHEMA]
        [-v] db_port database file

SQL Rewriter

positional arguments:
  db_port            Port for database
  database           Name for database
  file               File containing SQL statements which need to rewrite

optional arguments:
  -h, --help         show this help message and exit
  --db-host DB_HOST  Host for database
  --db-user DB_USER  Username for database log-in
  --schema SCHEMA    Schema name for the current business data
  -v, --version      show program's version number and exit
```

密码通过管道输入或交互式输入，对于免密用户，任意输入都可通过检验。

1.10.3.5.4 命令参考

表 1-29 命令行参数说明

参数名称	释义
db_port	数据库端口号
database	数据库名称

 版权所有 © 华为云计算技术有限公司

参数名称	释义
file	包含多个查询语句的文件路径
db-host	（可选）数据库主机号
db-user	（可选）数据库用户名
schema	（可选，模式为public）模式

1.10.3.5.5 常见问题处理

- SQL无法改写：请查看SQL是否符合改写规则或SQL语法是否正确。

1.10.3.6 Anomaly detection

1.10.3.6.1 概述

Anomaly detection 异常检测模块主要基于统计方法来实现时序数据来发现数据中存在的可能的异常情况。该模块框架解耦，可以实现不同异常检测算法的灵活替换，而且该模块功能可以根据时序数据的不同特征来自动选择算法，支持异常值检测、阈值检测、箱型图检测、梯度检测、增长率检测、波动率检测和状态转换检测。

须知

- 异常检测器的落盘存储依赖于元数据库，请勿在元数据库中对异常检测器进行手动修改。
- 当前版本仅支持，在主备切换、扩容和剔除节点的场景下，同一集群的检测器配置参数会被继承和保留，其他场景均不支持。

1.10.3.6.2 使用指导

假设指标采集系统运行正常，并且用户已经初始化了配置文件目录confpath，则可以通过下述命令实现本特性的功能：

仅启动异常检测功能：

```
gs_dbmind service start --conf confpath --only-run anomaly_detection
```

对于某一指标，在全部节点上，从timestamps1到timestamps2时间段内的数据进行概览：

```
gs_dbmind component anomaly_detection --conf confpath --action overview --metric metric_name --start-time timestamps1 --end-time timestamps2
```

对于某一指标，在特定节点上，从timestamps1到timestamps2时间段内的数据进行概览：

```
gs_dbmind component anomaly_detection --conf confpath --action overview --metric metric_name --start-time timestamps1 --end-time timestamps2 --host ip_address
```

对于某一指标，在全部节点上，从timestamps1到timestamps2时间段内的数据，以特定异常检测方式进行概览：

```
gs_dbmind component anomaly_detection --conf confpath --action overview --metric metric_name --start-time timestamps1 --end-time timestamps2 --anomaly anomaly_type
```


版权所有 © 华为云计算技术有限公司

对于某一指标，在特定节点，从timestamps1到timestamps2时间段内的数据，以特定异常检测方式进行概览：

```
gs_dbmind component anomaly_detection --conf confpath --action overview --metric metric_name --start-time timestamps1 --end-time timestamps2 --host ip_address --anomaly anomaly_type
```

对于某一指标，在特定节点，从timestamps1到timestamps2时间段内的数据，以特定异常检测方式进行可视化展示：

```
gs_dbmind component anomaly_detection --conf confpath --action plot --metric metric_name --start-time timestamps1 --end-time timestamps2 --host ip_address --anomaly anomaly_type
```

运行异常诊断后台任务：

```
参考10.3.3.3中方法，其对应定时任务为： anomaly_detection
```

停止已启动的服务：

```
gs_dbmind service stop --conf confpath
```

说明

- 在输入anomaly detection的参数时，start-time至少要比end-time早30秒以上。
- 异常检测功能依赖于异常检测器，可以通过异常检测器的查询接口/v1/api/app/anomaly-detection/detectors/{name}查看当前已添加的全部异常检测器。
- 添加检测器或更改检测器参数会将检测器状态变为启用。
- 对于初始化时默认的长期指标检测器（如slow_disk_detector和mem_leak_detector），其检测器的监测时间窗口长度是固定的，不支持修改，对于其duration参数的修改是无效的。
- 对于长期指标检测器，当收集到的数据低于一个星期时，不会进行检测。当数据已经长达一小时没有更新时，不会进行检测。

警告

- 异常检测器的落盘存储依赖于元数据库，请勿在元数据库中对异常检测器进行手动修改。
- 当前版本仅支持，在主备切换、扩容和剔除节点的场景下，同一集群的检测器配置参数会被继承与保留，其他场景均不支持。

1.10.3.6.3 获取帮助

模块命令行说明：

```
gs_dbmind component anomaly_detection --help
```

显示如下帮助信息：

```
usage: [-h] --action {overview,plot} -c CONF -m METRIC -s START_TIME -e END_TIME [-H HOST] [-a {level_shift,spike,seasonal,volatility_shift}]

Workload Anomaly detection: Anomaly detection of monitored metric.

optional arguments:
  -h, --help            show this help message and exit
  --action {overview,plot}
                        choose a functionality to perform
  -c CONF, --conf CONF  set the directory of configuration files
  -m METRIC, --metric METRIC
                        set the metric name you want to retrieve
  -s START_TIME, --start-time START_TIME
                        set the start time of for retrieving in ms, supporting UNIX-timestamp with microsecond or datetime format
  -e END_TIME, --end-time END_TIME
                        set the end time of for retrieving in ms, supporting UNIX-timestamp with microsecond or datetime format
```


版权所有 © 华为云计算技术有限公司

```
-H HOST, --host HOST  set a host of the metric, ip only or ip and port.
-a {level_shift,spike,seasonal,volatility_shift}, --anomaly {level_shift,spike,seasonal,volatility_shift}
                      set a anomaly detector of the metric from: "level_shift", "spike", "seasonal", "volatility_shift"
```

1.10.3.6.4 命令参考

表 1-30 命令行参数说明

参数	参数说明	取值范围
-h, --help	帮助命令	-
--action	动作参数	overview：概览 plot：可视化
-c，--conf	配置文件目录	-
-m，--metric	指定显示指标名	任意能采集到的指标。例如os_mem_usage、os_disk_usage等
-H, --host	指定数据来源地址信息，通过地址信息进行过滤	-ip地址或者ip地址加端口号
-a, --anomaly	指定异常检测方式，用于过滤	level_shift、spike、seasonal、volatility_shift
-s, --start-time	显示开始时间的时间戳，单位毫秒；或日期时间格式为 %Y-%m-%d %H:%M:%S.	正整数或日期时间格式
-e, --end-time	显示结束时间的时间戳，单位毫秒；或日期时间格式为 %Y-%m-%d %H:%M:%S.	正整数或日期时间格式

1.10.3.6.5 常见问题处理

- 概览场景失败：请检查配置文件路径是否正确，且配置文件信息是否完整。检查指标名称是否准确，检查host地址是否正确，检查异常检测类型是否准确，检查起止时间内指标是否存在对应数据。
- 可视化场景失败：请检查配置文件路径是否正确，且配置文件信息是否完整。检查指标名称是否准确，检查host地址是否正确，检查异常检测类型是否准确，检查起止时间内指标是否存在对应数据。

1.10.3.7 Distributed-key-advisor

对于分布式数据库来说，一个有效的分布键是十分重要的。可以通过使用本功能进行数据库分布键的自动化推荐。

分布键推荐功能提供数据库外部工具和SQL内置接口两种方式，支持数据迁移前与数据迁移后两种场景的分布键推荐。

1.10.3.7.1 外部工具

版权所有 © 华为云计算技术有限公司

概述

本分布键推荐功能在数据库内核外部实现，**以独立的工具形式呈现**，支持数据迁移前与数据迁移后两种场景下的分布键推荐，用户只需提供业务数据与少量相关数据库统计信息。

使用指导

前提条件

- 数据库运行状态良好，无资源紧张状况。
- 该特性是以外部工具的形式存在，需要用户在使用工具时提供元数据。
- 该特性当前仅针对**Hash分布**做推荐。
- 本特性暂不考虑计算倾斜、访问倾斜。
- 采用WDR报告中获取业务数据需提前设置guc参数track_stmt_stat_level='L0,L0'，并且要求执行节点是CN，以及指定用户具有monitor admin权限。
- 配置python3.7+环境。

操作步骤

步骤1 准备好包含有多条DML语句的文件作为输入的workload，对于数据迁移前场景，同时准备数据库相关统计信息的json文件。

数据库相关统计信息的JSON文件数据格式为：

```
{ table_name: # 表名
   {"size": size_value, # 表大小(整型)
    "rows": rows_value, # 表行数（整型）
    "primaryKey": "column_value", # 主键第一列（字符串）
    "column":           # 表列名及该列对应的distinct value值(浮点型)
       {"col1": col1_distinct_value,
        "col2": col2_distinct_value,
        "col3": col3_distinct_value
        }
    }
}
```

workload数据格式为：

```
SQL语句文本
```

步骤2 运行命令如下：

```
gs_dbmind component dkr [-f FILE] [-m MODE] [-s STATISTICS] [-p PORT] [-d DATABASE] [--schema SCHEMA] [--host HOST] [-U USER] [--dn DN] [--start_time START_TIME] [--end_time END_TIME] [--min_distinct_threshold MIN_DISTINCT_THRESHOLD] [--cost_type COST_TYPE][--max_replication_table_size MAX_REPLICATION_RTABLE_SIZE] [--prior_distribute_transaction] [--standard][--driver]
```

----结束

使用方法示例

当需要将业务数据从其他类型的数据库迁移到GaussDB 分布式数据库时（即“数据迁移前”场景），则可以使用下述命令启动推荐：

```
gs_dbmind component dkr -f test.sql -m offline -s table_info.json --dn 6 --min_distinct_threshold 0.5 --cost_type naive
```

当业务数据已经迁移到GaussDB 分布式数据库中后，对于需要对分布键进行调优的场景（即“数据迁移后”场景），则可使用下述命令启动推荐：

 版权所有 © 华为云计算技术有限公司

```
echo huawei.123 | gs_dbmind component dkr -f test.sql -m online -p 53400 -d tpch500high --host linux173 -
U dba --dn 6  --min_distinct_threshold 0.5 --cost_type optimizer
```

运行结果示例如下，其中“CREATE TABLE”表示应该创建分布键的列名，“HINT”提示可以创建为复制表的表名：

```
################################## Distribution key recommended result
##################################
CREATE TABLE customer(...) DISTRIBUTE BY HASH(c_custkey);
CREATE TABLE lineitem(...) DISTRIBUTE BY HASH(l_orderkey);
CREATE TABLE orders(...) DISTRIBUTE BY HASH(o_orderkey);
CREATE TABLE part(...) DISTRIBUTE BY HASH(p_partkey);
CREATE TABLE partsupp(...) DISTRIBUTE BY HASH(ps_partkey);
CREATE TABLE region(...) DISTRIBUTE BY HASH(r_regionkey);
CREATE TABLE supplier(...) DISTRIBUTE BY HASH(s_suppkey);
HINT: table nation can be set as a replication table. (table rows = 25)
```

获取帮助

启动调优程序之前，可以通过如下命令获取帮助信息：

```
gs_dbmind component dkr --help
```

输出帮助信息结果如下：

```
usage:              [-h] [-f FILE] [--standard STANDARD] [-s STATISTICS]
                    [-m {offline,online}] [--dn DN] [-p PORT] [-d DATABASE]
                    [--host HOST] [--schema SCHEMA] [-U USER]
                    [--start_time START_TIME] [--end_time END_TIME]
                    [--min_distinct_threshold MIN_DISTINCT_THRESHOLD]
                    [--cost_type {naive,optimizer}]
                    [--max_replication_table_size MAX_REPLICATION_TABLE_SIZE]
                    [--prior_distribute_transaction]
                    [--driver]

Generate a set of distributed keys for workload. two modes are supported:offline: (--file -m
-s) online: (--file -p -d --host -U --start_time --end_time --schema)

optional arguments:
  -h, --help            show this help message and exit
  -f FILE, --file FILE  File contains workload queries (One query per line) or
                        oracle stored procedure
  --standard STANDARD   queries in the file as standard format
  -s STATISTICS, --statistics STATISTICS
                        File contains statistical information
  -m {offline,online}, --mode {offline,online}
                        the current mode
  --dn DN               The number of data nodes
  -p PORT, --port PORT  Port of database
  -d DATABASE, --database DATABASE
                        Name of database
  --host HOST           Host for database
  --schema SCHEMA       Schema name for the current business data
  -U USER, --user USER  Username for database log-in
  --start_time START_TIME
                        Collect WDR report start time
  --end_time END_TIME   Collect WDR report end time
  --min_distinct_threshold MIN_DISTINCT_THRESHOLD
                        Minimum value of distinct value
  --cost_type {naive,optimizer}
                        Which cost estimation algorithm to use
  --max_replication_table_size MAX_REPLICATION_TABLE_SIZE
                        Maximum number of rows in the replication table
  --prior_distribute_transaction
                        Prioritize the processing of distributed transaction
  --driver              Whether to employ python-driver
```

 版权所有 © 华为云计算技术有限公司

常见问题处理

- 提示No recommended results for distribution key and replication table。检查数据库参数或**schema参数**是否输入正确。
- 获取WDR报告为空，检查时间参数格式是否正确，或者确认指定用户是否有monitor admin权限。

命令参考

表 1-31 命令行参数

参数	参数说明	取值范围
MODE	指定当前业务场景。	offline,online
STATISTICS	数据迁移前场景下，数据库统计信息文件路径；和UGO对接需要。	-
FILE	包含workload语句的文件路径。	-
DATABASE	连接数据库的名字。	-
SCHEMA	（可选）模式名称。	-
PORT	连接数据库的端口号。	-
DN	DN节点个数。	-
HOST	（可选）连接数据库主机号。	-
USER	（可选）连接数据库用户名。	-
START_TIME	（可选）收集WDR报告业务数据开始时间。	timestamp with time zone
END_TIME	（可选）收集WDR报告业务数据结束时间。	timestamp with time zone
MIN_DISTINCT_THRESHOLD	（可选）列的distinct value值最低，默认0.5。	-
COST_TYPE	（可选）代价计算类型，默认naive。	naive,optimizer
standard	（可选）指定当前业务文件格式与索引推荐章节介绍的extract_log脚本输出文件格式一致。	-
max_replication_table_size	（可选）指定复制表最大行数。	-
prior_distribute_transaction	（可选）指定对整个业务优先考虑分布式事物场景。	-
driver	（可选）指定使用python驱动连接数据库。	-

版权所有 © 华为云计算技术有限公司

1.10.3.7.2 SQL 内置接口

概述

分布键推荐SQL内置接口针对的是在分布式数据库下分布列以及分布方式的推荐，目的是在进行业务迁移或业务上线时，减少选择表分布列的人力成本。

使用指导

功能介绍

分布键推荐主要分为三种模式：

- 启发式模式：基于专家规则、预置权重的方式进行推荐。
- WhatIf代价模式：依赖于优化器的代价进行推荐。
- 半在线模式：在线收集普通SQL、存储过程中的DML、DQL语句，离线调用启发式和WhatIf代价模式进行分析。

应用场景

适用于有数据、无数据两种情况。无数据仅支持Hash分布推荐，有数据支持Hash分布、复制表推荐。

业务场景有以下几类：

1. 业务无数据，但是建立好了表模型，以及设计好了SQL负载，在此种情况下用户明确表模型中的维度表、事实表，所以在此种情况下复制表推荐的原则为用户指定维度表，然后对事实表采用启发式规则推荐。
2. 业务有数据，但无意向导入数据，则按照1进行。
3. 业务有数据，且有意向使用实际数据进行推荐。采用接近实际业务的数据量和SQL负载进行测试，有益于推荐更加准确的分布列，否则可能会出现分布列和分布方式推荐不准确的问题。
4. 在业务已有数据的测试环境上，不想导入SQL负载，可以通过使用跑业务的方式收集负载，并离线分析。

使用限制

- 使用推荐函数前需要创建对应的表，分析不支持临时表。
- 如果使用WhatIf代价模式，必须analyze。
- 需要关闭负载均衡，不关闭会导致收集部分负载。
- 推荐过程中不能做表结构的变更。
- 半在线模式只能有系统管理员调用。
- 半在线模式只会收集业务负载DML、DQL语句。
- 同一时间只能有一个数据库的负载收集功能打开。

命令参考

分布键推荐SQL内置接口主要使用以下系统函数，具体描述请参见《 开发者指南 》中“SQL参考 > 函数和操作符 > 分布列推荐函数”章节。

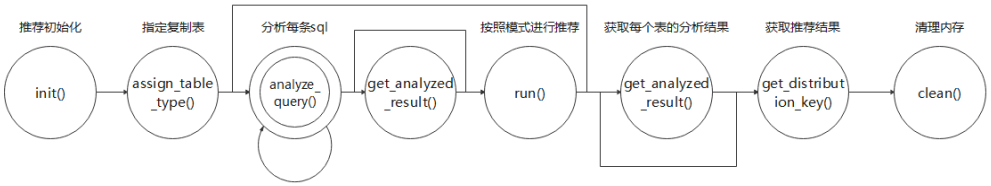
 版权所有 © 华为云计算技术有限公司

表 1-32 内置系统函数

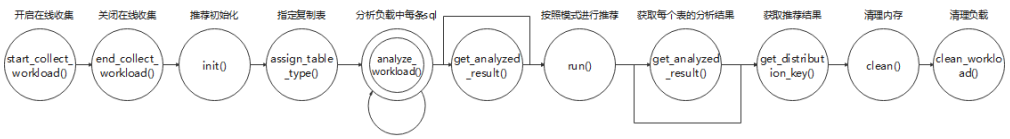
函数名	函数说明
sqladvisor.init	初始化分布键推荐参数。
sqladvisor.set_weight_params	设置启发式规则不同成分的权重。
sqladvisor.set_cost_params	使用Whtif代价模式可以设置的参数。
sqladvisor.analyze_query	导入需要推荐的SQL语句，并对语句的成分进行分析。
sqladvisor.analyze_workload	分析在线收集的负载信息。
sqladvisor.get_analyzed_result	获取当前表提取出来的有益成分。
sqladvisor.run	根据指定的模式和输入的SQL进行计算分析。
sqladvisor.clean	清理session中推荐过程中的全部内存。
sqladvisor.start_collect_workload	开启在线收集负载.
sqladvisor.end_collect_workload	关闭在线收集负载。
sqladvisor.clean_workload	清理负载中的内存。

使用建议

- 启发式、WhatIf代价推荐模式调用状态机。



- 半在线推荐模式调用状态机。



1.10.3.8 Cluster Diagnosis

1.10.3.8.1 概述

在现网业务中需要对发生的故障原因进行快速定位定界，本功能可以通过收集数据库集群中各个组件（如CMS、DN）等的信息和即时状态（如网络连通性），来判断集群环境是否存在故障，以及故障根因。可用于实现集群级别的故障根因诊断。

版权所有 © 华为云计算技术有限公司

DBMind对cmd_exporter进行加强，本版本支持CN、DN、CMS、CMA、ffic、OM_Monitor等日志采集，同时也支持基于节点间网络连通（如ping）状态采集。同时DBMind对现网故障场景进行了梳理，并对数据集进行枚举扩充，最终实现DN、CN故障快速定位。

说明

由于该功能是根据日志来进行诊断的，所以诊断结果中的时间可能因为日志的延迟或者日志的延迟处理，导致诊断结果中的时间晚于故障发生的时间。

表 1-33 现支持诊断的 CN、DN 故障根因列表

CN故障根因	DN故障根因
未知原因/Unknown	未知原因/Unknown
CN心跳超时/CN heartbeat timeout	实例被停止/DN manual stop
CN进程僵死/CN phony dead	磁盘故障/DN disk Damage
CORE/Core	网卡故障/DN NIC down
CN IP丢失/CN ip lost	端口冲突/DN port conflict
磁盘损坏或磁盘满/CN disk Damage	CM Server仲裁重启DN/DN restarted by cms
端口冲突/CN port conflict	进程僵死重启/DN phony dead
网卡故障/CN NIC down	CORE/Core
手动停止/CN manual stop	只读/DN read only
主机断网或宕机/CN down/ disconnection	主机断网或宕机/DN down/disconnection
CN与所有主DN断连/CN disconnected from dn	主备DN间网络异常/DN Primary disconnected with Standby
只读/CN read only	DN IP丢失/DN ip lost

说明

当cm_ctl query的集群状态输出结果异常时，一般是发生了调用栈输出，这种情况下难以获取集群状态，无法获取集群的诊断结果，相关状态标记为"abnormal_output_from_cm_ctl_query"，诊断结果为Unknown。

当dn节点处于"Offline"状态时，不对其进行集群故障诊断，返回状态为Normal，状态码-1。

1.10.3.8.2 使用指导

在DN集群产生异常告警时，一个完整的，用于启动集群故障分析功能的命令是：

```
gs_dbmind component cluster_diagnosis --conf {confpath} --host {ip_address} --role dn --time "2023-04-20 16:00:00" --method tree
```

输入此命令后，系统读取所设定时间前3分钟的日志记录，并对选定的DN集群使用选定方法进行分析，分析的结果示例如图所示：

版权所有 © 华为云计算技术有限公司

```
+---------------------+-----------------------+
|         Item        |         Result        |
+---------------------+-----------------------+
|         ping        |          Bad          |
|      dn_status      |         Normal        |
|    bind_ip_failed   |          Good         |
|   dn_ping_standby   |          Good         |
|     ffic_updated    |          Good         |
|    cms_phonydead    |          Good         |
| cms_restart_pending |          Good         |
|     dn_read_only    |          Good         |
|    dn_manual_stop   |          Good         |
|    dn_disk_damage   |          Good         |
|     dn_nic_down     |          Good         |
|   dn_port_conflict  |          Good         |
|     dn_writable     |          Good         |
|        Output       | DN down/disconnection |
+---------------------+-----------------------+
```

返回结果前半部分的字典给出对日志的解析结果，其中Good表示该项正常，Bad表示该项有异常；最后的Output表示输出结果，详情见**表1-33**。

说明

- 虽然单次诊断读取的是诊断时间点之前三分钟的日志和节点状态，但是由于网络延迟，模型计算时间等因素，实际时间会略短于3分钟，综合各种因素，以150秒内的诊断结果更为准确。

1.10.3.8.3 获取帮助

模块命令行说明：

```
gs_dbmind component cluster_diagnosis --help
```

显示如下帮助信息：

```
usage: [-h] --conf CONF --host HOST --role {cn,dn} [--time TIME] [--method {logical,tree}]
Cluster diagnosis.
optional arguments:
  -h, --help            show this help message and exit
  --conf CONF           set the directory of configuration files
  --host HOST           set the host of the cluster node, ip only.
  --role {cn,dn}        set the role of instance for diagnosis. roles: [cn]
                        are not supported for centralized DB.
  --time TIME           set time for diagnosis in timestamp(ms) or datetime format
  --method {logical,tree}
                        set method for the model: logical: if-else, tree: xgboost.
```


版权所有 © 华为云计算技术有限公司

1.10.3.8.4 命令参考

表 1-34 命令行参数说明

参数	参数说明	取值范围
-h, --help	帮助命令。	-
--conf	连接时序数据库需要的配置文件地址。	-
--host	分析的目标节点的IP地址。	-
--role	分析的目标节点的角色。	目前仅支持{cn, dn}。
--time	分析的异常发生的时间点，默认值是当前时间，日期时间格式为 %Y-%m-%d %H:%M:%S。	日期时间格式或者以毫秒为单位的时间戳。
--method	分析的方法，目前提供故障定位逻辑模型以及决策树模型两种方法。	{logical, tree}。

1.10.3.8.5 常见问题处理

如果尝试启动集群诊断时发现系统不能返回预期的诊断结果，需按照以下顺序逐项排查：

- 检查连接时序数据库需要的配置文件地址是否输入正确，是否包含集群诊断需要读取的日志文件，所用账户是否有权限读写日志，以及日志文件是否损坏，能否正常读写。
- 检查目标节点的IP地址是否有误。
- 检查是否支持对所选定的集群进行诊断，集群诊断目前仅支持对CN与DN的诊断，后期将会加入对GTM与ETCD的支持，并检查参数是否输入正确。
- 检查所键入的异常发生时间点是否符合日期时间格式，此外，集群诊断仅支持对当前及过去时刻的诊断。
- 检查所选择的诊断方法是否有输入正确，并且是否超出支持范围。
- 对于网卡故障的监测，建议在时序数据库采集数据时将cmd_exporter同时挂载在多个网卡上，以便在其中某一网卡故障时，其他网卡仍然能正常将消息送出。

1.10.3.9 智能巡检

1.10.3.9.1 概述

在现网业务中需要定期对数据库实例进行巡检并输出健康报告。DBMind构建了20+个巡检项，包括硬件状态、实例状态、数据库资源、数据库性能与诊断优化等类别。对其中关键资源指标趋势进行风险判断，避免不易发现的潜在问题影响实例健康。同时，支持生成日报、周报、月报供用户查看，并基于当前巡检项情况给出健康评分。支持用户对巡检项自定义设置告警阈值，方便用户根据业务特性进行调整，避免无意义告警。

版权所有 © 华为云计算技术有限公司

说明

- 月报与周报生成基于日报：如果未构建日报，则无法生成周报与月报；当日报数量不满足连续7天时，无法生成这7天对应的周报；当日报数量不满足连续14天以上时，无法生成对应的月报。此外，周报和月报的生成基于实例，比如出现实例IP和port的变化，会导致周报/月报无法生成；比如节点实例被删除，会导致周报/月报无法生成。采用定时方式生成周报和月报时，建议控制并发量，避免影响实时任务。配置文件中需要将实例的IP映射关系写到ip_map中，否则无法保证巡检结果的实例为管理IP。
- 周报和月报仅支持所有巡检项的巡检，不支持部分巡检项的拼接；因此周报和月报的自定义阈值需要由前端传入，规则与实时巡检等一致，没有传入的巡检项默认不进行告警（除了部分由前端控制的巡检项，见**10.3.9.2-使用指导**）
- 调用巡检接口时需要传递实例，该实例需要为集群主节点实例，否则在节点出现异常时无法返回所有节点的巡检结果。
- 智能巡检中db_size、buffer_hit_rate、db_tmp_file、db_deadlock、db_transaction等巡检项仅展示部署了opengauss_exporter的CN和DN的数据库结果。如果出现DN主备切换的情况，需要部署opengauss_exporter才可以获取新的节点的结果。
- 智能巡检中db_size、buffer_hit_rate、db_transaction、db_tmp_file、db_deadlock等巡检项展示部署了opengauss_exporter的CN和DN的数据库结果；user_login_out、db_latency、thread_pool等巡检项展示CN的结果；xlog_accumulate巡检项展示DN的结果。
- log_error_check和core_dump两个巡检项为了不漏掉关键信息，在实时巡检时仅支持45小时内数据；其他巡检项不能超过TSDB数据存储范围。

表 1-35 巡检项与权重设置

巡检类	巡检项	巡检字段	分数权重
系统资源 system_resource	CPU使用率	os_cpu_usage	0.06
	系统磁盘占用率	os_disk_usage	0.06
	内存使用率	os_mem_usage	0.06
	磁盘IO使用率	os_disk_ioutils	0.06
	网络状况	network_packet_loss	0.06
实例状态 instance_status	组件异常	component_error	0.05
数据库资源 database_resource	数据库目录占用率	data_directory	0.04
	数据库日志目录占用率	log_directory	0.04
	数据库大小	db_size	0
数据库性能 database_performance	Buffer命中率	buffer_hit_rate	0.03
	用户登录或者退出登录次数	user_login_out	0
	活跃Session率	active_session_rate	0.03
	日志异常检查	log_error_check	0.03
	线程池占用率	thread_pool	0.03
	数据库时延	db_latency	0.08

版权所有 © 华为云计算技术有限公司

巡检类	巡检项	巡检字段	分数权重
	数据库事务	db_transaction	0
	数据库临时文件大小	db_tmp_file	0
	数据库执行语句	db_exec_statement	0
	数据库死锁	db_deadlock	0.03
	数据库TPS性能	db_tps	0
	数据库Top Query	db_top_query	0
	长事务	long_transaction	0.05
	xlog堆积	xlog_accumulate	0.05
	oldestXmin长时间未推进	xmin_stuck	0.05
诊断优化 diagnosis_optimization	Core dump	core_dump	0.04
	动态内存	dynamic_memory	0.04
	程序内存	process_memory	0.04
	其他内存	other_memory	0.03
	GUC参数	guc_params	0.04

各巡检项实现细节与告警设置

- 系统资源
  - CPU使用率：获取TSDB中的“os_cpu_user_usage”和“os_cpu_iowait_usage”两个指标。
    - 对于os_cpu_user_usage，观测是否持续上升，若是则进行告警；同时，观测数值是否大于0.7，若超出阈值，则进行告警；最后，进行趋势预测，判断未来24小时数值是否大于0.7，若超出阈值，则进行告警。
    - 对于os_cpu_iowait_usage，当发现持续上升时会进行告警，当数值大于0.3时，进行告警
  - 系统磁盘占用率：获取TSDB中的“os_disk_usage”指标，观测其中系统盘（mount_point='/'）的磁盘使用率，当数值大于0.8时，进行告警。
  - 内存使用率：获取TSDB中的“os_mem_usage”指标，当发现持续上升时会进行告警；当数值大于0.7时，进行告警；最后，进行趋势预测，如果判断未来24小时数值大于0.8，进行告警。
  - 磁盘IO使用率：获取TSDB中的“os_disk_ioutils”指标，当数值大于0.8时，进行告警。
  - 网络状况：获取TSDB中的“gaussdb_network_packet_loss”指标，得出数据库节点间的网络连接状况，当丢包率大于0.05时，进行告警。

版权所有 © 华为云计算技术有限公司

- 实例状态
  - 组件异常：根据instance从元数据库中获取节点的状态信息，得出时间范围内当前集群中CN和DN的状态，当出现异常状态（即值不为-1）时，进行告警。
- 数据库资源
  - 数据库目录占用率：获取TSDB中的“pg_node_info_uptime”指标，解析出各个数据库实例的数据目录，同时获取“gaussdb_progress_cpu_usage”指标，匹配判断是否存在change_root，存在则组合成正确的目录路径，然后获取“os_disk_usage”指标，匹配判断对应数据目录的占用率，当发现持续上升时，进行告警；当数值大于0.8时，进行告警；最后，进行趋势预测，如果判断未来24小时数值大于0.8，进行告警；判断文件系统是否有误，不匹配则进行告警。
  - 数据库日志目录占用率：获取TSDB中的“pg_node_info_uptime”指标，然后解析出各个数据库实例的日志目录，同时获取“gaussdb_progress_cpu_usage”指标，匹配判断是否存在change_root，存在则组合成正确的目录路径，，然后获取“os_disk_usage”指标，匹配判断对应日志目录的占用率，当发现持续上升时会进行告警，当数值大于0.8时，同样进行告警，再进行趋势预测，如果判断未来24小时数值大于0.8，也会进行告警，最后还会判断文件系统是否有误，不匹配则进行告警。
  - 数据库大小：获取TSDB中的“pg_database_size_bytes”指标，获取所有数据库的大小并进行记录，用于前端进行趋势展示。暂无告警。
- 数据库性能
  - Buffer命中率：获取TSDB中的“pg_db_blks_access”指标，当数值小于0.9时，进行告警。
  - 用户登录或者退出登录次数：获取TSDB中的“gaussdb_user_login”和“gaussdb_user_logout”两个指标，获取1分钟内平均每秒用户登录或者退出登录次数，用于前端进行趋势展示。暂无告警。
  - 活跃Session率：获取TSDB中的活跃会话数gaussdb_active_connection和总会话数gaussdb_total_connection指标，活跃session率即为gaussdb_active_connection / gaussdb_total_connection，当其比值小于0.8时，进行告警。
  - 日志异常检查：获取TSDB中“gaussdb_log_*”的指标，统计每种日志错误出现的次数并进行相加，如果出现日志异常，则将日志异常的出现次数返回前端，进行告警。
  - 线程池占用率：获取TSDB中的“pg_thread_pool_rate”指标，当发现持续上升时会进行告警，当数值小于0.9时，进行告警。
  - 数据库时延：获取TSDB中的“statement_responsetime_percentile_p95”和“statement_responsetime_percentile_p80”两个指标，当发现持续上升时会进行告警。
  - 数据库事务：获取TSDB中的“pg_db_xact_commit”和“pg_db_xact_rollback”两个指标，获取业务中提交和回滚的次数，用于前端进行趋势展示。暂无告警。
  - 数据库临时文件大小：获取TSDB中的“pg_db_temp_files”指标，获取数据库临时文件的大小，用于前端进行趋势展示。暂无告警。
  - 数据库执行语句：获取TSDB中的“pg_sql_count_select”、“pg_sql_count_update”、“pg_sql_count_insert”和“pg_sql_count_delete”四个指标，获取数据库中select、update、insert、delete的执行次数，用于前端进行趋势展示。暂无告警。

 版权所有 © 华为云计算技术有限公司

  - 数据库死锁：获取TSDB中的“pg_db_deadlocks”指标，获取数据库中出现的死锁情况，当死锁数不为0时，表示数据库出现死锁。数值表示死锁数量。暂无告警。
  - 数据库TPS性能：获取TSDB中的“gaussdb_qps_by_instance”和“qps”两个指标，获取数据库的QPS和TPS信息，用于前端进行趋势展示。暂无告警。
  - 数据库Top Query：通过执行SQL语句从dbe_perf.statement查询出现调用次数最多的TopK个查询，用于前端展示。当前K为10。暂无告警。
  - 长事务：通过执行SQL语句从pg_catalog.pg_stat_activity查询超过12小时未结束的活跃事务，当返回结果列表的长度大于0时，表示存在长事务，进行告警。
  - xlog堆积：获取TSDB中的“gaussdb_xlog_count”指标，获取xlog目录下的文件数量，当数量大于3000时，进行告警。
  - oldestXmin长时间未推进：获取TSDB中的“oldestxmin_increase”指标，当oldestxmin超过12小时未出现变化，则表示长时间为推进，此时巡检返回数据中数值为0，进行告警。
- 诊断优化
  - Core dump：获取TSDB中的“gaussdb_log_ffic”指标，当返回值中count大于0时，表示出现core dump，进行告警。
  - 动态内存：获取TSDB中的“pg_total_memory_detail_mbytes”指标，计算其中“dynamic_used_memory”相对于“max_dynamic_memory”的比例，当数值大于0.8时，进行告警；计算其中“dynamic_used_shrctx”相对于“max_dynamic_memory”的比例，当数值大于0.8时，同样进行告警。
  - 程序内存：获取TSDB中的“pg_total_memory_detail_mbytes”指标，计算其中“process_used_memory”相对于“max_process_memory”的比例，当数值大于0.8时，进行告警。
  - 其他内存：获取TSDB中的“pg_total_memory_detail_mbytes”指标，获取其中“other_used_memory”的值，当发现持续上升时会进行告警，当超过20G时，进行告警。
  - GUC参数：获取TSDB中的“pg_total_memory_detail_mbytes”和“pg_settings_setting”指标，获取其中“max_process_memory”、“shared_buffers”和“work_mem”的值，将当前的值与DBMind根据workload推算出的最优参数进行对比，如果当前参数与最优参数相差大，结果中的warning字段为true，进行告警。

版权所有 © 华为云计算技术有限公司

说明

健康评分根据各巡检项的重要性权重以及异常程度来给出评分，最后汇总成当前实例的健康评分，具体计算公式如下：

$$score=\sum_{i=1}^{n} w_i * S_i$$

上述：

$$\sum_{i=1}^{n} w_i = 100$$

S_i = 0 or 1（其中0代表巡检项异常，1代表正常）。

此外，结论中会提供TOP3列表，TOP3指的是异常情况最严重的三个巡检项，通过统计各个巡检项的异常数量，再结合巡检项的权重进行排序，取前3个作为TOP3列表，如果没有异常，则为空。

支持的巡检自定义阈值

当前巡检项的告警类型与默认设置如下表所示。

- √和数字表示该巡检项已设置默认告警阈值
- -表示该巡检项支持设置对应告警类型
- ×表示该巡检项不支持设置对应告警类型：

巡检项	巡检项	数值范围	异常类型				判断标准
			超过阈值	预测超过阈值	持续上升	文件类型有误	
CPU使用率	os_cpu_usage	[0, 1]	>0.7	>0.7	√	×	任意异常出现则告警
磁盘IO使用率	os_disk_ioutils	[0, 1]	>0.8	-	-	×	任意异常出现则告警
系统磁盘占用率	os_disk_usage	[0, 1]	>0.8	-	-	×	任意异常出现则告警
内存使用率	os_mem_usage	[0, 1]	>0.7	>0.8	√	×	任意异常出现则告警
网络状况	network_packet_loss	[0, 1]	>0.05	-	×	×	任意异常出现则告警
组件异常	component_error	×	×	×	×	×	任意异常出现则告警
数据库目录占用率	data_directory	[0, 1]	>0.8	>0.8	-	√	任意异常出现则告警

版权所有 © 华为云计算技术有限公司

数据库日志目录占用率	log_directory	[0, 1]	>0.8	>0.8	-	√	任意异常出现则告警
数据库大小	db_size	[0, ∞)	-	-	-	×	暂无告警
Buffer命中率	buffer_hit_rate	[0, 1]	<0.9	-	×	×	任意异常出现则告警
用户登录或者退出登录次数	user_login_out	[0, ∞)	-	-	-	×	暂无告警
活跃Session率	active_session_rate	[0, 1]	<0.8	-	×	×	任意异常出现则告警
日志异常检查	log_error_check	×	×	×	×	×	存在则告警
线程池占用率	thread_pool	[0, 1]	>0.9	-	√	×	任意异常出现则告警
数据库时延	db_latency	[0, ∞)	-	-	√	×	任意异常出现则告警
数据库事务	db_transaction	[0, ∞)	-	-	-	×	暂无告警
数据库临时文件大小	db_tmp_file	[0, ∞)	-	-	-	×	暂无告警
数据库执行语句	db_exec_statement	[0, ∞)	-	-	×	×	暂无告警
数据库死锁	db_deadlock	[0, ∞)	-	-	-	×	暂无告警
数据库TPS性能	db_tps	[0, ∞)	-	-	-	×	暂无告警
数据库Top Query	db_top_query	×	×	×	×	×	暂无告警
长事务	long_transaction	×	×	×	×	×	存在则告警
oldestXmin长时间未推进	xmin_stuck	[0, ∞)	×	×	×	×	任意异常出现则告警
xlog堆积	xlog_accumulate	[0, ∞)	>3000	-	-	×	任意异常出现则告警
Core dump	core_dump	×	×	×	×	×	存在则告警
动态内存	dynamic_memory	[0, 1]	>0.8	-	-	×	任意异常出现则告警
程序内存	process_memory	[0, 1]	>0.8	-	-	×	任意异常出现则告警

版权所有 © 华为云计算技术有限公司

其他内存	other_memory	[0, ∞)	>20*1024	-	√	×	任意异常出现则告警
GUC参数	guc_params	×	×	×	×	×	warning中存在值则告警

说明

- 持续上升和预测超过阈值两类告警涉及检测算法，会占用DBMind性能，配置时减少开启的数量，避免影响服务性能以及返回时间等。
- 预测超过阈值需要设置预测的时间范围，以分钟为单位，设置范围(0, 48 * 60]。建议设置的时间长度小于数据的时间长度。

1.10.3.9.2 使用指导

通过巡检相关 API 实现功能调用

- 智能巡检接口

  示例：

  ```
  curl -X 'POST' "http://127.0.0.1:8080/v1/api/app/real-time-inspection?inspection_type=real_time_check&start_time=1689210000000&end_time=1689296400000&instance=127.0.0.1:5432" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"system_resource": ["os_mem_usage"], "instance_status": [], "database_resource": [], "database_performance": [], "diagnosis_optimization": []}' -H "Authorization: bearer xxx"
  ```

  如果使用HTTPS协议，则查询示例：

  ```
  curl -X 'POST' "https://127.0.0.1:8080/v1/api/app/real-time-inspection?inspection_type=real_time_check&start_time=1689210000000&end_time=1689296400000&instance=127.0.0.1:5432" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"system_resource": ["os_mem_usage"], "instance_status": [], "database_resource": [], "database_performance": [], "diagnosis_optimization": []}' -H "Authorization: bearer xxx" --cacert xx.crt --key xx.key --cert xx.crt
  ```

  如果使用自定义阈值，查询示例：

  ```
  curl -X 'POST' "https://127.0.0.1:8080/v1/api/app/real-time-inspection?inspection_type=real_time_check&start_time=1689210000000&end_time=1689296400000&instance=127.0.0.1:5432" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"system_resource": [{"os_mem_usage": {"increase": false, "threshold": [], "forecast": [1440, 0.0, 0.8]}}], "instance_status": [], "database_resource": [], "database_performance": [], "diagnosis_optimization": []}' -H "Authorization: bearer xxx" --cacert xx.crt --key xx.key --cert xx.crt
  ```

  返回结构示例如下：

  ```
  {
      "data": {
          "conclusion": {
              "full_score": 0.06,
              "health_score": 0.06,
              "health_status": "bad",
              "top3": []
          },
          "database_performance": {},
          "database_resource": {},
          "diagnosis_optimization": {},
          "instance_status": {},
          "system_resource": {
              "os_mem_usage": {
                  "127.0.0.1": {
                      "data": [0.31643905373281667],
                      "statistic": {
                          "avg": 0.311,
                          "max": 0.3166,
  ```


版权所有 © 华为云计算技术有限公司

```
                    "min": 0.3057,
                    "the_95th": 0.3153
                },
                "timestamps": [1694674713000],
                "warnings": {
                    "increase_warning": true
                }
            }
        }
    }
  },
  "success": true
}
```

- 展示巡检任务接口示例：

```
curl -X 'GET' "http://127.0.0.1:8080/v1/api/app/real-time-inspection/list?instance=127.0.0.1:5432" -H 'accept: application/json' -H "Authorization: bearer xxx"
```

如果使用HTTPS协议，则查询示例：

```
curl -X 'GET' "https://127.0.0.1:8080/v1/api/app/real-time-inspection/list?instance=127.0.0.1:5432" -H 'accept: application/json' -H "Authorization: bearer xxx" --cacert xx.crt --key xx.key --cert xx.crt
```

返回结构如下：

```
{"data":{"header":["instance","start","end","id","state","cost_time","inspection_type"],"rows":[["127.0.0.1:5432",1689210000000,1689296400000,5,"success",0.033701,"real_time_check"]]},"success":true}
```

- 获取指定巡检任务的巡检结果接口示例：

```
curl -X 'GET' "http://127.0.0.1:8080/v1/api/summary/real-time-inspection?spec_id=5&instance=127.0.0.1:5432" -H 'accept: application/json' -H "Authorization: bearer xxx"
```

如果使用HTTPS协议，则查询示例为：

```
curl -X 'GET' "https://127.0.0.1:8080/v1/api/summary/real-time-inspection?spec_id=5&instance=127.0.0.1:5432" -H 'accept: application/json' -H "Authorization: bearer xxx" --cacert xx.crt --key xx.key --cert xx.crt
```

返回结构如下，与**智能巡检接口返回结构**一致。

- 删除指定的巡检任务接口示例：

```
curl -X 'DELETE' "http://127.0.0.1:8080/v1/api/app/real-time-inspection?spec_id=5&instance=127.0.0.1:5432" -H 'accept: application/json' -H "Authorization: bearer xxx"
```

如果使用HTTPS协议，则删除示例：

```
curl -X 'DELETE' "https://127.0.0.1:8080/v1/api/app/real-time-inspection?spec_id=5&instance=127.0.0.1:5432" -H 'accept: application/json' -H "Authorization: bearer xxx" --cacert xx.crt --key xx.key --cert xx.crt
```

返回结构如下：

```
{"data":{"success":true},"success":true}
```

【自定义阈值参数设置】

- 自定义阈值传参方式，各告警类型对应的key如下表所示：

告警类型	键
持续上升	increase
超过阈值	threshold
预测超过阈值	forecast
文件类型有误	ftype

版权所有 © 华为云计算技术有限公司

- 启用默认告警配置（505.1.0之前的版本）

```
{
"system_resource": [
"os_mem_usage"
],
"instance_status": [],
"database_resource": [],
"database_performance": [],
"diagnosis_optimization": []
}
```

- 启用默认告警配置（505.1.0及之后版本）

```
{
"system_resource": [
{
"os_mem_usage": true
}
],
"instance_status": [],
"database_resource": [],
"database_performance": [],
"diagnosis_optimization": []
}
```

- 不启用告警

```
{
"system_resource": [
{
"os_mem_usage": false
}
],
"instance_status": [],
"database_resource": [],
"database_performance": [],
"diagnosis_optimization": []
}
```

- 启用自定义告警。如下所示，os_mem_usage启用持续上升告警、自定义设置阈值告警和预测阈值告警；os_disk_usage不启用持续上升告警、阈值告警，启用预测阈值告警：

```
{
"system_resource": [
{
"os_mem_usage": {
"increase": true,
"threshold": [0.0, 0.8],
"forecast": [1440, 0.0, 0.8]
},
"os_disk_usage": {
"forecast": [1440, 0.0, 0.8]
}
}
],
"instance_status": [],
"database_resource": [],
"database_performance": [],
"diagnosis_optimization": []
}
```

- 部分巡检项不支持自定义配置阈值：组件异常、日志异常检查、数据库Top Query、长事务、oldestXmin长时间未推进、Core dump、GUC参数，是否告警由云侧前端控制，忽略由前端传入的参数。
- 各个巡检项支持的阈值类型见**10.3.9-概述**中自定义阈值表格，前端传入不支持的告警类型会报错。
- 对于部分巡检项，存在子巡检项，列表如下：

版权所有 © 华为云计算技术有限公司

巡检项	子巡检项
os_cpu_usage	cpu_user
	cpu_iowait
user_login_out	login
	logout
db_latency	p95
	p80
db_transaction	commit
	rollback
db_exec_statement	select
	update
	insert
	delete
db_tps	tps
	qps
dynamic_memory	dynamic_used_memory
	dynamic_used_shrctx

当子巡检项的告警配置一致时，可以省略重复值，具体方式如下：

- 重复设置os_cpu_usage阈值自定义

```
{
"system_resource": [
{
"os_cpu_usage": {
"cpu_user": {
"increase": false,
"threshold": [],
"forecast": [1440, 0.0, 0.8]
},
"cpu_iowait": {
"increase": false,
"threshold": [],
"forecast": [1440, 0.0, 0.8]
}
}
}
],
"instance_status": [],
"database_resource": [],
"database_performance": [],
"diagnosis_optimization": []
}
```

- 简化设置os_cpu_usage阈值自定义

```
{
"system_resource": [
```


版权所有 © 华为云计算技术有限公司

```
{
"os_cpu_usage": {
"increase": false,
"threshold": [],
"forecast": [1440, 0.0, 0.8]
}
}
],
"instance_status": [],
"database_resource": [],
"database_performance": [],
"diagnosis_optimization": []
}
```


巡检项结果返回值示例

- 集群：

  以3CN、3主DN、6备DN为例，共3个节点，且opengauss_exporter部署在CN和主DN上（分布式仅部署在CN上，结果仅展示CN，无法获取DN节点上信息）：

  3CN: 127.0.0.1:9080, 127.0.0.2:9080, 127.0.0.3:9080

  3主DN: 127.0.0.1:40001, 127.0.0.2:50001, 127.0.0.3:60001

  6备DN: 127.0.0.1:5000, 127.0.0.1:60001, 127.0.0.2:40001, 127.0.0.2:60001, 127.0.0.3:40001, 127.0.0.3:50001

- 返回值结构类型：
  - a)：以节点为key：{"127.0.0.1": xxx, "127.0.0.2": xxx, "127.0.0.3": xxx}
  - b)：以CN为key：{"127.0.0.1:9080": xxx, "127.0.0.2:9080": xxx, "127.0.0.3:9080": xxx}
  - c)：以cn和主dn为key(以opengauss_exporter部署的cn和dn为key)：{"127.0.0.1:9080": xxx, "127.0.0.2:9080": xxx, "127.0.0.3:9080": xxx, "127.0.0.1:40001": xxx, "127.0.0.2:50001": xxx, "127.0.0.3:60001": xxx}
  - d)：以dn为key：{"127.0.0.1:40001": xxx, "127.0.0.2:50001": xxx, "127.0.0.3:60001": xxx, "127.0.0.2:40001": xxx, "127.0.0.3:50001": xxx, "127.0.0.1:60001": xxx , "127.0.0.3:40001": xxx, "127.0.0.1:50001": xxx, "127.0.0.2:60001": xxx }

巡检项对应的返回结构

巡检项	分布式	备注
os_cpu_usage	a)	两层结构，子巡检项：cpu_user、cpu_iowait
os_disk_ioutils	a)	-
os_disk_usage	a)	-
os_mem_usage	a)	-
network_packet_loss	a)	两层结构，展示节点到节点的网络状况
component_error	a)	两层结构，子巡检项：cn、dn

版权所有 © 华为云计算技术有限公司

巡检项	分布式	备注
data_directory	c)	-
log_directory	c)	-
db_size	c)	两层结构，{"ip:port": {"db": xxx}}
buffer_hit_rate	c)	两层结构，{"ip:port": {"db": xxx}}
user_login_out	c)	两层结构，子巡检项：login、logout
active_session_rate	c)	-
log_error_check	a)	-
thread_pool	c)	-
db_latency	c)	两层结构，子巡检项：p80、p95
db_transaction	c)	三层结构：{"commit": {"ip:port": {"db": xxx}}}
db_tmp_file	c)	两层结构，{"ip:port": {"db": xxx}}
db_exec_statement	c)	两层结构，子巡检项：select、update、insert、delete
db_deadlock	c)	两层结构，{"ip:port": {"db": xxx}}
db_tps	c)	两层结构，子巡检项：qps、tps
db_top_query	b)	两层结构，{"ip:port": [{xxx}]}
long_transaction	b)	两层结构，{"ip:port": [{xxx}]}
xmin_stuck	c)	-
xlog_accumulate	d)	-
core_dump	a)	-
dynamic_memory	c)	两层结构，子巡检项：dynamic_used_memory、dynamic_used_shrctx
process_memory	c)	-
other_memory	c)	-
guc_params	c)	-

说明

巡检结果会存储到DBMind元数据库中，DBMind会定期清除老数据以避免磁盘膨胀。

版权所有 © 华为云计算技术有限公司

1.10.3.9.3 命令参考

表 1-36 智能巡检接口

API	入参	参数介绍	请求方法	功能描述与预期返回结果
/v1/api/app/real-time-inspection	inspection_type start_time end_time tz instance inspection_items	巡检类型，String，必选 起始时间，String，可选 终止时间，String，可选 时区信息，String，可选 实例IP，String，必选 巡检项，dict，必选	POST	执行智能巡检功能并返回巡检结果
/v1/api/app/real-time-inspection/list	instance	实例IP，String，必选	GET	展示巡检任务的基础信息
/v1/api/summary/real-time-inspection	instance spec_id	实例IP，String，必选 巡检任务ID，String，必选	GET	获取指定巡检任务的巡检结果
/v1/api/app/real-time-inspection	instance spec_id	实例IP，String，必选 巡检任务ID，String，必选	DELETE	删除指定的巡检任务

1.10.3.9.4 常见问题处理

- 月报与周报生成基于日报：如果未构建日报，则无法生成周报与月报；当日报数量不满足连续7天时，无法生成这7天对应的周报；当日报数量不满足连续14天以上时，无法生成对应的月报。此外，周报和月报的生成基于实例，比如出现实例IP和port的变化，会导致周报/月报无法生成；比如节点实例被删除，会导致周报/月报无法生成。采用定时方式生成周报和月报时，建议控制并发量，避免影响实时任务。配置文件中需要将实例的IP映射关系写到ip_map中，否则无法保证巡检结果的实例为管理IP。
- 周报和月报仅支持所有巡检项的巡检，不支持部分巡检项的拼接；因此周报和月报的自定义阈值需要由前端传入，规则与实时巡检等一致，没有传入的巡检项默认不进行告警（除了部分由前端控制的巡检项，见**10.3.9.2-使用指导**）。
- 调用巡检接口时需要传递实例，该实例需要为集群主节点实例，否则在节点出现异常时无法返回所有节点的巡检结果。
- 智能巡检中db_size、buffer_hit_rate、db_tmp_file、db_deadlock、db_transaction等巡检项仅展示部署了opengauss_exporter的CN和DN的数据库结

版权所有 © 华为云计算技术有限公司

果。如果出现DN主备切换的情况，需要部署opengauss_exporter才可以获取新的节点的结果。

- 智能巡检中db_size、buffer_hit_rate、db_transaction、db_tmp_file、db_deadlock等巡检项展示部署了opengauss_exporter的CN和DN的数据库结果；user_login_out、db_latency、thread_pool等巡检项展示CN的结果；xlog_accumulate巡检项展示DN的结果。
- log_error_check和core_dump两个巡检项为了不漏掉关键信息，在实时巡检时仅支持45小时内数据；其他巡检项不能超过TSDB数据存储范围。

1.11 ABO 优化器

1.11.1 智能基数估计

1.11.1.1 概述

智能基数估计使用库内贝叶斯网络模型对多列数据样本联和分布进行建模，从而能够对多列等值查询提供更加准确的基数估计。更加准确的基数估计能够显著提高优化器对于计划和算子的选择的准确性，从而提高数据库整体吞吐量。

1.11.1.2 前置条件

数据库运行正常，GUC参数enable_ai_stats设置为on，multi_stats_type设置为'BAYESNET'或者'ALL'。

1.11.1.3 使用指导

步骤1 设置采样方式为按照采样率采样，即设置GUC参数default_statistics_target为[-100, -1]之间的整数，表示采样百分比。

步骤2 使用ANALYZE(([column_name,])) 进行数据统计和模型创建。

步骤3 输入查询，如果查询涉及到的等值查询列上有统计模型创建，那么会自动是使用统计模型进行选择率估计。

步骤4 不再需要智能统计模型的时候，使用ALTER TABLE [table_name] DELETE STATISTICS (([column_name,]))进行统计信息以及模型删除。

----结束

其他使用方法和接口详见《开发者指南》中“SQL参考 > SQL语法 > ALTER TABLE和ANALYZE | ANALYSE”章节。

1.11.1.4 最佳实践

生成如下数据表：

```
benchmark=# \d part;
id             | integer                  |
p_brand        | character varying(256)   |
p_type         | character varying(256)   |
p_container    | character varying(256)   |
p_mfgr         | character varying(256)   |
```

插入10,000,000行数据：

 版权所有 © 华为云计算技术有限公司

```
benchmark=# select count(1) from part1;
10000000
```

在数据表上创建四种不同的多列索引：

```
benchmark=# select * from pg_indexes where tablename='part1';
public     | part1     | brand_type_container   |             | CREATE INDEX brand_type_container ON part1
USING btree (p_brand, p_type, p_container) TABLESPACE pg_default
public     | part1     | brand_type_mfgr        |             | CREATE INDEX brand_type_mfgr ON part1 USING
btree (p_brand, p_type, p_mfgr) TABLESPACE pg_default
public     | part1     | brand_container_mfgr   |             | CREATE INDEX brand_container_mfgr ON part1
USING btree (p_brand, p_container, p_mfgr) TABLESPACE pg_default
public     | part1     | type_container_mfgr    |             | CREATE INDEX type_container_mfgr ON part1
USING btree (p_type, p_container, p_mfgr) TABLESPACE pg_default
```

针对数据表生成一批包含多列等值条件的查询，如下：

```
explain analyze select * from part1 where p_container='LG CASE' AND p_brand='Brand#34' AND
p_mfgr='Manufacturer#2' AND p_type='SMALL BRUSHED COPPER';
```

分别测试不创建多列统计信息和创建ABO统计信息场景下的执行计划：

```
benchmark=# explain analyze select * from part1 where p_container='LG CASE' AND p_brand='Brand#34'
AND p_mfgr='Manufacturer#2' AND p_type='SMALL BRUSHED COPPER';
  Bitmap Heap Scan on part1 (cost=5.30..336.06 rows=17 width=56) (actual time=0.953..7.061 rows=103
loops=1)
  Recheck Cond: (((p_brand)::text = 'Brand#34'::text) AND ((p_type)::text = 'SMALL BRUSHED COPPER'::text)
AND ((p_container)::text = 'LG CASE'::text))
  Filter: ((p_mfgr)::text = 'Manufacturer#2'::text)
  Rows Removed by Filter: 773
  Heap Blocks: exact=871
  ->  Bitmap Index Scan on brand_type_container  (cost=0.00..5.30 rows=84 width=0) (actual
time=0.704..0.704 rows=876 loops=1)
      Index Cond: (((p_brand)::text = 'Brand#34'::text) AND ((p_type)::text = 'SMALL BRUSHED COPPER'::text)
AND ((p_container)::text = 'LG CASE'::text))
Total runtime: 7.213 ms
benchmark=# explain analyze select * from part1 where p_container='LG CASE' AND p_brand='Brand#34'
AND p_mfgr='Manufacturer#2' AND p_type='SMALL BRUSHED COPPER';
  Bitmap Heap Scan on part1  (cost=10.59..723.97 rows=110 width=56) (actual time=0.112..0.434 rows=103
loops=1)
  Recheck Cond: (((p_type)::text = 'SMALL BRUSHED COPPER'::text) AND ((p_container)::text = 'LG
CASE'::text) AND ((p_mfgr)::text = 'Manufacturer#2'::text))
  Filter: ((p_brand)::text = 'Brand#34'::text)
  Rows Removed by Filter: 64
  Heap Blocks: exact=167
  ->  Bitmap Index Scan on type_container_mfgr  (cost=0.00..10.54 rows=183 width=0) (actual
time=0.081..0.081 rows=167 loops=1)
      Index Cond: (((p_type)::text = 'SMALL BRUSHED COPPER'::text) AND ((p_container)::text = 'LG
CASE'::text) AND ((p_mfgr)::text = 'Manufacturer#2'::text))
Total runtime: 0.533 ms
```

通过以上操作可以看出，在这个场景下，ABO基数估计加速了查询10+倍。

1.11.1.5 常见问题处理

如果遇到异常场景导致模型无法创建，ABO优化器会只创建传统统计信息，请根据对应的告警信息进行相应处理。

1.12 Foreign Data Wrapper

GaussDB的FDW（ Foreign Data Wrapper ）可以实现各个GaussDB数据库及远程服务器（包括数据库、文件系统）之间的跨库操作。目前支持的外部数据封装器类型包括file_fdw。

 版权所有 © 华为云计算技术有限公司

1.12.1 file_fdw

file_fdw模块提供了外部数据封装器file_fdw， 可以用来在服务器的文件系统中访问数据文件。数据文件必须是COPY FROM可读的格式，具体请参见《 开发者指南 》中“SQL参考 > SQL语法 > COPY”章节。使用file_fdw访问的数据文件是当前可读的，不支持对该数据文件的写入操作。

当前GaussDB会默认编译file_fdw，initdb的时候会在pg_catalog schema中创建该插件。

file_fdw对应的server和外表只允许数据库的初始用户、系统管理员或开启运维模式时的运维管理员创建。

使用file_fdw创建的外部表可以有下列选项：

- filename
  指定要读取的文件，必需的参数，且必须是一个绝对路径名。
- format
  远端server的文件格式，支持text/csv/binary三种格式，和COPY语句的FORMAT选项相同。
- header
  指定的文件是否有标题行，与COPY语句的HEADER选项相同。
- delimiter
  指定文件的分隔符，与COPY的DELIMITER选项相同。
- quote
  指定文件的引用字符，与COPY的QUOTE选项相同。
- escape
  指定文件的转义字符，与COPY的ESCAPE选项相同。
- null
  指定文件的null字符串，与COPY的NULL选项相同。
- encoding
  指定文件的编码，与COPY的ENCODING选项相同。
- force_not_null
  这是一个布尔选项。如果为真，则声明字段的值不应该匹配空字符串（也就是，文件级别null选项）。与COPY的FORCE_NOT_NULL选项里的字段相同。

说明

- file_fdw不支持COPY的OIDS和 FORCE_QUOTE选项。
- 这些选项只能为外部表或外部表的字段声明，不是file_fdw的选项，也不是使用file_fdw的服务器或用户映射的选项。
- 修改表级别的选项需要系统管理员权限。因为安全原因，只有系统管理员能够决定读取的文件。
- 对于一个使用file_fdw的外部表，EXPLAIN可显示要读取的文件名和文件大小（单位：字节）。指定了COSTS OFF关键字之后，不显示文件大小。

使用 file_fdw

- 创建服务器对象：CREATE SERVER

 版权所有 © 华为云计算技术有限公司

- 创建用户映射：CREATE USER MAPPING
- 删除用户映射：DROP USER MAPPING
- 删除服务器对象：DROP SERVER

注意事项

- 使用file_fdw需要指定要读取的文件，请先准备好该文件，并授予数据库对该文件的读取权限。
- 不支持DROP EXTENSION file_fdw操作。

1.13 GTM 模式

为适应不同的并发和一致性要求，GaussDB提供了三种不同的GTM模式，GTM、GTM-Lite、GTM-Free。三种模式之间的主要区别是中心事务管理节点GTM的压力和事务处理流程不同，其中GTM模式下，中心事务处理节点的压力最大，比较容易成为性能和并发瓶颈；在GTM-Lite模式下，中心事务处理节点的压力得到减轻，事务处理流程进一步优化，GTM的性能和并发瓶颈得到减轻，在保证一致性的情况下，事务处理能力得到更大限度的提升；在GTM-Free模式下，中心事务管理节点不再参与事务管理，消除了GTM单点瓶颈，可达到更高的事务处理性能，但是在一致性方面，支持所有事务运行完，保证读的外部一致性，不支持分布式事务强一致性读，不支持insert into select * from等依赖于查询结果的事务一致性。

当前版本暂不支持三种模式之间的相互切换，建议使用安装时默认的GTM模式，支持升级前后GTM模式不变化。

相关的GUC参数包括**enable_gtm_free**和**gtm_option**，可通过gsql执行show语句查询当前GTM模式：

```
SHOW enable_gtm_free;
SHOW gtm_option;
```

具体模式判断方法如下。

- GTM模式：enable_gtm_free=off 且 gtm_option=0；
- GTM-Lite模式：enable_gtm_free=off 且 gtm_option=1;
- GTM-Free模式：enable_gtm_free=on 或 gtm_option=2。

GTM-lite模式和GTM模式下，GaussDB支持分布式事务强一致和完整的语法功能。GTM-FREE模式下，由于采用了分布式事务最终一致的执行和并发控制机制，因此约束了部分语法的使用场景和使用方式。如果业务确有使用被限制语法的需求，那么在明确了解最终一致性行为的基础之上，需要对业务进行一定程度的改造。主要涉及的约束语法和改造建议如下：

1. 总体原则：所有用户表必须指定分布键（DISTRIBUTE BY），且选择合理：
   - 考虑数据分布均匀。
   - 尽量选择查询中的关联条件作为分布键，保证关联查询不会引起DN节点之间的数据流动。
   - 考虑将表的主键作为分布键。
2. SELECT：
   - 表查询时，WHERE条件应包含所有分布键字段等值查询条件。
   - 避免在SELECT目标列中使用子查询，可能导致计划无法下推到DN执行，影响执行性能。

版权所有 © 华为云计算技术有限公司

3. DML:

   默认不支持跨节点事务，如果所执行的DML语句包含跨节点事务，会报错处理，具体分为两种场景：

   a. 如果用户语句在数据库内部被拆分成多条独立语句执行，会报错：INSERT/UPDATE/DELETE/MERGE contains multiple remote queries under GTM-free mode Unsupport DML two phase commit under gtm free mode. modify your SQL to generate light-proxy or fast-query-shipping plan。这时，需要修改语句，来单节点执行。

      举例：

      insert into t select * from b where b.c = xx;

      假设t表和b表的分布键不同，且上述where条件只会过滤出一条数据。在不打开enable_stream_operator的情况下， 上面的查询在数据内部会被拆分成两条独立语句串行执行：首先执行select * from b where b.c = xx从某个DN节点抽取到目标记录；然后再执行insert into t语句，将抽取的目标记录下发到另一个节点DN节点完成插入。在gtm-free模式下，这样的语句执行方式会返回上述报错。类似的，create table as select * from、带子查询的delete/join/insert等语句，也可能会出现类似报错。

      业务改造方案：在业务执行之前，需要加上set enable_stream_operator=on命令，打开流算子，使得业务语句可以被整体下推执行。

   b. 如果同一个用户语句在数据库内部涉及多节点执行，会报错：Your SQL needs more than one datanode to be involved in。这时，建议对语句进行修改，使得能够单节点执行。

      举例：

      insert into t values(3,3),(1,1);

      假设(3,3)和(1,1)被分布在不同的DN节点上，那么上述语句在数据库内部的执行过程会涉及两个DN节点。在gtm-free模式下，这样的语句执行方式会返回上述报错。

      业务改造方式：对于上述语句，如果业务确有需要在多个节点上执行，需要在语句中添加一个hint来避免报错，如下：

      insert /*+ multinode */ into t values(3,3),(1,1);

      类似的，对delete和update语句也有类似约束，一般建议用户在delete和update语句的where条件中加上分布键等值过滤条件。

4. 建议开发阶段在jdbc连接串内设置application_type=perfect_sharding_type，这样所有跨节点读写操作的SQL都会报错，用来提示开发人员尽早优化语句。

1.14 敏感数据发现

敏感数据发现功能提供函数gs_sensitive_data_discovery()和gs_sensitive_data_discovery_detail()，通过调用的不同函数，指定扫描对象和敏感数据分类器，得到对应扫描对象不同明细级别的敏感数据信息。

使用敏感数据发现函数扫描数据

步骤1 创建扫描目标。

```
gaussdb=# CREATE SCHEMA test_schema1;
gaussdb=# CREATE TABLE test_schema1.normal_table1 (email VARCHAR(50), phonenum text, creditcard bytea, name BPCHAR(50), encryted_data NVARCHAR2(500));
gaussdb=#  INSERT INTO test_schema1.normal_table1 VALUES ('Gauss_1@huawei.com', '13345678901',
```

 版权所有 © 华为云计算技术有限公司

```
'4951-7843-2960-0886', '高斯',  'r1Pj4DKQopSmZfSr/mXwieGcrE+O2iX3G04w1Tda0zqX/pD/
Az90tLuvy0ZnCmUg0/Kl/g89ZqxxAZzta8jQ6bvicRpdXrDCQTHAfsXYSEScB9Idqtwh7kdm1LHq89qs')
```

步骤2 调用敏感数据发现函数扫描目标。

- 调用gs_sensitive_data_discovery()函数，扫描目标数据，返回统计的扫描结果。

```
gaussdb=# SELECT * FROM gs_sensitive_data_discovery('test_schema1.normal_table1',  'all');
           target           | percentage_of_sensitive_data |
classifier_type_distribution
----------------------------+------------------------------
+-----------------------------------------------------------------------------------------
 test_schema1.normal_table1 | 5/5                          |  Email: 1/5  CreditCard: 1/5  PhoneNumber: 1/5
ChineseName: 1/5  EncryptedContent: 1/5
(1 row)
```

- 调用gs_sensitive_data_discovery_detail()函数，扫描目标数据，返回详细的扫描结果。

```
   schema    |     table     |    column     |    classifier    | laws_and_regulations
-------------+---------------+---------------+------------------+----------------------
 test_schema1 | normal_table1 | email         | Email            | GDPR
 test_schema1 | normal_table1 | phonenum      | PhoneNumber      | GDPR
 test_schema1 | normal_table1 | creditcard    | CreditCard       | GDPR
 test_schema1 | normal_table1 | name          | ChineseName      | GDPR
 test_schema1 | normal_table1 | encryted_data | EncryptedContent | GDPR
(5 rows)
```

步骤3 清理数据。

```
gaussdb=# DROP TABLE test_schema1.normal_table1;
gaussdb=# DROP SCHEMA test_schema1;
(3 row)
```

----结束

 版权所有 © 华为云计算技术有限公司

2 主备版

2.1 物化视图

物化视图是一种特殊的物理表，物化视图是相对普通视图而言的。普通视图是虚拟表，应用的局限性较大，任何对视图的查询实际上都是转换为对SQL语句的查询，性能并没有实际提高。物化视图实际上就是存储SQL执行语句的结果，起到缓存的效果。物化视图常用的操作包括创建、查询、删除和刷新。

根据创建规则，物化视图分为全量物化视图和增量物化视图。全量物化视图只支持全量刷新；增量物化视图支持全量刷新和增量刷新两种方式。全量刷新会将基表中的数据全部重新刷入物化视图中，而增量刷新只会将两次刷新间隔期间的基表产生的增量数据刷入物化视图中。

目前Ustore引擎不支持创建、使用物化视图。

2.1.1 全量物化视图

2.1.1.1 概述

全量物化视图仅支持对已创建的物化视图进行全量刷新，而不支持进行增量刷新。创建全量物化视图语法和CREATE TABLE AS语法类似（详情请参见《开发者指南》中的“SQL参考 > SQL语法 > CREATE TABLE AS”章节）。

2.1.1.2 支持和约束

支持场景

- 通常全量物化视图所支持的查询范围与CREATE TABLE AS语句一致。
- 全量物化视图上支持创建索引。
- 支持analyze、explain。
- Ustore引擎不支持全量物化视图的创建和使用。

不支持场景

物化视图不支持增删改操作，只支持查询语句。

 版权所有 © 华为云计算技术有限公司

约束

全量物化视图的刷新、删除过程中会给基表加高级别锁，若物化视图的定义涉及多张表，需要注意业务逻辑，避免死锁产生。

2.1.1.3 使用

语法格式

- 创建全量物化视图
  ```
  CREATE MATERIALIZED VIEW view_name AS query;
  ```
- 刷新全量物化视图
  ```
  REFRESH MATERIALIZED VIEW view_name;
  ```
- 删除物化视图
  ```
  DROP MATERIALIZED VIEW view_name;
  ```
- 查询物化视图
  ```
  SELECT * FROM view_name;
  ```

参数说明

- **view_name**

  要创建的物化视图名。

  取值范围：字符串，要符合标识符的命名规范。

- **AS query**

  一个SELECT VALUES命令或者一个运行预备好的SELECT或VALUES查询的EXECUTE命令。

示例

```
-- 修改表的默认类型
gaussdb=#  set enable_default_ustore_table=off;

--准备数据。
gaussdb=# CREATE TABLE t1(c1 int, c2 int);
gaussdb=# INSERT INTO t1 VALUES(1, 1);
gaussdb=# INSERT INTO t1 VALUES(2, 2);

--创建全量物化视图。
gaussdb=# CREATE MATERIALIZED VIEW mv AS select count(*) from t1;
CREATE MATERIALIZED VIEW

--查询物化视图结果。
gaussdb=# SELECT * FROM mv;
 count
-------
     2
(1 row)

--向物化视图中基表插入数据。
gaussdb=# INSERT INTO t1 VALUES(3, 3);
INSERT 0 1

--对全量物化视图做全量刷新。
gaussdb=# REFRESH MATERIALIZED VIEW mv;
REFRESH MATERIALIZED VIEW

--查询物化视图结果。
gaussdb=# SELECT * FROM mv;
 count
```

 版权所有 © 华为云计算技术有限公司

```
-------
    3
(1 row)

--删除物化视图，删除表。
gaussdb=# DROP MATERIALIZED VIEW mv;
DROP MATERIALIZED VIEW
gaussdb=# DROP TABLE t1;
DROP TABLE
```

2.1.2 增量物化视图

2.1.2.1 概述

增量物化视图可以对物化视图增量刷新，需要用户手动执行语句，刷新物化视图在一段时间内的增量数据。与全量创建物化视图的不同在于目前增量物化视图所支持场景较小。目前物化视图创建语句仅支持基表扫描语句或者UNION ALL语句。

2.1.2.2 支持和约束

支持场景

- 单表查询语句。
- 多个单表查询的UNION ALL。
- 物化视图上支持创建索引。
- 物化视图支持Analyze操作。

不支持场景

- 物化视图中不支持多表Join连接计划以及subquery计划。
- 除少部分ALTER操作外，不支持对物化视图中基表执行绝大多数DDL操作。
- 物化视图不支持增删改操作，只支持查询语句。
- 不支持用临时表/hashbucket/unlog/分区表创建物化视图。
- 不支持物化视图嵌套创建（即物化视图上创建物化视图）。
- 不支持UNLOGGED类型的物化视图，不支持WITH语法。
- Ustore引擎不支持增量物化视图的创建和使用。

约束

- 物化视图定义如果为UNION ALL，则其中每个子查询需使用不同的基表。
- 增量物化视图的创建、全量刷新、删除过程中会给基表加高级别锁，若物化视图的定义为UNION ALL，需要注意业务逻辑，避免死锁产生。

2.1.2.3 使用

语法格式

- 创建增量物化视图
  ```
  CREATE INCREMENTAL MATERIALIZED VIEW view_name AS query;
  ```
- 全量刷新物化视图
  ```
  REFRESH MATERIALIZED VIEW view_name;
  ```

 版权所有 © 华为云计算技术有限公司

- 增量刷新物化视图
  ```
  REFRESH INCREMENTAL MATERIALIZED VIEW view_name;
  ```
- 删除物化视图
  ```
  DROP MATERIALIZED VIEW view_name;
  ```
- 查询物化视图
  ```
  SELECT * FROM view_name;
  ```

参数说明

- **view_name**

  要创建的物化视图名。

  取值范围：字符串，要符合标识符的命名规范。

- **AS query**

  一个SELECT VALUES命令或者一个运行预备好的SELECT或VALUES查询的EXECUTE命令。

示例

```
-- 修改表的默认类型
gaussdb=#  set enable_default_ustore_table=off;

--准备数据。
gaussdb=# CREATE TABLE t1(c1 int, c2 int);
gaussdb=# INSERT INTO t1 VALUES(1, 1);
gaussdb=# INSERT INTO t1 VALUES(2, 2);

--创建增量物化视图。
gaussdb=# CREATE INCREMENTAL MATERIALIZED VIEW mv AS SELECT * FROM t1;
CREATE MATERIALIZED VIEW

--插入数据。
gaussdb=# INSERT INTO t1 VALUES(3, 3);
INSERT 0 1

--增量刷新物化视图。
gaussdb=# REFRESH INCREMENTAL MATERIALIZED VIEW mv;
REFRESH MATERIALIZED VIEW

--查询物化视图结果。
gaussdb=# SELECT * FROM mv;
 c1 | c2
----+----
  1 |  1
  2 |  2
  3 |  3
(3 rows)

--插入数据。
gaussdb=# INSERT INTO t1 VALUES(4, 4);
INSERT 0 1

--全量刷新物化视图。
gaussdb=# REFRESH MATERIALIZED VIEW mv;
REFRESH MATERIALIZED VIEW

--查询物化视图结果。
gaussdb=# select * from mv;
 c1 | c2
----+----
  1 |  1
  2 |  2
  3 |  3
  4 |  4
```

 版权所有 © 华为云计算技术有限公司

```
(4 rows)

--删除物化视图，删除表。
gaussdb=# DROP MATERIALIZED VIEW mv;
DROP MATERIALIZED VIEW
gaussdb=# DROP TABLE t1;
DROP TABLE
```

2.2 设置密态等值查询

2.2.1 密态等值查询概述

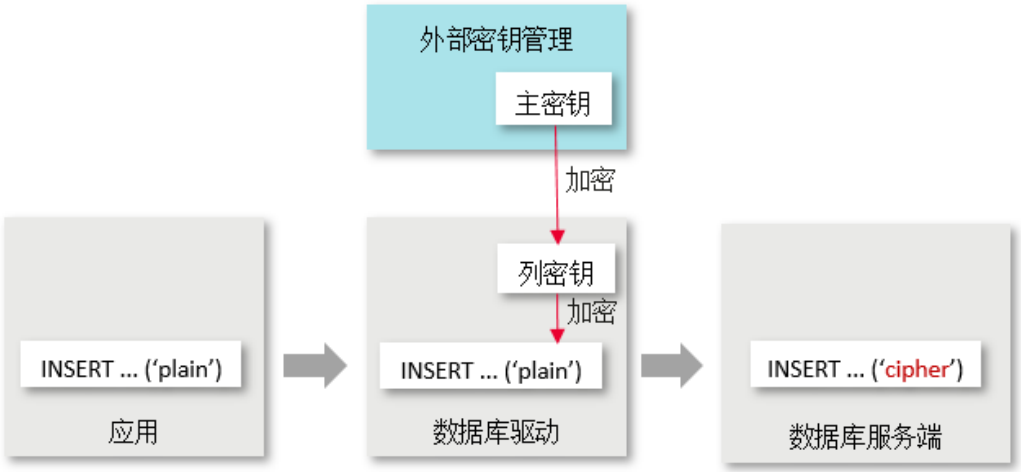
随着企业数据上云，数据的安全隐私保护面临越来越严重的挑战。密态数据库将解决数据整个生命周期中的隐私保护问题，涵盖网络传输、数据存储以及数据运行状态；更进一步，密态数据库可以实现云化场景下的数据隐私权限分离，即实现数据拥有者和实际数据管理者的数据读取能力分离。密态等值查询将优先解决密文数据的等值类查询问题。

加密模型

全密态数据库使用多级加密模型，加密模型中涉及3个对象：数据、列密钥和主密钥，以下是对3个对象的介绍：

- **数据**：
  a. 指SQL语法中包含的数据，比如INSERT...VALUES ('data')语法中包含'data'。
  b. 指从数据库服务端返回的查询结果，比如执行SELECT...语法返回的查询结果。

  密态数据库会在驱动中对SQL语法中属于加密列的数据进行加密，对数据库服务端返回的属于加密列的查询结果进行解密。
- **列密钥**：数据由列密钥进行加密，列密钥由数据库驱动生成或由用户手动导入，列密钥密文存储在数据库服务端。
- **主密钥**：列密钥由主密钥加密，主密钥由外部密钥管理者生成并存储。数据库驱动会自动访问外部密钥管理者，以实现对列密钥进行加解密。



版权所有 © 华为云计算技术有限公司

整体流程

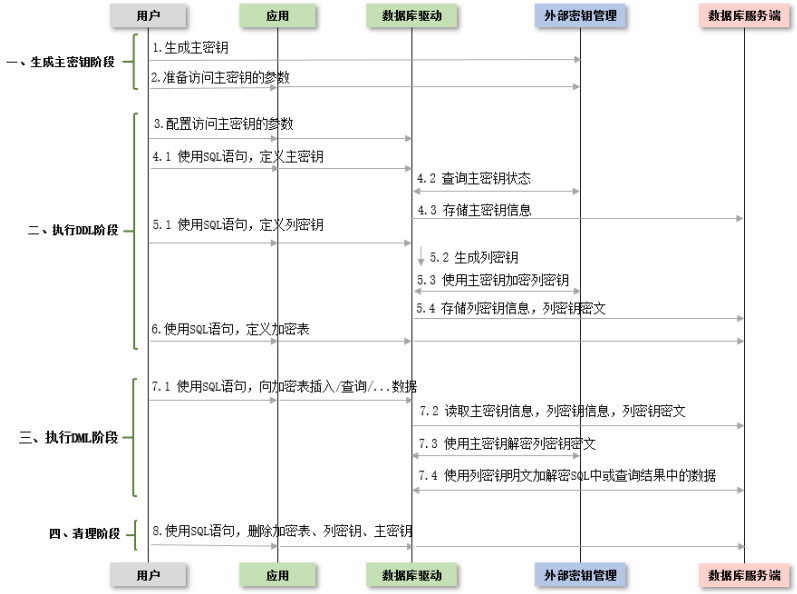
在使用全密态数据库的过程中，主要流程包括如下四个阶段，本节介绍整体流程，**使用gsql操作密态数据库**、**使用JDBC操作密态数据库**、**使用Go驱动操作密态数据库**章节介绍详细使用流程。

一、生成主密钥阶段：首先，用户需在外部密钥管理中生成主密钥。外部密钥管理分为多种，包括：华为云密钥服务，根据使用场景选择其中一种。生成主密钥后，需在外部密钥管理中，准备访问主密钥的参数，以供数据库使用。

二、执行DDL阶段：在本阶段，用户可使用密态数据库的密钥语法依次定义主密钥和列密钥，然后定义表并指定表中某列为加密列。定义主密钥和列密钥的过程中，需访问上一阶段生成的主密钥。

三、执行DML阶段：在创建加密表后，用户可直接执行包含但不限于INSERT、SELECT、UPDATE、DELETE等语法，数据库驱动会自动根据上一阶段的加密定义自动对加密列中的数据进行加解密。

四、清理阶段：依次删除加密表、列密钥和主密钥。

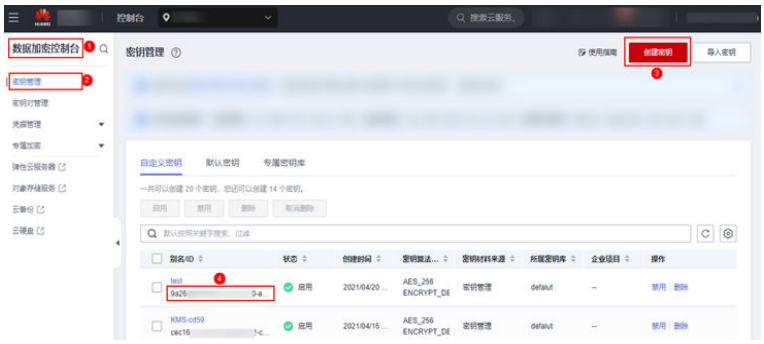


生成主密钥阶段

首次使用密态数据库时，需使用外部密钥管理服务生成至少一个主密钥，生成方式如下：

- **华为混合云场景**
  a. 进入密钥管理服务：登录华为云管控，搜索并进入"数据加密服务"。
  b. 创建主密钥：选择"密钥管理"功能，并通过"创建密钥"按钮创建至少1个密钥，即主密钥。

版权所有 © 华为云计算技术有限公司

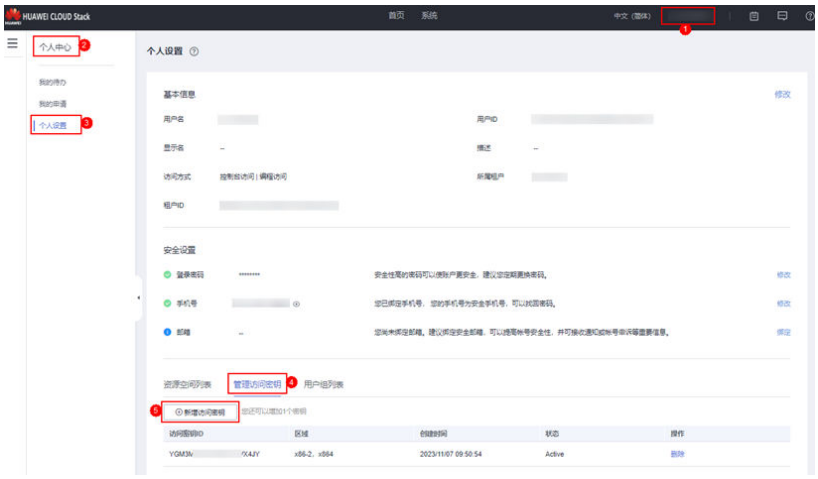


c. 记住主密钥ID：成功创建主密钥后，每个主密钥都有1个密钥ID。在后续使用密态数据过程中，需配置主密钥ID，数据库驱动会通过Restful接口访问该主密钥。

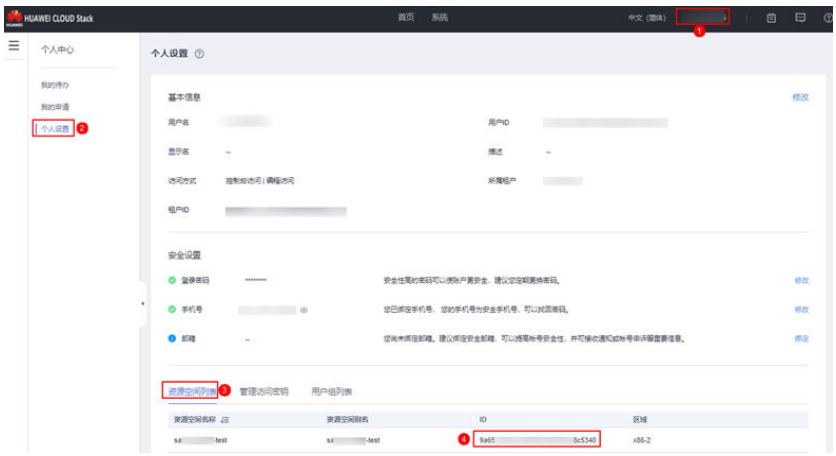
在生成主密钥后，需为数据驱动准备访问主密钥的参数，比如用户名、资源ID等参数。华为云支持两种身份认证方式，两种方式需要的参数个数与参数类型不同，选择其中一种方式即可。下述步骤介绍如何获取这些参数：

- **方式一 aksk认证**

  a. AK、SK：登录华为云管控，单击右上角"用户名"，进入"个人设置"页面。在"管理访问密钥"中，创建新的访问密钥，即AK和SK。初次创建访问密钥时，可下载密钥，下载后可在文件中分别找到AK和SK。



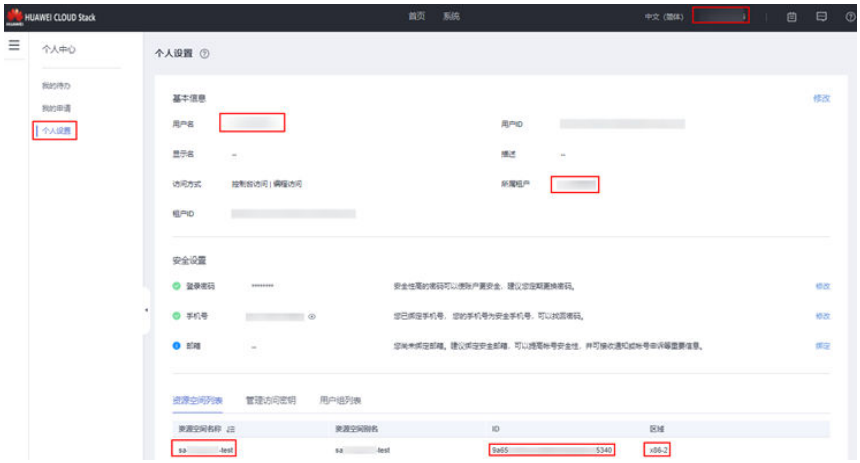
  b. 资源ID：进入"个人设置"页面，在"资源空间列表"中，可获取"资源ID"。



版权所有 © 华为云计算技术有限公司

- **方式二 账号密码认证**

  a. 用户名、所属租户、资源空间名称、资源空间ID、区域：登录华为云管控，单击右上角"用户名"，进入"个人设置"页面。进入"个人设置"页面。可依次获取所需参数。

  b. IAM服务器地址、KMS服务器地址：请自行根据部署方式查询。

  c. 用户密码：即用户名对应的密码。

2.2.2 使用 gsql 操作密态数据库

执行 SQL 语句

执行本节的SQL语句前，请确保已提前生成主密钥，并确认访问主密钥的参数。

本节以完整的执行流程为例，介绍如何使用密态数据库语法，包括三个阶段：使用DDL阶段、使用DML阶段、清理阶段。

步骤1 连接数据库，并通过-C参数开启全密态开关

```
gsql -p PORT -d DATABASE -h HOST -U USER -W PASSWORD -r -C
```

步骤2 通过元命令设置访问主密钥的参数

注意：从keyType字符串开始，不要添加换行，不要添加空格，否则gsql工具无法识别完整参数。

华为云支持两种认证方式，两种认证方式的参数个数与参数类型不同，选择其中一种方式即可。

- **认证方式一** aksk认证

  ```
  gaussdb=# \key_info keyType=hcs_kms,kmsProjectId={项目ID},ak={AK},sk={SK}
  ```

  参数获取：生成主密钥阶段介绍了如何获取相关参数：项目ID、AK、SK。

  示例：\key_info keyType=hcs_kms,kmsProjectId=0b59929e8100268a2f22c01429802728,ak=XMAUMJY******DFWLQW,sk=ga6rO8lx1Q4uB*********2gf80muIzUX

- **认证方式二** 账号密码认证

  ```
  gaussdb=# \key_info keyType=hcs_kms,iamUrl={IAM服务器地址},iamUser={用户名},iamPassword={用户密码},iamDomain={所属租户},kmsProject={项目名称}
  ```

  参数获取：生成主密钥阶段介绍了如何获取上述提到的5个参数。

版权所有 © 华为云计算技术有限公司

示例：\key_info keyType=hcs_kms,iamUrl=https://iam.xxx.com/v3/auth/tokens,iamUser=test_user,iamPassword=*********,iamDomain=test_account,kmsProject=xxx

步骤3 定义主密钥

在生成主密钥阶段，密钥服务已生成并存储主密钥，执行本语法只是将主密钥的相关信息存储在数据库中，方便以后访问。该语法详细格式参考：《开发者指南》中“SQL参考 > SQL语法 > CREATE CLIENT MASTER KEY”章节。

```
gaussdb=# CREATE CLIENT MASTER KEY cmk1 WITH ( KEY_STORE = hcs_kms , KEY_PATH = '{KMS服务器地址}/{密钥ID}', ALGORITHM = AES_256);
CREATE CLIENT MASTER KEY
```

- 参数获取：生成主密钥阶段介绍了如何获取如下参数：KMS服务器地址、密钥ID。

  KEY_PATH示例：https://kms-scc-apig.sa-fb-1.console-arm2.com/v1.0/0b59929e8100268a2f22c01429802728/kms/9a262917-8b31-41af-a1e0-a53235f32de9

步骤4 定义列密钥

列密钥由上一步定义的主密钥加密。详细语法参考：《开发者指南》中“SQL参考 > SQL语法 > CREATE COLUMN ENCRYPTION KEY”章节。

```
gaussdb=# CREATE COLUMN ENCRYPTION KEY cek1 WITH VALUES (CLIENT_MASTER_KEY = cmk1, ALGORITHM = AES_256_GCM);
```

步骤5 定义加密表

本示例中，通过语法指定表中name和credit_card为加密列。

```
gaussdb=# CREATE TABLE creditcard_info (
  id_number int,
  name text encrypted with (column_encryption_key = cek1, encryption_type = DETERMINISTIC),
  credit_card varchar(19) encrypted with (column_encryption_key = cek1, encryption_type = DETERMINISTIC));
NOTICE:  The 'DISTRIBUTE BY' clause is not specified. Using 'id_number' as the distribution column by default.
HINT:  Please use 'DISTRIBUTE BY' clause to specify suitable data distribution column.
CREATE TABLE
```

步骤6 对加密表进行其他操作

```
-- 向加密表写入数据。
gaussdb=# INSERT INTO creditcard_info VALUES (1,'joe','6217986500001288393');
INSERT 0 1
gaussdb=# INSERT INTO creditcard_info VALUES (2, 'joy','6219985678349800033');
INSERT 0 1

-- 从加密表中查询数据。
gaussdb=# select * from creditcard_info where name = 'joe';
 id_number | name |     credit_card
-----------+------+---------------------
         1 | joe  | 6217986500001288393

-- 更新加密表中数据。
gaussdb=# update creditcard_info set credit_card = '80000000011111111' where name = 'joy';
UPDATE 1

-- 向表中新增一列加密列。
gaussdb=# ALTER TABLE creditcard_info ADD COLUMN age int ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY = cek1, ENCRYPTION_TYPE = DETERMINISTIC);
ALTER TABLE

-- 从表中删除一列加密列。
gaussdb=# ALTER TABLE creditcard_info DROP COLUMN age;
ALTER TABLE
```

 版权所有 © 华为云计算技术有限公司

```
-- 从系统表中查询主密钥信息。
gaussdb=# SELECT * FROM gs_client_global_keys;
 global_key_name | key_namespace | key_owner | key_acl |        create_date
-----------------+---------------+-----------+---------+----------------------------
 cmk1            |          2200 |        10 |         | 2021-04-21 11:04:00.656617
(1 rows)

-- 从系统表中查询列密钥信息。
gaussdb=# SELECT column_key_name,column_key_distributed_id ,global_key_id,key_owner FROM
gs_column_keys;
 column_key_name | column_key_distributed_id | global_key_id | key_owner
-----------------+---------------------------+---------------+-----------
 cek1            |                 760411027 |         16392 |        10
(1 rows)

-- 查看表中列的元信息。
gaussdb=# \d creditcard_info
        Table "public.creditcard_info"
   Column    |       Type        | Modifiers
-------------+-------------------+-----------
 id_number   | integer           |
 name        | text              |  encrypted
 credit_card | character varying |  encrypted
```

步骤7 清理阶段

```
-- 删除加密表。
gaussdb=# DROP TABLE creditcard_info;
DROP TABLE

-- 删除列密钥。
gaussdb=# DROP COLUMN ENCRYPTION KEY cek1;
DROP COLUMN ENCRYPTION KEY

-- 删除主密钥。
gaussdb=# DROP CLIENT MASTER KEY cmk1;
DROP CLIENT MASTER KEY
```

----结束

2.2.3 使用 JDBC 操作密态数据库

配置 JDBC 驱动

1. 获取JDBC驱动包，JDBC驱动获取及使用可参考《 开发者指南 》中“应用程序开发教程 > 基于JDBC开发”章节。

   密态数据库支持的JDBC驱动包为gsjdbc4.jar、opengaussjdbc.jar、gscejdbc.jar。

   - gsjdbc4.jar：主类名为“org.postgresql.Driver”，数据库连接的url前缀为“jdbc:postgresql”。
   - opengaussjdbc.jar：主类名为“com.huawei.opengauss.jdbc.Driver”，数据库连接的url前缀为“jdbc:opengauss”。
   - gscejdbc.jar（目前仅支持EulerOS操作系统）：主类名为“com.huawei.gaussdb.jdbc.Driver”，数据库连接的url前缀为“jdbc:gaussdb”，密态场景推荐使用此驱动包。

2. 配置LD_LIBRARY_PATH

   密态场景使用JDBC驱动包时，需要先设置环境变量LD_LIBRARY_PATH。

   - 使用gscejdbc.jar驱动包时，gscejdbc.jar驱动包中密态数据库需要的依赖库会自动复制到该路径下，并在开启密态功能连接数据库的时候加载。
   - 使用opengaussjdbc.jar或gsjdbc4.jar时，需要同时解压包名为GaussDB-Kernel_数据库版本号_操作系统版本号_64bit_libpq.tar.gz的压缩包解压到指

 版权所有 © 华为云计算技术有限公司

定目录，并将lib文件夹所在目录路径，添加至LD_LIBRARY_PATH环境变量中。

> **⚠ 注意**
>
> 全密态场景使用JDBC驱动包时需要有System.loadLibrary权限，以及环境变量LD_LIBRARY_PATH中第一优先路径的文件读写权限，建议使用独立目录作为全密态依赖库的存放路径。若在执行的时候指定java.library.path，需要与LD_LIBRARY_PATH的第一优先路径保持一致。

使用gscejdbc.jar时，jvm加载class文件需要依赖系统的libstdc++库，若开启密态则gscejdbc.jar会自动复制密态数据库依赖的动态库（包括libstdc++库）到用户设定的LD_LIBRARY_PATH路径下。如果依赖库与现有系统库版本不匹配，则首次运行仅部署依赖库，再次调用后即可正常使用。

执行 SQL 语句

在执行本节的SQL语句之前，请确保已完成前两阶段：准备阶段、配置阶段。

本节以完整的执行流程为例，介绍如何使用密态数据库语法，包括三个阶段：使用DDL阶段、使用DML阶段、清理阶段。

JDBC开发中与非密态场景操作一致的部分请参考《开发者指南》中“应用程序开发教程 > 基于JDBC开发”章节。

- 密态数据库连接参数

  enable_ce：String类型。其中enable_ce=0表示不开启全密态开关，enable_ce=1表示支持密态等值查询基本能力，enable_ce=3表示在密态等值查询能力的基础上支持内存解密逃生通道。

```
// 以下用例以gscejdbc.jar驱动为例，如果使用其他驱动包，仅需修改驱动类名和数据库连接的url前缀。
// gsjdbc4.jar: 主类名为“org.postgresql.Driver”，数据库连接的url前缀为“jdbc:postgresql”。
// opengaussjdbc.jar：主类名为“com.huawei.opengauss.jdbc.Driver”，数据库连接的url前缀为“jdbc:opengauss”。
// gscejdbc.jar：主类名为“com.huawei.gaussdb.jdbc.Driver”，数据库连接的url前缀为“jdbc:gaussdb”

public static void main(String[] args) {
    // 驱动类。
    String driver = "com.huawei.gaussdb.jdbc.Driver";
    // 数据库连接描述符。enable_ce=1表示支持密态等值查询基本能力。
    String sourceURL = "jdbc:gaussdb://127.0.0.1:8000/postgres?enable_ce=1";
    // 在环境变量USER、PASSWORD分别配置用户名密码。
    String username = System.getenv("USER");
    String passwd = System.getenv("PASSWORD");
    Connection conn = null;
    try {
        // 加载驱动
        Class.forName(driver);
        // 创建连接
        conn = DriverManager.getConnection(sourceURL, username, passwd);
        System.out.println("Connection succeed!");
        // 创建语句对象
        Statement stmt = conn.createStatement();
       // 设置访问主密钥的参数
        // 此处介绍2种方式，选择其中1种方式即可：
        // 认证方式一 aksk认证（生成主密钥阶段介绍了如何获取相关参数：项目ID、AK、SK）

        connsetClientInfo("key_info", "keyType=hcs_kms, kmsProjectId={项目ID}, ak={AK}, sk={SK}");
        /* 示例:
            conn.setClientInfo("key_info",
```


版权所有 © 华为云计算技术有限公司

```
"keyType=hcs_kms,kmsProjectId=0b59929e8100268a2f22c01429802728," +
                "ak=XMAUMJY******DFWLQW, sk=ga6rO8lx1Q4uB*********2gf80muIzUX,");
        */
        // 认证方式二 账号密码认证（生成主密钥阶段介绍了如何获取相关参数：IAM服务器地址、IAM用户名、IAM用户密码、账号名、项目）
        conn.setClientInfo("key_info", "keyType=hcs_kms," +
            "iamUrl={IAM服务器地址}," +
            "iamUser={IAM用户名}," +
            "iamPassword={IAM用户密码}," +
            "iamDomain={账号名}," +
            "kmsProject={项目}");
        /* 示例：
        conn.setClientInfo("key_info", "keyType=hcs_kms," +
                "iamUrl=https://iam.xxx.com/v3/auth/tokens," +
                "iamUser=test," +
                "iamPassword=*********," +
                "iamDomain=test_account," +
                "kmsProject=xxx");
        */
        // 定义客户端主密钥：cmk1为主密钥名字，可自行取名

        // 生成主密钥阶段介绍了如何获取如下参数：KMS服务器地址、密钥ID

        int rc = stmt.executeUpdate("CREATE CLIENT MASTER KEY ImgCMK1 WITH ( KEY_STORE = hcs_kms , KEY_PATH = '{KMS服务器地址}/{密钥ID}', ALGORITHM = AES_256);");
        /*
            KEY_PATH示例：https://kms-scc-apig.sa-fb-1.console-arm2.com/v1.0/0b59929e8100268a2f22c01429802728/kms/9a262917-8b31-41af-a1e0-a53235f32de9
            解释：在生成主密钥阶段，密钥服务已生成并存储主密钥，执行本语法只是将主密钥的相关信息存储在数据库中，方便以后访问
            提示：KEY_PATH格式请参考：《开发者指南》中“SQL参考 > SQL语法 > CREATE CLIENT MASTER KEY”章节
        */
        // 定义列加密密钥
        int rc2 = stmt.executeUpdate("CREATE COLUMN ENCRYPTION KEY ImgCEK1 WITH VALUES (CLIENT_MASTER_KEY = ImgCMK1, ALGORITHM  = AES_256_GCM);");
        // 定义加密表
        int rc3 = stmt.executeUpdate("CREATE TABLE creditcard_info (id_number int, name varchar(50) encrypted with (column_encryption_key = ImgCEK1, encryption_type = DETERMINISTIC),credit_card varchar(19) encrypted with (column_encryption_key = ImgCEK1, encryption_type = DETERMINISTIC));");
        // 插入数据
        int rc4 = stmt.executeUpdate("INSERT INTO creditcard_info VALUES (1,'joe','6217986500001288393');");
        // 查询加密表
        ResultSet rs = null;
        rs = stmt.executeQuery("select * from creditcard_info where name = 'joe';");
        // 删除加密表
        int rc5 = stmt.executeUpdate("DROP TABLE IF EXISTS creditcard_info;");
        // 删除列加密密钥
        int rc6 = stmt.executeUpdate("DROP COLUMN ENCRYPTION KEY IF EXISTS ImgCEK1;");
        // 删除客户端主密钥
        int rc7 = stmt.executeUpdate("DROP CLIENT MASTER KEY IF EXISTS ImgCMK1;");
        // 关闭语句对象
        stmt.close();
        // 关闭连接
        conn.close();
    } catch (Exception e) {
        e.printStackTrace();
        return;
    }
}
```

 版权所有 © 华为云计算技术有限公司

说明

- 使用JDBC操作密态数据库时，一个数据库连接对象对应一个线程，否则，不同线程变更可能导致冲突。
- 使用JDBC操作密态数据库时，不同connection对密态配置数据有变更，由客户端调用isvalid方法保证connection能够持有变更后的密态配置数据，此时需要保证参数refreshClientEncryption为1(默认值为1)，在单客户端操作密态数据场景下，refreshClientEncryption参数可以设置为0。

调用 isValid 方法刷新缓存示例

```
// 创建连接conn1
Connection conn1 = DriverManager.getConnection("url","user","password");
// 在另外一个连接conn2中创建客户端主密钥
...
// conn1通过调用isValid刷新缓存，刷新conn1密钥缓存
try {
 if (!conn1.isValid(60)) {
  System.out.println("isValid Failed for connection 1");
 }
} catch (SQLException e) {
 e.printStackTrace();
       return null;
}
```

执行密态等值密文解密

数据库连接接口PgConnection类型新增解密接口，可以对全密态数据库的密态等值密文进行解密。解密后返回其明文值，通过schema.table.column找到密文对应的加密列并返回其原始数据类型。

表 2-1 新增 org.postgresql.jdbc.PgConnection 函数接口

方法名	返回值类型	支持JDBC 4
decryptData(String ciphertext, Integer len, String schema, String table, String column)	ClientLogicDecryptResult	Yes

参数说明：

- **ciphertext**
  需要解密的密文。
- **len**
  密文长度。当取值小于实际密文长度时，解密失败。
- **schema**
  加密列所属schema名称。
- **table**
  加密列所属table名称。
- **column**
  加密列所属column名称。

 版权所有 © 华为云计算技术有限公司

说明

下列场景可以解密成功，但不推荐：

- 密文长度入参比实际密文长。
- schema.table.column指向其他加密列，此时将返回被指向的加密列的原始数据类型。

表 2-2 新增 org.postgresql.jdbc.clientlogic.ClientLogicDecryptResult 函数接口

方法名	返回值类型	描述	支持JDBC4
isFailed()	Boolean	解密是否失败，若失败返回True，否则返回False。	Yes
getErrMsg()	String	获取错误信息。	Yes
getPlaintext()	String	获取解密后的明文。	Yes
getPlaintextSize()	Integer	获取解密后的明文长度。	Yes
getOriginalType()	String	获取加密列的原始数据类型。	Yes

```
// 通过非密态连接、逻辑解码等其他方式获得密文后，可使用该接口对密文进行解密
import org.postgresql.jdbc.PgConnection;
import org.postgresql.jdbc.clientlogic.ClientLogicDecryptResult;

// conn为密态连接
// 调用密态PgConnection的decryptData方法对密文进行解密，通过列名称定位到该密文的所属加密列，并返回其原始数据类型
ClientLogicDecryptResult decrypt_res = null;
decrypt_res = ((PgConnection)conn).decryptData(ciphertext, ciphertext.length(), schemaname_str,
        tablename_str, colname_str);
// 检查返回结果类解密成功与否，失败可获取报错信息，成功可获得明文及长度和原始数据类型
if (decrypt_res.isFailed()) {
    System.out.println(String.format("%s\n", decrypt_res.getErrMsg()));
} else {
    System.out.println(String.format("decrypted plaintext: %s size: %d type: %s\n", decrypt_res.getPlaintext(),
        decrypt_res.getPlaintextSize(), decrypt_res.getOriginalType()));
}
```

执行加密表的预编译 SQL 语句

```
// 调用Connection的prepareStatement方法创建预编译语句对象。
PreparedStatement pstmt = con.prepareStatement("INSERT INTO creditcard_info VALUES (?, ?, ?);");
// 调用PreparedStatement的setShort设置参数。
pstmt.setInt(1, 2);
pstmt.setString(2, "joy");
pstmt.setString(3, "6219985678349800033");
// 调用PreparedStatement的executeUpdate方法执行预编译SQL语句。
int rowcount = pstmt.executeUpdate();
// 调用PreparedStatement的close方法关闭预编译语句对象。
pstmt.close();
```

执行加密表的批处理操作

```
// 调用Connection的prepareStatement方法创建预编译语句对象。
Connection conn = DriverManager.getConnection("url","user","password");
```

 版权所有 © 华为云计算技术有限公司

```
PreparedStatement pstmt = conn.prepareStatement("INSERT INTO creditcard_info (id_number, name,
credit_card) VALUES (?,?,?)");
// 针对每条数据都要调用setShort设置参数，以及调用addBatch确认该条设置完毕。
int loopCount = 20;
 for (int i = 1; i < loopCount + 1; ++i) {
      pstmt.setInt(1, i);
      pstmt.setString(2, "Name " + i);
      pstmt.setString(3, "CreditCard " + i);
      // Add row to the batch.
      pstmt.addBatch();
}
// 调用PreparedStatement的executeBatch方法执行批处理。
int[] rowcount = pstmt.executeBatch();
// 调用PreparedStatement的close方法关闭预编译语句对象。
pstmt.close();
```

2.2.4 使用 Go 驱动操作密态数据库

执行本节的SQL语句前，请确保已提前生成主密钥，并确认访问主密钥的参数。

本节以完整的执行流程为例，介绍如何使用密态数据库语法，包括三个阶段：连接阶段、使用DDL阶段、使用DML阶段。

连接密态数据库

连接密态数据库需要使用Go驱动包openGauss-connector-go-pq，驱动包当前不支持在线导入，需要将解压缩后的Go驱动源码包放在本地工程，同时需要配置环境变量。具体Go驱动开发请参考《开发者指南》中“应用程序开发教程 > 基于Go驱动开发”章节。另外，需保证环境上已安装7.3以上版本的gcc。

Go驱动支持密态数据库相关操作，需要设置密态特性参数enable_ce，同时在编译时添加标签-tags=enable_ce，并解压包名为GaussDB-Kernel_数据库版本号_操作系统版本号_64bit_libpq.tar.gz的压缩包解压到指定目录，并将lib文件夹所在目录路径，添加至LD_LIBRARY_PATH环境变量中。密态操作示例如下：

```
// 以单ip:port为例，本示例以用户名和密码保存在环境变量中为例，运行本示例前请先在本地环境中设置环境变量(环境变量名称请根据自身情况进行设置)
func main() {
    // 设置访问主密钥的参数
    // 此处介绍2种方式，选择其中1种方式即可：
    // 认证方式一 aksk认证（生成主密钥阶段介绍了如何获取相关参数：项目ID、AK、SK）
    kmsarg := "key_info='keyType=hcs_kms, kmsProjectId={项目ID}, ak={AK}, sk={SK}'"
    // 示例： kmsarg :=
"key_info='keyType=hcs_kms,kmsProjectId=0b59929e8100268a2f22c01429802728,ak=XMAUMJY******DFWLQW, sk=ga6rO8lx1Q4uB********2gf80muIzUX'"
    // 认证方式二 账号密码认证（生成主密钥阶段介绍了如何获取相关参数：IAM服务器地址、IAM用户名、IAM用户密码、账号名、项目）
     kmsarg := "key_info='keyType=hcs_kms," +
           "iamUrl={IAM服务器地址}," +
           "iamUser={IAM用户名}," +
           "iamPassword={IAM用户密码}," +
           "iamDomain={账号名}," +
           "kmsProject={项目}'"
       /* 示例：
           kmsarg := "key_info='keyType=hcs_kms," +
               "iamUrl=https://iam.xxx.com/v3/auth/tokens," +
               "iamUser=test," +
               "iamPassword=*********," +
               "iamDomain=test_account," +
               "kmsProject=xxx'";
       */

    hostip := os.Getenv("GOHOSTIP")   //GOHOSTIP为写入环境变量的IP地址
    port := os.Getenv("GOPORT")       //GOPORT为写入环境变量的port
    usrname := os.Getenv("GOUSRNAME") //GOUSRNAME为写入环境变量的用户名
```

 版权所有 © 华为云计算技术有限公司

```
    passwd := os.Getenv("GOPASSWD")   //GOPASSWDW为写入环境变量的用户密码
    str := "host=" + hostip + " port=" + port + " user=" + usrname + " password=" + passwd + "
dbname=postgres enable_ce=1" + kmsarg // DSN连接串
    // str := "opengauss://" + usrname + ":" + passwd + "@" + hostip + ":" + port + "/postgres?enable_ce=1" +
kmsarg  // URL连接串

    // 获取数据库连接池句柄
    db, err:= sql.Open("opengauss", str)
    if err != nil {
        log.Fatal(err)
    }
    defer db.Close()

    // Open函数仅是验证参数，Ping方法可检查数据源是否合法
    err = db.Ping()
    if err == nil {
        fmt.Printf("Connection succeed!\n")
    } else {
        log.Fatal(err)
    }
}
```

执行密态等值查询相关的创建密钥语句

```
// 定义主密钥
// 生成主密钥阶段介绍了如何获取如下参数：KMS服务器地址、密钥ID
_, err = db.Exec("CREATE CLIENT MASTER KEY ImgCMK1 WITH ( KEY_STORE = hcs_kms , KEY_PATH = '{KMS
服务器地址}/{密钥ID}', ALGORITHM = AES_256);")
// KEY_PATH示例：https://kms-scc-apig.sa-fb-1.console-arm2.com/
v1.0/0b59929e8100268a2f22c01429802728/kms/9a262917-8b31-41af-a1e0-a53235f32de9
// 解释：在生成主密钥阶段，密钥服务已生成并存储主密钥，执行本语法只是将主密钥的相关信息存储在数据库
中，方便以后访问
// 提示：KEY_PATH格式请参考：《开发者指南》中“SQL参考 > SQL语法 > CREATE CLIENT MASTER KEY”章
节
// 定义列密钥
_, err = db.Exec("CREATE COLUMN ENCRYPTION KEY ImgCEK1 WITH VALUES (CLIENT_MASTER_KEY =
ImgCMK1, ALGORITHM = AEAD_AES_256_CBC_HMAC_SHA256);")
```

执行密态等值查询加密表相关的语句

```
// 定义加密表
_, err = db.Exec("CREATE TABLE creditcard_info (id_number int, name varchar(50) encrypted with
(column_encryption_key = ImgCEK1, encryption_type = DETERMINISTIC), credit_card varchar(19) encrypted
with (column_encryption_key = ImgCEK1, encryption_type = DETERMINISTIC));")
// 插入数据
_, err = db.Exec("INSERT INTO creditcard_info VALUES (1,'joe','6217986500001288393'),
(2,'mike','6217986500001722485'), (3,'joe','6315892300001244581');");
var var1 int
var var2 string
var var3 string
// 查询数据
rows, err := db.Query("select * from creditcard_info where name = 'joe';")
defer rows.Close()
// 逐行打印
for rows.Next() {
    err = rows.Scan(&var1, &var2, &var3)
    if err != nil {
        log.Fatal(err)
    } else {
        fmt.Printf("var1:%v, var2:%v, var3:%v\n", var1, var2, var3)
    }
}
```

执行加密表的预编译 SQL 语句

```
// 调用DB实例的Prepare方法创建预编译对象
delete_stmt, err := db.Prepare("delete from creditcard_info where name = $1;")
defer delete_stmt.Close()
```

 版权所有 © 华为云计算技术有限公司

```
// 调用预编译对象的Exec方法绑定参数并执行SQL语句
_, err = delete_stmt.Exec("mike")
```

执行加密表的 Copy In 操作

```
// 调用DB实例的Begin、Prepare方法创建事务对象、预编译对象
tx, err := db.Begin()
copy_stmt, err := tx.Prepare("Copy creditcard_info from stdin")
// 声明并初始化待导入数据
var records = []struct {
    field1 int
    field2 string
    field3 string
}{
    {4, "james", "6217986500001234567"},
    {
        field1: 5,
        field2: "john",
        field3: "6217986500007654321",
    },
}
// 调用预编译对象的Exec方法绑定参数并执行SQL语句
for _, record := range records {
    _, err = copy_stmt.Exec(record.field1, record.field2, record.field3)
    if err != nil {
        log.Fatal(err)
    }
}
// 调用事务对象的Commit方法完成事务提交
err = copy_stmt.Close()
err = tx.Commit()
```

说明

当前Go驱动Copy In语句存在强约束，仅能在事务中通过预编译方式执行。

2.2.5 前向兼容与安全增强

前向兼容

在上文中，支持通过key_info设置访问外部密钥管理的参数：

1. 使用gsql时，通过元命令\key_info xxx设置
2. 使用JDBC时，通过连接参数conn.setProperty(“key_info”，“xxx”)设置

为保持前向兼容，还支持通过环境变量等方式设置访问主密钥的参数。

注意

第一次配置使用密态数据库时，可忽略下述方法。如果以前使用下述方法配置密态数据库，建议改用’key_info’配置

使用系统级环境变量配置的方式如下：

```
export HUAWEI_KMS_INFO='iamUrl=https://iam.{项目}.myhuaweicloud.com/v3/auth/tokens,iamUser={IAM用户名},iamPassword={IAM用户密码},iamDomain={账号名},kmsProject={项目}'
# 该方法中操作系统日志可能会记录环境变量中的敏感信息，使用过程中注意及时清理。
```

还可通过标准库接口设置进程级环境变量，不同语言设置方法如下：

1. C/C++

```
setenv("HIS_KMS_INFO", "xxx");
```

 版权所有 © 华为云计算技术有限公司

2. GO
```
os.Setenv("HIS_KMS_INFO", "xxx");
```

外部密钥服务的身份验证

当数据库驱动访问华为云密钥管理服务时，为避免攻击者伪装为密钥服务，在数据库驱动与密钥服务建立https连接的过程中，可通过CA证书验证密钥服务器的合法性。为此，需提前配置CA证书，如果未配置，将不会验证密钥服务的身份。本节介绍如何下载与配置CA证书。

配置方法

在key_info参数的中，增加证书相关参数即可。

- 使用gsql时
```
gaussdb=# \key_info keyType=hcs_kms,iamUrl=https://iam.xxx.com/v3/auth/tokens,iamUser={IAM用户名},iamPassword={IAM用户密码},iamDomain={账号名},kmsProject={项目},iamCaCert=/路径/IAM的CA证书文件,kmsCaCert=/路径/KMS的CA证书文件

gaussdb=# \key_info keyType=hcs_kms,kmsProjectId={项目ID},ak={AK},sk={SK},kmsCaCert=/路径/KMS的CA证书文件
```
- 使用JDBC时
```
conn.setProperty("key_info", "keyType=hcs_kms," +
    "iamUrl=https://iam.xxx.com/v3/auth/tokens," +
    "iamUser={IAM用户名}," +
    "iamPassword={IAM用户密码}," +
    "iamDomain={账号名}," +
    "kmsProject={项目}," +
    "iamCaCert=/路径/IAM的CA证书文件," +
    "kmsCaCert=/路径/KMS的CA证书文件");

conn.setProperty("key_info", "keyType=hcs_kms, kmsProjectId={项目ID}, ak={AK}, sk={SK},
kmsCaCert=/路径/KMS的CA证书文件");
```

获取证书

大部分浏览器均会自动下载网站对应的CA证书，并提供证书导出功能。虽然，诸如**https://www.ssleye.com/ssltool/certs_down.html**等很多网站也提供自动下载CA证书的功能，但可能因本地环境中存在代理或网关，导致CA证书无法正常使用。所以，建议借助浏览器下载CA证书。下载方式如下：

⚠ 注意

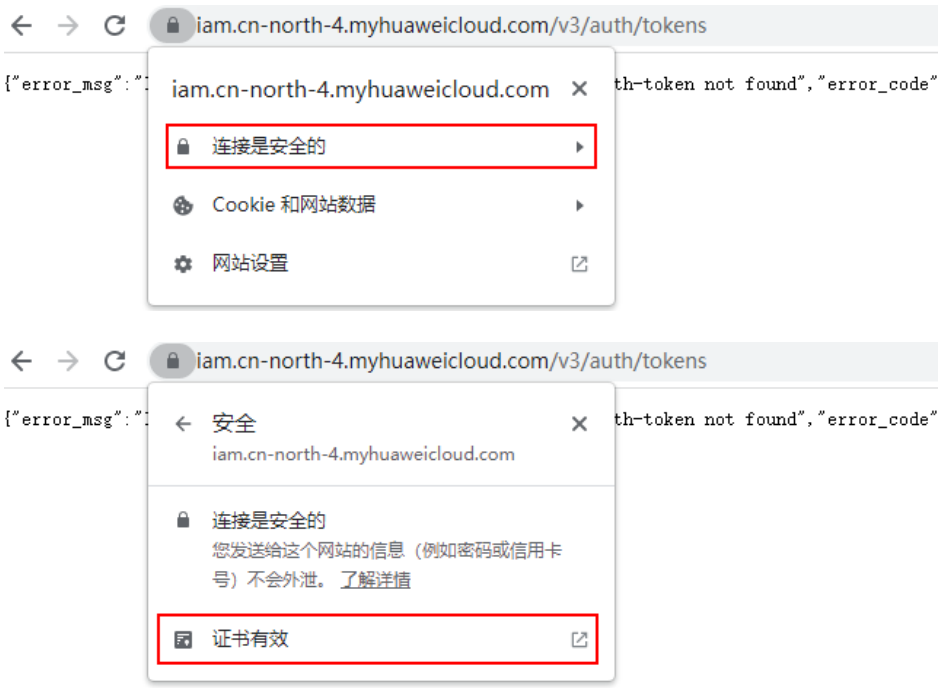
由于使用restful接口访问密钥服务，当在浏览器输入接口对应的url时，可忽略下述**步骤2**中的失败页面，因为即使在失败的情况下，浏览器也早已提前自动下载CA证书。

步骤1 输入域名：打开浏览器，在华为云场景中，分别输入IAM服务器地址和KMS服务器地址，地址获取方式参见：**生成主密钥阶段**。

步骤2 查找证书：在每次输入域名后，找到SSL连接相关信息，单击后会发现证书，继续单击可查看证书内容。

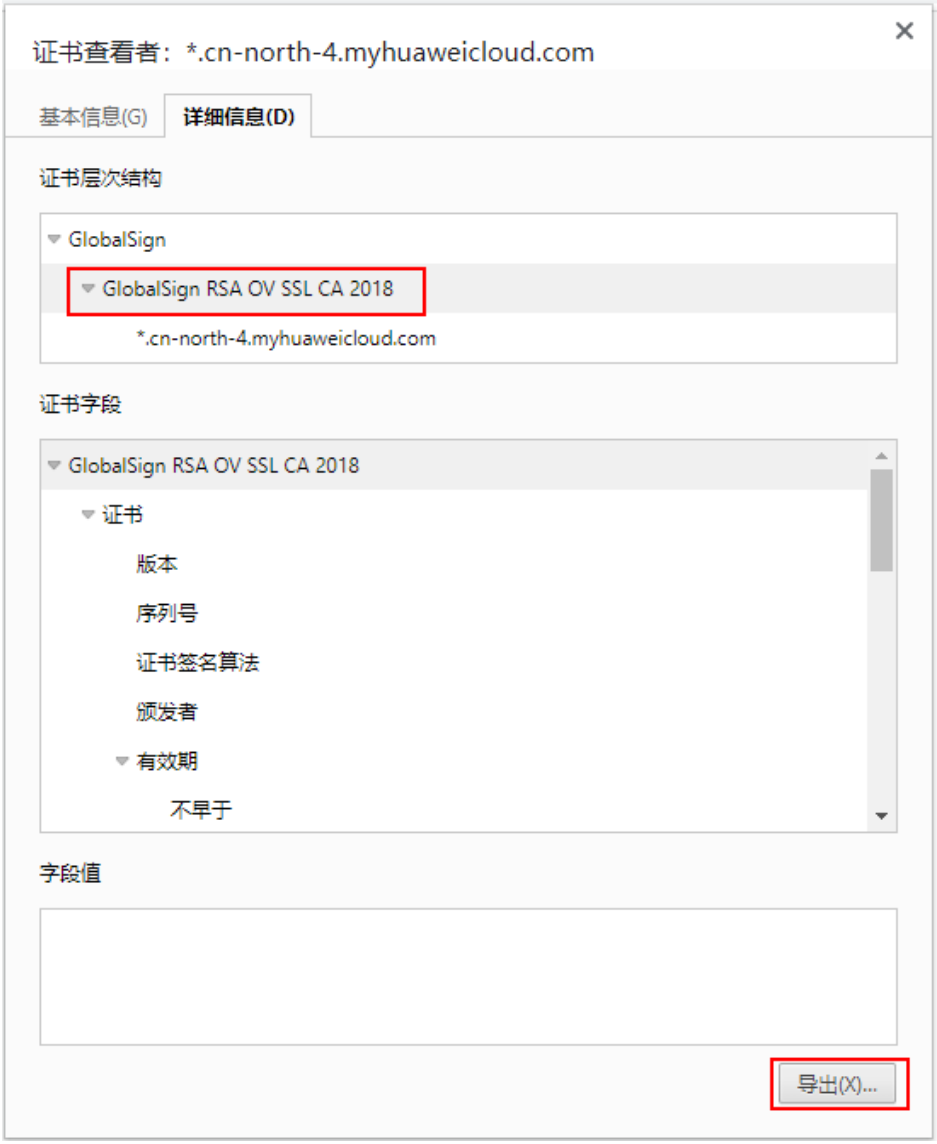


 版权所有 © 华为云计算技术有限公司



步骤3 导出证书：在证书查看页面，可能会看到证书分为很多级，仅需要域名的上一级证书即可，选择该证书并单击导出，便可直接生成证书文件，即需要的证书文件。

版权所有 © 华为云计算技术有限公司



步骤4 上传证书：将导出的证书上传至应用端，并配置到上述参数中即可。

----结束

2.2.6 密态支持函数/存储过程

密态支持函数/存储过程，当前版本只支持sql和plpgsql两种语言。 由于密态支持存储过程中创建和执行函数/存储过程对用户是无感知的，因此语法和非密态无区别。

函数/存储过程语法参考《 开发者指南 》中“用户自定义函数”章节和“存储过程”章节。

密态等值查询支持函数存储过程新增系统表gs_encrypted_proc，用于存储参数返回的原始数据类型。

系统表具体字段含义可参考《 开发者指南 》中“系统表和系统视图 > 系统表 > GS_ENCRYPTED_PROC”章节。

版权所有 © 华为云计算技术有限公司

创建并执行涉及加密列的函数/存储过程

步骤1 创建密钥，详细步骤请参见**使用gsql操作密态数据库**。

步骤2 创建加密表。

```
gaussdb=# CREATE TABLE creditcard_info (
  id_number int,
  name  text,
  credit_card varchar(19) encrypted with (column_encryption_key = cek1, encryption_type = DETERMINISTIC)
 ) with (orientation=row);
CREATE TABLE
```

步骤3 插入数据。

```
gaussdb=# insert into creditcard_info values(1, 'Avi', '1234567890123456');
INSERT 0 1
gaussdb=# insert into creditcard_info values(2, 'Eli', '2345678901234567');
INSERT 0 1
```

步骤4 创建函数支持密态等值查询。

```
gaussdb=# CREATE FUNCTION f_encrypt_in_sql(val1 text, val2 varchar(19)) RETURNS text AS 'SELECT name from creditcard_info where name=$1 or credit_card=$2 LIMIT 1' LANGUAGE SQL;
CREATE FUNCTION
gaussdb=# CREATE FUNCTION f_encrypt_in_plpgsql (val1 text, val2 varchar(19), OUT c text) AS $$
 BEGIN
 SELECT into c name from creditcard_info where name=$1 or credit_card =$2 LIMIT 1;
 END; $$
 LANGUAGE plpgsql;
CREATE FUNCTION
```

步骤5 执行函数。

```
gaussdb=# SELECT f_encrypt_in_sql('Avi','1234567890123456');
 f_encrypt_in_sql
------------------
 Avi
(1 row)

gaussdb=# SELECT f_encrypt_in_plpgsql('Avi', val2=>'1234567890123456');
 f_encrypt_in_plpgsql
----------------------
 Avi
(1 row)
```

----结束

 版权所有 © 华为云计算技术有限公司

说明

- 函数/存储过程中“执行动态查询语句”中的查询在执行过程中编译，因此函数/存储过程中的表名、列名不能在创建阶段未知，输入参数不能用于表名、列名或以任何方式连接。
- 函数/存储过程中的“执行动态查询语句”不支持EXECUTE 'query'中带有需要加密的数据值。
- 在RETURNS、IN和OUT的参数中，不支持混合使用加密和非加密类型参数。虽然参数类型都是原始数据类型，但实际类型不同。
- 在高级包接口中，如dbe_output.print_line()等在服务端打印输出的接口不会做解密操作，由于加密数据类型在强转成明文原始数据类型时会打印出该数据类型的默认值。
- 当前版本函数/存储过程的LANGUAGE只支持SQL和plpgsql，不支持C和JAVA等其他过程语言。
- 不支持在函数/存储过程中执行其他查询加密列的函数/存储过程。
- 当前版本不支持default、DECLARE中为变量赋予默认值，且不支持对DECLARE中的返回值进行解密，用户可以在执行函数时用输入参数、输出参数来代替使用。
- 不支持gs_dump对涉及加密列的function进行备份。
- 不支持在函数/存储过程中创建密钥。
- 该版本密态函数/存储过程不支持触发器
- 密态等值查询函数/存储过程不支持对plpgsql语言对语法进行转义，对于语法主体带有引号的语法CREATE FUNCTION AS ‘语法主体'，可以用CREATE FUNCTION AS $$语法主体$$代替。
- 不支持在密态等值查询函数/存储过程中执行修改加密列定义的操作，包括对创建加密表，添加加密列，由于执行函数是在服务端，客户端没法判断是否需要刷新缓存，需断开连接后或触发刷新客户端加密列缓存才可以对该列做加密操作。
- 密态函数/存储过程不支持编译检查。创建密态函数时，不能将behavior_compat_options设置为'allow_procedure_compile_check'。
- 不支持使用密态数据类型（byteawithoutorderwithequalcol、byteawithoutordercol、_byteawithoutorderwithequalcol、_byteawithoutordercol）创建函数和存储过程。
- 密态函数若返回值有加密类型，不支持返回不确定的行类型结果，如RETURN [SETOF] RECORD，可以使用返回可确定的行类型结果替代，如RETURN TABLE(columnname typename[,...])。
- 密态支持函数在创建加密函数时会在系统表gs_encrypted_proc中添加参数对应的加密列的OID，因此删除表后重建同名表可能会使密态函数失效，需要重新创建密态函数。

2.3 内存解密逃生通道

2.3.1 内存解密逃生通道概述

内存解密作为密态等值查询的一个逃生通道使用。在该逃生通道中，会将密钥传输到数据库内存中，对数据进行解密，以实现密文字段的特殊计算或查询功能，包括范围查询、排序。其他涉及密文操作、隐式或显式类型转换时，进行自动加解密。

须知

内存解密逃生通道，会将客户端密钥发送到数据库服务端内存缓存，实际的解密和数据明文运算都是在数据库内存中进行，断开连接或session中断后会对密钥和数据进行清理。在使用该特性前，请充分考虑该风险。

版权所有 © 华为云计算技术有限公司

2.3.2 使用 gsql 操作内存解密逃生通道

要开启全密态数据库的内存解密逃生通道功能，需要在使用gsql命令的时候添加-C3或--enable-client-encryption=3参数。

说明

使用gsql操作内存解密逃生通道全密态数据库需要完成密钥配置相关操作，请参见**使用gsql操作密态数据库**。

操作步骤

步骤1 执行以下命令打开密态开关，连接密态数据库。

```
gsql -p PORT -d gaussdb -h host -U user -W password -r -C3
```

步骤2 通过元命令设置访问主密钥的参数

注意：从keyType字符串开始，不要添加换行，不要添加空格，否则gsql工具无法识别完整参数。

华为云支持两种认证方式，两种认证方式的参数个数与参数类型不同，选择其中一种方式即可。

- **认证方式一** aksk认证

  ```
  gaussdb=# \key_info keyType=hcs_kms,kmsProjectId={项目ID},ak={AK},sk={SK}
  ```

  参数获取：生成主密钥阶段介绍了如何获取相关参数：项目ID、AK、SK。

  示例：\key_info keyType=hcs_kms,kmsProjectId=0b59929e8100268a2f22c01429802728,ak=XMAUMJY******DFWLQW,sk=ga6rO8lx1Q4uB*********2gf80muIzUX

- **认证方式二** 账号密码认证

  ```
  gaussdb=# \key_info keyType=hcs_kms,iamUrl={IAM服务器地址},iamUser={用户名},iamPassword={用户密码},iamDomain={所属租户},kmsProject={项目名称}
  ```

  参数获取：生成主密钥阶段介绍了如何获取上述提到的5个参数。

  示例：\key_info keyType=hcs_kms,iamUrl=https://iam.xxx.com/v3/auth/tokens,iamUser=test_user,iamPassword=*********,iamDomain=test_account,kmsProject=xxx

步骤3 定义主密钥

在生成主密钥阶段，密钥服务已生成并存储主密钥，执行本语法只是将主密钥的相关信息存储在数据库中，方便以后访问。该语法详细格式参考：《开发者指南》中“SQL参考 > SQL语法 > CREATE CLIENT MASTER KEY”章节。

```
gaussdb=# CREATE CLIENT MASTER KEY cmk1 WITH ( KEY_STORE = hcs_kms , KEY_PATH = '{KMS服务器地址}/{密钥ID}', ALGORITHM = AES_256);
CREATE CLIENT MASTER KEY
```

- 参数获取：生成主密钥阶段介绍了如何获取如下参数：KMS服务器地址、密钥ID。

  KEY_PATH示例：https://kms-scc-apig.sa-fb-1.console-arm2.com/v1.0/0b59929e8100268a2f22c01429802728/kms/9a262917-8b31-41af-a1e0-a53235f32de9

步骤4 定义列密钥

列密钥由上一步定义的主密钥加密。详细语法参考：《开发者指南》中“SQL参考 > SQL语法 > CREATE COLUMN ENCRYPTION KEY”章节。

版权所有 © 华为云计算技术有限公司

```
gaussdb=# CREATE COLUMN ENCRYPTION KEY cek1 WITH VALUES (CLIENT_MASTER_KEY = cmk1,
ALGORITHM = AES_256_GCM);
```

步骤5 创建加密表并向加密表插入数据。

```
gaussdb=# CREATE TABLE contacts (id int unique, credit float8 encrypted with (column_encryption_key =
cek1, encryption_type = DETERMINISTIC), name text encrypted with (column_encryption_key = cek1,
encryption_type = DETERMINISTIC));
NOTICE:  CREATE TABLE / UNIQUE will create implicit index "contacts_id_key" for table "contacts"
CREATE TABLE
gaussdb=# CREATE TABLE contacts_plain (id int unique, credit float8, name text);
NOTICE:  CREATE TABLE / UNIQUE will create implicit index "contacts_plain_id_key" for table
"contacts_plain"
CREATE TABLE
gaussdb=# INSERT INTO contacts VALUES (1, 8000, 'zhangsan');
INSERT 0 1
gaussdb=# INSERT INTO contacts VALUES (2, 7056.6, 'lisi');
INSERT 0 1
gaussdb=# INSERT INTO contacts VALUES (3, 16050, 'wangwu');
INSERT 0 1
```

步骤6 传输密钥。

```
gaussdb=# \st
Token cache enabled in Trusted Domain.
```

步骤7 对密文表进行范围查询。

```
gaussdb=# SELECT id,credit FROM contacts WHERE credit > 10000;
 id | credit
----+--------
  3 |  16050
(1 row)

gaussdb=# SELECT id,credit FROM contacts WHERE credit < 10000;
 id | credit
----+--------
  1 |   8000
  2 | 7056.6
(2 row)

gaussdb=# SELECT id,credit FROM contacts WHERE credit >= 8000;
 id | credit
----+--------
  1 |   8000
  3 |  16050
(2 row)

gaussdb=# SELECT id,credit FROM contacts WHERE credit <= 8000;
  id | credit
----+--------
  1 |   8000
  2 | 7056.6
(2 row)
```

步骤8 对密文表进行排序。

```
gaussdb=# SELECT id,credit FROM contacts ORDER BY credit;
  id | credit
----+--------
  2 | 7056.6
  1 |   8000
  3 |  16050
(3 row)

gaussdb=# SELECT id,credit FROM contacts ORDER BY credit DESC;
 id | credit
----+--------
  3 |  16050
  1 |   8000
  2 | 7056.6
(3 row)
```

 版权所有 © 华为云计算技术有限公司

步骤9 对密文进行计算算子操作。

```
gaussdb=# SELECT credit*2 FROM contacts LIMIT 1;
 ?column?
----------
    16000
(1 row)
```

步骤10 对密文进行聚合计算。

```
gaussdb=# SELECT sum(credit) FROM contacts;
   sum
---------
 31106.6
(1 row)
```

步骤11 对密文执行case语句操作。

```
gaussdb=# SELECT CASE WHEN credit > 9000 THEN name END FROM contacts;
  case
--------



 wangwu
(3 rows)
```

步骤12 对密文执行显式cast语句操作。

```
gaussdb=# SELECT credit::text, credit::int FROM contacts OFFSET 1 LIMIT 1;
 credit | credit
--------+--------
 7056.6 |   7057
(1 row)
```

步骤13 对密文执行like语句操作。

```
gaussdb=# SELECT credit FROM contacts WHERE name LIKE 'zhang%';
 credit
--------
   8000
(1 row)
```

步骤14 明文密文列互相迁移。

```
gaussdb=# INSERT INTO contacts_plain (id, name)
  SELECT id,
    ce_decrypt_deterministic(name,
      (SELECT column_key_distributed_id FROM gs_column_keys WHERE column_key_name='cek1'))
  FROM contacts;
INSERT 0 3
gaussdb=# DELETE FROM contacts;
DELETE 3
gaussdb=# INSERT INTO contacts (id, name)
  SELECT id,
    ce_encrypt_deterministic(name,
      (SELECT column_key_distributed_id FROM gs_column_keys WHERE column_key_name='cek1'))
  FROM contacts_plain;
INSERT 0 3
```

步骤15 删除密钥，清理环境。

```
gaussdb=# \ct
Token cache cleared in Trusted Domain.
gaussdb=# DROP TABLE contacts, contacts_plain;
DROP TABLE
gaussdb=# DROP COLUMN ENCRYPTION KEY cek1;
DROP COLUMN ENCRYPTION KEY
gaussdb=# DROP CLIENT MASTER KEY cmk1;
DROP CLIENT MASTER KEY
```

----结束

 版权所有 © 华为云计算技术有限公司

2.3.3 使用 JDBC 驱动操作内存解密逃生通道

⚠ 注意

- 服务端和客户端仅在GaussDB 503.1.0.SPC1200版本及以后版本支持JDBC操作内存解密逃生通道。
- 使用JDBC驱动操作内存解密逃生通道全密态数据库需要完成配置JDBC驱动和密钥配置等相关操作，详细请参见**使用JDBC操作密态数据库**。

全密态数据库开启内存解密逃生通道，JDBC连接需要设置enable_ ce=3参数。

```
public static Connection getConnect(String username, String passwd)
  {
     //驱动类。
     String driver = "org.postgresql.Driver";
     //数据库连接描述符。
     String sourceURL = "jdbc:postgresql://127.0.0.1:8000/postgres?enable_ce=3";
     Connection conn = null;

     try
     {
        //加载驱动。
        Class.forName(driver);
     }
     catch( Exception e )
     {
        e.printStackTrace();
        return null;
     }

     try
     {
         //创建连接。
        conn = DriverManager.getConnection(sourceURL, username, passwd);
        System.out.println("Connection succeed!");
     }
     catch(Exception e)
     {
        e.printStackTrace();
        return null;
     }

     return conn;
  };
```

设置完成后，可以进行**调用isValid方法刷新缓存**、**执行密态等值密文解密**、**执行加密表的预编译SQL语句**、**执行加密表的批处理操作**。

传输和销毁密钥示例

```
// 创建客户端连接。
Connection conn1 = DriverManager.getConnection("url","user","password");

// conn1通过调用setClientInfo传输密钥。
try {
 conn1.setClientInfo("send_token", null);
} catch (SQLException e) {
 System.out.println("Failed to cache token in Trusted Domain.\n");
}

// conn1通过调用setClientInfo销毁密钥。
try {
 conn1.setClientInfo("clear_token", null);
} catch (SQLException e) {
```


版权所有 © 华为云计算技术有限公司

```
 System.out.println("Failed to clear token in Trusted Domain.\n");
}
```

2.3.4 使用 Go 驱动操作内存解密逃生通道

说明

使用Go驱动操作内存解密逃生通道全密态数据库需要完成配置Go驱动和密钥配置等相关操作，详细请参见**使用Go驱动操作密态数据库**。

全密态数据库开启内存解密逃生通道，Go驱动连接串需要配置enable_ ce=3参数。

```
// 以单ip:port为例，本示例以用户名和密码保存在环境变量中为例，运行本示例前请先在本地环境中设置环境变量(环境变量名称请根据自身情况进行设置)
func main() {
    hostip := os.Getenv("GOHOSTIP")   //GOHOSTIP为写入环境变量的IP地址
    port := os.Getenv("GOPORT")       //GOPORT为写入环境变量的port
    usrname := os.Getenv("GOUSRNAME") //GOUSRNAME为写入环境变量的用户名
    passwd := os.Getenv("GOPASSWD")   //GOPASSWDW为写入环境变量的用户密码
    str := "host=" + hostip + " port=" + port + " user=" + usrname + " password=" + passwd + "
dbname=postgres enable_ce=3" // DSN连接串
    // str := "opengauss://" + usrname + ":" + passwd + "@" + hostip + ":" + port + "/postgres?
enable_ce=3"  // URL连接串

    // 获取数据库连接池句柄
    db, err := sql.Open("opengauss", str)
    if err != nil {
        log.Fatal(err)
    }
    defer db.Close()

    // Open函数仅是验证参数，Ping方法可检查数据源是否合法
    err = db.Ping()
    if err == nil {
        fmt.Printf("Connection succeed!\n")
    } else {
        log.Fatal(err)
    }
}
```

设置完成后，可以进行**执行密态等值查询相关的创建密钥语句**、**执行密态等值查询加密表相关的语句**、**执行加密表的预编译SQL语句**、**执行加密表的Copy In操作**。

传输和销毁密钥示例

```
hostip := os.Getenv("GOHOSTIP")   //GOHOSTIP为写入环境变量的IP地址
port := os.Getenv("GOPORT")       //GOPORT为写入环境变量的port
usrname := os.Getenv("GOUSRNAME") //GOUSRNAME为写入环境变量的用户名
passwd := os.Getenv("GOPASSWD")   //GOPASSWDW为写入环境变量的用户密码
// 配置自动传输密钥模式；此模式设置为yes，则将在连接初始化时自动将当前用户有权访问的所有列加密密钥发送给服务端，并在连接关闭时自动销毁
str := "host=" + hostip + " port=" + port + " user=" + usrname + " password=" + passwd + "
dbname=postgres enable_ce=3 auto_sendtoken=yes"
db, err := sql.Open("opengauss", str)
if err != nil {
   log.Fatal(err)
}
defer db.Close()

// 手动执行传输与销毁密钥命令
_, err = db.Exec("send_token")
if err != nil {
    log.Fatal(err)
}
_, err = db.Exec("clear_token")
if err != nil {
    log.Fatal(err)
}
```

 版权所有 © 华为云计算技术有限公司

2.3.5 使用 libpq_ce 操作内存解密逃生通道

libpq_ce.so为密态特性动态库，包含libpq.so的所有内容和密态客户端解析、加密的相关功能。使用libpq_ce操作内存解密逃生通道，需要设置连接参数enable_ce=3。

```
#include "libpq-fe.h"
int main()
{
    char *conninfo = "dbname=postgres port=8000 host='localhost' application_name=test user='testuser'
password=****** enable_ce=3";
    PGconn *conn = PQconnectdb(conninfo);
    if (PQstatus(conn) != CONNECTION_OK) {
        printf("conn err\n");
        return 0;
    }
    send_clear_token(conn);
    return 0;
}
```

传输和销毁密钥示例

创建好客户端加密主密钥（CMK）和数据加密密钥（CEK）后，可以通过PQenableTrustedDomain()传输和销毁密钥。

```
void send_clear_token(PGconn *conn)
{
    // 传输密钥，将该用户有权限使用的密钥传输到服务端
    if (PQenableTrustedDomain(conn, true)) {
        printf("send ok\n");
    } else {
        printf("%s\n", PQerrorMessage(conn));
        printf("send failed\n");
    }
    // 销毁密钥，清零传输到服务端的密钥
    if (PQenableTrustedDomain(conn, false)) {
        printf("clear ok\n");
    } else {
        printf("%s\n", PQerrorMessage(conn));
        printf("clear failed\n");
    }
    PQfinish(conn);
    return;
}
```

2.4 透明数据加密

透明加密提供表级数据加密存储功能。当用户使用本特性提供的语法创建加密表后，数据库向磁盘写入加密表数据前，会自动将其加密；同时，数据库从磁盘读取加密表数据后，会自动将其解密。向加密表中进行数据插入、更新、查询和删除等语法与非加密表一致。

说明

- 透明加密基本原理请参见《特性描述》的“数据库安全 > 透明数据加密”章节。
- 使用透明加密前需联系管理员配置开启该功能。

查看透明加密基本配置

步骤1 查看透明加密功能是否已开启。

enable_tde取值为on时表示开启，取值为off是表示关闭。该参数由管理员设置。

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# SHOW enable_tde;
 enable_tde
------------
 on
(1 row)
```

步骤2 查看是否已设置访问密钥管理服务的参数。

tde_key_info参数为空时表示未设置，tde_key_info不为空时表示已设置。该参数由管理员设置。

```
gaussdb=# show tde_key_info;
      tde_key_info
-------------------------
 keyType=...
```

----结束

操作加密表

步骤1 创建加密表。

创建表时，通过在WITH子句中设置enable_tde=on参数，即可设置该表为加密表。

数据库默认使用'AES_128_CTR'算法对加密表进行加密，如需使用其他算法，可通过encrypt_algo参数设置。

```
gaussdb=# CREATE TABLE t1 (c1 INT, c2 TEXT) WITH (enable_tde = on);
CREATE TABLE
gaussdb=# CREATE TABLE t2 (c1 INT, c2 TEXT) WITH (enable_tde = on, encrypt_algo = 'SM4_CTR');
CREATE TABLE
```

步骤2 查看加密表基本信息。

加密表基本信息存储在pg_class系统表中的reloptions字段中。其中，dek_cipher为数据密钥密文，由数据库自动生成，并由密钥管理服务加密。每个加密表都有1个独立的数据密钥。

```
gaussdb=# SELECT relname,reloptions FROM pg_class WHERE relname = 't1';
 relname | reloptions
---------
+-----------------------------------------------------------------------------------------------------------------------------
 t1      |
{orientation=row,enable_tde=on,encrypt_algo=AES_128_CTR,compression=no,storage_type=USTORE,key_type
=...,dek_cipher=...
```

步骤3 向加密表写入数据。

操作加密表与非加密表的语法一致。数据库将表中数据写入磁盘前，才会自动对加密表的数据进行加密。

```
gaussdb=# INSERT INTO t1 VALUES (1, 'tde plain 123');
INSERT 0 1
```

步骤4 从加密表查询数据。

对于合法用户而言，查询加密表与非加密表的语法一致，加解密操作由数据库自动实现。如果攻击者绕过数据库，直接读取磁盘上加密表对应的数据文件，会发现文件中的数据均已被加密。

```
gaussdb=# SELECT * FROM t1;
 c1 |      c2
----+---------------
  1 | tde plain 123
(1 row)
```

步骤5 轮转加密表的密钥。

 版权所有 © 华为云计算技术有限公司

为提高安全性，建议定期使用以下语法轮转加密表的数据密钥，即使用新的密钥对数据进行加密。

```
gaussdb=# ALTER TABLE t1 ENCRYPTION KEY ROTATION;
ALTER TABLE
```

轮转密钥后，数据库仍可以正常解密由旧密钥加密的数据。

步骤6 加密表与非加密表转换。

透明加密支持将加密表转换为非加密表，以及将非加密表转换为加密表。建议在每次转换后，手动执行VACUUM FULL *tablename*命令，以强制同步转换表中所有数据。

```
gaussdb=# CREATE TABLE t3 (c1 INT, c2 TEXT);
CREATE TABLE
gaussdb=# ALTER TABLE t3 SET (enable_tde = on);
ALTER TABLE
gaussdb=# VACUUM FULL t3;
VACUUM
gaussdb=# ALTER TABLE t3 SET (enable_tde = off);
ALTER TABLE
gaussdb=# VACUUM FULL t3;
VACUUM
```

步骤7 删除加密表。

```
gaussdb=# DROP TABLE IF EXISTS t1, t2, t3;
DROP TABLE
```

----结束

操作加密索引

步骤1 创建加密表。

创建索引的基表，需确保基表也是加密表。

```
gaussdb=# CREATE TABLE t1 (c1 INT, c2 TEXT) WITH (enable_tde = on);
CREATE TABLE
```

步骤2 创建加密索引。

与创建加密表的方式相同，通过在WITH子句中设置enable_tde=on参数，即将索引设置为加密索引。

索引与基表使用相同的加密算法和密钥，对基表进行密钥轮转时，索引也会使用新密钥。

```
gaussdb=# CREATE INDEX i1 ON t1(c2) WITH (enable_tde = on);
CREATE INDEX
```

步骤3 查看加密索引基本信息。

与加密表一样，索引基本信息也存储在pg_class系统表中的reloptions字段中，索引的dek_cipher、encrypt_algo等参数与基表保持一致。

```
gaussdb=# SELECT relname,reloptions FROM pg_class WHERE relname = 'i1';
 relname | reloptions
---------
+-----------------------------------------------------------------------------------------------------------------------
 i1      |
{orientation=row,enable_tde=on,encrypt_algo=AES_128_CTR,compression=no,storage_type=USTORE,key_type
=...,dek_cipher=...
```

步骤4 加密索引与非加密索引转换。

透明加密支持将非加密索引转换为加密索引，将加密索引转换为非加密索引。

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# CREATE TABLE t2 (c1 INT, c2 TEXT) WITH (enable_tde = on);
ALTER TABLE
gaussdb=# CREATE INDEX i2 ON t2(c2);
CREATE INDEX
gaussdb=# ALTER INDEX i2 SET (enable_tde = on);
ALTER INDEX
gaussdb=# ALTER INDEX i2 SET (enable_tde = off);
ALTER INDEX
```

步骤5 自动对索引进行加密。

默认情况下，主动设置enable_tde参数才可创建加密索引。当设置GUC参数tde_index_default_encrypt=on，且以加密表为基表创建索引时，数据库会自动将索引转换为加密索引。示例如下：

```
gaussdb=# CREATE TABLE t3 (c1 INT, c2 TEXT) WITH (enable_tde = on);
ALTER TABLE
gaussdb=# CREATE INDEX i3 ON t3(c2);
CREATE INDEX
gaussdb=# SELECT relname,reloptions FROM pg_class WHERE relname = 'i3';
 relname | reloptions
---------
+-----------------------------------------------------------------------------------------------------------------------
 i1      |
{orientation=row,enable_tde=on,encrypt_algo=AES_128_CTR,compression=no,storage_type=USTORE,key_type=...,dek_cipher=...

-- 解释：虽然未指定i3为加密索引，但是开启了tde_index_default_encrypt=on，且基表t3是加密表，数据库自动将i3转换为加密索引
```

步骤6 删除加密表和索引。

```
gaussdb=# DROP TABLE IF EXISTS t1, t2, t3;
DROP TABLE
```

----结束

2.5 设置账本数据库

2.5.1 账本数据库概述

背景信息

账本数据库融合了区块链思想，将用户操作记录至两种历史表：用户历史表和全局区块表中。当用户创建防篡改用户表时，系统将自动为该表添加一个hash列来保存每行数据的hash摘要信息，同时在blockchain模式下会创建一张用户历史表来记录对应用户表中每条数据的变更行为；而用户对防篡改用户表的每一次修改行为将被记录到全局区块表中。由于历史表具有只可追加不可修改的特点，因此历史表记录串联起来便形成了用户对防篡改用户表的修改历史。

用户历史表命名和结构如下：

表 2-3 用户历史表 blockchain.<schemaname>_<tablename>_hist 所包含的字段

字段名	类型	描述
rec_num	bigint	行级修改操作在历史表中的执行序号。

 版权所有 © 华为云计算技术有限公司

字段名	类型	描述
hash_ins	hash16	INSERT或UPDATE操作插入的数据行的hash值。
hash_del	hash16	DELETE或UPDATE操作删除数据行的hash值。
pre_hash	hash32	当前用户历史表的数据整体摘要。

表 2-4 hash_ins 与 hash_del 场景对应关系

-	hash_ins	hash_del
INSERT	(√) 插入行的hash值。	空
DELETE	空	(√) 删除行的hash值。
UPDATE	(√) 新插入数据的hash值。	(√) 删除前该行的hash值。

操作步骤

步骤1 创建防篡改模式。

例如，创建防篡改模式ledgernsp。

```
gaussdb=# CREATE SCHEMA ledgernsp WITH BLOCKCHAIN;
```

说明

如果需要创建防篡改模式或更改普通模式为防篡改模式，则需设置enable_ledger参数为on。enable_ledger默认参数为off。

步骤2 在防篡改模式下创建防篡改用户表。

例如，创建防篡改用户表ledgernsp.usertable。

```
gaussdb=# CREATE TABLE ledgernsp.usertable(id int, name text);
```

查看防篡改用户表结构及其对应的用户历史表结构。

```
gaussdb=# \d+ ledgernsp.usertable;
gaussdb=# \d+ blockchain.ledgernsp_usertable_hist;
```

执行结果如下：

```
gaussdb=# \d+ ledgernsp.usertable;
                    Table "ledgernsp.usertable"
 Column     |  Type   | Modifiers | Storage  | Stats target | Description
--------+---------+-----------+----------+--------------+-------------
 id         | integer |           | plain    |              |
 name       | text    |           | extended |              |
 hash_1d2d14   | hash16  |           | plain    |              |
Has OIDs: no
Options: orientation=row, compression=no
History table name: ledgernsp_usertable_hist
```


版权所有 © 华为云计算技术有限公司

```
gaussdb=# \d+ blockchain.ledgernsp_usertable_hist;
            Table "blockchain.ledgernsp_usertable_hist"
  Column  |  Type  | Modifiers | Storage | Stats target | Description
----------+--------+-----------+---------+--------------+-------------
 rec_num  | bigint |           | plain   |              |
 hash_ins | hash16 |           | plain   |              |
 hash_del | hash16 |           | plain   |              |
 pre_hash | hash32 |           | plain   |              |
Indexes:
    "gs_hist_16388_index" PRIMARY KEY, btree (rec_num int4_ops) TABLESPACE pg_default
Has OIDs: no
Options: internal_mask=263
```

说明

防篡改表在创建时会自动增加一个名为hash的系统列，所以防篡改表单表最大列数为1599。

步骤3 修改防篡改用户表数据。

例如，对防篡改用户表执行INSERT、UPDATE、DELETE操作。

```
gaussdb=# INSERT INTO ledgernsp.usertable VALUES(1, 'alex'), (2, 'bob'), (3, 'peter');
INSERT 0 3
gaussdb=# SELECT *, hash_1d2d14 FROM ledgernsp.usertable ORDER BY id;
 id | name  |       hash_1d2d14
----+-------+------------------
  1 | alex  | 1f2e543c580cb8c5
  2 | bob   | 8fcd74a8a6a4b484
  3 | peter | f51b4b1b12d0354b
(3 rows)

gaussdb=# UPDATE ledgernsp.usertable SET name = 'bob2' WHERE id = 2;
UPDATE 1
gaussdb=# SELECT *, hash_1d2d14 FROM ledgernsp.usertable ORDER BY id;
 id | name  |       hash_1d2d14
----+-------+------------------
  1 | alex  | 1f2e543c580cb8c5
  2 | bob2  | 437761affbb7c605
  3 | peter | f51b4b1b12d0354b
(3 rows)

gaussdb=# DELETE FROM ledgernsp.usertable WHERE id = 3;
DELETE 1
gaussdb=# SELECT *, hash_1d2d14 FROM ledgernsp.usertable ORDER BY id;
 id | name |       hash_1d2d14
----+------+------------------
  1 | alex | 1f2e543c580cb8c5
  2 | bob2 | 437761affbb7c605
(2 rows)
```

步骤4 删除表和模式。

若要执行其他账本数据库章节的示例，请在执行完之后再执行当前步骤，否则请直接执行当前步骤。

```
gaussdb=# DROP TABLE ledgernsp.usertable;
DROP TABLE
gaussdb=# DROP SCHEMA ledgernsp;
DROP SCHEMA
```

----结束

 版权所有 © 华为云计算技术有限公司

2.5.2 查看账本历史操作记录

前提条件

- 系统中需要有审计管理员或者具有审计管理员权限的角色。
- 数据库正常运行，并且对防篡改数据库执行了一系列增、删、改等操作，保证在查询时段内有账本操作记录结果产生。

背景信息

- 只有拥有AUDITADMIN属性的用户才可以查看账本历史操作记录。有关数据库用户及创建用户的办法请参见《开发者指南》中“数据库安全 > 用户及权限 > 用户”章节。
- 查询全局区块表命令是直接查询gs_global_chain表，操作为：

  ```
  SELECT * FROM gs_global_chain;
  ```

  该表有10个字段，每个字段的含义请参见《开发者指南》中“系统表和系统视图 > 系统表 > GS_GLOBAL_CHAIN”章节。
- 查询用户历史表的命令是直接查询BLOCKCHAIN模式下的用户历史表，操作为：

  例如用户表所在的模式为ledgernsp，表名为usertable，则对应的用户历史表名为blockchain.ledgernsp_usertable_hist：

  ```
  SELECT * FROM blockchain.ledgernsp_usertable_hist;
  ```

  用户历史表有4个字段，每个字段的含义请参见**表2-3**。

  说明

  用户历史表的表名一般为blockchain.<schemaname>_<tablename>_hist形式。当防篡改用户表模式名或者表名过长导致前述方式生成的表名超出表名长度限制，则会采用blockchain.<schema_oid>_<table_oid>_hist的方式命名。

操作步骤

步骤1 连接数据库，具体操作请参考《开发者指南》中“使用数据库入门 > 连接数据库 > 使用gsql连接”章节。

步骤2 查询全局区块表记录。

```
gaussdb=# SELECT * FROM gs_global_chain;
```

查询结果如下：

```
 blocknum |  dbname  | username |          starttime          | relid |  relnsp   |  relname  |     relhash
|           globalhash           |
              txcommand
----------+----------+----------+-----------------------------+-------+-----------+-----------+------------------
+--------------------------------+------------------
-------------------------------------------------------------
       0 | testdb | omm      | 2021-04-14 07:00:46.32757+08  | 16393 | ledgernsp | usertable |
a41714001181a294 | 6b5624e039e8aee36bff3e8295c75b40 | insert into ledge
rnsp.usertable values(1, 'alex'), (2, 'bob'), (3, 'peter');
       1 | testdb | omm      | 2021-04-14 07:01:19.767799+08 | 16393 | ledgernsp | usertable |
b3a9ed0755131181 | 328b48c4370faed930937869783c23e0 | update ledgernsp.
usertable set name = 'bob2' where id = 2;
       2 | testdb | omm      | 2021-04-14 07:01:29.896148+08 | 16393 | ledgernsp | usertable |
0ae4b4e4ed2fcab5 | aa8f0a236357cac4e5bc1648a739f2ef | delete from ledge
rnsp.usertable where id = 3;
```

该结果表明，用户omm连续执行了三条DML命令，包括INSERT、UPDATE和DELETE操作。

版权所有 © 华为云计算技术有限公司

步骤3 查询历史表记录。

```
gaussdb=# SELECT * FROM blockchain.ledgernsp_usertable_hist;
```

查询结果如下：

```
 rec_num |     hash_ins     |     hash_del     |             pre_hash
---------+------------------+------------------+----------------------------------
       0 | 1f2e543c580cb8c5 |                  | e1b664970d925d09caa295abd38d9b35
       1 | 8fcd74a8a6a4b484 |                  | dad3ed8939a141bf3682043891776b67
       2 | f51b4b1b12d0354b |                  | 53eb887fc7c4302402343c8914e43c69
       3 | 437761affbb7c605 | 8fcd74a8a6a4b484 | c2868c5b49550801d0dbbbaa77a83a10
       4 |                  | f51b4b1b12d0354b | 9c512619f6ffef38c098477933499fe3
(5 rows)
```

查询结果显示，用户omm对ledgernsp.usertable表插入了3条数据，更新了1条数据，随后删除了1行数据，最后剩余2行数据，hash值分别为1f2e543c580cb8c5和437761affbb7c605。

步骤4 查询用户表数据及校验列。

```
gaussdb=# SELECT *, hash_1d2d14 FROM ledgernsp.usertable;
```

查询结果如下：

```
 id | name |       hash_1d2d14
----+------+------------------
  1 | alex | 1f2e543c580cb8c5
  2 | bob2 | 437761affbb7c605
(2 rows)
```

查询结果显示，用户表中剩余2条数据，与4中的记录一致。

----结束

2.5.3 校验账本数据一致性

前提条件

数据库正常运行，并且对防篡改数据库执行了一系列增、删、改等操作，保证在查询时段内有账本操作记录结果产生。

背景信息

- 账本数据库校验功能目前提供两种校验接口，分别为：ledger_hist_check(text, text)和ledger_gchain_check(text, text)。普通用户调用校验接口，仅能校验自己有权限访问的表。
- 校验防篡改用户表和用户历史表的接口为pg_catalog.ledger_hist_check，操作为：

  ```
  SELECT pg_catalog.ledger_hist_check(schema_name text,table_name text);
  ```

  如果校验通过，函数返回t，反之则提示失败原因并返回f。
- 校验防篡改用户表、用户历史表和全局区块表三者是否一致的接口为pg_catalog.ledger_gchain_check，操作为：

  ```
  SELECT pg_catalog.ledger_gchain_check(schema_name text, table_name text);
  ```

  如果校验通过，函数返回t，反之则提示失败原因并返回f。

操作步骤

步骤1 校验防篡改用户表ledgernsp.usertable与其对应的历史表是否一致。

```
gaussdb=# SELECT pg_catalog.ledger_hist_check('ledgernsp', 'usertable');
```

 版权所有 © 华为云计算技术有限公司

查询结果如下：

```
 ledger_hist_check
-------------------
 t
(1 row)
```

该结果表明防篡改用户表和用户历史表中记录的结果能够一一对应，保持一致。

步骤2 查询防篡改用户表ledgernsp.usertable与其对应的历史表以及全局区块表中关于该表的记录是否一致。

```
gaussdb=# SELECT pg_catalog.ledger_gchain_check('ledgernsp', 'usertable');
```

查询结果如下：

```
 ledger_gchain_check
---------------------
 t
(1 row)
```

查询结果显示，上述三表中关于ledgernsp.usertable的记录保持一致，未发生篡改行为。

----结束

2.5.4 归档账本数据库

前提条件

- 系统中需要有审计管理员或者具有审计管理员权限的角色。
- 数据库正常运行，并且对防篡改数据库执行了一系列增、删、改等操作，保证在查询时段内有账本操作记录结果产生。
- 数据库已经正确配置审计文件的存储路径audit_directory。

背景信息

- 账本数据库归档功能目前提供两种校验接口，分别为：ledger_hist_archive(text, text)和ledger_gchain_archive(text, text)。账本数据库接口仅审计管理员可以调用。
- 归档用户历史表的接口为pg_catalog.ledger_hist_archive，操作为：

  ```
  SELECT pg_catalog.ledger_hist_archive(schema_name text,table_name text);
  ```

  如果归档成功，函数返回t，反之则提示失败原因并返回f。
- 归档全局区块表的接口为pg_catalog.ledger_gchain_archive，操作为：

  ```
  SELECT pg_catalog.ledger_gchain_archive();
  ```

  如果归档成功，函数返回t，反之则提示失败原因并返回f。

操作步骤

步骤1 对指定用户历史表进行归档操作。

```
gaussdb=# SELECT pg_catalog.ledger_hist_archive('ledgernsp', 'usertable');
```

执行结果如下：

```
 ledger_hist_archive
---------------------
 t
(1 row)
```

用户历史表将归档为一条数据：

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# SELECT * FROM blockchain.ledgernsp_usertable_hist;
 rec_num |     hash_ins     |     hash_del     |             pre_hash
---------+------------------+------------------+----------------------------------
       3 | e78e75b00d396899 | 8fcd74a8a6a4b484 | fd61cb772033da297d10c4e658e898d7
(1 row)
```

该结果表明当前节点用户历史表导出成功。

步骤2 执行全局区块表导出操作。

```
gaussdb=# SELECT pg_catalog.ledger_gchain_archive();
```

执行结果如下：

```
 ledger_gchain_archive
-----------------------
 t
(1 row)
```

全局历史表将以用户表为单位归档为N（用户表数量）条数据：

```
gaussdb=# SELECT * FROM gs_global_chain;
 blocknum |  dbname  | username |           starttime           | relid |  relnsp   |  relname  |     relhash
|            globalhash            | txcommand
----------+----------+----------+-------------------------------+-------+-----------+-----------+------------------
+----------------------------------+-----------
        1 | testdb | libc     | 2021-05-10 19:59:38.619472+08 | 16388 | ledgernsp | usertable |
57c101076694b415 | be82f98ee68b2bc4e375f69209345406 | Archived.
(1 row)
```

该结果表明，当前节点全局区块表导出成功。

----结束

2.5.5 修复账本数据库

前提条件

- 系统中需要有审计管理员或者具有审计管理员权限的角色。
- 数据库正常运行，并且对防篡改数据库执行了一系列增、删、改等操作，保证在查询时段内有账本操作记录结果产生。

背景信息

- 当在异常情况或表被损坏时，需要使用ledger_gchain_repair(text, text)接口对全局区块表进行修复，或使用ledger_hist_repair(text, text)接口对用户历史表进行修复，修复后调用全局区块表或用户历史表校验接口结果为true。
- 修复用户历史表的接口为pg_catalog.ledger_hist_repair，操作为：

  ```
  SELECT pg_catalog.ledger_hist_repair(schema_name text,table_name text);
  ```

  如果修复成功，函数返回修复过程中用户历史表hash的增量。

  注：对用户表执行闪回DROP时，可使用该函数恢复用户表和用户历史表名称，请参见**恢复用户表和用户历史表名称**。
- 修复全局区块表的接口为pg_catalog.ledger_gchain_repair，操作为：

  ```
  SELECT pg_catalog.ledger_gchain_repair(schema_name text,table_name text);
  ```

  如果修复成功，函数返回修复过程中全局区块表中指定表的hash总和。

恢复用户表数据和全局区块表数据

步骤1 执行历史表修复操作。

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# SELECT pg_catalog.ledger_hist_repair('ledgernsp', 'usertable');
```

查询结果如下：

```
 ledger_hist_repair
--------------------
 84e8bfc3b974e9cf
(1 row)
```

该结果表明当前节点用户历史表修复成功，修复造成的用户历史表hash增量为84e8bfc3b974e9cf。

步骤2 执行全局区块表修复操作。

```
gaussdb=# SELECT pg_catalog.ledger_gchain_repair('ledgernsp', 'usertable');
```

查询结果如下：

```
 ledger_gchain_repair
----------------------
 a41714001181a294
(1 row)
```

该结果表明，全局区块表修复成功，且插入一条修复数据，其hash值为a41714001181a294。

----结束

恢复用户表和用户历史表名称

已通过enble_recyclebin参数和recyclebin_retention_time参数开启闪回DROP功能，恢复用户表和用户历史表名称。示例如下：

- DROP用户表，对用户表执行闪回DROP。使用ledger_hist_repair对用户表、用户历史表进行表名恢复。

```
-- 对用户表执行闪回drop，使用ledger_hist_repair对用户历史表进行表名恢复。
gaussdb=# CREATE TABLE ledgernsp.tab2(a int, b text);
CREATE TABLE
gaussdb=# DROP TABLE ledgernsp.tab2;
DROP TABLE
gaussdb=# SELECT rcyrelid, rcyname, rcyoriginname FROM gs_recyclebin;
 rcyrelid |           rcyname              |    rcyoriginname
----------+--------------------------------+---------------------
    16717 | BIN$38242338414D$42EB978==$0 | tab2
    16725 | BIN$382423384155$42EC678==$0 | gs_hist_tab2_index
    16722 | BIN$382423384152$42ECC30==$0 | ledgernsp_tab2_hist
    16720 | BIN$382423384150$42ED3E0==$0 | pg_toast_16717
(4 rows)
-- 对用户表执行闪回drop。
gaussdb=# TIMECAPSULE TABLE ledgernsp.tab2 TO BEFORE DROP;
TimeCapsule Table
-- 使用ledger_hist_repair恢复用户历史表表名。
gaussdb=# SELECT ledger_hist_repair('ledgernsp', 'tab2');
 ledger_hist_repair
--------------------
 0000000000000000
(1 row)
gaussdb=# TIMECAPSULE TABLE ledgernsp.tab2 TO BEFORE DROP;
TimeCapsule Table
gaussdb=# SELECT ledger_hist_repair('ledgernsp', 'tab2');
 ledger_hist_repair
--------------------
 0000000000000000
(1 row)
gaussdb=#  \d+ ledgernsp.tab2;
                        Table "ledgernsp.tab2"
  Column   |  Type   | Modifiers | Storage  | Stats target | Description
-----------+---------+-----------+----------+--------------+-------------
```

 版权所有 © 华为云计算技术有限公司

```
 a           | integer |           | plain    |              |
 b           | text    |           | extended |              |
 hash_1d2d14 | hash16  |           | plain    |              |
Has OIDs: no
Options: orientation=row, compression=no, storage_type=USTORE, segment=off,
toast.storage_type=USTORE, toast.toast_storage_type=enhanced_toast
History table name: ledgernsp_tab2_hist

-- 对用户表执行闪回drop，使用ledger_hist_repair对用户表进行表名恢复。
gaussdb=# CREATE TABLE ledgernsp.tab3(a int, b text);
CREATE TABLE
gaussdb=# DROP TABLE ledgernsp.tab3;
DROP TABLE
gaussdb=# SELECT rcyrelid, rcyname, rcyoriginname FROM gs_recyclebin;
 rcyrelid |           rcyname              |    rcyoriginname
----------+--------------------------------+---------------------
    17574 | BIN$44A4233844A6$B18E7A0==$0 | tab3
    17582 | BIN$44A4233844AE$B18F488==$0 | gs_hist_tab3_index
    17579 | BIN$44A4233844AB$B18FA40==$0 | ledgernsp_tab3_hist
    17577 | BIN$44A4233844A9$B190208==$0 | pg_toast_17574
(4 rows)
-- 对用户历史表执行闪回drop。
gaussdb=# TIMECAPSULE TABLE blockchain.ledgernsp_tab3_hist TO BEFORE DROP;
TimeCapsule Table
-- 拿到回收站中用户表对应的rcyname，使用ledger_hist_repair恢复用户表表名。
gaussdb=# SELECT ledger_hist_repair('ledgernsp', 'BIN$44A4233844A6$B18E7A0==$0');
 ledger_hist_repair
--------------------
 0000000000000000
(1 row)

gaussdb=# \d+ ledgernsp.tab3;
                        Table "ledgernsp.tab3"
   Column    |  Type   | Modifiers | Storage  | Stats target | Description
-------------+---------+-----------+----------+--------------+-------------
 a           | integer |           | plain    |              |
 b           | text    |           | extended |              |
 hash_7a0c87 | hash16  |           | plain    |              |
Has OIDs: no
Options: orientation=row, compression=no, storage_type=USTORE, segment=off,
toast.storage_type=USTORE, toast.toast_storage_type=enhanced_toast
History table name: ledgernsp_tab3_hist

-- 删除表。
gaussdb=# DROP TABLE ledgernsp.tab2 PURGE;
DROP TABLE
gaussdb=# DROP TABLE ledgernsp.tab3 PURGE;
DROP TABLE
```

2.6 基于标签的强制访问控制

2.6.1 定义安全标签

前提条件

数据库正常运行，执行操作的用户需要具有SYSADMIN权限或者继承了内置角色gs_role_seclabel的权限。

背景信息

提供系统表gs_security_label来查看系统中已创建好的安全标签：

```
SELECT * FROM gs_security_label;
```

 版权所有 © 华为云计算技术有限公司

操作步骤

以某个实际业务应用举例说明，比如某公司的信息等级为公开、秘密和绝密，那么就可分别对应成等级L1、L2和L3。信息存储范围为一层到五层各层员工的信息，那么可分别对应成范围G1、G2、G3、G4、G5，以此为基础定义安全标签。

步骤1 创建一个安全标签，标志信息等级为公开，范围为二层和四层。

```
gaussdb=# CREATE SECURITY LABEL label1 'L1:G2,G4';
```

步骤2 创建一个安全标签，标志信息等级为秘密，范围为二层到四层。

```
gaussdb=# CREATE SECURITY LABEL label2 'L2:G2-G4';
```

步骤3 创建一个安全标签，标志信息等级为绝密，范围为一层到五层。

```
gaussdb=# CREATE SECURITY LABEL label3 'L3:G1-G5';
```

步骤4 查询系统表gs_security_label。

```
gaussdb=# SELECT * FROM gs_security_label;
```

查询结果如下：

```
gaussdb=# SELECT * FROM gs_security_label;
 label_name | label_content
------------+---------------
 label1     | L1:G2,G4
 label2     | L2:G2-G4
 label3     | L3:G1-G5
(3 rows)
```

步骤5 删除安全标签。

```
gaussdb=# DROP SECURITY LABEL label1;
gaussdb=# DROP SECURITY LABEL label2;
gaussdb=# DROP SECURITY LABEL label3;
```

----结束

2.6.2 应用安全标签

前提条件

数据库正常运行，执行操作的用户需要具有SYSADMIN权限或者继承了内置角色gs_role_seclabel的权限。

背景信息

- 提供系统表pg_seclabel来查看表或表列上的安全标签：
  ```
  SELECT * FROM pg_seclabel;
  ```
- 提供系统表pg_shseclabel来查看用户/角色上的安全标签：
  ```
  SELECT * FROM pg_shseclabel;
  ```
- 提供系统视图pg_seclabels来查看系统中所有的安全标签记录：
  ```
  SELECT * FROM pg_seclabels;
  ```

操作步骤

数据库中已有用户user1、user2和数据表tbl。

步骤1 对用户user1应用安全标签label1，表示用户user1对应的信息等级为公开，范围为二层和四层。

```
gaussdb=# SECURITY LABEL ON USER user1 is 'label1';
```

 版权所有 © 华为云计算技术有限公司

步骤2 对用户user2应用安全标签label3，表示用户user2对应的信息等级为绝密，范围为一层到五层。

```
gaussdb=# SECURITY LABEL ON USER user2 is 'label3';
```

步骤3 对表tbl应用安全标签label2，表示表tbl对应的信息等级为秘密，范围为二层到四层。

```
gaussdb=# SECURITY LABEL ON TABLE tbl is 'label2';
```

步骤4 查询系统视图pg_seclabels。

```
gaussdb=# SELECT * FROM pg_seclabels;
```

查询结果如下：

```
gaussdb=# SELECT * FROM pg_seclabels;
 objoid | classoid | objsubid | objtype | objnamespace | objname | provider | label
--------+----------+----------+---------+--------------+---------+----------+--------
  16399 |     1259 |        0 | table   |         2200 | tbl     | maclabel | label2
  16391 |     1260 |        0 | role    |              | user1   | maclabel | label1
  16395 |     1260 |        0 | role    |              | user2   | maclabel | label3
(3 rows)
```

----结束

2.6.3 基于标签的强制访问控制检查

前提条件

- 数据库正常运行，强制访问控制检查开关（enable_mac_check=on）打开。
- 执行操作的用户已具有所需要访问表的ACL权限。

背景信息

基于安全标签的强制访问控制策略规则由系统内置设定，用户不能更改：

- 插入（INSERT）策略：只有主体安全标记等级小于等于客体安全标记等级且主体安全标记范围是客体安全标记范围的子集时才允许插入数据。
- 查询（SELECT）策略：只有主体安全标记等级大于等于客体安全标记等级且主体安全标记范围是客体安全标记范围的超集时才允许查询数据。
- 修改（UPDATE）和删除（DELETE）策略：只有主体安全标记等级等于客体安全标记等级且主体安全标记范围等于客体安全标记范围时才允许修改和删除数据。
- 若客体未标记，则强制访问控制策略对该客体不生效。
- 若主体未标记，则不能访问任意带有标记的客体。

操作步骤

步骤1 授予用户user1和user2数据表tbl的SELECT和INSERT操作权限。

```
gaussdb=# GRANT SELECT,INSERT ON tbl TO user1,user2;
```

步骤2 用户user1登录数据库执行SELECT和INSERT操作，SELECT失败，INSERT成功，符合强制访问控制策略规则。

```
gaussdb=> SELECT current_user;
 current_user
--------------
 user1
(1 row)
gaussdb=> SELECT * FROM tbl;
ERROR:  permission denied for relation tbl
DETAIL:  N/A
```

 版权所有 © 华为云计算技术有限公司

```
gaussdb=> INSERT INTO tbl VALUES (1);
INSERT 0 1
```

步骤3 用户user2登录数据库执行SELECT和INSERT操作，SELECT成功，INSERT失败，符合强制访问控制策略规则。

```
gaussdb=> SELECT current_user;
 current_user
--------------
 user2
(1 row)
gaussdb=> SELECT * FROM tbl;
 fir
-----
   1
(1 row)
gaussdb=> INSERT INTO tbl VALUES (1);
ERROR:  permission denied for relation tbl
DETAIL:  N/A
```

----结束

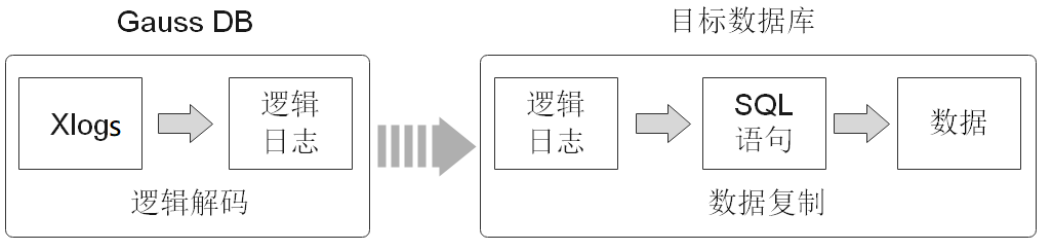
2.7 逻辑复制

GaussDB对数据复制能力的支持情况为：

支持通过数据迁移工具定期向异构数据库（如Oracle等）进行数据同步，不具备实时数据复制能力，因此不足以支撑与异构数据库间并网运行实时数据同步的诉求。

GaussDB提供了逻辑解码功能，通过反解xLog的方式生成逻辑日志。目标数据库解析逻辑日志以实时进行数据复制。具体如**图 逻辑复制**所示。逻辑复制降低了对目标数据库的形态限制，支持异构数据库、同构异形数据库对数据的同步，支持目标库进行数据同步期间的数据可读写，数据同步时延低。

图 2-1 逻辑复制



逻辑复制由两部分组成：逻辑解码和数据复制。逻辑解码会输出以事务为单位组织的逻辑日志。业务或数据库中间件将会对逻辑日志进行解析并最终实现数据复制。GaussDB当前只提供逻辑解码功能。

2.7.1 逻辑解码

版权所有 © 华为云计算技术有限公司

2.7.1.1 逻辑解码概述

功能描述

逻辑解码为逻辑复制提供事务解码的基础能力，GaussDB可以使用SQL函数接口进行逻辑解码。此方法调用方便，不需使用工具，对接外部工具接口也比较清晰，不需要额外适配。

由于逻辑日志是以事务为单位的，在事务提交后才能输出，且逻辑解码是由用户驱动的。因此，为了防止事务开始时的xLog被系统回收，或所需的事务信息被VACUUM回收，GaussDB新增了逻辑复制槽，用于阻塞xLog的回收。

一个逻辑复制槽表示一个更改流，这些更改可以在其他数据库中以它们在原数据库上产生的顺序重新执行。每个逻辑复制槽都由其对应逻辑日志的获取者维护。如果处于流式解码中的逻辑复制槽所在库不存在业务，则该复制槽会依照其他库的日志位置来推进。活跃状态的LSN序逻辑复制槽在处理到活跃事务快照日志时可以根据当前日志的LSN推进复制槽；活跃状态的CSN序逻辑复制槽在处理到虚拟事务日志时可以根据当前日志的CSN推进复制槽。

前提条件

- 逻辑日志目前从DN中抽取，如果进行逻辑复制，应使用SSL连接，因此需要保证相应DN上的GUC参数ssl设置为on。

  说明

  为避免安全风险，请保证启用SSL连接。

- 设置GUC参数wal_level为logical。
- 设置GUC参数max_replication_slots>=每个节点所需的（物理流复制槽数+备份槽数+逻辑复制槽数）。

  说明

  关于逻辑复制槽数，请按如下规则考虑。

  - 一个逻辑复制槽只能解码一个Database的修改，如果需要解码多个Database，则需要创建多个逻辑复制槽。
  - 如果需要多路逻辑复制同步给多个目标数据库，在源端数据库需要创建多个逻辑复制槽，每个逻辑复制槽对应一条逻辑复制链路。
  - 同一实例上，最多支持同时开启20个逻辑复制槽进行解码。

- 仅限初始用户和拥有REPLICATION权限的用户进行操作。三权分立关闭时数据库管理员可进行逻辑复制操作，三权分立开启时不允许数据库管理员进行逻辑复制操作。

注意事项

- 逻辑解码支持DDL约束请参见**规格约束**。
- 在执行特定的DDL语句（如普通表truncate或分区表exchange）时，可能造成解码数据丢失。
- 不支持数据页复制的DML解码。
- 不支持级联备机进行逻辑解码。
- 当执行DDL语句（如alter table）后，该DDL语句前尚未解码的物理日志可能会丢失。

 版权所有 © 华为云计算技术有限公司

- 单条元组大小不超过1GB，考虑解码结果可能大于插入数据，因此建议单条元组大小不超过500MB。
- GaussDB支持解码的数据类型为：INTEGER、BIGINT、SMALLINT、TINYINT、SERIAL、SMALLSERIAL、BIGSERIAL、FLOAT、DOUBLE PRECISION、BOOLEAN、BIT(n)、BIT VARYING(n)、DATE、TIME[WITHOUT TIME ZONE]、TIMESTAMP[WITHOUT TIME ZONE]、CHAR(n)、VARCHAR(n)、TEXT、CLOB（解码成TEXT格式）。
- 支持M-Compatibility数据库的DML逻辑解码，解码数据类型如下：
  - 整型：tinyint(m)、smallint(m)、mediumint(m)、bigint(m)、int(m)/integer、bool/boolean。
  - 浮点型：float(m,d)、double(m,d)。
  - 定点型：decimal(m,d)、numeric(m,n)。
  - BIT类型：BIT。
  - 字符串二进制类型：char(n)、varchar(n)、binary、varbinary。
  - 文本类型：tinytext、text、mediumtext、longtext。
  - 日期类型：data、time、timestamp、year。
  - 大对象类型：tinyblob、blob、mediumblob、longblob。
  - 数据类型属性：unsigned(部分支持)、zerofil(不支持)。
  - 不支持类型：enum枚举类型、set类型。
- 逻辑复制槽名称必须小于64个字符，仅支持小写字母、数字以及_?-.字符，且不支持“.”或“..”单独作为复制槽名称。
- 当逻辑复制槽所在数据库被删除后，这些复制槽变为不可用状态，需要用户手动删除。
- 对多库的解码需要分别在库内创建流复制槽并开始解码，每个库的解码都需要单独扫描一遍日志。
- 不支持强切，强切后需要重新全量导出数据。
- 备机解码时，switchover和failover时可能出现解码数据变多，需用户手动过滤。Quorum协议下，switchover和failover选择升主的备机，需要与当前主机日志同步。
- 不允许主备，多个备机同时使用同一个复制槽解码，否则会产生数据不一致的情况。
- 只支持主机创建删除复制槽。当删除的复制槽为最后一个复制槽时，删除完成后会产生告警"replicationSlotMinLSN is INVALID_WAL_REC_PTR!!!"和"replicationSlotMaxLSN is INVALID_WAL_REC_PTR!!!"。
- 数据库故障重启或逻辑复制线程重启后，解码数据可能存在重复，用户需手动过滤。
- 计算机内核故障后，解码可能存在乱码，需手动或自动过滤。
- 请确保在创建逻辑复制槽过程中未启动长事务，启动长事务会阻塞逻辑复制槽的创建。
- 不支持interval partition表DML复制。
- 不支持全局临时表的DML解码。
- 不支持本地临时表的DML解码。
- M兼容性下，不支持SELECT INTO语句的解码。非M兼容性下，SELECT INTO语句会解码创建目标表的DDL操作，数据插入的DML操作不解码。

 版权所有 © 华为云计算技术有限公司

- 禁止在使用逻辑复制槽时在其他节点对该复制槽进行操作，删除复制槽的操作需在该复制槽停止解码后执行。
- 为解析某个astore表的UPDATE和DELETE语句，需为此表配置REPLICA IDENTITY属性，在此表无主键时需要配置为FULL，具体配置方式请参考《开发者指南》中“SQL参考 > SQL语法 > ALTER TABLE”章节中“REPLICA IDENTITY { DEFAULT | USING INDEX index_name | FULL | NOTHING }”字段。
- 基于目标库可能需要源库的系统状态信息考虑，逻辑解码仅自动过滤模式'pg_catalog'和'pg_toast'下OID小于16384的系统表的逻辑日志。若目标库不需要复制其他相关系统表的内容，逻辑日志回放过程中需要对相关系统表进行过滤。
- 在开启逻辑复制的场景下，如需创建包含系统列的主键索引，必须将该表的REPLICA IDENTITY属性设置为FULL或是使用USING INDEX指定不包含系统列的、唯一的、非局部的、不可延迟的、仅包括标记为NOT NULL的列的索引。
- 若一个事务的子事务过多导致落盘文件过多，退出解码时需执行SQL函数pg_terminate_backend(*逻辑解码的walsender线程id*)来手动停止解码，而且退出时延增加约为1分钟/30万个子事务。因此在开启逻辑解码时，若一个事务的子事务数量达到5万时，会打印一条WARNING日志。
- 当逻辑复制槽处于非活跃状态，且设置GUC参数enable_xlog_prune=on、enable_logicalrepl_xlog_prune=on、max_size_for_xlog_retention为非零值，且备份槽或逻辑复制槽导致保留日志段数已超过GUC参数wal_keep_segments，同时其他复制槽并未导致更多的保留日志段数时，如果max_size_for_xlog_retention大于0且当前逻辑复制槽导致保留日志的段数（每段日志大小为16MB）超过max_size_for_xlog_retention，或者max_size_for_xlog_retention小于0且磁盘使用率达到(-max_size_for_xlog_retention)/100，当前逻辑复制槽会强制失效，其restart_lsn将被设置为7FFFFFFF/FFFFFFFF。该状态的逻辑复制槽不参与阻塞日志回收或系统表历史版本的回收，但仍占用复制槽的限制数量，需要手动删除。
- 备机解码启动后，向主机发送复制槽推进指令后会占用主机上对应的逻辑复制槽（即标识为活跃状态）。在此之前主机上对应逻辑复制槽为非活跃状态，此状态下如果满足逻辑复制槽强制失效条件则会被标记为失效（即restart_lsn将被设置为7FFFFFFF/FFFFFFFF），备机将无法推进主机复制槽，且备机回放完成复制槽失效日志后当前复制槽的备机解码断开后将无法重连。
- 不活跃的逻辑复制槽将阻塞WAL日志回收和系统表元组历史版本清理，导致磁盘日志堆积和系统表扫描性能下降，因此不再使用的逻辑复制槽请及时清理。
- 通过协议连接DN创建逻辑复制槽仅支持LSN序复制槽。
- 解码使用JSON格式输出时不支持数据列包含特殊字符（如'\0'空字符），解码输出列内容将出现被截断现象。
- 不支持无日志表的DML解码。
- 当同一事务产生大量需要落盘的子事务时，同时打开的文件句柄可能会超限，需将GUC参数max_files_per_process配置成大于子事务数量上限的两倍。
- 不支持账本数据库功能，当前版本如果开启解码任务的数据库中有关于账本数据库的DML操作，则解码结果中会包含hash列，从而导致回放失败。
- 不支持M-Compatibility数据库的DDL解码（对M-Compatibility数据库使用逻辑解码功能时，enable-ddl-decoding选项必须关闭）。

SQL 函数解码性能

在Benchmarksql-5.0的100warehouse场景下，采用pg_logical_slot_get_changes时：

 版权所有 © 华为云计算技术有限公司

- 单次解码数据量4K行（对应约5MB～10MB日志），解码性能0.3MB/s～0.5MB/s。
- 单次解码数据量32K行（对应约40MB～80MB日志），解码性能3MB/s～5MB/s。
- 单次解码数据量256K行（对应约320MB～640MB日志），解码性能3MB/s～5MB/s。
- 单次解码数据量再增大，解码性能无明显提升。

如果采用pg_logical_slot_peek_changes + pg_replication_slot_advance方式，解码性能相比采用pg_logical_slot_get_changes时要下降30%～50%。

2.7.1.2 逻辑解码选项

逻辑解码选项可以用来为本次逻辑解码提供限制或额外功能，如“解码结果是否包含事务号”、“解码时是否忽略空事务”等。对于具体配置方法，SQL函数解码请参考《开发者指南》中“SQL参考 > 函数和操作符 > 系统管理函数 > 逻辑复制函数”章节中函数pg_logical_slot_peek_changes的可选入参''options_name'和'options_value'，JDBC流式解码请参考《开发者指南》中“应用程序开发教程 > 基于JDBC开发 > 示例：逻辑复制代码示例”章节示例代码中函数withSlotOption的使用方法。

通用选项（串行解码和并行解码均可配置，但可能无效，请参考相关选项详细说明）

- include-xids：

  解码出的data列是否包含xid信息。

  取值范围：boolean型，默认值为true。

  - false：设为false时，解码出的data列不包含xid信息。
  - true：设为true时，解码出的data列包含xid信息。

- skip-empty-xacts：

  解码时是否忽略空事务信息。

  取值范围：boolean型，默认值为false。

  - false：设为false时，解码时不忽略空事务信息。
  - true：设为true时，解码时会忽略空事务信息。

- include-timestamp：

  解码信息是否包含commit时间戳。

  取值范围：boolean型，针对并行解码场景默认值为false，针对SQL函数解码和串行解码场景默认值为true。

  - false：设为false时，解码信息不包含commit时间戳。
  - true：设为true时，解码信息包含commit时间戳。

- only-local：

  是否仅解码本地日志。

  取值范围：boolean型，默认值为true。

  - false：设为false时，解码非本地日志和本地日志。
  - true：设为true时，仅解码本地日志。

- white-table-list：

  白名单参数，包含需要进行解码的schema和表名。

 版权所有 © 华为云计算技术有限公司

取值范围：包含白名单中表名的字符串，不同的表以','为分隔符进行隔离；使用'*'来模糊匹配所有情况；schema名和表名间以'.'分割，不允许存在任意空白符。例如：

```
select * from pg_logical_slot_peek_changes('slot1', NULL, 4096, 'white-table-list',
'public.t1,public.t2,*.t3,my_schema.*');
```

- max-txn-in-memory：

  内存管控参数，单位为MB，单个事务占用内存大于该值即进行落盘。

  串行解码-取值范围：0~100的整型，默认值为0，即不开启此种管控。

  并行解码-取值范围：0~max_process_memory总量的25%，默认值为max_process_memory/4/1024，其中1024为kB到MB的单位转换，0表示不开启此条内存管控项。

- max-reorderbuffer-in-memory

  内存管控参数，单位为GB，拼接-发送线程中正在拼接的事务总内存（包含缓存）大于该值则对当前解码事务进行落盘。

  串行解码-取值范围：0~100的整型，默认值为0，即不开启此种管控。

  并行解码-取值范围：0~max_process_memory总量的50%，默认值为max_process_memory/2/1048576，其中1048576为kB到GB的单位转换，0表示不开启此条内存管控项。

- include-user：

  事务的BEGIN逻辑日志是否输出事务的用户名。事务的用户名特指授权用户——执行事务对应会话的登录用户，它在事务的整个执行过程中不会发生变化。

  取值范围：boolean型，默认值为false。

  - false：设为false时，事务的BEGIN逻辑日志不输出事务的用户名。
  - true：设为true时，事务的BEGIN逻辑日志输出事务的用户名。

- exclude-userids：

  黑名单用户的OID参数。

  取值范围：字符串类型，指定黑名单用户的OID，多个OID通过','分隔，不校验用户OID是否存在。

- exclude-users：

  黑名单用户的名称列表。

  取值范围：字符串类型，指定黑名单用户名，通过','分隔，不校验用户名是否存在。

- dynamic-resolution：

  是否动态解析黑名单用户名。

  取值范围：boolean型，默认值为true。

  - false：设为false时，当解码观测到黑名单exclude-users中用户不存在时将会报错并退出逻辑解码。
  - true：设为true时，当解码观测到黑名单exclude-users中用户不存在时继续解码。

- standby-connection：

  仅流式解码设置，是否仅限制备机解码。

  取值范围：boolean型，默认值为false。

  - true：设为true时，仅允许连接备机解码，连接主机解码时会报错退出。

 版权所有 © 华为云计算技术有限公司

- false：设为false时，不做限制，允许连接主机或备机解码。

说明

如果主机资源使用率较大，且业务对增量数据同步的实时性不敏感，建议进行备机解码；如果业务对增量数据同步的实时性要求高，并且主机业务压力较小，建议使用主机解码。

- sender-timeout：

  仅流式解码设置，内核与客户端的心跳超时阈值。如果该时间段内没有收到客户端任何消息，逻辑解码将主动停止，并断开和客户端的连接。单位为毫秒（ms）。

  取值范围：0~2147483647的int型，默认值取决于GUC参数logical_sender_timeout的配置值。

- change-log-max-len:

  逻辑日志缓存长度上限参数，单位为字节，仅并行解码有效，串行解码及SQL函数解码无效。如果单条解码结果长度超过上限，则会销毁重新分配大小为1024字节的内存并缓存。过长会增加内存占用，过短会频繁触发内存申请和释放的操作，不建议设置成小于1024的值。

  取值范围：1~65535，默认值为4096。

- max-decode-to-sender-cache-num:

  并行解码日志的缓存条数阈值，仅并行解码有效，串行解码及SQL函数解码无效。缓存中的日志条数未超过这个阈值时，使用完毕的解码日志将置入缓存，否则直接释放。

  取值范围：1~65535，默认值为4096。

- enable-heartbeat：

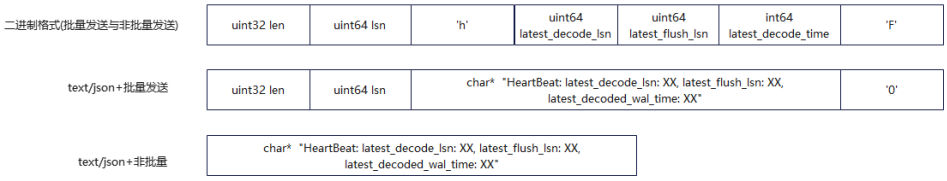
  仅流式解码设置，代表是否输出心跳日志。

  取值范围：boolean型，默认值为false。

  - true：设为true时，输出心跳日志。
  - false：设为false时，不输出心跳日志。

说明

若开启心跳日志选项，此处说明并行解码场景心跳日志如何解析：二进制格式首先是字符'h'表示消息是心跳日志，之后是心跳日志内容，分别是8字节uint64代表LSN，表示发送心跳逻辑日志时读取的WAL日志结束位置；8字节uint64代表LSN，表示发送心跳逻辑日志时刻已经落盘的WAL日志的位置；8字节int64代表时间戳（从1970年1月1日开始），表示最新解码到的事务日志或检查点日志的产生时间戳。关于消息结束符：如果是二进制格式则为字符'F'，如果格式为text或者json且为批量发送则结束符为0，否则没有结束符。具体解析见下图：



- parallel-decode-num：

  仅流式解码设置有效，并行解码的Decoder线程数量；系统函数调用场景下此选项无效，仅校验取值范围。

  取值范围：1~20的int型，取1表示按照原有的串行逻辑进行解码，取其余值即为开启并行解码，默认值为1。

版权所有 © 华为云计算技术有限公司

须知

当parallel-decode-num不配置（即为默认值1）或显式配置为1时，下述“并行解码”中的选项不可配置。

- output-order：

  仅流式解码设置有效，代表是否使用CSN顺序输出解码结果；系统函数调用场景下此选项无效，仅校验取值范围。

  取值范围：0或1的int型，默认值为0。

  - 0：设为0时，解码结果按照事务的COMMIT LSN排序，当且仅当解码复制槽的confirmed_csn列值为0（即不显示）时可使用该方式，否则报错。
  - 1：设为1时，解码结果按照事务的CSN排序，当且仅当解码复制槽的confirmed_csn列值为非零时可使用该方式，否则报错。

- auto-advance：

  仅流式解码设置有效，代表是否允许自主推进逻辑复制槽。

  取值范围：boolean型，默认值为false。

  - true：设为true时，在已发送日志都被确认推进且没有待发送事务时，推进逻辑复制槽到当前解码位置。
  - false：设为false时，完全交由复制业务调用日志确认接口推进逻辑复制槽。

- enable-ddl-decoding：

  逻辑解码控制参数，用于控制DDL的反解析流程以及输出形式。

  取值范围：boolean型，默认值为false。

  - true：值为true时，开启DDL语句的逻辑解码。
  - false：值为false时，不开启DDL语句的逻辑解码。

- enable-ddl-json-format：

  逻辑解码控制参数，用于控制DDL的反解析流程以及输出形式。

  取值范围：boolean型，默认值为false。

  - true：值为true时，传送JSON格式的DDL反解析结果。
  - false：设为false时，传送TEXT格式的DDL反解析结果。

- skip-generated-columns：

  逻辑解码控制参数，用于跳过生成列的输出。对UPDATE和DELETE的旧元组无效，相应元组始终会输出生成列。

  取值范围：boolean型，默认值为false。

  - true：值为true时，不输出生成列的解码结果。
  - false：设为false时，输出生成列的解码结果。

串行解码

- force-binary：

  是否以二进制格式输出解码结果，针对不同场景呈现不同行为。

  - 针对系统函数pg_logical_slot_get_binary_changes和pg_logical_slot_peek_binary_changes：

    取值范围：boolean型，默认值为false。此值无实际意义，均以二进制格式输出解码结果。

版权所有 © 华为云计算技术有限公司

  - 针对系统函数pg_logical_slot_get_changes、pg_logical_slot_peek_changes和pg_logical_get_area_changes：

    取值范围：仅取false值的boolean型。以文本格式输出解码结果。
  - 针对流式解码：

    取值范围：boolean型，默认值为false。此值无实际意义，均以文本格式输出解码结果。

并行解码

以下配置选项仅限流式解码设置。

- decode-style：

  指定解码格式。

  取值范围：char型的字符'j'、't'或'b'，分别代表json格式、text格式及二进制格式。默认值为'b'即二进制格式解码。

  对于json格式和text格式解码，开启批量发送选项时的解码结果中，每条解码语句的前4字节组成的uint32代表该条语句总字节数（不包含该uint32类型占用的4字节，0代表本批次解码结束），8字节uint64代表相应lsn（begin对应first_lsn，commit对应end_lsn，其他场景对应该条语句的lsn）。

版权所有 © 华为云计算技术有限公司

说明

二进制格式编码规则如下所示：

1. 前4字节代表接下来到语句级别分隔符字母P（不含）或者该批次结束符F（不含）的解码结果的总字节数，该值如果为0代表本批次解码结束。
2. 接下来8字节uint64代表相应lsn（begin对应first_lsn，commit对应end_lsn，其他场景对应该条语句的lsn）。
3. 接下来1字节的字母有5种B/C/I/U/D，分别代表begin/commit/insert/update/delete。
4. 第3步字母为B时：
    1. 接下来的8字节uint64代表CSN。
    2. 接下来的8字节uint64代表first_lsn。
    3. 【该部分为可选项】接下来的1字节字母如果为T，则代表后面4字节uint32表示该事务commit时间戳长度，再后面等同于该长度的字符为时间戳字符串。
    4. 【该部分为可选项】接下来的1字节字母如果为N，则代表后面4字节uint32表示该事务用户名的长度，再后面等同于该长度的字符为事务的用户名字。
    5. 因为之后仍可能有解码语句，接下来会有1字节字母P或F作为语句间的分隔符，P代表本批次仍有解码的语句，F代表本批次解码完成。
5. 第3步字母为C时：
    1. 【该部分为可选项】接下来1字节字母如果为X，则代表后面的8字节uint64表示xid。
    2. 【该部分为可选项】接下来的1字节字母如果为T，则代表后面4字节uint32表示时间戳长度，再后面等同于该长度的字符为时间戳字符串。
    3. 因为批量发送日志时，一个COMMIT日志解码之后可能仍有其他事务的解码结果，接下来的1字节字母如果为P则表示该批次仍需解码，如果为F则表示该批次解码结束。
6. 第3步字母为I/U/D时：
    1. 接下来的2字节uint16代表schema名的长度。
    2. 按照上述长度读取schema名。
    3. 接下来的2字节uint16代表table名的长度。
    4. 按照上述长度读取table名。
    5. 【该部分为可选项】接下来1字节字母如果为N代表为新元组，如果为O代表为旧元组，这里先发送新元组。
        1. 接下来的2字节uint16代表该元组需要解码的列数，记为attrnum。
        2. 以下流程重复attrnum次。
            1. 接下来2字节uint16代表列名的长度。
            2. 按照上述长度读取列名。
            3. 接下来4字节uint32代表当前列类型的OID。
            4. 接下来4字节uint32代表当前列值（以字符串格式存储）的长度，如果为0xFFFFFFFF则表示NULL，如果为0则表示长度为0的字符串。
            5. 按照上述长度读取列值。
    6. 因为之后仍可能有解码语句，接下来的1字节字母如果为P则表示该批次仍需解码，如果为F则表示该批次解码结束。

- sending-batch：

  指定是否批量发送。

  取值范围：0或1的int型，默认值为0。

  - 0：设为0时，表示逐条发送解码结果。
  - 1：设为1时，表示解码结果累积到达1MB则批量发送解码结果。

  开启批量发送的场景中，当解码格式为'j'或't'时，在原来的每条解码语句之前会附加一个uint32类型，表示本条解码结果长度（长度不包含当前的uint32类型），以及一个uint64类型，表示当前解码结果对应的lsn。

 版权所有 © 华为云计算技术有限公司

须知

在CSN序解码（即output-order设置为1）场景下，批量发送仅限于单个事务内（即如果一个事务有多条较小的语句会采用批量发送），即不会使用批量发送功能在同一批次里发送多个事务，且BEGIN和COMMIT语句不会批量发送。

- parallel-queue-size：

  指定并行逻辑解码线程间进行交互的队列长度。

  取值范围：2~1024的int型，且必须为2的整数幂，默认值为128。

  队列长度和解码过程的内存使用量正相关。

- logical-reader-bind-cpu:

  reader线程绑定cpu核号的参数。

  取值范围：-1~65535，不使用该参数则为不绑核。

  默认-1，为不绑核。-1不可手动设置，核号应确保在机器总逻辑核数以内，不然会返回报错。多个线程绑定同一个核会导致该核负担加重，从而导致性能下降。

- logical-decoder-bind-cpu-index：

  逻辑解码线程绑定cpu核号的参数。

  取值范围：-1~65535，不使用该参数则为不绑核。

  默认-1，不绑核。-1不可手动设置，核号应确保在机器总逻辑核数以内且小于[cpu核数 - 并行逻辑解码数]，不然会返回报错。

  从给定的核号参数开始，新拉起的线程会依次递增加一。

  多个线程绑定同一个核会导致该核负担加重，从而导致性能下降。

版权所有 © 华为云计算技术有限公司

说明

GaussDB在进行逻辑解码和日志回放时，会占用大量的CPU资源，相关线程如Walwriter、WalSender、WALreceiver、pageredo就处于性能瓶颈，如果能将这些线程绑定在固定的CPU上运行，可以减少因操作系统调度线程频繁切换CPU，导致缓存未命中带来的性能开销，从而提高流程处理速度，如用户场景有性能需求，可根据以下的绑核样例进行配置优化。

- 参数样例：
  1. walwriter_cpu_bind=1
  2. walwriteraux_bind_cpu=2
  3. wal_receiver_bind_cpu=4
  4. wal_rec_writer_bind_cpu=5
  5. wal_sender_bind_cpu_attr='cpuorderbind:7-14'
  6. redo_bind_cpu_attr='cpuorderbind:16-19'
  7. logical-reader-bind-cpu=20
  8. logical-decoder-bind-cpu-index=21
- 样例中1.2.3.4.5.6通过GUC工具设置，使用指令如

  gs_guc set -Z datanode -N all -I all -c “walwriter_cpu_bind=1”。

  样例中7.8通过JDBC客户端发起解码请求时添加。
- 样例中如walwriter_cpu_bind=1是限定该线程在1号CPU上运行。

  cpuorderbind：7-14意为拉起的每个线程依次绑定7号到14号CPU，如果范围内的CPU用完，则新拉起的线程不参与绑核。

  logical-decoder-bind-cpu-index意为拉起的线程从21号CPU依次开始绑定，后续拉起的线程分别绑定21、22、23，依次类推。
- 绑核的原则是一个线程占用一个CPU，样例中的GUC参数说明可参考管理员指南。
- 不恰当的绑核，例如将多个线程绑定在一个CPU上很有可能带来性能劣化。
- 可以通过lscpu指令查看“CPU(s):”得知自己环境的CPU逻辑核心数。

CPU逻辑核心数低于36则不建议使用此套绑核策略，此时建议使用默认配置（不进行参数设置）。

2.7.1.3 使用 SQL 函数接口进行逻辑解码

GaussDB可以通过调用SQL函数，进行创建、删除、推进逻辑复制槽，获取解码后的事务日志。

操作步骤

步骤1 以具有REPLICATION权限的用户登录GaussDB数据库主DN。

步骤2 使用如下命令连接数据库。

```
gsql -U user1 -W password -d db1 -p 16000 -r
```

其中，user1为用户名，password为密码，db1为需要连接的数据库名称，16000为数据库端口号，用户可根据实际情况替换。

步骤3 创建名称为slot1的逻辑复制槽。

```
db1=> SELECT * FROM pg_create_logical_replication_slot('slot1', 'mppdb_decoding');
slotname | xlog_position
----------+---------------
slot1    | 0/601C150
(1 row)
```


版权所有 © 华为云计算技术有限公司

步骤4 在数据库中创建表t，并向表t中插入数据。

```
db1=> CREATE TABLE t(a int PRIMARY KEY, b int);
db1=> INSERT INTO t VALUES(3,3);
```

步骤5 读取复制槽slot1解码结果，解码条数为4096。

说明

逻辑解码选项请参见**逻辑解码选项**。

```
db1=> SELECT * FROM pg_logical_slot_peek_changes('slot1', NULL, 4096);
location  |  xid  | data
-----------+-------
+-----------------------------------------------------------------------------------------------------------------------------------------------
---------------------
-------------------------------------------
 0/601C188 | 1010023 | BEGIN 1010023
 0/601ED60 | 1010023 | COMMIT 1010023 (at 2023-09-14 16:03:51.394287+08) CSN 1010022
 0/601ED60 | 1010024 | BEGIN 1010024
 0/601ED60 | 1010024 | {"table_name":"public.t","op_type":"INSERT","columns_name":
["a","b"],"columns_type":["integer","integer"],"columns_val":["3","3"],"old_keys_name":[],"old_keys_type":
[],"old_keys_val":[]}
 0/601EED8 | 1010024 | COMMIT 1010024 (at 2023-09-14 16:03:57.239821+08) CSN 1010023
(5 rows)
```

步骤6 删除逻辑复制槽slot1，删除业务表t。

```
db1=> SELECT * FROM pg_drop_replication_slot('slot1');
 pg_drop_replication_slot
--------------------------

(1 row)

db1=> DROP TABLE t;
DROP TABLE
```

----结束

2.7.1.4 使用流式解码实现数据逻辑复制

第三方复制工具通过流式逻辑解码从GaussDB抽取逻辑日志后到对端数据库回放。对于使用JDBC连接数据库的复制工具，具体代码请参考《开发者指南》中“应用程序开发教程 > 基于JDBC开发 > 示例：逻辑复制代码示例”章节。

2.7.1.5 逻辑解码支持 DDL

GaussDB主机上正常执行DDL语句，通过逻辑解码工具可以获取到DDL语句。

版权所有 © 华为云计算技术有限公司

表 2-5 具体支持的 DDL 类型

表	索引	自定义函数	自定义存储过程	触发器	Sequence	视图	物化视图	Package	Schema
CREATE TABLE [PARTITION] ALTER TABLE DROP TABLE	CREATE INDEX ALTER INDEX DROP INDEX REINDEX	CREATE FUNCTION ALTER FUNCTION DROP FUNCTION	CREATE PROCEDURE ALTER PROCEDURE DROP PROCEDURE	CREATE TRIGGER ALTER TRIGGER DROP TRIGGER	CREATE SEQUENCE ALTER SEQUENCE DROP SEQUENCE	CREATE VIEW ALTER VIEW DROP VIEW	CREATE MATERIALIZED VIEW ALTER MATERIALIZED VIEW DROP MATERIALIZED VIEW	CREATE PACKAGE ALTER PACKAGE DROP PACKAGE	CREATE SCHEMA ALTER SCHEMA DROP SCHEMA

功能描述

数据库在执行DML的时候，存储引擎会生成对应的DML日志，用于进行恢复，对这些DML日志进行解码，即可还原对应的DML语句，生成逻辑日志。而对于DDL语句，数据库并不记录DDL原语句的日志，而是记录DDL语句涉及的系统表的DML日志。DDL种类多样、语法复杂，逻辑复制要支持DDL语句，通过这些系统表的DML日志来解码原DDL语句是非常困难的。新增DDL日志记录原DDL信息，并在解码时通过DDL日志可以得到DDL原语句。

在DDL语句执行过程中，SQL引擎解析器会对原语句进行语法、词法解析，并生成解析树（不同的DDL语法会生成不同类型的解析树，解析树中包含DDL语句的全部信息）。随后，执行器通过这些信息执行对应操作，生成、修改对应元信息。

本文通过新增DDL日志的方式，来支持逻辑解码DDL，其内容由解析器结果（解析树）以及执行器结果生成，并在执行器执行完成后生成该日志。

从语法树反解析出DDL，DDL反解析能够将DDL命令转换为JSON格式的语句，并提供必要的信息在目标位置重建DDL命令。与原始DDL命令字符串相比，使用DDL反解析的好处包括：

1. 解析出来的每个数据库对象都带有Schema，因此如果使用不同的search_path，也不会有歧义。
2. 结构化的JSON和格式化的输出能支持异构数据库。如果用户使用的是不同的数据库版本，并且存在某些DDL语法差异，需要在应用之前解决这些差异。

反解析输出的结果是规范化后的形式，结果与用户输入等价，不保证完全相同，例如：

示例1：在函数体中没有单引号'时，函数体的分隔符$$会被解析为单引号'。

原始SQL语句：

```
CREATE FUNCTION func(a INT) RETURNS INT AS
$$
```

 版权所有 © 华为云计算技术有限公司

```
BEGIN
a:= a+1;
CREATE TABLE test(col1 INT);
INSERT INTO test VALUES(1);
DROP TABLE test;
RETURN a;
END;
$$
LANGUAGE plpgsql;
```

反解析结果：

```
CREATE  FUNCTION public.func ( IN a pg_catalog.int4 ) RETURNS pg_catalog.int4 LANGUAGE plpgsql
VOLATILE CALLED ON NULL INPUT SECURITY INVOKER COST 100 AS '
BEGIN
a:= a+1;
CREATE TABLE test(col1 INT);
INSERT INTO test VALUES(1);
DROP TABLE test;
RETURN a;
END;
';
```

示例2：“CREATE MATERIALIZED VIEW v46_4 AS SELECT a, b FROM t46 ORDER BY a OFFSET 10 ROWS FETCH NEXT 3 ROWS ONLY”会被反解析为“CREATE MATERIALIZED VIEW public.v46_4 AS SELECT a, b FROM public.t46 ORDER BY a OFFSET 10 LIMIT 3”；。

示例3：“ALTER INDEX "Alter_Index_Index" REBUILD PARTITION "CA_ADDRESS_SK_index2"”会被反解析为“REINDEX INDEX public."Alter_Index_Index" PARTITION "CA_ADDRESS_SK_index2"”。

示例4：创建/修改范围分区表，START END语法格式均解码转化为LESS THAN语句：

```
gaussdb=# CREATE TABLE test_create_table_partition2 (c1 INT, c2 INT)
PARTITION BY RANGE (c2) (
   PARTITION p1 START(1) END(1000) EVERY(200) ,
   PARTITION p2 END(2000),
   PARTITION p3 START(2000) END(2500),
   PARTITION p4 START(2500),
   PARTITION p5 START(3000) END(5000) EVERY(1000)
);
```

会被反解析为：

```
gaussdb=# CREATE TABLE test_create_table_partition2 (c1 INT, c2 INT)
PARTITION BY RANGE (c2) (
   PARTITION p1_0 VALUES LESS THAN ('1'), PARTITION p1_1 VALUES LESS THAN ('201'), PARTITION p1_2
VALUES LESS THAN ('401'), PARTITION p1_3 VALUES LESS THAN ('601'), PARTITION p1_4 VALUES LESS
THAN ('801'), PARTITION p1_5 VALUES LESS    THAN ('1000'),
   PARTITION p2 VALUES LESS THAN ('2000'),
   PARTITION p3 VALUES LESS THAN ('2500'),
   PARTITION p4 VALUES LESS THAN ('3000'),
   PARTITION p5_1 VALUES LESS THAN ('4000'),
   PARTITION p5_2 VALUES LESS THAN ('5000')
);
```

示例5：新增表的列字段时，使用IF NOT EXISTS判断。

- 原始SQL语句：

  ```
  gaussdb=# CREATE TABLE IF NOT EXISTS tb5 (c1 int,c2 int) with (ORIENTATION=ROW,
  STORAGE_TYPE=USTORE);
  gaussdb=# ALTER TABLE IF EXISTS tb5 * ADD COLUMN IF NOT EXISTS c2 char(5) after c1; -- 可解码。
  TABLE中已有int型的列c2，语句执行跳过，反解析结果中c2列的类型仍为原来的类型。
  ```

  反解析结果：

  ```
  gaussdb=# ALTER TABLE IF EXISTS public.tb5 ADD COLUMN IF NOT EXISTS c2 pg_catalog.int4 AFTER
  c1;
  ```

 版权所有 © 华为云计算技术有限公司

- 原始SQL语句：
  ```
  gaussdb=# ALTER TABLE IF EXISTS tb5 * ADD COLUMN IF NOT EXISTS c2 char(5) after c1，ADD COLUMN IF NOT EXISTS c3 char(5) after c1; -- 解码。新增列c3的反解析结果类型正确。
  ```
  反解析结果：
  ```
  gaussdb=# ALTER TABLE IF EXISTS public.tb5 ADD COLUMN IF NOT EXISTS c2 pg_catalog.int4 AFTER c1, ADD COLUMN IF NOT EXISTS c3 pg_catalog.bpchar(5) AFTER c1;
  ```
- 原始SQL语句：
  ```
  gaussdb=# ALTER TABLE IF EXISTS tb5 * ADD COLUMN c2 char(5) after c1，ADD COLUMN IF NOT EXISTS c4 int after c1; --不解码，语句执行错误。
  ```

规格约束

- 逻辑解码支持DDL规格：
  - 无DDL时，原只有DML场景的逻辑解码性能不下降。
  - 纯DDL逻辑解码性能标准环境下约为100MB/S，DDL/DML混合事务逻辑解码性能标准环境下约为100MB/S。
  - 开启此功能后，对DDL语句影响性能下降小于15%。
- 解码通用约束（串行和并行）：
  - 不支持对本地临时对象做DDL解码。例如：LOCAL临时表、临时Schema。
  - alter table add column的default值不支持stable类型和volatile类型的函数；create table和alter table的column的check表达式不支持stable类型和volatile类型的函数；alter table如果有多条子语句，只要其中一条子语句存在上述两种情况，则该条alter table整条语句不反解析。
    ```
    gaussdb=# ALTER TABLE tbl_28 ADD COLUMN b1 TIMESTAMP DEFAULT NOW(); -- 's' NOT DEPARSE
    gaussdb=# ALTER TABLE tbl_28 ADD COLUMN b2 INT DEFAULT RANDOM(); -- 'v' NOT DEPARSE
    gaussdb=# ALTER TABLE tbl_28 ADD COLUMN b3 INT DEFAULT ABS(1); -- 'i' DEPARSE
    ```
  - 不支持CREATE TABLE LIKE的DDL解码。

    说明

    对于CREATE TABLE LIKE创建的表，仍会解码其ALTER以及DROP语句。
  - 创建对象时语句中存在IF NOT EXIES时，如果对象已存在，则不进行解码。删除对象时语句中存在IF EXISTS时，如果对象不存在，则不进行解码。
  - 不对ALTER PACKAGE COMPILE语句进行解码，但会解码实例化内容中包含的DDL/DML语句。如果PACKAGE里没有DDL或DML部分的实例化内容，则alter package compile会被逻辑解码忽略。
  - 仅支持本版本之前版本的商用DDL语法，以下SQL语句不支持逻辑解码。
    - B兼容模式下创建表，列字段添加ON UPDATE事件。

      原始SQL语句：
      ```
      -- B兼容模式下创建表，列字段添加ON UPDATE事件
      gaussdb=# CREATE TABLE IF NOT EXISTS tb1 (c1 time without time zone ON UPDATE CURRENT_TIMESTAMP) with (ORIENTATION=ROW, STORAGE_TYPE=USTORE);
      -- B兼容模式下修改表，列字段添加ON UPDATE事件
      gaussdb=# ALTER TABLE IF EXISTS ONLY tb2 MODIFY COLUMN c2 time without time zone ON UPDATE LOCALTIMESTAMP;
      ```
      反解析结果：
      ```
      gaussdb=# CREATE TABLE IF NOT EXISTS public.tb1 (c1 TIME) WITH (orientation = 'row', storage_type = 'ustore', compression = 'no') NOCOMPRESS;
      gaussdb=# ALTER TABLE IF EXISTS ONLY public.tb2 MODIFY COLUMN c2 TIME;
      ```

 版权所有 © 华为云计算技术有限公司

- 创建行存表，设置ILM策略。

  原始SQL语句：

  ```
  gaussdb=# CREATE TABLE IF NOT EXISTS tb3 (c1 int) with
  (storage_type=USTORE,ORIENTATION=ROW) ILM ADD POLICY ROW STORE COMPRESS
  ADVANCED ROW AFTER 7 day OF NO MODIFICATION;
  ```

  反解析结果：

  ```
  gaussdb=# CREATE TABLE IF NOT EXISTS public.tb3 (c1 pg_catalog.int4) WITH
  (storage_type = 'ustore', orientation = 'row', compression = 'no') NOCOMPRESS;
  ```

- 创建表时，列字段添加IDENTITY约束。

  ```
  CREATE TABLE IF NOT EXISTS tb4 (c1 int GENERATED ALWAYS AS IDENTITY (INCREMENT
  BY 2 MINVALUE 10 MAXVALUE 20 CYCLE SCALE)); -- 整条语句不解析
  ```

- B兼容模式下创建或修改表时，给表或列添加注释。

  原始SQL语句：

  ```
  gaussdb=# CREATE TABLE IF NOT EXISTS tb6 (c1 integer comment 'Mysql兼容注释语法')
  with (ORIENTATION=ROW, STORAGE_TYPE=USTORE);
  ```

  反解析结果：

  ```
  gaussdb=# CREATE TABLE IF NOT EXISTS public.tb6 (c1 pg_catalog.int4) WITH
  (storage_type = 'ustore', orientation = 'row', compression = 'no') NOCOMPRESS;
  ```

– 逻辑解码不支持DDL（DCL）/DML混合事务的，混合事务中DDL之后的DML解码不支持。

```
-- 均不反解析，DCL为不支持语句故不解析，DML处于DCL之后也不反解析
gaussdb=# BEGIN;
gaussdb=# GAINT ALL PRIVILEGES to u01;
gaussdb=# INSERT INTO test1(col1) values(1);
gaussdb=# COMMIT;

-- 只反解析第一句和第三句SQL语句
gaussdb=# BEGIN;
gaussdb=# CREATE TABLE mix_tran_t4(id int);
gaussdb=# INSERT INTO mix_tran_t4 VALUES(111);
gaussdb=# CREATE TABLE mix_tran_t5(id int);
gaussdb=# COMMIT;

-- 只反解析第一句和第二句SQL语句
gaussdb=# BEGIN;
gaussdb=# INSERT INTO mix_tran_t4 VALUES(111);
gaussdb=# CREATE TABLE mix_tran_t6(id int);
gaussdb=# INSERT INTO mix_tran_t4 VALUES(111);
gaussdb=# COMMIT;

-- 全反解析
gaussdb=# BEGIN;
gaussdb=# INSERT INTO mix_tran_t4 VALUES(111);
gaussdb=# CREATE TABLE mix_tran_t7(id int);
gaussdb=# CREATE TABLE mix_tran_t8(id int);
gaussdb=# COMMIT;

-- 只反解析第一句和第三句SQL语句
gaussdb=# BEGIN;
gaussdb=# CREATE TABLE mix_tran_t7(id int);
gaussdb=# CREATE TYPE compfoo AS (f1 int, f2 text);
gaussdb=# CREATE TABLE mix_tran_t8(id int);
gaussdb=# COMMIT;

-- 全反解析
gaussdb=# BEGIN;
gaussdb=# INSERT INTO mix_tran_t4 VALUES(111);
gaussdb=# INSERT INTO mix_tran_t4 VALUES(111);
gaussdb=# INSERT INTO mix_tran_t4 VALUES(111);
gaussdb=# COMMIT;
```

 版权所有 © 华为云计算技术有限公司

```
-- 只反解析第一句SQL语句
gaussdb=# BEGIN;
gaussdb=# INSERT INTO mix_tran_t4 VALUES(111);
gaussdb=# CREATE TYPE compfoo AS (f1 int, f2 text);
gaussdb=# INSERT INTO mix_tran_t4 VALUES(111);
gaussdb=# COMMIT;

-- 只反解析第一句和第三句SQL语句
gaussdb=# BEGIN;
gaussdb=# INSERT INTO mix_tran_t4 VALUES(111);
gaussdb=# CREATE TYPE compfoo AS (f1 int, f2 text);
gaussdb=# CREATE TABLE mix_tran_t9(id int);
gaussdb=# COMMIT;
```

  - 逻辑解码语句CREATE TABLE AS SELECT、SELECT INTO和CREATE TABLE AS仅能解码出CREATE TABLE语句，暂不支持解码INSERT语句。

    说明

    对于CTAS创建的表，仍会解码其ALTER和DROP语句。

    示例：

    原始SQL语句：

    ```
    CREATE TABLE IF NOT EXISTS tb35_2 (c1 int) with (storage_type=USTORE,ORIENTATION=ROW);
    INSERT INTO tb35_2 VALUES (6);
    CREATE TABLE tb35_1 with (storage_type=USTORE,ORIENTATION=ROW) AS SELECT * FROM tb35_2;
    ```

    最后一句SQL语句反解析结果：

    ```
     CREATE  TABLE  public.tb35_1 (c1 pg_catalog.int4) WITH (storage_type = 'ustore', orientation = 'row', compression = 'no') NOCOMPRESS;
    ```

  - 执行存储过程/函数/高级包时，若其本身包含DDL/DML混合事务或者其本身与同事务内其他语句组成DDL/DML混合事务，则按照混合事务原则执行解码。
  - 不支持账本数据库功能，当前版本如果有账本数据库的DDL操作，例如创表语句，解码的DDL语句中会解析出hash列，导致目的端与源端的数据库表结构不一致。
- 串行逻辑解码支持DDL特有约束：
  - sql_decoding插件不支持json格式的DDL。

解码格式

- JSON形式

  对于输入的DDL语句，SQL引擎解析器会通过语法、词法分析将其分解为解析树，解析树节点中包含了DDL的全部信息，并且执行器会根据解析树内容，执行系统元信息的修改。在执行器执行完成之后，便可以获取到DDL操作数据对象的search_path。本特性在执行器执行成功之后，对解析树信息以及执行器结果进行反解析，以还原出DDL原语句的全部信息。反解析的方式可以分解整个DDL语句，以方便输出JSON形式的DDL，用以适配异构数据库场景。

  CREATE TABLE语句在经过词法、语法分析之后，得到对应的CreateStmt解析树节点，节点中包含了表信息、列信息、分布式信息（DistributeBy结构体）、分区信息（PartitionState结构）等。通过反解析后，可输出的JSON形式如下：

  ```
  {"JDDL":{"fmt":"CREATE %{persistence}s TABLE %{if_not_exists}s %{identity}D %{table_elements}s %{with_clause}s %{compression}s","identity":{"object_name":"test_create_table_a","schema_name":"public"},"compression":"NOCOMPRESS","persistence":"","with_clause":{"fmt":"WITH (%{with:, }s)","with":[{"fmt":"%{label}s = %{value}L","label":{"fmt":"%{label}I","label":"orientation"},"value":"row"},{"fmt":"%{label}s = %{value}L","label":{"fmt":"%{label}I","label":"compression"},"value":"no"}]},"if_not_exists":"","table_elements":{"fmt":"(%{elements:, }s)","elements":[{"fmt":"%{name}I %{column_type}T","name":"a","column_type":{"typmod":"","typarray":false,"type_name":"int4","schema_name":"pg_catalog"}}]}}}
  ```

 版权所有 © 华为云计算技术有限公司

可以看到，JSON形式中包含对象的search_path，其中的identity键标识schema为public，表名为test_create_table_a，其中%{persistence}s对应的字段如下，此SQL语句不含此字段所以为空。

```
[ [ GLOBAL | LOCAL ] [ TEMPORARY | TEMP ] | UNLOGGED ]
```

%{if_not_exists}s对应SQL语句中的字段，不含此字段所以为空：

```
[ IF NOT EXISTS ]
```

%{identity}D对应SQL语句中的字段：

```
table_name
```

%{table_elements}s对应SQL语句中的字段：

```
( column_name data_type )
```

%{with_clause}s对应SQL语句中的字段：

```
[ WITH ( {storage_parameter = value} [, ... ] ) ]
```

%{compression}s对应SQL语句中的字段：

```
[ COMPRESS | NOCOMPRESS ]
```

- SQL形式

  输出的SQL形式（文本方式）为：

  ```
  {"TDDL":"CREATE  TABLE  public.test_create_table_a (a pg_catalog.int4) WITH (orientation = 'row', compression = 'no') NOCOMPRESS"}
  ```

  其中语句中也包含Schema名称。

接口设计

- 新增控制参数

  a. 新增逻辑解码控制参数，用于控制DDL的反解析流程以及输出形式。可通过pg_recvlogical -o或者pg_logical_slot_peek_changes开启。

     - enable-ddl-decoding：默认false，不开启DDL语句的逻辑解码；值为true时，开启DDL语句的逻辑解码。
     - enable-ddl-json-format：默认false，传送TEXT格式的DDL反解析结果；值为true时，传送JSON格式的DDL反解析结果。

  b. 新增GUC参数

     - enable_logical_replication_ddl：默认为ON，ON状态下，逻辑复制可支持DDL，否则，不支持DDL。只有当ON状态下，才会对DDL执行结果进行反解析，并生成DDL的WAL日志。否则，不反解析也不生成WAL日志。

       enable_logical_replication_ddl的开关日志，以证明是否是用户修改了该参数导致逻辑解码不支持DDL。

- 新增日志

  新增DDL日志xl_logical_ddl_message，其类型为RM_LOGICALDDLMSG_ID。其定义如下：

名称	类型	意义
db_id	Oid	数据库ID
rel_id	Oid	表ID
csn	CommitSeqNo	CSN快照

版权所有 © 华为云计算技术有限公司

名称	类型	意义
cid	CommandId	Command ID
tag_type	NodeTag	DDL类型
message_size	Size	日志内容长度
filter_message_size	Size	日志中白名单过滤信息长度
message	char *	DDL内容

使用步骤

步骤1 逻辑解码特性需提前设置GUC参数wal_level为logical，该参数需要重启生效。

```
gs_guc set -Z datanode -D $node_dir -c "wal_level = logical"
```

其中，$node_dir为数据库节点路径，用户可根据实际情况替换。

步骤2 以具有REPLICATION权限的用户登录GaussDB数据库主节点，使用如下命令连接数据库。

```
gsql -U user1 -W password -d db1 -p 16000 -r
```

其中，user1为用户名，password为密码，db1为需要连接的数据库名称，16000为数据库端口号，用户可根据实际情况替换。

步骤3 创建名称为slot1的逻辑复制槽。

```
db1=>SELECT * FROM pg_create_logical_replication_slot('slot1', 'mppdb_decoding');
 slotname | xlog_position
----------+---------------
 slot1    | 0/3764C788
(1 row)
```

说明

串行逻辑解码支持mppdb_decoding和sql_decoding，并行逻辑解码只支持mppdb_decoding。

步骤4 在数据库中创建Package。

```
db1=> CREATE OR REPLACE PACKAGE ldp_pkg1 IS
   var1 int:=1;  --公有变量
   var2 int:=2;
   PROCEDURE testpro1(var3 int);  --公有存储过程，可以被外部调用
END ldp_pkg1;
/
```

步骤5 读取复制槽slot1解码结果，可通过pg_recvlogical -o或者pg_logical_slot_peek_changes推进复制槽。

说明

- 逻辑解码选项请参见**逻辑解码选项**和新增控制参数。
- 并行解码中，在pg_recvlogical中改变参数decode_style可以决定解码格式：
  - -o decode_style='b'：DDL分解析为bin格式，
  - -o decode_style='j'：DDL分解析为json格式，
  - -o decode_style='t'：DDL分解析为text格式。

```
db1=> SELECT data FROM pg_logical_slot_peek_changes('slot1', NULL, NULL, 'enable-ddl-decoding', 'true',
'enable-ddl-json-format', 'false') WHERE data not like 'BEGIN%' AND data not like 'COMMIT%' AND data
```


版权所有 © 华为云计算技术有限公司

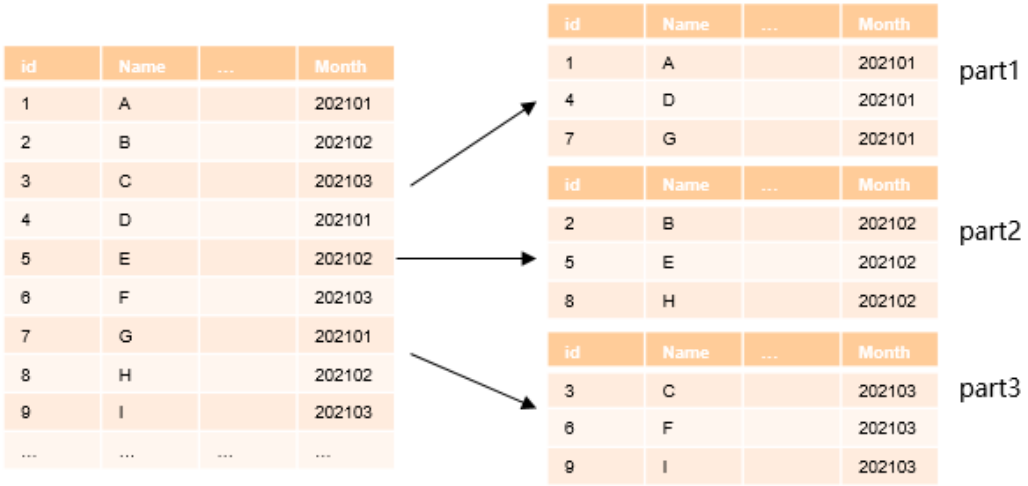
```
not like '%dbe_pldeveloper.gs_source%';
                                                
```

1. 提升大容量数据场景查询效率：由于表内数据按照分区键进行逻辑分区，查询结果可以通过访问分区的子集而不是整个表来实现。这种分区剪枝技术可以提供数量级的性能增益。
2. 降低运维与查询的并发操作影响：降低DML语句、DDL语句并发场景的相互影响，在对一些大数据量以时间维度进行分区的场景下会明显受益。例如，新数据分区进行入库、实时点查操作，老数据分区进行数据清洗、分区合并等运维性质操作。
3. 提供大容量场景下灵活的数据运维管理方式：由于分区表从物理上对不同分区的数据做了表文件层面的隔离，每个分区可以具有单独的物理属性，如启用或禁用压缩、物理存储设置和表空间。同时它支持数据管理操作，如数据加载、索引创建和重建，以及分区级别的备份和恢复，而不是对整个表进行操作，从而减少了操作时间。

2.8.1.3 数据分区查找优化

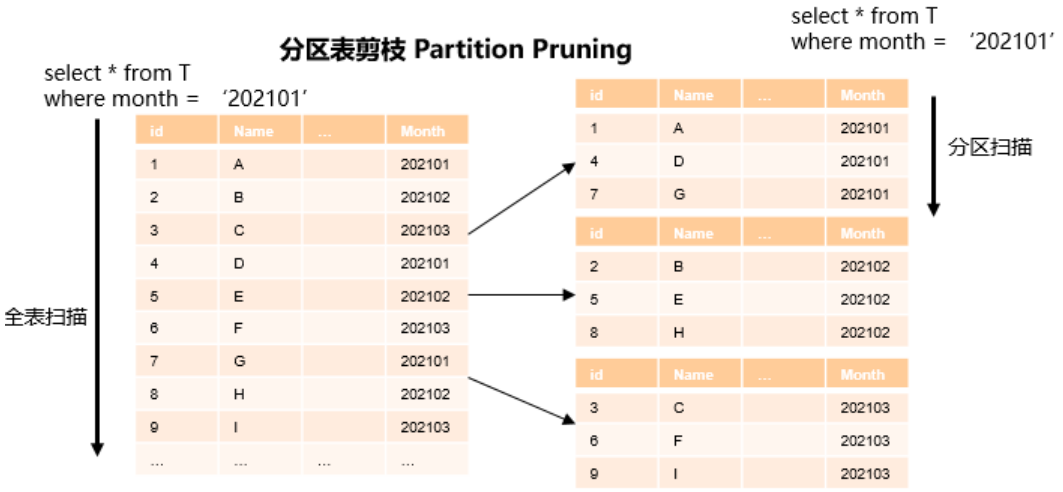
分区表对数据查找方面的帮助主要体现在对分区键进行谓词查询场景，例如一张以月份Month作为分区键的表，如图2-2所示。如果以普通表的方式设计表结构则需要访问表全量的数据（Full Table Scan），如果以日期为分区键重新设计该表，那么原有的全表扫描会被优化成为分区扫描。当表内的数据量很大同时具有很长的历史周期时，由于扫描数据缩减所带来的性能提升会有明显的效果，如图2-3所示。

图 2-2 分区表示例图



版权所有 © 华为云计算技术有限公司

图 2-3 分区表剪枝示例图



2.8.1.4 数据分区运维管理

分区表技术为数据生命周期管理（Data Life Cycle Management，DLM）提供了灵活性的支持，数据生命周期管理是一组用于在数据的整个使用寿命中管理数据的过程和策略。其中一个重要组成部分是确定在数据生命周期的任何时间点存储数据的最合适和最经济高效的介质：日常操作中使用的较新数据存储在最快、可用性最高的存储层上，而不经常访问的较旧数据可能存储在成本较低、效率较低的存储层。较旧的数据也可能更新的频率较低，因此将数据压缩并存储为只读是有意义的。

分区表为实施DLM解决方案提供了理想的环境，通过不同分区使用不同表空间，最大限度在确保易用性的同时，实现了有效的数据生命周期的成本优化。这部分的设置由数据库运维人员在服务端设置操作完成，实际用户并不感知这一层面的优化设置，对用户而言逻辑上仍然是对同一张表的查询操作。此外不同分区可以分别实施备份、恢复、索引重建等运维性质的操作，能够对单个数据集不同子类进行分治操作，满足用户业务场景的差异化需求。

2.8.2 分区表介绍

分区表（Partitioned Table）指在单节点内对表数据内容按照分区键以及围绕分区键的分区策略对表进行逻辑切分。从数据分区的角度来看是一种水平分区（horizontal partition）策略方式。分区表增强了数据库应用程序的性能、可管理性和可用性，并有助于降低存储大量数据的总体拥有成本。分区允许将表、索引和索引组织的表细分为更小的部分，使这些数据库对象能够在更精细的粒度级别上进行管理和访问。GaussDB提供了丰富的分区策略和扩展，以满足不同业务场景的需求。由于分区策略的实现完全由数据库内部实现，对用户是完全透明的，因此它几乎可以在实施分区表优化策略以后做平滑迁移，无需潜在耗费人力物力的应用程序更改。本章围绕GaussDB分区表的基本概念从以下几个方面展开介绍：

1. 分区表基本概念：从表分区的基本概念出发，介绍分区表的catalog存储方式以及内部对应原理。
2. 分区策略：从分区表所支持的基本类型出发，介绍各种分区模式下对应的特性以及能够达到的优化特点和效果。

2.8.2.1 基本概念

版权所有 © 华为云计算技术有限公司

2.8.2.1.1 分区表（母表）

实际对用户体现的表，用户对该表进行常规DML语句的增、删、查、改操作。通常使用在建表DDL语句显式的使用PARTITION BY语句进行定义，创建成功以后在pg_class表中新增一个entry，并且parttype列内容为'p'（一级分区）或者's'（二级分区），表明该entry为分区表的母表。分区母表通常是一个逻辑形态，对应的表文件并不存放数据。

示例1：t1_hash为一个一级分区表，分区类型为hash：

```
gaussdb=# CREATE TABLE t1_hash (c1 INT, c2 INT, c3 INT)
PARTITION BY HASH(c1)
(
    PARTITION p0,
    PARTITION p1,
    PARTITION p2,
    PARTITION p3,
    PARTITION p4,
    PARTITION p5,
    PARTITION p6,
    PARTITION p7,
    PARTITION p8,
    PARTITION p9
);

gaussdb=# \d+ t1_hash
                    Table "public.t1_hash"
 Column |  Type   | Modifiers | Storage | Stats target | Description
--------+---------+-----------+---------+--------------+-------------
 c1     | integer |           | plain   |              |
 c2     | integer |           | plain   |              |
 c3     | integer |           | plain   |              |
Partition By HASH(c1)
Number of partitions: 10 (View pg_partition to check each partition range.)
Has OIDs: no
Options: orientation=row, compression=no, storage_type=USTORE, segment=off

--查询t1_hash分区类型
gaussdb=# SELECT relname, parttype FROM pg_class WHERE relname = 't1_hash';
 relname | parttype
---------+----------
 t1_hash | p
(1 row)

gaussdb=# DROP TABLE t1_hash;
```

示例2：t1_sub_rr为一个二级分区表，分区类型为range-list：

```
gaussdb=# CREATE TABLE t1_sub_rr (
    c1 INT,
    c2 INT,
    c3 INT
)
PARTITION BY RANGE (c1)
SUBPARTITION BY LIST (c2)
(
    PARTITION p_2021 VALUES LESS THAN (2022) (
        SUBPARTITION p_2021_1 VALUES (1),
        SUBPARTITION p_2021_2 VALUES (2),
        SUBPARTITION p_2021_3 VALUES (3)
    ),
    PARTITION p_2022 VALUES LESS THAN (2023) (
        SUBPARTITION p_2022_1 VALUES (1),
        SUBPARTITION p_2022_2 VALUES (2),
        SUBPARTITION p_2022_3 VALUES (3)
    ),
    PARTITION p_2023 VALUES LESS THAN (2024) (
        SUBPARTITION p_2023_1 VALUES (1),
        SUBPARTITION p_2023_2 VALUES (2),
        SUBPARTITION p_2023_3 VALUES (3)
```

 版权所有 © 华为云计算技术有限公司

```
    ),
    PARTITION p_2024 VALUES LESS THAN (2025) (
        SUBPARTITION p_2024_1 VALUES (1),
        SUBPARTITION p_2024_2 VALUES (2),
        SUBPARTITION p_2024_3 VALUES (3)
    ),
    PARTITION p_2025 VALUES LESS THAN (2026) (
        SUBPARTITION p_2025_1 VALUES (1),
        SUBPARTITION p_2025_2 VALUES (2),
        SUBPARTITION p_2025_3 VALUES (3)
    ),
    PARTITION p_2026 VALUES LESS THAN (2027) (
        SUBPARTITION p_2026_1 VALUES (1),
        SUBPARTITION p_2026_2 VALUES (2),
        SUBPARTITION p_2026_3 VALUES (3)
    )
);


gaussdb=# \d+ t1_sub_rr
                  Table "public.t1_sub_rr"
 Column |  Type   | Modifiers | Storage | Stats target | Description
--------+---------+-----------+---------+--------------+-------------
 c1     | integer |           | plain   |              |
 c2     | integer |           | plain   |              |
 c3     | integer |           | plain   |              |
Partition By RANGE(c1) Subpartition By LIST(c2)
Number of partitions: 6 (View pg_partition to check each partition range.)
Number of subpartitions: 18 (View pg_partition to check each subpartition range.)
Has OIDs: no
Options: orientation=row, compression=no, storage_type=USTORE, segment=off

--查询t1_sub_rr分区类型
gaussdb=# SELECT relname, parttype FROM pg_class WHERE relname = 't1_sub_rr';
  relname  | parttype
-----------+----------
 t1_sub_rr | s
(1 row)

gaussdb=# DROP TABLE t1_sub_rr;
```

2.8.2.1.2 分区（分区子表、子分区）

分区表中实际保存数据的表，对应的entry通常保存在pg_partition中，各个子分区的parentid作为外键关联其分区母表在pg_class表中的OID列。

示例：t1_hash为一个一级分区表：

```
gaussdb=# CREATE TABLE t1_hash (c1 INT, c2 INT, c3 INT)
PARTITION BY HASH(c1)
(
    PARTITION p0,
    PARTITION p1,
    PARTITION p2,
    PARTITION p3,
    PARTITION p4,
    PARTITION p5,
    PARTITION p6,
    PARTITION p7,
    PARTITION p8,
    PARTITION p9
);

--查询t1_hash分区类型
gaussdb=# SELECT oid, relname, parttype FROM pg_class WHERE relname = 't1_hash';
  oid  | relname | parttype
-------+---------+----------
 16685 | t1_hash | p
(1 row)
```

 版权所有 © 华为云计算技术有限公司

```
--查询t1_hash的分区信息
gaussdb=# SELECT oid, relname, parttype, parentid FROM pg_partition WHERE parentid = 16685;
  oid  | relname | parttype | parentid
-------+---------+----------+----------
 16688 | t1_hash | r        |    16685
 16689 | p0      | p        |    16685
 16690 | p1      | p        |    16685
 16691 | p2      | p        |    16685
 16692 | p3      | p        |    16685
 16693 | p4      | p        |    16685
 16694 | p5      | p        |    16685
 16695 | p6      | p        |    16685
 16696 | p7      | p        |    16685
 16697 | p8      | p        |    16685
 16698 | p9      | p        |    16685
(11 rows)

gaussdb=# DROP TABLE t1_hash;
```

2.8.2.1.3 分区键

分区键由一个或多个列组成，分区键值结合对应分区方法能够唯一确定某一元组所在的分区，通常在建表时通过PARTITION BY语句指定：

```
CREATE TABLE table_name (…) PARTITION BY part_strategy (partition_key) (…)
```

须知

范围分区表和列表分区表支持最多16列分区键，其他分区表只支持1列分区键。

2.8.2.2 分区策略

分区策略在使用DDL语句建表语句时通过PARTITION BY语句的语法指定，分区策略描述了在分区表中数据和分区路由映射规则。常见的分区类型有基于条件的Range分区/Interval分区、基于哈希散列函数的Hash分区、基于数据枚举的List列表分区：

```
CREATE TABLE table_name (…) PARTITION BY partition_strategy (partition_key) (…)
```

2.8.2.2.1 范围分区

范围分区（Range Partition）根据为每个分区建立分区键的值范围将数据映射到分区。范围分区是生产系统中最常见的分区类型，通常在以时间维度（Date、Time Stamp）描述数据场景中使用。范围分区有两种语法格式，示例如下：

1. VALUES LESS THAN的语法格式

   对于从句是VALUE LESS THAN的语法格式，范围分区策略的分区键最多支持16列。

   - 单列分区键示例如下：

     ```
     gaussdb=# CREATE TABLE range_sales_single_key
     (
        product_id      INT4 NOT NULL,
        customer_id     INT4 NOT NULL,
        time            DATE,
        channel_id      CHAR(1),
        type_id         INT4,
        quantity_sold   NUMERIC(3),
        amount_sold     NUMERIC(10,2)
     )
     PARTITION BY RANGE (time)
     (
     ```

 版权所有 © 华为云计算技术有限公司

```
    PARTITION date_202001 VALUES LESS THAN ('2020-02-01'),
    PARTITION date_202002 VALUES LESS THAN ('2020-03-01'),
    PARTITION date_202003 VALUES LESS THAN ('2020-04-01'),
    PARTITION date_202004 VALUES LESS THAN ('2020-05-01')
);
gaussdb=# DROP TABLE range_sales_single_key;
```

其中date_202002表示2020年2月的分区，将包含分区键值从2020年2月1日到2020年2月29日的数据。

每个分区都有一个VALUES LESS子句，用于指定分区的非包含上限。大于或等于该分区键的任何值都将添加到下一个分区。除第一个分区外，所有分区都具有由前一个分区的VALUES LESS子句指定的隐式下限。可以为最高分区定义MAXVALUE关键字，MAXVALUE表示一个虚拟无限值，其排序高于分区键的任何其他可能值，包括空值。

- 多列分区键示例如下：

```
gaussdb=# CREATE TABLE range_sales
(
    c1      INT4 NOT NULL,
    c2      INT4 NOT NULL,
    c3      CHAR(1)
)
PARTITION BY RANGE (c1,c2)
(
    PARTITION p1 VALUES LESS THAN (10,10),
    PARTITION p2 VALUES LESS THAN (10,20),
    PARTITION p3 VALUES LESS THAN (20,10)
);
INSERT INTO range_sales VALUES(9,5,'a');
INSERT INTO range_sales VALUES(9,20,'a');
INSERT INTO range_sales VALUES(9,21,'a');
INSERT INTO range_sales VALUES(10,5,'a');
INSERT INTO range_sales VALUES(10,15,'a');
INSERT INTO range_sales VALUES(10,20,'a');
INSERT INTO range_sales VALUES(10,21,'a');
INSERT INTO range_sales VALUES(11,5,'a');
INSERT INTO range_sales VALUES(11,20,'a');
INSERT INTO range_sales VALUES(11,21,'a');

gaussdb=# SELECT * FROM range_sales PARTITION (p1);
 c1 | c2 | c3
----+----+----
  9 |  5 | a
  9 | 20 | a
  9 | 21 | a
 10 |  5 | a
(4 rows)

gaussdb=# SELECT * FROM range_sales PARTITION (p2);
 c1 | c2 | c3
----+----+----
 10 | 15 | a
(1 row)

gaussdb=# SELECT * FROM range_sales PARTITION (p3);
 c1 | c2 | c3
----+----+----
 10 | 20 | a
 10 | 21 | a
 11 |  5 | a
 11 | 20 | a
 11 | 21 | a
(5 rows)

gaussdb=# DROP TABLE range_sales;
```

 版权所有 © 华为云计算技术有限公司

说明

多列分区的分区规则如下：

1. 从第一列开始比较。
2. 如果插入的当前列小于分区当前列边界值，则直接插入。
3. 如果插入的当前列等于分区当前列的边界值，则比较插入值的下一列与分区下一列边界值的大小，同规则**1.ii**~**1.iv**。
4. 如果插入的当前列大于分区当前列的边界值，则换下一个分区进行比较。

2. START END语法格式

对于从句是START END语法格式，范围分区策略的分区键最多支持1列。

示例如下：

```
gaussdb=#
-- 创建表空间
CREATE TABLESPACE startend_tbs1 LOCATION '/home/omm/startend_tbs1';
CREATE TABLESPACE startend_tbs2 LOCATION '/home/omm/startend_tbs2';
CREATE TABLESPACE startend_tbs3 LOCATION '/home/omm/startend_tbs3';
CREATE TABLESPACE startend_tbs4 LOCATION '/home/omm/startend_tbs4';
-- 创建临时schema
CREATE SCHEMA tpcds;
SET CURRENT_SCHEMA TO tpcds;
-- 创建分区表，分区键是integer类型
CREATE TABLE tpcds.startend_pt (c1 INT, c2 INT)
TABLESPACE startend_tbs1
PARTITION BY RANGE (c2) (
    PARTITION p1 START(1) END(1000) EVERY(200) TABLESPACE startend_tbs2,
    PARTITION p2 END(2000),
    PARTITION p3 START(2000) END(2500) TABLESPACE startend_tbs3,
    PARTITION p4 START(2500),
    PARTITION p5 START(3000) END(5000) EVERY(1000) TABLESPACE startend_tbs4
)
ENABLE ROW MOVEMENT;

-- 查看分区表信息
gaussdb=# SELECT relname, boundaries, spcname FROM pg_partition p JOIN pg_tablespace t ON
    p.reltablespace=t.oid and p.parentid='tpcds.startend_pt'::regclass ORDER BY 1;
     relname | boundaries | spcname
-------------+------------+--------------
       p1_0 | {1}        | startend_tbs2
       p1_1 | {201}      | startend_tbs2
       p1_2 | {401}      | startend_tbs2
       p1_3 | {601}      | startend_tbs2
       p1_4 | {801}      | startend_tbs2
       p1_5 | {1000}     | startend_tbs2
         p2 | {2000}     | startend_tbs1
         p3 | {2500}     | startend_tbs3
         p4 | {3000}     | startend_tbs1
       p5_1 | {4000}     | startend_tbs4
       p5_2 | {5000}     | startend_tbs4
 startend_pt |            | startend_tbs1
(12 rows)

--清理示例
gaussdb=# DROP TABLE tpcds.startend_pt;
```

2.8.2.2.2 间隔分区

间隔分区（Interval Partition）可以看成是范围分区的一种增强和扩展方式，相比之下间隔分区定义分区时无需为新增的每个分区指定上限和下限值，只需要确定每个分区的长度，实际插入的过程中会自动进行分区的创建和扩展。间隔分区在创建初始时必须至少指定一个范围分区，范围分区键值确定范围分区的高值称为转换点，数据库为值超出该转换点的数据自动创建间隔分区。每个区间分区的下边界是先前范围或区间分区的非包容性上边界。示例如下：

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# CREATE TABLE interval_sales
(
    prod_id       NUMBER(6),
    cust_id       NUMBER,
    time_id       DATE,
    channel_id    CHAR(1),
    promo_id      NUMBER(6),
    quantity_sold NUMBER(3),
    amount_sold   NUMBER(10, 2)
)
PARTITION BY RANGE (time_id) INTERVAL ('1 month')
(
    PARTITION date_2015 VALUES LESS THAN ('2016-01-01'),
    PARTITION date_2016 VALUES LESS THAN ('2017-01-01'),
    PARTITION date_2017 VALUES LESS THAN ('2018-01-01'),
    PARTITION date_2018 VALUES LESS THAN ('2019-01-01'),
    PARTITION date_2019 VALUES LESS THAN ('2020-01-01')
);
gaussdb=# DROP TABLE interval_sales;
```

上述例子中，初始创建分区以2015年到2019年以年为单位创建分区，当数据插入到2020-01-01以后的数据时，由于超过的预先定义Range分区的上边界，会自动创建一个分区。

⚠ 注意

间隔分区仅支持数值类型和日期/时间类型，不支持字符类型和其他类型。支持类型白名单如下：

INT1/UINT1、INT2/UINT2、INT4/UINT4、INT8/UINT8、FLOAT4、FLOAT8、NUMERIC、DATE、TIMESTAMP、TIMESTAMP WITH TIME ZONE。

2.8.2.2.3 哈希分区

哈希分区（Hash Partition）基于对分区键使用哈希算法将数据映射到分区。使用的哈希算法为GaussDB内置哈希算法，在分区键取值范围不倾斜（no data skew）的场景下，哈希算法在分区之间均匀分布行，使分区大小大致相同。因此哈希分区是实现分区间均匀分布数据的理想方法。哈希分区也是范围分区的一种易于使用的替代方法，尤其是当要分区的数据不是历史数据或没有明显的分区键时，示例如下：

```
CREATE TABLE bmsql_order_line (
    ol_w_id         INTEGER   NOT NULL,
    ol_d_id         INTEGER   NOT NULL,
    ol_o_id         INTEGER   NOT NULL,
    ol_number       INTEGER   NOT NULL,
    ol_i_id         INTEGER   NOT NULL,
    ol_delivery_d   TIMESTAMP,
    ol_amount       DECIMAL(6,2),
    ol_supply_w_id  INTEGER,
    ol_quantity     INTEGER,
    ol_dist_info    CHAR(24)
)
--预先定义100个分区
PARTITION BY HASH(ol_d_id)
(
    PARTITION p0,
    PARTITION p1,
    PARTITION p2,
    ...
    PARTITION p99
);
```

上述例子中，bmsql_order_line表的ol_d_id进行了分区，ol_d_id列是一个identifier性质的属性列，本身并不带有时间或者某一个特定维度上的区分。使用哈希分区策略来

版权所有 © 华为云计算技术有限公司

对其进行分表处理则是一个较为理想的选择。相比其他分区类型，除了预先确保分区键没有过多数据倾斜（某一、某几个值重复度高），只需要指定分区键和分区数即可创建分区，同时还能够确保每个分区的数据均匀，提升了分区表的易用性。

2.8.2.2.4 列表分区

列表分区（List Partition）能够通过在每个分区的描述中为分区键指定离散值列表来显式控制行如何映射到分区。列表分区的优势在于可以以枚举分区值方式对数据进行分区，可以对无序和不相关的数据集进行分组和组织。对于未定义在列表中的分区键值，可以使用默认分区（DEFAULT）来进行数据的保存，这样所有未映射到任何其他分区的行都不会生成错误。示例如下：

```
gaussdb=# CREATE TABLE bmsql_order_line (
    ol_w_id         INTEGER   NOT NULL,
    ol_d_id         INTEGER   NOT NULL,
    ol_o_id         INTEGER   NOT NULL,
    ol_number       INTEGER   NOT NULL,
    ol_i_id         INTEGER   NOT NULL,
    ol_delivery_d   TIMESTAMP,
    ol_amount       DECIMAL(6,2),
    ol_supply_w_id  INTEGER,
    ol_quantity     INTEGER,
    ol_dist_info    CHAR(24)
)
PARTITION BY LIST(ol_d_id)
(
    PARTITION p0 VALUES (1,4,7),
    PARTITION p1 VALUES (2,5,8),
    PARTITION p2 VALUES (3,6,9),
    PARTITION p3 VALUES (DEFAULT)
);
gaussdb=# DROP TABLE bmsql_order_line;
```

上述例子和之前给出的哈希分区的例子类似，同样通过ol_d_id列进行分区，但是在List分区中直接通过对ol_d_id的可能取值范围进行限定，不在列表中的数据会进入p3分区（DEFAULT）。相比哈希分区，List列表分区对分区键的可控性更好，往往能够准确的将目标数据保存在预想的分区中，但是如果列表值较多在分区定义时变得麻烦，该情况下推荐使用Hash分区。List、Hash分区往往都是处理无序、不相关的数据集进行分组和组织。

⚠ 注意

列表分区的分区键最多支持16列。如果分区键定义为1列，子分区定义时List列表中的枚举值不允许为NULL值；如果分区键定义为多列，子分区定义时List列表中的枚举值允许有NULL值。

2.8.2.2.5 二级分区

二级分区（Sub Partition，也叫组合分区）是基本数据分区类型的组合，将表通过一种数据分布方法进行分区，然后使用第二种数据分布方式将每个分区进一步细分为子分区。给定分区的所有子分区表示数据的逻辑子集。常见的二级分区组合如下所示：

1. Range-Range
2. Range-List
3. Range-Hash
4. List-Range

版权所有 © 华为云计算技术有限公司

5. List-List
6. List-Hash
7. Hash-Range
8. Hash-List
9. Hash-Hash

示例如下：

```
gaussdb=#
--Range-Range
CREATE TABLE t_range_range (
    c1 INT,
    c2 INT,
    c3 INT
)
PARTITION BY RANGE (c1)
SUBPARTITION BY RANGE (c2)
(
    PARTITION p1 VALUES LESS THAN (10) (
        SUBPARTITION p1sp1 VALUES LESS THAN (5),
        SUBPARTITION p1sp2 VALUES LESS THAN (10)
    ),
    PARTITION p2 VALUES LESS THAN (20) (
        SUBPARTITION p2sp1 VALUES LESS THAN (15),
        SUBPARTITION p2sp2 VALUES LESS THAN (20)
    )
);
DROP TABLE t_range_range;

--Range-List
CREATE TABLE t_range_list (
    c1 INT,
    c2 INT,
    c3 INT
)
PARTITION BY RANGE (c1)
SUBPARTITION BY LIST (c2)
(
    PARTITION p1 VALUES LESS THAN (10) (
        SUBPARTITION p1sp1 VALUES (1, 2),
        SUBPARTITION p1sp2 VALUES (3, 4)
    ),
    PARTITION p2 VALUES LESS THAN (20) (
        SUBPARTITION p2sp1 VALUES (1, 2),
        SUBPARTITION p2sp2 VALUES (3, 4)
    )
);
DROP TABLE t_range_list;

--Range-Hash
CREATE TABLE t_range_hash (
    c1 INT,
    c2 INT,
    c3 INT
)
PARTITION BY RANGE (c1)
SUBPARTITION BY HASH (c2)
SUBPARTITIONS 2
(
    PARTITION p1 VALUES LESS THAN (10),
    PARTITION p2 VALUES LESS THAN (20)
);
DROP TABLE t_range_hash;

--List-Range
CREATE TABLE t_list_range (
    c1 INT,
```

 版权所有 © 华为云计算技术有限公司

```
    c2 INT,
    c3 INT
)
PARTITION BY LIST (c1)
SUBPARTITION BY RANGE (c2)
(
    PARTITION p1 VALUES (1, 2) (
        SUBPARTITION p1sp1 VALUES LESS THAN (5),
        SUBPARTITION p1sp2 VALUES LESS THAN (10)
    ),
    PARTITION p2 VALUES (3, 4) (
        SUBPARTITION p2sp1 VALUES LESS THAN (5),
        SUBPARTITION p2sp2 VALUES LESS THAN (10)
    )
);
DROP TABLE t_list_range;

--List-List
CREATE TABLE t_list_list (
    c1 INT,
    c2 INT,
    c3 INT
)
PARTITION BY LIST (c1)
SUBPARTITION BY LIST (c2)
(
    PARTITION p1 VALUES (1, 2) (
        SUBPARTITION p1sp1 VALUES (1, 2),
        SUBPARTITION p1sp2 VALUES (3, 4)
    ),
    PARTITION p2 VALUES (3, 4) (
        SUBPARTITION p2sp1 VALUES (1, 2),
        SUBPARTITION p2sp2 VALUES (3, 4)
    )
);
DROP TABLE t_list_list;

--List-Hash
CREATE TABLE t_list_hash (
    c1 INT,
    c2 INT,
    c3 INT
)
PARTITION BY LIST (c1)
SUBPARTITION BY HASH (c2)
SUBPARTITIONS 2
(
    PARTITION p1 VALUES (1, 2),
    PARTITION p2 VALUES (3, 4)
);
DROP TABLE t_list_hash;

--Hash-Range
CREATE TABLE t_hash_range (
    c1 INT,
    c2 INT,
    c3 INT
)
PARTITION BY HASH (c1)
PARTITIONS 2
SUBPARTITION BY RANGE (c2)
(
    PARTITION p1 (
        SUBPARTITION p1sp1 VALUES LESS THAN (5),
        SUBPARTITION p1sp2 VALUES LESS THAN (10)
    ),
    PARTITION p2 (
        SUBPARTITION p2sp1 VALUES LESS THAN (5),
        SUBPARTITION p2sp2 VALUES LESS THAN (10)
```

 版权所有 © 华为云计算技术有限公司

```
    )
);
DROP TABLE t_hash_range;

--Hash-List
CREATE TABLE t_hash_list (
    c1 INT,
    c2 INT,
    c3 INT
)
PARTITION BY HASH (c1)
PARTITIONS 2
SUBPARTITION BY LIST (c2)
(
    PARTITION p1 (
        SUBPARTITION p1sp1 VALUES (1, 2),
        SUBPARTITION p1sp2 VALUES (3, 4)
    ),
    PARTITION p2 (
        SUBPARTITION p2sp1 VALUES (1, 2),
        SUBPARTITION p2sp2 VALUES (3, 4)
    )
);
DROP TABLE t_hash_list;

--Hash-Hash
CREATE TABLE t_hash_hash (
    c1 INT,
    c2 INT,
    c3 INT
)
PARTITION BY HASH (c1)
PARTITIONS 2
SUBPARTITION BY HASH (c2)
SUBPARTITIONS 2
(
    PARTITION p1,
    PARTITION p2
);
DROP TABLE t_hash_hash;
```

⚠ 注意

Interval分区看成是范围分区的一种特殊形式，目前不支持二级分区场景中定义Interval分区。

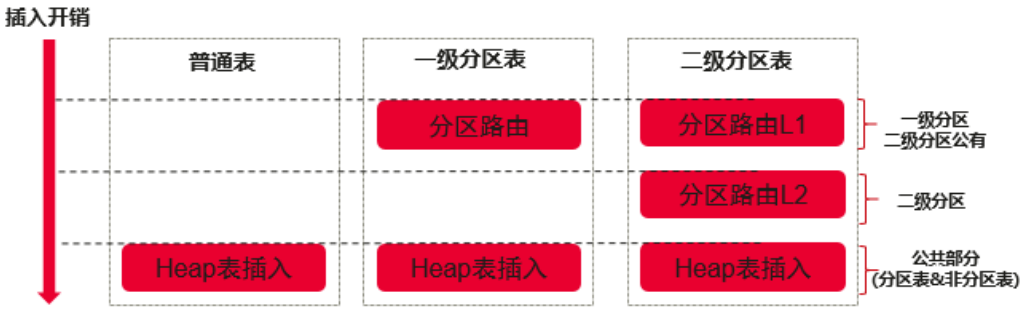
二级分区表的一级分区和二级分区分区键均只支持1列。

2.8.2.2.6 分区表对导入操作的性能影响

在GaussDB内核实现中，分区表数据插入的处理过程相比非分区表增加分区路由部分的开销，因从整体上分区表场景的数据插入开销主要看成：（1）heap-insert基表插入；（2）partition-routing分区路由两个部分，如图2-4所示。其中heap基表插入解决tuple入库对应heap表的问题并且该部分普通表和分区表共用，而分区路由部分解决分区路由即tuple元组插入到对应partRel的问题，并且分区路由算法本身作为一级、二级分区共用，不同之处在于二级分区相比一级分区多一层路由操作，对路由算法为两次调用。

版权所有 © 华为云计算技术有限公司

图 2-4 普通表&分区表数据插入



因此对数据插入优化的侧重点如下：

1. 分区表基表Heap表插入：
   a. 算子底噪优化
   b. heap数据插入
   c. 索引插入build优化（带索引）
2. 分区表分区路由：
   a. 路由查找算法逻辑优化
   b. 路由底噪优化，包括分区表partRel句柄开启、新增的函数调用逻辑开销

> **说明**
>
> 分区路由的性能主要通过大数据量的单条INSERT语句体现，UPDATE场景内部包含了查找对应要更新的元组进行DELETE操作然后再进行INSERT，因此不如单条INSERT语句场景直接。

不同分区类型的路由算法逻辑如表2-6所示：

表 2-6 路由算法逻辑

分区方式	路由算法复杂度	实现概述说明
范围分区（Range Partition）	O(logN)	基于二分binary-search实现
间隔分区（Interval Partition）	O(logN)	基于二分binary-search实现
哈希分区（Hash-Partition）	O(1)	基于key-partOid哈希表实现
列表分区（List-Partition）	O(1)	基于key-partOid哈希表实现
二级分区（List/List）	O(1) + O(1)	哈希+哈希
二级分区（List/Range）	O(1) + O(1) = O(1)	哈希+二分查找
二级分区（List/Hash）	O(1) + O(1) = O(1)	哈希+哈希

版权所有 © 华为云计算技术有限公司

分区方式	路由算法复杂度	实现概述说明
二级分区（Range/List）	O(1) + O(1) = O(1)	二分查找+哈希
二级分区（Range/Range）	O(1) + O(1) = O(1)	二分查找+二分查找
二级分区（Range/Hash）	O(1) + O(1) = O(1)	二分查找+哈希
二级分区（Hash/List）	O(1) + O(1) = O(1)	哈希+哈希
二级分区（Hash/Range）	O(1) + O(1) = O(1)	哈希+二分查找
二级分区（Hash/Hash）	O(1) + O(1) = O(1)	哈希+哈希

⚠ 注意

分区路由的主要处理逻辑根据导入数据元组的分区键计算其所在分区的过程，相比非分区表这部分为额外增加的开销，这部分开销在最终数据导入上的具体性能损失和服务器CPU处理能力、表宽度、磁盘/内存的实际容量相关，通常可以粗略认为：

- x86服务器场景下一级分区表相比普通表的导入性能会略低10%以内，二级分区表比普通表略低20%以内。
- ARM服务器场景下为20%、30%，造成x86和ARM指向性能略微差异的主要原因是分区路由为in-memory计算强化场景，主流x86体系CPU在单核指令处理能力上略优于ARM。

2.8.2.3 分区基本使用

2.8.2.3.1 创建分区表

创建普通分区表（创建一级分区表）

由于SQL语言功能强大和灵活多样性，SQL语法树通常比复杂，分区表同样如此，分区表的创建可以理解成在原有非分区表的基础上新增表分区属性，因此分区表的语法接口可以看成是对原有非分区表CREATE TABLE语句进行扩展PARTITION BY语句部分，同时指定分区相关的三个核元素：

1. 分区类型（partType）：描述分区表的分区策略，分别有RANGE/INTERVAL/LIST/HASH。
2. 分区键（partKey）：描述分区表的分区列，目前RANGE/LIST分区支持多列（不超过16列）分区键，INTERVAL/HASH分区只支持单列分区。
3. 分区表达式（partExpr）：描述分区表的具体分区表方式，即键值与分区的对应映射关系。

版权所有 © 华为云计算技术有限公司

这三部分重要元素在建表语句的Partition By Clause子句中体现，*PARTITION BY partType* (*partKey*) (*partExpr[,partExpr]*…) 。示例如下：

```
CREATE TABLE [ IF NOT EXISTS ] partition_table_name
(
    [ /* 该部份继承于普通表的Create Table */
    { column_name data_type [ COLLATE collation ] [ column_constraint [ ... ] ]
    | table_constraint
    | LIKE source_table [ like_option [...] ] }[, ... ]
    ]
)
[ WITH ( {storage_parameter = value} [, ... ] ) ]
[ COMPRESS | NOCOMPRESS ]
[ TABLESPACE tablespace_name ]
/* 范围分区场景，若申明INTERVAL子句则为间隔分区场景 */
PARTITION BY RANGE (partKey) [ INTERVAL ('interval_expr') [ STORE IN (tablespace_name [, ... ] ) ] ] (
    partition_start_end_item [, ... ]
    partition_less_then_item [, ... ]
)
/* 列表分区场景，若申明AUTOMATIC则支持list自动扩展 */
PARTITION BY LIST (partKey) [ AUTOMATIC ]
(
    PARTITION partition_name VALUES (list_values_clause) [ TABLESPACE tablespace_name [, ... ] ]
...
)
/* 哈希分区场景 */
PARTITION BY HASH (partKey) (
    PARTITION partition_name [ TABLESPACE tablespace_name [, ... ] ]
...
)
/* 开启/关闭分区表行迁移 */
[ { ENABLE | DISABLE } ROW MOVEMENT ];
```

规格约束：

1. RANGE/LIST分区最大支持16个分区键，INTERVAL/HASH分区均只支持1个分区键，二级分区只支持1个分区键。
2. INTERVAL分区仅支持数值类型和日期/时间类型，INTERVAL分区不支持在二级分区表中创建。
3. INTERVAL分区表定义不能有MAXVALUE分区，LIST自动扩展分区表不能定义有DEFAULT分区。
4. 除哈希分区外，分区键不能插入空值，否则DML语句会进行报错处理。唯一例外：RANGE分区表定义有MAXVALUE分区/LIST分区表定义有DEFAULT分区。
5. 分区数最大值为1048575个，可以满足大部分业务场景的诉求。但分区数增加会导致系统中文件数增加，影响系统的性能，一般对于单个表而言不建议分区数超过200。

创建二级分区表

二级分区表，可以看成是对一级分区表的扩展，在二级分区表中第一层分区是一张逻辑表并不实际存储数据，数据实际是存储在二级分区节点上的。从实现上而言，二级分区表的分区方案是由两个一级分区的嵌套而来，一级分区的分区方案详见章节CREATE TABLE PARTITION。常见的二级分区表组合方案有：Range-Range分区、Range-List分区、Range-Hash分区、List-Range分区、List-List分区、List-Hash分区、Hash-Range分区、Hash-List分区、Hash-Hash分区等。目前二级分区仅支持行存表，二级分区创建的示例如下：

```
CREATE TABLE [ IF NOT EXISTS ] subpartition_table_name
(
    [ /* 该部份继承于普通表的Create Table */
    { column_name data_type [ COLLATE collation ] [ column_constraint [ ... ] ]
    | table_constraint
    | LIKE source_table [ like_option [...] ] } [, ... ]
```

 版权所有 © 华为云计算技术有限公司

```
    ]
)
[ WITH ( {storage_parameter = value} [, ... ] ) ]
[ COMPRESS | NOCOMPRESS ]
[ TABLESPACE tablespace_name ]
/* 二级分区定义的部分，只有list分区策略支持申明AUTOMATIC */
PARTITION BY {RANGE | LIST | HASH} (partKey) [ AUTOMATIC ] SUBPARTITOIN BY {RANGE | LIST | HASH}
(partKey) [ AUTOMATIC ]
(
    PARTITION partition_name partExpr…  /* 第一层分区 */
    (
        SUBPARTITION partition_name partExpr … /* 第二层分区 */
        SUBPARTITION partition_name partExpr … /* 第二层分区 */
    ),
    PARTITION partition_name partExpr…  /* 第一层分区 */
    (
        SUBPARTITION partition_name partExpr … /* 第二层分区 */
        SUBPARTITION partition_name partExpr … /* 第二层分区 */
    ),
    …
)
[ { ENABLE | DISABLE } ROW MOVEMENT ];
```

规格约束：

1. 二级分区表支持LIST/HASH/RANGE分区的任意两两组合。
2. 二级分区表支持任一层LIST申明自动扩展。
3. 二级分区表的第二层分区采用LIST自动扩展时，建表语句中第二层分区定义不能为空。
4. 二级分区表场景中仅支持单分区键。
5. 二级分区表中不支持INTERVAL类型分区的组合。
6. 二级分区表场景中，分区总数上限为1048575。

修改分区属性

分区表和分区相关的部分属性可以使用类似非分区表的ALTER-TABLE命令进行分区属性修改，常用的分区属性修改语句包括：

1. 增加分区
2. 删除分区
3. 删除/清空分区数据
4. 切割分区
5. 合并分区
6. 移动分区
7. 交换分区
8. 重命名分区

以上常用的分区属性变更语句基于对普通表ALTER TABLE语句进行扩展，在使用方式上大部分使用方式类似，分区表属性变更的基本语法框架示例如下：

```
/* 基本alter table语法 */
ALTER TABLE [ IF EXISTS ] { table_name [*] | ONLY table_name | ONLY ( table_name )}
action [, ... ];
```

分区表ALTER TABLE语句使用方法请参见**分区表运维管理**、《开发者指南》中“SQL参考 > SQL语法 > ALTER TABLE PARTITION和ALTER TABLE SUBPARTITION”章节。

 版权所有 © 华为云计算技术有限公司

2.8.2.3.2 使用和管理分区表

分区表支持大部分非分区表的相关功能，具体可以参考《开发者指南》中常规表的各类操作语法相关资料。

除此之外，分区表还支持大量的分区级操作命令，包括分区级DQL/DML（如SELECT、INSERT、UPDATE、DELETE、UPSERT、MERGE INTO）、分区级DDL（如ADD、DROP、TRUNCATE、EXCHANGE、SPLIT、MERGE、MOVE、RENAME）、分区VACUUM/ANALYZE、分类分区索引等。相关命令使用方法请参见**分区表DQL/DML**、**分区索引**、**分区表运维管理**、以及《开发者指南》中各个语法命令对应的章节。

分区级操作命令一般通过指定分区名或者分区值的方式进行，比如语法命令可能是如下情形：

```
sql_action [ t_name ] { PARTITION | SUBPARTITION } { p_name | (p_name) };
sql_action [ t_name ] { PARTITION | SUBPARTITION } FOR (p_value);
```

通过指定分区名p_name或指定分区值p_value来定向操作某个特定分区，此时业务只会作用于对象分区，而不会影响其他任何分区。如果通过指定分区名p_name来执行业务，数据库会匹配p_name对应的分区，该分区不存在则业务提示异常；如果通过指定分区值p_value来执行业务，数据库会匹配p_value值所属分区。

比如定义有如下的分区表：

```
gaussdb=# CREATE TABLE list_01
(
    id   INT,
    role VARCHAR(100),
    data VARCHAR(100)
)
PARTITION BY LIST (id)
(
    PARTITION p_list_1 VALUES(0,1,2,3,4),
    PARTITION p_list_2 VALUES(5,6,7,8,9),
    PARTITION p_list_3 VALUES(DEFAULT)
);
gaussdb=# DROP TABLE list_01;
```

指定分区业务中，PARTITION p_list_1与PARTITION FOR (4）等价，为同一个分区；PARTITION p_list_3与PARTITION FOR (12）等价，为同一个分区。

2.8.2.3.3 分区表 DQL/DML

由于分区的实现完全体现在数据库内核中，用户对分区表的DQL/DML与非分区表相比，在语法上没有任何区别。

出于分区表的易用性考虑，GaussDB支持指定分区的DQL/DML操作，指定分区可以通过PARTITION (partname)或者PARTITION FOR (partvalue)来实现，对于二级分区表还可以通过SUBPARTITION (subpartname)或者SUBPARTITION FOR (subpartvalue)指定具体的二级分区。指定分区执行DQL/DML时，若插入的数据不属于目标分区，则业务会产生报错；若查询的数据不属于目标分区，则会跳过该数据的处理。

指定分区DQL/DML支持以下几类语法：

1. 查询（SELECT）
2. 插入（INSERT）
3. 更新（UPDATE）
4. 删除（DELETE）
5. 插入或更新（UPSERT）

 版权所有 © 华为云计算技术有限公司

6. 合并（MERGE INTO）

指定分区做DQL/DML的示例如下：

```
gaussdb=#
/* 创建二级分区表 list_list_02 */
CREATE TABLE IF NOT EXISTS list_list_02
(
    id   INT,
    role VARCHAR(100),
    data VARCHAR(100)
)
PARTITION BY LIST (id) SUBPARTITION BY LIST (role)
(
    PARTITION p_list_2 VALUES(0,1,2,3,4,5,6,7,8,9)
    (
        SUBPARTITION p_list_2_1 VALUES ( 0,1,2,3,4,5,6,7,8,9 ),
        SUBPARTITION p_list_2_2 VALUES ( DEFAULT ),
        SUBPARTITION p_list_2_3 VALUES ( 10,11,12,13,14,15,16,17,18,19),
        SUBPARTITION p_list_2_4 VALUES ( 20,21,22,23,24,25,26,27,28,29 ),
        SUBPARTITION p_list_2_5 VALUES ( 30,31,32,33,34,35,36,37,38,39 )
    ),
    PARTITION p_list_3 VALUES(10,11,12,13,14,15,16,17,18,19)
    (
        SUBPARTITION p_list_3_2 VALUES ( DEFAULT )
    ),
    PARTITION p_list_4 VALUES( DEFAULT ),
    PARTITION p_list_5 VALUES(20,21,22,23,24,25,26,27,28,29)
    (
        SUBPARTITION p_list_5_1 VALUES ( 0,1,2,3,4,5,6,7,8,9 ),
        SUBPARTITION p_list_5_2 VALUES ( DEFAULT ),
        SUBPARTITION p_list_5_3 VALUES ( 10,11,12,13,14,15,16,17,18,19),
        SUBPARTITION p_list_5_4 VALUES ( 20,21,22,23,24,25,26,27,28,29 ),
        SUBPARTITION p_list_5_5 VALUES ( 30,31,32,33,34,35,36,37,38,39 )
    ),
    PARTITION p_list_6 VALUES(30,31,32,33,34,35,36,37,38,39),
    PARTITION p_list_7 VALUES(40,41,42,43,44,45,46,47,48,49)
    (
        SUBPARTITION p_list_7_1 VALUES ( DEFAULT )
    )
) ENABLE ROW MOVEMENT;
/* 导入数据 */
INSERT INTO list_list_02 VALUES(null, 'alice', 'alice data');
INSERT INTO list_list_02 VALUES(2, null, 'bob data');
INSERT INTO list_list_02 VALUES(null, null, 'peter data');

/* 对指定分区进行查询 */
-- 查询分区表全部数据
gaussdb=# SELECT * FROM list_list_02 ORDER BY data;
 id | role  |    data
----+-------+------------
    | alice | alice data
2   |       | bob data
    |       | peter data
(3 rows)
-- 查询分区p_list_4数据
gaussdb=# SELECT * FROM list_list_02 PARTITION (p_list_4) ORDER BY data;
 id | role  |    data
----+-------+------------
    | alice | alice data
    |       | peter data
(2 rows)
-- 查询(100, 100)所对应的二级分区的数据，即二级分区p_list_4_subpartdefault1，这个分区是在p_list_4下自动创建的一个分区范围定义为DEFAULT的分区
gaussdb=# SELECT * FROM list_list_02 SUBPARTITION FOR(100, 100) ORDER BY data;
 id | role  |    data
----+-------+------------
    | alice | alice data
    |       | peter data
(2 rows)
```

 版权所有 © 华为云计算技术有限公司

```
-- 查询分区p_list_2 数据
gaussdb=# SELECT * FROM list_list_02 PARTITION (p_list_2) ORDER BY data;
 id | role |   data
----+------+----------
  2 |      | bob data
(1 row)
-- 查询(0, 100)所对应的二级分区的数据，即二级分区p_list_2_2
gaussdb=# SELECT * FROM list_list_02 SUBPARTITION FOR (0, 100) ORDER BY data;
 id | role |   data
----+------+----------
  2 |      | bob data
(1 row)

/* 对指定分区做IUD */
-- 删除分区p_list_5中的全部数据
gaussdb=# DELETE FROM list_list_02 PARTITION (p_list_5);
-- 指定分区p_list_7_1插入数据，由于数据不符合该分区约束，插入报错
gaussdb=# INSERT INTO list_list_02 SUBPARTITION (p_list_7_1) VALUES(null, 'cherry', 'cherry data');
ERROR:  inserted subpartition key does not map to the table subpartition
-- 将一级分区值100所属分区的数据进行更新
gaussdb=# UPDATE list_list_02 PARTITION FOR (100) SET id = 1;

--upsert
gaussdb=# INSERT INTO list_list_02 (id, role, data) VALUES (1, 'test', 'testdata') ON DUPLICATE KEY UPDATE
role = VALUES(role), data = VALUES(data);

--merge into
gaussdb=#
CREATE TABLE IF NOT EXISTS list_tmp
(
    id   INT,
    role VARCHAR(100),
    data VARCHAR(100)
)
PARTITION BY LIST (id)
(
    PARTITION p_list_2 VALUES(0,1,2,3,4,5,6,7,8,9),
    PARTITION p_list_3 VALUES(10,11,12,13,14,15,16,17,18,19),
    PARTITION p_list_4 VALUES( DEFAULT ),
    PARTITION p_list_5 VALUES(20,21,22,23,24,25,26,27,28,29),
    PARTITION p_list_6 VALUES(30,31,32,33,34,35,36,37,38,39),
    PARTITION p_list_7 VALUES(40,41,42,43,44,45,46,47,48,49)) ENABLE ROW MOVEMENT;

gaussdb=# MERGE INTO list_tmp target
USING list_list_02 source
ON (target.id = source.id)
WHEN MATCHED THEN
  UPDATE SET target.data = source.data,
             target.role = source.role
WHEN NOT MATCHED THEN
  INSERT (id, role, data)
  VALUES (source.id, source.role, source.data);

gaussdb=#
DROP TABLE list_tmp;
DROP TABLE list_list_02;
```

2.8.3 分区表查询优化

说明

本节示例对应explain_perf_mode参数值为normal。

2.8.3.1 分区剪枝

分区剪枝是GaussDB提供的一种分区表查询优化技术，数据库SQL引擎会根据查询条件，只扫描特定的部分分区。分区剪枝是自动触发的，当分区表查询条件符合剪枝场

 版权所有 © 华为云计算技术有限公司

景时，会自动触发分区剪枝。根据剪枝阶段的不同，分区剪枝分为静态剪枝和动态剪枝，静态剪枝在优化器阶段进行，在生成计划之前，数据库已经知道需要访问的分区信息；动态剪枝在执行器阶段进行（执行开始/执行过程中），在生成计划时，数据库并不知道需要访问的分区信息，只是判断“可以进行分区剪枝”，具体的剪枝信息由执行器决定。

只有分区表页面扫描和Local索引扫描才会触发分区剪枝，Global索引没有分区的概念，不需要进行剪枝。

2.8.3.1.1 分区表静态剪枝

对于检索条件中分区键上带有常数的分区表查询语句，在优化器阶段将对indexscan、bitmap indexscan、indexonlyscan等算子中包含的检索条件作为剪枝条件，完成分区的筛选。算子包含的检索条件中需要至少包含一个分区键字段，对于含有多个分区键的分区表，包含任意分区键子集即可。

静态剪枝支持范围如下所示：

1. 支持分区级别：一级分区、二级分区。
2. 支持分区类型：范围分区、间隔分区、哈希分区、列表分区。
3. 支持表达式类型：比较表达式（<，<=，=，>=，>）、逻辑表达式、数组表达式。

说明

目前静态剪枝不支持子查询表达式。

为了支持分区表剪枝，在计划生成时会将分区键上的过滤条件强制转换为分区键类型，该操作与隐式类型转换规则存在差异，可能导致相同条件在分区键上转换报错，非分区键无报错的情况。

- 静态剪枝支持的典型场景具体示例如下：

  a. 比较表达式

```
gaussdb=#
--创建分区表
CREATE TABLE t1 (c1 int, c2 int)
PARTITION BY RANGE (c1)
(
    PARTITION p1 VALUES LESS THAN(10),
    PARTITION p2 VALUES LESS THAN(20),
    PARTITION p3 VALUES LESS THAN(MAXVALUE)
);

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 = 1;
                QUERY PLAN
------------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: 1
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: (t1.c1 = 1)
         Selected Partitions:  1
(7 rows)

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 < 1;
                QUERY PLAN
------------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: 1
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: (t1.c1 < 1)
```

 版权所有 © 华为云计算技术有限公司

```
      Selected Partitions:  1
(7 rows)

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 > 11;
             QUERY PLAN
-------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: 2
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: (t1.c1 > 11)
         Selected Partitions:  2..3
(7 rows)

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 is NULL;
             QUERY PLAN
-------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: 1
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: (t1.c1 IS NULL)
         Selected Partitions:  3
(7 rows)
```

b. 逻辑表达式

```
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 = 1 AND c2 = 2;
               QUERY PLAN
------------------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: 1
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: ((t1.c1 = 1) AND (t1.c2 = 2))
         Selected Partitions:  1
(7 rows)

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 = 1 OR c1 = 2;
               QUERY PLAN
------------------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: 1
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: ((t1.c1 = 1) OR (t1.c1 = 2))
         Selected Partitions:  1
(7 rows)

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE NOT c1 = 1;
             QUERY PLAN
-------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: 3
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: (t1.c1 <> 1)
         Selected Partitions:  1..3
(7 rows)
```

c. 数组表达式

```
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 IN (1, 2, 3);
                  QUERY PLAN
-----------------------------------------------------
 Partition Iterator
   Output: c1, c2
```

 版权所有 © 华为云计算技术有限公司

```
  Iterations: 1
  ->  Partitioned Seq Scan on public.t1
        Output: c1, c2
        Filter: (t1.c1 = ANY ('{1,2,3}'::integer[]))
        Selected Partitions:  1
(7 rows)

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 = ALL(ARRAY[1,
2, 3]);
                    QUERY PLAN
-----------------------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: 0
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: (t1.c1 = ALL ('{1,2,3}'::integer[]))
         Selected Partitions:  NONE
(7 rows)

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 = ANY(ARRAY[1,
2, 3]);
                    QUERY PLAN
-----------------------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: 1
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: (t1.c1 = ANY ('{1,2,3}'::integer[]))
         Selected Partitions:  1
(7 rows)

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 =
SOME(ARRAY[1, 2, 3]);
                    QUERY PLAN
-----------------------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: 1
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: (t1.c1 = ANY ('{1,2,3}'::integer[]))
         Selected Partitions:  1
(7 rows)
```

- 静态剪枝不支持的典型场景具体示例如下：

  子查询表达式

```
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 WHERE c1 = ALL(SELECT c2
FROM t1 WHERE c1 > 10);
                      QUERY PLAN
-------------------------------------------------------------
 Partition Iterator
   Output: public.t1.c1, public.t1.c2
   Iterations: 3
   ->  Partitioned Seq Scan on public.t1
         Output: public.t1.c1, public.t1.c2
         Filter: (SubPlan 1)
         Selected Partitions:  1..3
(7 rows)

gaussdb=# DROP TABLE t1;
```

2.8.3.1.2 分区表动态剪枝

对于检索条件中存在带有变量的分区表查询语句，由于优化器阶段无法获取用户的绑定参数，因此优化器阶段仅能完成indexscan、bitmapindexscan、indexonlyscan等算子检索条件的解析，后续会在执行器阶段获得绑定参数后，完成分区筛选。算子包含

的检索条件中需要至少包含一个分区键字段，对于含有多个分区键的分区表，包含任意分区键子集即可。目前分区表动态剪枝仅支持PBE（Prepare/Bind/Execute）场景和参数化路径场景。

PBE 动态剪枝

PBE动态剪枝支持范围如下所示：

1. 支持分区级别：一级分区、二级分区。
2. 支持分区类型：范围分区、间隔分区、哈希分区、列表分区。
3. 支持表达式类型：比较表达式（<，<=，=，>=，>）、逻辑表达式、数组表达式。
4. 支持部分类型转换和函数：对于类型可以相互转换的场景和immutable函数可以支持PBE动态剪枝

⚠ 注意

- PBE动态剪枝支持表达式、隐式转换、immutable函数和stable函数，不支持子查询表达式和volatile函数。对于stable函数，如to_timestamp等类型转换函数，可能会受GUC参数变化，影响剪枝结果。为了保持性能优化，此情况可以通过analyze表重新生成gplan解决。
- 由于PBE动态剪枝是基于generic plan的剪枝，所以判断语句是否能PBE动态剪枝时，需要设置参数plan_cache_mode = 'force_generic_plan'，排除custom plan的干扰。
- 使用动态剪枝时，由于在生成计划阶段计划实际执行的分区尚未确定，不支持生成分类索引计划。

- PBE动态剪枝支持的典型场景具体示例如下：

  a. 比较表达式

```
gaussdb=#
--创建分区表
CREATE TABLE t1 (c1 int, c2 int)
PARTITION BY RANGE (c1)
(
    PARTITION p1 VALUES LESS THAN(10),
    PARTITION p2 VALUES LESS THAN(20),
    PARTITION p3 VALUES LESS THAN(MAXVALUE)
);
--设置参数
gaussdb=# set plan_cache_mode = 'force_generic_plan';

gaussdb=# PREPARE p1(int) AS SELECT * FROM t1 WHERE c1 = $1;
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p1(1);
              QUERY PLAN
----------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: PART
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: (t1.c1 = $1)
         Selected Partitions:  1 (pbe-pruning)
(7 rows)

gaussdb=# PREPARE p2(int) AS SELECT * FROM t1 WHERE c1 < $1;
PREPARE
```


版权所有 © 华为云计算技术有限公司

```
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p2(1);
              QUERY PLAN
---------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: PART
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: (t1.c1 < $1)
         Selected Partitions:  1 (pbe-pruning)
(7 rows)

gaussdb=# PREPARE p3(int) AS SELECT * FROM t1 WHERE c1 > $1;
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p3(1);
              QUERY PLAN
---------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: PART
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: (t1.c1 > $1)
         Selected Partitions:  1..3 (pbe-pruning)
(7 rows)
```

b. 逻辑表达式

```
gaussdb=# PREPARE p5(INT, INT) AS SELECT * FROM t1 WHERE c1 = $1 AND c2 = $2;
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p5(1, 2);
                 QUERY PLAN
-------------------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: PART
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: ((t1.c1 = $1) AND (t1.c2 = $2))
         Selected Partitions:  1 (pbe-pruning)
(7 rows)

gaussdb=# PREPARE p6(INT, INT) AS SELECT * FROM t1 WHERE c1 = $1 OR c2 = $2;
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p6(1, 2);
                 QUERY PLAN
-------------------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: PART
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: ((t1.c1 = $1) OR (t1.c2 = $2))
         Selected Partitions:  1 (pbe-pruning)
(7 rows)
gaussdb=# PREPARE p7(INT) AS SELECT * FROM t1 WHERE NOT c1 = $1;
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) execute p7(1);
              QUERY PLAN
---------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: PART
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: (t1.c1 <> $1)
         Selected Partitions:  1 (pbe-pruning)
(7 rows)
```

c. 数组表达式

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# PREPARE p8(INT, INT, INT) AS SELECT * FROM t1 WHERE c1 IN ($1, $2, $3);
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p8(1, 2, 3);
                  QUERY PLAN
-----------------------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: PART
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: (t1.c1 = ANY (ARRAY[$1, $2, $3]))
         Selected Partitions:  1 (pbe-pruning)
(7 rows)
gaussdb=# PREPARE p9(INT, INT, INT) AS SELECT * FROM t1 WHERE c1 NOT IN ($1, $2, $3);
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p9(1, 2, 3);
                   QUERY PLAN
-----------------------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: PART
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: (t1.c1 <> ALL (ARRAY[$1, $2, $3]))
         Selected Partitions:  1 (pbe-pruning)
(7 rows)
gaussdb=# PREPARE p10(INT, INT, INT) AS SELECT * FROM t1 WHERE c1 = ALL(ARRAY[$1, $2,
$3]);
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p10(1, 2, 3);
                  QUERY PLAN
-----------------------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: PART
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: (t1.c1 = ALL (ARRAY[$1, $2, $3]))
         Selected Partitions:  1 (pbe-pruning)
(7 rows)
gaussdb=# PREPARE p11(INT, INT, INT) AS SELECT * FROM t1 WHERE c1 = ANY(ARRAY[$1, $2,
$3]);
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p11(1, 2, 3);
                  QUERY PLAN
-----------------------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: PART
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: (t1.c1 = ANY (ARRAY[$1, $2, $3]))
         Selected Partitions:  1 (pbe-pruning)
(7 rows)
gaussdb=# PREPARE p12(INT, INT, INT) AS SELECT * FROM t1 WHERE c1 = SOME(ARRAY[$1,
$2, $3]);
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p12(1, 2, 3);
                  QUERY PLAN
-----------------------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: PART
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: (t1.c1 = ANY (ARRAY[$1, $2, $3]))
         Selected Partitions:  1 (pbe-pruning)
(7 rows)
```

d. 类型转换触发隐式转换

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# set plan_cache_mode = 'force_generic_plan';
gaussdb=# PREPARE p13(TEXT) AS SELECT * FROM t1 WHERE c1 = $1;
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p13('12');
                 QUERY PLAN
-----------------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: PART
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: (t1.c1 = ($1)::bigint)
         Selected Partitions:  1 (pbe-pruning)
(7 rows)
```

e. immutable函数

```
gaussdb=# PREPARE p14(TEXT) AS SELECT * FROM t1 WHERE c1 = LENGTHB($1);
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p14('hello');
                 QUERY PLAN
-----------------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: PART
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: (t1.c1 = lengthb($1))
         Selected Partitions:  1 (pbe-pruning)
(7 rows)
```

- PBE动态剪枝不支持的典型场景具体示例如下：

a. 子查询表达式

```
gaussdb=# PREPARE p15(INT) AS SELECT * FROM t1 WHERE c1 = ALL(SELECT c2 FROM t1
WHERE c1 > $1);
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p15(1);
                      QUERY PLAN
-------------------------------------------------------------
 Partition Iterator
   Output: public.t1.c1, public.t1.c2
   Iterations: 3
   ->  Partitioned Seq Scan on public.t1
         Output: public.t1.c1, public.t1.c2
         Filter: (SubPlan 1)
         Selected Partitions:  1..3
(7 rows)
```

b. 类型转换无法直接触发隐式转换

```
gaussdb=# PREPARE p16(name) AS SELECT * FROM t1 WHERE c1 = $1;
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p16('12');
                 QUERY PLAN
-----------------------------------------------
 Partition Iterator
   Output: c1, c2
   Iterations: 3
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: ((t1.c1)::text = ($1)::text)
         Selected Partitions:  1..3
(7 rows)
```

c. stable/volatile函数

```
gaussdb=# create sequence seq;
gaussdb=# PREPARE p17(TEXT) AS SELECT * FROM t1 WHERE c1 = currval($1);--volatile函数不支
持剪枝
PREPARE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) EXECUTE p17('seq');
                      QUERY PLAN
-------------------------------------------------------------
```

 版权所有 © 华为云计算技术有限公司

```
 Partition Iterator
   Output: c1, c2
   Iterations: 3
   ->  Partitioned Seq Scan on public.t1
         Output: c1, c2
         Filter: ((t1.c1)::numeric = currval(($1)::regclass))
         Selected Partitions:  1..3
(7 rows)

gaussdb=# DROP TABLE t1;
```

参数化路径动态剪枝

参数化路径动态剪枝支持范围如下所示：

1. 支持分区级别：一级分区、二级分区。
2. 支持分区类型：范围分区、间隔分区、哈希分区、列表分区。
3. 支持算子类型：indexscan、indexonlyscan、bitmapscan。
4. 支持表达式类型：比较表达式（<，<=，=，>=，>）、逻辑表达式。

⚠ 注意

参数化路径动态剪枝不支持子查询表达式，不支持stable和volatile函数，不支持跨QueryBlock参数化路径，不支持BitmapOr、BitmapAnd算子。

- 参数化路径动态剪枝支持的典型场景具体示例如下：
  a. 比较表达式

```
gaussdb=#
--创建分区表和索引
gaussdb=# CREATE TABLE t1 (c1 INT, c2 INT)
PARTITION BY RANGE (c1)
(
    PARTITION p1 VALUES LESS THAN(10),
    PARTITION p2 VALUES LESS THAN(20),
    PARTITION p3 VALUES LESS THAN(MAXVALUE)
);
CREATE TABLE t2 (c1 INT, c2 INT)
PARTITION BY RANGE (c1)
(
    PARTITION p1 VALUES LESS THAN(10),
    PARTITION p2 VALUES LESS THAN(20),
    PARTITION p3 VALUES LESS THAN(MAXVALUE)
);
CREATE INDEX t1_c1 ON t1(c1) LOCAL;
CREATE INDEX t2_c1 ON t2(c1) LOCAL;
CREATE INDEX t1_c2 ON t1(c2) LOCAL;
CREATE INDEX t2_c2 ON t2(c2) LOCAL;

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t2 JOIN t1 ON t1.c1 = t2.c2;
                        QUERY PLAN
------------------------------------------------------------------
 Hash Join
   Output: t2.c1, t2.c2, t1.c1, t1.c2
   Hash Cond: (t2.c2 = t1.c1)
   ->  Partition Iterator
         Output: t2.c1, t2.c2
         Iterations: 3
         ->  Partitioned Seq Scan on public.t2
               Output: t2.c1, t2.c2
               Selected Partitions:  1..3
   ->  Hash
         Output: t1.c1, t1.c2
```


版权所有 © 华为云计算技术有限公司

```
      ->  Partition Iterator
            Output: t1.c1, t1.c2
            Iterations: 3
            ->  Partitioned Seq Scan on public.t1
                  Output: t1.c1, t1.c2
                  Selected Partitions:  1..3
(17 rows)

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t2 JOIN t1 ON t1.c1 < t2.c2;
                     QUERY PLAN
-------------------------------------------------------------
 Nested Loop
   Output: t2.c1, t2.c2, t1.c1, t1.c2
   ->  Partition Iterator
         Output: t2.c1, t2.c2
         Iterations: 3
         ->  Partitioned Seq Scan on public.t2
               Output: t2.c1, t2.c2
               Selected Partitions:  1..3
   ->  Partition Iterator
         Output: t1.c1, t1.c2
         Iterations: PART
         ->  Partitioned Index Scan using t2_c1 on public.t1
               Output: t1.c1, t1.c2
               Index Cond: (t1.c1 < t2.c2)
               Selected Partitions:  1..3 (ppi-pruning)
(15 rows)

gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t2 JOIN t1 ON t1.c1 > t2.c2;
                     QUERY PLAN
-------------------------------------------------------------
 Nested Loop
   Output: t2.c1, t2.c2, t1.c1, t1.c2
   ->  Partition Iterator
         Output: t2.c1, t2.c2
         Iterations: 3
         ->  Partitioned Seq Scan on public.t2
               Output: t2.c1, t2.c2
               Selected Partitions:  1..3
   ->  Partition Iterator
         Output: t1.c1, t1.c2
         Iterations: PART
         ->  Partitioned Index Scan using t2_c1 on public.t1
               Output: t1.c1, t1.c2
               Index Cond: (t1.c1 > t2.c2)
               Selected Partitions:  1..3 (ppi-pruning)
(15 rows)
```

b. 逻辑表达式

```
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t2 JOIN t1 ON t1.c1 = t2.c2
AND t1.c2 = 2;
                     QUERY PLAN
-------------------------------------------------------------
 Hash Join
   Output: t2.c1, t2.c2, t1.c1, t1.c2
   Hash Cond: (t2.c2 = t1.c1)
   ->  Partition Iterator
         Output: t2.c1, t2.c2
         Iterations: 3
         ->  Partitioned Seq Scan on public.t2
               Output: t2.c1, t2.c2
               Selected Partitions:  1..3
   ->  Hash
         Output: t1.c1, t1.c2
         ->  Partition Iterator
               Output: t1.c1, t1.c2
               Iterations: 3
               ->  Partitioned Bitmap Heap Scan on public.t1
                     Output: t1.c1, t1.c2
                     Recheck Cond: (t1.c2 = 2)
```

 版权所有 © 华为云计算技术有限公司

```
                  Selected Partitions:  1..3
                  ->  Partitioned Bitmap Index Scan on t1_c2
                        Index Cond: (t1.c2 = 2)
(20 rows)
```

- 参数化路径动态剪枝不支持的典型场景具体示例如下：

  a. BitmapOr/BitmapAnd算子

```
gaussdb=# set enable_seqscan=off;
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t2 JOIN t1 ON t1.c1 = t2.c2 OR
t1.c1 = 2;
                          QUERY PLAN
------------------------------------------------------------------
 Nested Loop
   Output: t2.c1, t2.c2, t1.c1, t1.c2
   ->  Partition Iterator
         Output: t2.c1, t2.c2
         Iterations: 3
         ->  Partitioned Seq Scan on public.t2
               Output: t2.c1, t2.c2
               Selected Partitions:  1..3
   ->  Partition Iterator
         Output: t1.c1, t1.c2
         Iterations: 3
         ->  Partitioned Bitmap Heap Scan on public.t1
               Output: t1.c1, t1.c2
               Recheck Cond: ((t1.c1 = t2.c2) OR (t1.c1 = 2))
               Selected Partitions:  1..3
               ->  BitmapOr
                     ->  Partitioned Bitmap Index Scan on t1_c1
                           Index Cond: (t1.c1 = t2.c2)
                     ->  Partitioned Bitmap Index Scan on t1_c1
                           Index Cond: (t1.c1 = 2)
(20 rows)
```

  b. 隐式转换

```
gaussdb=# CREATE TABLE t3(c1 TEXT, c2 INT);
CREATE TABLE
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 JOIN t3 ON t1.c1 = t3.c1;
                        QUERY PLAN
-----------------------------------------------------------------
 Nested Loop
   Output: t1.c1, t1.c2, t3.c1, t3.c2
   ->  Seq Scan on public.t3
         Output: t3.c1, t3.c2
   ->  Partition Iterator
         Output: t1.c1, t1.c2
         Iterations: 3
         ->  Partitioned Index Scan using t1_c1 on public.t1
               Output: t1.c1, t1.c2
               Index Cond: (t1.c1 = (t3.c1)::bigint)
               Selected Partitions:  1..3
(11 rows)
```

  c. 函数

```
gaussdb=# EXPLAIN (VERBOSE ON, COSTS OFF) SELECT * FROM t1 JOIN t3 ON t1.c1 =
LENGTHB(t3.c1);
                        QUERY PLAN
-----------------------------------------------------------------
 Nested Loop
   Output: t1.c1, t1.c2, t3.c1, t3.c2
   ->  Seq Scan on public.t3
         Output: t3.c1, t3.c2
   ->  Partition Iterator
         Output: t1.c1, t1.c2
         Iterations: 3
         ->  Partitioned Index Scan using t1_c1 on public.t1
               Output: t1.c1, t1.c2
               Index Cond: (t1.c1 = lengthb(t3.c1))
               Selected Partitions:  1..3
(11 rows)
```

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# DROP TABLE t1;
gaussdb=# DROP TABLE t2;
gaussdb=# DROP TABLE t3;
```

2.8.3.2 分区算子执行优化

2.8.3.2.1 Partition Iterator 算子消除

场景描述

在当前分区表架构中，执行器通过Partition Iterator算子去迭代访问每一个分区。当分区剪枝结果只有一个分区时，Partition Iterator算子已经失去了迭代器的作用，在此情况下消除Partition Iterator算子，可以避免执行时一些不必要的开销。由于执行器的PIPELINE架构，Partition Iterator算子会重复执行，在数据量较大的场景下消除Partition Iterator算子的收益十分可观。

示例

消除Partition Iterator算子在GUC参数partition_iterator_elimination开启后才能生效，示例如下：

```
gaussdb=#
CREATE TABLE test_range_pt (a INT, b INT, c INT)
PARTITION BY RANGE (a)
(
    PARTITION p1 VALUES LESS THAN (2000),
    PARTITION p2 VALUES LESS THAN (3000),
    PARTITION p3 VALUES LESS THAN (4000),
    PARTITION p4 VALUES LESS THAN (5000),
    PARTITION p5 VALUES LESS THAN (MAXVALUE)
)ENABLE ROW MOVEMENT;

gaussdb=# EXPLAIN SELECT * FROM test_range_pt WHERE a = 3000;
                                QUERY PLAN
---------------------------------------------------------------------------------
 Partition Iterator  (cost=0.00..25.31 rows=10 width=12)
   Iterations: 1
   ->  Partitioned Seq Scan on test_range_pt  (cost=0.00..25.31 rows=10 width=12)
         Filter: (a = 3000)
         Selected Partitions:  3
(5 rows)

gaussdb=# SET partition_iterator_elimination = on;
SET
gaussdb=# EXPLAIN SELECT * FROM test_range_pt WHERE a = 3000;
                              QUERY PLAN
-------------------------------------------------------------------------------
 Partitioned Seq Scan on test_range_pt  (cost=0.00..25.31 rows=10 width=12)
   Filter: (a = 3000)
   Selected Partitions:  3
(3 rows)

gaussdb=# DROP TABLE test_range_pt;
```

注意事项及约束条件

1. GUC参数partition_iterator_elimination开启后，且优化器剪枝结果只有一个分区时，目标场景优化才能生效。
2. 消除Partition Iterator算子不支持二级分区表。

 版权所有 © 华为云计算技术有限公司

3. 支持cplan，支持部分gplan场景，如分区键a = $1（即优化器阶段可以剪枝到一个分区的场景）。
4. 支持SeqScan、Indexscan、Indexonlyscan、Bitmapscan、RowToVec、Tidscan算子。
5. 支持行存，astore/ustore存储引擎，支持SQLBypass。

2.8.3.2.2 Merge Append

场景描述

当对分区表进行全局排序时，通常SQL引擎的实现方式是先通过Partition Iterator + PartitionScan对分区表做全量扫描，然后进行Sort排序操作，这样难以利用数据分区分治的算法思想进行全局排序，假如ORDER BY排序列包含索引，本身局部有序的前提条件则无法利用。针对这类问题，目前分区表支持分区归并排序执行策略，利用Merge Append的执行机制改进分区表的排序机制。

示例

分区表Merge Append的执行机制示例如下：

```
gaussdb=#
CREATE TABLE test_range_pt (a INT, b INT, c INT)
PARTITION BY RANGE(a)
(
   PARTITION p1 VALUES LESS THAN (2000),
   PARTITION p2 VALUES LESS THAN (3000),
   PARTITION p3 VALUES LESS THAN (4000),
   PARTITION p4 VALUES LESS THAN (5000),
   PARTITION p5 VALUES LESS THAN (MAXVALUE)
)ENABLE ROW MOVEMENT;
INSERT INTO test_range_pt VALUES
(generate_series(1,10000),generate_series(1,10000),generate_series(1,10000));
CREATE INDEX idx_range_b ON test_range_pt(b) LOCAL;
ANALYZE test_range_pt;

gaussdb=# EXPLAIN ANALYZE SELECT * FROM test_range_pt WHERE b >10 AND b < 5000 ORDER BY b
LIMIT 10;
                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
--------------------------------
 Limit  (cost=0.06..1.02 rows=10 width=12) (actual time=0.990..1.041 rows=10 loops=1)
   ->  Result  (cost=0.06..480.32 rows=10 width=12) (actual time=0.988..1.036 rows=10 loops=1)
         ->  Merge Append  (cost=0.06..480.32 rows=10 width=12) (actual time=0.985..1.026 rows=10 loops=1)
               Sort Key: b
               ->  Partitioned Index Scan using idx_range_b on test_range_pt  (cost=0.00..44.61 rows=998
width=12) (actual time=0.256..0.284 rows=10 loops=1)
                     Index Cond: ((b > 10) AND (b < 5000))
                     Selected Partitions:  1
               ->  Partitioned Index Scan using idx_range_b on test_range_pt  (cost=0.00..44.61 rows=998
width=12) (actual time=0.208..0.208 rows=1 loops=1)
                     Index Cond: ((b > 10) AND (b < 5000))
                     Selected Partitions:  2
               ->  Partitioned Index Scan using idx_range_b on test_range_pt  (cost=0.00..44.61 rows=998
width=12) (actual time=0.205..0.205 rows=1 loops=1)
                     Index Cond: ((b > 10) AND (b < 5000))
                     Selected Partitions:  3
               ->  Partitioned Index Scan using idx_range_b on test_range_pt  (cost=0.00..44.61 rows=998
width=12) (actual time=0.212..0.212 rows=1 loops=1)
                     Index Cond: ((b > 10) AND (b < 5000))
                     Selected Partitions:  4
               ->  Partitioned Index Scan using idx_range_b on test_range_pt  (cost=0.00..44.61 rows=998
width=12) (actual time=0.092..0.092 rows=0 loops=1)
                     Index Cond: ((b > 10) AND (b < 5000))
```

 版权所有 © 华为云计算技术有限公司

```
                  Selected Partitions:  5
 Total runtime: 1.656 ms
(20 rows)

--关闭分区表Merge Append算子
gaussdb=# SET sql_beta_feature = 'disable_merge_append_partition';
SET
gaussdb=# EXPLAIN ANALYZE SELECT * FROM test_range_pt WHERE b >10 AND b < 5000 ORDER BY b
LIMIT 10;
                                                    QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------
------------------
 Limit  (cost=330.92..330.95 rows=10 width=12) (actual time=6.728..6.730 rows=10 loops=1)
   ->  Sort  (cost=330.92..343.40 rows=10 width=12) (actual time=6.727..6.729 rows=10 loops=1)
         Sort Key: b
         Sort Method: top-N heapsort  Memory: 26kB
         ->  Partition Iterator  (cost=0.00..223.07 rows=4991 width=12) (actual time=0.102..4.503 rows=4989
loops=1)
               Iterations: 5
               ->  Partitioned Index Scan using idx_range_b on test_range_pt  (cost=0.00..223.07 rows=4991
width=12) (actual time=0.253..3.666 rows=4989 loops=5)
                     Index Cond: ((b > 10) AND (b < 5000))
                     Selected Partitions:  1..5
 Total runtime: 6.945 ms
(10 rows)

gaussdb=# DROP TABLE test_range_pt;
```

Merge Append的执行代价远小于普通执行方式。

注意事项及约束条件

1. 当分区扫描路径为Index/Index Only时，才支持Merge Append执行机制。
2. 当分区剪枝结果大于1时，才支持Merge Append执行机制。
3. 当分区索引全部有效且为btree索引时，才支持Merge Append执行机制。
4. 当SQL含有Limit子句时，才支持Merge Append执行机制。
5. 当分区扫描时如果存在Filter，不支持Merge Append执行机制。
6. 当GUC参数sql_beta_feature = 'disable_merge_append_partition'时，不再生成Merge Append路径。

2.8.3.2.3 Max/Min

场景描述

当对分区表使用Max/Min函数时，通常SQL引擎的实现方式是先通过Partition Iterator + PartitionScan对分区表做全量扫描然后进行Sort + Limit操作。如果分区是索引扫描，可以先对每个分区进行Limit操作，计算Max/Min值，最后在分区表上做Sort + Limit操作。这样分区表上做Sort时，由于每个分区已经获取Max/Min值，所以Sort的数据量跟分区数相同，这时极大的减少了Sort开销。

示例

分区表Max/Min优化示例如下：

```
gaussdb=#
CREATE TABLE test_range_pt (a INT, b INT, c INT)
PARTITION BY RANGE(a)
(
    PARTITION p1 VALUES LESS THAN (2000),
    PARTITION p2 VALUES LESS THAN (3000),
    PARTITION p3 VALUES LESS THAN (4000),
```

 版权所有 © 华为云计算技术有限公司

```
   PARTITION p4 VALUES LESS THAN (5000),
   PARTITION p5 VALUES LESS THAN (MAXVALUE)
)ENABLE ROW MOVEMENT;
CREATE INDEX idx_range_b ON test_range_pt(b) LOCAL;
INSERT INTO test_range_pt VALUES(generate_series(1,10000), generate_series(1,10000),
generate_series(1,10000));
```

优化前：

```
gaussdb=# EXPLAIN ANALYZE SELECT min(b) FROM test_range_pt;
                                                    QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------
-----------
Aggregate  (cost=164.00..164.01 rows=1 width=8) (actual time=6.779..6.780 rows=1 loops=1)
  ->  Partition Iterator  (cost=0.00..139.00 rows=10000 width=4) (actual time=0.099..4.588 rows=10000
loops=1)
      Iterations: 5
      ->  Partitioned Seq Scan on test_range_pt  (cost=0.00..139.00 rows=10000 width=4) (actual
time=0.326..3.516 rows=10000 loops=5)
            Selected Partitions:  1..5
 Total runtime: 6.942 ms
(6 rows)
```

优化后：

```
gaussdb=# EXPLAIN  ANALYZE SELECT min(b) FROM test_range_pt;
                                                                QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------
-----------------------------------------------
 Result  (cost=441.25..441.26 rows=1 width=0) (actual time=0.554..0.555 rows=1 loops=1)
   InitPlan 1 (returns $2)
     ->  Limit  (cost=441.25..441.25 rows=1 width=4) (actual time=0.547..0.547 rows=1 loops=1)
           ->  Sort  (cost=441.25..466.25 rows=1 width=4) (actual time=0.544..0.544 rows=1 loops=1)
                 Sort Key: public.test_range_pt.b
                 Sort Method: top-N heapsort  Memory: 25kB
                 ->  Partition Iterator  (cost=0.00..391.25 rows=10000 width=4) (actual time=0.135..0.502 rows=5
loops=1)
                       Iterations: 5
                       ->  Limit  (cost=0.00..0.04 rows=1 width=4) (actual time=0.322..0.322 rows=5 loops=5)
                             ->  Partitioned Index Only Scan using idx_range_b on test_range_pt  (cost=0.00..391.25
rows=1 width=4) (actual time=0.319..0.319 rows=5 loops=5)
                                   Index Cond: (b IS NOT NULL)
                                   Heap Fetches: 5
                                   Selected Partitions:  1..5
 Total runtime: 0.838 ms
(14 rows)
```

优化后时间消耗远小于优化前。

```
--清理示例
gaussdb=# DROP TABLE test_range_pt;
```

注意事项及约束条件

1. 当分区扫描路径为Index、Index Only时，才支持Max/Min优化。
2. 当分区索引全部有效且为Btree索引时，才支持Max/Min优化。

2.8.3.2.4 分区导入数据性能优化

场景描述

当向分区表插入数据的时候，如果插入的数据为常量/参数/表达式等简单类型，会自动对INSERT算子进行执行优化（FastPath）。可以通过执行计划来判断是否触发了执行优化，触发执行优化时Insert计划前会带有FastPath关键字。

 版权所有 © 华为云计算技术有限公司

示例

```
gaussdb=#
CREATE TABLE fastpath_t1
(
    col1 INT,
    col2 TEXT
)
PARTITION BY RANGE(col1)
(
    PARTITION p1 VALUES LESS THAN(10),
    PARTITION p2 VALUES LESS THAN(MAXVALUE)
);

--INSERT常量，执行FastPath优化
gaussdb=# EXPLAIN INSERT INTO fastpath_t1 VALUES (0, 'test_insert');
                          QUERY PLAN
---------------------------------------------------------------------
 FastPath Insert on fastpath_t1  (cost=0.00..0.01 rows=1 width=0)
   ->  Result  (cost=0.00..0.01 rows=1 width=0)
(2 rows)

--INSERT带参数/简单表达式，执行FastPath优化
gaussdb=# PREPARE insert_t1 AS INSERT INTO fastpath_t1 VALUES($1 + 1 + $2, $2);
PREPARE
gaussdb=# EXPLAIN EXECUTE insert_t1(10, '0');
                          QUERY PLAN
---------------------------------------------------------------------
 FastPath Insert on fastpath_t1  (cost=0.00..0.02 rows=1 width=0)
   ->  Result  (cost=0.00..0.02 rows=1 width=0)
(2 rows)

--INSERT为子查询，无法执行FastPath优化，走标准执行器模块
gaussdb=# CREATE TABLE test_1(col1 int, col3 text);
gaussdb=# EXPLAIN INSERT INTO fastpath_t1 SELECT * FROM test_1;
                          QUERY PLAN
---------------------------------------------------------------------
 Insert on fastpath_t1  (cost=0.00..22.38 rows=1238 width=36)
   ->  Seq Scan on test_1  (cost=0.00..22.38 rows=1238 width=36)
(2 rows)

gaussdb=# DROP TABLE fastpath_t1;
gaussdb=# DROP TABLE test_1;
```

注意事项及约束条件

1. 只支持INSERT VALUES语句下的执行优化，且VALUES子句后的数据为常量/参数/表达式等类型。
2. 不支持触发器。
3. 不支持UPSERT语句的执行优化。
4. 在CPU为资源瓶颈时能获得较好的提升。

2.8.3.3 分区索引

分区表上的索引共有三种类型：

1. Global Non-Partitioned Index
2. Global Partitioned Index
3. Local Partitioned Index

目前GaussDB支持Global Non-Partitioned Index和Local Partitioned Index类型索引。

 版权所有 © 华为云计算技术有限公司

图 2-5 Global Non-Partitioned Index

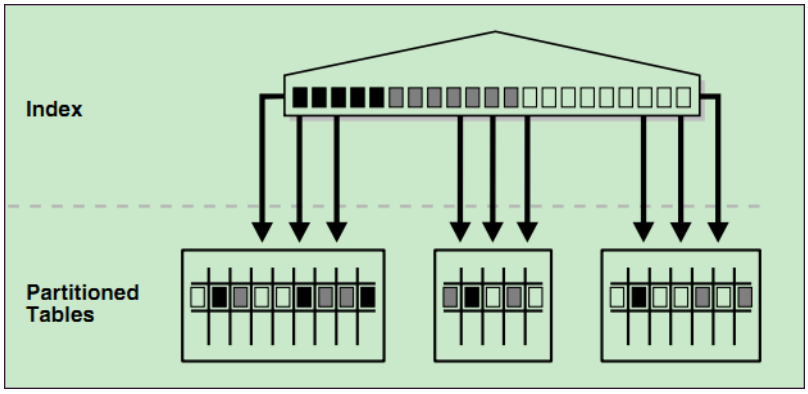


图 2-6 Global Partitioned Index

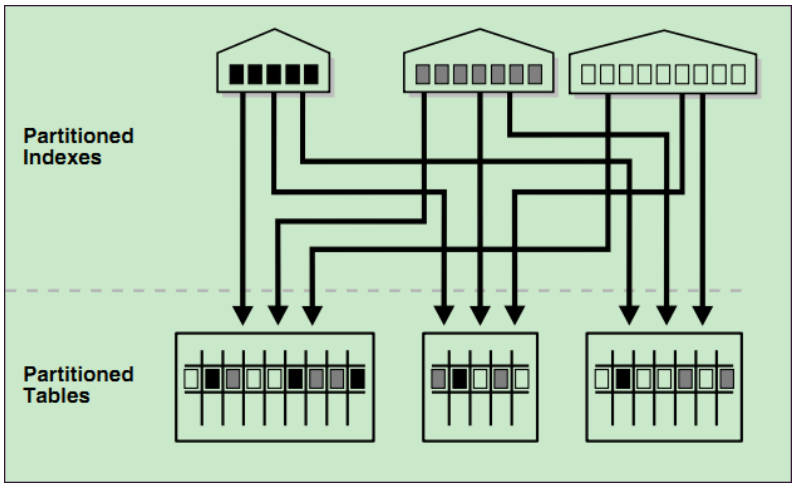
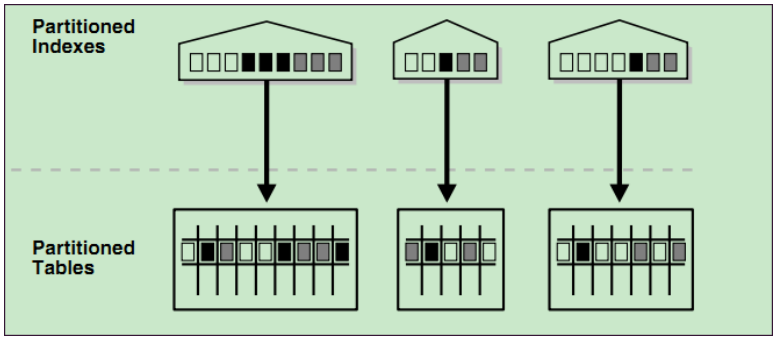


图 2-7 Local Partitioned Index



约束

- 分区表索引分为LOCAL索引与GLOBAL索引：LOCAL索引与某个具体分区绑定，而GLOBAL索引则对应整个分区表。
- 唯一约束和主键约束的约束键包含所有分区键则创建LOCAL索引，否则创建GLOBAL索引。
- 在创建LOCAL索引时，可以通过FOR { partition_name | (partition_value [, ...]) }子句，指定在单个分区上创建LOCAL索引，此类索引在其他分区上不生

版权所有 © 华为云计算技术有限公司

效，后续新增的分区也不会自动创建该索引。需要注意的是，当前仅静态剪枝到单个分区的计划支持生成分类索引的查询路径。

说明

当查询语句在查询数据涉及多个目标分区时，建议使用GLOBAL索引，反之建议使用LOCAL索引。但需要注意GLOBAL索引在分区维护语法中存在额外的开销。

示例

- 创建表

```
gaussdb=# CREATE TABLE web_returns_p2
(
    ca_address_sk INTEGER NOT NULL ,
    ca_address_id CHARACTER(16) NOT NULL ,
    ca_street_number CHARACTER(10) ,
    ca_street_name CHARACTER VARYING(60) ,
    ca_street_type CHARACTER(15) ,
    ca_suite_number CHARACTER(10) ,
    ca_city CHARACTER VARYING(60) ,
    ca_county CHARACTER VARYING(30) ,
    ca_state CHARACTER(2) ,
    ca_zip CHARACTER(10) ,
    ca_country CHARACTER VARYING(20) ,
    ca_gmt_offset NUMERIC(5,2) ,
    ca_location_type CHARACTER(20)
)
PARTITION BY RANGE (ca_address_sk)
(
    PARTITION P1 VALUES LESS THAN(5000),
    PARTITION P2 VALUES LESS THAN(10000),
    PARTITION P3 VALUES LESS THAN(15000),
    PARTITION P4 VALUES LESS THAN(20000),
    PARTITION P5 VALUES LESS THAN(25000),
    PARTITION P6 VALUES LESS THAN(30000),
    PARTITION P7 VALUES LESS THAN(40000),
    PARTITION P8 VALUES LESS THAN(MAXVALUE)
)
ENABLE ROW MOVEMENT;
```

- 创建索引
  - 创建分区表LOCAL索引tpcds_web_returns_p2_index1，不指定索引分区的名称。

    ```
    gaussdb=# CREATE INDEX tpcds_web_returns_p2_index1 ON web_returns_p2 (ca_address_id)
    LOCAL;
    ```

    当结果显示为如下信息，则表示创建成功。

    ```
    CREATE INDEX
    ```

  - 创建分区表LOCAL索引tpcds_web_returns_p2_index2，并指定索引分区的名称。

    ```
    gaussdb=# CREATE TABLESPACE example2 LOCATION '/home/omm/example2';
    CREATE TABLESPACE example3 LOCATION '/home/omm/example3';
    CREATE TABLESPACE example4 LOCATION '/home/omm/example4';

    gaussdb=# CREATE INDEX tpcds_web_returns_p2_index2 ON web_returns_p2 (ca_address_sk)
    LOCAL
    (
        PARTITION web_returns_p2_P1_index,
        PARTITION web_returns_p2_P2_index TABLESPACE example3,
        PARTITION web_returns_p2_P3_index TABLESPACE example4,
        PARTITION web_returns_p2_P4_index,
        PARTITION web_returns_p2_P5_index,
        PARTITION web_returns_p2_P6_index,
        PARTITION web_returns_p2_P7_index,
        PARTITION web_returns_p2_P8_index
    ) TABLESPACE example2;
    ```

 版权所有 © 华为云计算技术有限公司

当结果显示为如下信息，则表示创建成功。

```
CREATE INDEX
```

- 创建分区表GLOBAL索引tpcds_web_returns_p2_global_index。

```
gaussdb=# CREATE INDEX tpcds_web_returns_p2_global_index ON web_returns_p2
(ca_street_number) GLOBAL;
```

当结果显示为如下信息，则表示创建成功。

```
CREATE INDEX
```

- 创建分类分区索引

指定分区名：

```
gaussdb=# CREATE INDEX tpcds_web_returns_for_p1 ON web_returns_p2 (ca_address_id)
LOCAL(partition ind_part for p1);
```

指定分区键的值：

```
gaussdb=# CREATE INDEX tpcds_web_returns_for_p2 ON web_returns_p2 (ca_address_id)
LOCAL(partition ind_part for (5000));
```

当结果显示为如下信息，则表示创建成功。

```
CREATE INDEX
```

- 修改索引分区的表空间
  - 修改索引分区web_returns_p2_P2_index的表空间为example1。

```
gaussdb=# ALTER INDEX tpcds_web_returns_p2_index2 MOVE PARTITION
web_returns_p2_P2_index TABLESPACE example1;
```

当结果显示为如下信息，则表示修改成功。

```
ALTER INDEX
```

  - 修改索引分区web_returns_p2_P3_index的表空间为example2。

```
gaussdb=# ALTER INDEX tpcds_web_returns_p2_index2 MOVE PARTITION
web_returns_p2_P3_index TABLESPACE example2;
```

当结果显示为如下信息，则表示修改成功。

```
ALTER INDEX
```

- 重命名索引分区
  - 执行如下命令对索引分区web_returns_p2_P8_index重命名web_returns_p2_P8_index_new。

```
gaussdb=# ALTER INDEX tpcds_web_returns_p2_index2 RENAME PARTITION
web_returns_p2_P8_index TO web_returns_p2_P8_index_new;
```

当结果显示为如下信息，则表示重命名成功。

```
ALTER INDEX
```

- 查询索引
  - 执行如下命令查询系统和用户定义的所有索引。

```
gaussdb=# SELECT RELNAME FROM PG_CLASS WHERE RELKIND='i' or RELKIND='I';
```

  - 执行如下命令查询指定索引的信息。

```
gaussdb=# \di+ tpcds_web_returns_p2_index2
```

- 删除索引

```
gaussdb=# DROP INDEX tpcds_web_returns_p2_index1;
```

当结果显示为如下信息，则表示删除成功。

```
DROP INDEX
```

- 清理示例

```
gaussdb=# DROP TABLE web_returns_p2;
```

2.8.3.4 分区表统计信息

对于分区表，支持收集分区级统计信息，相关统计信息可以在pg_partition和pg_statistic系统表以及pg_stats和pg_ext_stats视图中查询。分区级统计信息适用于分区表进行静态剪枝后，分区表的扫描范围剪枝到单分区的场景下。分区级统计信息的

 版权所有 © 华为云计算技术有限公司

支持范围为：分区级的page数和tuple数、单列统计信息、多列统计信息、表达式索引统计信息。

分区表统计信息有以下收集方式：

- 级联收集统计信息
- 指定具体单个分区收集统计信息

2.8.3.4.1 级联收集统计信息

在ANALYZE | ANALYSE分区表时，系统会根据用户指定的或默认的PARTITION_MODE，自动收集分区表中所有符合语义的分区级统计信息，PARTITION_MODE的相关信息请参见《开发者指南》中“SQL参考 > SQL语法 > ANALYZE | ANALYSE”中的PARTITION_MODE参数。

⚠ 注意

- 分区级统计信息级联收集不支持default_statistics_target为负数的场景。

示例

- 创建分区表并插入数据

```
gaussdb=# CREATE TABLE t1_range_int
(
    c1 INT,
    c2 INT,
    c3 INT,
    c4 INT
)
PARTITION BY RANGE(c1)
(
    PARTITION range_p00 VALUES LESS THAN(10),
    PARTITION range_p01 VALUES LESS THAN(20),
    PARTITION range_p02 VALUES LESS THAN(30),
    PARTITION range_p03 VALUES LESS THAN(40),
    PARTITION range_p04 VALUES LESS THAN(50)
);
gaussdb=# INSERT INTO t1_range_int SELECT v,v,v,v FROM generate_series(0, 49) AS v;
```

- 级联收集统计信息

```
gaussdb=# ANALYZE t1_range_int WITH ALL;
```

- 查看分区级统计信息

```
gaussdb=# SELECT relname, parttype, relpages, reltuples FROM pg_partition WHERE
parentid=(SELECT oid FROM pg_class WHERE relname='t1_range_int') ORDER BY relname;
   relname    | parttype | relpages | reltuples
--------------+----------+----------+-----------
 range_p00    | p        |        1 |        10
 range_p01    | p        |        1 |        10
 range_p02    | p        |        1 |        10
 range_p03    | p        |        1 |        10
 range_p04    | p        |        1 |        10
 t1_range_int | r        |        0 |         0
(6 rows)

gaussdb=# SELECT
schemaname,tablename,partitionname,subpartitionname,attname,inherited,null_frac,avg_width,n_distinct,n_dndistinct,most_common_vals,most_common_freqs,histogram_bounds FROM pg_stats WHERE
tablename='t1_range_int' ORDER BY tablename, partitionname, attname;
 schemaname |   tablename    | partitionname | subpartitionname | attname | inherited | null_frac |
avg_width | n_distinct | n_dndistinct | most_common_vals | most_common_freqs
|                                                  histogram_bounds
```


版权所有 © 华为云计算技术有限公司

```
------------+--------------+---------------+------------------+---------+-----------+-----------+-----------
+------------+-------------+-----------------+------------------
+-----------------------------------------------------------------------------------------------------
-------------------------------------------------
 public     | t1_range_int | range_p00     |               | c1      | f         |       0 |       4 |       -1
|          0 |             |               | {0,1,2,3,4,5,6,7,8,9}
 public     | t1_range_int | range_p00     |               | c2      | f         |       0 |       4 |       -1
|          0 |             |               | {0,1,2,3,4,5,6,7,8,9}
 public     | t1_range_int | range_p00     |               | c3      | f         |       0 |       4 |       -1
|          0 |             |               | {0,1,2,3,4,5,6,7,8,9}
 public     | t1_range_int | range_p00     |               | c4      | f         |       0 |       4 |       -1
|          0 |             |               | {0,1,2,3,4,5,6,7,8,9}
 public     | t1_range_int | range_p01     |               | c1      | f         |       0 |       4 |       -1
|          0 |             |               | {10,11,12,13,14,15,16,17,18,19}
 public     | t1_range_int | range_p01     |               | c2      | f         |       0 |       4 |       -1
|          0 |             |               | {10,11,12,13,14,15,16,17,18,19}
 public     | t1_range_int | range_p01     |               | c3      | f         |       0 |       4 |       -1
|          0 |             |               | {10,11,12,13,14,15,16,17,18,19}
 public     | t1_range_int | range_p01     |               | c4      | f         |       0 |       4 |       -1
|          0 |             |               | {10,11,12,13,14,15,16,17,18,19}
 public     | t1_range_int | range_p02     |               | c1      | f         |       0 |       4 |       -1
|          0 |             |               | {20,21,22,23,24,25,26,27,28,29}
 public     | t1_range_int | range_p02     |               | c2      | f         |       0 |       4 |       -1
|          0 |             |               | {20,21,22,23,24,25,26,27,28,29}
 public     | t1_range_int | range_p02     |               | c3      | f         |       0 |       4 |       -1
|          0 |             |               | {20,21,22,23,24,25,26,27,28,29}
 public     | t1_range_int | range_p02     |               | c4      | f         |       0 |       4 |       -1
|          0 |             |               | {20,21,22,23,24,25,26,27,28,29}
 public     | t1_range_int | range_p03     |               | c1      | f         |       0 |       4 |       -1
|          0 |             |               | {30,31,32,33,34,35,36,37,38,39}
 public     | t1_range_int | range_p03     |               | c2      | f         |       0 |       4 |       -1
|          0 |             |               | {30,31,32,33,34,35,36,37,38,39}
 public     | t1_range_int | range_p03     |               | c3      | f         |       0 |       4 |       -1
|          0 |             |               | {30,31,32,33,34,35,36,37,38,39}
 public     | t1_range_int | range_p03     |               | c4      | f         |       0 |       4 |       -1
|          0 |             |               | {30,31,32,33,34,35,36,37,38,39}
 public     | t1_range_int | range_p04     |               | c1      | f         |       0 |       4 |       -1
|          0 |             |               | {40,41,42,43,44,45,46,47,48,49}
 public     | t1_range_int | range_p04     |               | c2      | f         |       0 |       4 |       -1
|          0 |             |               | {40,41,42,43,44,45,46,47,48,49}
 public     | t1_range_int | range_p04     |               | c3      | f         |       0 |       4 |       -1
|          0 |             |               | {40,41,42,43,44,45,46,47,48,49}
 public     | t1_range_int | range_p04     |               | c4      | f         |       0 |       4 |       -1
|          0 |             |               | {40,41,42,43,44,45,46,47,48,49}
 public     | t1_range_int |               |               | c1      | f         |       0 |       4 |       -1 |          0
|             |               |
{0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,
34,35,36,37,38,39,40,41,42,43,44,45,46,47,48,49}
 public     | t1_range_int |               |               | c2      | f         |       0 |       4 |       -1 |          0
|             |               |
{0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,
34,35,36,37,38,39,40,41,42,43,44,45,46,47,48,49}
 public     | t1_range_int |               |               | c3      | f         |       0 |       4 |       -1 |          0
|             |               |
{0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,
34,35,36,37,38,39,40,41,42,43,44,45,46,47,48,49}
 public     | t1_range_int |               |               | c4      | f         |       0 |       4 |       -1 |          0
|             |               |
{0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,
34,35,36,37,38,39,40,41,42,43,44,45,46,47,48,49}
(24 rows)
```

- 生成多列数据的分区级统计信息

```
gaussdb=# ALTER TABLE t1_range_int ADD STATISTICS ((c2, c3));
gaussdb=# ANALYZE t1_range_int WITH ALL;
```

- 查看多列数据的分区级统计信息

```
gaussdb=# SELECT
schemaname,tablename,partitionname,subpartitionname,attname,inherited,null_frac,avg_width,n_disti
```

 版权所有 © 华为云计算技术有限公司

```
nct,n_dndistinct,most_common_vals,most_common_freqs,histogram_bounds FROM pg_ext_stats
WHERE tablename='t1_range_int' ORDER BY tablename,partitionname,attname;
 schemaname |   tablename    | partitionname | subpartitionname | attname | inherited | null_frac |
avg_width | n_distinct | n_dndistinct | most_common_vals | most_common_freqs | histogram_bounds
------------+--------------+---------------+------------------+---------+-----------+-----------
+------------+--------------+-----------------+------------------+-----------------
 public     | t1_range_int | range_p00     |                  | 2 3     | f         |         0 |         8 |        -1
|          0 |              |                 |
 public     | t1_range_int | range_p01     |                  | 2 3     | f         |         0 |         8 |        -1
|          0 |              |                 |
 public     | t1_range_int | range_p02     |                  | 2 3     | f         |         0 |         8 |        -1
|          0 |              |                 |
 public     | t1_range_int | range_p03     |                  | 2 3     | f         |         0 |         8 |        -1
|          0 |              |                 |
 public     | t1_range_int | range_p04     |                  | 2 3     | f         |         0 |         8 |        -1
|          0 |              |                 |
 public     | t1_range_int |               |                  | 2 3     | f         |         0 |         8 |        -1 |          0
|            |              |
(6 rows)
```

- 创建表达式索引并生成对应的分区级统计信息

```
gaussdb=# CREATE INDEX t1_range_int_index ON t1_range_int(text(c1)) LOCAL;
gaussdb=# ANALYZE t1_range_int WITH ALL;
```

- 查看表达式索引的分区级统计信息

```
gaussdb=# SELECT
schemaname,tablename,partitionname,subpartitionname,attname,inherited,null_frac,avg_width,n_disti
nct,n_dndistinct,most_common_vals,most_common_freqs,histogram_bounds FROM pg_stats WHERE
tablename='t1_range_int_index' ORDER BY tablename,partitionname,attname;
 schemaname |      tablename       |    partitionname     | subpartitionname | attname | inherited |
null_frac | avg_width | n_distinct | n_dndistinct | most_common_vals | most_common_freqs
|                                                  histogram_bounds

------------+--------------------+--------------------+------------------+---------+-----------+-----------
+-----------+------------+--------------+------------------+-------------------
+-------------------------------------------------------------------------------
---------------------------------------------------------------
 public     | t1_range_int_index | range_p00_text_idx |                  | text    | f         |         0 |         5
|         -1 |          0 |              |                  | {0,1,2,3,4,5,6,7,8,9}
 public     | t1_range_int_index | range_p01_text_idx |                  | text    | f         |         0 |         6
|         -1 |          0 |              |                  | {10,11,12,13,14,15,16,17,18,19}
 public     | t1_range_int_index | range_p02_text_idx |                  | text    | f         |         0 |         6
|         -1 |          0 |              |                  | {20,21,22,23,24,25,26,27,28,29}
 public     | t1_range_int_index | range_p03_text_idx |                  | text    | f         |         0 |         6
|         -1 |          0 |              |                  | {30,31,32,33,34,35,36,37,38,39}
 public     | t1_range_int_index | range_p04_text_idx |                  | text    | f         |         0 |         6
|         -1 |          0 |              |                  | {40,41,42,43,44,45,46,47,48,49}
 public     | t1_range_int_index |                    |                  | text    | f         |         0 |         5 |        -1
|          0 |              |                 |
{0,1,10,11,12,13,14,15,16,17,18,19,2,20,21,22,23,24,25,26,27,28,29,3,30,31,32,33,3
4,35,36,37,38,39,4,40,41,42,43,44,45,46,47,48,49,5,6,7,8,9}
(6 rows)
```

- 删除分区表

```
gaussdb=# DROP TABLE t1_range_int;
```

2.8.3.4.2 分区级统计信息

指定单分区统计信息收集

当前分区表支持指定单分区统计信息收集，已收集统计信息的分区会在再次收集时自动更新维护。该功能适用于列表分区、哈希分区和范围分区。

```
gaussdb=# CREATE TABLE only_first_part(id int,name varchar)PARTITION BY RANGE (id)
 (PARTITION id11 VALUES LESS THAN (1000000),
PARTITION id22 VALUES LESS THAN (2000000),
PARTITION max_id1 VALUES LESS THAN (MAXVALUE));
```

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# INSERT INTO only_first_part SELECT generate_series(1,5000),'test';

gaussdb=# ANALYZE only_first_part PARTITION (id11);
gaussdb=# ANALYZE only_first_part PARTITION (id22);
gaussdb=# ANALYZE only_first_part PARTITION (max_id1);

gaussdb=# SELECT relname, relpages, reltuples FROM pg_partition WHERE relname IN ('id11', 'id22',
'max_id1');
 relname | relpages | reltuples
---------+----------+-----------
 id11    |       20 |      5000
 id22    |        0 |         0
 max_id1 |        0 |         0
(3 rows)

gaussdb=# \x
gaussdb=# SELECT * FROM pg_stats WHERE tablename ='only_first_part' AND partitionname ='id11';
-[ RECORD 1 ]----------
+-----------------------
```

```
-- delete partition table
gaussdb=# DROP TABLE only_first_part;
```

优化器使用指定分区统计信息

优化器优先使用指定分区的统计信息。如果指定分区未收集统计信息，优化器使用改写分区子句剪枝优化，请参见**通过改写分区子句剪枝优化**。

```
gaussdb=# CREATE TABLE only_first_part_two
(
    c1 INT,
    c2 BIGINT
)
PARTITION BY LIST(c2)
(
    PARTITION p_1 VALUES (10000, 20000),
    PARTITION p_2 VALUES (300000, 400000, 500000),
    PARTITION p_3 VALUES (DEFAULT)
);

gaussdb=# EXPLAIN SELECT * FROM only_first_part_two PARTITION (p_2);

                                  QUERY PLAN
-------------------------------------------------------------------------------------------
 Partition Iterator  (cost=0.00..29.45 rows=1945 width=12)
   Iterations: 1
   ->  Partitioned Seq Scan on only_first_part_two  (cost=0.00..29.45 rows=1945 width=12)
         Selected Partitions:  2
(4 rows)

gaussdb=# EXPLAIN SELECT * FROM only_first_part_two PARTITION (p_1) WHERE c2 = 1;
                                  QUERY PLAN
-------------------------------------------------------------------------------------------
 Partition Iterator  (cost=0.00..34.31 rows=10 width=12)
   Iterations: 0
   ->  Partitioned Seq Scan on only_first_part_two  (cost=0.00..34.31 rows=10 width=12)
         Filter: (c2 = 1)
         Selected Partitions:  NONE
(5 rows)
-- drop partition table
gaussdb=# DROP TABLE only_first_part_two;
```

通过改写分区子句剪枝优化

当没有分区级统计信息时，在优化器行数估算模块，通过在逻辑上对分区子句进行伪谓词的改写，利用改写后的伪谓词影响选择率的计算和整表的统计信息获取一个比较准确的行数估算值。

说明

- 特性只作用于选择率的计算。
- 特性适用于一级分区和二级分区。
- 特性只支持范围分区（range partition）、间隔分区（interval partition）和列表分区（list partition）。
- 对于范围分区和间隔分区，特性只支持单列分区键的改写，不支持多列分区键的改写。
- 对于列表分区，出于性能考虑，设置列表指定分区的枚举值个数的阈值为40个。
  - 当指定分区的列表枚举值个数超过40时，本特性不再适用。
  - 对于default分区，其列表枚举值个数是所有非default分区的枚举值个数的总和。

示例1：对于范围/间隔分区的改写

```
gaussdb=# CREATE TABLE test_int4_maxvalue(id INT, name VARCHAR)
PARTITION BY RANGE(id)
```

 版权所有 © 华为云计算技术有限公司

```
(
    PARTITION id1 VALUES LESS THAN(1000),
    PARTITION id2 VALUES LESS THAN(2000),
    PARTITION max_id VALUES LESS THAN(MAXVALUE)
);
gaussdb=# INSERT INTO test_int4_maxvalue SELECT GENERATE_SERIES(1,5000),'test';
gaussdb=# ANALYZE test_int4_maxvalue WITH GLOBAL;

-- 查询指定分区id1
gaussdb=# EXPLAIN SELECT * FROM test_int4_maxvalue PARTITION(id1);
                                  QUERY PLAN
-------------------------------------------------------------------------------------------
 Partition Iterator  (cost=0.00..51.00 rows=1000 width=9)
   Iterations: 1
   ->  Partitioned Seq Scan on test_int4_maxvalue  (cost=0.00..51.00 rows=1000 width=9)
         Selected Partitions:  1
(4 rows)

-- 查询指定分区max_id
gaussdb=# EXPLAIN SELECT * FROM test_int4_maxvalue PARTITION(max_id);
                                  QUERY PLAN
-------------------------------------------------------------------------------------------
 Partition Iterator  (cost=0.00..51.00 rows=3000 width=9)
   Iterations: 1
   ->  Partitioned Seq Scan on test_int4_maxvalue  (cost=0.00..51.00 rows=3000 width=9)
         Selected Partitions:  3
(4 rows)

-- drop partition table
gaussdb=# DROP TABLE test_int4_maxvalue;
```

示例2：对于列表分区的改写

```
gaussdb=# CREATE TABLE test_default
(
    c1 INT,
    c2 BIGINT
)
PARTITION BY LIST(c2)
(
    PARTITION p_1 VALUES (10000, 20000),
    PARTITION p_2 VALUES (300000, 400000, 500000),
    PARTITION p_3 VALUES (DEFAULT)
);
gaussdb=# INSERT INTO test_default SELECT GENERATE_SERIES(1, 1000), 10000;
gaussdb=# INSERT INTO test_default SELECT GENERATE_SERIES(1001, 2000), 600000;
gaussdb=# ANALYZE test_default WITH GLOBAL;

-- 查询指定分区p_1
gaussdb=# EXPLAIN SELECT * FROM test_default PARTITION(p_1);
                                QUERY PLAN
-------------------------------------------------------------------------------------
 Partition Iterator  (cost=0.00..28.00 rows=1000 width=12)
   Iterations: 1
   ->  Partitioned Seq Scan on test_default  (cost=0.00..28.00 rows=1000 width=12)
         Selected Partitions:  1
(4 rows)

-- 查询指定分区p_3
gaussdb=# EXPLAIN SELECT * FROM test_default PARTITION(p_3);
                                QUERY PLAN
-------------------------------------------------------------------------------------
 Partition Iterator  (cost=0.00..28.00 rows=1000 width=12)
   Iterations: 1
   ->  Partitioned Seq Scan on test_default  (cost=0.00..28.00 rows=1000 width=12)
         Selected Partitions:  3
(4 rows)

-- drop partition table
gaussdb=# DROP TABLE test_default;
```

 版权所有 © 华为云计算技术有限公司

2.8.4 分区自动扩展

分区的自动扩展功能是分区表的一种能力增强。当DML业务（INSERT、UPDATE、UPSERT、MERGE INTO、COPY）新增数据无法匹配到已有的任一分区时，会自动创建一个新的分区。此外，以partition/subpartition for partition_value的方式创建分类索引时，若指定的分区不存在，也会自动创建一个新的分区。当前支持范围分区自动扩展和列表分区自动扩展。

2.8.4.1 范围分区自动扩展

范围分区的自动扩展即**间隔分区**。开启范围分区自动扩展功能，需要在创建分区时明确指定INTERVAL子句。当前只支持一级间隔分区表，且只支持单列分区键。

```
-- 创建分区表并指定INTERVAL子句，表示支持范围分区自动扩展。
gaussdb=# CREATE TABLE interval_int (c1 int, c2 int)
PARTITION BY RANGE (c1) INTERVAL (5)
(
    PARTITION p1 VALUES LESS THAN (5),
    PARTITION p2 VALUES LESS THAN (10),
    PARTITION p3 VALUES LESS THAN (15)
);
```

当插入数据无法匹配到已有的任意分区时，会自动创建一个新的分区，新分区的范围定义由上一个分区范围和INTERVAL值决定。

```
-- 分区键插入数据23，自动创建分区sys_p1，分区范围定义为[20, 25）。
gaussdb=# INSERT INTO interval_int VALUES (23, 0);
-- 清理示例
gaussdb=# DROP TABLE interval_int;
```

2.8.4.2 列表分区自动扩展

开启列表分区的自动扩展功能，需要在创建分区时指定AUTOMATIC关键字。列表分区自动扩展支持一级分区和二级分区自动扩展。

说明

- 使用列表分区的自动扩展功能，要求分区表中不能包含分区键值为DEFAULT的分区。
- 使用列表分区的自动扩展功能，需合理设计分区列，避免大量扩展分区行为导致分区数急剧增加，影响业务性能。

2.8.4.2.1 一级分区表自动扩展

开启列表分区的自动扩展功能，需要在创建一级列表分区表时指定AUTOMATIC关键字。一级列表分区表自动扩展支持多列分区键。

例如，创建一个支持自动扩展的列表分区表。

```
gaussdb=# CREATE TABLE auto_list (c1 int, c2 int)
PARTITION BY LIST (c1) AUTOMATIC
(
    PARTITION p1 VALUES (1, 2, 3),
    PARTITION p2 VALUES (4, 5, 6)
);
```

当插入数据无法匹配到已有的任意分区时，会自动创建一个新的分区，新分区的范围定义为单key。

```
--分区键插入数据9，自动创建分区sys_p1，分区定义为VALUES (9)
gaussdb=# INSERT INTO auto_list VALUES (9, 0);
```

这一功能与如下命令等价：

版权所有 © 华为云计算技术有限公司

```
ALTER TABLE auto_list ADD PARTITION sys_p1 VALUES (9);
INSERT INTO auto_list VALUES (9, 0);
gaussdb=# DROP TABLE auto_list;
```

2.8.4.2.2 二级分区表自动扩展

创建二级分区表时，可以在创建列表分区定义上指定AUTOMATIC关键字，以支持二级分区表的一级自动扩展/二级自动扩展。二级列表分区表自动扩展只支持单列分区键。

- 创建二级分区表时，在创建一级分区定义上指定AUTOMATIC，以支持一级自动扩展。

```
gaussdb=# CREATE TABLE autolist_range (c1 int, c2 int)
PARTITION BY LIST (c1) AUTOMATIC SUBPARTITION BY RANGE (c2)
(
   PARTITION p1 VALUES (1, 2, 3) (
      SUBPARTITION sp11 VALUES LESS THAN (5),
      SUBPARTITION sp12 VALUES LESS THAN (10)
   ),
   PARTITION p2 VALUES (4, 5, 6) (
      SUBPARTITION sp21 VALUES LESS THAN (5),
      SUBPARTITION sp22 VALUES LESS THAN (10)
   )
);
```

  当插入数据无法匹配到已有的任意一级分区时，会自动创建一个新的一级分区，新一级分区的范围定义为单key（新数据对应的新分区键值），其下面会定义一个全集的二级分区。

```
--一级分区键插入数据9，因为现有的一级分区p1、p2的键值中不包含9，所以自动创建一个新的一级分区sys_p1，分区定义为VALUES (9)
gaussdb=# INSERT INTO autolist_range VALUES (9, 0);
```

  这一功能与如下命令等价：

```
gaussdb=# ALTER TABLE autolist_range ADD PARTITION sys_p1 VALUES (9);
gaussdb=# INSERT INTO autolist_range VALUES (9, 0);
gaussdb=# DROP TABLE autolist_range;
```

- 创建二级分区表时，在二级分区定义上指定AUTOMATIC，以支持二级自动扩展。

```
gaussdb=# CREATE TABLE range_autolist (c1 int, c2 int)
PARTITION BY RANGE (c1) SUBPARTITION BY LIST (c2) AUTOMATIC
(
   PARTITION p1 VALUES LESS THAN (5) (
      SUBPARTITION sp11 VALUES (1, 2, 3),
      SUBPARTITION sp12 VALUES (4, 5, 6)
   ),
   PARTITION p2 VALUES LESS THAN (10) (
      SUBPARTITION sp21 VALUES (1, 2, 3),
      SUBPARTITION sp22 VALUES (4, 5, 6)
   )
);
```

  当插入数据无法匹配到已有的任意二级分区时，会在对应的一级分区下自动创建一个新的二级分区，新二级分区的范围定义为单key（新数据对应的新分区键值）。

```
--二级分区键插入数据0，因为现有的二级分区的键值中不包含0，所以自动创建一个新的二级分区sys_sp1，分区定义为VALUES (0)
gaussdb=# INSERT INTO range_autolist VALUES (4, 0);
```

  这一功能与如下命令等价：

```
gaussdb=# ALTER TABLE range_autolist MODIFY PARTITION p1 ADD SUBPARTITION sys_sp1 VALUES (0);
gaussdb=# INSERT INTO range_autolist VALUES (4, 0);

-- 清理示例
gaussdb=# DROP TABLE range_autolist;
```

- 创建二级分区表时，在一级/二级分区定义上同时指定AUTOMATIC，表示支持一级自动扩展/二级自动扩展。

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# CREATE TABLE autolist_autolist (c1 int, c2 int)
PARTITION BY LIST (c1) AUTOMATIC SUBPARTITION BY LIST (c2) AUTOMATIC
(
   PARTITION p1 VALUES (1, 2, 3) (
      SUBPARTITION sp11 VALUES (1, 2, 3),
      SUBPARTITION sp12 VALUES (4, 5, 6)
   ),
   PARTITION p2 VALUES (4, 5, 6) (
      SUBPARTITION sp21 VALUES (1, 2, 3),
      SUBPARTITION sp22 VALUES (4, 5, 6)
   )
);
```

- 当插入数据无法匹配到已有的任意一级分区时，会自动创建一个新的一级分区，新一级分区的范围定义为单key（新数据对应的新分区键值），其下面会定义一个范围定义为单key的二级分区。

  ```
  --一级分区键插入数据9，因为现有的一级分区p1、p2的键值中不包含9，所以自动创建一个新的一级分区sys_p1，分区定义为VALUES (9)；同时二级分区键插入数据0，因为现有的二级分区的键值中不包含0，所以会在新的一级分区sys_p1定义一个新的二级分区sys_sp1，分区定义为VALUES (0)。
  gaussdb=# INSERT INTO autolist_autolist VALUES (9, 0);
  ```

  这一功能与如下命令等价：

  ```
  gaussdb=# ALTER TABLE autolist_autolist ADD PARTITION sys_p1 VALUES (9) (SUBPARTITION sys_sp1 VALUES (0));
  gaussdb=# INSERT INTO autolist_autolist VALUES (9, 0);
  ```

- 当插入数据无法匹配到已有的任意二级分区时，会在对应的一级分区下自动创建一个新的二级分区，新二级分区的范围定义为单key（新数据对应的新分区键值）。

  ```
  --二级分区键插入数据0，因为现有的二级分区的键值中不包含0，所以自动创建二级分区sys_sp2，分区定义为VALUES (0)
  gaussdb=# INSERT INTO autolist_autolist VALUES (4, 0);
  ```

  这一功能与如下命令等价：

  ```
  gaussdb=# ALTER TABLE autolist_autolist MODIFY PARTITION p2 ADD SUBPARTITION sys_sp2 VALUES (0);
  gaussdb=# INSERT INTO autolist_autolist VALUES (4, 0);

  -- 清理示例
  gaussdb=# DROP TABLE autolist_autolist;
  ```

说明

二级分区表的列表分区自动扩展行为受AUTOMATIC关键字的指定位置影响：

- 若在一级分区后指定了AUTOMATIC关键字，则仅支持一级分区自动扩展，不支持二级分区的自动扩展，且不能定义有一级分区键值为DEFAULT分区。
- 若在二级分区后指定了AUTOMATIC关键字，则仅支持二级分区自动扩展，不支持一级分区的自动扩展，且不能定义有二级分区键值为DEFAULT分区。
- 若在一级分区和二级分区后同时指定了AUTOMATIC关键字，则同时支持一级分区和二级分区自动扩展，一级分区键值和二级分区键值均不能定义有DEFAULT分区。

2.8.4.3 开启/关闭分区自动扩展

用户可以通过ALTER命令来对已创建的分区表开启/关闭分区自动扩展功能。这一操作会对分区表持有SHARE_UPDATE_EXCLUSIVE级别的表锁，与常规DQL/DML业务互不影响，但与DDL业务相互排斥。若DML业务触发自动扩展分区，也会与之互斥。不同级别锁的行为控制请参见**常规锁设计**。

2.8.4.3.1 开启/关闭范围分区自动扩展

使用ALTER TABLE SET INTERVAL可以开启/关闭范围分区自动扩展。

例如，开启范围分区自动扩展。

版权所有 © 华为云计算技术有限公司

```
gaussdb=# CREATE TABLE range_int (c1 int, c2 int)
PARTITION BY RANGE (c1)
(
    PARTITION p1 VALUES LESS THAN (5),
    PARTITION p2 VALUES LESS THAN (10),
    PARTITION p3 VALUES LESS THAN (15)
);

gaussdb=# ALTER TABLE range_int SET INTERVAL (5);
```

说明

- 开启范围分区自动扩展要求分区表中不能存在分区键值为MAXVALUE的分区。
- 开启范围分区自动扩展只支持一级分区表、单列分区键。

关闭范围分区自动扩展。

```
gaussdb=# ALTER TABLE range_int SET INTERVAL ();

-- 清理示例
gaussdb=# DROP TABLE range_int;
```

2.8.4.3.2 开启/关闭一级列表分区自动扩展

使用ALTER TABLE SET PARTITIONING 可以开启/关闭一级列表分区自动扩展。

例如：

- 开启一级列表分区表自动扩展。

  ```
  gaussdb=# CREATE TABLE list_int (c1 int, c2 int)
  PARTITION BY LIST (c1)
  (
      PARTITION p1 VALUES (1, 2, 3),
      PARTITION p2 VALUES (4, 5, 6)
  );

  gaussdb=# ALTER TABLE list_int SET PARTITIONING AUTOMATIC;
  ```

  或者：

  ```
  gaussdb=# CREATE TABLE list_range (c1 int, c2 int)
  PARTITION BY LIST (c1) SUBPARTITION BY RANGE (c2)
  (
      PARTITION p1 VALUES (1, 2, 3) (
          SUBPARTITION sp11 VALUES LESS THAN (5),
          SUBPARTITION sp12 VALUES LESS THAN (10)
      ),
      PARTITION p2 VALUES (4, 5, 6) (
          SUBPARTITION sp21 VALUES LESS THAN (5),
          SUBPARTITION sp22 VALUES LESS THAN (10)
      )
  );

  gaussdb=# ALTER TABLE list_range SET PARTITIONING AUTOMATIC;
  ```

  说明

  开启一级列表分区自动扩展功能要求一级分区表、一级分区中不能存在分区键值为DEFAULT的分区。

- 关闭一级列表分区表自动扩展。

  ```
  gaussdb=# ALTER TABLE list_int SET PARTITIONING MANUAL;
  ```

  或者：

  ```
  gaussdb=# ALTER TABLE list_range SET PARTITIONING MANUAL;
  ```

  清理示例：

  ```
  gaussdb=# DROP TABLE list_int;
  gaussdb=# DROP TABLE list_range;
  ```

 版权所有 © 华为云计算技术有限公司

2.8.4.3.3 开启/关闭二级列表分区自动扩展

使用ALTER TABLE SET SUBPARTITIONING可以开启/关闭二级列表分区自动扩展功能。

例如：

- 开启二级列表分区自动扩展。

```
gaussdb=# CREATE TABLE range_list (c1 int, c2 int)
PARTITION BY RANGE (c1) SUBPARTITION BY LIST (c2)
(
   PARTITION p1 VALUES LESS THAN (5) (
      SUBPARTITION sp11 VALUES (1, 2, 3),
      SUBPARTITION sp12 VALUES (4, 5, 6)
   ),
   PARTITION p2 VALUES LESS THAN (10) (
      SUBPARTITION sp21 VALUES (1, 2, 3),
      SUBPARTITION sp22 VALUES (4, 5, 6)
   )
);
gaussdb=# ALTER TABLE range_list SET SUBPARTITIONING AUTOMATIC;
```

> **说明**
>
> 开启二级列表分区自动扩展要求二级分区中不能存在分区键值为DEFAULT的分区。

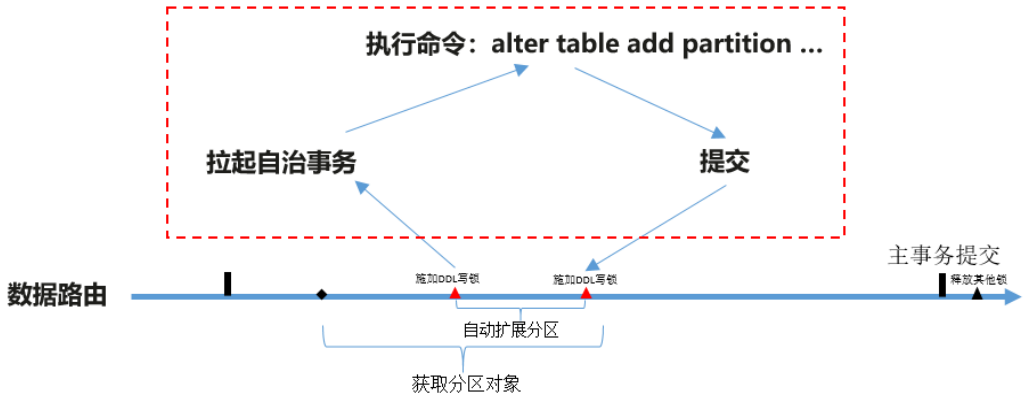
- 关闭二级列表分区自动扩展。

```
gaussdb=# ALTER TABLE range_list SET SUBPARTITIONING MANUAL;

--清理示例
gaussdb=# DROP TABLE range_list;
```

2.8.4.4 自动扩展分区的创建策略

分区自动扩展是一个自动提交的过程，当DML插入的数据无法匹配到已有的任意分区或创建分类索引指定的分区不存在时，会触发自治事务执行分区自动扩展。这一过程会对分区表施加短暂的锁定，与其他分区DDL命令相互阻塞。阻塞周期极为短暂，对系统运行或用户操作基本无影响。



分区自动扩展的行为表现如下：

- 通过DML业务自动扩展的分区不支持回滚，即当前事务回滚后，新建的分区依然存在。可以通过查询系统表PG_PARTITION查看新建的分区。
- 通过创建分类索引触发自动扩展分区时，若创建索引事务异常回滚，新建的分区可能存在（异常回滚点在触发自动扩展分区之后），也可能不存在（异常回滚点在触发自动扩展分区之前）。可以通过查询系统表PG_PARTITION查看新建的分区。

版权所有 © 华为云计算技术有限公司

- 分区自动扩展与常规分区DQL/DML业务互不阻塞，支持这两类业务的并发。
- 分区自动扩展过程会短暂施加锁定，系统运行或用户操作基本无影响，支持多个线程因DML/分类索引业务同时触发分区自动扩展场景的并发。
- 分区自动扩展过程与分区DDL互斥，若有其他线程执行分区DDL且未提交，如果当前线程DML/分类索引业务触发分区自动扩展，则会被阻塞；但若其他线程执行DML/分类索引业务触发分区自动扩展且未提交，则不会阻塞当前线程的分区DDL业务。

自动扩展分区可以通过以下两种方式创建：

- DML业务导致的新增数据，无法匹配到任意已有分区，此时会优先基于规则创建一个新的分区，再向新分区插入对应数据。
- 以PARTITION/SUBPARTITION FOR partition_value的方式创建分类索引，若指定的分区不存在，此时优先基于规则创建一个新的分区，再在新分区上创建分类索引。

2.8.5 分区表运维管理

分区表运维管理包括分区管理、分区表管理、分区索引管理和分区表业务并发支持等。

- 分区管理：也称分区级DDL，包括新增（Add）、删除（Drop）、交换（Exchange）、清空（Truncate）、分割（Split）、合并（Merge）、移动（Move）、重命名（Rename）共8种。

⚠ 注意

- 对于哈希分区，涉及分区数的变更会导致数据re-shuffling，故当前GaussDB不支持导致Hash分区数变更的操作，包括新增（Add）、删除（Drop）、分割（Split）、合并（Merge）这4种。
- 涉及分区数据变更的操作会使得Global索引失效，可以通过UPDATE GLOBAL INDEX子句来同步更新Global索引，包括删除（Drop）、交换（Exchange）、清空（Truncate）、分割（Split）、合并（Merge）这5种。

说明

- 大部分分区DDL支持partition/subpartition和partition/subpartition for指定分区两种写法，前者需要指定分区名，后者需要指定分区定义范围内的任一分区值。比如假设分区part1的范围定义为[100, 200），那么partition part1和partition for(150)这两种写法是等价的。
- 不同分区DDL的执行代价各不相同，由于在执行分区DDL过程中目标分区会被锁住，用户需要评估其代价以及对业务的影响。一般而言，分割（Split）、合并（Merge）的执行代价远大于其他分区DDL，与源分区的大小正相关；交换（Exchange）的代价主要源于Global索引的重建和validation校验；移动（Move）的代价限制于磁盘I/O；其余分区DDL的执行代价都很低。

- 分区表管理：除了继承普通表的功能外，还支持开启/关闭分区表行迁移的功能。
- 分区索引管理：支持用户设置索引/索引分区不可用，或者重建不可用的索引/索引分区，比如由于分区管理操作导致的Global索引失效场景。
- 分区表业务并发支持：当分区级DDL与分区DQL/DML作用于不同分区时，支持二者执行层面的并发。

版权所有 © 华为云计算技术有限公司

2.8.5.1 新增分区

用户可以在已建立的分区表中新增分区，来维护新业务的进行。当前各种分区表支持的分区上限为1048575，如果达到了上限则不能继续添加分区。同时需要考虑分区占用内存的开销，分区表使用内存大致为（分区数 * 3 / 1024）MB，分区占用内存不允许大于local_syscache_threshold的值，同时还需要预留部分空间以供其他功能使用。

⚠ 注意

- 新增分区不能作用于HASH分区上。
- 新增分区不继承表上的分类索引属性。

2.8.5.1.1 向范围分区表新增分区

使用ALTER TABLE ADD PARTITION可以将分区添加到现有分区表的最后面，新增分区的上界值必须大于当前最后一个分区的上界值。

例如，对范围分区表range_sales新增一个分区。

```
ALTER TABLE range_sales ADD PARTITION date_202005 VALUES LESS THAN ('2020-06-01') TABLESPACE tb1;
```

须知

当范围分区表有MAXVALUE分区时，无法新增分区。可以使用ALTER TABLE SPLIT PARTITION命令分割分区。分割分区同样适用于需要在现有分区表的前面/中间添加分区的情形，请参见**对范围分区表分割分区**。

2.8.5.1.2 向间隔分区表新增分区

不支持通过ALTER TABLE ADD PARTITION命令向间隔分区表新增分区。当用户插入数据超出现有间隔分区表范围时，数据库会自动根据间隔分区的INTERVAL值创建一个分区。

例如，对间隔分区表interval_sales插入如下数据后，数据库会创建一个分区，该分区范围为['2020-07-01', '2020-08-01')，间隔分区的新增分区命名从sys_p1开始递增。

```
INSERT INTO interval_sales VALUES (263722,42819872,'2020-07-09','E',432072,213,17);
```

2.8.5.1.3 向列表分区表新增分区

使用ALTER TABLE ADD PARTITION可以在列表分区表中新增分区，新增分区的枚举值不能与已有的任一个分区的枚举值重复。

例如，对列表分区表list_sales新增一个分区。

```
ALTER TABLE list_sales ADD PARTITION channel5 VALUES ('X') TABLESPACE tb1;
```

须知

当列表分区表有DEFAULT分区时，无法新增分区。可以使用ALTER TABLE SPLIT PARTITION命令分割分区。

版权所有 © 华为云计算技术有限公司

2.8.5.1.4 向二级分区表新增一级分区

使用ALTER TABLE ADD PARTITION可以在二级分区表中新增一个一级分区，这个行为可以作用在一级分区策略为RANGE或者LIST的情况。如果这个新增一级分区下申明了二级分区定义，则数据库会根据定义创建对应的二级分区；如果这个新增一级分区下没有申明二级分区定义，则数据库会自动创建一个默认的二级分区。

例如，对二级分区表range_list_sales新增一个一级分区，并在下面创建四个二级分区。

```
ALTER TABLE range_list_sales ADD PARTITION date_202005 VALUES LESS THAN ('2020-06-01')
TABLESPACE tb1
(
    SUBPARTITION date_202005_channel1 VALUES ('0', '1', '2'),
    SUBPARTITION date_202005_channel2 VALUES ('3', '4', '5') TABLESPACE tb2,
    SUBPARTITION date_202005_channel3 VALUES ('6', '7'),
    SUBPARTITION date_202005_channel4 VALUES ('8', '9')
);
```

或者对二级分区表range_list_sales只进行新增一级分区操作。

```
ALTER TABLE range_list_sales ADD PARTITION date_202005 VALUES LESS THAN ('2020-06-01')
TABLESPACE tb1;
```

上面这种行为与如下SQL语句等价。

```
ALTER TABLE range_list_sales ADD PARTITION date_202005 VALUES LESS THAN ('2020-06-01')
TABLESPACE tb1
(
    SUBPARTITION date_202005_channel1 VALUES (DEFAULT)
);
```

须知

当二级分区表的一级分区策略为HASH时，不支持通过ALTER TABLE ADD PARTITION命令新增一级分区。

2.8.5.1.5 向二级分区表新增二级分区

使用ALTER TABLE MODIFY PARTITION ADD SUBPARTITION可以在二级分区表中新增一个二级分区，这个行为可以作用在二级分区策略为RANGE或者LIST的情况。

例如，对二级分区表range_list_sales的date_202004新增一个二级分区。

```
ALTER TABLE range_list_sales MODIFY PARTITION date_202004 ADD SUBPARTITION date_202004_channel5
VALUES ('X') TABLESPACE tb2;
```

须知

当二级分区表的二级分区策略为HASH时，不支持通过ALTER TABLE MODIFY PARTITION ADD SUBPARTITION命令新增二级分区。

2.8.5.2 删除分区

用户可以使用删除分区的命令来移除不需要的分区。删除分区可以通过指定分区名或者分区值来进行。

版权所有 © 华为云计算技术有限公司

⚠ 注意

- 删除分区不能作用于HASH分区上。
- 执行删除分区命令会使得Global索引失效，可以通过UPDATE GLOBAL INDEX子句来同步更新Global索引，或者用户自行重建Global索引。
- 删除分区时，如果该分区上带有仅属于当前分区的分类索引时，则会级联删除分类索引。

2.8.5.2.1 对一级分区表删除分区

使用ALTER TABLE DROP PARTITION可以删除指定分区表的任何一个分区，这个行为可以作用在范围分区表、间隔分区表、列表分区表上。

例如，通过指定分区名删除范围分区表range_sales的分区date_202005，并更新Global索引。

```
ALTER TABLE range_sales DROP PARTITION date_202005 UPDATE GLOBAL INDEX;
```

或者，通过指定分区值来删除范围分区表range_sales中'2020-05-08'所对应的分区。由于不带UPDATE GLOBAL INDEX子句，执行该命令后Global索引会失效。

```
ALTER TABLE range_sales DROP PARTITION FOR ('2020-05-08');
```

须知

- 当分区表只有一个分区时，不支持通过ALTER TABLE DROP PARTITION命令删除分区。
- 当分区表为哈希分区表时，不支持通过ALTER TABLE DROP PARTITION命令删除分区。

2.8.5.2.2 对二级分区表删除一级分区

使用ALTER TABLE DROP PARTITION可以删除二级分区表的一个一级分区，这个行为可以作用在一级分区策略为RANGE或者LIST的情况。数据库会将这个一级分区，以及一级分区下的所有二级分区都删除。

例如，通过指定分区名删除二级分区表range_list_sales的一级分区date_202005，并更新Global索引。

```
ALTER TABLE range_list_sales DROP PARTITION date_202005 UPDATE GLOBAL INDEX;
```

或者，通过指定分区值来删除二级分区表range_list_sales中('2020-05-08')所对应的一级分区。由于不带UPDATE GLOBAL INDEX子句，执行该命令后Global索引会失效。

```
ALTER TABLE range_list_sales DROP PARTITION FOR ('2020-05-08');
```

须知

- 当二级分区表只有一个一级分区时，不支持通过ALTER TABLE DROP PARTITION命令删除一级分区。
- 当二级分区表的一级分区策略为HASH时，不支持通过ALTER TABLE DROP PARTITION命令删除一级分区。

版权所有 © 华为云计算技术有限公司

2.8.5.2.3 对二级分区表删除二级分区

使用ALTER TABLE DROP SUBPARTITION可以删除二级分区表的一个二级分区，这个行为可以作用在二级分区策略为RANGE或者LIST的情况。

例如，通过指定分区名删除二级分区表range_list_sales的二级分区date_202005_channel1，并更新Global索引。

```
ALTER TABLE range_list_sales DROP SUBPARTITION date_202005_channel1 UPDATE GLOBAL INDEX;
```

或者，通过指定分区值来删除二级分区表range_list_sales中('2020-05-08', '0')所对应的二级分区。由于不带UPDATE GLOBAL INDEX子句，执行该命令后Global索引会失效。

```
ALTER TABLE range_list_sales DROP SUBPARTITION FOR ('2020-05-08', '0');
```

须知

- 当二级分区表所删除的目标分区只有一个二级分区时，不支持通过ALTER TABLE DROP SUBPARTITION命令删除二级分区。
- 当二级分区表的二级分区策略为HASH时，不支持通过ALTER TABLE DROP SUBPARTITION命令删除二级分区。

2.8.5.3 交换分区

用户可以使用交换分区的命令来将分区与普通表的数据进行交换。交换分区可以快速将数据导入/导出分区表，实现数据高效加载的目的。在业务迁移的场景，使用交换分区比常规导入会快很多。交换分区可以通过指定分区名或者分区值来进行。

须知

- 执行交换分区命令会使得Global索引失效，可以通过UPDATE GLOBAL INDEX子句来同步更新Global索引，或者用户自行重建Global索引。
- 执行交换分区时，可以申明WITH/WITHOUT VALIDATION，表明是否校验普通表数据满足目标分区的分区键约束规则（默认校验）。数据校验活动开销较大，如果能确保交换的数据属于目标分区，可以申明WITHOUT VALIDATION来提高交换性能。
- 可以申明WITH VALIDATION VERBOSE，此时数据库会校验普通表的每一行，将不满足目标分区的分区键约束规则的数据，插入到分区表的其他分区中，最后再进行普通表与目标分区的交换。

例如，给出如下分区定义和普通表exchange_sales的数据分布，并将分区DATE_202001和普通表exchange_sales做交换，则根据申明子句的不同，存在以下三种行为：

- 申明WITHOUT VALIDATION，数据全部交换到分区DATE_202001中，由于'2020-02-03', '2020-04-08'不满足分区DATE_202001的范围约束，后续业务可能会出现异常。
- 申明WITH VALIDATION，由于'2020-02-03', '2020-04-08'不满足分区DATE_202001的范围约束，数据库给出相应的报错。

版权所有 © 华为云计算技术有限公司

- 申明WITH VALIDATION VERBOSE，数据库会将'2020-02-03'插入分区DATE_202002，将'2020-04-08'插入分区DATE_202004，再将剩下的数据交换到分区DATE_202001中。

```
--分区定义
PARTITION DATE_202001 VALUES LESS THAN ('2020-02-01'),
PARTITION DATE_202002 VALUES LESS THAN ('2020-03-01'),
PARTITION DATE_202003 VALUES LESS THAN ('2020-04-01'),
PARTITION DATE_202004 VALUES LESS THAN ('2020-05-01')
-- exchange_sales的数据分布
('2020-01-15', '2020-01-17', '2020-01-23', '2020-02-03', '2020-04-08')
```

警告

如果交换的数据不完全属于目标分区，请不要申明WITHOUT VALIDATION交换分区，否则会破坏分区约束规则，导致分区表后续DML业务结果异常。

进行交换的普通表和分区必须满足如下条件：

- 普通表和分区的列数目相同，对应列的信息严格一致。
- 普通表和分区的表压缩信息严格一致。
- 普通表索引和分区Local索引个数相同，且对应索引的信息严格一致。
- 普通表和分区的表约束个数相同，且对应表约束的信息严格一致。
- 普通表不可以是临时表。
- 普通表和分区表上不可以有动态数据脱敏，行访问控制约束。

2.8.5.3.1 对一级分区表交换分区

使用ALTER TABLE EXCHANGE PARTITION可以对一级分区表交换分区。

例如，通过指定分区名将范围分区表range_sales的分区date_202001和普通表exchange_sales进行交换，不进行分区键校验，并更新Global索引。

```
ALTER TABLE range_sales EXCHANGE PARTITION (date_202001) WITH TABLE exchange_sales WITHOUT VALIDATION UPDATE GLOBAL INDEX;
```

或者，通过指定分区值将范围分区表range_sales中'2020-01-08'所对应的分区和普通表exchange_sales进行交换，进行分区校验并将不满足目标分区约束的数据插入到分区表的其他分区中。由于不带UPDATE GLOBAL INDEX子句，执行该命令后Global索引会失效。

```
ALTER TABLE range_sales EXCHANGE PARTITION FOR ('2020-01-08') WITH TABLE exchange_sales WITH VALIDATION VERBOSE;
```

2.8.5.3.2 对二级分区表交换二级分区

使用ALTER TABLE EXCHANGE SUBPARTITION可以对二级分区表交换二级分区。

例如，通过指定分区名将二级分区表range_list_sales的二级分区date_202001_channel1和普通表exchange_sales进行交换，不进行分区键校验，并更新Global索引。

```
ALTER TABLE range_list_sales EXCHANGE SUBPARTITION (date_202001_channel1) WITH TABLE exchange_sales WITHOUT VALIDATION UPDATE GLOBAL INDEX;
```

或者，通过指定分区值将二级分区表range_list_sales中('2020-01-08', '0')所对应的二级分区和普通表exchange_sales进行交换，进行分区校验并将不满足目标分区约束的数据插入到分区表的其他分区中。由于不带UPDATE GLOBAL INDEX子句，执行该命令后Global索引会失效。

版权所有 © 华为云计算技术有限公司

```
ALTER TABLE range_list_sales EXCHANGE SUBPARTITION FOR ('2020-01-08', '0') WITH TABLE
exchange_sales WITH VALIDATION VERBOSE;
```

须知

不支持对二级分区表的一级分区交换分区。

2.8.5.4 清空分区

用户可以使用清空分区的命令来快速清空分区的数据。与删除分区功能类似，区别在于清空分区只会删除分区中的数据，分区的定义和物理文件都会保留。清空分区可以通过指定分区名或者分区值来进行。

⚠ 注意

执行清空分区命令会使得Global索引失效，可以通过UPDATE GLOBAL INDEX子句来同步更新Global索引，或者用户自行重建Global索引。

2.8.5.4.1 对一级分区表清空分区

使用ALTER TABLE TRUNCATE PARTITION可以清空指定分区表的任何一个分区。

例如，通过指定分区名清空范围分区表range_sales的分区date_202005，并更新Global索引。

```
ALTER TABLE range_sales TRUNCATE PARTITION date_202005 UPDATE GLOBAL INDEX;
```

或者，通过指定分区值来清空范围分区表range_sales中'2020-05-08'所对应的分区。由于不带UPDATE GLOBAL INDEX子句，执行该命令后Global索引会失效。

```
ALTER TABLE range_sales TRUNCATE PARTITION FOR ('2020-05-08');
```

2.8.5.4.2 对二级分区表清空一级分区

使用ALTER TABLE TRUNCATE PARTITION可以清空二级分区表的一个一级分区，数据库会将这个一级分区下的所有二级分区都进行清空。

例如，通过指定分区名清空二级分区表range_list_sales的一级分区date_202005，并更新Global索引。

```
ALTER TABLE range_list_sales TRUNCATE PARTITION date_202005 UPDATE GLOBAL INDEX;
```

或者，通过指定分区值来清空二级分区表range_list_sales中('2020-05-08')所对应的一级分区。由于不带UPDATE GLOBAL INDEX子句，执行该命令后Global索引会失效。

```
ALTER TABLE range_list_sales TRUNCATE PARTITION FOR ('2020-05-08');
```

2.8.5.4.3 对二级分区表清空二级分区

使用ALTER TABLE TRUNCATE SUBPARTITION可以清空二级分区表的一个二级分区。

例如，通过指定分区名清空二级分区表range_list_sales的二级分区date_202005_channel1，并更新Global索引。

```
ALTER TABLE range_list_sales TRUNCATE SUBPARTITION date_202005_channel1 UPDATE GLOBAL INDEX;
```

或者，通过指定分区值来清空二级分区表range_list_sales中('2020-05-08', '0')所对应的二级分区。由于不带UPDATE GLOBAL INDEX子句，执行该命令后Global索引会失效。

版权所有 © 华为云计算技术有限公司

```
ALTER TABLE range_list_sales TRUNCATE SUBPARTITION FOR ('2020-05-08', '0');
```

2.8.5.5 分割分区

用户可以使用分割分区的命令来将一个分区分割为两个或多个新分区。当分区数据太大，或者需要对有MAXVALUE的范围分区/DEFAULT的列表分区新增分区时，可以考虑执行该操作。分割分区可以指定分割点将一个分区分割为两个新分区，也可以不指定分割点将一个分区分割为多个新分区。分割分区可以通过指定分区名或者分区值来进行。

⚠ 注意

- 分割分区不能作用于哈希分区上。
- 不支持对二级分区表的一级分区进行分割。
- 执行分割分区命令会使得Global索引失效，可以通过UPDATE GLOBAL INDEX子句来同步更新Global索引，或者用户自行重建Global索引。
- 分割的目标分区如果包含分类索引时，该分区不支持分割。
- 分割后的新分区，可以与源分区名字相同，比如将分区p1分割为p1,p2。但数据库不会将分割前后相同名的分区视为同一个分区，这会影响分割期间对源分区p1查询，具体请参见**DQL/DML-DDL并发**。

2.8.5.5.1 对范围分区表分割分区

使用ALTER TABLE SPLIT PARTITION可以对范围分区表分割分区。

例如，假设范围分区表range_sales的分区date_202001定义范围为['2020-01-01', '2020-02-01')。可以指定分割点'2020-01-16'将分区date_202001分割为两个分区，并更新Global索引。

```
ALTER TABLE range_sales SPLIT PARTITION date_202001 AT ('2020-01-16') INTO
(
    PARTITION date_202001_p1, --第一个分区上界是'2020-01-16'
    PARTITION date_202001_p2  --第二个分区上界是'2020-02-01'
) UPDATE GLOBAL INDEX;
```

或者，不指定分割点，将分区date_202001分割为多个分区，并更新Global索引。

```
ALTER TABLE range_sales SPLIT PARTITION date_202001 INTO
(
    PARTITION date_202001_p1 VALUES LESS THAN ('2020-01-11'),
    PARTITION date_202001_p2 VALUES LESS THAN ('2020-01-21'),
    PARTITION date_202001_p3 --第三个分区上界是'2020-02-01'
)UPDATE GLOBAL INDEX;
```

又或者，通过指定分区值而不是指定分区名来分割分区。

```
ALTER TABLE range_sales SPLIT PARTITION FOR ('2020-01-15') AT ('2020-01-16') INTO
(
    PARTITION date_202001_p1, --第一个分区上界是'2020-01-16'
    PARTITION date_202001_p2  --第二个分区上界是'2020-02-01'
) UPDATE GLOBAL INDEX;
```

须知

若对MAXVALUE分区进行分割，前面几个分区不能申明MAXVALUE范围，最后一个分区会继承MAXVALUE分区范围。

版权所有 © 华为云计算技术有限公司

2.8.5.5.2 对间隔分区表分割分区

对间隔分区表分割分区的命令与范围分区表相同。

须知

对间隔分区表的间隔分区完成分割分区操作之后，源分区之前的间隔分区会变成范围分区。

例如，创建如下间隔分区表，并插入数据新增三个分区sys_p1、sys_p2、sys_p3。

```
CREATE TABLE interval_sales
(
    prod_id       NUMBER(6),
    cust_id       NUMBER,
    time_id       DATE,
    channel_id    CHAR(1),
    promo_id      NUMBER(6),
    quantity_sold NUMBER(3),
    amount_sold   NUMBER(10, 2)
)
PARTITION BY RANGE (TIME_ID) INTERVAL ('1 MONTH')
(
    PARTITION date_2015 VALUES LESS THAN ('2016-01-01'),
    PARTITION date_2016 VALUES LESS THAN ('2017-01-01'),
    PARTITION date_2017 VALUES LESS THAN ('2018-01-01'),
    PARTITION date_2018 VALUES LESS THAN ('2019-01-01'),
    PARTITION date_2019 VALUES LESS THAN ('2020-01-01')
);
INSERT INTO interval_sales VALUES (263722,42819872,'2020-07-09','E',432072,213,17); --新增分区sys_p1
INSERT INTO interval_sales VALUES (345724,72651233,'2021-03-05','A',352451,146,9);  --新增分区sys_p2
INSERT INTO interval_sales VALUES (153241,65143129,'2021-05-07','H',864134,89,34);  --新增分区sys_p3
```

如果对分区sys_p2进行分割，则会将分区sys_p1变为范围分区，分区范围下界值从依赖间隔分区值变成依赖前一个分区的上界值，也就是分区范围从['2020-07-01', '2020-08-01')变成['2020-01-01', '2020-08-01')；分区sys_p3依然为间隔分区，其分区范围为['2021-05-01', '2021-06-01')。

2.8.5.5.3 对列表分区表分割分区

使用ALTER TABLE SPLIT PARTITION可以对列表分区表分割分区。

例如，假设列表分区表list_sales的分区channel2定义范围为('6', '7', '8', '9')。可以指定分割点('6', '7') 将分区channel2分割为两个分区，并更新Global索引。

```
ALTER TABLE list_sales SPLIT PARTITION channel2 VALUES ('6', '7') INTO
(
    PARTITION channel2_1, --第一个分区范围是('6', '7' )
    PARTITION channel2_2  --第二个分区范围是('8', '9' )
) UPDATE GLOBAL INDEX;
```

或者，不指定分割点，将分区channel2分割为多个分区，并更新Global索引。

```
ALTER TABLE list_sales SPLIT PARTITION channel2 INTO
(
    PARTITION channel2_1 VALUES ('6'),
    PARTITION channel2_2 VALUES ('8'),
    PARTITION channel2_3 --第三个分区范围是('7', '9' )
)UPDATE GLOBAL INDEX;
```

又或者，通过指定分区值而不是指定分区名来分割分区。

```
ALTER TABLE list_sales SPLIT PARTITION FOR ('6') VALUES ('6', '7') INTO
(
    PARTITION channel2_1, --第一个分区范围是('6', '7' )
    PARTITION channel2_2  --第二个分区范围是('8', '9' )
) UPDATE GLOBAL INDEX;
```


版权所有 © 华为云计算技术有限公司

⚠ 注意

若对DEFAULT分区进行分割，前面几个分区不能申明DEFAULT范围，最后一个分区会继承DEFAULT分区范围。

2.8.5.5.4 对*-RANGE 二级分区表分割二级分区

使用ALTER TABLE SPLIT SUBPARTITION可以对*-RANGE二级分区表分割二级分区。

例如，假设*-RANGE二级分区表list_range_sales的二级分区channel1_customer4的定义范围为[1000, MAXVALUE)。可以指定分割点1200将二级分区channel1_customer4分割为两个分区，并更新Global索引。

```
ALTER TABLE list_range_sales SPLIT SUBPARTITION channel1_customer4 AT (1200) INTO
(
   SUBPARTITION channel1_customer4_p1, --第一个分区上界是1200
   SUBPARTITION channel1_customer4_p2  --第二个分区上界是MAXVALUE
) UPDATE GLOBAL INDEX;
```

或者，不指定分割点，将分区channel1_customer4分割为多个分区，并更新Global索引。

```
ALTER TABLE list_range_sales SPLIT SUBPARTITION channel1_customer4 INTO
(
   SUBPARTITION channel1_customer4_p1 VALUES LESS THAN (1200),
   SUBPARTITION channel1_customer4_p2 VALUES LESS THAN (1400),
   SUBPARTITION channel1_customer4_p3 --第三个分区上界是MAXVALUE
)UPDATE GLOBAL INDEX;
```

又或者，通过指定分区值而不是指定分区名来分割分区。

```
ALTER TABLE range_sales SPLIT SUBPARTITION FOR ('1', 1200) AT (1200) INTO
(
   PARTITION channel1_customer4_p1,
   PARTITION channel1_customer4_p2
) UPDATE GLOBAL INDEX;
```

须知

若对MAXVALUE分区进行分割，前面几个分区不能申明MAXVALUE范围，最后一个分区会继承MAXVALUE分区范围。

2.8.5.5.5 对*-LIST 二级分区表分割二级分区

使用ALTER TABLE SPLIT SUBPARTITION可以对*-LIST二级分区表分割二级分区。

例如，假设*-LIST二级分区表hash_list_sales的二级分区product2_channel2的定义范围为DEFAULT。可以指定分割点将其分割为两个分区，并更新Global索引。

```
ALTER TABLE hash_list_sales SPLIT SUBPARTITION product2_channel2 VALUES ('6', '7', '8', '9') INTO
(
   SUBPARTITION product2_channel2_p1, --第一个分区范围是('6', '7', '8', '9')
   SUBPARTITION product2_channel2_p2  --第二个分区范围是DEFAULT
) UPDATE GLOBAL INDEX;
```

或者，不指定分割点，将分区product2_channel2分割为多个分区，并更新Global索引。

```
ALTER TABLE hash_list_sales SPLIT SUBPARTITION product2_channel2 INTO
(
   SUBPARTITION product2_channel2_p1 VALUES ('6', '7', '8'),
   SUBPARTITION product2_channel2_p2 VALUES ('9', '10'),
   SUBPARTITION product2_channel2_p3 --第三个分区范围是DEFAULT
) UPDATE GLOBAL INDEX;
```


版权所有 © 华为云计算技术有限公司

又或者，通过指定分区值而不是指定分区名来分割分区。

```
ALTER TABLE hash_list_sales SPLIT SUBPARTITION FOR (1200, '6') VALUES ('6', '7', '8', '9') INTO
(
    SUBPARTITION product2_channel2_p1, --第一个分区范围是('6', '7', '8', '9')
    SUBPARTITION product2_channel2_p2  --第二个分区范围是DEFAULT
) UPDATE GLOBAL INDEX;
```

⚠ 注意

若对DEFAULT分区进行分割，前面几个分区不能申明DEFAULT范围，最后一个分区会继承DEFAULT分区范围。

2.8.5.6 合并分区

用户可以使用合并分区的命令来将多个分区合并为一个分区。合并分区只能通过指定分区名来进行，不支持指定分区值的写法。

⚠ 注意

- 合并分区不能作用于哈希分区上。
- 执行合并分区命令会使得Global索引失效，可以通过UPDATE GLOBAL INDEX子句来同步更新Global索引，或者用户自行重建Global索引。
- 合并前的分区如果包含分类索引则不支持合并。

须知

合并后的新分区，对于范围/间隔分区，可以与最后一个源分区名字相同，比如将p1,p2合并为p2；对于列表分区，可以与任一源分区名字相同，比如将p1,p2合并为p1。

如果新分区与源分区名字相同，数据库会将新分区视为对源分区的继承，这会影响合并期间对源分区查询的行为，具体请参见**DQL/DML-DDL并发**。

2.8.5.6.1 对一级分区表合并分区

使用ALTER TABLE MERGE PARTITIONS可以将多个分区合并为一个分区。

例如，将范围分区表range_sales的分区date_202001和date_202002合并为一个新的分区，并更新Global索引。

```
ALTER TABLE range_sales MERGE PARTITIONS date_202001, date_202002 INTO
    PARTITION date_2020_old UPDATE GLOBAL INDEX;
```

须知

对间隔分区表的间隔分区完成合并分区操作之后，源分区之前的间隔分区会变成范围分区。

版权所有 © 华为云计算技术有限公司

2.8.5.6.2 对二级分区表合并二级分区

使用ALTER TABLE MERGE SUBPARTITIONS可以将多个二级分区合并为一个分区。

例如，将二级分区表hash_list_sales的分区product1_channel1、product1_channel2、product1_channel3合并为一个新的分区，并更新Global索引。

```
ALTER TABLE hash_list_sales MERGE SUBPARTITIONS product1_channel1, product1_channel2,
product1_channel3 INTO
    SUBPARTITION product1_channel1 UPDATE GLOBAL INDEX;
```

2.8.5.7 移动分区

用户可以使用移动分区的命令来将一个分区移动到新的表空间中。移动分区可以通过指定分区名或者分区值来进行。

2.8.5.7.1 对一级分区表移动分区

使用ALTER TABLE MOVE PARTITION可以对一级分区表移动分区。

例如，通过指定分区名将范围分区表range_sales的分区date_202001移动到表空间tb1中。

```
ALTER TABLE range_sales MOVE PARTITION date_202001 TABLESPACE tb1;
```

或者，通过指定分区值将列表分区表list_sales中'0'所对应的分区移动到表空间tb1中。

```
ALTER TABLE list_sales MOVE PARTITION FOR ('0') TABLESPACE tb1;
```

2.8.5.7.2 对二级分区表移动二级分区

使用ALTER TABLE MOVE SUBPARTITION可以对二级分区表移动二级分区。

例如，通过指定分区名将二级分区表range_list_sales的分区date_202001_channel1移动到表空间tb1中。

```
ALTER TABLE range_list_sales MOVE SUBPARTITION date_202001_channel1 TABLESPACE tb1;
```

或者，通过指定分区值将二级分区表range_list_sales中('2020-01-08', '0')所对应的分区移动到表空间tb1中。

```
ALTER TABLE range_list_sales MOVE SUBPARTITION FOR ('2020-01-08', '0') TABLESPACE tb1;
```

2.8.5.8 重命名分区

用户可以使用重命名分区的命令来将一个分区命名为新的名称。重命名分区可以通过指定分区名或者分区值来进行。

2.8.5.8.1 对一级分区表重命名分区

使用ALTER TABLE RENAME PARTITION可以对一级分区表重命名分区。

例如，通过指定分区名将范围分区表range_sales的分区date_202001重命名。

```
ALTER TABLE range_sales RENAME PARTITION date_202001 TO date_202001_new;
```

或者，通过指定分区值将列表分区表list_sales中'0'所对应的分区重命名。

```
ALTER TABLE list_sales RENAME PARTITION FOR ('0') TO channel_new;
```

2.8.5.8.2 对二级分区表重命名一级分区

使用ALTER TABLE RENAME PARTITION可以对二级分区表重命名一级分区。具体方法与一级分区表相同。

 版权所有 © 华为云计算技术有限公司

2.8.5.8.3 对二级分区表重命名二级分区

使用ALTER TABLE RENAME SUBPARTITION可以对二级分区表重命名二级分区。

例如，通过指定分区名将二级分区表range_list_sales的分区date_202001_channel1重命名。

```
ALTER TABLE range_list_sales RENAME SUBPARTITION date_202001_channel1 TO date_202001_channelnew;
```

或者，通过指定分区值将二级分区表range_list_sales中('2020-01-08', '0')所对应的分区重命名。

```
ALTER TABLE range_list_sales RENAME SUBPARTITION FOR ('2020-01-08', '0') TO date_202001_channelnew;
```

2.8.5.8.4 对 Local 索引重命名索引分区

使用ALTER INDEX RENAME PARTITION可以对Local索引重命名索引分区。具体方法与一级分区表重命名分区相同。

2.8.5.9 分区表行迁移

用户可以使用ALTER TABLE ENABLE/DISABLE ROW MOVEMENT来开启/关闭分区表行迁移。

开启行迁移时，允许通过更新操作将一个分区中的数据迁移到另一个分区中；关闭行迁移时，如果出现这种更新行为，则业务报错。

须知

如果业务明确不允许对分区键所在列进行更新操作，建议关闭分区表行迁移。

例如，创建列表分区表，并开启分区表行迁移，此时可以跨分区更新分区键所在列；关闭分区表行迁移后，对分区键所在列进行跨分区更新会业务报错。

```
CREATE TABLE list_sales
(
    product_id      INT4 NOT NULL,
    customer_id     INT4 PRIMARY KEY,
    time_id         DATE,
    channel_id      CHAR(1),
    type_id         INT4,
    quantity_sold   NUMERIC(3),
    amount_sold     NUMERIC(10,2)
)
PARTITION BY LIST (channel_id)
(
    PARTITION channel1 VALUES ('0', '1', '2'),
    PARTITION channel2 VALUES ('3', '4', '5'),
    PARTITION channel3 VALUES ('6', '7'),
    PARTITION channel4 VALUES ('8', '9')
) ENABLE ROW MOVEMENT;
INSERT INTO list_sales VALUES (153241,65143129,'2021-05-07','0',864134,89,34);
--跨分区更新成功，数据从分区channel1迁移到分区channel2
UPDATE list_sales SET channel_id = '3' WHERE channel_id = '0';
--关闭分区表行迁移
ALTER TABLE list_sales DISABLE ROW MOVEMENT;
--跨分区更新失败，报错fail to update partitioned table "list_sales"
UPDATE list_sales SET channel_id = '0' WHERE channel_id = '3';
--分区内更新依然成功
UPDATE list_sales SET channel_id = '4' WHERE channel_id = '3';
```


版权所有 © 华为云计算技术有限公司

2.8.5.10 分区表索引重建/不可用

用户可以通过命令使得一个分区表索引或者一个索引分区不可用，此时该索引/索引分区不再维护。使用重建索引命令可以重建分区表索引，恢复索引的正常功能。

此外，部分分区级DDL操作也会使得Global索引失效，包括删除drop、交换exchange、清空truncate、分割split、合并merge。如果在DDL操作中带UPDATE GLOBAL INDEX子句，则会同步更新Global索引，否则需要用户自行重建索引。

2.8.5.10.1 索引重建/不可用

使用ALTER INDEX可以设置索引是否可用。

例如，假设分区表range_ sales上存在索引range_sales_idx，可以通过如下命令设置其不可用。

```
ALTER INDEX range_sales_idx UNUSABLE;
```

可以通过如下命令重建索引range_sales_idx。

```
ALTER INDEX range_sales_idx REBUILD;
```

2.8.5.10.2 Local 索引分区重建/不可用

- 使用ALTER INDEX PARTITION可以设置Local索引分区是否可用。
- 使用ALTER TABLE MODIFY PARTITION可以设置分区表上指定分区的所有索引分区是否可用。这个语法如果作用于二级分区表的一级分区，数据库会将这个一级分区下的所有二级分区均进行设置。
- 使用ALTER TABLE MODIFY SUBPARTITION可以设置二级分区表上指定二级分区的所有索引分区是否可用。

例如，假设分区表range_sales上存在两张Local索引range_sales_idx1和range_sales_idx2，假设其在分区date_202001上对应的索引分区名分别为range_sales_idx1_part1和range_sales_idx2_part1。

下面给出了维护分区表分区索引的语法：

- 可以通过如下命令设置分区date_202001上的所有索引分区均不可用。
  ```
  ALTER TABLE range_sales MODIFY PARTITION date_202001 UNUSABLE LOCAL INDEXES;
  ```
- 或者通过如下命令单独设置分区date_202001上的索引分区range_sales_idx1_part1不可用。
  ```
  ALTER INDEX range_sales_idx1 MODIFY PARTITION range_sales_idx1_part1 UNUSABLE;
  ```
- 可以通过如下命令重建分区date_202001上的所有索引分区。
  ```
  ALTER TABLE range_sales MODIFY PARTITION date_202001 REBUILD UNUSABLE LOCAL INDEXES;
  ```
- 或者通过如下命令单独重建分区date_202001上的索引分区range_sales_idx1_part1。
  ```
  ALTER INDEX range_sales_idx1 REBUILD PARTITION range_sales_idx1_part1;
  ```

假设二级分区表list_range_sales上存在两张Local索引list_range_sales_idx1和list_range_sales_idx2，表下有一级分区channel1，其下属二级分区有channel1_product1、channel1_product2、channel1_product3，二级分区channel1_product1上对应的索引分区名分别为channel1_product1_idx1和channel1_product1_idx2。

下面给出了维护二级分区表一级分区索引的语法：

- 可以通过如下命令设置分区channel1下属二级分区的所有索引分区均不可用，包括二级分区channel1_product1、channel1_product2、channel1_product3。
  ```
  ALTER TABLE list_range_sales MODIFY PARTITION channel1 UNUSABLE LOCAL INDEXES;
  ```

 版权所有 © 华为云计算技术有限公司

- 可以通过如下命令重建分区channel1下属二级分区的所有索引分区。
  ```
  ALTER TABLE list_range_sales MODIFY PARTITION channel1 REBUILD UNUSABLE LOCAL INDEXES;
  ```

下面给出了维护二级分区表二级分区索引的语法：

- 可以通过如下命令单独设置二级分区channel1_product1上的所有索引分区均不可用。
  ```
  ALTER TABLE list_range_sales MODIFY SUBPARTITION channel1_product1 UNUSABLE LOCAL INDEXES;
  ```
- 可以通过如下命令重建二级分区channel1_product1上的所有索引分区。
  ```
  ALTER TABLE list_range_sales MODIFY SUBPARTITION channel1_product1 REBUILD UNUSABLE LOCAL INDEXES;
  ```
- 或者通过如下命令单独设置二级分区channel1_product1上的索引分区channel1_product1_idx1不可用。
  ```
  ALTER INDEX list_range_sales_idx1 MODIFY PARTITION channel1_product1_idx1 UNUSABLE;
  ```
- 通过如下命令单独重建二级分区channel1_product1上的索引分区channel1_product1_idx1。
  ```
  ALTER INDEX list_range_sales_idx1 REBUILD PARTITION channel1_product1_idx1;
  ```

2.8.6 分区并发控制

分区并发控制给出了分区表DQL、DML、DDL并发过程中的行为规格限制。用户在设计分区表并发业务时，尤其是在进行分区维护操作时，可以参考本章节指导。

2.8.6.1 常规锁设计

分区表通过表锁+分区锁两重设计，在表和分区上分别施加8个不同级别的常规锁，来保证DQL、DML、DDL并发过程中的合理行为控制。下表给出了不同级别锁的互斥行为，标记为√的两种常规锁互不阻塞，可以并行。

表 2-7 常规锁行为

-	ACCESS_SHARE	ROW_SHARE	ROW_EXCLUSIVE	SHARE_UPDATE_EXCLUSIVE	SHARE	SHARE_ROW_EXCLUSIVE	EXCLUSIVE	ACCESS_EXCLUSIVE
ACCESS_SHARE	√	√	√	√	√	√	√	×
ROW_SHARE	√	√	√	√	√	√	×	×
ROW_EXCLUSIVE	√	√	√	√	×	×	×	×
SHARE_UPDATE_EXCLUSIVE	√	√	√	×	×	×	×	×
SHARE	√	√	×	×	√	×	×	×
SHARE_ROW_EXCLUSIVE	√	√	×	×	×	×	×	×

版权所有 © 华为云计算技术有限公司

-	ACCESS_SHARE	ROW_SHARE	ROW_EXCLUSIVE	SHARE_UPDATE_EXCLUSIVE	SHARE	SHARE_ROW_EXCLUSIVE	EXCLUSIVE	ACCESS_EXCLUSIVE
EXCLUSIVE	√	×	×	×	×	×	×	×
ACCESS_EXCLUSIVE	×	×	×	×	×	×	×	×

分区表的不同业务最终都是作用于目标分区上，数据库会给分区表和目标分区施加不同级别的表锁+分区锁，来控制并发行为。下表给出了不同业务的锁粒度控制。其中数字1~8代表上表给出的8种级别常规锁。

表 2-8 分区表业务锁粒度

业务模型	一级分区表锁级别(表锁+分区锁)	二级分区表锁级别(表锁+一级分区锁+二级分区锁)
SELECT	1-1	1-1-1
SELECT FOR UPDATE	2-2	2-2-2
DML业务，包括INSERT、UPDATE、DELETE、UPSERT、MERGE INTO、COPY	3-3	3-3-3
分区DDL，包括ADD、DROP、EXCHANGE、TRUNCATE、SPLIT、MERGE、MOVE、RENAME；打开/关闭分区自动扩展	4-8	4-8-8（作用二级分区表的一级分区） 4-4-8（作用二级分区表的二级分区）
CREATE INDEX（非分类索引）、REBUILD INDEX	5-5	5-5-5
CREATE INDEX（分类索引）	3-5	3-3-5
REBUILD INDEX PARTITION	1-5	1-5-5
ANALYZE、VACUUM	4-4	4-4-4
其他分区表DDL	8-8	8-8-8

版权所有 © 华为云计算技术有限公司

2.8.6.2 DQL/DML-DQL/DML 并发

DQL/DML操作会给表和分区施加1~3级别的常规锁，DQL/DML操作自身互不阻塞，支持DQL/DML-DQL/DML并发。

说明

支持自动扩展的分区表由于INSERT、UPDATE、UPSERT、MERGE INTO、COPY等业务导致的新增分区行为视为一个分区DDL操作。

2.8.6.3 DQL/DML-DDL 并发

表级DDL会给分区表施加8级锁，阻塞全部的DQL/DML操作。

分区级DDL会给分区表施加4级锁，并给目标分区施加8级锁。当DQL/DML与DDL作用不同分区时，支持二者执行层面的并发；当DQL/DML与DDL作用相同分区时，后触发业务会被阻塞。

须知

- 业务在进行自动扩展分区时，应避免同时进行分区DDL操作，避免触发死锁问题。
- 业务在进行分区DDL维护操作时，应尽可能避免期间同时对目标分区进行DQL/DML操作。
- 如果并发的DDL与DQL/DML作用目标分区有重叠，由于串行阻塞，DQL/DML既可能先于DDL发生，也可能后于DDL发生，用户应该明确知晓其可能的预期结果。比如当Truncate与Insert作用同一分区时，如果Truncate先于Insert触发，则业务完成后目标分区存在数据，如果Truncate后于Insert触发，则业务完成后目标分区不存在数据。

DQL/DML-DDL 跨分区并发

GaussDB支持跨分区的DQL/DML-DDL并发。

例如，定义如下分区表range_sales，下面给出了一些支持并发的例子。

```
CREATE TABLE range_sales
(
    product_id      INT4 NOT NULL,
    customer_id     INT4 NOT NULL,
    time_id         DATE,
    channel_id      CHAR(1),
    type_id         INT4,
    quantity_sold   NUMERIC(3),
    amount_sold     NUMERIC(10,2)
)
PARTITION BY RANGE (time_id)
(
    PARTITION time_2008 VALUES LESS THAN ('2009-01-01'),
    PARTITION time_2009 VALUES LESS THAN ('2010-01-01'),
    PARTITION time_2010 VALUES LESS THAN ('2011-01-01'),
    PARTITION time_2011 VALUES LESS THAN ('2012-01-01')
);

CREATE TABLE temp
(
    product_id      INT4 NOT NULL,
    customer_id     INT4 NOT NULL,
    time_id         DATE,
    channel_id      CHAR(1),
```


版权所有 © 华为云计算技术有限公司

```
    type_id         INT4,
    quantity_sold   NUMERIC(3),
    amount_sold     NUMERIC(10,2)
);
```

分区表支持的并发业务可以为如下场景。

```
--并发case1，插入分区time_2011与清空分区time_2008互不阻塞
\parallel on
INSERT INTO range_sales VALUES (455124, 92121433, '2011-09-17', 'X', 4513, 7, 17);
ALTER TABLE range_sales TRUNCATE PARTITION time_2008 UPDATE GLOBAL INDEX;
\parallel off

--并发case2，指定分区time_2010查询与交换分区time_2009互不阻塞
\parallel on
SELECT COUNT(*) FROM range_sales PARTITION (time_2010);
ALTER TABLE range_sales EXCHANGE PARTITION (time_2009) WITH TABLE temp UPDATE GLOBAL INDEX;
\parallel off

--并发case3，对分区表range_ sales做更新与删除分区time_2008互不阻塞，这是因为更新SQL带条件剪枝到分区time_2010和time_2011上
\parallel on
UPDATE range_sales SET channel_id = 'T' WHERE channel_id = 'X' AND time_id > '2010-06-01';
ALTER TABLE range_sales DROP PARTITION time_2008 UPDATE GLOBAL INDEX;
\parallel off

--并发case4，对分区表range_ sales的任何DQL/DML操作与新增分区time_2012互不阻塞，这是因为新增分区对其他进行的业务不可见
\parallel on
DELETE FROM range_sales WHERE channel_id = 'T';
ALTER TABLE range_sales ADD PARTITION time_2012 VALUES LESS THAN ('2013-01-01');
\parallel off
```

DQL/DML-DDL 同分区并发

GaussDB不支持同分区的DQL/DML-DDL并发，后触发业务会被先触发业务阻塞。

原则上，不建议用户在进行分区DDL时，同时对该分区进行DQL/DML操作，因为目标分区存在一个状态的突变过程，可能会导致业务的查询结果不符合预期。

如果由于业务模型不合理、无法剪枝等场景导致的DQL/DML和DDL作用分区有重叠时，考虑两种场景：

场景一：先触发DQL/DML，再触发DDL。DDL会被阻塞，等DQL/DML提交后再进行。

场景二：先触发DDL，再触发DQL/DML。DQL/DML会被阻塞，等DDL提交后再进行，由于分区元信息发生了变更，可能导致预期不合理。为了保证数据一致性，预期结果按照如下规则制定。

⚠ 注意

在DQL/DML业务期间，如果对执行DQL/DML操作的分区，同时做多次分区DDL操作，有低概率出现报错，报错原因：分区找不到，分区已经被DDL删除。

- **ADD分区**

  ADD分区会产生一个新的分区，这个新分区对期间触发的DQL/DML操作均是不可见的，无阻塞期。

- **DROP分区**

  DROP分区会将已有分区进行删除，期间触发的目标分区DQL/DML操作会被阻塞，阻塞完成后跳过对该分区的处理。

版权所有 © 华为云计算技术有限公司

- **TRUNCATE分区**

  TRUNCATE分区会将已有分区清空数据，期间触发的目标分区DQL/DML操作会被阻塞，阻塞完成后继续对该分区进行处理。

  注意期间触发的目标分区查询是查不到数据的，因为TRUNCATE操作提交后目标分区中不存有任何数据。

- **EXCHANGE分区**

  EXCHANGE分区会将一个已有分区与普通表进行交换，期间触发的目标分区DQL/DML操作会被阻塞，阻塞完成后继续对该分区进行处理，该分区的实际数据对应原普通表。

  例外：如果分区表上存在GLOBAL索引，EXCHANGE命令带来UPDATE GLOBAL INDEX子句，且期间触发的分区表查询使用了GLOBAL索引，由于无法查询到交换后分区上的数据，在阻塞完成后查询业务会报错。

  ERROR: partition xxxxxx does not exist on relation "xxxxxx"

  DETAIL: this partition may have already been dropped by cocurrent DDL operations EXCHANGE PARTITION

- **SPLIT分区**

  SPLIT分区会将一个分区分割为多个分区，即使其中一个新分区与旧分区名字相同，也视为不同的分区。期间触发的目标分区DQL/DML操作会被阻塞，阻塞完成后业务报错。

  ERROR: partition xxxxxx does not exist on relation "xxxxxx"

  DETAIL: this partition may have already been dropped by cocurrent DDL operations SPLIT PARTITION

- **MERGE分区**

  MERGE分区会将多个分区合并为一个分区，如果合并后的分区与其中一个旧分区A名字相同，逻辑上视为相同分区。期间触发的目标分区DQL/DML操作会被阻塞，阻塞完成后，根据目标分区类型判断，如果目标分区是旧分区A，则作用于新分区；如果目标分区为其他旧分区，则业务报错。

  ERROR: partition xxxxxx does not exist on relation "xxxxxx"

  DETAIL: this partition may have already been dropped by cocurrent DDL operations MERGE PARTITION

- **RENAME分区**

  RENAME分区不会变更分区结构信息，期间触发的DQL/DML操作不会出现任何异常，但会被阻塞，直到RENAME操作提交。

- **MOVE分区**

  MOVE分区不会变更分区结构信息，期间触发的DQL/DML操作不会出现任何异常，但会被阻塞，直到MOVE操作提交。

2.8.6.4 DDL-DDL 并发

GaussDB不支持DDL操作自身的并发，后触发业务会被先触发业务阻塞。

2.8.7 分区表系统视图&DFX

 版权所有 © 华为云计算技术有限公司

2.8.7.1 分区表相关系统视图

分区表系统视图根据权限分为3类，具体字段信息请参考《 开发者指南 》中“系统表和系统视图 > 系统视图”章节。

1. 所有分区视图：
   - ADM_PART_TABLES：所有分区表信息。
   - ADM_TAB_PARTITIONS：所有一级分区信息。
   - ADM_TAB_SUBPARTITIONS：所有二级分区信息。
   - ADM_PART_INDEXES：所有Local索引信息。
   - ADM_IND_PARTITIONS：所有一级分区表索引分区信息。
   - ADM_IND_SUBPARTITIONS：所有二级分区表索引分区信息。
2. 当前用户可访问的视图：
   - DB_PART_TABLES：当前用户可访问的分区表信息。
   - DB_TAB_PARTITIONS：当前用户可访问的一级分区信息。
   - DB_TAB_SUBPARTITIONS：当前用户可访问的二级分区信息。
   - DB_PART_INDEXES：当前用户可访问的Local索引信息。
   - DB_IND_PARTITIONS：当前用户可访问的一级分区表索引分区信息。
   - DB_IND_SUBPARTITIONS：当前用户可访问的二级分区表索引分区信息。
3. 当前用户拥有的视图：
   - MY_PART_TABLES：当前用户拥有的分区表信息。
   - MY_TAB_PARTITIONS：当前用户拥有的一级分区信息。
   - MY_TAB_SUBPARTITIONS：当前用户拥有的二级分区信息。
   - MY_PART_INDEXES：当前用户拥有的Local索引信息。
   - MY_IND_PARTITIONS：当前用户拥有的一级分区表索引分区信息。
   - MY_IND_SUBPARTITIONS：当前用户拥有的二级分区表索引分区信息。

2.8.7.2 分区表相关内置工具函数

前置建表相关信息

- 前置建表：

```
CREATE TABLE test_range_pt (a INT, b INT, c INT)
PARTITION BY RANGE (a)
(
    PARTITION p1 VALUES LESS THAN (2000),
    PARTITION p2 VALUES LESS THAN (3000),
    PARTITION p3 VALUES LESS THAN (4000),
    PARTITION p4 VALUES LESS THAN (5000),
    PARTITION p5 VALUES LESS THAN (MAXVALUE)
)ENABLE ROW MOVEMENT;
```

- 查看分区表OID：

```
SELECT oid FROM pg_class WHERE relname = 'test_range_pt';
  oid
-------
 49290
(1 row)
```

- 查看分区信息：

```
SELECT oid,relname,parttype,parentid,boundaries FROM pg_partition WHERE parentid = 49290;
  oid |    relname    | parttype | parentid | boundaries
-------+---------------+----------+----------+------------
```

 版权所有 © 华为云计算技术有限公司

```
 49293 | test_range_pt | r       |    49290 |
 49294 | p1            | p       |    49290 | {2000}
 49295 | p2            | p       |    49290 | {3000}
 49296 | p3            | p       |    49290 | {4000}
 49297 | p4            | p       |    49290 | {5000}
 49298 | p5            | p       |    49290 | {NULL}
(6 rows)
```

- 创建索引：

```
CREATE INDEX idx_range_a ON test_range_pt(a) LOCAL;
CREATE INDEX
--查看分区索引oid
SELECT oid FROM pg_class WHERE relname = 'idx_range_a';
  oid
-------
 90250
(1 row)
```

- 查看索引分区信息：

```
SELECT oid,relname,parttype,parentid,boundaries,indextblid FROM pg_partition WHERE parentid =
90250;
  oid  | relname  | parttype | parentid | boundaries | indextblid
-------+----------+----------+----------+------------+------------
 90255 | p5_a_idx | x        |    90250 |            |      49298
 90254 | p4_a_idx | x        |    90250 |            |      49297
 90253 | p3_a_idx | x        |    90250 |            |      49296
 90252 | p2_a_idx | x        |    90250 |            |      49295
 90251 | p1_a_idx | x        |    90250 |            |      49294
(5 rows)
```

工具函数示例

- pg_get_tabledef获取分区表的定义，入参可以为表的OID或者表名。

```
SELECT pg_get_tabledef('test_range_pt');

pg_get_tabledef
----------------------------------------------------------------------------------------------------------------------------------------------------------------
 SET search_path =
public;
                                                +
 CREATE TABLE test_range_pt
(
                                              +
    a
integer,
                                                       +
    b
integer,
                                                       +
    c
integer
                                                       +
 )
                                                          +
 WITH (orientation=row, compression=no, storage_type=USTORE,
segment=off)
           +
 PARTITION BY RANGE
(a)
                                           +

(
                                                        +
    PARTITION p1 VALUES LESS THAN (2000) TABLESPACE
pg_default,
                +
    PARTITION p2 VALUES LESS THAN (3000) TABLESPACE
pg_default,
```

 版权所有 © 华为云计算技术有限公司

```
                +
    PARTITION p3 VALUES LESS THAN (4000) TABLESPACE
pg_default,
                +
    PARTITION p4 VALUES LESS THAN (5000) TABLESPACE
pg_default,
                +
    PARTITION p5 VALUES LESS THAN (MAXVALUE) TABLESPACE
pg_default
              +
)
                                                    +
 ENABLE ROW
MOVEMENT;
                                                  +
 CREATE INDEX idx_range_a ON test_range_pt USING ubtree (a) LOCAL(PARTITION p1_a_idx,
PARTITION p2_a_idx, PARTITION p3_a_idx, PARTITION p4_a_idx, PARTITION p5_a_idx)  WITH
(storage_type=USTORE) TABLESPACE pg_default;
(1 row)
```

- pg_stat_get_partition_tuples_hot_updated返回给定分区id的分区热更新元组数的统计。

  在分区p1中插入10条数据并更新，统计分区p1的热更新元组数。

```
INSERT INTO test_range_pt VALUES(generate_series(1,10),1,1);
INSERT 0 10
SELECT pg_stat_get_partition_tuples_hot_updated(49294);
pg_stat_get_partition_tuples_hot_updated
------------------------------------------
0
(1 row)
UPDATE test_range_pt SET b = 2;
UPDATE 10
SELECT pg_stat_get_partition_tuples_hot_updated(49294);
pg_stat_get_partition_tuples_hot_updated
------------------------------------------
10
(1 row)
```

- pg_partition_size(oid,oid)指定OID代表的分区使用的磁盘空间。其中，第一个oid为表的OID，第二个oid为分区的OID。

  查看分区p1的磁盘空间。

```
SELECT pg_partition_size(49290, 49294);
pg_partition_size
-------------------
90112
(1 row)
```

- pg_partition_size(text, text)指定名称的分区使用的磁盘空间。其中，第一个text为表名，第二个text为分区名。

  查看分区p1的磁盘空间。

```
SELECT pg_partition_size('test_range_pt', 'p1');
pg_partition_size
-------------------
90112
(1 row)
```

- pg_partition_indexes_size(oid,oid)指定OID代表的分区索引使用的磁盘空间。其中，第一个oid为表的OID，第二个oid为分区的OID。

  查看分区p1的索引分区磁盘空间。

```
SELECT pg_partition_indexes_size(49290, 49294);
pg_partition_indexes_size
---------------------------
204800
(1 row)
```

 版权所有 © 华为云计算技术有限公司

- pg_partition_indexes_size(text,text)指定名称的分区索引使用的磁盘空间。其中，第一个text为表名，第二个text为分区名。

  查看分区p1的索引分区磁盘空间。

  ```
  SELECT pg_partition_indexes_size('test_range_pt', 'p1');
  pg_partition_indexes_size
  ---------------------------
  204800
  (1 row)
  ```

- pg_partition_filenode(partition_oid)获取到指定分区表的OID所对应的filenode。

  查看分区p1的filenode。

  ```
  SELECT pg_partition_filenode(49294);
  pg_partition_filenode
  -----------------------
  49294
  (1 row)
  ```

- pg_partition_filepath(partition_oid)指定分区的文件路径名。

  查看分区p1的文件路径。

  ```
  SELECT pg_partition_filepath(49294);
  pg_partition_filepath
  -----------------------
  base/16521/49294
  (1 row)
  ```

2.9 存储引擎

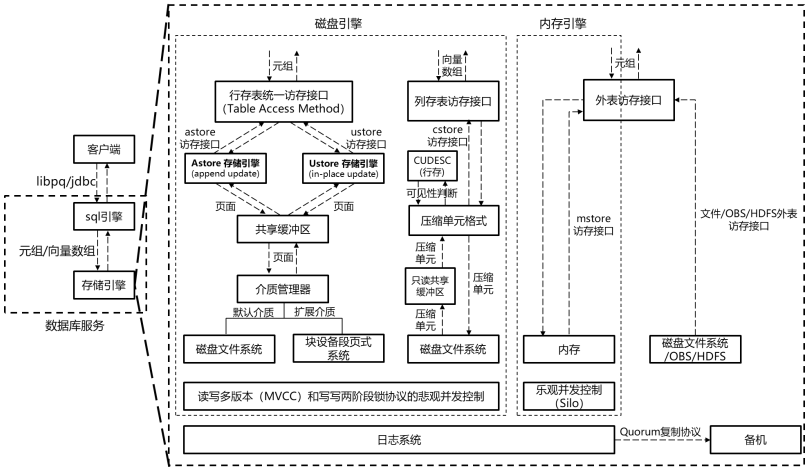
2.9.1 存储引擎体系架构

2.9.1.1 存储引擎体系架构概述

2.9.1.1.1 静态编译架构

从整个数据库服务的组成构架来看，存储引擎向上对接SQL引擎，为SQL引擎提供或接收标准化的数据格式（元组或向量数组）。存储引擎向下对接存储介质，按照特定的数据组织方式，以页面、压缩单元（Compress Unit）或其他形式为单位，通过存储介质提供的特定接口，对存储介质中的数据完成读写操作。GaussDB通过静态编译使数据库专业人员可以为特定的应用程序需求选择专用的存储引擎。为了减少对执行引擎的干扰，提供行存访问接口层TableAM，用来屏蔽底层行存引擎带来的差异，使得不同行存引擎可以分别独立演进。如下图所示。

 版权所有 © 华为云计算技术有限公司



在此基础之上，存储引擎通过日志系统提供数据的持久化和可靠性能力。通过并发控制（事务）系统保证同时执行的、多个读写操作之间的原子性、一致性和隔离性，通过索引系统提供对特定数据的加速寻址和查询能力，通过主备复制系统提供整个数据库服务的高可用能力。

行存引擎主要面向OLTP（OnLine Transaction Processing）类业务应用场景，适合高并发、小数据量的单点或小范围数据读写操作。行存引擎向上为SQL引擎提供元组形式的读写接口，向下以页面为单位通过可扩展的介质管理器对存储介质进行读写操作，并通过页面粒度的共享缓冲区来优化读写操作的效率。对于读写并发操作，采用多版本并发控制（MVCC，Multi-Version Concurrency Control）；对于写写并发操作，采用基于两阶段锁协议（2PL，Two-Phase Locking）的悲观并发控制（PCC，Pessimistic Concurrency Control）。当前，行存引擎默认的介质管理器采用磁盘文件系统接口，后续可扩展支持块设备等其他类型的存储介质。GaussDB行存引擎可以选择基于Append update 的Astore或基于In-place update的Ustore。

2.9.1.1.2 通用数据库服务层

从技术角度来看，存储引擎需要一些基础架构组件，主要包括：

并发：不同存储引擎选择正确的锁可以减少开销，从而提高整体性能。此外提供多版本并发控制或“快照”读取等功能。

事务：均需满足ACID的要求，提供事务状态查询等功能。

内存缓存：不同存储引擎在访问索引和数据时一般会对其进行缓存。缓存池允许直接从内存中处理经常使用的数据，从而加快了处理速度。

检查点：不同存储引擎一般都支持增量checkpoint/double write或全量checkpoint/full page write模式。应用可以根据不同条件进行选择增量或者全量，这个对存储引擎是透明的。

日志：GaussDB采用的是物理日志，其写入/传输/回放对存储引擎透明。

2.9.1.2 设置存储引擎

存储引擎会对数据库整体效率和性能具有巨大影响，请根据实际需求选择适当的存储引擎。用户可使用WITH ([ORIENTATION | STORAGE_TYPE] [= value] [, ...])为表或索引指定一个可选的存储参数。参数的详细描述如下所示：

ORIENTATION	STORAGE_TYPE

版权所有 © 华为云计算技术有限公司

ROW（缺省值）：表的数据将以行式存储。	[USTORE(缺省值)|ASTORE|空]

如果ORIENTATION指定为ROW，且STORAGE_TYPE为空的情况下创建出的表类型取决于GUC参数enable_default_ustore_table（取值为on/off，默认情况为on，参数详情请参见《管理员指南》中“配置运行参数 > GUC参数说明”章节）：如果参数设置为on，创建出的表为Ustore类型；如果为off，创建出的表为Astore类型。

具体示例如下：

```
gaussdb=# CREATE TABLE TEST(a int);
gaussdb=# \d+ test
                    Table "public.test"
 Column |  Type   | Modifiers | Storage | Stats target | Description
--------+---------+-----------+---------+--------------+-------------
 a      | integer |           | plain   |              |
Has OIDs: no
Options: orientation=row, compression=no, storage_type=USTORE, segment=off

gaussdb=# CREATE TABLE TEST1(a int) with(orientation=row, storage_type=ustore);
gaussdb=# \d+ test1
Table "public.test1"
 Column |  Type   | Modifiers | Storage | Stats target | Description
--------+---------+-----------+---------+--------------+-------------
 a      | integer |           | plain   |              |
Has OIDs: no
Options: orientation=row, storage_type=ustore, compression=no, segment=off

gaussdb=# CREATE TABLE TEST2(a int) with(orientation=row, storage_type=astore);
gaussdb=# \d+ test2
Table "public.test2"
 Column |  Type   | Modifiers | Storage | Stats target | Description
--------+---------+-----------+---------+--------------+-------------
 a      | integer |           | plain   |              |
Has OIDs: no
Options: orientation=row, storage_type=astore, compression=no
gaussdb=#  CREATE TABLE test4(a int) with(orientation=row);
gaussdb=# \d+
                                          List of relations
 Schema | Name  | Type |   Owner   | Size  |                        Storage                         | Description
--------+-------+-------+-----------+---------+--------------------------------------------------------------------------------
+-------------
 public | test  | table | z7ee88f3a | 0 bytes |
{orientation=row,compression=no,storage_type=USTORE,segment=off} |
 public | test1 | table | z7ee88f3a | 0 bytes |
{orientation=row,storage_type=ustore,compression=no,segment=off} |
 public | test2 | table | z7ee88f3a | 0 bytes | {orientation=row,storage_type=astore,compression=no}            |
 public | test3 | table | z7ee88f3a | 16 kB   |
{orientation=column,storage_type=astore,compression=low}        |
 public | test4 | table | z7ee88f3a | 0 bytes |
{orientation=row,compression=no,storage_type=USTORE,segment=off} |
(5 rows)

gaussdb=# show enable_default_ustore_table;
 enable_default_ustore_table
-----------------------------
 on
(1 row)

gaussdb=#  DROP TABLE test;
gaussdb=#  DROP TABLE test1;
gaussdb=#  DROP TABLE test2;
gaussdb=#  DROP TABLE test4;
```

 版权所有 © 华为云计算技术有限公司

2.9.1.3 存储引擎更新说明

2.9.1.3.1 GaussDB 内核 505 版本

- Ustore支持柔性字段高效存储。
- Ustore支持Toast规模商用。
- Ustore增加页面恢复与逃生技术。
- Ustore支持SMP技术。

2.9.1.3.2 GaussDB 内核 503 版本

- Ustore适配分布式/并行查询/Global Temp Table/Vacuum full/列约束DEFERRABLE以及INITIALLY DEFERRED。
- Ustore增加在线重建索引。
- Ustore增加增强版本B-tree空页面估算，提升优化器代价估算准确度。
- Ustore增加存储引擎可靠性验证框架，Dignose Page/Page Verify。
- Ustore增强存储引擎相关的解析/检测/修复视图。
- Ustore增强基于WAL日志的定位能力，新增gs_redo_upage系统视图，支持对单页面的不断重放，获取并打印该页面的任何一个历史版本，加速页面损坏类问题的定位。
- Ustore扩展事务槽TD物理格式，为事务内空间复用做好铺垫。
- Ustore增加在线创建索引。
- Ustore适配闪回功能（ for Ustore ）/极致RTO。

2.9.1.3.3 GaussDB 内核 R2 版本

- Ustore增加新的基于原位更新的行存储引擎Ustore，首次实现新、旧版本的记录的分离存储。
- Ustore增加回滚段模块。
- Ustore增加回滚过程，支持同步/异步/页内模式。
- Ustore增加支持事务的增强版本B-tree。
- Astore增加闪回功能，支持闪回表/闪回查询/闪回Drop/闪回Truncate。
- Ustore不支持的特性包括分布式并行查询/Table Sampling/Global Temp Table/在线创建/重建索引/极致RTO/Vacuum Full/列约束DEFERRABLE以及INITIALLY DEFERRED。

2.9.2 Astore 存储引擎

2.9.2.1 Astore 简介

Astore与Ustore的多版本实现最大的区别在于最新版本和历史版本是否分离存储。Astore不进行分离存储，而Ustore当前也只是分离了数据，索引本身没有分开。

 版权所有 © 华为云计算技术有限公司

使用 Astore 的优势

1. Astore没有回滚段，而Ustore有回滚段。对于Ustore来说，回滚段是非常重要的，回滚段损坏会导致数据丢失甚至数据库无法启动的严重问题。且Ustore恢复时同步需要Redo和Undo。由于Astore没有回滚段，旧数据都是记录在原先的文件中，所以当数据库异常crash后，恢复时，不会像Ustore数据库那样进行复杂的恢复。
2. 由于旧的数据是直接记录在数据文件中，而不是回滚段中，所以不会经常报Snapshot Too Old错误。
3. 回滚可以很快完成，因为回滚并不删除数据。

> **⚠ 注意**
>
> 回滚时很复杂，在事务回滚时必须清理该事务所进行的修改，插入的记录要删除，更新的记录要更新回来，同时回滚的过程也会再次产生大量的Redo日志。

4. WAL日志要简单一些，仅需要记录数据文件的变化，不需要记录回滚段的变化。
5. 支持回收站（闪回DROP、闪回Truncate）功能。

2.9.3 Ustore 存储引擎

2.9.3.1 Ustore 简介

Ustore（Unified Storage）是GaussDB推出的一款原位更新的存储引擎，其多版本的实现较Astore最大的区别在于最新版本和历史版本的数据是分离存储的，而索引当前还没有分离。Ustore目前已发展为GaussDB的默认行存引擎。

使用 Ustore 的优势

- 最新版本和历史版本分离存储，相比Astore扫描范围小。去除Astore的HOT chain，非索引列/索引列更新，Heap均可原位更新，ROWID可保持不变。历史版本可批量回收，空间膨胀可控。
- B-tree索引增加了事务信息，能够独立进行MVCC。增加了IndexOnlyScan的比例，大大减少回表次数。
- 不依赖Vacuum进行旧版本清理。独立的空间回收能力，索引与堆表解耦，可独立清理，IO平稳度更优。
- 大并发更新同一行的场景，相对于Astore的ROWID会偏移，Ustore的原位更新机制保证了元组ROWID稳定，先到先得，更新时延相对稳定。
- 支持闪回功能。

> **⚠ 注意**
>
> Ustore DML在修改数据页面时，也需要同步生成Undo，因此更新操作开销会稍大一些。此外单条Tuple扫描开销由于需要复制（Astore返回指针）也会大一些。

2.9.3.1.1 Ustore 特性与规格

版权所有 © 华为云计算技术有限公司

特性约束

类别	特性	是否支持
事务	Serializable	×
	在事务块中对分区表执行DDL操作	×
可扩展性	Hashbucket	×
SQL	Table sampling/物化视图/键值锁	×

存储规格

1. 数据表最大列数不能超过1600列。
2. init_td（TD（Transaction Directory，事务目录）是Ustore表独有的用于存储页面事务信息的结构，TD的数量决定该页面支持的最大并发数。在创建表或索引时可以指定初始的TD大小init_td）取值范围[2, 128]，默认值4。单页面支持的最大并发不超过128个。
3. Ustore表（不含toast情况）最大Tuple长度不能超过（8192 - MAXALIGN(56 + init_td * 26 + 4)）, 其中MAXALIGN表示8字节对齐。当插入数据长度超过阈值时，用户会收到元组长度过长无法插入的报错。其中init_td对于Tuple长度的影响如下：
   - 表init_td数量为最小值2时，Tuple长度不能超过8192 - MAXALIGN(56+2*26+4) = 8080B。
   - 表init_td数量为默认值4时，Tuple长度不能超过8192 - MAXALIGN(56+4*26+4) = 8024B。
   - 表init_td数量为最大值128时，Tuple长度不能超过8192 - MAXALIGN(56+128*26+4) = 4800B。
4. 索引最大列数不能超过32列。全局分区索引最大列数不能超过31列。
5. 索引元组长度不能超过(8192 - MAXALIGN(28 + 3 * 4 + 3 * 10) - MAXALIGN(42))/3, 其中MAXALIGN表示8字节对齐。当插入数据长度超过阈值时，用户会收到索引元组长度过长无法插入的报错，其中索引页头为28B，行指针为4B，元组CTID+INFO标记位为10B，页尾为42B。
6. 回滚段容量最大支持16TB。

2.9.3.1.2 使用 Ustore 进行测试

创建Ustore表

使用CREATE TABLE语句创建Ustore表。

```
gaussdb=# CREATE TABLE ustore_table(a INT PRIMARY KEY, b CHAR (20)) WITH (STORAGE_TYPE=USTORE);
NOTICE:  CREATE TABLE / PRIMARY KEY will create implicit index "ustore_table_pkey" for table
"ustore_table"
CREATE TABLE
gaussdb=# \d+ ustore_table
Table "public.ustore_table"
Column |     Type      | Modifiers | Storage  | Stats target | Description
--------+---------------+-----------+----------+--------------+-------------
a      | integer       | not null  | plain    |              |
b      | character(20) |           | extended |              |
Indexes:
"ustore_table_pkey" PRIMARY KEY, ubtree (a) WITH (storage_type=USTORE) TABLESPACE pg_default
```

 版权所有 © 华为云计算技术有限公司

```
Has OIDs: no
Options: orientation=row, storage_type=ustore, compression=no, segment=off
```

删除Ustore表

```
gaussdb=# DROP TABLE ustore_table;
 DROP TABLE
```

为Ustore表创建索引

Ustore当前仅支持BTree类型的多版本索引。在一些场景中，为了区别于Astore的BTree索引，也会将Ustore表的多版本BTee索引称为UBTree（Ustore BTree，UBTree介绍详见**UBTree**章节）。用户可以参照以下方式使用CREATE INDEX语句为Ustore表的“a”属性创建一个UBTree索引。

Ustore表不指定创建索引类型，默认创建的是UBTree索引。

⚠ 注意

UBTree索引分为RCR版本和PCR版本，默认创建RCR版本的UBTree。若在创建索引时，with选项指定(index_txntype=pcr)或者指定GUC的index_txntype=pcr，则创建的是PCR版本的UBTree。

```
gaussdb=# CREATE TABLE test(a int);
CREATE TABLE
gaussdb=# CREATE INDEX UB_tree_index ON test(a);
CREATE INDEX
gaussdb=# \d+ test
Table "public.test"
 Column |  Type   | Modifiers | Storage | Stats target | Description
--------+---------+-----------+---------+--------------+-------------
 a      | integer |           | plain   |              |
Indexes:
    "ub_tree_index" ubtree (a) WITH (storage_type=USTORE) TABLESPACE pg_default
Has OIDs: no
Options: orientation=row, compression=no, storage_type=USTORE, segment=off

--删除Ustore表索引。
gaussdb=# DROP TABLE test;
DROP TABLE
```

2.9.3.1.3 Ustore 的最佳实践

怎么配置 init_td 大小

TD（Transaction Directory，事务目录）是Ustore表独有的用于存储页面事务信息的结构，TD的数量决定该页面支持的最大并发数。在创建表或索引时可以指定初始的TD大小init_td，默认值为4，即同时支持4个并发事务修改该页面，最大值为128。

用户需要结合业务并发度分析是否需要手动配置init_td。另外也可以结合业务运行过程中“wait available td”等待事件出现的频率来分析是否需要调整，一般“wait available td”等于0，如果“wait available td”一直不为0，就存在等待TD的事件，此时建议增大init_td再进行观察，反复几次，如果大于0的情况属于偶发，不建议调整，多余的TD槽位会占用更多的空间。推荐的增大的方法可以按照倍数进行测试，建议可从小到大尝试8、16、32、48、...、128，并观测对应的等待事件是否有明显减少，尽量取等待事件较少中init_td数量最小的值作为默认值以节省空间。wait available td是wait_status的值之一，wait_status表示当前线程的等待状态，包含等待状态详细信息。通过PG_THREAD_WAIT_STATUS视图可以查询wait_status的值（none表示没在等待任意事件，如果有等待事件即可看到对应wait available td的

 版权所有 © 华为云计算技术有限公司

值）示例如下。init_td的配置和详细描述参见《开发者指南》的“SQL参考 > SQL语法 > CREATE TABLE”章节。init_td查看和修改方法具体实例如下：

```
gaussdb=# CREATE TABLE test1(name varchar) WITH(storage_type = ustore, init_td=2);
gaussdb=# \d+ test1
                        Table "public.test1"
 Column |       Type        | Modifiers | Storage  | Stats target | Description
--------+-------------------+-----------+----------+--------------+-------------
 name   | character varying |           | extended |              |
Has OIDs: no
Options: orientation=row, storage_type=ustore, init_td=2, compression=no, segment=off,
toast.storage_type=ustore, toast.toast_storage_type=enhanced_toast

gaussdb=# ALTER TABLE test1 set(init_td=8);
gaussdb=# \d+ test1
                        Table "public.test1"
 Column |       Type        | Modifiers | Storage  | Stats target | Description
--------+-------------------+-----------+----------+--------------+-------------
 name   | character varying |           | extended |              |
Has OIDs: no
Options: orientation=row, storage_type=ustore, compression=no, segment=off, init_td=8,
toast.storage_type=ustore, toast.toast_storage_type=enhanced_toast

gaussdb=# SELECT * FROM pg_thread_wait_status;
 node_name | db_name  |           thread_name           |    query_id     |       tid       |    sessionid    | lwtid |
psessionid | tlevel | smpid | wait_status | wait_event | locktag | lo
ckmode | block_sessionid | global_sessionid
-----------+----------+---------------------------------+-----------------+-----------------+-----------------+-------
+------------+--------+-------+-------------+------------+---------+---
-------+-----------------+------------------
 sgnode    |          | PageWriter                      |           0 | 139769678919424 | 139769678919424 | 16915
|          |     0 |     0 | none        | none       |         |
      |              | 0:0#0
 sgnode    |          | PageWriter                      |           0 | 139769736066816 | 139769736066816 | 16913
|          |     0 |     0 | none        | none       |         |
      |              | 0:0#0
 sgnode    |          | PageWriter                      |           0 | 139769707755264 | 139769707755264 | 16914
|          |     0 |     0 | none        | none       |         |
      |              | 0:0#0
 sgnode    |          | PageWriter                      |           0 | 139769761756928 | 139769761756928 | 16912
|          |     0 |     0 | none        | none       |         |
      |              | 0:0#0
 sgnode    |          | PageWriter                      |           0 | 139769783772928 | 139769783772928 | 16911
|          |     0 |     0 | none        | none       |         |
      |              | 0:0#0


gaussdb=# DROP TABLE test1;
DROP TABLE
```

怎么配置 fillfactor 大小

fillfactor是用于描述页面填充率的参数，该参数与页面能存放的元组数量、大小以及表的物理空间直接相关。Ustore表的默认页面填充率为92%，预留的8%空间用于更新的扩展，也可以用于TD列表的扩展空间。fillfactor的配置和详细描述参见《开发者指南》的“SQL参考 > SQL语法 > CREATE TABLE”章节。

用户需要结合业务分析是否需要手动配置fillfactor。如果表数据导入后只有查询或定长更新操作，可将页面填充率调整为100%。如果数据导入后存在大量非定长更新操作，建议为不调整页面填充率或者将页面填充率数值调整更小，以减少跨页更新带来的性能损耗。fillfactor查看和修改方法具体实例如下：

```
gaussdb=# CREATE TABLE test(a int) WITH(fillfactor=100);
gaussdb=# \d+ test
                      Table "public.test"
 Column |  Type   | Modifiers | Storage | Stats target | Description
```

 版权所有 © 华为云计算技术有限公司

```
--------+---------+-----------+---------+--------------+-------------
 a      | integer |           | plain   |              |
Has OIDs: no
Options: orientation=row, fillfactor=100, compression=no, storage_type=USTORE, segment=off

gaussdb=# ALTER TABLE test SET(fillfactor=92);
gaussdb=# \d+ test
                    Table "public.test"
 Column |  Type   | Modifiers | Storage | Stats target | Description
--------+---------+-----------+---------+--------------+-------------
 a      | integer |           | plain   |              |
Has OIDs: no
Options: orientation=row, compression=no, storage_type=USTORE, segment=off, fillfactor=92

gaussdb=# DROP TABLE test;
 DROP TABLE
```

在线校验功能

在线校验是Ustore特有的，在运行过程中可以有效预防页面因编码逻辑错误导致的逻辑损坏，默认开启UPAGE:UBTREE:UNDO三个模块校验。业务现网请保持开启，性能场景除外。

关闭：

```
gs_guc reload -Z datanode -N all -I all -c "ustore_attr=''"
```

打开：

```
gs_guc reload -Z datanode -N all -I all -c
"ustore_attr='ustore_verify_level=fast;ustore_verify_module=upage:ubtree:undo'"
```

怎么配置回滚段大小

一般情况下回滚段大小的参数使用默认值即可。为了达到最佳性能，部分场景下可调整回滚段大小的相关参数，具体场景与设置方法如下。

1. 保留给定时间内的历史版本数据。

   当使用闪回或者支撑问题定位时，通常希望保留更多历史版本数据，此时需要修改undo_retention_time（ guc参数在gaussdb.conf文件修改 ）。undo_retention_time默认值是0，取值范围为 0~3天，输入有效单位为s,min,h,d。

   调整的推荐值为900s，需要注意的是，undo_retention_time的取值越大，对业务的影响除了Undo空间占用增多，也会造成数据空间膨胀，进一步影响数据扫描更新性能。当不使用闪回或者希望减少历史旧版本的磁盘空间占用时，需要将undo_retention_time调小来达到最佳性能。可以通过如下方法选择更适合自己业务模型的取值：

   使用Undo统计信息的系统函数gs_stat_undo，如果入参为false，查看输出的info列中推荐的undo_retention_time 参数。具体详细参数值参见《 开发者指南 》的“SQL参考 > 函数和操作符 > undo系统函数”章节。

2. 保留给定空间大小的历史版本数据。

   如果业务中存在长事务或大事务可能导致Undo空间膨胀时，需要将undo_space_limit_size调大，undo_space_limit_size默认值为256GB，取值范围为800MB~16TB。

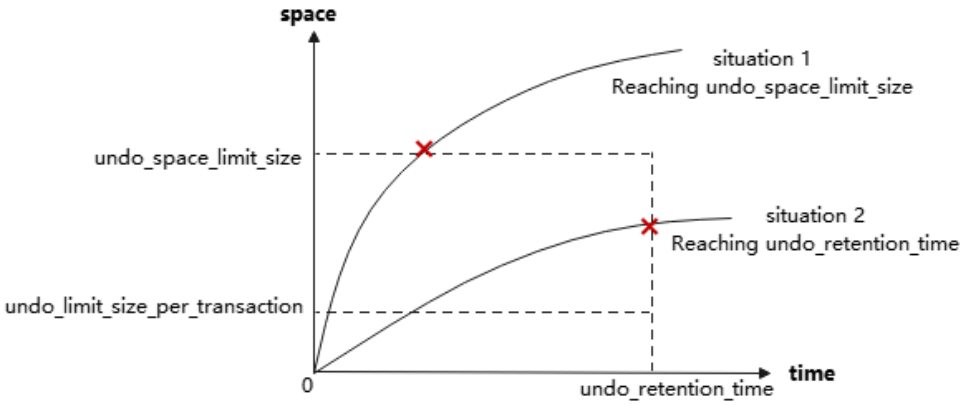
   在磁盘空间允许的条件下，推荐undo_space_limit_size设置翻倍。同时undo_space_limit_size的取值越大则占用磁盘空间越大，可能降低性能。如果查询视图系统函数gs_stat_undo的curr_used_undo_size发现不存在Undo空间膨胀，可以恢复为原值。

 版权所有 © 华为云计算技术有限公司

调整undo_space_limit_size后可相应提高单事务平均占用undo空间undo_limit_size_per_transaction的取值，undo_limit_size_per_transaction取值范围为2MB~16TB，默认值为32GB。设置时建议undo_limit_size_per_transaction不超过undo_space_limit_size，即单事务Undo分配空间阈值不大于Undo总空间阈值。

3. 历史版本的保留参数的调整优先级。

   在undo_retention_time、undo_space_limit_size、undo_limit_size_per_transaction中，先触发的空间阈值会先进行约束限制。



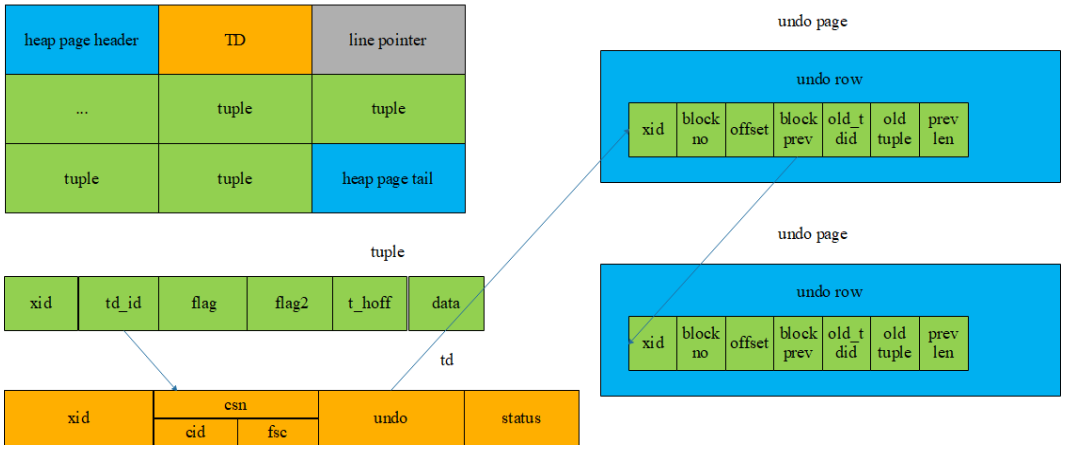
例如：Undo强制回收阈值参数undo_space_limit_size设置为1GB，Undo旧版本保留时间undo_retention_time为900s。如果900s内产生的历史版本数据不足1GB*0.8，则按照900s进行回收限制否则按照1GB*0.8进行回收限制。遇到该情况时，如果磁盘空闲空间充足，则上调undo_space_limit_size；如果磁盘空闲空间紧缺，则下调undo_retention_time。

2.9.3.2 存储格式

2.9.3.2.1 RCR Uheap

RCR Uheap 多版本管理

Ustore对其使用的heap做了如下重要的增强，简称Uheap。



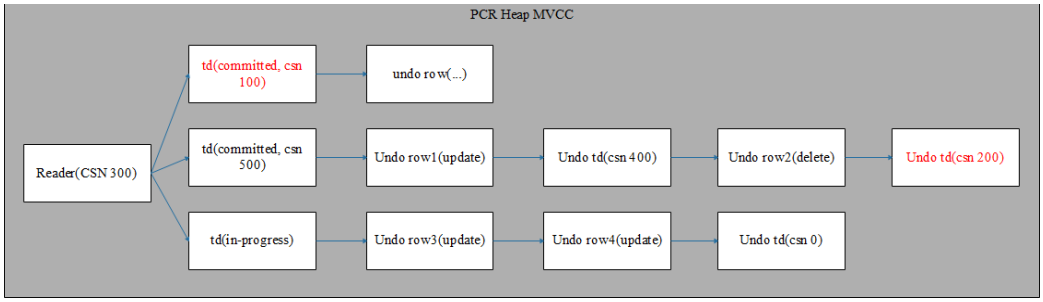
Ustore RCR(Row Consistency Read)的多版本管理是基于数据行的行级多版本管理，不过Ustore将XID记录在了页面的TD(Transaction Directory)区域，区别于常见的将

版权所有 © 华为云计算技术有限公司

XID存储在数据行上，节省了页面空间。事务修改记录时，会将历史数据记录到Undo Row中，在Tuple中的td_id指向的TD槽上记录产生的Undo Row地址(zone_id, block no, page offset)，并将新的数据覆盖写入页面。访问元组时，沿着版本链还原该元组，直到找到自己对应的版本。

RCR Uheap 可见性机制

Ustore可见性判断是通过构建数据行的一致性版本获得的，老快照可通过Undo记录获取历史版本。



RCR Uheap 空闲空间管理

Ustore使用Free Space Map（FSM）文件记录了每个数据页的潜在空闲空间，并且以树的结构组织起来。每当用户想要对某个表执行插入操作或者是非原位更新操作时，就会从该表对应的FSM中进行快速查找，查看当前FSM上记录的最大空闲空间是否可以满足插入所需的空间要求。如果满足则返回对应的blocknum用于执行插入操作，否则执行拓展页面逻辑。

每一个表或者分区对应的FSM结构存放在一个独立的FSM文件中，该FSM文件与表数据放在相同的目录下。例如，假设表t1对应的数据文件为32181，则其对应的FSM文件为32181_fsm。FSM内部同样是以数据块的格式存储，这里称为FSM block，FSM block之间的逻辑结构组成了一棵有三层节点的树，树的节点在逻辑上是大顶堆关系。每次在FSM上查找时从根节点进行，一直查找到叶子节点，然后在叶子节点内搜索到一个可用的页面并返回给业务用于执行后续操作。

该结构不保证和数据页实际可用空间保持实时一致，会在DML的执行过程中进行维护。Ustore会在Auto Vacuum的过程中概率性对该FSM进行修复重建。当用户执行插入类型的DML语句，类似Insert/Non-Inplace Update(新页面)/Multi Insert时，会查询FSM结构，寻找到一个可以插入当前记录的空间。用户执行完DML操作后会根据当前页面的潜在空闲空间与实际空闲空间的差值来决定是否将该页面的空闲空间刷新到FSM上。该差值越大，即潜在空间大于实际空间越多，则该页面被更新至FSM的几率越大。FSM上会记录数据页的潜在空闲空间 ，在用户执行插入操作找到一个页面时，如果该页面上的空闲空间较大则直接插入，否则如果潜在空间较大则对页面执行清理后插入，最后如果空间不够则重新搜索FSM结构或者拓展总页面数量。更新FSM结构主要有以下几个位置，DML、页面清理、vacuum、拓展页面、分区合并、页面扫描等。

2.9.3.2.2 UBTree

其使用的btree做了如下重要的增强，简称UBTree。

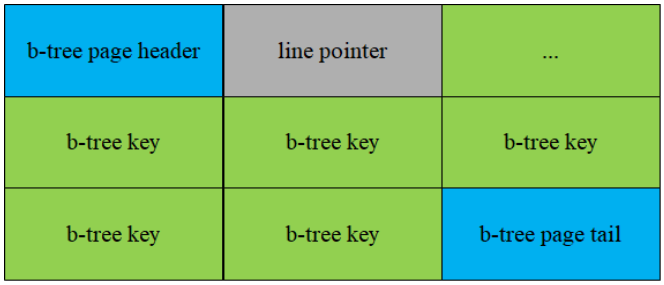
- UBTree索引增加了事务信息，能够独立进行MVCC。增加了IndexOnlyScan的比例，大大减少回表次数。
- 不依赖Vacuum进行旧版本清理。独立的空间回收能力，索引与堆表解耦，可独立清理，IO平稳度更优。

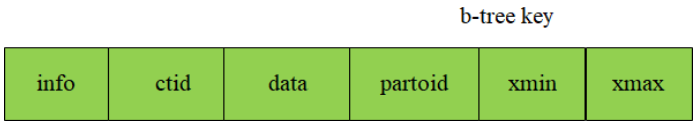
版权所有 © 华为云计算技术有限公司

RCR UBTree

?.1. RCR UBTree 多版本管理

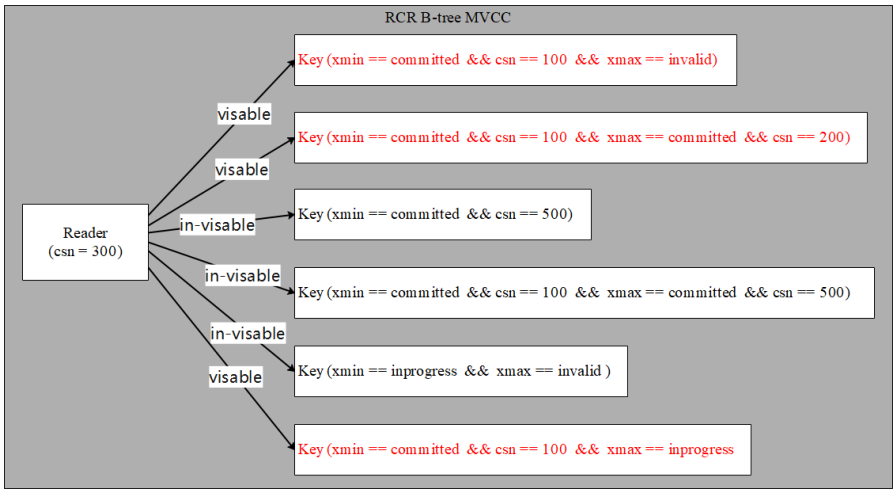
RCR(Row Consistency Read) UBtree的多版本管理是基于数据行的行级多版本管理，将XID记录在了数据行上，会增加Key的大小，索引会有5-20%左右的膨胀。最新版本和历史版本均在btree上，索引没有记录Undo信息。插入或者删除key时按照key + TID的顺序排列，索引列相同的元组按照对应元组的TID作为第二关键字进行排序。会将xmin、xmax追加到key的后面。索引分裂时，多版本信息随着key的迁移而迁移。





?.2. RCR UBTree 可见性机制

RCR UBTree可见性判断是通过判断Key上xmin/xmax来确定的，与Astore堆表数据行上xmin/xmax作用类似。

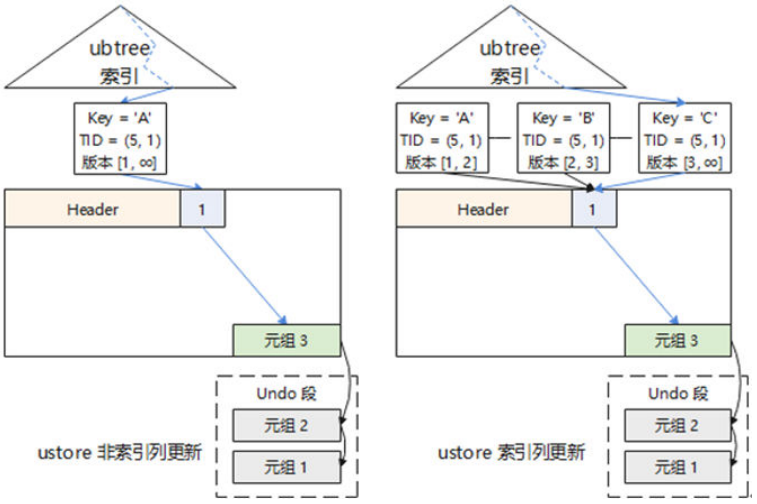


?.3. RCR UBTree 增删改查

- **Insert操作**：UBTree的插入逻辑基本不变，只需增加索引插入时直接获取事务信息填写xmin字段。
- **Delete操作**：UBTree额外增加了索引删除流程，索引删除主要步骤与插入相似，获取事务信息填写xmax字段（BTree索引不维护版本信息，不需要删除操作），同时更新页面上的active_tuple_count，若active_tuple_count被减为0，则尝试页面回收。

版权所有 © 华为云计算技术有限公司

- **Update操作**：对于Ustore而言，数据更新对UBTree索引列的操作也与Astore有所不同，数据更新包含两种情况：索引列和非索引列更新，下图给出了UBTree在数据发生更新时的处理。



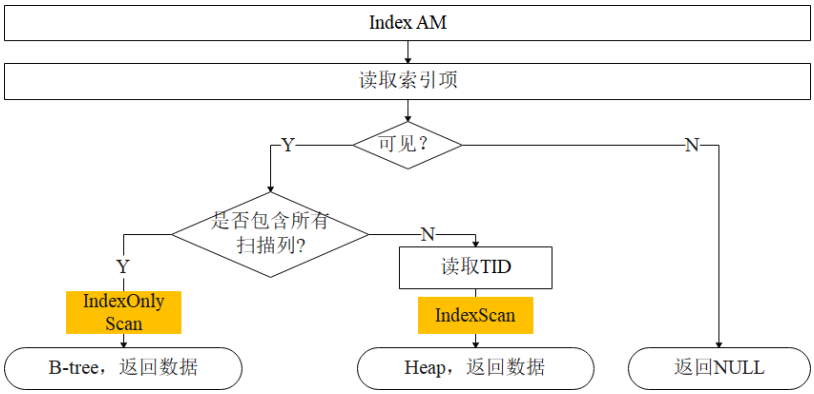
上图展示UBTree在索引列和非索引列更新的差异：

  - 在非索引列更新的情况下，索引不发生任何变化。index tuple仍指向第一次插入的data tuple，Uheap不会插入新的data tuple，而是修改当下data tuple并将历史数据存入Undo中。
  - 在索引列更新的情况下，UBTree也会插入新的index tuple，但是会指向同一个data linepointer和同一个data tuple，扫描旧版本的数据则需要从Undo中读取。

- **Scan操作**：用户在读取数据时，可通过使用索引扫描加速。UBTree支持索引数据的多版本管理及可见性检查，索引层的可见性检查使得索引扫描（Index Scan）及仅索引扫描（IndexOnly Scan）性能有所提升。

  对于索引扫描：

  - 若索引列包含所有扫描列（IndexOnly Scan），则通过扫描条件在索引上进行二分查找，找到符合条件元组即可返回数据。
  - 若索引列不包含所有扫描列（Index Scan），则通过扫描条件在索引上进行二分查找，找到符合条件元组的TID，再通过TID到数据表上查找对应的数据元组。如下图所示。

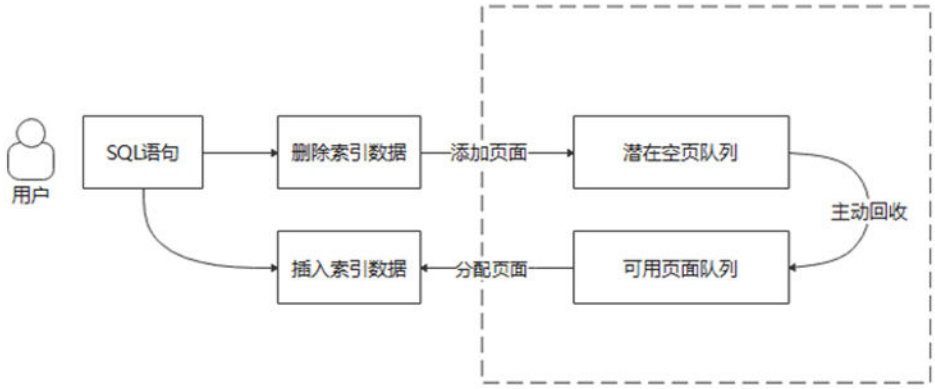


?.4. RCR UBTree 空间管理

当前Astore的索引依赖AutoVacuum和Free Space Map（FSM）进行空间管理，存在回收不及时的问题。而Ustore的索引使用其特有的URQ（UBTree Recycle Queue，一

版权所有 © 华为云计算技术有限公司

种基于循环队列的数据结构，即双循环队列），对索引空闲空间进行管理。双循环队列是指有两个循环队列，一个潜在空页队列，另一个可用空页队列，在DML过程中完成索引的空间管理，能有效地缓解DML过程中造成的空间急剧膨胀问题。索引回收队列单独储存在BTree索引对应的FSM文件中。



如上图所示，索引页面在双循环队列间流动如下：

1. **索引空页流动到潜在队列**

   索引页尾字段中记录了页面上活跃元组个数（activeTupleCount），在DML过程中，删空一个页面的所有元组，即activeTupleCount为零时会将索引页放入潜在队列中。

2. **潜在队列流动到可用队列**

   潜在队列到可用队列的转化主要是达到一个潜在队列收支平衡以及可用队列在拿页时有页可拿的目的，即当从可用队列拿出一个索引空页用完后，建议从潜在队列转化至少一个索引页面到可用队列中，以及每当潜在队列新加入一个索引页面时，能从潜在队列中移除至少一个索引页插入可用队列中，达到潜在队列的收支平衡，以及可用队列有页可用的目的。

3. **可用队列流动到索引空页**

   索引在分裂等获取一个索引空页面时，会先从可用队列中进行查找是否有可以复用的索引页。如果找到则直接进行复用，没有可复用页面则进行物理扩页。

PCR UBTree

相比于RCR版本的UBTree，PCR版本的UBTree有以下特点。

- 索引元组的事务信息统一由TD槽进行管理。
- 增加了Undo操作，插入和删除前需要先写入Undo，事务abort时需要进行回滚操作。
- 支持闪回。

PCR UBTree通过在创建索引时with选项设置“index_txntype=pcr”或者设置GUC参数“index_txntype=pcr”进行创建，若没有显示指定with选项或者GUC，则默认创建RCR版本的UBTree。

注意，当前版本PCR索引在大数据量的回滚上耗时可能较长（回滚时间随数据量增长可能呈指数型增长，数据量太大可能导致会回滚无法完全执行），回滚时间会在新的版本进行优化。以下是当前版本回滚时间的具体规格：

版权所有 © 华为云计算技术有限公司

表 2-9 PCR 索引回滚时间的规格

类型/数据量	100	1000	1万	10万	100万
带PCR索引的回滚时间	0.692 ms	9.610 ms	544.678 ms	52,963.754 ms	89,440,029.048 ms
不带PCR索引的回滚时间	0.226 ms	0.916 ms	8.974 ms	94.903 ms	1206.177 ms
两者比值	3.06	10.49	60.70	558.08	74,151.66

?.1. PCR UBTree 多版本管理

与RCR UBTree的区别是，PCR(Page Consistency Read) 的多版本管理是基于页面的多版本管理，所有元组的事务信息统一由TD槽进行管理。

?.2. PCR UBTree 可见性机制

PCR UBTree可见性判断是通过把页面回滚到快照可见的时刻得到元组全部可见的页面完成的。

?.3. PCR UBTree 增删改查

- **Insert操作**：操作与RCR UBTree基本一致，区别是：插入前需要先申请TD和写入Undo。
- **Delete操作**：操作与RCR UBTree基本一致，区别是：删除前需要先申请TD和写入Undo。
- **Update操作**：操作与RCR UBTree无区别，均转换为一条Delete操作和和一条Insert操作。
- **Scan操作**：操作与RCR UBTree基本一致，区别是：查询操作需要将页面复制一个CR页面出来，将CR页面回滚到扫描快照可见的状态，从而整个页面的元组对于快照都是可见版本。

?.4. PCR UBTree 空间管理

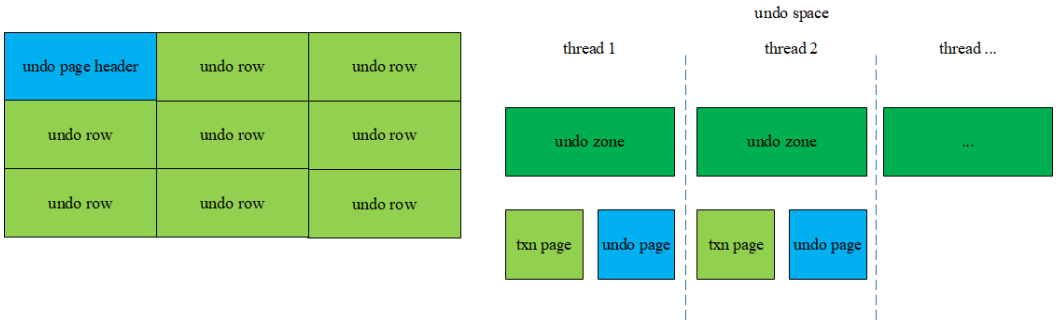
空间管理操作与RCR UBTree基本一致，区别是：PCR UBTree支持闪回，所以页面可回收的时间点由OldestXmin改成了GlobalRecycleXid。

2.9.3.2.3 Undo

历史版本数据集中存放在$node_dir/undo目录中，其中$node_dir为数据库节点路径，回滚段日志是与单个写事务关联的所有撤销日志的集合。支持permanent/unlogged/temp三种表类型。

版权所有 © 华为云计算技术有限公司

回滚段管理



1. 每个undo zone除了管理部分transaction page（用于存储事务回滚的元数据）外，还管理undo page。
2. Undo页面中存储undo row，对数据的修改会将历史版本记录到Undo中。
3. Undo记录也是数据，因此对Undo页面的修改同样会记录Redo。

文件组织结构

如需查询当前回滚段使用的存储方式是页式或段页式，可以查询系统表。当前仅支持页式。

示例：

```
gaussdb=# SELECT * FROM gs_global_config WHERE name LIKE '%undostoragetype%';
      name       |  value
-----------------+---------
 undostoragetype | page
(1 row)
```

- 当回滚段使用的存储方式为页式：
  - txn page所在文件组织结构：
    ```
    $node_dir/undo/{permanent|unlogged|temp}/$undo_zone_id.meta.$segno
    ```
  - undo row所在文件组织结构：
    ```
    $node_dir/undo/{permanent|unlogged|temp}/$undo_zone_id.$segno
    ```

空间管理

Undo子系统依赖后台回收线程进行空闲空间回收，负责主机上Undo模块的空间回收，备机通过回放xLog进行回收。回收线程遍历使用中的undo zone，对该zone中的txn page扫描，依据xid从小到大的顺序进行遍历。回收已提交或者已回滚完成的事务，且该事务的提交时间应早于$(current_time-undo_retention_time)。对于遍历过程中需要回滚的事务，后台回收线程会为该事务添加异步回滚任务。

当数据库中存在运行时间长、修改数据量大的事务，或者开启闪回时间较长的时候，可能出现undo空间持续膨胀的情况。当undo占用空间接近undo_space_limit_size时，就会触发强制回收。只要事务已提交或者已回滚完成，即使事务提交时间晚于$(current_time-undo_retention_time)，在这种情况下也可能被回收掉。

2.9.3.2.4 Enhanced Toast

概述

Enhanced Toast是一种用于处理超大字段的技术。首先，减少了Toast Pointer中的冗余信息，存储支持单表超长字段列数超过500列。其次，优化了主表与线外存储表之间

版权所有 © 华为云计算技术有限公司

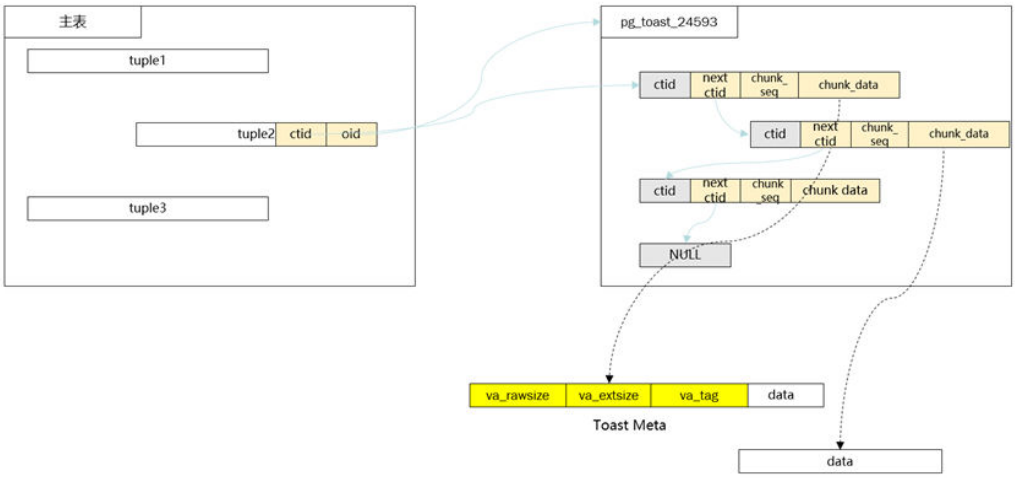
的映射关系，无需通过pg_toast_index来存储主表数据与线外存储表数据的关系，降低了用户存储空间。最后，Enhanced Toast技术通过让分割数据自链接，消除了Oid分配的依赖，极大地加快了写入效率。

说明

- Astore存储引擎不支持Enhanced Toast。
- 不支持对Enhanced Toast类型的线外存储表单独进行Vacuum Full操作。

Enhanced Toast 存储结构

Enhanced Toast技术使用自链接的方式来处理元组间的依赖关系。线外存储表把超长数据按照2K分割成链表块，主表的Toast Pointer指向线外存储表的对应数据链表头。这样极大简化了主表与线外存储表间的映射关系，有效的提升了数据写入与查询的性能。



?.1. Enhanced Toast 使用

新增的GUC参数enable_enhance_toast_table用于控制线外存储结构。
“enable_enhance_toast_table=on”表示使用Enhanced Toast线外存储表，
“enable_enhance_toast_table=off”表示使用Toast线外存储表。

```
gs_guc reload -D datadir -c "enable_enhance_toast_table=on"
```

Enhanced Toast 增删改查

Insert操作：触发Enhanced Toast的写入条件保持与原有Toast一致，除了数据写入时增加了数据间的链接信息之外，插入基本逻辑保持不变。

Delete操作：Enhanced Toast的数据删除流程不再依赖Toast数据索引，仅依靠数据间的链接信息将对应的数据进行遍历删除。

Update操作：Enhanced Toast的更新流程与原有Toast保持一致。

Enhanced Toast 相关 DDL 操作

Enhanced Toast表的创建

建表时指定Toast表的存储类型为Enhanced Toast或者Toast：

```
gaussdb=#  CREATE TABLE test_toast (id int, content text) WITH(toast.toast_storage_type=toast);
CREATE TABLE
```


版权所有 © 华为云计算技术有限公司

```
gaussdb=#  \d+ test_toast
                    Table "public.test_toast"
 Column  |  Type   | Modifiers | Storage  | Stats target | Description
---------+---------+-----------+----------+--------------+-------------
 id      | integer |           | plain    |              |
 content | text    |           | extended |              |
Has OIDs: no
Options: orientation=row, compression=no, storage_type=USTORE, segment=off,
toast.storage_type=USTORE, toast.toast_storage_type=toast
gaussdb=#  drop table test_toast;
DROP TABLE
gaussdb=#  CREATE TABLE test_toast (id int, content text) WITH(toast.toast_storage_type=enhanced_toast);
CREATE TABLE
gaussdb=#  \d+ test_toast
                    Table "public.test_toast"
 Column  |  Type   | Modifiers | Storage  | Stats target | Description
---------+---------+-----------+----------+--------------+-------------
 id      | integer |           | plain    |              |
 content | text    |           | extended |              |
Has OIDs: no
Options: orientation=row, compression=no, storage_type=USTORE, segment=off,
toast.storage_type=USTORE, toast.toast_storage_type=enhanced_toast
gaussdb=#  DROP TABLE test_toast;
DROP TABLE
```

建表时不指定线外存储表的类型，则创建线外存储表类型依赖于GUC参数enable_enhance_toast_table：

```
gaussdb=#  show enable_enhance_toast_table;
 enable_enhance_toast_table
----------------------------
 on
(1 row)
gaussdb=#  CREATE TABLE test_toast (id int, content text);
CREATE TABLE
gaussdb=#   \d+ test_toast
                    Table "public.test_toast"
 Column  |  Type   | Modifiers | Storage  | Stats target | Description
---------+---------+-----------+----------+--------------+-------------
 id      | integer |           | plain    |              |
 content | text    |           | extended |              |
Has OIDs: no
Options: orientation=row, compression=no, storage_type=USTORE, segment=off,
toast.storage_type=USTORE, toast.toast_storage_type=enhanced_toast

gaussdb=#   DROP TABLE test_toast;
DROP TABLE
gaussdb=#  SET enable_enhance_toast_table = off;
SET
gaussdb=#  show enable_enhance_toast_table;
 enable_enhance_toast_table
----------------------------
 off
(1 row)
gaussdb=#   CREATE TABLE test_toast (id int, content text);
CREATE TABLE
gaussdb=#  \d+ test_toast
                    Table "public.test_toast"
 Column  |  Type   | Modifiers | Storage  | Stats target | Description
---------+---------+-----------+----------+--------------+-------------
 id      | integer |           | plain    |              |
 content | text    |           | extended |              |
Has OIDs: no
Options: orientation=row, compression=no, storage_type=USTORE, segment=off,
toast.storage_type=USTORE, toast.toast_storage_type=toast
gaussdb=#  DROP TABLE test_toast;
DROP TABLE
```

线外存储表结构的升级

 版权所有 © 华为云计算技术有限公司

当GUC参数“enable_enhance_toast_table=on”时，线外存储表支持通过Vacuum Full操作将Toast升级为Enhanced Toast结构。

```
gaussdb=#   CREATE TABLE test_toast (id int, content text);
CREATE TABLE
gaussdb=#  \d+ test_toast
                    Table "public.test_toast"
 Column  |  Type   | Modifiers | Storage  | Stats target | Description
---------+---------+-----------+----------+--------------+-------------
 id      | integer |           | plain    |              |
 content | text    |           | extended |              |
Has OIDs: no
Options: orientation=row, compression=no, storage_type=USTORE, segment=off,
toast.storage_type=USTORE, toast.toast_storage_type=toast
gaussdb=#  VACUUM FULL test_toast;
VACUUM
gaussdb=#  \d+ test_toast
                    Table "public.test_toast"
 Column  |  Type   | Modifiers | Storage  | Stats target | Description
---------+---------+-----------+----------+--------------+-------------
 id      | integer |           | plain    |              |
 content | text    |           | extended |              |
Has OIDs: no
Options: orientation=row, compression=no, storage_type=USTORE, segment=off,
toast.storage_type=USTORE, toast.toast_storage_type=enhanced_toast
gaussdb=#  DROP TABLE test_toast;
DROP TABLE
```

分区表merge操作

支持将分区表的分区间不同的线外存储表类型进行合并操作。

说明

- 对于相同类型的线外存储分区，合并与原有逻辑保持一致，进行物理合并。
- 对于不同类型的线外存储分区，合并后的分区线外存储表为Enhanced Toast表，需要进行逻辑合并，性能劣于物理合并。

```
gaussdb=#  CREATE TABLE test_partition_table(a int, b text)PARTITION BY range(a)(partition p1 values less
than (2000),partition p2 values less than (3000));
gaussdb=# SELECT relfilenode FROM pg_partition WHERE relname='p1';
 relfilenode
-------------
       17529
(1 row)

gaussdb=#  \d+ pg_toast.pg_toast_part_17529
TOAST table "pg_toast.pg_toast_part_17529"
   Column   |  Type   | Storage
------------+---------+---------
 chunk_id   | oid     | plain
 chunk_seq  | integer | plain
 chunk_data | bytea   | plain
Options: storage_type=ustore, toast_storage_type=toast

gaussdb=# SELECT relfilenode from pg_partition WHERE relname='p2';
 relfilenode
-------------
       17528
(1 row)

gaussdb=#  \d+ pg_toast.pg_toast_part_17528
TOAST table "pg_toast.pg_toast_part_17528"
   Column   |  Type   | Storage
------------+---------+---------
 chunk_seq  | integer | plain
 next_chunk | tid     | plain
```

 版权所有 © 华为云计算技术有限公司

```
 chunk_data | bytea   | plain
Options: storage_type=ustore, toast_storage_type=enhanced_toast
gaussdb=#  ALTER TABLE test_partition_table MERGE PARTITIONS p1,p2 INTO partition p1_p2;
ALTER TABLE
gaussdb=#  SELECT reltoastrelid::regclass FROM pg_partition WHERE relname='p1_p2';
        reltoastrelid
------------------------------
 pg_toast.pg_toast_part_17559
(1 row)
gaussdb=#  \d+ pg_toast.pg_toast_part_17559
TOAST table "pg_toast.pg_toast_part_17559"
   Column   |  Type   | Storage
------------+---------+---------
 chunk_seq  | integer | plain
 next_chunk | tid     | plain
 chunk_data | bytea   | plain
Options: storage_type=ustore, toast_storage_type=enhanced_toast

gaussdb=# DROP TABLE test_partition_table;
DROP TABLE
```

Enhanced Toast 运维管理

通过gs_parse_page_bypath解析主表中的ToastPointer信息。

```
gaussdb=#  SELECT ctid,next_chunk,chunk_seq FROM pg_toast.pg_toast_part_17559;
 ctid  | next_chunk | chunk_seq
-------+------------+-----------
 (0,1) | (0,0)      |         1
 (0,2) | (0,1)      |         0
 (0,3) | (0,0)      |         1
 (0,4) | (0,3)      |         0
(4 rows)
gaussdb=#  SELECT gs_parse_page_bypath((SELECT * FROM
pg_relation_filepath('test_toast')),0,'uheap',false);
                      gs_parse_page_bypath
-----------------------------------------------------------------------
 ${data_dir}/gs_log/dump/1663_13113_17603_0.page
(1 row)
```

解析文件1663_13113_17603_0.page中存储了ToastPointer的相关信息，具体如下：

```
Toast_Pointer:
 column_index: 1
  toast_relation_oid: 17608    --线外存储表OID信息
  ctid: (4, 1)                 --线外存储数据链，头指针
  bucket id: -1                --bucket id信息
 column_index: 2
  toast_relation_oid: 17608
  ctid: (2, 1)
  bucket id: -1
```

Enhanced Toast数据查询，通过直接查询到的Enhanced Toast表数据可以判断其链式结构的完整性。

```
gaussdb=#  SELECT ctid,next_chunk,chunk_seq FROM pg_toast.pg_toast_part_17559;
 ctid  | next_chunk | chunk_seq
-------+------------+-----------
 (0,1) | (0,0)      |         1
 (0,2) | (0,1)      |         0
 (0,3) | (0,0)      |         1
 (0,4) | (0,3)      |         0
(4 rows)
```

2.9.3.3 Ustore 事务模型

GaussDB事务基础：

 版权所有 © 华为云计算技术有限公司

1. 事务启动时不会自动分配XID，该事务中的第一条DML/DDL语句运行时才会真正为该事务分配XID。
2. 事务结束时，会产生代表事务提交状态的CLOG（Commit Log），CLOG共有四种状态：事务运行中、事务提交、事务同步回滚、子事务提交。每个事务的 CLOG 状态位为2 bits，CLOG页面上每个字节可以表示四个事务的提交状态。
3. 事务结束时，还会产生代表事务提交顺序的CSN（Commit sequence number），CSN为实例级变量，每个XID都有自己对应的唯一CSN。CSN可以标记事务的以下状态：事务提交中、事务提交、事务回滚、事务已冻结等。

2.9.3.3.1 事务提交

针对隐式事务和显式事务，其提交策略如下所示：

1. 隐式事务。单条DML/DDL语句自动触发隐式事务，这种事务没有显式的事务块控制语句（START TRANSACTION/BEGIN/COMMIT/END），DML语句结束后自动提交。
2. 显式事务。显式事务由显式的START TRANSACTION/BEGIN语句控制事务的开始，由COMMIT/END语句控制事务的提交。

   子事务必须存在于显式事务或存储过程中，由SAVEPOINT语句控制子事务开始，由RELEASE SAVEPOINT语句控制子事务结束。如果一个事务在提交时还存在未释放的子事务，该事务提交前会先执行子事务的提交，所有子事务提交完毕后才会进行父事务的提交。

   Ustore支持读已提交隔离级别。语句在执行开始时，获取当前系统的CSN作为当前语句的查询CSN。整个语句的可见结果由语句开始那一刻决定，不受后续其他事务修改影响。Ustore中read committed默认是保持一致性读的。Ustore也支持标准的2PC事务。

2.9.3.3.2 事务回滚

回滚是在事务运行的过程中发生了故障等异常情形下，事务不能继续执行，系统需要将事务中已完成的修改操作进行撤销。Astore、Ubtree没有回滚段，自然没有这个专门的回滚动作。Ustore为了性能考虑，它的回滚流程结合了同步、异步和页面级回滚等3种形式。

- **同步回滚**

  有三种情况会触发事务的同步回滚：

  - 事务块中的ROLLBACK关键字会触发同步回滚。
  - 事务运行过程中如果发生ERROR级别报错，此时的COMMIT关键字与ROLLBACK功能相同，也会触发同步回滚。
  - 事务运行过程中如果发生FATAL/PANIC级别报错，在线程退出前会尝试将该线程绑定的事务进行一次同步回滚。

- **异步回滚**

  同步回滚失败或者在系统宕机后再次重启时，会由Undo回收线程为未回滚完成的事务发起异步回滚任务，立即对外提供服务。由异步回滚任务发起线程undo launch负责拉起异步回滚工作线程undo worker，再由异步回滚工作线程实际执行回滚任务。undo launch线程最多可以同时拉起5个undo worker线程。

- **页面级回滚**

  当事务需要回滚但还未回滚到本页面时，如果其他事务需要复用该事务所占用的TD，就会在复用前对该事务在本页面的所有修改执行页面级回滚。页面级回滚只负责回滚事务在本页面的修改，不涉及其他页面。

 版权所有 © 华为云计算技术有限公司

Ustore子事务的回滚由ROLLBACK TO SAVEPOINT语句控制，子事务回滚后父事务可以继续运行，子事务的回滚不影响父事务的事务状态。如果一个事务在回滚时还存在未释放的子事务，该事务回滚前会先执行子事务的回滚，所有子事务回滚完毕后才会进行父事务的回滚。

2.9.3.4 闪回恢复

闪回恢复功能是数据库恢复技术的一环，可以有选择性地撤销一个已提交事务的影响，将数据从人为不正确的操作中进行恢复。在采用闪回技术之前，只能通过备份恢复、PITR等手段找回已提交的数据库修改，恢复时长需要数分钟甚至数小时。采用闪回技术后，通过闪回Drop和闪回Truncate恢复已提交的数据库Drop/Truncate的数据，只需要秒级，而且恢复时间和数据库大小无关。

说明

- Astore引擎只支持闪回DROP/TRUNCATE功能。
- 备机不支持闪回操作。
- 用户可以根据需要开启闪回功能，开启后会带来一定的性能劣化。

2.9.3.4.1 闪回查询

背景信息

闪回查询可以查询过去某个时间点表的某个snapshot数据，这一特性可用于查看和逻辑重建意外删除或更改的受损数据。闪回查询基于MVCC多版本机制，通过检索查询旧版本，获取指定老版本数据。

前提条件

整体方案分为三部分：旧版本保留、快照的维护和旧版本检索。旧版本保留：新增undo_retention_time配置参数，用来设置旧版本保留的时间，超过该时间的旧版本将被回收清理，若使用闪回查询则需将该参数设置为大于0的值，请联系管理员修改。

语法

```
{[ ONLY ] table_name [ * ] [ partition_clause ] [ [ AS ] alias [ ( column_alias [, ...] ) ] ]
[ TABLESAMPLE sampling_method ( argument [, ...] ) [ REPEATABLE ( seed ) ] ]
[TIMECAPSULE { TIMESTAMP | CSN } expression ]
|( select ) [ AS ] alias [ ( column_alias [, ...] ) ]
|with_query_name [ [ AS ] alias [ ( column_alias [, ...] ) ] ]
|function_name ( [ argument [, ...] ] ) [ AS ] alias [ ( column_alias [, ...] | column_definition [, ...] ) ]
|function_name ( [ argument [, ...] ] ) AS ( column_definition [, ...] )
|from_item [ NATURAL ] join_type from_item [ ON join_condition | USING ( join_column [, ...] ) ]}
```

语法树中“TIMECAPSULE {TIMESTAMP | CSN} expression”为闪回功能新增表达方式，其中TIMECAPSULE表示使用闪回功能，TIMESTAMP以及CSN表示闪回功能使用具体时间点信息或使用CSN（commit sequence number）信息。

参数说明

- TIMESTAMP
  - 指要查询某个表在TIMESTAMP这个时间点上的数据，TIMESTAMP指一个具体的历史时间。
- CSN

 版权所有 © 华为云计算技术有限公司

- 指要查询整个数据库逻辑提交序下某个CSN点的数据，CSN指一个具体逻辑提交时间点，数据库中的CSN为写一致性点，每个CSN代表整个数据库的一个一致性点，查询某个CSN下的数据代表SQL查询数据库在该一致性点的相关数据。

⚠ 注意

使用时间点进行闪回时，可能会有3s的误差。想要闪回到精确的操作点，需要使用CSN进行闪回。

使用示例

- 示例（需将undo_retention_time参数设置为大于0的值）：

```
gaussdb=# DROP TABLE IF EXISTS "public".flashtest;
NOTICE:  table "flashtest" does not exist, skipping
DROP TABLE
--创建表flashtest
gaussdb=# CREATE TABLE "public".flashtest (col1 INT,col2 TEXT) WITH(storage_type=ustore);
CREATE TABLE
--查询csn
gaussdb=# SELECT int8in(xidout(next_csn)) FROM gs_get_next_xid_csn();
  int8in
----------
 79351682
(1 rows)
--查询当前时间戳
gaussdb=# select now();
              now
-------------------------------
 2023-09-13 19:35:26.011986+08
(1 row)
--插入数据
gaussdb=# INSERT INTO flashtest VALUES(1,'INSERT1'),(2,'INSERT2'),(3,'INSERT3'),(4,'INSERT4'),
(5,'INSERT5'),(6,'INSERT6');
INSERT 0 6
gaussdb=# SELECT * FROM flashtest;
 col1 |  col2
------+---------
    3 | INSERT3
    1 | INSERT1
    2 | INSERT2
    4 | INSERT4
    5 | INSERT5
    6 | INSERT6
(6 rows)
--闪回查询某个csn处的表
gaussdb=# SELECT * FROM flashtest TIMECAPSULE CSN 79351682;
 col1 | col2
------+------
(0 rows)
gaussdb=# SELECT * FROM flashtest;
 col1 |  col2
------+---------
    1 | INSERT1
    2 | INSERT2
    4 | INSERT4
    5 | INSERT5
    3 | INSERT3
    6 | INSERT6
(6 rows)
--闪回查询某个时间戳处的表
gaussdb=# SELECT * FROM flashtest TIMECAPSULE TIMESTAMP '2023-09-13 19:35:26.011986';
 col1 | col2
------+------
```


版权所有 © 华为云计算技术有限公司

```
(0 rows)
gaussdb=# SELECT * FROM flashtest;
 col1 |  col2
------+---------
    1 | INSERT1
    2 | INSERT2
    4 | INSERT4
    5 | INSERT5
    3 | INSERT3
    6 | INSERT6
(6 rows)
--闪回查询某个时间戳处的表
gaussdb=# SELECT * FROM flashtest TIMECAPSULE TIMESTAMP to_timestamp ('2023-09-13
19:35:26.011986', 'YYYY-MM-DD HH24:MI:SS.FF');
 col1 | col2
------+------
(0 rows)
--闪回查询某个csn处的表，并对表进行重命名
gaussdb=# SELECT * FROM flashtest AS ft TIMECAPSULE CSN 79351682;
 col1 | col2
------+------
(0 rows)
gaussdb=# DROP TABLE IF EXISTS "public".flashtest;
DROP TABLE
```

2.9.3.4.2 闪回表

背景信息

闪回表可以将表恢复至特定时间点，当逻辑损坏仅限于一个或一组表，而不是整个数据库时，此特性可以快速恢复表的数据。闪回表基于MVCC多版本机制，通过删除指定时间点和该时间点之后的增量数据，并找回指定时间点和当前时间点删除的数据，实现表级数据还原。

前提条件

整体方案分为三部分：旧版本保留、快照的维护和旧版本检索。旧版本保留：新增undo_retention_time配置参数，用来设置旧版本保留的时间，超过该时间的旧版本将被回收清理，请联系管理员修改。

语法

```
TIMECAPSULE TABLE table_name TO { TIMESTAMP | CSN } expression
```

使用示例

```
gaussdb=# DROP TABLE IF EXISTS "public".flashtest;
NOTICE:  table "flashtest" does not exist, skipping
DROP TABLE
--创建表flashtest
gaussdb=# CREATE TABLE "public".flashtest (col1 INT,col2 TEXT) WITH(storage_type=ustore);
CREATE TABLE
--查询csn
gaussdb=# SELECT int8in(xidout(next_csn)) FROM gs_get_next_xid_csn();
  int8in
----------
 79352065
(1 rows)
--查询当前时间戳
gaussdb=# SELECT now();
              now
--------------------------------
 2023-09-13 19:46:34.102863+08
(1 row)
```

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# SELECT * FROM flashtest;
 col1 | col2
------+------
(0 rows)
--插入数据
gaussdb=# INSERT INTO flashtest VALUES(1,'INSERT1'),(2,'INSERT2'),(3,'INSERT3'),(4,'INSERT4'),
(5,'INSERT5'),(6,'INSERT6');
INSERT 0 6
gaussdb=# SELECT * FROM flashtest;
 col1 |  col2
------+---------
    3 | INSERT3
    6 | INSERT6
    1 | INSERT1
    2 | INSERT2
    4 | INSERT4
    5 | INSERT5
(6 rows)
--闪回表至特定的时间戳
gaussdb=# TIMECAPSULE TABLE flashtest TO TIMESTAMP to_timestamp ('2023-09-13 19:52:21.551028',
'YYYY-MM-DD HH24:MI:SS.FF');
TimeCapsule Table
gaussdb=# SELECT * FROM flashtest;
 col1 | col2
------+------
(0 rows)
gaussdb=# SELECT now();
              now
-------------------------------
 2023-09-13 19:54:00.641506+08
(1 row)
--插入数据
gaussdb=# INSERT INTO flashtest VALUES(1,'INSERT1'),(2,'INSERT2'),(3,'INSERT3'),(4,'INSERT4'),
(5,'INSERT5'),(6,'INSERT6');
INSERT 0 6
gaussdb=# SELECT * FROM flashtest;
 col1 |  col2
------+---------
    3 | INSERT3
    6 | INSERT6
    1 | INSERT1
    2 | INSERT2
    4 | INSERT4
    5 | INSERT5
(6 rows)
--闪回表至特定的时间戳
gaussdb=# TIMECAPSULE TABLE flashtest TO TIMESTAMP '2023-09-13 19:54:00.641506';
TimeCapsule Table
gaussdb=# SELECT * FROM flashtest;
 col1 | col2
------+------
(0 rows)
gaussdb=# DROP TABLE IF EXISTS "public".flashtest;
DROP TABLE
```

2.9.3.4.3 闪回 DROP/TRUNCATE

背景信息

- 闪回DROP：可以恢复意外删除的表，从回收站（recyclebin）中恢复被删除的表及其附属结构如索引、表约束等。闪回drop是基于回收站机制，通过还原回收站中记录的表的物理文件，实现已drop表的恢复。
- 闪回TRUNCATE：可以恢复误操作或意外被进行truncate的表，从回收站中恢复被truncate的表及索引的物理数据。闪回truncate基于回收站机制，通过还原回收站中记录的表的物理文件，实现已truncate表的恢复。

 版权所有 © 华为云计算技术有限公司

前提条件

- 开启enable_recyclebin参数（GUC参数在gaussdb.conf文件修改），启用回收站，请联系管理员修改。
- recyclebin_retention_time参数用于设置回收站对象保留时间，超过该时间的回收站对象将被自动清理，请联系管理员修改。

相关语法

- 删除表
```
DROP TABLE table_name [PURGE]
```
- 清理回收站对象
```
PURGE { TABLE { table_name }
| INDEX { index_name }
| RECYCLEBIN
}
```
- 闪回被删除的表
```
TIMECAPSULE TABLE { table_name } TO BEFORE DROP [RENAME TO new_tablename]
```
- 截断表
```
TRUNCATE TABLE { table_name } [ PURGE ]
```
- 闪回截断的表
```
TIMECAPSULE TABLE { table_name } TO BEFORE TRUNCATE
```

参数说明

- **DROP/TRUNCATE TABLE table_name PURGE**
  - 默认将表数据放入回收站中，PURGE直接清理。
- **PURGE RECYCLEBIN**
  - 表示清理回收站对象。
- **TO BEFORE DROP**

  使用这个子句检索回收站中已删除的表及其子对象。

  可以指定原始用户指定的表的名称，或对象删除时数据库分配的系统生成名称。
  - 回收站中系统生成的对象名称是唯一的。因此，如果指定系统生成名称，那么数据库检索指定的对象。使用“select * from gs_recyclebin;”语句查看回收站中的内容。
  - 在指定了用户指定的名称且回收站中包含多个该名称的对象的情况下，数据库检索回收站中最近移动的对象，如果想要检索更早版本的表，你可以这样做：
    - 指定你想要检索的表的系统生成名称。
    - 执行TIMECAPSULE TABLE ... TO BEFORE DROP语句，直到你要检索的表。
  - 恢复DROP表时，只恢复基表名，其他子对象名均保持回收站对象名。用户可根据需要，执行DDL命令手工调整子对象名。
  - 回收站对象不支持DML、DCL、DDL等写操作，不支持DQL查询操作（后续支持）。
  - 闪回点和当前点之间，执行过修改表结构或影响物理结构的语句，闪回失败。执行过DDL的表进行闪回操作报错：“ERROR：The table definition of %s has been changed.”。涉及namespace、表名改变等操作的DDL执行闪回操作报错： ERROR: recycle object %s desired does not exis；

 版权所有 © 华为云计算技术有限公司

  - 如果基表有truncate trigger，truncate表时，无法触发trigger，不能同时truncate目标表。用户想要truncate目标表，需要手动操作。

- **RENAME TO**

  为从回收站中检索的表指定一个新名称。

- **TO BEFORE TRUNCATE**

  闪回到TRUNCATE之前。

语法示例

```
-- PURGE TABLE table_name; --
--查看回收站
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid | rcyname | rcyoriginname | rcyoperation | rcytype | rcyrecyclecsn |
rcyrecycletime | rcycreatecsn | rcychangecsn | rcynamespace | rcyowner | rcytablespace
 | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+---------+---------------+--------------+---------+---------------+----------------
+--------------+--------------+--------------+----------+--------------
-+----------------+---------------+-------------+--------------+----------------+-----------
(0 rows)

gaussdb=# DROP TABLE IF EXISTS flashtest;
NOTICE:  table "flashtest" does not exist, skipping
DROP TABLE
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid | rcyname | rcyoriginname | rcyoperation | rcytype | rcyrecyclecsn |
rcyrecycletime | rcycreatecsn | rcychangecsn | rcynamespace | rcyowner | rcytablespace
 | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+---------+---------------+--------------+---------+---------------+----------------
+--------------+--------------+--------------+----------+--------------
-+----------------+---------------+-------------+--------------+----------------+-----------
(0 rows)
--创建表flashtest
gaussdb=# CREATE TABLE IF NOT EXISTS flashtest(id int, name text) with (storage_type = ustore);
CREATE TABLE
--插入数据
gaussdb=# INSERT INTO flashtest VALUES(1, 'A');
INSERT 0 1
gaussdb=# SELECT * FROM flashtest;
 id | name
----+------
  1 | A
(1 row)

--DROP表flashtest
gaussdb=# DROP TABLE IF EXISTS flashtest;
DROP TABLE
--查看回收站，删除的表被放入回收站
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid |          rcyname           |    rcyoriginname     | rcyoperation | rcytype |
rcyrecyclecsn |        rcyrecycletime         | rcycreatecsn | rcychangecs
n | rcynamespace | rcyowner | rcytablespace | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid |
rcyfrozenxid64 | rcybucket
-----------+---------+----------+----------------------------+----------------------+--------------+---------
+---------------+--------------------------------+--------------+------------
--+--------------+----------+---------------+----------------+---------------+-------------+--------------
+----------------+-----------
    18591 |   12737 |    18585 | BIN$31C14EB4899$9737$0==$0   | flashtest            | d            |       0 |
79352606 | 2023-09-13 20:01:28.640664+08 |     79352595 |     7935259
5 |         2200 |       10 |            0 |         18585 | t            | t           | 225492       |        225492 |
    18591 |   12737 |    18588 | BIN$31C14EB489C$12D1BF60==$0 | pg_toast_18585       | d            |       2
|     79352606 | 2023-09-13 20:01:28.641018+08 |            0 |
0 |          99 |       10 |            0 |         18588 | f            | f           | 225492       |        225492 |
(2 rows)
--查看表flashtest，表不存在
gaussdb=# SELECT * FROM flashtest;
ERROR:  relation "flashtest" does not exist
```

 版权所有 © 华为云计算技术有限公司

```
LINE 1: select * from flashtest;
                    ^
--PURGE表，将回收站中的表删除
gaussdb=# PURGE TABLE flashtest;
PURGE TABLE
--查看回收站，回收站中的表被删除
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid | rcyname | rcyoriginname | rcyoperation | rcytype | rcyrecyclecsn |
rcyrecycletime | rcycreatecsn | rcychangecsn | rcynamespace | rcyowner | rcytablespace
 | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+---------+---------------+--------------+---------+---------------+----------------
+--------------+--------------+--------------+----------+--------------
-+----------------+---------------+-------------+--------------+----------------+-----------
(0 rows)


-- PURGE INDEX index_name; --
gaussdb=# DROP TABLE IF EXISTS flashtest;
NOTICE:  table "flashtest" does not exist, skipping
DROP TABLE
--创建表flashtest
gaussdb=# CREATE TABLE IF NOT EXISTS flashtest(id int, name text) with (storage_type = ustore);
CREATE TABLE
--为表flashtest创建索引flashtest_index
gaussdb=# CREATE INDEX flashtest_index ON flashtest(id);
CREATE INDEX
--DROP表
gaussdb=# DROP TABLE IF EXISTS flashtest;
DROP TABLE
--查看回收站
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid |          rcyname           |    rcyoriginname     | rcyoperation | rcytype |
rcyrecyclecsn |        rcyrecycletime         | rcycreatecsn | rcychangecs
n | rcynamespace | rcyowner | rcytablespace | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid |
rcyfrozenxid64 | rcybucket
-----------+---------+----------+-----------------------------+----------------------+--------------+---------
+---------------+-----------------------------+--------------+------------
--+--------------+----------+--------------+----------------+---------------+-------------+--------------
+----------------+-----------
    18648 |   12737 |    18641 | BIN$31C14EB48D1$9A85$0==$0   | flashtest           | d            |       0 |
79354509 | 2023-09-13 20:40:11.360638+08 |     79354506 |     7935450
8 |         2200 |       10 |             0 |          18641 | t            | t           | 226642       |        226642 |
    18648 |   12737 |    18644 | BIN$31C14EB48D4$12E236A0==$0 | pg_toast_18641       | d            |       2
|     79354509 | 2023-09-13 20:40:11.36112+08  |             0 |
0 |          99 |       10 |             0 |          18644 | f            | f           | 226642       |        226642 |
    18648 |   12737 |    18647 | BIN$31C14EB48D7$9A85$0==$0   | flashtest_index      | d            |       1 |
79354509 | 2023-09-13 20:40:11.361246+08 |     79354508 |     7935450
8 |         2200 |       10 |             0 |          18647 | f            | t           | 0            |             0 |
(3 rows)

--PURGE索引flashtest_index
gaussdb=# PURGE INDEX flashtest_index;
PURGE INDEX
--查看回收站，回收站中的索引flashtest_index被删除
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid |          rcyname           |    rcyoriginname     | rcyoperation | rcytype |
rcyrecyclecsn |        rcyrecycletime         | rcycreatecsn | rcychangecs
n | rcynamespace | rcyowner | rcytablespace | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid |
rcyfrozenxid64 | rcybucket
-----------+---------+----------+-----------------------------+----------------------+--------------+---------
+---------------+-----------------------------+--------------+------------
--+--------------+----------+--------------+----------------+---------------+-------------+--------------
+----------------+-----------
    18648 |   12737 |    18641 | BIN$31C14EB48D1$9A85$0==$0   | flashtest           | d            |       0 |
79354509 | 2023-09-13 20:40:11.360638+08 |     79354506 |     7935450
8 |         2200 |       10 |             0 |          18641 | t            | t           | 226642       |        226642 |
    18648 |   12737 |    18644 | BIN$31C14EB48D4$12E236A0==$0 | pg_toast_18641       | d            |       2
|     79354509 | 2023-09-13 20:40:11.36112+08  |             0 |
0 |          99 |       10 |             0 |          18644 | f            | f           | 226642       |        226642 |
```

 版权所有 © 华为云计算技术有限公司

```
(2 rows)

-- PURGE RECYCLEBIN --
--PURGE回收站
gaussdb=# PURGE RECYCLEBIN;
PURGE RECYCLEBIN
--查看回收站，回收站被清空
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid | rcyname | rcyoriginname | rcyoperation | rcytype | rcyrecyclecsn |
rcyrecycletime | rcycreatecsn | rcychangecsn | rcynamespace | rcyowner | rcytablespace
 | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+---------+---------------+--------------+---------+---------------+----------------
+--------------+--------------+--------------+----------+--------------
-+----------------+--------------+-------------+--------------+----------------+-----------
(0 rows)

-- TIMECAPSULE TABLE { table_name } TO BEFORE DROP [RENAME TO new_tablename] --
gaussdb=# DROP TABLE IF EXISTS flashtest;
NOTICE:  table "flashtest" does not exist, skipping
DROP TABLE
--创建表flashtest
gaussdb=# CREATE TABLE IF NOT EXISTS flashtest(id int, name text) with (storage_type = ustore);
CREATE TABLE
--插入数据
gaussdb=# INSERT INTO flashtest VALUES(1, 'A');
INSERT 0 1
gaussdb=# SELECT * FROM flashtest;
 id | name
----+------
  1 | A
(1 row)

--DROP表
gaussdb=# DROP TABLE IF EXISTS flashtest;
DROP TABLE
--查看回收站，表被放入回收站
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid |           rcyname            |     rcyoriginname      | rcyoperation | rcytype |
rcyrecyclecsn |        rcyrecycletime         | rcycreatecsn | rcychangecs
n | rcynamespace | rcyowner | rcytablespace | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid |
rcyfrozenxid64 | rcybucket
-----------+---------+----------+------------------------------+----------------------+--------------+---------
+---------------+------------------------------+--------------+------------
--+--------------+----------+---------------+----------------+---------------+-------------+--------------
+----------------+-----------
    18658 |   12737 |    18652 | BIN$31C14EB48DC$9B2B$0==$0   | flashtest            | d            |       0 |
79354760 | 2023-09-13 20:47:57.075907+08 |     79354753 |     7935475
3 |         2200 |       10 |             0 |          18652 | t            | t           | 226824       |         226824 |
    18658 |   12737 |    18655 | BIN$31C14EB48DF$12E46400==$0 | pg_toast_18652       | d            |       2
|      79354760 | 2023-09-13 20:47:57.07621+08  |             0 |
0 |          99 |       10 |             0 |          18655 | f            | f           | 226824       |         226824 |
(2 rows)

--查看表，表不存在
gaussdb=# SELECT * FROM flashtest;
ERROR:  relation "flashtest" does not exist
LINE 1: select * from flashtest;
                      ^
--闪回drop表
gaussdb=# TIMECAPSULE TABLE flashtest to before drop;
TimeCapsule Table
--查看表，表被恢复到drop之前
gaussdb=# SELECT * FROM flashtest;
 id | name
----+------
  1 | A
(1 row)

--查看回收站，回收站中的表被删除
```

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid | rcyname | rcyoriginname | rcyoperation | rcytype | rcyrecyclecsn |
rcyrecycletime | rcycreatecsn | rcychangecsn | rcynamespace | rcyowner | rcytablespace
 | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+---------+--------------+--------------+---------+--------------+----------------
+--------------+--------------+--------------+----------+--------------
-+----------------+---------------+-------------+--------------+----------------+-----------
(0 rows)

--DROP表
gaussdb=# DROP TABLE IF EXISTS flashtest;
DROP TABLE
gaussdb=# SELECT * FROM flashtest;
ERROR:  relation "flashtest" does not exist
LINE 1: select * from flashtest;
                      ^
--查看回收站，表被放入回收站
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid |           rcyname            |        rcyoriginname         | rcyoperation | rcytype |
rcyrecyclecsn |        rcyrecycletime         | rcycreatecsn | rcy
changecsn | rcynamespace | rcyowner | rcytablespace | rcyrelfilenode | rcycanrestore | rcycanpurge |
rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+------------------------------+------------------------------+--------------+---------
+---------------+--------------------------------+--------------+----
----------+--------------+----------+---------------+----------------+---------------+-------------+--------------
+----------------+-----------
    18664 |   12737 |    18652 | BIN$31C14EB48DC$9B4E$0==$0   | flashtest                    | d            |       0
|      79354845 | 2023-09-13 20:49:17.762977+08 |     79354753 |
 79354753 |         2200 |       10 |             0 |          18652 | t            | t           | 226824       |         226824 |
    18664 |   12737 |    18657 | BIN$31C14EB48E1$12E680A8==$0 | BIN$31C14EB48E1$12E45E00==$0 |
d            |       3 |      79354845 | 2023-09-13 20:49:17.763271+08 |     79354753 |
 79354753 |          99 |       10 |             0 |          18657 | f            | f           | 0            |             0 |
    18664 |   12737 |    18655 | BIN$31C14EB48DF$12E68698==$0 | BIN$31C14EB48DF$12E46400==$0 |
d            |       2 |      79354845 | 2023-09-13 20:49:17.763343+08 |            0 |
       0 |          99 |       10 |             0 |          18655 | f            | f           | 226824       |         226824 |
(3 rows)

--闪回drop表，表名用回收站中的rcyname
gaussdb=# TIMECAPSULE TABLE "BIN$31C14EB48DC$9B4E$0==$0" to before drop;
TimeCapsule Table
--查看回收站，回收站中的表被删除
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid | rcyname | rcyoriginname | rcyoperation | rcytype | rcyrecyclecsn |
rcyrecycletime | rcycreatecsn | rcychangecsn | rcynamespace | rcyowner | rcytablespace
 | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+---------+--------------+--------------+---------+--------------+----------------
+--------------+--------------+--------------+----------+--------------
-+----------------+---------------+-------------+--------------+----------------+-----------
(0 rows)

gaussdb=# SELECT * FROM flashtest;
 id | name
----+------
  1 | A
(1 row)

--DROP表
gaussdb=# DROP TABLE IF EXISTS flashtest;
DROP TABLE
--查看回收站，表被放入回收站
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid |           rcyname            |        rcyoriginname         | rcyoperation | rcytype |
rcyrecyclecsn |        rcyrecycletime         | rcycreatecsn | rcy
changecsn | rcynamespace | rcyowner | rcytablespace | rcyrelfilenode | rcycanrestore | rcycanpurge |
rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+------------------------------+------------------------------+--------------+---------
+---------------+--------------------------------+--------------+----
----------+--------------+----------+---------------+----------------+---------------+-------------+--------------
+----------------+-----------
```

 版权所有 © 华为云计算技术有限公司

```
    18667 |   12737 |    18652 | BIN$31C14EB48DC$9B8D$0==$0   | flashtest                    | d            |       0
|      79354943 | 2023-09-13 20:52:14.525946+08 |      79354753 |
 79354753 |         2200 |       10 |             0 |          18652 | t             | t           | 226824       |         226824 |
    18667 |   12737 |    18657 | BIN$31C14EB48E1$1320B4F0==$0 | BIN$31C14EB48E1$12E680A8==$0 |
d            |       3 |      79354943 | 2023-09-13 20:52:14.526319+08 |      79354753 |
 79354753 |          99 |       10 |             0 |          18657 | f             | f           | 0            |             0 |
    18667 |   12737 |    18655 | BIN$31C14EB48DF$1320BAE0==$0 | BIN$31C14EB48DF$12E68698==$0 |
d            |       2 |      79354943 | 2023-09-13 20:52:14.526423+08 |            0 |
       0 |          99 |       10 |             0 |          18655 | f             | f           | 226824       |         226824 |
(3 rows)

--查看表，表不存在
gaussdb=# SELECT * FROM flashtest;
ERROR:  relation "flashtest" does not exist
LINE 1: SELECT * FROM flashtest;
                      ^
--闪回drop表，并重命名表
gaussdb=# TIMECAPSULE TABLE flashtest to before drop rename to flashtest_rename;
TimeCapsule Table
--查看原表，表不存在
gaussdb=# SELECT * FROM flashtest;
ERROR:  relation "flashtest" does not exist
LINE 1: SELECT * FROM flashtest;
                      ^
--查看重命名后的表，表存在
gaussdb=# SELECT * FROM flashtest_rename;
 id | name
----+------
  1 | A
(1 row)

--查看回收站，回收站中的表被删除
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid | rcyname | rcyoriginname | rcyoperation | rcytype | rcyrecyclecsn |
rcyrecycletime | rcycreatecsn | rcychangecsn | rcynamespace | rcyowner | rcytablespace
 | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+---------+---------------+--------------+---------+---------------+----------------
+--------------+--------------+--------------+----------+--------------
-+----------------+---------------+-------------+--------------+----------------+-----------
(0 rows)
--drop表
gaussdb=# DROP TABLE IF EXISTS flashtest_rename;
DROP TABLE
--清空回收站
gaussdb=# PURGE RECYCLEBIN;
PURGE RECYCLEBIN
--查看回收站，回收站被清空
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid | rcyname | rcyoriginname | rcyoperation | rcytype | rcyrecyclecsn |
rcyrecycletime | rcycreatecsn | rcychangecsn | rcynamespace | rcyowner | rcytablespace
 | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+---------+---------------+--------------+---------+---------------+----------------
+--------------+--------------+--------------+----------+--------------
-+----------------+---------------+-------------+--------------+----------------+-----------
(0 rows)


-- TIMECAPSULE TABLE { table_name } TO BEFORE TRUNCATE --
gaussdb=# DROP TABLE IF EXISTS flashtest;
NOTICE:  table "flashtest" does not exist, skipping
DROP TABLE
--创建表flashtest
gaussdb=# CREATE TABLE IF NOT EXISTS flashtest(id int, name text) with (storage_type = ustore);
CREATE TABLE
--插入数据
gaussdb=# INSERT INTO flashtest VALUES(1, 'A');
INSERT 0 1
gaussdb=# SELECT * FROM flashtest;
 id | name
```

 版权所有 © 华为云计算技术有限公司

```
----+------
  1 | A
(1 row)

--truncate表
gaussdb=# TRUNCATE TABLE flashtest;
TRUNCATE TABLE
--查看回收站，表的数据被放入回收站
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid |           rcyname            |     rcyoriginname     | rcyoperation | rcytype |
rcyrecyclecsn |        rcyrecycletime         | rcycreatecsn | rcychangecs
n | rcynamespace | rcyowner | rcytablespace | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid |
rcyfrozenxid64 | rcybucket
-----------+---------+----------+------------------------------+----------------------+--------------+---------
+---------------+------------------------------+--------------+------------
--+--------------+----------+---------------+----------------+---------------+-------------+--------------
+----------------+-----------
    18703 |   12737 |    18697 | BIN$31C14EB4909$9E4C$0==$0   | flashtest            | t            |       0 |
79356608 | 2023-09-13 21:24:42.819863+08 |     79356606 |     7935660
6 |         2200 |       10 |             0 |          18697 | t             | t           | 227927       |         227927 |
    18703 |   12737 |    18700 | BIN$31C14EB490C$132FE3F0==$0 | pg_toast_18697       | t            |       2
|      79356608 | 2023-09-13 21:24:42.820358+08 |            0 |
0 |          99 |       10 |             0 |          18700 | f             | f           | 227927       |         227927 |
(2 rows)

--查看表，表中的数据为空
gaussdb=# SELECT * FROM flashtest;
 id | name
----+------
(0 rows)

--闪回truncate表
gaussdb=# TIMECAPSULE TABLE flashtest to before truncate;
TimeCapsule Table
--查看表，表中的数据被恢复
gaussdb=# SELECT * FROM flashtest;
 id | name
----+------
  1 | A
(1 row)

--查看回收站
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid |           rcyname            |     rcyoriginname     | rcyoperation | rcytype |
rcyrecyclecsn |        rcyrecycletime         | rcycreatecsn | rcychangecs
n | rcynamespace | rcyowner | rcytablespace | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid |
rcyfrozenxid64 | rcybucket
-----------+---------+----------+------------------------------+----------------------+--------------+---------
+---------------+------------------------------+--------------+------------
--+--------------+----------+---------------+----------------+---------------+-------------+--------------
+----------------+-----------
    18703 |   12737 |    18700 | BIN$31C14EB490C$13300228==$0 | pg_toast_18697       | t            |       2
|      79356610 | 2023-09-13 21:24:42.872732+08 |            0 |
0 |          99 |       10 |             0 |          18706 | f             | f           | 0            |         227928 |
    18703 |   12737 |    18697 | BIN$31C14EB4909$9E4D$0==$0   | flashtest            | t            |       0 |
79356610 | 2023-09-13 21:24:42.872792+08 |     79356606 |     7935660
6 |         2200 |       10 |             0 |          18704 | t             | t           | 0            |         227928 |
(2 rows)

--drop表
gaussdb=# DROP TABLE IF EXISTS flashtest;
DROP TABLE
--清空回收站
gaussdb=# PURGE RECYCLEBIN;
PURGE RECYCLEBIN
--查看回收站，回收站被清空
gaussdb=# SELECT * FROM gs_recyclebin;
 rcybaseid | rcydbid | rcyrelid | rcyname | rcyoriginname | rcyoperation | rcytype | rcyrecyclecsn |
rcyrecycletime | rcycreatecsn | rcychangecsn | rcynamespace | rcyowner | rcytablespace
```

 版权所有 © 华为云计算技术有限公司

```
 | rcyrelfilenode | rcycanrestore | rcycanpurge | rcyfrozenxid | rcyfrozenxid64 | rcybucket
-----------+---------+----------+---------+--------------+-------------+---------+--------------+----------------
+-------------+-------------+-------------+----------+--------------
-+----------------+--------------+-------------+-------------+----------------+-----------
(0 rows)
```

2.9.3.5 常用视图工具

视图类型	类型	功能描述	使用场景	函数名称
解析	全类型	用于解析指定表页面，并返回存放解析内容的路径。	• 查看页面信息。 • 查看元组（非用户数据）信息。 • 页面或者元组损坏。 • 元组可见性问题。 • 校验报错问题。	gs_parse_page_bypath
	索引回收队列（URQ）	用于解析UB-tree索引回收队列关键信息。	• UB-tree索引空间膨胀。 • UB-tree索引空间回收异常。 • 校验报错问题。	gs_urq_dump_stat

版权所有 © 华为云计算技术有限公司

视图类型	类型	功能描述	使用场景	函数名称
	回滚段（Undo）	用于解析指定Undo Record的内容，不包含旧版本元组的数据。	• undo空间膨胀。 • undo回收异常。 • 回滚异常。 • 日常巡检。 • 校验报错。 • 可见性判断异常。 • 修改参数。	gs_undo_dump_record
		用于解析指定事务生成的所有Undo Record，不包含旧版本元组的数据。		gs_undo_dump_xid
		用于解析指定UndoZone中所有Transaction Slot信息。		gs_undo_translot_dump_slot
		用于解析指定事务对应Transaction Slot信息，包括事务XID和该事务生成的Undo Record范围.		gs_undo_translot_dump_xid
		用于解析指定Undo Zone的元信息，显示Undo Record和Transaction Slot指针使用情况。		gs_undo_meta_dump_zone
		用于解析指定Undo Zone对应Undo Space的元信息，显示Undo Record文件使用情况。		gs_undo_meta_dump_spaces
		用于解析指定Undo Zone对应Slot Space的元信息，显示Transaction Slot文件使用情况。		gs_undo_meta_dump_slot
		用于解析数据页和数据页上数据的所有历史版本，并返回存放解析内容的路径。		gs_undo_dump_parsepage_mv
	预写日志（WAL）	用于解析指定LSN范围之内的xLog日志，并返回存放解析内容的路径。可以通过pg_current_xlog_location()获取当前xLog位置。	• WAL日志出错。 • 日志回放出错。 • 页面损坏。	gs_xlogdump_lsn
		用于解析指定XID的xLog日志，并返回存放解析内容的路径。可以通过txid_current()获取当前事务ID。		gs_xlogdump_xid
		用于解析指定表页面对应的日志，并返回存放解析内容的路径。		gs_xlogdump_tablepath

版权所有 © 华为云计算技术有限公司

视图类型	类型	功能描述	使用场景	函数名称
		用于解析指定表页面和表页面对应的日志，并返回存放解析内容的路径。可以看做一次执行gs_parse_page_bypath和gs_xlogdump_tablepath。该函数执行的前置条件是表文件存在。如果想查看已删除的表的相关日志，请直接调用gs_xlogdump_tablepath。		gs_xlogdump_parsepage_tablepath
统计	回滚段（Undo）	用于显示Undo模块的统计信息，包括Undo Zone使用情况、Undo链使用情况、Undo模块文件创建删除情况和Undo模块参数设置推荐值。	• Undo空间膨胀。 • Undo资源监控。	gs_stat_undo
	预写日志（WAL）	用于统计预写日志（WAL）写盘时的内存状态表内容。	• WAL写/刷盘监控。 • WAL写/刷盘hang住。	gs_stat_wal_entrytable
		用于统计预写日志（WAL）刷盘状态、位置统计信息。		gs_walwriter_flush_position
		用于统计预写日志（WAL）写刷盘次数频率、数据量以及刷盘文件统计信息。		gs_walwriter_flush_stat
校验	堆表/索引	用于离线校验表或者索引文件磁盘页面数据是否异常。	• 页面损坏或者元组损坏。 • 可见性问题。 • 日志回放出错问题。	ANALYZE VERIFY
		用于校验当前实例当前库物理文件是否存在丢失。	文件丢失。	gs_verify_data_file
	索引回收队列（URQ）	用于校验UB-tree索引回收队列（潜在队列/可用队列/单页面）数据是否异常。	• UB-tree索引空间膨胀。 • UB-tree索引空间回收异常。	gs_verify_urq

版权所有 © 华为云计算技术有限公司

视图类型	类型	功能描述	使用场景	函数名称
	回滚段（Undo）	用于离线校验Undo Record数据是否存在异常。	• Undo Record异常或者损坏。 • 可见性问题。 • 回滚出错或者异常。	gs_verify_undo_record
		用于离线校验Transaction Slot数据是否存在异常。	• Undo Record异常或者损坏。 • 可见性问题。 • 回滚出错或者异常。	gs_verify_undo_slot
		用于离线校验Undo元信息数据是否存在异常。	• 因Undo meta引起的节点无法启动问题。 • Undo空间回收异常。 • Snapshot too old问题。	gs_verify_undo_meta
修复	堆表/索引/Undo文件	用于基于备机修复主机丢失的物理文件。	堆表/索引/Undo文件丢失。	gs_repair_file
	堆表/索引/Undo页面	用于校验并基于备机修复主机受损页面。	堆表/索引/Undo页面损坏。	gs_verify_and_tryrepair_page
		用于基于备机页面直接修复主机页面。		gs_repair_page
		用于基于偏移量对页面的备份进行字节修改。		gs_edit_page_bypath

版权所有 © 华为云计算技术有限公司

视图类型	类型	功能描述	使用场景	函数名称
		用于将修改后的页面覆盖写入到目标页面。		gs_repair_page_bypath
	回滚段（Undo）	用于重建Undo元信息，如果校验发现Undo元信息没有问题则不重建。	Undo元信息异常或者损坏。	gs_repair_undo_byzone
	索引回收队列（URQ）	用于重建UB-tree索引回收队列。	索引回收队列异常或者损坏。	gs_repair_urq

2.9.3.6 常见问题及定位手段

2.9.3.6.1 snapshot too old

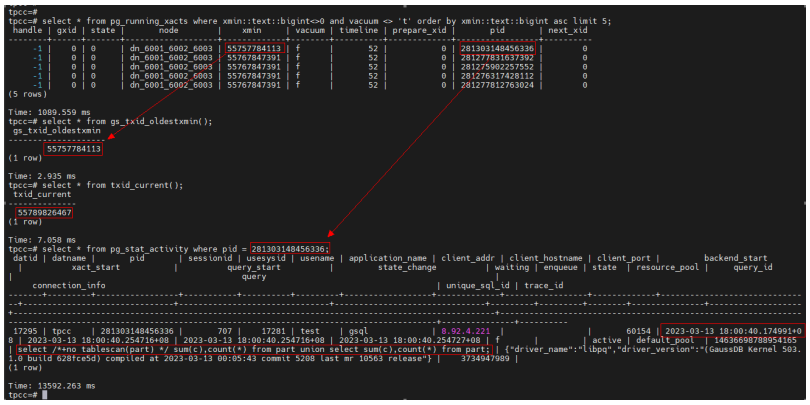
查询SQL执行时间过长或者其它一些原因，Undo无法保存太久的历史数据就可能因为历史版本被强制回收报错。一般情况下需要扩容回滚段空间，但具体问题需要具体分析。

长事务阻塞 Undo 空间回收

问题现象

1. gs_log中打印如下错误：
   ```
   snapshot too old! the undo record has been forcibly discarded
   xid xxx, the undo size xxx of the transaction exceeds the threshold xxx. trans_undo_threshold_size xxx,undo_space_limit_size xxx.
   ```
   在真实报错信息中，上文中的xxx为实际数据。
2. global_recycle_xid（Undo子系统的全局回收事务XID）长时间不发生变化。
   ```
   gaussdb=# select * from gs_undo_meta_dump_slot(1,-1);
    zone_id | allocate | recycle | frozen_xid | global_frozen_xid | recycle_xid | global_recycle_xid
   ---------+----------+---------+------------+-------------------+-------------+--------------------
          1 | 280      | 248     | 17028      | 17028             | 17025       | 17028
   (1 row)
   ```
3. pg_running_xacts与pg_stat_activity视图查询存在长事务，阻塞oldestxmin和global_recycle_xid推进。如果pg_running_xacts中查询活跃事务的xmin和gs_txid_oldestxmin相等，且通过pid查询pg_stat_activity查询线程执行语句时间过长，则表明有长事务卡主了回收。
   ```
   SELECT * FROM pg_running_xacts WHERE xmin::text::bigint<>0 AND vacuum <> 't' ORDER BY xmin::text::bigint ASC LIMIT 5;
   SELECT * FROM gs_txid_oldestxmin();
   SELECT * FROM pg_stat_activity where pid = 长事务所在线程PID;
   ```


版权所有 © 华为云计算技术有限公司



处理方法

通过pg_terminate_session(pid, sessionid)终止长事务所在的会话（提醒：长事务无固定快速恢复手段，强制结束SQL语句为其中一种常用操作，属于高危操作，执行需谨慎，执行前需与业务及华为技术确认，避免造成业务失败或报错）。

大量回滚事务拖慢 Undo 空间回收

问题现象

使用gs_async_rollback_xact_status视图查看有大量的待回滚事务，且待回滚的事务数量维持不变或者持续增高。

```
SELECT * FROM gs_async_rollback_xact_status();
```

处理方法

调大异步回滚线程数量，调整方式有以下两种：

方式1：在gaussdb.conf中配置max_undo_workers，然后重启节点。

方式2：gs_guc reload -Z NODE-TYPE [-N NODE-NAME] [-I INSTANCE-NAME | -D DATADIR] -c max_undo_workers=100 重启实例。

2.9.3.6.2 storage test error

业务执行过程中，数据页、索引或者Undo页面发生变更后，该页面放锁之前会主动进行逻辑损坏检测，发现页面损坏问题后会输出包含“storage test error”关键字的日志信息到数据库运行日志（gs_log文件），执行事务回滚，页面会恢复到修改前的状态。

问题现象

gs_log中打印“storage test error”关键字。

处理方法

请联系华为技术支持解决。

版权所有 © 华为云计算技术有限公司

2.9.3.6.3 备机读业务报错:"UBTreeSearch::read_page has conflict with recovery, please try again later"

问题现象

业务在使用备机读时，出现报错（错误码43244），错误信息中包含“UBTreeSearch::read_page has conflict with recovery, please try again later”关键字。

问题分析

在开启并行回放或串行回放的情况下（查询GUC参数recovery_parse_workers和recovery_max_workers均是1为串行回放；recovery_parse_workers是1，recovery_max_workers大于1为并行回放），备机的查询线程在做索引扫描时，会先对索引页面加读锁，每当扫到一个元组时会去判可见性，如果该元组对应的事务处于committing状态，需要等待该事务提交后再判断。而备机上的事务提交是依赖日志回放线程推进的，这个过程中会对索引页面进行修改，因此需要加锁。查询线程在等待过程中会释放索引页面的锁，否则会出现查询线程等待回放线程进行事务提交，而回放线程在等待查询线程释放锁。

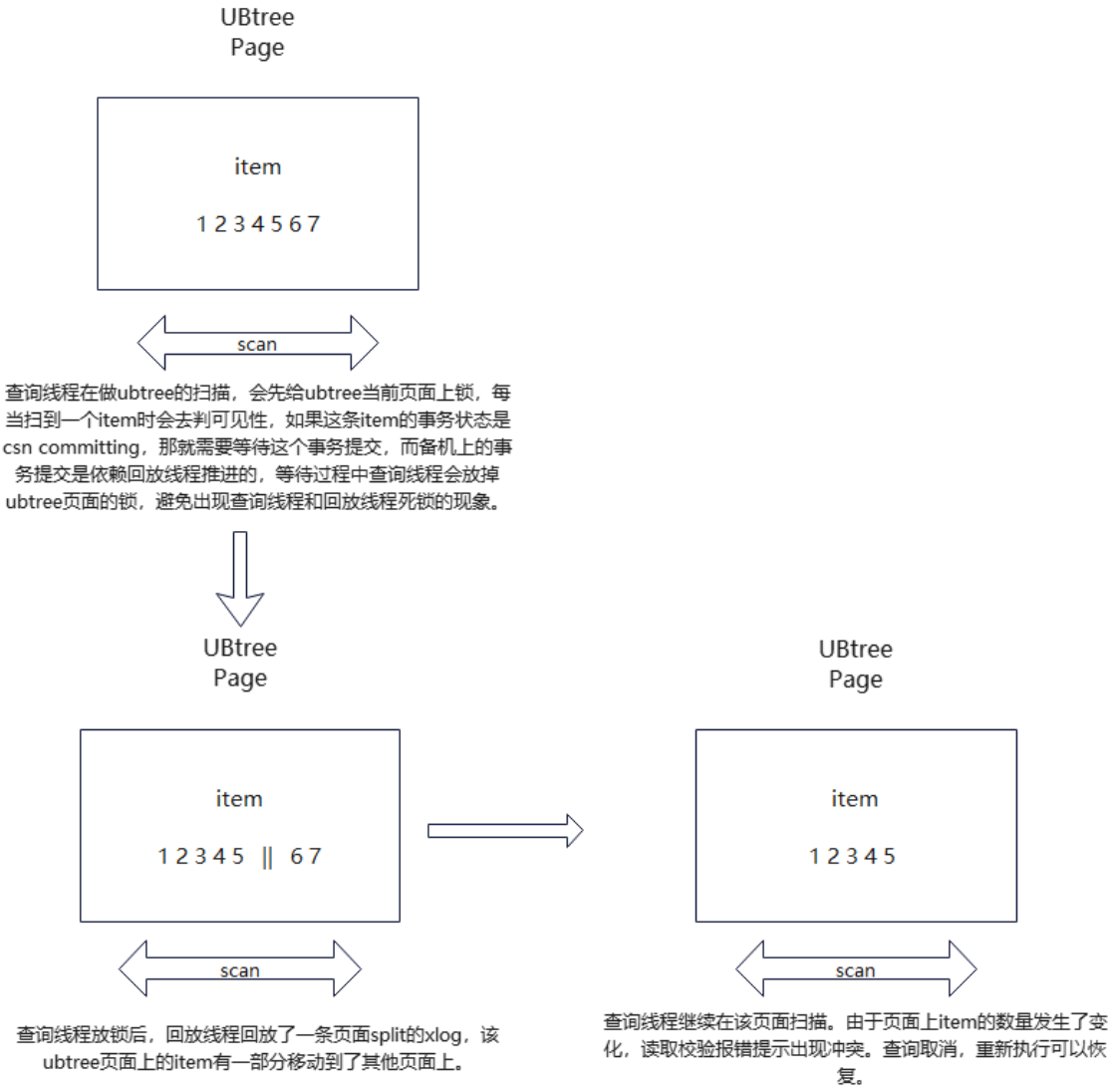
该报错仅出现在查询与回放都需要访问同一个索引页面的场景下，查询线程在释放锁并等待事务结束过程中，访问的页面出现被修改的情况。具体流程图如下图1所示：

说明

- 备机查询在扫到committing状态的元组时，需要等待事务提交是因为事务提交的顺序与产生日志的顺序可能是乱序的，例如主机上tx_1的事务比tx_2先提交，而备机上tx_1的commit日志在tx_2的commit日志之后回放，按照事务提交顺序来看tx_1对tx_2应当是可见的，所以需要等待事务提交。
- 备机查询在扫描索引页面时，发现页面元组数量（包含死元组）发生变化后不可重试，是因为在扫描时可能为正向或反向扫描，而举例来说页面发生分裂后一部分元组移动到右页面，在反向扫描的情况下即使重试只能向左扫描读取，无法再保证结果的正确性，并且由于无法分辨发生分裂或者插入，所以不可重试。

版权所有 © 华为云计算技术有限公司

图 2-8 问题分析



处理方法

出现报错时，建议重试查询。另外建议选择非频繁更新的索引字段、采用软删除的方式（物理删除操作在业务低谷期执行），可以降低出现该报错的概率。

2.9.4 数据生命周期管理-OLTP 表压缩

2.9.4.1 特性简介

OLTP表压缩是GaussDB高级压缩中的一个特性，基于全新的压缩算法、细粒度的自动冷热判定和支持块内压缩等技术创新，可以在提供合理压缩率的同时大幅度降低对业务的影响、增加后台调度、增加查询Job执行状态以及节约空间，能够在支持关键在线业务的容量控制中发挥重要价值。

版权所有 © 华为云计算技术有限公司

2.9.4.2 特性约束

- 不支持系统表、内存表、全局临时表、本地临时表和序列表。
- 支持用户为普通表、分区、二级分区设置ILM策略。
- 支持普通表、分区、二级分区的Astore/Ustore用户表和透明加密表。
- 特性仅在A兼容模式与PG模式下有效。
- Ustore不支持编解码，压缩率小于2:1。

2.9.4.3 特性规格

- TPCC只开启策略、不开调度对原有业务无影响。
- TPCC不开启压缩策略对原有业务无影响。
- TPCC.bmsql_order_line设置ILM策略（只识别完成派送的订单为冷行）不调度，TPmC劣化不高于2%（56核CPU370GB内存+3TB SSD硬盘，350GB SharedBuffer）。
- TPCC.bmsql_order_line设置ILM策略（只识别完成派送的订单为冷行）后台默认参数调度时，TPmC劣化不高于5%（56核CPU370GB内存+3TB SSD硬盘，350GB SharedBuffer）。
- 单线程ILM Job带宽约100MB/秒（56核CPU370GB内存+3TB SSD硬盘，350GB SharedBuffer）。

  度量方式：根据执行压缩的开始时间和结束时间以及压缩的页面个数计算带宽。
- get查询访问压缩数据比非压缩数据性能劣化，驱动侧不高于10%，plsql侧不高于15%（32MB SharedBuffer，6万页面数据）。
- multi-get查询访问压缩数据比非压缩数据性能劣化，驱动侧不高于30%，plsql侧不高于40%（32MB SharedBuffer，6万页面数据）。
- table-scan查询访问压缩数据比非压缩数据性能劣化，驱动侧不高于30%，plsql侧不高于40%（32MB SharedBuffer，6万页面数据）。
- TPCH.lineitem表压缩比（全冷行）不小于2:1。
- 对于TPC-C的Orderline表，以及TPC-H的Lineitem、Orders、Customer、Part表的测试表明，数值型字段较多时，压缩率高于LZ4和ZLIB；而文本型字段较多时，压缩率介于LZ类和LZ+Huffman组合类的压缩算法之间。

2.9.4.4 使用说明

使用高级压缩的功能，用户必须购买License才能使用。具体情况请联系华为工程师。

步骤1 执行如下命令开启压缩功能：

```
gaussdb=# ALTER DATABASE SET ilm = on;
```

检查当前数据库的public schema中是否存在gsilmpolicy_seq和gsilmtask_seq。

```
gaussdb=# \d
                                List of relations
 Schema |      Name       |   Type   |   Owner    |                  Storage
--------+-----------------+----------+------------+-------------------------------------------------------------
 public | gsilmpolicy_seq | sequence | omm |
|
 public | gsilmtask_seq   | sequence | omm |
|
```

或者：

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# SELECT a.oid, a.relname from pg_class a inner join pg_namespace b on a.relnamespace = b.oid
    WHERE (a.relname = 'gsilmpolicy_seq' OR a.relname = 'gsilmtask_seq') AND b.nspname = 'public';

 oid |     relname
-------+-----------------
 17002 | gsilmpolicy_seq
 17004 | gsilmtask_seq
(2 rows)
```

生成异常会报warning：

```
WARNING:  ILM sequences are already existed while initializing
```

步骤2 为表添加压缩策略。

- 新建带策略的表：
  ```
  gaussdb=# CREATE TABLE ilm_table_1 (col1 int, col2 text)
     ilm add policy row store compress advanced row
     after 3 days of no modification on (col1 < 1000);
  ```
- 为存量表添加策略：
  ```
  gaussdb=# CREATE TABLE ilm_table_2 (col1 int, col2 text);
  gaussdb=# ALTER TABLE ilm_table_2 ilm add policy row store
     compress advanced row after 3 days of no modification;
  ```
- 检查策略视图中是否新增数据：
  ```
  gaussdb=# SELECT * FROM gs_my_ilmpolicies;

   policy_name |  policy_type  | tablespace | enabled | deleted
  -------------+---------------+------------+---------+---------
   p1          | DATA MOVEMENT |            | YES     | NO
   p2          | DATA MOVEMENT |            | YES     | NO
  (2 rows)
  ```
- 检查策略详细信息视图中是否新增了符合刚刚设置的策略：
  ```
  gaussdb=# SELECT * FROM gs_my_ilmdatamovementpolicies;

   policy_name | action_type | scope | compression_level | tier_tablespace | tier_status |
  condition_type     | condition_days | custom_function | policy_subtype | action_clause | tier_to
  -------------+-------------+-------+-------------------+-----------------+-------------+-------------------------
  +----------------+-----------------+---------------+---------------+---------
   p1          | COMPRESSION | ROW   | ADVANCED          |                 |             | LAST MODIFICATION
  TIME |              3 |                 |               |               |
   p2          | COMPRESSION | ROW   | ADVANCED          |                 |             | LAST MODIFICATION
  TIME |              3 |                 |               |               |
  (2 rows)
  ```
- 检查策略与目标表是否对应：
  ```
  gaussdb=# SELECT * FROM gs_my_ilmobjects;
   policy_name | object_owner | object_name | subobject_name | object_type |    inherited_from    |
  tbs_inherited_from | enabled | deleted
  -------------+--------------+-------------+----------------+-------------+----------------------
  +--------------------+---------+---------
   p1          | public       | ilm_table_1 |                | TABLE       | POLICY NOT INHERITED |                    |
  YES     | NO
   p2          | public       | ilm_table_2 |                | TABLE       | POLICY NOT INHERITED |                    |
  YES     | NO
  (2 rows)
  ```

步骤3 执行压缩评估。

- 手动执行压缩评估。

> **⚠ 注意**
>
> 为方便测试，本功能环境参数中提供POLICY_TIME属性，决定时间条件以天为单位还是以秒为单位。通过下面语句调整：

版权所有 © 华为云计算技术有限公司

```
gaussdb=# CALL DBE_ILM_ADMIN.CUSTOMIZE_ILM(11, 1);
```

插入随机数据用于测试：

```
gaussdb=# INSERT INTO ilm_table_1 select *, 'test_data' FROM generate_series(1, 10000);

gaussdb=# DECLARE
    v_taskid number;
gaussdb=# BEGIN
    DBE_ILM.EXECUTE_ILM(OWNER           => 'public',
                        OBJECT_NAME     => 'ilm_table_1',
                        TASK_ID         => v_taskid,
                        SUBOBJECT_NAME  => NULL,
                        POLICY_NAME     => 'ALL POLICIES',
                        EXECUTION_MODE  => 2);
    RAISE INFO 'Task ID is:%', v_taskid;
gaussdb=# END;
/
```

如入参有误，会报对应的错误信息。无误则无输出（上述代码段添加了RAISE INFO语句打印当前task的id）。

```
INFO:  Task ID is:1
```

检查task信息：

```
gaussdb=# SELECT * FROM gs_my_ilmtasks;

task_id | task_owner |   state   |         creation_time         |          start_time           |
completion_time
---------+------------+-----------+-------------------------------+-------------------------------
+-------------------------------
       1 | omm | COMPLETED | 2023-08-29 17:36:38.779555+08 | 2023-08-29 17:36:38.779555+08 |
2023-08-29 17:36:38.879485+08
(1 row)
```

检查评估结果：

```
gaussdb=# SELECT * FROM gs_my_ilmevaluationdetails;

 task_id | policy_name | object_owner | object_name | subobject_name | object_type |
selected_for_execution |      job_name      | comments
---------+-------------+--------------+-------------+----------------+-------------+------------------------
+--------------------+----------
       1 | p1          | public       | ilm_table_1 |                | TABLE       | SELECTED FOR EXECUTION | ilmjob
$_postgres1 |
(1 row)
```

检查压缩job信息：

```
gaussdb=# SELECT * FROM gs_my_ilmresults;

 task_id |     job_name      |       job_state        |          start_time           |       completion_time         |
comments |               statistics
---------+-------------------+------------------------+-------------------------------
+-------------------------------+----------+-----------------------------------------
       1 | ilmjob$_postgres1 | COMPLETED SUCCESSFULLY | 2023-08-29 17:36:38.779555+08 |
2023-08-29 17:36:38.879485+08 |          | SpaceSaving=0,BoundTime=0,LastBlkNum=0
(1 row)
```

- 触发后台自动调度评估。

  使用初始用户登录template1数据库，创建维护窗口：

```
gaussdb=#
DECLARE
    V_HOUR INT := 22;
    V_MINUTE INT := 0;
    V_SECOND INT := 0;
    C_ADO_WINDOW_SCHEDULE_NAME TEXT := 'ado_window_schedule';
    C_ADO_WINDOW_PROGRAM_NAME TEXT := 'ado_window_program';
    C_MAINTENANCE_WINDOW_JOB_NAME TEXT := 'maintenance_window_job';
    V_MAINTENCE_WINDOW_REPEAT TEXT;
    V_MAINTENCE_WINDOW_START TIMESTAMPTZ;
    V_BE_SCHEDULE_ENABLE BOOL;
    V_MAINTENANCE_WINDOW_EXIST INT;
BEGIN
```

 版权所有 © 华为云计算技术有限公司

```
    SELECT COUNT(*) INTO V_MAINTENANCE_WINDOW_EXIST FROM PG_CATALOG.PG_JOB WHERE
JOB_NAME = 'maintenance_window_job' AND DBNAME = 'template1';
    IF CURRENT_DATABASE() != 'template1' THEN
        RAISE EXCEPTION 'Create maintenance_window FAILED, current database is not tempalte1';
    END IF;
    IF V_MAINTENANCE_WINDOW_EXIST = 0 AND CURRENT_DATABASE() = 'template1' THEN
        SELECT
            CASE
                WHEN NOW() < CURRENT_DATE + INTERVAL '22 HOUR' THEN CURRENT_DATE +
INTERVAL '22 HOUR'
                ELSE CURRENT_DATE + INTERVAL '1 DAY 22 HOUR'
            END INTO V_MAINTENCE_WINDOW_START;
        --1. prepare for maintence window schedule
        SELECT 'freq=daily;interval=1;byhour='||V_HOUR||';byminute='||V_MINUTE||';bysecond='||
V_SECOND INTO V_MAINTENCE_WINDOW_REPEAT;
        BEGIN
            SELECT
                CASE
                    WHEN VALUE = 1 THEN TRUE -- DBE_ILM_ADMIN.ILM_ENABLED
                    ELSE FALSE
                END INTO V_BE_SCHEDULE_ENABLE
                FROM PG_CATALOG.GS_ILM_PARAM WHERE IDX = 7; -- DBE_ILM_ADMIN.ENABLED
        EXCEPTION
            WHEN OTHERS THEN
                V_BE_SCHEDULE_ENABLE := FALSE;
        END;
        --2. Create ado window schedule
        DBE_SCHEDULER.CREATE_SCHEDULE(
            SCHEDULE_NAME => C_ADO_WINDOW_SCHEDULE_NAME,
            START_DATE => '9999-01-01 00:00:01',
            REPEAT_INTERVAL => NULL,
            END_DATE => NULL,
            COMMENTS => 'ado window schedule');
        --3. Create ado window program
        DBE_SCHEDULER.CREATE_PROGRAM(
            PROGRAM_NAME => C_ADO_WINDOW_PROGRAM_NAME,
            PROGRAM_TYPE => 'plsql_block',
            PROGRAM_ACTION => 'call prvt_ilm.be_execute_ilm();',
            NUMBER_OF_ARGUMENTS => 0,
            ENABLED => TRUE,
            COMMENTS => NULL);
        --4. Create maintenance window master job
        DBE_SCHEDULER.CREATE_JOB(
            JOB_NAME => C_MAINTENANCE_WINDOW_JOB_NAME,
            START_DATE => V_MAINTENCE_WINDOW_START,
            REPEAT_INTERVAL => V_MAINTENCE_WINDOW_REPEAT,
            END_DATE => NULL,
            JOB_TYPE => 'STORED_PROCEDURE'::TEXT,
            JOB_ACTION => 'prvt_ilm.be_active_ado_window'::TEXT,
            NUMBER_OF_ARGUMENTS => 0,
            ENABLED => V_BE_SCHEDULE_ENABLE,
            AUTO_DROP => FALSE,
            COMMENTS => 'maintenance window job');
    END IF;
END;
```

自动调度提供若干参数用于调整：

```
gaussdb=# SELECT * FROM gs_adm_ilmparameters;
        name         | value
---------------------+-------
 EXECUTION_INTERVAL |    15
 RETENTION_TIME     |    30
 ENABLED            |     1
 POLICY_TIME        |     0
 ABS_JOBLIMIT       |    10
 JOB_SIZELIMIT      |  1024
 WIND_DURATION      |   240
 BLOCK_LIMITS       |    40
(8 rows)
```

 版权所有 © 华为云计算技术有限公司

- EXECUTION_INTERVAL：自动调度任务执行间隔，默认每15分钟执行一次。
- RETENTION_TIME：历史压缩任务记录清理间隔，默认每30天清理一次。
- ENABLED：当前自动调度启用情况，默认为开启。
- POLICY_TIME：策略评估的时间单位，测试使用。默认以天为单位。
- ABS_JOBLIMIT：单次评估生成压缩任务数量上限，默认为10个。
- JOB_SIZELIMIT：单个压缩任务的IO上限，默认为1GB。
- WIND_DURATION：单次维护窗口的持续时间。
- BLOCK_LIMITS：控制实例级的行存压缩速率上限，默认是40，取值范围是0到10000（0表示不限制）。单位是block/ms，表示每毫秒最多压缩多少个block。速率上限计算方法：BLOCK_LIMITS*1000*BLOCKSIZE，以默认值40为例，其速率上限为：40*1000*8KB=320000KB/s。

以上参数均可通过DBE_ILM_ADMIN.CUSTOMIZE_ILM()接口调整。

维护窗口默认为每天22：00（北京时间）开启，可通过DBE_SCHEDULER提供的接口SET_ATTRIBUTE进行设置：

```
\c template1
 CALL DBE_ILM_ADMIN.DISABLE_ILM();
 CALL DBE_ILM_ADMIN.ENABLE_ILM();
DECLARE
    newtime timestamptz := CLOCK_TIMESTAMP() + to_interval('2 seconds');
BEGIN
    DBE_SCHEDULER.set_attribute(
        name          =>          'maintenance_window_job',
        attribute     =>          'start_date',
        value         =>          TO_CHAR(newtime, 'YYYY-MM-DD HH24:MI:SS')
    );
END;
/
```

----结束

2.9.4.5 维护窗口参数配置

- RETENTION_TIME：评估与压缩记录的保留时长，单位天，默认值30。用户可根据自己存储容量自行调节。
- EXECUTION_INTERVAL：评估任务的执行频率，单位分钟，默认值15。用户可根据自己维护窗口期间业务与资源情况调节。该参数与ABS_JOBLIMIT相互影响。单日单线程最大可产生的I/O为WIND_DURATION/EXECUTION_INTERVAL*JOB_SIZELIMIT。
- JOB_SIZELIMIT：控制单个压缩Job可以处理的最大字节数，单位兆，默认值1024。压缩带宽约为100MB/秒，每个压缩Job限制IO为1GB时，最多10秒完成。用户可根据自己业务闲时情况以及需要压缩的数据量自行调节。
- ABS_JOBLIMIT：控制一次评估最多生成多少个压缩Job。用户可根据自己设置策略的分区及表数量自己调节。建议最大不超过10，可以使用“select count(*) from gs_adm_ilmobjects where enabled = true”命令查询。
- POLICY_TIME：控制判定冷行的条件单位是天还是秒，秒仅用来做测试用。取值为：ILM_POLICY_IN_SECONDS或ILM_POLICY_IN_DAYS（默认值）。
- WIND_DURATION：维护窗口持续时长，单位分钟，默认240分钟（4小时）。维护窗口默认从22：00（北京时间）开始持续240分钟，用户可根据自己业务闲时情况自行调节。
- BLOCK_LIMITS：控制实例级的行存压缩速率上限，默认是40，取值范围是0到10000(0表示不限制)，单位是block/ms，表示每毫秒最多压缩多少个block。速率

 版权所有 © 华为云计算技术有限公司

上限计算方法：BLOCK_LIMITS*1000*BLOCKSIZE，以默认值40为例，其速率上限为:40*1000*8KB=320000KB/s。

示例分析：

```
EXECUTION_INTERVAL：15
JOB_SIZELIMIT：10240
WIND_DURATION：240
BLOCK_LIMITS：0
```

此配置下单表分区或子分区在一个维护窗口期间可完成240/15*10240MB=160GB数据的评估压缩。压缩带宽为100MB/秒，实际压缩仅耗时160GB/(100MB/秒)=27分钟。其他时间对业务无影响。用户可根据自己业务闲时可支配给压缩的时长来调节参数。

2.9.4.6 运维 TIPS

须知

高级压缩特性使用前必须先联系工作人员购买License，否则执行相关命令会报错。

1. 手动触发一次压缩（示例中一次压缩102400MB）。
   a. 给表加上冷热分离策略：

   ```
   gaussdb=#  DROP TABLE IF EXISTS ILM_TABLE;
   gaussdb=#  CREATE TABLE ILM_TABLE(a int);
   gaussdb=# ALTER TABLE ILM_TABLE ILM ADD POLICY ROW STORE COMPRESS ADVANCED
      ROW AFTER 3 MONTHS OF NO MODIFICATION;
   ```

   b. 手动触发压缩：

   ```
   DECLARE
    v_taskid number;
   BEGIN
    DBE_ILM_ADMIN.CUSTOMIZE_ILM(11, 1);
   DBE_ILM_ADMIN.CUSTOMIZE_ILM(13, 102400);
    DBE_ILM.EXECUTE_ILM(OWNER         => '$schema_name',
                        OBJECT_NAME    => 'ilm_table',
                        TASK_ID        => v_taskid,
                        SUBOBJECT_NAME => NULL,
                        POLICY_NAME    => 'ALL POLICIES',
                        EXECUTION_MODE => 2);
    RAISE INFO 'Task ID is:%', v_taskid;
   END;
   /
   ```

   c. 查看压缩JOB是否完成，可以看到具体的执行信息：

   ```
   gaussdb=# SELECT * FROM gs_adm_ilmresults ORDER BY task_id desc;

    task_id |                job_name                |     start_time      |   completion_time
   |                    statistics
   ---------+----------------------------------------+---------------------+---------------------
   +-----------------------------------------------------
      17267 | ilmjob$_2 | 2023-03-29 08:11:25 | 2023-03-29 08:11:25 |
   SpaceSaving=453048,BoundTime=1680145883,LastBlkNum=128
   ```

2. 手动停止压缩：

   ```
   gaussdb=# DBE_ILM.STOP_ILM (task_id => V_TASK, p_drop_running_Jobs => FALSE, p_Jobname =>
   V_JOBNAME);
   ```

表 2-10 DBE_ILM.STOP_ILM 输入参数

名称	描述
task_id	指定待停止ADO task的描述符ID。

版权所有 © 华为云计算技术有限公司

名称	描述
p_drop_running_Jobs	标识是否停止正在运行的JOB。
p_Jobname	标识待停止的特定JobName，通过GS_MY_ILMEVALUATIONDETAILS视图可以查询。

3. 为表生成策略及后台调度压缩任务。

   a. 给表加上冷热分离策略：

   ```
   gaussdb=# DROP TABLE IF EXISTS ILM_TABLE;
   gaussdb=# CREATE TABLE ILM_TABLE(a int);
   gaussdb=# ALTER TABLE ILM_TABLE ILM ADD POLICY ROW STORE COMPRESS ADVANCED
      ROW AFTER 3 MONTHS OF NO MODIFICATION;
   ```

   b. 设置ILM执行相关参数：

   ```
   BEGIN
     DBE_ILM_ADMIN.CUSTOMIZE_ILM(11, 1);
     DBE_ILM_ADMIN.CUSTOMIZE_ILM(12, 10);
     DBE_ILM_ADMIN.CUSTOMIZE_ILM(1, 1);
     DBE_ILM_ADMIN.CUSTOMIZE_ILM(13, 512);
   END;
   /
   ```

   c. 开启后台的定时调度：

   ```
   gaussdb=# CALL DBE_ILM_ADMIN.DISABLE_ILM();
   gaussdb=# CALL DBE_ILM_ADMIN.ENABLE_ILM();
   ```

   d. 用户可以根据需要，调用DBE_SCHEDULER.set_attribute设置后台维护窗口的开启时间。当前默认22:00开启。

4. 设置ILM执行相关参数。

   控制ADO的条件单位是天还是秒，秒仅用来做测试用。取值为ILM_POLICY_IN_SECONDS = 1或ILM_POLICY_IN_DAYS = 0（默认值）：

   ```
   gaussdb=# CALL DBE_ILM_ADMIN.CUSTOMIZE_ILM(11， 1);
   ```

   控制一次ADO Task最多生成多少个ADO Job。取值范围大于等于0小于等于2147483647的整数或浮点数，作用时向下取整：

   ```
   gaussdb=# CALL DBE_ILM_ADMIN.CUSTOMIZE_ILM(12, 10);
   ```

   ADO Task的执行频率，单位分钟，默认值15。取值范围大于等于1小于等于2147483647的整数或浮点数，作用时向下取整：

   ```
   gaussdb=# CALL DBE_ILM_ADMIN.CUSTOMIZE_ILM(1, 1);
   ```

   控制单个ADO Job可以处理的最大字节数，单位兆。取值范围大于等于0小于等于2147483647的整数或浮点数，作用时向下取整：

   ```
   gaussdb=# CALL DBE_ILM_ADMIN.CUSTOMIZE_ILM(13, 512);
   ```

5. 评估一张表是否适合压缩及评估压缩后带来多少收益。

   ```
   gaussdb=# DBE_COMPRESSION.GET_COMPRESSION_RATIO (
     scratchtbsname        IN     VARCHAR2,
     ownname               IN     VARCHAR2,
     objname               IN     VARCHAR2,
     subobjname            IN     VARCHAR2,
     comptype              IN     NUMBER,
     blkcnt_cmp            OUT    PLS_INTEGER,
     blkcnt_uncmp          OUT    PLS_INTEGER,
     row_cmp               OUT    PLS_INTEGER,
     row_uncmp             OUT    PLS_INTEGER,
     cmp_ratio             OUT    NUMBER,
     comptype_str          OUT    VARCHAR2,
     sample_ratio    IN  INTEGER DEFAULT 20,
     objtype               IN     PLS_INTEGER DEFAULT OBJTYPE_TABLE);
   ```

 版权所有 © 华为云计算技术有限公司

表 2-11 DBE_COMPRESSION.GET_COMPRESSION_RATIO 输入参数

名称	描述
scratchtbsname	数据所在空间名称。
ownname	数据对象的拥有者名称。
objname	数据对象名称。
subobjname	数据的分区名称，默认为NULL。
comptype	压缩类型：COMP_NOCOMPRESS和COMP_ADVANCED。
sample_ratio	采样比例，输入为0-100的整数或浮点数，对应为百分之N的采样比例。默认为20，即对20%的行数进行采样。
objtype	对象类型，本期支持的类型为`OBJTYPE_TABLE`。

表 2-12 DBE_COMPRESSION.GET_COMPRESSION_RATIO 输出参数

名称	描述
blkcnt_cmp	样本被压缩后占用的块数。
blkcnt_uncmp	样本未压缩占用的块数。
row_cmp	样本被压缩后单个块内可容纳的行数。
row_uncmp	样本未被压缩时单个数据块可容纳的行数。
cmp_ratio	压缩比，blkcnt_uncmp除以blkcnt_cmp。
comptype_str	描述压缩类型的字符串。

示例：

```
gaussdb=# ALTER DATABASE set ilm = on;
gaussdb=# CREATE user user1 IDENTIFIED BY '********';
gaussdb=# CREATE user user2 IDENTIFIED BY '********';
gaussdb=# SET ROLE user1 PASSWORD '********';
gaussdb=# CREATE TABLE TEST_DATA (ORDER_ID INT, GOODS_NAME TEXT, CREATE_TIME
TIMESTAMP)
  ILM ADD POLICY ROW STORE COMPRESS ADVANCED ROW AFTER 1 DAYS OF NO MODIFICATION;
INSERT INTO TEST_DATA VALUES (1, '零食大礼包A', NOW());

DECLARE
o_blkcnt_cmp       integer;
o_blkcnt_uncmp     integer;
o_row_cmp          integer;
o_row_uncmp        integer;
o_cmp_ratio        number;
o_comptype_str     varchar2;
begin
dbe_compression.get_compression_ratio(
    SCRATCHTBSNAME  =>  NULL,
    OWNNAME         =>  'user1',
```


版权所有 © 华为云计算技术有限公司

```
    OBJNAME          =>  'test_data',
    SUBOBJNAME       =>  NULL,
    COMPTYPE         =>  2,
    BLKCNT_CMP       =>  o_blkcnt_cmp,
    BLKCNT_UNCMP     =>  o_blkcnt_uncmp,
    ROW_CMP          =>  o_row_cmp,
    ROW_UNCMP        =>  o_row_uncmp,
    CMP_RATIO        =>  o_cmp_ratio,
    COMPTYPE_STR     =>  o_comptype_str,
    SAMPLE_RATIO     =>  100,
    OBJTYPE          =>  1 ) ;
RAISE INFO 'Number of blocks used by the compressed sample of the object         : %', o_blkcnt_cmp;
RAISE INFO 'Number of blocks used by the uncompressed sample of the object         : %',
o_blkcnt_uncmp;
RAISE INFO 'Number of rows in a block in compressed sample of the object         : %', o_row_cmp;
RAISE INFO 'Number of rows in a block in uncompressed sample of the object         : %', o_row_uncmp;
RAISE INFO 'Estimated Compression Ratio of Sample                                 : %', o_cmp_ratio;
RAISE INFO 'Compression Type                                                      : %', o_comptype_str;
end;
/
INFO:  Number of blocks used by the compressed sample of the object         : 0
INFO:  Number of blocks used by the uncompressed sample of the object         : 0
INFO:  Number of rows in a block in compressed sample of the object         : 0
INFO:  Number of rows in a block in uncompressed sample of the object         : 0
INFO:  Estimated Compression Ratio of Sample                                  : 1
INFO:  Compression Type                                                       : Compress Advanced
```

6. 查询每一行的最后修改时间。

```
gaussdb=# DBE_HEAT_MAP.ROW_HEAT_MAP(
 owner     IN VARCHAR2,
 segment_name  IN VARCHAR2,
 partition_name  IN VARCHAR2 DEFAULT NULL,
ctid     IN  VARCHAR2,);
```

表 2-13 DBE_HEAT_MAP.ROW_HEAT_MAP 输入参数

名称	描述
owner	数据对象的所有者。
segment_name	数据对象名称。
partition_name	数据对象分区名称，可选参数。
ctid	目标行的ctid，即block_id或row_id。

表 2-14 DBE_HEAT_MAP.ROW_HEAT_MAP 输出参数

名称	描述
owner	数据对象的所有者。
segment_name	数据对象名称。
partition_name	数据对象分区名称，可选参数。
tablespace_name	数据所属的表空间名称。
file_id	行所属的绝对文件ID。
relative_fno	行所属的相对文件ID（GaussDB中无此逻辑，因此取值同上）。

 版权所有 © 华为云计算技术有限公司

名称	描述
ctid	行的ctid，即block_id或row_id。
writetime	行的最后修改时间。

示例：

```
gaussdb=# ALTER DATABASE set ilm = on;
gaussdb=# CREATE Schema HEAT_MAP_DATA;
gaussdb=# SET current_schema=HEAT_MAP_DATA;

gaussdb=# CREATE TABLESPACE example1 RELATIVE LOCATION 'tablespace1';
gaussdb=# CREATE TABLE HEAT_MAP_DATA.heat_map_table(id INT, value TEXT) TABLESPACE
example1;
gaussdb=# INSERT INTO HEAT_MAP_DATA.heat_map_table VALUES (1, 'test_data_row_1');

gaussdb=# SELECT * from DBE_HEAT_MAP.ROW_HEAT_MAP(
    owner          =>  'heat_map_data',
    segment_name  =>  'heat_map_table',
    partition_name  => NULL,
    ctid           =>  '(0,1)');
     owner     |  segment_name  | partition_name | tablespace_name | file_id | relative_fno | ctid  |
writetime
---------------+----------------+----------------+-----------------+---------+--------------+-------+-----------
 heat_map_data | heat_map_table |                | example1        |   17291 |        17291 | (0,1) |
(1 row)
```

7. 查询ILM调度与执行的相关环境参数。

```
gaussdb=# SELECT * FROM GS_ADM_ILMPARAMETERS;
         name         | value
----------------------+-------
 EXECUTION_INTERVAL |    15
 RETENTION_TIME     |    30
 ENABLED            |     1
 POLICY_TIME        |     0
 ABS_JOBLIMIT       |    10
 JOB_SIZELIMIT      |  1024
 WIND_DURATION      |   240
 BLOCK_LIMITS       |    40
 (8 rows)
```

8. 查询ILM策略的概要信息，包含策略名称、类型、启用禁用状态、删除状态。

```
gaussdb=# SELECT * FROM GS_ADM_ILMPOLICIES;
policy_name |  policy_type  | tablespace | enabled | deleted
-------------+---------------+------------+---------+---------
 p1          | DATA MOVEMENT |            | YES     | NO


gaussdb=# SELECT * FROM GS_MY_ILMPOLICIES;
 policy_name |  policy_type  | tablespace | enabled | deleted
-------------+---------------+------------+---------+---------
 p1          | DATA MOVEMENT |            | YES     | NO
```

9. 查询ILM策略的数据移动概要信息，包含策略名称、动作类型、条件等。

```
gaussdb=# SELECT * FROM GS_ADM_ILMDATAMOVEMENTPOLICIES;
 policy_name | action_type | scope | compression_level | tier_tablespace | tier_status |
condition_type     | condition_days | custom_function | policy_subtype | action_clause | tier_to
-------------+-------------+-------+-------------------+-----------------+-------------+-------------------------
+----------------+----------------+---------------+--------------+---------
 p1          | COMPRESSION | ROW   | ADVANCED          |                 |             | LAST MODIFICATION
TIME |             90 |                |               |              |

gaussdb=# SELECT * FROM GS_MY_ILMDATAMOVEMENTPOLICIES;
 policy_name | action_type | scope | compression_level | tier_tablespace | tier_status |
condition_type     | condition_days | custom_function | policy_subtype | action_clause | tier_to
-------------+-------------+-------+-------------------+-----------------+-------------+-------------------------
```

 版权所有 © 华为云计算技术有限公司

```
+----------------+-----------------+---------------+---------------+---------
 p1          | COMPRESSION | ROW   | ADVANCED          |               |             | LAST MODIFICATION
TIME |            90 |               |               |             |
(1 row)
```

10. 查询所有存在ILM策略应用的数据对象与相应策略的概要信息，包含策略名称、数据对象名称、策略的来源、策略的启用删除状态。

```
gaussdb=# SELECT * FROM  GS_ADM_ILMOBJECTS;
 policy_name | object_owner | object_name | subobject_name | object_type |    inherited_from    |
tbs_inherited_from | enabled | deleted
-------------+--------------+-------------+----------------+-------------+----------------------
+--------------------+---------+---------
 p1          | public       | lineitem    |                | TABLE       | POLICY NOT INHERITED |                    |
YES     | NO

gaussdb=# SELECT * FROM  GS_MY_ILMOBJECTS;
 policy_name | object_owner | object_name | subobject_name | object_type |    inherited_from    |
tbs_inherited_from | enabled | deleted
-------------+--------------+-------------+----------------+-------------+----------------------
+--------------------+---------+---------
 p1          | public       | lineitem    |                | TABLE       | POLICY NOT INHERITED |                    |
YES     | NO
```

11. 查询ADO Task的概要信息，包含Task ID，Task Owner，状态以及时间信息。

```
gaussdb=# SELECT * FROM GS_ADM_ILMTASKS;

 task_id | task_owner |   state   |         creation_time         |          start_time           |
completion_time
---------+------------+-----------+-------------------------------+-------------------------------
+-------------------------------
       1 | omm        | COMPLETED | 2023-10-16 12:03:55.113296+08 | 2023-10-16 12:03:55.113296+08 |
2023-10-16 12:03:56.326864+08
(1 row)

gaussdb=# SELECT * FROM GS_MY_ILMTASKS;
 task_id | task_owner |   state   |         creation_time         |          start_time           |
completion_time
---------+------------+-----------+-------------------------------+-------------------------------
+-------------------------------
       1 | omm        | COMPLETED | 2023-10-16 12:03:55.113296+08 | 2023-10-16 12:03:55.113296+08 |
2023-10-16 12:03:56.326864+08
(1 row)
```

12. 查询ADO Task的评估详情信息，包含Task ID，策略信息、对象信息、评估结果以及ADO JOB名称。

```
gaussdb=# SELECT * FROM GS_ADM_ILMEVALUATIONDETAILS;
 task_id | policy_name | object_owner | object_name | subobject_name | object_type |
selected_for_execution |      job_name       | comments

---------+-------------+--------------+-------------+----------------+-------------+-----------------------
+--------------------+---------
-
       1 | p2          | public       | ilm_table_1 |                | TABLE       | SELECTED FOR EXECUTION | ilmjob
$_postgres1 |
(1 row)

gaussdb=# SELECT * FROM GS_MY_ILMEVALUATIONDETAILS;
 task_id | policy_name | object_owner | object_name | subobject_name | object_type |
selected_for_execution |      job_name       | comments

---------+-------------+--------------+-------------+----------------+-------------+-----------------------
+--------------------+---------
-
       1 | p2          | public       | ilm_table_1 |                | TABLE       | SELECTED FOR EXECUTION | ilmjob
$_postgres1 |
(1 row)
```

13. 查询ADO JOB的执行详情信息，包含Task ID，JOB名称、JOB状态、JOB时间信息等。

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# SELECT * FROM GS_ADM_ILMRESULTS;
 task_id |     job_name      |       job_state        |          start_time           |       completion_time         |
comments |                    statistics
---------+-------------------+------------------------+-------------------------------
+-------------------------------+----------+-----------------------------------------------------
       1 | ilmjob$_postgres1 | COMPLETED SUCCESSFULLY | 2023-10-16 12:03:56.290176+08 |
2023-10-16 12:03:56.319829+08 |          | SpaceSaving=0,BoundTime=1697429033,LastBlkNum=40
(1 row)


gaussdb=# SELECT * FROM GS_MY_ILMRESULTS;
 task_id |     job_name      |       job_state        |          start_time           |       completion_time         |
comments |                    statistics
---------+-------------------+------------------------+-------------------------------
+-------------------------------+----------+-----------------------------------------------------
       1 | ilmjob$_postgres1 | COMPLETED SUCCESSFULLY | 2023-10-16 12:03:56.290176+08 |
2023-10-16 12:03:56.319829+08 |          | SpaceSaving=0,BoundTime=1697429033,LastBlkNum=40
(1 row)
```

2.10 内存管理

Linux 内存机制

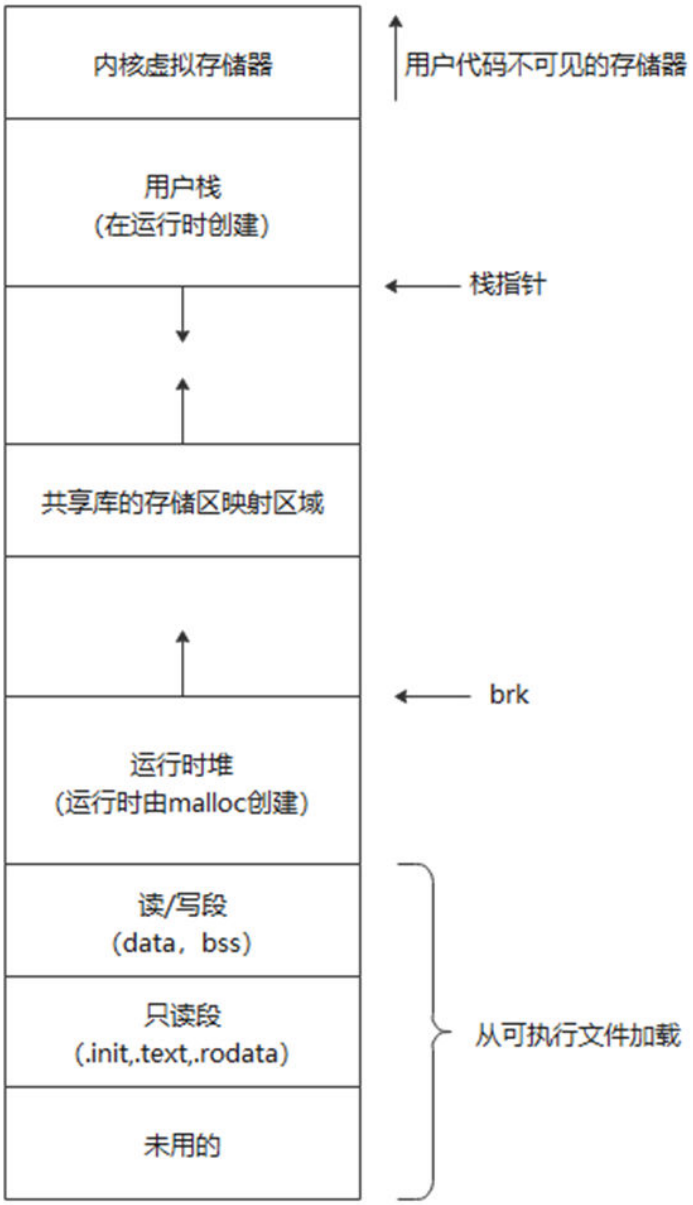
- 物理内存

  物理可见的硬件，即内存条。

- 虚拟内存

  进程运行时所有内存空间的总和，以32位系统为例，每个进程有4GB的内存空间，各进程的内存空间具有类似的结构。

 版权所有 © 华为云计算技术有限公司



  - 一个新进程建立的时候，将会建立起自己的内存空间，并将此进程的数据、代码等从磁盘复制到自己的内存空间。进程中每一条数据的地址都由进程控制表中的task_struct记录。task_struct包含一张链表，用来记录内存空间的分配情况，哪些地址有数据、哪些地址无数据、哪些可读、哪些可写都可以通过这个链表记录。
  - 每个进程已经分配的内存空间，都与对应的磁盘空间存在映射关系。

    说明

    建议不要对进程虚拟内存大小进行限制。

- 物理内存地址和虚拟内存地址
  - 内存分页机制
    - 虚拟内存中的页（page）和物理内存页帧（page frame）相同大小。

版权所有 © 华为云计算技术有限公司

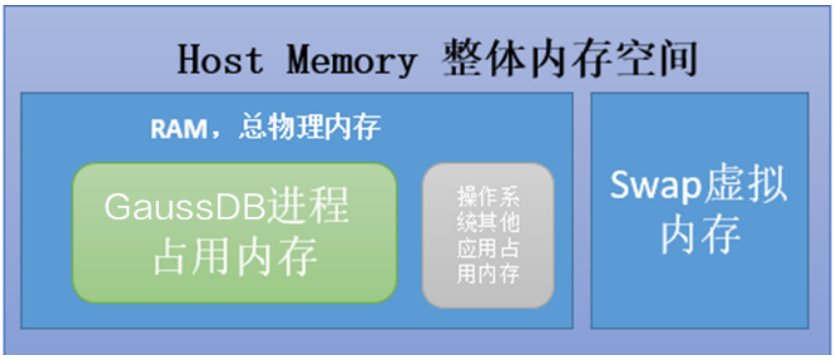
        - 页表（page table）维护虚拟内存页到物理内存页的映射。
        - 页面失效（page fault）功能：操作系统找到一个最少使用的页帧，让他失效，并把它写入磁盘，随后把需要访问的页放到页帧中，并修改页表中的映射。
    - 虚拟内存地址：页号（与页表中的页号关联）和偏移量（页的大小）组成。
    - 虚存寻址：页表中找到页帧号，若失效则调入页，页帧号及偏移量传给内存管理单元，组成一个物理存在的地址，即可访问物理内存数据。
- Buffer
    - 根据磁盘的读写设计的，把分散的写操作集中进行，减少磁盘碎片和硬盘的反复寻道，从而提高系统性能。
    - Linux有一个守护进程定期清空缓冲内容（即写入磁盘），也可以通过sync命令手动清空缓冲。
- Cache
    - 将读取过的数据保存起来，重新读取时若命中（找到需要的数据）就不需要读硬盘了，若没有命中就读硬盘。其中的数据会根据读取频率进行组织，把读取最频繁的内容放在最容易找到的位置，把不再读的内容不断往后排，直至从中删除。
- OOM
    - 内存溢出（Out Of Memory，简称 OOM）是指应用系统中存在无法回收的内存或使用的内存过多，最终使得程序运行要用到的内存大于能提供的最大内存。此时程序无法运行，系统会提示内存溢出，有时会自动关闭软件，通过重启机器或软件释放掉一部分内存后又可以正常运行该软件，而由系统配置、数据流、用户代码等原因而导致的内存溢出错误，即使用户重新执行任务依然无法避免。

GaussDB 内存资源管理

- 内存管理简介
    - GaussDB的动态内存使用方法均基于内存上下文管理，在内存上下文的机制上，引入了逻辑内存管理机制，同时提供多项视图来追踪内存使用情况。

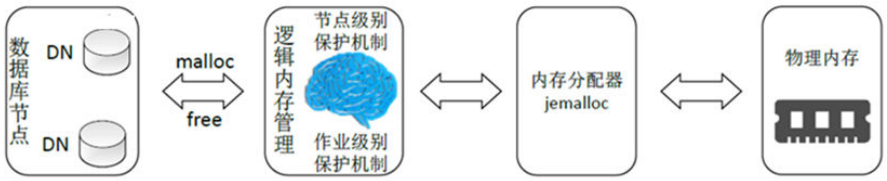
      当前GaussDB的内存管理分为两级，分别是：
        - 内存节点级别控制：通过max_process_memory参数限制DN上可以使用的内存上限。
        - 内存作业级别控制：限制单条query语句最大可使用的内存上限。
- 节点级别内存控制
    - 在Linux操作系统中，内存空间通常由物理内存RAM和虚拟内存Swap组成。其中，物理内存由数据库程序和非数据库程序共同使用；而数据库程序的内存空间由GaussDB进程共同使用。

 版权所有 © 华为云计算技术有限公司

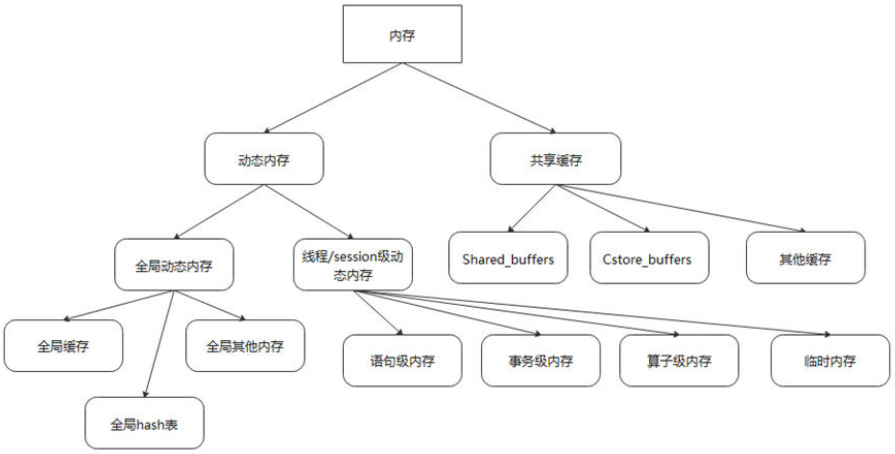


  - 由于非数据库程序也需要使用内存，一般建议GaussDB占系统可用内存的80%。

- 作业级别内存控制
  - 提供query_mem参数，根据能使用的内存量和能同时运行的作业数，指定每个作业执行时能够使用的内存量；提供query_max_mem参数，指定每个作业执行时能够使用的内存上限。
  - 提供work_mem参数，设置内部排序操作和Hash表在开始写入临时磁盘文件之前使用的内存大小；提供maintenance_work_mem参数，设置在维护性操作（比如VACUUM、CREATE INDEX、ALTER TABLE ADD FOREIGN KEY等）中可使用的最大内存。
- 逻辑内存管理机制
  - GaussDB内存管理是通过逻辑内存管理来实现的。其实现原理是在原始的内存分配之上增加一层逻辑内存管理，通过检查已分配内存是否超过规定的内存来决定是否为作业分配内存。逻辑内存管理没有更改原有的内存资源分配机制，仅在分配内存之前增加一个逻辑判断层，查看是否达到允许使用的内存上限，来决定是否分配内存。



- 内存分类



版权所有 © 华为云计算技术有限公司

GaussDB的内存使用分类如上所述，其每种类型的说明如下：

- 动态内存
  - 全局动态内存
    - 全局缓存：全局缓存（Cache）用来提高访问效率，Cache中包括一个系统表元组Cache和一个表模式信息Cache（RelCache），SysCache中存放的是最近使用过的系统表的元组，而RelCache中包含所有最近访问过的表的模式信息（包含系统表的信息）。RelCache中存放的不是元组，而是RelationData数据结构，每一个RelationData结构表示一个表的模式信息，这些信息都由系统表元组中的信息构造而来。
    - 全局hash表：保存全局的统计信息或其他数据的内存hash结构。
    - 全局其他内存：保存全局变量等内容。
  - 线程/session级动态内存
    - 语句级内存：生命周期为整个SQL语句执行阶段，语句执行前内存被申请，执行结束后内存被回收释放。
    - 事务级内存：生命周期为整个事务内，事务开始前内存被申请，事务结束后内存被回收释放。
    - 算子级内存：生命周期为单个算子有效期内，算子执行结束后内存被释放回收。
    - 临时内存：一些临时变量申请的内存，使用后及时释放。
- 共享缓存
  - Shared_buffers

    缓存从磁盘读取的数据所使用的内存。
  - 其他缓存

    线程所使用的槽位，锁等资源占用的内存。

- 内存上下文机制
  - 内存上下文介绍
    - 概念：内存上下文用来统一控制内存的申请和释放，内存上下文中记录了其所申请的总内存大小，每一处内存申请的大小，申请内存所在的文件，行号等信息以及与其他内存上下文之间的关联信息。
    - 目的：基于数据库中作业执行逻辑，通过内存上下文管理机制，提升查询内存分配、释放效率。
  - 逻辑内存管理
    - 通过数据库进程内部记账，控制单进程内存使用的上限。
    - 可统计进程总体使用、共享和非共享内存上下文使用及重点模块内存使用（通信库、pooler等）。
    - 逻辑内存的生效机制：enable_memory_limit为on，并且max_process_memory减掉缓存等其他预留内存 > 2GB，具体详情请联系管理员。

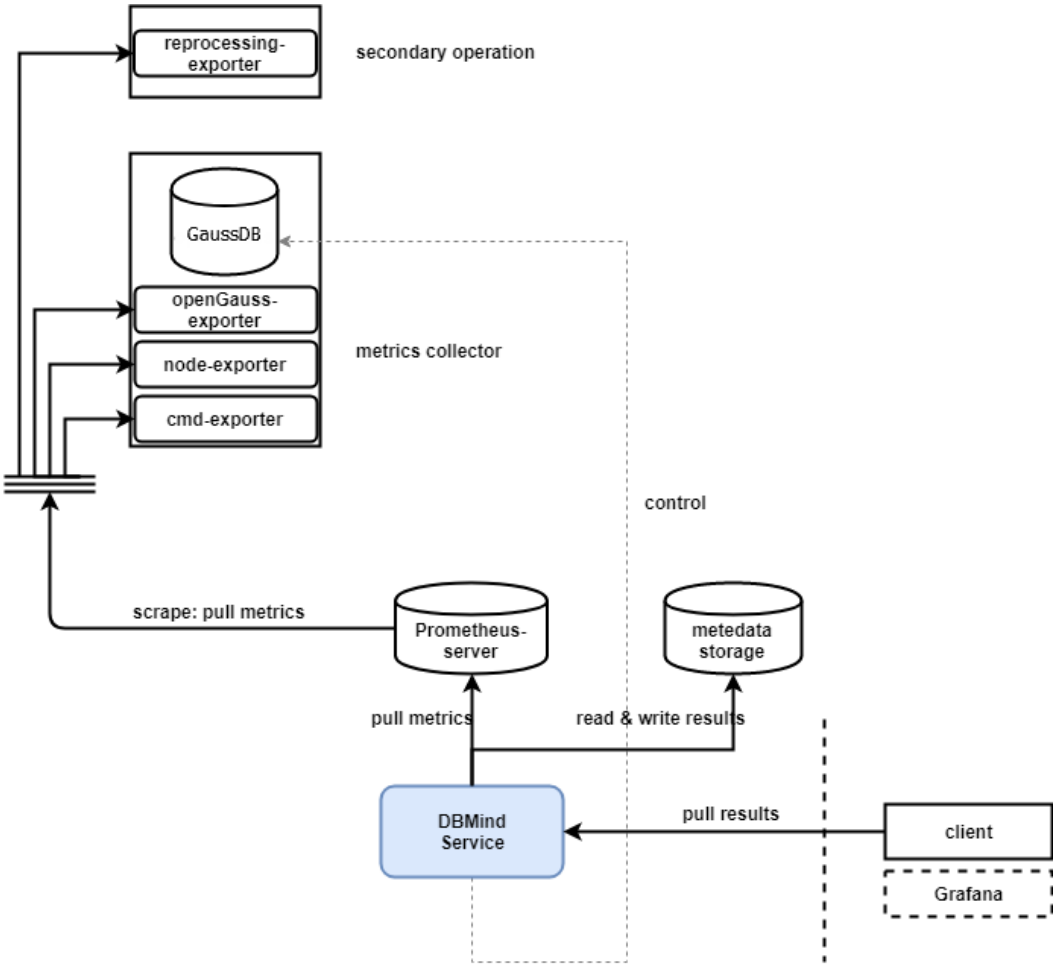
 版权所有 © 华为云计算技术有限公司

- GS_TOTAL_MEMORY_DETAIL视图用于查询数据库进程的内存使用情况，便于分析内存失败后问题，具体字段可参考《开发者指南》中“系统表和系统视图 > 系统视图 > GS_TOTAL_MEMORY_DETAIL”章节。

2.11 DBMind: 数据库自治运维

DBMind承载了GaussDB的AI for Database特性，主要用于对数据库进行自治运维和管理，从而帮助数据库运维人员减少运维工作量。在实现上，DBMind具有监控和服务化的性质，同时也提供即时AI工具包，提供开箱即用的AI自治功能（如：索引推荐）。DBMind的监控平台以开源Prometheus为主，并提供了数据生产者exporter，可与Prometheus平台完成对接。DBMind服务架构如下图所示：

图 2-9 DBMind 服务架构



图中各关键组件说明：

- DBMind Service：DBMind后台服务，可用于定期离线计算，包括慢SQL根因分析、时序预测等；
- Prometheus-server：存储Prometheus监控指标的服务器；
- metadatabase：DBMind在离线计算结束后，将计算结果存储在此处，支持GaussDB、SQLite等数据库；

版权所有 © 华为云计算技术有限公司

- client：用户读取DBMind离线计算结果的客户端，目前该客户端仅支持命令行操作；若采用GaussDB等数据库存储DBMind的计算结果，则用户可以自行配置Grafana等可视化工具，用于对该结果进行可视化；
- openGauss-exporter：用户从GaussDB数据库节点上采集监控指标，供DBMind服务进行计算；
- node-exporter：Prometheus官方提供的exporter，可用于监控对应节点的系统指标，如CPU和内存使用情况；
- cmd-exporter：在用户安装数据库的环境上执行命令行，并采集该命令行的执行结果，同时，也尝试将数据库日志内容转化为监控指标；例如通过执行cm_ctl命令，查看数据库实例的状态；
- reprocessing-exporter：用于对Prometheus采集到的指标进行二次加工处理，例如计算CPU使用率等。

DBMind 支持的场景规格

- DBMind目前不支持在元数据库变更的情况下保留原始诊断数据；
- DBMind不支持纳管容灾集群中的备节点；
- DBMind支持纳管多集群，但是随着纳管集群节点越多，消耗的资源也对应增多。需要在DBMind消耗的资源过多时，停止纳管更多节点，以避免CPU和内存资源不足。
- DBMind实例纳管建议为1个CPU核对应3个节点（例如3个节点（包含主备）的集中式则需要1个CPU核，9个节点（包含DN等）的分布式需要3个CPU核），如果纳管的实例节点数太多，会导致DBMind服务接口超时或服务异常。
- DBMind只支持文档中写明的场景，不支持文档中没有明确的场景、示例中不包括的部署形态、使用方法等超规格使用形式的情况。
- DBMind服务当前不支持ipv6。
- DBMind在M-compatibility数据库下仅支持异常检测与根因分析、实例故障诊断功能，不支持其他兼容形态。

说明

云侧网络情况和DBMind云上部署说明：

- 云上的实例节点拥有多层网络结构，集中式节点拥有两层结构：管理层（CM）和数据层（DN）。
- 对DBMind主程序部署情况的说明如下：
  - 启动prometheus，监听在数据层ip。
  - 启动reprocessing_exporter，监听在数据层ip。
  - 启动DBMind，暴露webservice 在管理层ip（管理ip是因为接口调用需要走管理面发起，需要保证管理面网络互通）。
- 对于被纳管的数据库实例，插件部署情况如下：
  - node_exporter、cmd_exporter、opengauss_exporter都监听在数据层ip。
  - 对于仲裁、冷备、offline的节点，只会部署(node_exporter、cmd_exporter)。
  - 其他情况均会部署node_exporter、cmd_exporter、opengauss_exporter。
- dbmind指标采集主要为本身功能服务，在采集过程中可能存在网络故障、采集超时等异常，另外prometheus本身采集机制为通过标签区分不同指标，因此不能保证采集指标的连续性，如果用户需要将指标用于前端展示，在指标不连续的情况下需要自己补齐。

 版权所有 © 华为云计算技术有限公司

环境配置

DBMind外置AI功能需要运行在Python 3.7版本以上，当前DBMind独立出包并且包中已包含工具所有三方依赖项，因此用户不用单独安装。如果用户需要自己配置依赖项，DBMind需要的第三方依赖包记录在AI功能根目录的requirements.txt文件中（包括requirements-x86.txt与requirements-arrch64.txt，用户可根据自己平台类型选择）中，可以通过pip install命令安装依赖，命令如下：

```
pip3 install -r requirements-x86.txt
```

如果用户没有安装齐全所需的依赖，则当用户执行gs_dbmind命令时，会再次提醒用户安装第三方依赖。需要注意，该文件提供了DBMind所需的第三方依赖，若用户环境存在第三方包冲突等情况，可由用户根据实际情况进行处理。

如果pip版本过低，可能会出现装包失败的情况，此时可使用以下命令对pip进行升级：

```
pip3 install --upgrade pip
```

2.11.1 DBMind 模式说明

用户可通过gs_dbmind命令调用DBMind下列基本功能：

- 服务功能：service子命令，包括创建并初始化配置目录、启动后台服务、关闭后台服务等；
- 调用组件：component子命令（如索引推荐、参数调优等）可通过该模式进行调用；
- 设置参数：set子命令，通过该命令，可以一键修改配置目录中的配置文件值；用户也可以通过文本编辑器进行手动修改。

用户可以通过 --help 选项获得上述模式的帮助信息，例如：

```
gs_dbmind --help
usage: [-h] [-v] {service,set,component} ...

openGauss DBMind: An autonomous platform for GaussDB

optional arguments:
  -h, --help            show this help message and exit
  -v, --version         show program's version number and exit

available subcommands:
  {service,set,component}
                        type '<subcommand> -h' for help on a specific subcommand
    service             send a command to DBMind to change the status of the service
    set                 set a parameter
    component           pass command line arguments to each sub-component.
```

表 2-15 gs_dbmind 选项基本说明

参数	参数说明	取值范围
-h, --help	帮助命令	-
--version	版本号	-
service	服务功能相关的子命令	-
component	调用组件的子命令	-
set	修改配置文件的子命令	-

 版权所有 © 华为云计算技术有限公司

2.11.1.1 service 子命令

该子命令可用于对配置目录进行初始化，同时也可以实现启动和停止后台任务。

配置目录初始化

用户可通过“gs_dbmind service setup”子命令进行配置目录的初始化。该配置文件中可包括DBMind的配置文件、日志等内容。该目录中的部分文件说明：

- dbmind.conf：DBMind的参数配置文件，用户可通过“gs_dbmind set”命令进行修改，也可通过文本编辑器进行手动修改。
- dynamic_config.db：DBMind服务保存在本地节点的元信息，主要包括算法的超参数、监控阈值等；该文件为DBMind服务元信息，不可由用户直接配置。
- metric_map.conf：监控指标映射表，可用于适配到不同采集平台中。例如，在DBMind中，监控到的系统cpu使用率名为os_cpu_usage，而用户自行实现的指标采集工具将cpu使用率命名为my_cpu_usage_rate。则在该种情况下，如果想要DBMind中代表cpu使用率的指标名为my_cpu_usage_rate，则需要修改该配置选项。即添加“os_cpu_usage = my_cpu_usage_rate”配置项进行映射。对于普通用户，建议直接使用DBMind配套的采集组件和方案，则不涉及修改该配置文件。
- metric_value_range.conf：指定某些具体监控指标的上下限，以防止对该指标进行预测时，超过合理区间；未在该列表中的指标范围默认为[0, +∞)。
- dbmind.pid：该文件保存正在使用该DBMind配置目录的进程PID。
- dbmind.children.pid：该文件保存DBMind子进程的PID信息，pid之间用逗号隔开。
- VERSION：生成该配置文件目录的DBMind版本，通过验证版本来避免版本不兼容情况下的使用。
- logs：该目录用于存储DBMind服务产生的日志，请不要手动修改该目录下的日志文件，否则可能会导致日志变为不可写状态，无法继续更新DBMind操作记录。
- backtrace.stack：DBMind自身用于定位调试产生的临时文件，包含了当前各个线程的调用栈信息，是通过接收到SIGUSR2信号而产生的。

用户可通过两种方式进行配置目录的初始化，一种为交互式，另一种为非交互式。例如，待初始化的配置目录名为confpath，则分别通过下述方法进行配置：

交互式模式

```
gs_dbmind service setup -c confpath --interactive
```

执行完毕上述命令后，用户可通过命令行终端对配置项进行交互式配置。

非交互式模式

非交互式模式总共分为三个步骤，即创建配置文件、修改配置项、初始化配置。其中第二个步骤需要用户通过文本编辑器手动编辑配置文件。具体步骤如下：

步骤1 创建配置文件，执行下述命令：

```
gs_dbmind service setup -c confpath
```

步骤2 执行完上述命令后，会在confpath目录下生成dbmind.conf配置文件，用户需要利用文本编辑器进行手动修改。相关参数的说明如下：

```
# METADATABASE部分用于指定DBMind生成的分析结果的存储位置。
```

 版权所有 © 华为云计算技术有限公司

```
# 当前支持的数据库类型有SQLite和GaussDB Kernel。如果使用openGauss数据库，使用时需要注意Python驱动psycopg2的兼容性问题，用户可以选择使用openGauss官方提供的驱动，也可以通过自行编译或修改GUC参数进行适配。
# 其他信息为连接到该数据库的连接信息，注意用户需要有数据库创建权限。
[METADATABASE]
dbtype = sqlite # 元数据库类型，选项：sqlite，postgresql
host = # 元数据库地址，如果配置多个元数据库地址，需要用逗号（,）分隔。
port = # 元数据库端口，如果配置多个元数据库地址，且port相同，填写一个port即可。如果port不相同，port数量需要和host数量一致，用逗号（,）分隔。
username = # 元数据库用户名
password = (null) # 元数据库用户密码
database = # 元数据库名

# WORKER用于指定DBMind可以使用的worker子进程数量，如果写0则会进行自适应，即尽可能多地使用CPU资源。
[WORKER]
process_num = 0  # 本地节点上的工作进程数，小于或等于零表示自适应。

# AGENT部分用于指定DBMind连接到openGauss Agent的信息。通过使用该Agent，可以让DBMind获取到被监控实例的即时状态，从而提高分析准确性。同时，也可以向数据库实例下发一些变更动作，如结束某条慢SQL语句（这取决于此处配置的用户是否有足够的权限）。
# 该master_url地址即为Agent的地址，由于openGauss-exporter承担了Agent的角色，故该地址也就是openGauss-exporter 的地址。
# 同时，openGauss-exporter是支持HTTPS协议的，所以，此处也可以根据配置指定SSL证书。
[AGENT]
master_url = # 不配置该参数，即可启动自动发现模式，此时所有在TSDB中注册的代理都会被找到，但是需要配置一套统一的用户名/密码/SSL连接信息。否则，您可以键入主节点的代理 URL，例如，https://127.0.0.1:9187，如果您有多个数据库实例，则需要用逗号（,）分隔它们。
username =  # 登录监控数据库的用户名。代理凭证。如果配置监控多个数据库实例，且这些数据库实例的用户名和密码不同，则需要输入多组配置用户名，各配置用户名之间用逗号（,）分隔，且需要与上述master_url选项一一对应。
password = (null) # 监控数据库的登录密码。代理凭证。如果配置监控多个数据库实例，需要用逗号（,）分隔。具体配置方法与上述参数相同。
ssl_certfile = (null) # SSL连接的证书文件路径。这是可选的。如果配置监控多个数据库实例，需要用逗号（,）分隔。具体配置方法与上述参数相同。
ssl_keyfile = (null) # SSL连接的私钥文件路径。这是可选的。如果配置监控多个数据库实例，需要用逗号（,）分隔。具体配置方法与上述参数相同。
ssl_keyfile_password = (null) # SSL私钥文件的密码。这是可选的。如果配置监控多个数据库实例，需要用逗号（,）分隔。
ssl_ca_file = (null)  # 用于验证请求的CA证书文件路径。这是可选的。如果配置监控多个数据库实例，需要用逗号（,）分隔。具体配置方法与上述参数相同。

[TIMED_TASK_LIST]
explanation = Configure scheduled tasks including start tasks and intervals # 配置计划任务，包括启动任务和间隔
current_support_task_list = List of currently supported tasks:  # 当前支持的任务列表
task1 = anomaly_detection          # 1. 指标异常检测.
task2 = discard_expired_results    # 2. 清理过期数据.
task3 = knob_recommend             # 3. 参数推荐
task4 = slow_query_killer          # 4. 慢SQL查杀
task5 = slow_query_diagnosis       # 5. 慢SQL诊断
task6 = cluster_diagnose           # 6. 集群故障诊断
task7 = agent_update_detect        # 7. 当检测到代理更新时自动更新异常检测器
task8 = calibrate_security_metrics # 8. 校准security_scenarios.yml提到的安全指标
task9 = check_security_metrics     # 9. 检查security_scenarios.yml提到的安全指标
task10 = update_statistics         # 10. 自动更新指定指标的统计信息

TIMED_TASK表示后台任务配置，通过修改参数启动或停止DBMind运行的后台任务以及任务的运行间隔，单位为秒。
运行间隔不能低于30秒，最大值为python3支持的整型数字上限，2的63次方减1。
[TIMED_TASK]
task = discard_expired_results,anomaly_detection, cluster_diagnose, agent_update_detect, calibrate_security_metrics, check_security_metrics, update_statistics # 默认reload启动数据清理、异常检测、集群故障诊断、更新异常检测器、校准security_scenarios.yml指标、检查security_scenarios.yml指标和更新指定指标的统计信息任务
anomaly_detection_interval = 180  # 异常检测任务运行间隔，默认180秒
slow_query_diagnosis_interval = 120  # 慢SQL诊断任务运行间隔，默认120秒
knob_recommend_interval = 3600  # 参数推荐任务运行间隔，默认一小时
slow_query_killer_interval = 30  # 慢SQL查杀任务运行间隔，默认30秒
```

 版权所有 © 华为云计算技术有限公司

```
cluster_diagnose_interval = 30  # 集群故障诊断运行间隔，默认30秒
discard_expired_results_interval = 3600  # 数据清理任务运行间隔，默认一小时
agent_update_detect_interval = 30  # 检测agent更新并自动并自动更新异常检测器间隔，默认30秒
calibrate_security_metrics_interval = 600 # 校准security_scenarios.yml安全指标间隔，默认10分钟
check_security_metrics_interval = 600 # 检查security_scenarios.yml安全指标间隔，默认10分钟
update_statistic_interval = 1800  # 自动更新指定指标统计信息间隔，默认30分钟

# WEB-SERVICE用于DBMind的前台界面展示
[WEB-SERVICE]
ssl = true # 默认使用安全的协议，用户需要提供证书路径，否则DBMind服务不能启动
host = 127.0.0.1  # DBMind-service监听地址
port = 8080  # DBMind-service监听端口
ssl_certfile = (null) # SSL连接的证书文件路径
ssl_keyfile = (null) # SSL连接的私钥文件路径
ssl_keyfile_password = (null) # SSl私钥文件的密码，如果没有密码则忽略
ssl_ca_file = (null)  # 用于验证请求的CA证书文件路径

# LOG表示设置DMBind的日志记录信息
[LOG]
maxbytes = 10485760 # 默认值为10Mb。单个日志文件的最大大小。如果 maxbytes 为零，则文件无限增长
backupcount = 1 # 日志文件备份数量
level = INFO  # 日志级别，选项：DEBUG, INFO, WARNING, ERROR
log_directory = logs # 日志文件存放的目录

# 下列内容表示给用户进行交互配置时的提示信息，用户无需配置。
[COMMENT]
worker = The form of executing compute-intensive tasks. Tasks can be executed locally or distributed to
multiple nodes for execution.
tsdb = Configure the data source for time series data, which come from monitoring the openGauss instance.
metadatabase = Configure the database to record meta-data, which the database can store meta-data for
the forecasting and diagnosis process. The database should be an openGauss instance.
self-monitoring = Set up parameters for monitoring and diagnosing openGauss instance.
self-optimization = Set up parameters for openGauss optimization.

[IP_MAP]
ip_map = (null) # 对于数据ip和管理ip不一致的环境，需要提供对应关系。以下以一主两备数据库实例为例：
“primary_data_ip:primary_management_ip,standby1_data_ip:standby1_management_ip,standby2_data_ip:st
andby2_management_ip”
```

步骤3 待用户手动修改完上述参数后，需要执行下述命令进行配置项的初始化。在该阶段中，DBMind会初步检查配置项的正确性、初始化用于存储结果数据的元数据库表结构和内容，同时也加密配置项中出现的密码。

```
gs_dbmind service setup --initialize -c confpath
```

步骤4 完成配置目录初始化过程，可基于该配置目录启动DBMind后台服务。

----结束

说明

1. 配置文件注释信息用于在交互模式下对用户进行提示，有特殊含义不要手动修改或删除；
2. 需要确保配置项的值与注释信息之间通过空格符分割，否则系统会将注释信息识别为配置项的值；
3. 配置项中的特殊字符，如果需要转义，则通过转义符“百分号”（%）来转义，例如，用户配置的密码为 "password%"，则应通过“百分号”进行转义，即 "password%%";
4. DBMind Service对外以标准RestfulAPI的方式提供接口，其默认支持https通信，建议用户使用。
5. DBMind云侧部署主机目前最小规格8U64G，目前云管控支持容灾主集群纳管，不支持容灾备集群纳管。
6. DBMind当前不支持M兼容库作为元数据库进行初始化。

 版权所有 © 华为云计算技术有限公司

启动服务

当用户完成配置目录的初始化后，可基于此配置目录启动DBMind后台服务。例如配置目录为confpath，则启动命令如下：

```
gs_dbmind service start -c confpath
```

当执行上述命令后，会提示服务已启动。在未指定任何附加参数时，该命令默认会启动所有的后台任务。如果用户只想启动某一个后台任务，需要添加“--only-run”选项。例如，用户只想启动慢SQL根因分析服务，则为：

```
gs_dbmind service start -c confpath --only-run slow_query_diagnosis
```

DBMind运行过程中如果想启或停止定时任务，首先将配置文件TIMED_TASK下task中相关功能删除，在执行以下命令，此时DBMind会刷新后台定时任务：

```
gs_dbmind service reload -c confpath
```

由于DBMind是一个在后台定期执行的服务，故如果用户只是想运行一次DBMind服务进行测试或调试，则可以通过添加“--dry-run”选项，如：

```
gs_dbmind service start -c confpath --dry-run
```

“--dry-run”选项可以与“--only-run”选项合用。

如果当前进程已经在运行，用户希望重启启动该DBMind服务，则可以使用重启命令，即：

```
gs_dbmind service restart -c confpath
```

默认情况下，restart会等待当前DBMind正在执行的任务执行完毕后才会退出并重启服务。如果用户希望强行终止当前正在运行的任务，则可以添加“-f”或“--force”选项，例如：

```
gs_dbmind service restart --force -c confpath
```

关闭服务

关闭服务与启动服务类似，其命令行结构更加简单，只需指定配置目录的地址即可。例如配置目录为confpath，则为：

```
gs_dbmind service stop -c confpath
```

DBMind服务会在后台执行完正在运行的任务后自行退出，如需要强行退出，则使用“--force”或“-f”选项。

DBMind 的高可用

⚠ 注意

DBMind高可用功能需要在未纳管实例的情况下实现，因此接口不需要token认证。应当使用默认的HTTPS协议以进行SSL双向认证，使用HTTP协议则可能引入攻击者直接发起未授权请求的安全风险。

为保证DBMind云侧使用时的高可靠，DBMind对外提供了服务状态查询和部分异常修复的接口，详细请参见**表2-23**。

- DBMind服务状态查询接口示例：

  ```
  curl -X 'POST'"http://127.0.0.1:8080/v1/api/check-status" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"cmd": "./gs_dbmind service start -c dbmindconf "}'
  ```


版权所有 © 华为云计算技术有限公司

如果使用HTTPS协议，则查询示例为：

```
curl -X 'POST'"https://127.0.0.1:8080/v1/api/check-status" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"cmd": "./gs_dbmind service start -c dbmindconf "}' --cacert xx.crt --key xx.key --cert xx.crt
```

返回结果示例：

```
{"data":{"error_msg":"","result":{},"state":"NORMAL"},"success":true}
```

- DBMind异常修复接口示例：

```
curl -X 'POST'"http://127.0.0.1:8080/v1/api/repair" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"cmd": "./gs_dbmind service start -c dbmindconf "}'
```

如果使用HTTPS协议，则修复示例为：

```
curl -X 'POST'"https://127.0.0.1:8080/v1/api/repair" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"cmd": "./gs_dbmind service start -c dbmindconf "}' --cacert xx.crt --key xx.key --cert xx.crt
```

返回结果示例：

```
{"data":{"error_msg":"","result":{},"state":"SUCCESS"},"success":true}
```


版权所有 © 华为云计算技术有限公司

说明

当前DBMind服务支持查询的异常场景如下：

- PID文件丢失或文件内容异常：当用户启动服务/组件后，会自动生成“*.pid”文件来记录进程对应的进程标识符，也就是PID。PID文件用于云管控判断进程状态，发现进程级别的异常并进行修复。为防止PID文件的误删除与内容的误修改，需要定期判断PID文件是否存在以及其内容是否正确。
- 日志文件丢失：当用户启动服务/组件后，会自动生成*.log文件来记录进程运行的日志。此日志文件用于快速定位问题的根源、追踪程序执行的过程等。为防止日志文件的误删除，需要定期判断日志文件是否存在。
- 资源占用异常：DBMind服务会执行索引推荐、指标分析、智能巡检等功能，需要占用系统资源；exporter组件会采集服务器上的指标，用于后续分析，也会占用系统资源；当服务/组件的CPU或内存占用超过阈值时，说明资源占用过高，服务/组件出现异常。因此需要监控服务和组件的资源占用情况。
- 元数据库主备切换：DBMind支持数据库的主备切换功能，在主服务器发生故障等条件下，自动将备份服务器切换为主服务器，确保系统的持续运行。当数据库出现主备切换时，需要DBMind能够正确识别并访问对应的服务器，需要定期判断数据库是否发生了主备切换。如果配置文件中只配置一个元数据库的ip，不支持主备切换。
- 元数据库异常：DBMind需要将配置信息、检测到的异常结果、故障信息等内容进行持久化，存储在元数据库中，因此需要确保元数据库的正常运行；如果元数据库发生异常，则可能会导致依赖元数据库存储结果的任务无法向其插入数据，导致数据遗漏。

资源占用阈值及告警设置如下：

- 内存占用阈值：max(总内存的1%, 200MB)。
- CPU多核占用阈值：cmd_exporter 10%，opengauss_exporter 10%，reprocessing_exporter 10%， DBMind 80%。
- CPU单核占用阈值：cmd_exporter 50%，opengauss_exporter 50%，reprocessing_exporter 50%，DBMind不做限制。
- 调用高可用接口会检查资源占用，连续超过3次阈值才会触发资源占用告警。

风险：当占用资源连续超过阈值三次时，会触发告警，管控侧会重启服务，导致正在运行的任务中断。

当前针对DBMind服务异常的修复性说明如下，不在该范围的异常不支持修复：

- PID文件丢失：可自动修复；
- 日志文件丢失：可自动修复；
- 组件资源占用异常：不可通过接口修复；
- 元数据库主备切换：可自动修复；
- 元数据库异常：不可通过接口修复；
- 针对Exporter组件不可修复的异常，需要云侧进行处理，如重启进程。

版权所有 © 华为云计算技术有限公司

⚠ 注意

- [METADATABASE]中的元数据库用户需要具有在该数据库下的创表和数据插入更新权限，否则执行时会出现异常。而且，需要用户提前创建好数据库，否则，会提示无法连接到该数据库的错误。
- DBMind尝试为每个配置文件目录启动一个DBMind实例，并通过PID文件记录DBMind实例PID，不支持同一配置文件下启动多个服务。
- DBMind提供了requirement.txt文件，用户可以通过该文件安装所需的第三方依赖。
- DBMind只支持特定场景的异常修复，对于不能修复的异常则需要云侧进行处理。
- DBMind支持元数据库主备切换场景的自动修复，但在修复期间对元数据库的业务操作会执行失败，导致数据无法正常插入、查询或更新。
- 如果用户在配置目录的dbmind.conf中填写了AGENT下的master_url，当调用**DBMind的AI子功能**中/v1/api/agents接口更新agent时，如果参数force=true时则不支持更新，该参数只有mater_url没有填写时支持（会自动查找）。
- DBMind当前使用进程池执行特性功能，因此接口侧的响应时间与进程池参数设置和环境等有关，比如用户在confpath目录下的dbmind.conf配置文件中process_num设置不合理或硬件性能较差等，使进程池负载较重导致**DBMind的AI子功能**中部分接口超时，比如/v1/api/risk-analysis/{metric}。

命令参考

用户可以通过“--help”选项获得该模式的帮助信息，例如：

```
gs_dbmind service --help
usage:  service [-h] -c DIRECTORY
              [--only-run
{discard_expired_results,anomaly_detection,cluster_diagnose,agent_update_detect,knob_recommend,slow_query_killer,slow_query_diagnosis}] [--dry-run] [-f]
 [--interactive | --initialize]
              {setup,start,stop,restart,reload}

positional arguments:
  {setup,start,stop,restart,reload}
                        perform an action for service

optional arguments:
  -h, --help            show this help message and exit
  -c DIRECTORY, --conf DIRECTORY
                        set the directory of configuration files
  --only-run
{discard_expired_results,anomaly_detection,cluster_diagnose,agent_update_detect,knob_recommend,slow_query_killer,slow_query_diagnosis}
                        explicitly set a certain task running in the backend
  --dry-run             run the backend task(s) once. the task to run can be
                        specified by the --only-run argument
  -f, --force           force to stop the process and cancel all in-progress
                        tasks
  --interactive         configure and initialize with interactive mode
  --initialize          initialize and check configurations after configuring.
```


版权所有 © 华为云计算技术有限公司

表 2-16 gs_dbmind service 子命令说明

参数	参数说明	取值范围
action	动作参数	• setup：初始化配置目录。 • start：启动服务。 • restart: 重启服务。 • reload: 重新加载配置文件参数。 • stop：停止服务。
-c, --conf	配置文件目录地址	-
--initialize	配置参数初始化	-
--interactive	交互式输入配置参数	-
--only-run	选择只运行的模块	• slow_query_diagnosis：慢SQL根因分析模块。 • anomaly_detection：异常检测模块。 • slow_query_killer：慢SQL查杀。 • cluster_diagnose：集群故障诊断。 • agent_update_detect：代理更新。 • 下列模块也可输入，但是DBMind内部模块使用，无命令行获取接口，对用户透明：knob_recommend、discard_expired_results。
--dry-run	只跑一次DBMind的任务，任务执行完毕后退出	-
-f, --force	强行退出当前正在执行的任务，可用于restart和stop命令中	-
-h, --help	帮助命令	-

2.11.1.2 component 子命令

该子命令可以用于启动DBMind的子组件（或插件），包括可用于监控指标的exporter，以及AI功能等。该命令可以将用户通过命令行传入的命令转发给对应的子组件，故不同的子组件命令需参考其功能的对应说明，详见后文各个子组件对应章节，此处不再赘述。

版权所有 © 华为云计算技术有限公司

命令参考

用户可以通过“--help”选项获得该模式的帮助信息，例如：

```
gs_dbmind component --help
usage:  component [-h] COMPONENT_NAME ...

positional arguments:
  COMPONENT_NAME  choice a component to start. ['anomaly_detection',
                  'cluster_diagnosis', 'cmd_exporter', 'dkr',
                  'extract_log', 'index_advisor', 'opengauss_exporter',
                  'reprocessing_exporter', 'slow_query_diagnosis',
                  'sql_rewriter', 'sqldiag', 'xtuner']
  ARGS            arguments for the component to start

optional arguments:
  -h, --help      show this help message and exit
```

表 2-17 gs_dbmind component 子命令说明

参数	参数说明	取值范围
COMPONENT_NAME	子组件（即插件）名	'anomaly_detection', 'cluster_diagnosis', 'cmd_exporter', 'dkr','extract_log', 'index_advisor', 'opengauss_exporter', 'reprocessing_exporter', 'slow_query_diagnosis', 'sql_rewriter', 'sqldiag', 'xtuner'
ARGS	子组件的参数	参考子组件的命令说明
-h, --help	帮助命令	-

2.11.1.3 set 子命令

该命令用于修改配置文件dbmind.conf中的参数值，与用户手动修改配置文件dbmind.conf一般无差异。例如修改配置目录confpath中的配置文件dbmind.conf中TSDB配置部分，host参数的值，并将其设置为127.0.0.1。则可通过下述命令实现：

```
gs_dbmind set TSDB host 127.0.0.1 -c confpath
```

在修改上述普通参数时，与手动修改配置文件dbmind.conf无差异。但由于DBMind的配置文件中不保存明文密码（如果用户使用明文密码，则DBMind会提示并退出），故当用户想要修改密码项时，有两种方法进行修改，一种是先修改dbmind.conf，并通过以下命令实现配置文件的重新初始化：

```
gs_dbmind service setup --initialize -c confpath
```

另一种方法则是直接通过set子命令进行设置，如：

```
gs_dbmind set METADATABASE password xxxxx -c confpath
```

 版权所有 © 华为云计算技术有限公司

说明

1. 该命令对于字符串是大小写敏感的，如果输错则可能出现执行过程错误；
2. 由于set子命令涉及的参数值类型很多，故只会对设置值进行初步检查，用户需要保证输入值的内容正确，如某些值应为正整数而非负数。

命令参考

用户可以通过“--help”选项获得该模式的帮助信息，例如：

```
gs_dbmind set --help
usage:  set [-h] -c DIRECTORY section option target

positional arguments:
  section               which section (case sensitive) to set
  option                which option to set
  target                the parameter target to set

optional arguments:
  -h, --help            show this help message and exit
  -c DIRECTORY, --conf DIRECTORY
                        set the directory of configuration files
```

表 2-18 gs_dbmind set 子命令说明

参数	参数说明	取值范围
-h, --help	帮助命令	-
-c，--conf	配置文件目录confpath	-
section	设置区	-
option	设置项	-
target	设置值	-

2.11.1.4 upgrade 子命令

该子命令用于升降级DBMind配置文件和元数据库，DBMind整体升降级还需配合DBMind主服务与exporter组件重启等操作，具体步骤如下（以配置文件路径confpath为例）：

步骤1 在当前版本DBMind文件路径下，停止DBMind服务。

```
gs_dbmind service stop -c confpath
```

步骤2 在高版本DBMind文件路径下，升级配置文件和元数据库。

```
gs_dbmind upgrade -c confpath --target dbmindpath
```

步骤3 查询reprocessing_exporter、cmd_exporter、opengauss_exporter的进程号并重启。以reprocessing_exporter为例。

1. 查询进程号。
   ```
   ps ux | grep reprocessing_exporter
   ```
2. 终止进程。
   ```
   kill -9 pid
   ```
3. 使用目标版本DBMind启动进程，详细见**component子命令**。
   ```
   gs_dbmind components reprocessing_exporter
   ```


版权所有 © 华为云计算技术有限公司

步骤4 在目标版本DBMind文件路径下，启动DBMind服务。

```
gs_dbmind service start -c confpath && gs_dbmind service reload -c confpath
```

----结束

须知

- 升降级前需确认DBMind服务的API接口正常，否则，DBMind无法升级或升级后API接口不可用。
- 配置文件与元数据库升降级过程中，如果出现元数据无法连接等异常，会导致升级失败，此时，相关文件会自动回滚，如果回滚失败，返回255。
- 升降级前需保证配置文件夹可读写。
- 升降级过程中的硬件故障如磁盘故障、机器停止工作等，无法保证回滚成功。
- 升降级中的回滚失败时，可重新执行升降级命令进行升降级。
- 升级时回滚不成功，如需回退至旧版本，请先进行升级后再进行降级操作；降级时回滚不成功，如需回退至旧版本，请先进行降级再进行升级。
- 升降级回滚失败时，会保留${confpath}_backup和${confpath}_new_backup两个文件夹，用于升降级的重试，需保证这两个文件夹与用户文件不冲突。

命令参考

用户可以通过 --help 选项获得该模式的帮助信息，例如：

```
gs_dbmind upgrade --help
usage:  upgrade [-h] -c DIRECTORY [--target DIRECTORY]

optional arguments:
  -h, --help            show this help message and exit
  -c DIRECTORY, --conf DIRECTORY
                        set the directory of configuration files
  --target DIRECTORY    set the directory for dbmind of the target version
```

表 2-19 gs_dbmind upgrade 子命令说明

参数	参数说明	取值范围
-h, --help	帮助命令	-
-c，--conf	配置文件目录confpath	文件夹路径
--target	配置目标版本的DBMind目录	文件夹路径，默认为gs_dbmind的文件夹路径

2.11.2 DBMind 的支持组件

支持组件是指DBMind提供的用于支撑整个服务、确保解决方案能够部署和实施的模块。它们本身不是AI功能，却是整个服务体系中非常重要的一环，用于支撑整个自治运维解决方案的快速实施，如用于采集数据库指标的exporter等。

版权所有 © 华为云计算技术有限公司

2.11.2.1 Prometheus Exporter 组件

2.11.2.1.1 概述

Prometheus是业内非常流行的开源监控系统，同时本身也是一款时序数据库。Prometheus的采集端被称为exporter，用来收集被监控模块的指标项。为了与Prometheus平台完成对接，AI工具自带了几款exporter，分别是用来采集数据库指标的opengauss-exporter，用来执行cmd命令并获取返回结果以及采集日志信息的cmd-exporter，以及对采集到的指标进行二次加工的reprocessing-exporter。

须知

- Prometheus和exporter是业内流行的监控和采集平台，部署在内网环境中，不对外部暴露接口，仅供内部监控平台使用。因此，为了增强该平台的安全性，一般需要用户或运维人员配置防火墙等，以便隔离外部访问，从而增强监控平台的安全性。
- Prometheus平台在默认情况下，采用Http协议、并且没有任何安全访问限制。这是因为，该平台一般部署在内网环境中，攻击风险可控。如果用户希望提高安全性，可自行修改Prometheus的TLS配置选项或增加basic_auth_users相关选项，但仍不建议对外部直接暴露访问接口。
- Prometheus默认绑定的IP地址是0.0.0.0，DBMind无法自动获知用户希望绑定的IP地址，故用户应该显式设置Prometheus的绑定IP地址，以减小安全风险。对于Prometheus的其他开源exporter（如node-exporter）也是同理。
- DBMind的exporter与Prometheus交互时，若使用SSL证书，则默认会进行双向认证。
- Prometheus的CA证书默认只支持SAN（Subject Alternative Name）形式，用户在配置Prometheus的CA证书，以及与Prometheus进行认证的证书时，需要使用SAN格式。同时，Prometheus不支持密码保护的私钥文件，用户在为Prometheus生成私钥文件时，请不要使用密码保护。例如，用户可以参考下述示例，通过openssl工具生成SAN形式的自签发证书和私钥文件：

  ```
  openssl req -new -newkey rsa:2048 -days 365 -nodes -x509 -keyout prometheus.key -out prometheus.crt -subj "/C=CN/ST=GaussDB/L=GaussDB/O=GaussDB/CN=localhost" -addext "subjectAltName = DNS:localhost"
  ```

  通过执行上述命令，可以生成两个文件prometheus.crt和prometheus.key，其中prometheus.crt文件可以同时作为CA证书使用。
- DBMind的exporter默认支持HTTPS协议，需要用户手动指定证书文件路径，建议用户使用。
- DBMind Service的部分功能会从Prometheus中获取时序数据，该通道也可以使用SSL协议进行加固，prometheus默认支持https通信，建议用户使用。

2.11.2.1.2 环境部署

用户可以从Prometheus的官网上下载Prometheus-server和node-exporter，然后根据官方文档中的说明启动它们；

其中，提供的exporter组件默认采用Https通信协议，因此需要用户默认提供SSL证书和密钥文件，并通过“--ssl-keyfile”、“--ssl-certfile”以及“--ssl-ca-file”选项进行配置。若用户不希望使用HTTPS协议，则可以通过“--disable-https”选项禁用该模式。

版权所有 © 华为云计算技术有限公司

说明

由于GaussDB默认模式下的通信加密协议与PostgreSQL不兼容，故导致通过PyPI源安装的基于PostgreSQL编译的Python驱动psycopg2-binary默认无法连接至数据库。

因此，需要用户自行编译psycopg2或修改GUC参数进行适配。也可通过openGauss官方网站下载基于GaussDB编译的psycopg2（官方网站仅提供部分Python版本的编译包，需要用户鉴别是否与当前Python运行时版本一致）。

2.11.2.1.3 使用指导

用户可通过gs_dbmind命令启动对应的exporter。下面为用户演示一个完整的Prometheus监控平台的搭建过程。

步骤1 部署Prometheus主进程，运行如下命令：

```
prometheus --config.file=prometheus.yml
```

步骤2 部署openGauss-exporter：启动openGauss-exporter，采用默认侦听端口号9187，侦听地址为192.168.1.100，采用HTTPS协议，则命令可以为：

```
gs_dbmind component opengauss_exporter --url postgresql://user:password@ip:port/dbname --web.listen-address 192.168.1.100 --ssl-keyfile server.key --ssl-certfile server.crt --ssl-ca-file server.crt
```

步骤3 部署reprocessing-exporter：启动reprocessing-exporter，采用默认侦听端口号8181，侦听地址为192.168.1.101，Prometheus-server IP与端口号为192.168.1.100:9090，采用HTTPS协议，则命令可以为：

```
gs_dbmind component reprocessing_exporter 192.168.1.100 9090 --web.listen-address 192.168.1.101 --ssl-keyfile server.key --ssl-certfile server.crt --ssl-ca-file server.crt
```

步骤4 部署cmd-exporter: 启动cmd-exporter，使用默认参数，设置采集的数据库日志目录路径，并给定证书信息：

```
gs_dbmind component cmd_exporter --ssl-keyfile server.key --ssl-certfile server.crt --ssl-ca-file server.crt --pg-log-dir /path/to/pglog
```

步骤5 部署node-exporter：一般而言，Prometheus监控平台都需要部署node-exporter用于监控Linux操作系统，后文提到的部分AI功能也需要依赖node-exporter采集Linux系统指标，故也需要用户来部署；使用方法详见：https://prometheus.io/docs/guides/node-exporter/#installing-and-running-the-node-exporter。用户可直接运行该node-exporter进程，其默认端口号为9100，启动命令行为：

```
node_exporter
```

----结束

注意

DBMind高可用功能需要在未纳管实例的情况下实现，因此接口不需要token认证。应当使用默认的HTTPS协议以进行SSL双向认证，使用HTTP协议则可能引入攻击者直接发起未授权请求的安全风险。

为保证DBMind云上使用时高可靠，exporter组件提供了组件状态查询和部分异常修复接口，详细请参见**表2-23**。

- 组件状态查询接口示例：

  ```
  curl -X 'POST'"http://127.0.0.1:8080/v1/api/check-status" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"cmd": "./gs_dbmind component cmd_exporter --web.listen-address 0.0.0.0 --web.listen-port 9181 --log.filepath /xxx/dbmind_cmd_exporter.log --disable-https"}'
  ```

  如果使用HTTPS协议，则查询示例为：

  ```
  curl -X 'POST'"https://127.0.0.1:8080/v1/api/check-status" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"cmd": "./gs_dbmind component cmd_exporter --web.listen-address 0.0.0.0
  ```


版权所有 © 华为云计算技术有限公司

```
--web.listen-port 9181 --log.filepath /xxx/dbmind_cmd_exporter.log --ssl-keyfile xx.key --ssl-certfile xx.crt --ssl-ca-file xx.crt"}' --cacert xx.crt --key xx.key --cert xx.crt
```

返回结果示例：

```
{"data":{"error_msg":"","result":{},"state":"NORMAL"},"success":true}
```

- 组件异常修复接口示例：

```
curl -X 'POST'"http://127.0.0.1:8080/v1/api/repair" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"cmd": "./gs_dbmind component cmd_exporter --web.listen-address 0.0.0.0 --web.listen-port 9181 --log.filepath /xxx/dbmind_cmd_exporter.log --disable-https"}'
```

如果使用HTTPS协议，则修复示例为：

```
curl -X 'POST'"https://127.0.0.1:8080/v1/api/repair" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"cmd": "./gs_dbmind component cmd_exporter --web.listen-address 0.0.0.0 --web.listen-port 9181 --log.filepath /xxx/dbmind_cmd_exporter.log --ssl-keyfile xx.key --ssl-certfile xx.crt --ssl-ca-file xx.crt"}' --cacert xx.crt --key xx.key --cert xx.crt
```

返回结果示例：

```
{"data":{"error_msg":"","result":{},"state":"SUCCESS"},"success":true}
```

 版权所有 © 华为云计算技术有限公司

说明

exporter启动后，会根据yamls目录下的配置文件信息采集特定的指标。指标的配置信息如下例所示：

```
indicator_name:
  name: indicator_name
  desc: xxx
  query:
    - name: indicator_name
      sql: xxx
      version: xxx
      timeout: 10
      status: enable
  metrics:
    ...
  status: enable
  ttl: 0
  timeout: 10
```

当status值为enable时，会在指定的超时时间内对该指标进行采集。用户可以对默认开启状态status和超时时间timeout进行修改，修改后需重新启动exporter才能生效。

组件状态查询和修复接口只针对DBMind自研exporter，包括opengauss_exporter、cmd_exporter、reprocessing_exporter，对于开源的node_exporter则不支持。

当前Exporter组件支持查询的异常场景如下：

- PID文件丢失或文件内容异常：当用户启动服务/组件后，会自动生成*.pid文件来记录进程对应的进程标识符，也就是PID。PID文件用于云管控来判断进程状态，发现进程级别的异常并进行修复。为防止PID文件的误删除与内容的误修改，需要定期判断PID文件是否存在以及其内容是否正确。
- 日志文件丢失：当用户启动服务/组件后，会自动生成*.log文件来记录进程运行的日志。此日志文件用于快速定位问题的根源、追踪程序执行的过程等。为防止日志文件的误删除，需要定期判断日志文件是否存在。
- 资源占用异常：DBMind服务会执行索引推荐、指标分析、智能巡检等功能，需要占用系统资源；同样的，exporter组件会采集服务器上的指标，用于后续分析，也会占用系统资源；当服务/组件的CPU或内存占用超过阈值时，说明资源占用过高，服务/组件出现异常。因此需要监控服务和组件的资源占用情况。
- 数据库异常：opengauss_exporter需要从数据库中获取数据，用于生成采集结果，所以需要判断数据库是否能够正常连接。

资源占用阈值及告警设置如下：

- 内存占用阈值：max(总内存的1%, 200MB)。
- CPU多核占用阈值：cmd_exporter 10%，opengauss_exporter 10%，reprocessing_exporter 10%， DBMind 80%。
- CPU单核占用阈值：cmd_exporter 50%，opengauss_exporter 50%，reprocessing_exporter 50%，DBMind不做限制。
- 调用高可用接口会检查资源占用，连续超过3次阈值才会触发资源占用告警。

风险：当占用资源连续超过阈值三次时，会触发告警，管控侧会重启服务，导致正在运行的任务中断。

当前Exporter针对组件异常的修复性说明如下：

- PID文件丢失：可自动修复；
- 日志文件丢失：可自动修复；
- 组件资源占用异常：不可通过接口修复；
- 数据库异常：不可通过接口修复；
- 针对Exporter组件不可修复的异常，需要云侧进行处理，如重启进程。

 版权所有 © 华为云计算技术有限公司

⚠ 注意

- openGauss-exporter中连接数据库的用户需要monitor admin或以上权限，否则会出现部分指标无法采集的情况。同时openGauss-exporter不支持使用数据库初始用户来进行数据采集。
- openGauss-exporter中连接数据库的用户需要获取dbe_perf模式下的数据，因此需要保证其具有该模式的权限。
- openGauss-exporter会从dbe_perf.statement_history中抽样慢SQL信息，dbe_perf.statement_history视图慢SQL记录与GUC参数log_min_duration_statement和track_stmt_stat_level相关，其中log_min_duration_statement是慢SQL阈值，单位毫秒，具体值由用户设置；track_stmt_stat_level是SQL记录级别，默认为'OFF,L0'，即只记录慢SQL信息，级别为L0，建议用户在详细了解参数意义与作用的情况下谨慎修改。
- openGauss-exporter采集数据库相关信息，主要包括部分系统表和视图中的数据（具体参见代码中opengauss_exporter中的配置文件）。node-exporter采集系统指标信息，主要与系统磁盘、CPU等相关。reprocessing_exporter基于prometheus-server中的某些指标（具体参见代码中reprocessing_exporter中的配置文件）进行二次加工，最终提供加工后的数据供用户使用。
- 当数据库部署在多层网络上时（集中式：管理层-数据层），openGauss-exporter的启动命令中的url参数必须使用数据层的ip地址，否则会影响DBMind的其他功能。
- prometheus-server在拉取exporter数据时有超时机制，超时时间由scrape_timeout（默认10s）控制，因此当exporter采集数据量较大时，用户可根据实际情况增大scrape_timeout以防止超时报错，另外需要注意的是scrape_interval（采集间隔，默认15s）不能比scrape_timeout小，否则会出现异常。
- 如果数据库时区设置和系统不相同，可能会出现时间相关指标时间与系统时间不一致的情况，因此需要将数据库时区与系统保持同步。
- 当使用https通信时，工具会检测证书与密钥文件权限以及证书有效期，如果文件权限大于600则会出现报警，证书有效期小于90天会出现报警。
- 当存在指标重复采集时，openGauss-exporter会出现异常，异常信息会打印到日志中。
- 使用openGauss-exporter的--config、--disable-settings-metrics、--disable-statement-history-metrics三个参数时需要注意，其存在以下几种情况：
  - 用户不指定其中任何参数，则工具会同时对yamls目录下的三个配置文件中的指标进行采集。
  - 用户显式指定--config，则工具不会采集yamls目录下default.yml中的指标，而会采集用户指定配置文件中的指标，同时pg_settings.yml和statements.yml正常采集，此时用户需要注意指定的配置文件中的指标和pg_settings.yml、statements.yml中的指标不能存在重复采集的现象。
  - 用户显式指定--disable-settings-metrics，则工具不会采集yamls目录下pg_settings.yml中的指标；用户显式指定--disable-statement-history-metrics，则工具不会采集yamls目录下statements.yml（慢SQL相关）中的指标。
- exporter启动后，一般情况下不会终止进程并退出（如连接的数据库地址不可用、连接的数据库用户被删除或禁用等），而是会在日志中记录报错信息，并在后台重新尝试。
- cmd_exporter默认Linux系统定义了需要监控的gaussDB的日志路径：$GAUSSLOG，如果没有定义$GAUSSLOG则需要进行定义。

版权所有 © 华为云计算技术有限公司

- cmd_exporter默认匹配的是GUC参数“log_statement='none'”时的日志格式，当log_statement为其他参数时cmd_exporter会无法识别日志格式。
- 由于cmd_exporter需要访问数据库日志和使用cm工具，所以需要由数据库用户部署cmd_exporter来获取相应的状态信息。
- 实例升级过程中，opengauss_exporter无法正常采集数据，cmd_exporter无法采集数据库的cm相关信息。
- cmd_exporter的日志采集功能对于内核日志的日志文件名和日志格式都有要求，在gaussdb.conf文件中，log_filename的格式要求为'gaussdb-%Y-%m-%d_%H%M%S.log'，log_line_prefix的格式要求为'%m %n %u %d %h %p %S %x %a'。
- cmd_exporter采集的日志指标为散点图形式，各个采集点之间标签不同，所以不能合并到同一个集合中，不适合用折线图的方式进行展示，请采用表格或者散点图的方式进行展示。
- 当cmd_exporter在一个采集周期内没有采集到日志指标时（如gaussdb_log_ffic为空，则表示没有生成ffic文件），则该指标数据为空。
- 在某些特殊情况下由于日志文件激增，考虑到cpu的处理能力和内存限制，cmd_exporter为日志的消息队列设置了长度上限以防止内存泄露。当队列满时将停止新的日志事件入列直到队列有空间为止。这一设计可能导致在日志负载高的时候丢失部分日志指标。
- reprocessing_exporter中采集指标os_disk_usage时，只针对EXT和XFS文件系统，其他文件系统不会采集。

2.11.2.1.4 获取帮助

用户可以通过“--help”命令获取Exporter组件启动或停止等操作的帮助信息，例如：

```
gs_dbmind component opengauss_exporter --help
gs_dbmind component reprocessing_exporter --help
gs_dbmind component cmd_exporter --help
```

Exporter组件状态查询和修复的RESETful API列表如下：

API	入参	参数介绍	请求方法	功能描述与预期返回结果
/v1/api/check-status	cmd	组件启动命令，String，必选	POST	获取exporter组件的状态信息并返回状态详情
/v1/api/repair	cmd	组件启动命令，String，必选	POST	修复exporter组件并返回修复结果

2.11.2.1.5 命令参考

reprocessing-exporter的使用帮助详情：

```
gs_dbmind component reprocessing_exporter --help
usage:  [-h] [--prometheus-auth-user PROMETHEUS_AUTH_USER]
        [--prometheus-auth-password PROMETHEUS_AUTH_PASSWORD]
        [--disable-https] [--ssl-keyfile SSL_KEYFILE]
        [--ssl-certfile SSL_CERTFILE] [--ssl-ca-file SSL_CA_FILE]
        [--tsdb-ssl-keyfile TSDB_SSL_KEYFILE] [--tsdb-ssl-certfile TSDB_SSL_CERTFILE]
        [--tsdb-ssl-ca-file TSDB_SSL_CA_FILE]
        [--web.listen-address WEB.LISTEN_ADDRESS]
        [--web.listen-port WEB.LISTEN_PORT]
        [--collector.config COLLECTOR.CONFIG] [--log.filepath LOG.FILEPATH]
```

 版权所有 © 华为云计算技术有限公司

```
      [--log.level {debug,info,warn,error,fatal}] [-v]
      prometheus_host prometheus_port

Reprocessing Exporter: A re-processing module for metrics stored in the
Prometheus server.

positional arguments:
  prometheus_host       from which host to pull data
  prometheus_port       the port to connect to the Prometheus host

optional arguments:
  -h, --help            show this help message and exit
  --prometheus-auth-user PROMETHEUS_AUTH_USER
                        use this user for basic authorization to connect to
                        the Prometheus server
  --prometheus-auth-password PROMETHEUS_AUTH_PASSWORD
                        use this password for basic authorization to connect
                        to the Prometheus server
  --disable-https       disable Https scheme
  --ssl-keyfile SSL_KEYFILE
                        set the path of ssl key file
  --ssl-certfile SSL_CERTFILE
                        set the path of ssl certificate file
  --ssl-ca-file SSL_CA_FILE
                        set the path of ssl ca file
  --tsdb-ssl-keyfile TSDB_SSL_KEYFILE
                        set the path of tsdb ssl key file
  --tsdb-ssl-certfile TSDB_SSL_CERTFILE
                        set the path of tsdb ssl certificate file
  --tsdb-ssl-ca-file TSDB_SSL_CA_FILE
                        set the path of tsdb ssl ca file
  --web.listen-address WEB.LISTEN_ADDRESS
                        address on which to expose metrics and web interface
  --web.listen-port WEB.LISTEN_PORT
                        listen port to expose metrics and web interface
  --collector.config COLLECTOR.CONFIG, --config COLLECTOR.CONFIG
                        according to the content of the yaml file for metric
                        collection
  --log.filepath LOG.FILEPATH
                        the path to log
  --log.level {debug,info,warn,error,fatal}
                        only log messages with the given severity or above.
                        Valid levels: [debug, info, warn, error, fatal]
  -v, --version         show program's version number and exit
```

表 2-20 reprocessing-exporter 的命令行参数详情表

参数	参数说明	取值范围
prometheus_host	Prometheus-server的IP地址。	-
prometheus_port	Prometheus-server的服务侦听端口号。	1024-65535。
-h, --help	帮助选项。	-
--prometheus-auth-user	prometheus用户名称。	-
--prometheus-auth-password	prometheus用户密码。	-
--disable-https	禁用HTTPS协议。	-

 版权所有 © 华为云计算技术有限公司

参数	参数说明	取值范围
--ssl-keyfile	HTTPS协议使用的证书私钥文件路径，如果为密文私钥，需要通过管道传入私钥密码，传输内容为json格式，密码填充在ssl-keyfile-password字段。如：`echo {"ssl-keyfile-password":"password"} | gs_dbmind component reprocessing_exporter ...` 。	-
--ssl-certfile	HTTPS协议使用的证书文件路径。	-
--ssl-ca-file	HTTPS协议使用的CA证书文件路径。	-
--tsdb-ssl-keyfile	TSDB数据库使用的HTTPS协议证书私钥文件路径。	-
--tsdb-ssl-certfile	TSDB数据库使用的HTTPS协议证书文件路径。	-
--tsdb-ssl-ca-file	TSDB数据库使用的HTTPS协议CA证书文件路径。	-
--web.listen-address	该exporter服务的绑定IP。	-
--web.listen-port	该exporter服务的侦听端口。	1024-65535。
--collector.config	显性指定的待采集指标配置文件路径。	-
--log.filepath	日志文件保存路径，默认保存在当前目录下。	-
--log.level	日志文件的打印级别，默认为INFO级别。	debug、info、warn、error、fatal。
--version	显示版本信息。	-

openGauss-exporter的使用帮助详情：

```
gs_dbmind component opengauss_exporter --help
usage:  [-h] --url URL [--config-file CONFIG_FILE]
        [--include-databases INCLUDE_DATABASES]
        [--exclude-databases EXCLUDE_DATABASES]
        [--constant-labels CONSTANT_LABELS]
        [--scrape-interval-seconds SCRAPE_INTERVAL_SECONDS]
        [--web.listen-address WEB.LISTEN_ADDRESS]
        [--web.listen-port WEB.LISTEN_PORT] [--disable-cache]
        [--disable-settings-metrics] [--disable-statement-history-metrics]
        [--disable-https] [--disable-agent] [--ssl-keyfile SSL_KEYFILE]
        [--ssl-certfile SSL_CERTFILE] [--ssl-ca-file SSL_CA_FILE]
```


版权所有 © 华为云计算技术有限公司

```
      [--parallel PARALLEL] [--connection-pool-size CONNECTION_POOL_SIZE]
      [--log.filepath LOG.FILEPATH]
      [--log.level {debug,info,warn,error,fatal}] [-v]

openGauss Exporter (DBMind): Monitoring or controlling for openGauss.

optional arguments:
 -h, --help            show this help message and exit
 --url URL, --dsn URL  openGauss database target url. It is recommended to
                       connect to the postgres database through this URL, so
                       that the exporter can actively discover and monitor
                       other databases.
 --config-file CONFIG_FILE, --config CONFIG_FILE
                       path to config file.
 --include-databases INCLUDE_DATABASES
                       only scrape metrics from the given database list. a
                       list of database name (format is label=dbname or
                       dbname) separated by comma(,).
 --exclude-databases EXCLUDE_DATABASES
                       scrape metrics from the all auto-discovered databases
                       excluding the list of database. a list of database
                       name (format is label=dbname or dbname) separated by
                       comma(,).
 --constant-labels CONSTANT_LABELS
                       a list of label=value separated by comma(,).
 --scrape-interval-seconds SCRAPE_INTERVAL_SECONDS
                       specify the scrape interval in seconds to reduce
                       redundant results. If set 0, it means automatically
                       calculate.
 --web.listen-address WEB.LISTEN_ADDRESS
                       address on which to expose metrics and web interface
 --web.listen-port WEB.LISTEN_PORT
                       listen port to expose metrics and web interface
 --disable-cache       force not using cache.
 --disable-settings-metrics
                       not collect pg_settings.yml metrics.
 --disable-statement-history-metrics
                       not collect statement-history metrics (including slow
                       queries).
 --disable-https       disable Https scheme
 --disable-agent       by default, this exporter also assumes the role of
                       DBMind-Agent, that is, executing database operation
                       and maintenance actions issued by the DBMind service.
                       With this argument, users can disable the agent
                       functionality, thereby prohibiting the DBMind service
                       from making changes to the database.
 --ssl-keyfile SSL_KEYFILE
                       set the path of ssl key file
 --ssl-certfile SSL_CERTFILE
                       set the path of ssl certificate file
 --ssl-ca-file SSL_CA_FILE
                       set the path of ssl ca file
 --parallel PARALLEL   number of parallels for metrics scrape.
 --connection-pool-size CONNECTION_POOL_SIZE
                       size of connection pool for each database. Set zero to
                       disable connection pool.
 --log.filepath LOG.FILEPATH
                       the path to log
 --log.level {debug,info,warn,error,fatal}
                       only log messages with the given severity or above.
                       Valid levels: [debug, info, warn, error, fatal]
 -v, --version         show program's version number and exit
```

 版权所有 © 华为云计算技术有限公司

表 2-21 openGauss-exporter 的命令行参数详情表

参数	参数说明	取值范围
--url	数据库server的连接地址，例如 postgres://user:pwd@host:port/dbname。 密码字段为空时，需要通过管道密码，传输内容为json格式，密码填充在db-password字段。如：`echo {"db-password":"password"} | gs_dbmind component opengauss_exporter ...`。	如果该url涉及到的各字段URL包含特殊字符（如@，/等），则需要通过URL编码进行转义，例如密码中的"@"应转义为%40，"/"应转义为%2F，否则各字段的含义会被错误识别和切分，具体转义规则可以参考URL编码的转义规则，该URL地址规则遵循RFC-1738标准。
--constant-labels	常量列表，k=v格式，用逗号隔开，表明该exporter自带的常量标签。	格式如“cluster_name=demo,cluster_id=1”。
-h, --help	帮助选项。	-
--disable-https	禁用HTTPS协议。	-
--ssl-keyfile	HTTPS协议使用的证书私钥文件路径，如果为密文私钥，需要通过管道传入私钥密码，传输内容为json格式，密码填充在ssl-keyfile-password字段。如：`echo {"ssl-keyfile-password":"password"} | gs_dbmind component opengauss_exporter ...`。	-
--ssl-certfile	HTTPS协议使用的证书文件路径。	-
--ssl-ca-file	HTTPS协议使用的CA证书文件路径。	-
--web.listen-address	该exporter服务的绑定IP。	-
--web.listen-port	该exporter服务的侦听端口。	1024-65535。
--config, --config-file	显性指定的待采集指标配置文件路径。	-
--log.filepath	日志文件保存路径，默认保存在当前目录下。	-
--log.level	日志文件的打印级别，默认为INFO级别。	debug、info、warn、error、fatal。

版权所有 © 华为云计算技术有限公司

参数	参数说明	取值范围
--version	显示版本信息。	-
--disable-cache	禁止使用缓存。	-
--disable-settings-metrics	禁止采集pg_settings表的值。	-
--disable-statement-history-metrics	禁止采集statement_history表中的慢SQL信息。	-
--disable-agent	禁止agent行为。	-
--include-databases	显性表明待采集的数据库名，指定多个数据库时用逗号（,）隔开。	-
--exclude-databases	显性表明不采集的数据库名，指定多个数据库时用逗号（,）隔开。	-
--parallel	指标采集的并行线程数，默认为5。	正整数。
--scrape-interval-seconds	明确抓取间隔避免重复采集指标，默认为0。	>=0的整数，如果为0，则根据上一次采集的时间对采集间隔进行动态调整。
--connection-pool-size	数据库的连接池大小，默认为0。	>=0的整数，如果为0，则不采用连接池连接。

cmd-exporter的使用帮助详情：

```
gs_dbmind component cmd_exporter --help
usage:  [-h] [--constant-labels CONSTANT_LABELS] [--web.listen-address WEB.LISTEN_ADDRESS] [--
web.listen-port WEB.LISTEN_PORT] [--disable-https]
       [--config CONFIG] [--ssl-keyfile SSL_KEYFILE] [--ssl-certfile SSL_CERTFILE] [--ssl-ca-file SSL_CA_FILE]
[--parallel PARALLEL]
       [--pg-log-dir PG_LOG_DIR] [--disable-log-exporter] [--log.filepath LOG.FILEPATH] [--log.level
{debug,info,warn,error,fatal}] [-v]

Command Exporter (DBMind): scrape metrics by performing shell commands.

optional arguments:
  -h, --help            show this help message and exit
  --constant-labels CONSTANT_LABELS
                        a list of label=value separated by comma(,).
  --web.listen-address WEB.LISTEN_ADDRESS
                        address on which to expose metrics and web interface
  --web.listen-port WEB.LISTEN_PORT
                        listen port to expose metrics and web interface
  --disable-https       disable Https scheme
  --config CONFIG       path to config dir or file.
  --ssl-keyfile SSL_KEYFILE
                        set the path of ssl key file
  --ssl-certfile SSL_CERTFILE
                        set the path of ssl certificate file
  --ssl-ca-file SSL_CA_FILE
                        set the path of ssl ca file
```


版权所有 © 华为云计算技术有限公司

```
--parallel PARALLEL   performing shell command in parallel.
--pg-log-dir PG_LOG_DIR
                      set the directory path of PGLOG, default value is  $GAUSSLOG.
--disable-log-exporter
                      disable log analysis
--log.filepath LOG.FILEPATH
                      the path to log
--log.level {debug,info,warn,error,fatal}
                      only log messages with the given severity or above. Valid levels: [debug, info, warn, error,
fatal]
-v, --version         show program's version number and exit
```

表 2-22 cmd-exporter 的命令行参数详情表

参数	参数说明	取值范围
-h, --help	帮助选项。	-
--disable-https	禁用HTTPS协议。	-
--ssl-keyfile	HTTPS协议使用的证书私钥文件路径，如果为密文私钥，需要通过管道传入私钥密码，为json格式，ssl-keyfile-password字段。如：`echo {"ssl-keyfile-password":"password"} | gs_dbmind component cmd_exporter ...`。	-
--ssl-certfile	HTTPS协议使用的证书文件路径。	-
--ssl-ca-file	HTTPS协议使用的CA证书文件路径。	-
--web.listen-address	该exporter服务的绑定IP。	-
--web.listen-port	该exporter服务的侦听端口。	1024-65535。
--config	显性指定的待采集指标配置文件路径。	默认是该功能yamls目录下的default.yml文件，可以参考该配置文件格式，错误配置会报错。
--log.filepath	日志文件保存路径，默认保存在当前目录下。	-
--log.level	日志文件的打印级别，默认为INFO级别。	debug、info、warn、error、fatal。
--parallel	并行执行shell命令的并发度。	正整数。
--pg-log-dir	日志侦听路径，默认路径为“${GAUSSLOG}/gs_log”。	-
--disable-log-exporter	禁用日志采集功能。	-

 版权所有 © 华为云计算技术有限公司

参数	参数说明	取值范围
--constant-labels	常量列表，k=v格式，用逗号隔开，表明该exporter自带的常量标签。	格式如“cluster_name=demo,cluster_id=1”。
--version	显示版本信息。	-

说明

当启用日志监测功能时，当监控的日志路径发生变动时，请重启cmd_exporter进程。

cmd_exporter进程对容器化环境部分支持。

2.11.2.1.6 常见问题处理

1. 提示需要用户提供--ssl-keyfile与--ssl-certfile选项：

   上述exporter默认采用HTTPS模式通信，因此需要用户指定证书及其私钥文件的路径。相反，如果用户只想采用Http模式，则需要显性指定--disable-https选项，从而禁用HTTPS协议。

2. 提示用户需要输入PEM密码（Enter PEM pass phrase）：

   如果用户采用Https模式，并给定了证书及其密钥文件的路径，且该密钥文件是经过加密的，则需要用户输入该加密私钥证书文件的密码。该密码也可以通过标准输入流传递。

   说明

   prometheus不支持私钥加密，若涉及prometheus通信的私钥加密时部分功能将无法实现。

   prometheus的query_range功能支持的最大序列长度是11000个数据，在使用时需要注意。

   prometheus的query_range功能可能会出现单点数据同时出现在连续两次查询结果中，此为正常现象。

2.11.3 DBMind 的 AI 子功能

用户可以通过gs_dbmind的component子命令启动对应的AI子功能，下述章节展示不同AI功能的具体内容和使用详情。

DBMind进程启动后，其提供的Http(s)接口信息如下：

版权所有 © 华为云计算技术有限公司

表 2-23 DBMind Restful API 列表

API	请求方法	功能描述	参数	返回值	示例
/v1/api/token	POST	获取数据库访问令牌	param username：必选，用户名 param grant_type：可选，授权类型 param password：必选，密码 param scope：必选，数据库实例 param client_id：可选，客户端id param client_secret：可选，客户端密钥	返回token	{"access_token":"qmWWgt28VSu0YAH5","token_type":"bearer","expires_in":600}
/v1/api/agents	GET	获取所有代理列表	无	返回所有代理列表	{"data":{"ip1":["ip1","ip2","ip3"]}
/v1/api/agents	PUT	更新代理信息（包含强制更新），由于数据库实例状态的更改需要一段时间，因此强制更新默认延迟15秒执行。	param force：可选，强制更新	返回更新代理结果	{"data":true,"success":true}
/v1/api/configs/dynamic-config	PUT	设置DBMind配置	param config：必选，配置项分类 param name：必选，配置项名称 param value：必选，配置项取值	返回更新配置结果	{"data":"success","success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/configs/dynamic-config	GET	获取DBMind配置	param config：必选，配置项分类 param name：必选，配置项名称	返回配置信息	{"data":"50","success":true}
/v1/api/configs/dynamic-config/list	GET	获取DBMind配置列表	param is_default：可选，默认为false。取值为false时从元数据库中获取配置信息；否则从本地配置文件中获取配置信息	返回配置信息列表	{"data":{"dynamic":{"detection_params":[["esd_test_alpha","0.05","The Significance level."]...]}...},"success":true}
/v1/api/configs/anomaly-detection/default-settings	GET	获取异常检测默认设置	无	返回异常检测默认设置信息	{"data":{"AlarmInfo":{"alarm_cause":null,"alarm_content":null,...,"window":10}},"success":true}
/v1/api/status/schedulers	GET	获取定时任务状态更新	无	返回定时任务的状态	{"data":{"header":["name","current_status","running_interval"],"rows":[["anomaly_detection","Running",180]...]},"success":true}
/v1/api/status/collection-system	GET	获取流水采集状态	param exporter_type：必选，exporter类型，取值范围：('opengauss_exporter','cmd_exporter','reprocessing_exporter') param ssl_context：必选，SSL上下文信息	返回采集器状态	{"data":{"header":["component","listen_address","is_alive"],"rows":[["opengauss_exporter","ip",true]...],"suggestions":[]},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/status/data-directory	GET	获取数据库数据目录状态	param instance：可选，实例节点的地址 param latest_minutes：可选，接口会统计从请求时间往前推latest_minutes分钟的数据，默认为5	返回数据库目录状态	{"data":{"free_space":2161.57,"tilt_rate":0.03,"total_space":3298.17...},"success":true}
/v1/api/status/overview	GET	获取数据库概览信息	param latest_minutes：可选，接口会统计从请求时间往前推latest_minutes分钟的数据，默认为3	返回数据库概览信息	{"data":{"deployment_mode":"centralized",...,"strength_version":"openGauss 2.1.0..."},"success":true}
/v1/api/status/instances	GET	获取数据库实例状态	无	返回数据库实例状态	{"data":{"header":["instance","role","state"],"rows":[["ip:port,","primary",fawulse]...]},"success":true}
/v1/api/status/agents	GET	获取数据库代理状态	无	返回数据库代理状态	{"data":{"agent_address":"ip","status":true},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/app/slow-sql-rca	GET	获取慢SQL的根因分析结果	param query：必选，SQL查询语句，字符串长度需小于10240 param db_name：必选，数据库名称 param schema_name：可选，schema名称，默认为public param start_time：可选，13位时间戳，建议传递，开始时间 param finish_time：可选，13位时间戳，建议传递，结束时间 param template_id：可选，建议传递，归一化SQL ID param debug_query_id：可选，建议传递，唯一SQL ID param n_soft_parse：可选，软解析次数 param n_hard_parse：可选，硬解析次数param query_plan: 建议传递，执行计划 param n_returned_rows：可选，结果中元组个数	返回慢SQL的根因分析结果	{"data":[true,"public","Seq Scan on t2 (cost=0.00..3.25 rows=1 width=70)\n Filter: (c1 = 'dadsadasds'::text)\n",[[["1. MISSING_INDEXES: (0.53) Missing required index.","2. HEAVY_SCAN_OPERATOR: (0.47) Existing expensive seq scans. Detail: (name: Seq Scan on t2, parent: None, rows:t2(100), cost rate: 100.0%)"]],[["1. Recommended index: (schema: public, index: t2(c1)).","2. According to business adjustments, try to avoid it"]]]],"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
			param n_tuples_fetched：可选，随机扫描行数 param n_tuples_returned：可选，顺序扫描行数 param n_tuples_inserted：可选，插入行数 param n_tuples_updated：可选，更新行数 param n_tuples_deleted：可选，删除行数 param n_blocks_fetched：可选，buffer的块访问次数 param n_blocks_hit：可选，buffer的块命中次数 param db_time：可选，有效的DB时间花费，单位：微秒 param cpu_time：可选，CPU时间，单位：微秒 param parse_time：可选，解析时间，单位：微秒 param plan_time：可		**说明** 详细参见**注意事项说明**。 • 当用户传入归一化QUERY语句时，内部会使用PBE方式获取执行计划，如果获取失败则不会返回执行计划内容。 • 如果用户执行诊断的用户权限不足，则无法正常返回结果。 • 接口响应时长受数据库负载和资源影响。 • 建议传入query_plan参数，避免内部获取。 • 如果传入的QUERY存在截断且没有传入query_plan参数，则无法进行诊断。 • 接口只支持一条QUERY语句，如果传入多条则只会诊断第一条QUERY。 • 如果用户不传schema信息，则schema默认为PUBLIC。 • 当前只支持DML语句。

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
			选，执行时间，单位：微秒 param data_io_time：可选，IO时间，单位：微秒 param hash_spill_count：可选，hash过程中，若发生落盘，写文件的次数 param sort_spill_count：可选，sort过程中，若发生落盘，写文件的次数 param n_calls：可选，调用次数 param lock_wait_time：可选，加锁等待耗时 param lwlock_wait_time：可选，轻量级加锁时间 param tz：可选，若存在时区差异则建议传递，慢SQL时区信息，只支持UTC标准		

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/app/correlation	GET	获取性能指标相关性分析结果	param metric_name：可选，metric名称 param instance：可选，实例地址 param start_time：可选，开始时间的时间戳 param end_time：可选，结束时间的时间戳 注：此处start_time和end_time是两个独立参数，并非为选取时间窗口，所以不强制要求start_time早于end_time param metric_filter：可选，metric的标签。 支持如下两种格式： ● “k1=v1,k2=v2”的kv pairs格式 ● “{'k1':'v1','k2':'v2'}”的json字符串格式	返回异常关联指标及其时序数据	{"data": {"os_mem_usage from ip": [["gs_total_memory_detail_mbytes{\"node name\": \"dn_6001_6002\", \"type\": \"dynamic_used_memory\"} from ip",1.0,0, [0.32950820234487...]]...]},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/app/memory-check	GET	获取内存占用情况	param latest_hours：可选，默认为0	返回当前数据库实例内存使用情况及其可能的风险	{"data": {"timestamps": [1684838536049],"values": [4618.0]},"remark":"too little data...","status":"unknown"},...,"topk_session_memory_sql": [{"application_name":"JobScheduler"...}...]},"success":true}
/v1/api/app/risk-analysis/{metric}	GET	获取风险分析结果	param metric：必选，metric名称 param instance：可选，实例地址 param warning_hours：可选，风险分析时长 param upper：可选，metric的上限 param lower：可选，metric的下限 param labels：可选，metric的标签。支持如下两种格式： ● “k1=v1,k2=v2”的kv pairs格式 ● “{'k1':'v1','k2':'v2'}”的json字符串格式 ● param tz：可选，时区参数，只支持UTC标准	返回当前指标未来风险分析结果	{"data": {},"success":true} **说明** 1. 该接口会基于历史数据对指标未来变化趋势进行预测，预测时会取三倍于warning_hours的历史数据作为训练数据，以预测未来metric的变化趋势，因此如果warning_hours取值过小导致历史数据不足，可能会导致训练数据过少而无法进行训练，最终无正常结果输出，另外大于等于48小时的预测结果可能会不准确。 2. 该接口适合预测具有趋势和周期变化的数据。 3. 当该接口中没有传递labels参数进行过滤时，如果满足条件的指标过多可能会导致接口超时。 4. 该预测功能基于AI模型，因此每次预测结果可能存在部分差异。

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例

/v1/api/app/anomaly-detection/detectors/{name}	PUT	添加新的异常检测检测器或修改当前检测器	param name：必选，检测器的名称
			需要传入json结构体:
			{
			"duration": 10,
			"forecasting_seconds": 0,
			"alarm_info": {
			"alarm_type": "SYSTEM",
			"alarm_level": "ERROR"
			},
			"detector_info": [
			{
			"detector_name": "EsdTestDetector",
			"metric_name": "gaussdb_blks_hit_ratio",
			"detector_kwargs": {
			"alpha": 0.05
			}
			}
			]
			}
			detector_info字段下检测算法的可选范围及对应检测参数，见表格下方说明
				返回添加状态
				成功返回Success: add {检测器名称} for {数据库实例IP}
				失败返回Failed: add {检测器名称} for {数据库实例IP}
					{"data":"Success: add detector for IP","success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/app/anomaly-detection/detectors/{name}	DELETE	删除指定的异常检测检测器	param name：必选，检测器的名称	返回删除状态	{"data":"Success: delete detector for IP","success":true}
/v1/api/app/anomaly-detection/detectors/{name}/pause	PUT	暂停指定的异常检测检测器	param name：必选，检测器的名称	返回暂停状态	{"data":"Success: pause detector for IP","success":true}
/v1/api/app/anomaly-detection/detectors/{name}/resume	PUT	继续指定的异常检测检测器	param name：必选，检测器的名称	返回继续状态	{"data":"Success: resume detector for IP","success":true}
/v1/api/app/anomaly-detection/detectors/{name}	GET	查看指定的异常检测检测器信息	param name：必选，检测器的名称	返回异常检测器信息	{"data":{"alarm_info":{"alarm_cause":null,"alarm_content":null,"alarm_level":"ERROR","alarm_type":"SYSTEM","extra":null},"detector_info":[...],"duration":10,"forecasting_seconds":0,"running":1},"success":true}
/v1/api/app/anomaly-detection/detectors/all/rebuild	PUT	根据后端落盘信息重建所有异常检测检测器	无	返回重建状态	{"data":"Success: rebuild detectors for IP","success":true}
/v1/api/app/anomaly-detection/detectors	DELETE	清空所有异常检测检测器	无	返回删除状态	{"data":"Success: clear detectors for IP","success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/app/workload-collection	GET	获取数据库工作负载数据	param data_source：必选，数据源，取值范围：('asp', 'statement_history', 'pg_stat_activity') param databases：可选，数据库列表 param schemas：可选，schema列表 param start_time：可选，开始时间 param end_time：可选，结束时间，仅适用于asp和statement_history param db_users：可选，数据库用户列表 param sql_types：可选，SQL语句类型，取值范围：('SELECT', 'UPDATE', 'DELETE', 'INSERT') param template_id：可选，SQL模板id param duration：可选，SQL语句的执行时间，单位ms，仅适用于statement_histo	返回工作负载信息	{"data":{"header":["user_name","db_name","schema_name","application_name","unique_query_id","start_time","finish_time","duration","n_returned_rows","n_tuples_fetched","n_tuples_returned","n_tuples_inserted","n_tuples_updated","n_tuples_deleted","n_blocks_fetched","n_blocks_hit","n_soft_parse","n_hard_parse","db_time","cpu_time","parse_time","plan_time","data_io_time","lock_wait_time","lwlock_wait_time","query"],"rows":[["user1","db1","public"...]...]},"success":true} **说明** • data_source参数数据源解释：asp为采样数据，statement_history为历史慢SQL数据，慢SQL阈值由用户指定，pg_stat_activity为实时SQL数据。 • 为避免接口超时，返回数据量上限1000条。

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
			ry和 pg_stat_activity		
/v1/api/app/cluster-diagnosis	GET	获取故障诊断结果	param instance：可选，被诊断的实例地址 param role：可选，实例的角色，取值范围：('cn', 'dn') param method：可选，诊断模型使用的方法，取值范围：('logical', 'tree') param timestamp：可选，诊断时间的时间戳，单位：时间戳（13位）	返回故障诊断结果	{"data":[{"bind_ip_failed":0,"cms_phonydead_restart":0,"cms_restart_pending":0,"dn_disk_damage":0,"dn_manual_stop":0,"dn_nic_down":0,"dn_ping_standby":0,"dn_port_conflict":0,"dn_read_only":0,"dn_status":0,"dn_writable":0,"ffic_updated":0,"ping":1},"DN down/disconnection"],"success":true}
/v1/api/summary/metrics	GET	获取所有指标列表	无	返回TSDB上的所有指标列表	{"data":["gaussdb_blks_hit_ratio","gaussdb_blks_read_rate"...],"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/metrics/{name}	GET	获取最新的指定名称序列的指标	param name：必选，指定metric的名称 param instance：可选，metric所属的实例地址 param latest_minutes：可选，指定的时间范围 param from_timestamp：可选，开始时间时间戳，适用于latest_minutes为空的情况 param to_timestamp：可选，结束时间时间戳，适用于latest_minutes为空的情况 param step：可选，获取数据的时间间隔 可选 param fetch_all：可选，是否获取所有序列或只获取一次 param regex：可选，当instance非空，用正则表达式过滤instance param labels：可选，用指定标签过滤序列。支持如下两种格式： ● “k1=v1,k2=v2”的kv pairs格式	返回指定序列的详情。 如果prometheus短期内存入了大量数据，或者查询的区间过大，有可能导致查询超时，或由于内存不足导致查询失败。	{"data":[{"labels":{"device":"device","from_instance":"ip","from_job":"exporter","fstype":"ext4","instance":"ip:port","job":"exporter","mountpoint":"mounpoint"},"name":"os_disk_usage","timestamps":[1684900929939,1684900944939,1684900959939,1684900974939],"values":[0.7115459280083043,0.7115460512887748,0.7115461745692453,0.7115462978497158]}],"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
			• “{'k1':'v1','k2':'v2'}”的json字符串格式 param regex_labels：可选，使用正则表达式过滤序列。支持如下两种格式： • “k1=v1,k2=v2”的kv pairs格式 • “{'k1':'v1','k2':'v2'}”的json字符串格式 param limit：可选，限制返回结果行数 param tz：可选，时区参数，只支持UTC标准		

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/metrics/{name}	DELETE	删除指定范围的指标	param name：必选，指定metric的名称 param instance：可选，metric所属的实例地址 param from_timestamp：可选，开始时间时间戳 param to_timestamp：可选，结束时间时间戳 param regex：可选，当instance非空，用正则表达式过滤instance param labels：可选，用指定标签过滤序列。支持如下两种格式： ● “k1=v1,k2=v2”的kv pairs格式 ● “{'k1':'v1','k2':'v2'}”的json字符串格式 param regex_labels：可选，使用正则表达式过滤序列。支持如下两种格式： ● “k1=v1,k2=v2”的kv pairs格式 ● “{'k1':'v1','k2':'v2'}”的	返回删除状态（from_timestamp和to_timestamp虽然支持毫秒粒度，但是实际删除时仅支持到秒粒度，所以会出现一些临近边界时间范围的值删不掉的情况）	{"data":null,"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
			json字符串格式 param flush：可选，删除指标后是否刷新磁盘 param tz：可选，时区参数，只支持UTC标准		

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/alarms	GET	获取历史告警列表	param pagesize：可选，每页展示个数，建议传递 param current：可选，当前页，建议传递 param instance：可选，告警所属实例，如果不传递则返回当前集群内的告警 param alarm_type：可选，告警类型，参数可选范围为['SYSTEM', 'SLOW_QUERY', 'ALARM_LOG', 'ALARM', 'SECURITY', 'PERFORMANCE'] param alarm_level：可选，告警级别，参数可选范围为['CRITICAL', 'FATAL', 'ERROR', 'WARNING', 'WARN', 'INFO', 'NOTICE', 'DEBUG', 'NOTSET'] param metric_name：可选，指标名称 param start_at：可选，查询开始时间戳，建议传递 param end_at：可选，查询结束	返回历史告警列表	{"data":{"header":["history_alarm_id","instance","metric_name","metric_filter","alarm_type","alarm_level","start_at","end_at","alarm_content","extra_info","anomaly_type","alarm_cause"],"rows":[[65,"ip","os_mem_usage",null,"SYSTEM",30,1684762097001,1684762547001,"mem_usage_spike_detector: Find obvious spikes in memory usage.",null,"Spike"]]},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
			时间戳，建议传递 param anomaly_type：可选，异常类型 param group：可选，是否根据告警类型和内容划分告警		

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/alarms/count	GET	获取历史告警数量	param instance：可选，告警所属实例，如果不传递则返回当前集群内的告警 param alarm_type：可选，告警类型，参数可选范围为['SYSTEM', 'SLOW_QUERY', 'ALARM_LOG', 'ALARM', 'SECURITY', 'PERFORMANCE'] param alarm_level：可选，告警级别，参数可选范围为['CRITICAL', 'FATAL', 'ERROR', 'WARNING', 'WARN', 'INFO', 'NOTICE', 'DEBUG', 'NOTSET'] param metric_name：可选，指标名称 param start_at：可选，查询开始时间戳，不得晚于end_at，建议传递 param end_at：可选，查询结束时间戳，不得早于start_at，建议传递 param anomaly_type：可选，异常类型	返回历史告警数量	{"data":30,"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
			param group：可选，是否根据告警类型和内容划分告警		

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/cluster-diagnosis	GET	获取历史集群诊断列表，集群诊断定时任务结果存储于元数据库中，若元数据库发生故障，结果无法插入表中，会导致部分数据遗漏	param pagesize：可选，每页展示个数，建议传递 param current：可选，当前页，建议传递 param instance：可选，实例IP，如果不传递则返回当前集群内的诊断结果 param instance_like：可选，实例IP模糊匹配，特性预埋字段，未实装 param start_at：可选，查询开始时间戳，仅返回timestamp大于等于该时间戳的结果，建议传递 param end_at：可选，查询结束时间戳，仅返回timestamp小于等于该时间戳的结果，建议传递（若start_at和end_at都不传递，则不对timestamp列进行任何值的筛选，返回全量时间戳的结果） param cluster_role：可选，节点类型 param diagnosis_meth	返回历史集群诊断列表	{"data":{"header":["diagnosis_id","instance","timestamp","cluster_role","diagnosis_method","cluster_feature","diagnosis_result","status_code","alarm_type","alarm_level"],"rows":[[1,ip,1695110107087,"cn","logical","{\"ping\": 0, \"cn_status\": 0, \"bind_ip_failed\": 0, \"panic\": 0, \"ffic_updated\": 0, \"cms_heartbeat_restart\": 0, \"cms_phonydead_restart\": 0, \"cn_dn_disconnected\": 0, \"cn_down_to_delete\": 0, \"cn_restart_time_exceed\": 0, \"cn_read_only\": 0, \"cn_restart\": 0, \"cn_start\": 0, \"cn_manual_stop\": 0, \"cn_disk_damage\": 0, \"cn_nic_down\": 0, \"cn_port_conflict\": 0}","Normal",-1,"ALARM",0]]},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
			od：可选，诊断算法 param status_code：可选，状态码，特性预埋字段，未实装 param alarm_type：可选，告警类型，参数可选范围为['SYSTEM', 'SLOW_QUERY', 'ALARM_LOG', 'ALARM', 'SECURITY', 'PERFORMANCE'] param alarm_level：可选，告警级别，参数可选范围为['CRITICAL', 'FATAL', 'ERROR', 'WARNING', 'WARN', 'INFO', 'NOTICE', 'DEBUG', 'NOTSET'] param is_normal：可选，是否包含诊断结果为Normal的数据		**说明** ● 返回结果中timestamp列所记录的时间戳以DBMind部署的服务器时间为准，如果元数据库所在服务器时间与DBMind部署服务器时间不一致，会导致元数据库中显示的时间戳结果与元数据库服务器实际时间不一致。 ● 返回结果按照以下两个规则排序： ● 按照timestamp从小到大排列； ● 当timestamp一致时按照diagnosis_id从小到大排列。

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/cluster-diagnosis/count	GET	获取历史集群诊断数量，集群诊断定时任务结果存储于元数据库中，若元数据库发生故障，结果无法插入表中，会导致部分数据遗漏	param instance：可选，实例IP，如果不传递则返回当前集群内的诊断结果 param instance_like：可选，实例IP模糊匹配，特性预埋字段，未生效 param start_at：可选，查询开始时间戳，仅返回timestamp大于等于该时间戳的结果，建议传递 param end_at：可选，查询结束时间戳，仅返回timestamp小于等于该时间戳的结果，建议传递 （若start_at和end_at都不传递，则不对timestamp列进行任何值的筛选，返回全量时间戳的结果） param cluster_role：可选，节点类型 param diagnosis_method：可选，诊断算法 param status_code：可选，状态码，特性预埋字段，未生效 param alarm_type：可	返回历史集群诊断数量	{"data":30,"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
			选，告警类型，参数可选范围为['SYSTEM', 'SLOW_QUERY', 'ALARM_LOG', 'ALARM', 'SECURITY', 'PERFORMANCE'] param alarm_level：可选，告警级别，参数可选范围为['CRITICAL', 'FATAL', 'ERROR', 'WARNING', 'WARN', 'INFO', 'NOTICE', 'DEBUG', 'NOTSET'] param is_normal：可选，是否包含诊断结果为Normal的数据		

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/sql/killed	GET	获取被终止的查询列表	param pagesize：可选，每页展示个数 param current：可选，当前页 param instance：可选，慢SQL所属实例 param query：可选，慢SQL文本 param start_time：可选，获取慢SQL的开始时间，单位：时间戳（13位） param end_time：可选，获取慢SQL的结束时间，单位：时间戳（13位）	返回被终止的查询列表	{"data":{"header":["instance","schema_name","db_name","query","template_id","hit_rate","fetch_rate","cpu_time","data_io_time","parse_time","plan_time","db_time","root_cause","suggestion","start_at","duration_time"],"rows":[(ip1, 'public', 'db1', 'select 1', t_id, ...), (...), ...]},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/sql/killed/count	GET	获取被终止的查询数量	param instance：可选，慢SQL所属实例 param query：可选，慢SQL文本 param start_time：可选，获取慢SQL的开始时间，单位：时间戳（13位） param end_time：可选，获取慢SQL的结束时间，单位：时间戳（13位）	返回被终止的查询数量	{"data":0,"success":true}
/v1/api/summary/sql/top	GET	获取执行最频繁的前十个SQL模板	无	返回执行最频繁的前十个SQL模板	{"data":{"header":["user_name","unique_sql_id","query","n_calls","min_elapse_time","max_elapse_time","avg_elapse_time","n_returned_rows","db_time","cpu_time","execution_time","parse_time","last_updated","sort_spill_count","hash_spill_count"],"rows":[["user",1961954918,"vacuum;",1,38958701,38958701,38958701,"3891978 1.218781218781",0,38959021...]]},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/sql/active	GET	获取当前活跃的SQL列表	无	返回当前活跃的SQL列表	{"data":{"header":["datname","usename","application_name","client_addr","query_start","waiting","state","query","connection_info"], "rows":[["data1","db1","app1","ip:port","time"...]]},"success":true}
/v1/api/summary/sql/locking	GET	获取当前锁等待的SQL列表	无	返回当前锁等待的SQL列表	{"data":{"header":null,"rows":[]},"success":true}
/v1/api/app/index-recommendation	POST	获取建议的索引列表	param instance：必选，数据库实例 param database：可选，数据库名称 param max_index_num：可选，推荐索引个数的最大值 param max_index_storage：可选，推荐索引的内存使用的最大值 param sqls：可选，SQL语句列表	返回建议的索引列表	{"data":[{"advise_indexes":[],"redundant_indexes":[],"total":0,"useless_indexes":[{"columns":"c1","schemaName":"public","statement":"DROP INDEX t1_c1_idx;","tbName":"t1","type":3},{"columns":"c2","schemaName":"public","statement":"DROP INDEX t2_c2_idx;","tbName":"t2","type":3}]},{}],"success":true}
/v1/api/summary/knob-recommendation/snapshots	GET	获取指标快照	param pagesize：可选，每页展示个数 param current：可选，当前页	返回指标快照	{"data":{"header":["instance","metric","value"],"rows":[]},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/knob-recommendation/snapshots/count	GET	获取快照数量	无	返回快照数量	{"data":0,"success":true}
/v1/api/summary/knob-recommendation/warnings	GET	获取指标报警值	param pagesize：可选，每页展示个数 param current：可选，当前页	返回指标报警值	{"data":{"header":["instance","level","comment"],"rows":[]},"success":true}
/v1/api/summary/knob-recommendation/warnings/count	GET	获取指标报警值数量	无	返回指标报警值数量	{"data":0,"success":true}
/v1/api/summary/knob-recommendation/details	GET	获取参数推荐列表	param pagesize：可选，每页展示个数 param current：可选，当前页	返回参数推荐列表	{"data":{"header":["instance","name","current","recommend","min","max"],"rows":[]},"success":true}
/v1/api/summary/knob-recommendation/details/count	GET	获取参数推荐数量	无	返回参数推荐数量	{"data":0,"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/sql/slow	GET	获取慢查询诊断统计信息	无	返回慢查询诊断统计信息	{"data":{"distribution":{"delete":0,"insert":0,"select":0,"update":0},"main_slow_queries":0,"mean_buffer_hit_rate":-1,"mean_cpu_time":-1,"mean_fetch_rate":-1,"mean_io_time":-1,"nb_unique_slow_queries":0,"slow_query_count":{"timestamps":[],"values":[]},"slow_query_template":{"header":["template_id","count","query"],"rows":[]},"slow_query_threshold":2.0,"statistics_for_database":{},"statistics_for_schema":{},"systable":{"business_table":0,"system_table":0}},"success":true}
/v1/api/summary/regular-inspection	GET	获取定期巡检结果列表	param inspection_type：必选，检测类型，取值范围：('daily_check', 'weekly_check', 'monthly_check')	返回定期巡检结果列表	{"data":{"header":["instance","report","start","end"],"rows":[["ip:port",{"connection":{"active_connection":{"avg":3.0,"max":4.0,"min":1.0,"the_95th":4.0},"total_connection":{"avg":3.2321,"max":5.0,"min":1.0,"the_95th":4.0}}...},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/regular-inspection/count	GET	获取定期巡检结果数量	param inspection_type：必选，检测类型，取值范围：('daily_check', 'weekly_check', 'monthly_check')	返回定期巡检结果数量	{"data":9,"success":true}
/v1/api/summary/database-list	GET	获取数据库列表	无	返回数据库列表	{"data":["db1","db2","db3","db4","db5"...],"success":true}
/v1/api/summary/sql/slow/latest	GET	获取最近的慢查询列表	param instance：可选，慢SQL所属的实例地址 param query：可选，query文本 param start_time：可选，慢SQL开始运行时间，单位：时间戳（13位） param end_time：可选，慢SQL结束运行时间，单位：时间戳（13位） param group：可选，查询时是否使用group逻辑	返回最近的慢查询列表	{"data":{"header":["instance","schema_name","db_name","query","template_id","hit_rate","fetch_rate","cpu_time","data_io_time","parse_time","plan_time","db_time","root_cause","suggestion","start_at","duration_time"],"rows":[(ip1, 'public', 'db1', 'select 1', t_id, ...), (...), ...]},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/summary/sql/slow/latest/count	GET	获取最近的慢查询数量	param pagesize：可选，查询页返回数量 param current：可选，当前页数 param instance：可选，慢SQL所属的实例地址 param distinct：可选，查询时是否使用distinct逻辑 param query：可选，query文本 param start_time：可选，慢SQL开始运行时间，单位：时间戳（13位） param end_time：可选，慢SQL结束运行时间，单位：时间戳（13位） param group：可选，查询时是否使用group逻辑	返回最近的慢查询数量	{"data":0,"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
/v1/api/app/metric-diagnosis	GET	执行指标异常诊断	param metric_name：必选，指标名称。取值范围：os_cpu_user_usage，pg_thread_pool_rate，os_mem_usage，os_disk_usage，os_disk_io_read_delay param metric_filter：必选，筛选指标。支持如下两种格式： ● “k1=v1,k2=v2”的kv pairs格式 ● “{'k1':'v1','k2':'v2'}”的json字符串格式 param alarm_cause：必选，选择分析方法。取值范围：high_cpu_usage，high_thread_pool_rate, high_dynamic_mem_usage, high_shared_mem_usage, high_disk_usage, high_io_delay param start：可选，分析指标开始时间戳，单位毫秒 param end：可选，分析指标结	返回指标异常根因	{"data":{[{'reason1': 0.0, 'reason2': 1.0}, 'conclusion', 'advice']},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
			束时间戳，单位毫秒		

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例

/v1/api/app/real-time-inspection	POST	执行智能巡检功能	param inspection_type：必选，巡检类型
			param start_time：可选，起始时间
			param end_time：可选，终止时间
			param instance：必选，实例IP
			param inspection_items：必选，巡检项
			param tz：可选，时区参数，只支持UTC标准
				返回巡检结果	{
					"system_resource": {
					"os_mem_usage": {
					"192.168.0.1": {
					"statistic": {
					"max": 0.4154,
					"min": 0.4128,
					"avg": 0.4149,
					"the_95th": 0.4153
					},
					"warnings": {
					"increase_warning": false,
					"threshold_warning": [],
					"forecast_warning": {
					"occur_time": "",
					"remaining_hours": 0.0,
					"risk": "",
					"timestamps": [],
					"values": []
					}
					},
					"timestamps": [1689210000000, 1689210360000],
					"data": [0.41450417776587234, 0.41450179472976234]
					},
					"192.168.0.2": {
					xxx
					}
					}
					},

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
					"instance_status": {}, "database_resource": {}, "database_performance": {}, "diagnosis_optimization": {}, "conclusion": {} }
/v1/api/app/real-time-inspection/list	GET	展示巡检任务的基础信息	param instance：必选，实例IP	返回巡检任务的基础信息	{"data":{"header":["instance","start","end","id","state","cost_time","inspection_type"],"rows":[["192.168.0.1:8080",1689210000000,1689296400000,5,"success",0.033701,"real_time_check"]]},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例

/v1/api/summary/real-time-inspection	GET	获取指定巡检任务的巡检结果	param instance：必选，实例IP
			param spec_id：必选，巡检任务ID
				返回指定巡检任务的巡检结果	{
					"system_resource": {
					"os_mem_usage": {
					"192.168.0.1": {
					"statistic": {
					"max": 0.4154,
					"min": 0.4128,
					"avg": 0.4149,
					"the_95th": 0.4153
					},
					"warnings": {
					"increase_warning": false,
					"threshold_warning": [],
					"forecast_warning": {
					"occur_time": "",
					"remaining_hours": 0.0,
					"risk": "",
					"timestamps": [],
					"values": []
					}
					},
					"timestamps": [1689210000000, 1689210360000],
					"data": [0.41450417776587234, 0.41450179472976234]
					},
					"192.168.0.2": {
					xxx
					}
					}
					},

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
					"instance_status": {}, "database_resource": {}, "database_performance": {}, "diagnosis_optimization": {}, "conclusion": {} }
/v1/api/app/real-time-inspection	DELETE	删除指定的巡检任务	param instance：必选，实例IP param spec_id：必选，巡检任务ID	返回删除状态	{"data": {"success":true},"success":true}
/v1/api/check-status	POST	获取DBMind服务的状态信息并返回状态详情	param cmd：可选，服务启动命令	返回状态详情	{"data": {"error_msg":"","result": {},"state":"NORMAL"},"success":true}
/v1/api/repair	POST	修复DBMind服务并返回修复结果	param cmd：可选，服务启动命令	返回修复结果	{"data": {"error_msg":"","result": {},"state":"SUCCESS"},"success":true}

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例

/v1/api/summary/sql-trace	GET	SQL全链路查询	param instance_id：必选，实例id
			param labels：必选，过滤使用的指标标签
			param is_online：必选，数据来源是否是在线的
			param from_timestamp：可选，查询开始时间
			param to_timestamp：可选，查询结束时间
			param tz：可选，时区参数，仅支持UTC标准
				返回SQL全链路信息	{
					"data": [
					{
					"all_time": 3477357,
					"application_name": "xxx",
					"client_addr": "",
					"client_port": 0,
					"component_id": "xxx",
					"db_name": "xxx",
					"execution_time_details": {
					"kernel_time": {
					"all_time": 3477357,
					"kernel_time_details": {
					"execution_time": 3466386,
					"other_time": 4958,
					"parse_time": 453,
					"plan_time": 5244,
					"rewrite_time": 316
					}
					},
					"resource_time": {
					"all_time": 3477357,
					"resource_time_details": {
					"cpu_time": 98756,
					"data_io_time": 0,
					"other_time": 3378601
					}
					},
					"wait_event_time": {
					"code_wait_event_time": {

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例

					"all_time": 3477357,
					"code_wait_event_time_details": {
					"events": [
					{
					"event_name": "xxx",
					"event_time": 16810
					},
					{
					"event_name": "xxx",
					"event_time": 5330
					},
					{
					"event_name": "xxx",
					"event_time": 5195
					},
					{
					"event_name": "xxx",
					"event_time": 197
					}
					],
					"left_time": 0,
					"other_time": 3449825
					}
					},
					"resource_wait_event_time": {
					"all_time": 3477357,
					"other_time": 3477357,
					"resource_wait_event_time_details": {
					"data_io_time": {
					"all_time": 0,
					"data_io_time_details": {
					"events": [

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例

					{
					"event_name": "xxx",
					"event_time": 742
					}
					],
					"left_time": 0,
					"other_time": -27532
					}
					},
					"lock_time": {
					"all_time": 0,
					"lock_time_details": {
					"events": [],
					"left_time": 0,
					"other_time": 0
					}
					},
					"lwlock_time": {
					"all_time": 0,
					"lwlock_time_details": {
					"events": [],
					"left_time": 0,
					"other_time": 0
					}
					}
					}
					}
					},
					"finish_time": "2023-08-01 17:50:04.923701+08",
					"node_id": "0",
					"schema_name": "user,public",

版权所有 © 华为云计算技术有限公司

API	请求方法	功能描述	参数	返回值	示例
					"session_id": 1658590, "sql_exec_id": 72339069024193250, "sql_id": 1951225884, "start_time": "2023-08-01 17:50:01.446446+08", "trace_id": "0", "transaction_id": "0", "user_name": "user" }], "success": true }
/v1/api/summary/metric-unit/{metric}	GET	指标单位查询	param metric：必选，指标名	返回**指标单位**	{"data":"{'en': 'rate', 'cn': '比率'}","success":true}
/v1/api/security/scenarios	GET	自安全	无	返回所有安全异常类型	{"data": ["scanning_attack", "brute_force_login_attack","user_violation_rate"],"success":true}
/v1/api/security/scenarios/{name}	GET	自安全	param name：必选，安全指标名称，参数可选范围为['scanning_attack', 'brute_force_login_attack', 'user_violation_rate']	返回安全异常包含的指标的校准状态	{"data": [{"metric":"gaussdb_log_errors_rate","status":"CALIBRATED"}, {"metric":"gaussdb_user_violation_rate","status":"CALIBRATED"}],"success":true}

版权所有 © 华为云计算技术有限公司

说明

对异常检测器检测算法的说明如下，也可以通过调用/v1/api/configs/anomaly-detection/default-settings获得：

1. 学生化残差检测器-EsdTestDetector：{"alpha":0.05}。
2. 梯度检测器-GradientDetector：{"max_coef":1,"side":["positive",["positive","negative","both"]]}。
3. 单调趋势检测器-IncreaseDetector：{"alpha":null,"side":["positive",["positive","negative","both"]]}。
4. 四分位间距检测器-InterQuartileRangeDetector：{"outliers":[3,3]}。
5. 均值漂移检测器-LevelShiftDetector：{"agg":["median",["median","mean","std"]],"outliers":[null,6], "side":["both",["positive","negative","both"]],"window":5}。
6. 分位数检测器-QuantileDetector：{"high":1,"low":0}。
7. 周期数据检测器-SeasonalDetector：{"high_ac_threshold":0.1,"min_seasonal_freq":2,"outliers":[null,3], "period":null,"side":["positive,["positive","negative","both"]],"window":10}"。
8. 尖峰检测器-SpikeDetector：{"agg":["median",["median","mean","std"]],"outliers":[null,3], "side":["both",["positive","negative","both"]],"window":1}。
9. 阈值检测器-ThresholdDetector：{"high":null,"low":null,"percentage":[null,[0,1]]}。
10. 波动率漂移检测器-VolatilityShiftDetector：{"agg":["std",["median","mean","std"]],"outliers":[null,6], "side":["both",["positive","negative","both"]],"window":10}。

说明

- alarm_level参数的所有级别及其对应的权重：CRITICAL、FATAL：50，ERROR：40，WARNING、WARN：30，INFO、NOTICE：20，DEBUG：10、NOTSET：0。
- alarm_type参数的所有类型：SYSTEM、SLOW_QUERY、ALARM_LOG、ALARM、SECURITY、PERFORMANCE。
- anomaly_type参数的所有类型：high_disk_usage_detector、high_mem_usage_detector、high_cpu_usage_detector、high_thread_pool_rate_detector、high_io_delay_detector、deadlock_detector、slow_disk_detector、mem_leak_detector、core_detector。

指标单位查询接口

/v1/api/summary/metric-unit/{metric}支持的指标列表如下表所示：

tps	gs_redo_stat_writetim	process_start_time_seconds	node_netstat_Udp6_SndbufErrors
qps	gs_index_idx_blks_hit	gs_audit_user_locked_total	node_netstat_UdpLite6_InErrors
node_load1	gaussdb_log_node_start	gs_connections_enqueue_sql	node_rapl_package_joules_total
node_load5	gaussdb_log_gtm_status	gs_session_memory_usedsize	node_timex_pps_frequency_hertz
node_load15	pg_checkpoint_redo_lsn	gaussdb_progress_leaked_fds	node_timex_pps_stability_hertz
os_disk_iops	pg_checkpoint_next_oid	gaussdb_log_dn_ping_standby	gs_long_xact_wait_events_count

版权所有 © 华为云计算技术有限公司

os_mem_usage	pg_database_size_bytes	pg_connections_idle_session	gs_checkpoint_full_page_writes
pg_time_value	pg_db_confl_tablespace	pg_global_ckpt_status_value	gs_lock_time_info_holding_time
os_cpu_iowait	pg_redo_stat_phyblkwrt	pg_memory_context_totalsize	gs_recovery_status_current_rto
os_disk_usage	pg_sql_count_mergeinto	pg_session_memory_totalsize	gs_replication_slots_delay_lsn
load_average1	pg_stat_activity_count	pg_stat_get_wal_senders_pid	gs_stat_bgwriter_total_seconds
os_disk_await	pg_index_idx_blks_read	pg_sql_statement_full_count	gs_stat_replication_flush_diff
gs_time_value	pg_tables_size_relsize	gaussdb_invalid_logins_rate	gs_stat_replication_replay_lag
pg_db_blks_hit	os_disk_io_write_bytes	gaussdb_user_violation_rate	gs_stat_replication_replay_lsn
gs_db_blks_hit	os_disk_io_write_delay	gaussdb_confl_snapshot_rate	gs_stat_replication_write_diff
pg_db_blks_read	gaussdb_blks_read_rate	gaussdb_confl_deadlock_rate	gs_summary_user_logout_counter
pg_db_conflicts	gaussdb_blks_hit_ratio	gaussdb_table_increase_rate	gs_wait_events_total_wait_time
pg_db_deadlocks	gaussdb_conflicts_rate	gaussdb_index_increase_rate	gs_tables_structure_n_tup_read
pg_locker_count	gaussdb_deadlocks_rate	node_context_switches_total	gs_tables_structure_n_live_tup
pg_class_relage	node_boot_time_seconds	node_entropy_available_bits	gs_tables_structure_n_dead_tup
pg_index_mbytes	node_cpu_seconds_total	node_entropy_pool_size_bits	gaussdb_log_cms_restart_pending
os_disk_ioutils	node_memory_Slab_bytes	node_filesystem_avail_bytes	pg_checkpoint_next_multixact_id
node_intr_total	node_network_mtu_bytes	node_hwmon_temp_max_celsius	pg_checkpoint_next_multi_offset
node_network_up	node_timex_sync_status	node_memory_AnonPages_bytes	pg_checkpoint_oldest_active_xid
node_udp_queues	node_vmstat_pgmajfault	node_memory_HugePages_Total	pg_stat_replication_replay_diff

版权所有 © 华为云计算技术有限公司

process_max_fds	gs_checkpoint_redo_lsn	node_memory_SwapTotal_bytes	pg_tables_structure_last_vacuum
gs_db_blks_read	gs_checkpoint_next_oid	node_memory_Writeback_bytes	node_disk_io_time_seconds_total
gs_db_conflicts	gs_database_size_bytes	node_netstat_Icmp6_InErrors	node_disk_reads_completed_total
gs_db_deadlocks	gs_db_confl_tablespace	node_netstat_IpExt_InOctets	node_memory_AnonHugePages_bytes
gs_locker_count	gs_redo_stat_phyblkwrt	node_rapl_dram_joules_total	node_memory_Inactive_anon_bytes
gs_class_relage	gs_sql_count_mergeinto	node_sockstat_TCP_mem_bytes	node_memory_Inactive_file_bytes
gs_index_mbytes	gs_stat_activity_count	node_sockstat_UDPLITE_inuse	node_netstat_TcpExt_ListenDrops
gaussdb_ping_lag	gs_index_idx_blks_read	node_sockstat_UDP_mem_bytes	node_netstat_TcpExt_TCPTimeouts
gaussdb_log_ffic	gs_tables_size_relsize	node_timex_maxerror_seconds	node_network_receive_drop_total
pg_db_confl_lock	gaussdb_log_cms_cn_down	node_timex_pps_jitter_total	node_network_receive_errs_total
pg_db_temp_bytes	pg_audit_ddl_user_total	gs_connections_idle_session	node_network_receive_fifo_total
pg_db_temp_files	pg_connections_max_conn	gs_global_ckpt_status_value	node_nf_conntrack_entries_limit
pg_sql_count_dcl	pg_memory_context_count	gs_memory_context_totalsize	node_schedstat_timeslices_total
pg_sql_count_ddl	pg_prepared_xacts_count	gs_session_memory_totalsize	gs_checkpoint_next_multixact_id
pg_sql_count_dml	pg_stat_replication_lsn	gs_stat_get_wal_senders_pid	gs_checkpoint_next_multi_offset
pg_class_relsize	os_disk_io_queue_length	gs_sql_statement_full_count	gs_checkpoint_oldest_active_xid
gaussdb_cpu_time	os_cpu_processor_number	pg_pagewriter_dirty_page_num	gs_stat_replication_replay_diff
node_arp_entries	gaussdb_qps_by_instance	pg_audit_user_unlocked_total	gs_tables_structure_last_vacuum
node_disk_io_now	gaussdb_log_errors_rate	pg_checkpoint_checkpoint_lsn	pg_audit_successful_logins_total

版权所有 © 华为云计算技术有限公司

node_forks_total	gaussdb_confl_lock_rate	pg_sesstype_memory_totalsize	pg_replication_slots_restart_lsn
process_open_fds	os_network_receive_drop	pg_stat_replication_sent_lsn	pg_never_used_indexes_index_size
gs_db_confl_lock	gaussdb_corrosion_index	pg_tables_structure_tbl_scan	pg_tables_structure_last_analyze
gs_db_temp_bytes	gaussdb_statement_calls	gaussdb_confl_bufferpin_rate	pg_tables_structure_vacuum_count
gs_db_temp_files	gaussdb_idle_connection	gaussdb_statement_sort_spill	gaussdb_table_increase_amplitude
gs_sql_count_dcl	node_hwmon_temp_celsius	gaussdb_statement_hash_spill	node_cpu_package_throttles_total
gs_sql_count_ddl	node_memory_Dirty_bytes	node_cpu_frequency_max_hertz	node_cpu_scaling_frequency_hertz
gs_sql_count_dml	node_memory_Shmem_bytes	node_cpu_frequency_min_hertz	node_disk_writes_completed_total
gs_class_relsize	node_netstat_Tcp_InErrs	node_cpu_guest_seconds_total	node_hwmon_power_is_battery_watt
gaussdb_nic_state	node_netstat_Tcp_InSegs	node_disk_reads_merged_total	node_network_address_assign_type
gaussdb_log_panic	node_sockstat_RAW_inuse	node_filesystem_device_error	node_network_receive_bytes_total
pg_checkpoint_tli	node_sockstat_TCP_alloc	node_hwmon_temp_crit_celsius	node_network_receive_frame_total
pg_db_blks_access	node_sockstat_TCP_inuse	node_memory_PageTables_bytes	node_network_transmit_drop_total
pg_db_reset_delay	node_sockstat_UDP_inuse	node_memory_SUnreclaim_bytes	node_network_transmit_errs_total
pg_db_tup_deleted	node_timex_tick_seconds	node_memory_SwapCached_bytes	node_network_transmit_fifo_total
pg_db_tup_fetched	scrape_duration_seconds	node_netstat_IpExt_OutOctets	node_timex_pps_calibration_total
pg_db_tup_updated	gs_audit_ddl_user_total	node_netstat_Tcp_ActiveOpens	process_virtual_memory_max_bytes
pg_db_xact_commit	gs_connections_max_conn	node_netstat_Tcp_RetransSegs	gs_audit_successful_logins_total
pg_class_relpages	gs_memory_context_count	node_netstat_Udp_InDatagrams	gs_replication_slots_restart_lsn

版权所有 © 华为云计算技术有限公司

pg_index_idx_scan	gs_prepared_xacts_count	node_network_iface_link_mode	gs_never_used_indexes_index_size
os_cpu_user_usage	gs_stat_replication_lsn	node_sockstat_UDPLITE6_inuse	gs_tables_structure_last_analyze
os_cpu_idle_usage	gaussdb_ping_packet_rate	node_softnet_processed_total	gs_tables_structure_vacuum_count
gaussdb_cpu_usage	gaussdb_log_login_denied	node_timex_pps_shift_seconds	gaussdb_log_user_cancel_statement
node_time_seconds	gaussdb_log_errors_total	process_virtual_memory_bytes	gaussdb_log_cms_phonyd ead_restart
node_timex_status	gaussdb_log_flow_control	gs_pagewriter_dirty_page_num	gaussdb_log_cms_heartbeat_timeout
gs_checkpoint_tli	gaussdb_log_node_restart	gs_audit_user_unlocked_total	pg_shared_memory_detail_totalsize
gs_db_blks_access	pg_checkpoint_oldest_xid	gs_checkpoint_checkpoint_lsn	pg_stat_replication_sync_priority
gs_db_reset_delay	pg_connections_used_conn	gs_sesstype_memory_totalsize	pg_tables_structure_n_tup_hot_upd
gs_db_tup_deleted	pg_connections_used_rate	gs_stat_replication_sent_lsn	pg_tables_structure_analyze_count
gs_db_tup_fetched	pg_lock_sql_locked_times	gs_tables_structure_tbl_scan	pg_sql_statement_history_exc_time
gs_db_tup_updated	pg_sesstype_memory_count	gaussdb_log_lock_wait_timeout	node_disk_read_time_seconds_total
gs_db_xact_commit	pg_tables_size_totalsize	gaussdb_log_undo_invalidation	node_hwmon_temp_crit_alar_celsius
gs_class_relpages	pg_tables_size_indexsize	gaussdb_log_undo_impermission	node_netstat_TcpExt_TCPSynRetrans
gs_index_idx_scan	pg_tables_size_toastsize	pg_session_memory_detail_size	node_network_transmit_bytes_total
gaussdb_ping_state	gaussdb_user_locked_rate	pg_autovacuum_worker_duration	node_network_transmit_colls_total
gaussdb_xlog_count	gaussdb_tup_fetched_rate	pg_audit_invalid_logins_total	node_softnet_times_squeezed_total
pg_txid_oldestxmin	gaussdb_tup_deleted_rate	pg_audit_user_violation_total	gs_shared_memory_detail_totalsize
pg_checkpoint_time	gaussdb_tup_updated_rate	pg_checkpoint_oldest_xid_dbid	gs_stat_replication_sync_priority

版权所有 © 华为云计算技术有限公司

pg_db_tup_inserted	os_network_receive_error	pg_stat_replication_flush_lsn	gs_tables_structure_n_tup_hot_upd
pg_db_tup_returned	os_network_transmit_drop	pg_stat_replication_sent_diff	gs_tables_structure_analyze_count
pg_class_reltuples	os_network_receive_bytes	pg_stat_replication_write_lsn	gs_sql_statement_history_exc_time
os_disk_iocapacity	gaussdb_wait_event_spike	pg_summary_user_login_counter	gaussdb_log_cn_restart_time_exceed
gaussdb_state_time	gaussdb_total_connection	pg_total_memory_detail_mbytes	pg_logical_replication_slots_count
gaussdb_user_login	node_filesystem_readonly	pg_tables_structure_n_tup_mod	pg_lock_time_info_p95_holding_time
os_mem_total_bytes	node_memory_Active_bytes	pg_tables_structure_n_tup_ins	pg_session_memory_detail_totalsize
node_network_flags	node_memory_Bounce_bytes	pg_tables_structure_n_tup_upd	pg_stat_replication_backend_uptime
node_procs_blocked	node_memory_Cached_bytes	pg_tables_structure_n_tup_del	node_disk_write_time_seconds_total
node_procs_running	node_memory_Mapped_bytes	pg_tables_structure_dead_rate	node_edac_correctable_errors_total
node_vmstat_pgpgin	node_netstat_Icmp_InMsgs	gaussdb_confl_temp_files_rate	node_hwmon_temp_crit_alarm_celsius
node_vmstat_pswpin	node_netstat_Tcp_OutRsts	gaussdb_confl_temp_bytes_rate	node_netstat_TcpExt_SyncookiesRecv
gs_txid_oldestxmin	node_netstat_Tcp_OutSegs	node_cooling_device_cur_state	node_netstat_TcpExt_SyncookiesSent
gs_checkpoint_time	node_netstat_Udp_NoPorts	node_cooling_device_max_state	node_network_carrier_changes_total
gs_db_tup_inserted	node_network_speed_bytes	node_cpu_core_throttles_total	node_network_receive_packets_total
gs_db_tup_returned	node_sockstat_FRAG_inuse	node_disk_writes_merged_total	node_network_transmit_queue_length
gs_class_reltuples	node_sockstat_RAW6_inuse	node_disk_written_bytes_total	node_timex_estimated_error_seconds
gaussdb_mount_state	node_sockstat_TCP6_inuse	node_hwmon_power_average_watt	gs_logical_replication_slots_count
pg_temp_files_count	node_sockstat_TCP_orphan	node_memory_Active_anon_bytes	gs_lock_time_info_p95_holding_time

版权所有 © 华为云计算技术有限公司

pg_downstream_count	node_sockstat_UDP6_inuse	node_memory_Active_file_bytes	gs_session_memory_detail_totalsize
pg_db_blk_read_time	gs_checkpoint_oldest_xid	node_memory_CommitLimit_bytes	gs_stat_replication_backend_uptime
pg_db_xact_rollback	gs_connections_used_conn	node_memory_DirectMap1G_bytes	pg_global_double_write_status_value
pg_node_info_uptime	gs_connections_used_rate	node_memory_DirectMap2M_bytes	pg_tables_structure_last_autovacuum
pg_sql_count_select	gs_lock_sql_locked_times	node_memory_DirectMap4k_bytes	pg_sql_statement_statistics_n_calls
pg_sql_count_insert	gs_sesstype_memory_count	node_memory_KernelStack_bytes	pg_sql_statement_statistics_db_time
pg_sql_count_delete	gs_tables_size_totalsize	node_memory_Unevictable_bytes	gaussdb_database_increase_amplitude
pg_sql_count_update	gs_tables_size_indexsize	node_memory_VmallocUsed_bytes	node_memory_HardwareCorrupted_bytes
pg_thread_pool_rate	gs_tables_size_toastsize	node_netstat_Tcp_PassiveOpens	node_netstat_TcpExt_ListenOverflows
pg_settings_setting	gaussdb_log_cms_read_only	node_netstat_Udp6_InDatagrams	node_network_transmit_carrier_total
oldestxmin_increase	pg_long_transaction_count	node_netstat_UdpLite_InErrors	node_network_transmit_packets_total
os_cpu_iowait_usage	pg_long_xact_mem_ctx_size	node_netstat_Udp_OutDatagrams	gs_global_double_write_status_value
os_cpu_system_usage	pg_state_memory_totalsize	node_netstat_Udp_RcvbufErrors	gs_tables_structure_last_autovacuum
os_process_fds_rate	gaussdb_tup_inserted_rate	node_netstat_Udp_SndbufErrors	gs_sql_statement_statistics_n_calls
gaussdb_user_logout	os_network_transmit_error	node_scrape_collector_success	gs_sql_statement_statistics_db_time
node_filefd_maximum	os_network_transmit_bytes	node_time_zone_offset_seconds	pg_tables_structure_last_autoanalyze
node_vmstat_pgfault	gaussdb_statement_db_time	node_timex_loop_time_constant	node_cpu_scaling_frequency_max_hertz
node_vmstat_pgpgout	gaussdb_active_connection	node_timex_pps_jitter_seconds	node_cpu_scaling_frequency_min_hertz
node_vmstat_pswpout	node_memory_Buffers_bytes	node_timex_tai_offset_seconds	node_edac_uncorrectable_errors_total

版权所有 © 华为云计算技术有限公司

gs_temp_files_count	node_memory_CmaFree_bytes	process_resident_memory_bytes	node_netstat_TcpExt_SyncookiesFailed
gs_downstream_count	node_memory_MemFree_bytes	gs_session_memory_detail_size	node_network_receive_multicast_total
gs_db_blk_read_time	node_memory_Mlocked_bytes	gs_autovacuum_worker_duration	node_schedstat_running_seconds_total
gs_db_xact_rollback	node_netstat_Icmp6_InMsgs	gs_audit_invalid_logins_total	node_schedstat_waiting_seconds_total
gs_node_info_uptime	node_netstat_Icmp_OutMsgs	gs_audit_user_violation_total	promhttp_metric_handler_errors_total
gs_sql_count_select	node_netstat_Ip6_InOctets	gs_checkpoint_oldest_xid_dbid	gs_tables_structure_last_autoanalyze
gs_sql_count_insert	node_netstat_Udp6_NoPorts	gs_stat_replication_flush_lsn	pg_summary_file_iostat_total_phyblkrd
gs_sql_count_delete	node_netstat_Udp_InErrors	gs_stat_replication_sent_diff	statement_responsetime_percentile_p80
gs_sql_count_update	node_nf_conntrack_entries	gs_stat_replication_write_lsn	statement_responsetime_percentile_p95
gs_thread_pool_rate	node_sockstat_FRAG6_inuse	gs_summary_user_login_counter	node_network_receive_compressed_total
gs_settings_setting	node_sockstat_FRAG_memory	gs_total_memory_detail_mbytes	node_timex_frequency_adjustment_ratio
pg_audit_grant_total	node_timex_offset_seconds	gs_tables_structure_n_tup_mod	gs_summary_file_iostat_total_phyblkrd
pg_checkpoint_elapse	process_cpu_seconds_total	gs_tables_structure_n_tup_ins	pg_summary_file_iostat_total_phyblkwrt
pg_db_blk_write_time	gs_long_transaction_count	gs_tables_structure_n_tup_upd	net_conntrack_dialer_conn_closed_total
pg_db_confl_deadlock	gs_long_xact_mem_ctx_size	gs_tables_structure_n_tup_del	net_conntrack_dialer_conn_failed_total
pg_db_confl_snapshot	gs_state_memory_totalsize	gs_tables_structure_dead_rate	node_network_transmit_compressed_total
gaussdb_state_memory	gaussdb_progress_cpu_usage	gaussdb_log_dn_writable_failed	node_scrape_collector_duration_seconds
pg_database_all_size	gaussdb_progress_mem_usage	pg_long_xact_wait_events_count	promhttp_metric_handler_requests_total
node_network_carrier	gaussdb_log_deadlock_count	pg_checkpoint_full_page_writes	gs_summary_file_iostat_total_phyblkwrt

版权所有 © 华为云计算技术有限公司

node_sockstat_TCP_tw	gaussdb_log_bind_ip_failed	pg_lock_time_info_holding_time	gaussdb_log_gtm_disconnected_to_primary
gs_database_all_size	pg_audit_user_locked_total	pg_recovery_status_current_rto	pg_session_connection_total_session_num
gs_audit_grant_total	pg_connections_enqueue_sql	pg_replication_slots_delay_lsn	node_timex_pps_stability_exceeded_total
gs_checkpoint_elapse	pg_session_memory_usedsize	pg_stat_bgwriter_total_seconds	gs_session_connection_total_session_num
gs_db_blk_write_time	node_disk_read_bytes_total	pg_stat_replication_flush_diff	net_conntrack_listener_conn_closed_total
gs_db_confl_deadlock	node_filesystem_files_free	pg_stat_replication_replay_lag	node_disk_io_time_weighted_seconds_total
gs_db_confl_snapshot	node_filesystem_free_bytes	pg_stat_replication_replay_lsn	node_edac_csrow_correctable_errors_total
gaussdb_cluster_state	node_filesystem_size_bytes	pg_stat_replication_write_diff	gaussdb_log_cms_heartbeat_timeout_restart
gaussdb_log_cn_status	node_memory_CmaTotal_bytes	pg_summary_user_logout_counter	pg_stat_bgwriter_checkpoint_avg_sync_time
gaussdb_log_dn_status	node_memory_HugePages_Free	pg_wait_events_total_wait_time	net_conntrack_dialer_conn_attempted_total
gaussdb_log_gtm_panic	node_memory_HugePages_Rsvd	pg_tables_structure_n_tup_read	node_hwmon_power_average_interval_seconds
pg_audit_logout_total	node_memory_HugePages_Surp	pg_tables_structure_n_live_tup	gs_stat_bgwriter_checkpoint_avg_sync_time
pg_audit_revoke_total	node_memory_Inactive_bytes	pg_tables_structure_n_dead_tup	net_conntrack_listener_conn_accepted_total
pg_cpu_load_total_cpu	node_memory_MemTotal_bytes	gaussdb_successful_logins_rate	node_edac_csrow_uncorrectable_errors_total
pg_db_confl_bufferpin	node_memory_SwapFree_bytes	gaussdb_connections_used_ratio	promhttp_metric_handler_requests_in_flight
pg_redo_stat_writetim	node_netstat_Icmp6_OutMsgs	gaussdb_dead_tup_increase_rate	pg_tables_structure_last_data_changed_delay
pg_index_idx_blks_hit	node_netstat_Icmp_InErrors	gaussdb_database_increase_rate	net_conntrack_dialer_conn_established_total
os_disk_io_read_bytes	node_netstat_Ip6_OutOctets	node_memory_Committed_AS_bytes	gs_tables_structure_last_data_changed_delay

版权所有 © 华为云计算技术有限公司

os_disk_io_read_delay	node_netstat_Ip_Forwarding	node_memory_Hugepagesize_bytes	pg_sql_statement_statistics_sort_spill_count
node_filefd_allocated	node_netstat_Tcp_CurrEstab	node_memory_MemAvailable_bytes	pg_sql_statement_statistics_hash_spill_count
node_filesystem_files	node_netstat_Udp6_InErrors	node_memory_NFS_Unstable_bytes	gs_sql_statement_statistics_sort_spill_count
node_sockstat_TCP_mem	node_network_protocol_type	node_memory_SReclaimable_bytes	gs_sql_statement_statistics_hash_spill_count
node_sockstat_UDP_mem	node_sockstat_FRAG6_memory	node_memory_VmallocChunk_bytes	node_hwmon_power_average_interval_max_seconds
gs_audit_logout_total	node_sockstat_sockets_used	node_memory_VmallocTotal_bytes	node_hwmon_power_average_interval_min_seconds
gs_audit_revoke_total	node_softnet_dropped_total	node_memory_WritebackTmp_bytes	pg_stat_bgwriter_checkpoint_proactive_triggering_ratio
gs_cpu_load_total_cpu	node_textfile_scrape_error	node_netstat_Udp6_OutDatagrams	gs_stat_bgwriter_checkpoint_proactive_triggering_ratio
gs_db_confl_bufferpin	node_timex_pps_error_total	node_netstat_Udp6_RcvbufErrors	-

2.11.3.1 X-Tuner

2.11.3.1.1 概述

X-Tuner是一款数据库集成的参数调优工具，通过结合深度强化学习和全局搜索算法等AI技术，实现在无需人工干预的情况下，获取最佳数据库参数配置。本功能不强制与数据库环境部署到一起，支持独立部署，脱离数据库安装环境独立运行。

说明

使用X-Tuner功能会尝试建立数据库连接，从而获取数据库规格。

2.11.3.1.2 使用准备

前提条件与使用事项

- 数据库状态正常、客户端能够正常连接、且要求数据库内导入数据，以便调优程序可以执行benchmark测试调优效果。
- 使用本工具需要指定登录到数据库的用户身份，要求该登录到数据库上的用户具有足够的权限，以便可以获得充足的数据库状态信息。
- 使用登录到数据库宿主机上的Linux用户，需要将**$GAUSSHOME/bin**添加到PATH环境变量中，即能够直接运行gsql、gs_guc、gs_ctl等数据库运维工具。

版权所有 © 华为云计算技术有限公司

- 本工具支持以三种模式运行，其中tune和train模式要求用户配置好benchmark运行环境，并导入数据，本工具将会通过迭代运行benchmark来判断修改后的参数是否有性能提升。
- recommend模式建议在数据库正在执行workload的过程中执行，以便获得更准确的实时workload信息。
- 本工具默认带有TPC-C、TPC-H、TPC-DS以及sysbench的benchmark运行脚本样例，如果用户使用上述benchmark对数据库系统进行压力测试，则可以对上述配置文件进行适度修改或配置。如果需要适配用户自己的业务场景，需要您参照benchmark目录中的template.py文件编写驱动您自定义benchmark的脚本文件。

原理简介

调优程序是一个独立于数据库内核之外的工具，需要提供数据库及其所在实例的用户名和登录密码信息，以便控制数据库执行benchmark进行性能测试；在启动调优程序前，要求用户测试环境交互正常，能够正常跑通benchmark测试脚本、能够正常连接数据库。

说明

如果需要调优的参数中，包含重启数据库后才能使修改生效的参数，那么在调优过程中数据库将会重启多次。如果用户的数据库正在执行作业，请慎用train与tune模式。

调优程序X-Tuner包含三种运行模式，分别是:

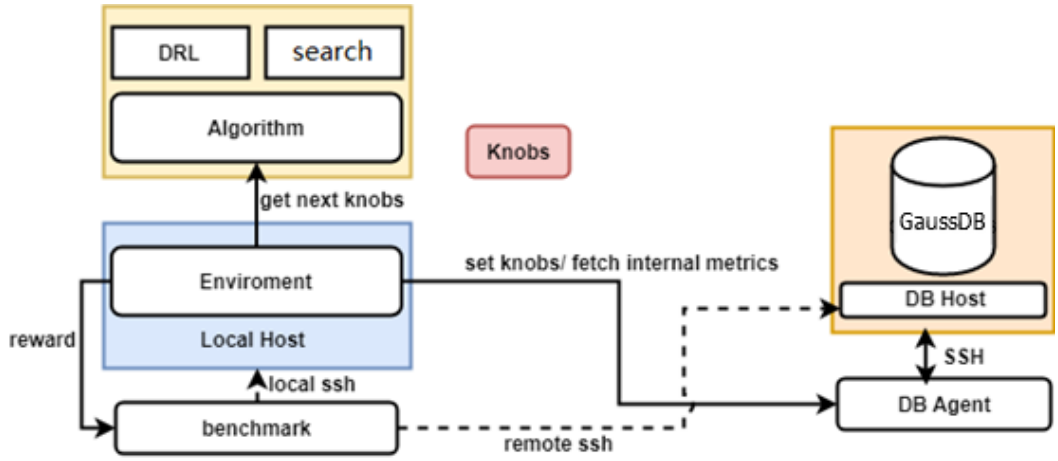
- recommend：通过用户指定的用户名等信息登录到数据库环境中，获取当前正在运行的workload特征信息，根据上述特征信息生成参数推荐报告。报告当前数据库中不合理的参数配置和潜在风险等；输出根据当前正在运行的workload行为和特征；输出推荐的参数配置。**该模式是秒级的，不涉及数据库的重启操作，其他模式可能需要反复重启数据库**。
- train：通过用户提供的benchmark信息，不断地进行参数修改和benchmark的执行。通过反复的迭代过程，训练强化学习模型，以便用户在后面通过tune模式加载该模型进行调优。
- tune：使用优化算法进行数据库参数的调优，当前支持两大类算法，一种是深度强化学习，另一种是全局搜索算法（全局优化算法）。深度强化学习模式要求先运行train模式，生成训练后的调优模型，而使用全局搜索算法则不需要提前进行训练，可以直接进行搜索调优。

须知

如果在tune模式下，使用深度强化学习算法，要求必须有一个训练好的模型，且要求**训练该模型时的参数与进行调优时的参数列表（包括max与min）必须一致**。

版权所有 © 华为云计算技术有限公司

图 2-10 X-Tuner 结构图



X-Tuner的整体架构如上方X-Tuner 结构图所示，系统可以分为：

- DB侧：通过DB_Agent模块对数据库实例进行抽象，通过该模块可以获取数据库内部的状态信息、当前数据库参数、以及设置数据库参数等。DB侧包括登录数据库环境使用的SSH连接。
- 算法侧：用于调优的算法包，包括全局搜索算法（如贝叶斯优化、粒子群算法等）和深度强化学习（如DDPG）。
- X-Tuner主体逻辑模块：通过Environment模块进行封装，每一个step就是一次调优过程。整个调优过程通过多个step进行迭代。
- benchmark：由用户指定的benchmark性能测试脚本，用于运行benchmark作业，通过跑分结果反映数据库系统性能优劣。

说明

应确保benchmark脚本跑分结果越大表示性能越好。

例如TPCH这种衡量SQL语句整体执行时长的benchmark，可以通过取总体执行时间的相反数作为benchmark的输出分数。

X-Tuner 的运行和安装方法

执行下述命令即可获取xtuner功能帮助

```
gs_dbmind component xtuner --help
```

用户可据此给定不同的命令行执行相应的功能。

X-Tuner 的配置文件说明

X-Tuner在运行前需要加载配置文件，用户可以通过 **--help** 命令查看默认加载的配置文件绝对路径：

```
-x TUNER_CONFIG_FILE, --tuner-config-file TUNER_CONFIG_FILE
                      This is the path of the core configuration file of the
                      X-Tuner. You can specify the path of the new
                      configuration file. The default path is DBMINDPATH/dbmind/components/xtuner/tuner/
xtuner.conf.
                      You can modify the configuration file to control the
                      tuning process.
```


版权所有 © 华为云计算技术有限公司

修改配置文件的配置项可以指引X-Tuner执行不同的动作，用户可以根据自己的不同需求来修改配置文件的内容，配置文件的配置项说明详见**表2-25**。如果需要修改配置文件的加载路径，则可以通过选项**-x**命令行选项来指定。

Benchmark 的选择与配置

Benchmark的驱动脚本存放路径为X-Tuner目录的子目录benchmark中。X-Tuner自带常用的benchmark驱动脚本，例如基于时间周期的探测脚本（**默认**）、TPC-C、TPC-H等。X-Tuner通过调用benchmark/__init__.py文件中 **get_benchmark_instance()**命令来加载不同的benchmark驱动脚本，获取benchmark驱动实例。其中，benchmark驱动脚本的格式说明如下：

- 驱动脚本文件名：表示benchmark的名字，该名字用于表示驱动脚本的唯一性，可通过在X-Tuner的配置文件中的配置项**benchmark_script**来指定选择加载哪一个benchmark驱动脚本。
- 驱动脚本内容三要素：path变量、cmd变量以及run函数。

下面分别介绍驱动脚本的内容三要素：

1. path变量：表示benchmark脚本的存放地址，可以直接在驱动脚本中修改，也可以通过配置文件的benchmark_path配置项来指定。
2. cmd变量：表示执行benchmark脚本需要运行的命令，可以直接在驱动脚本中修改，也可以通过配置文件的benchmark_cmd配置项来指定。cmd中的文本允许使用占位符，用于获取某些运行cmd命令时的必要信息，使用示例参见TPC-H驱动脚本示例。这些占位符包括：
   - {host}：数据库宿主机的IP地址
   - {port}：数据库实例的侦听端口号
   - {user}：登录数据库系统上的用户名
   - {password}：与登录数据库系统上的用户相匹配的密码
   - {db}：正在进行调优的数据库名
3. run函数：该函数的函数签名为：

   ```
   def run(remote_server, local_host) -> float:
   ```

   其中，返回数据类型为float，表示benchmark执行后的评估分数值，要求该值越大表示性能越好，例如使用TPC-C跑分结果tpmC即可作为返回值，TPC-H的全部SQL语句执行总时间的相反数（取相反数后可保证返回值越大则性能越好）也可作为返回值。

   remote_server变量是X-Tuner程序传递给脚本使用的远端主机（数据库宿主机）的shell命令接口，local_host变量是X-Tuner程序传递给脚本使用的本地主机（运行X-Tuner脚本的主机）的shell命令接口。上述shell命令接口提供的方法包括：

   ```
   exec_command_sync(command, timeout)
   功能：该方法用于在主机上执行shell命令。
   参数列表：
   command 必选，数据类型可以是str, 以及元素为str类型的list或tuple;
   timeout 可选，表示命令执行的超时时长，单位是秒。
   返回值：
   返回二元组 (stdout, stderr)，stdout表示标准输出流结果，stderr表示标准错误流结果，数据类型均为str.
   exit_status
   功能：该属性表示最近一条shell命令执行后的退出状态码(exit status code)。
   说明：一般情况，退出状态码为0表示执行正常，非0表示存在错误。
   ```

Benchmark驱动脚本示例说明

1. TPC-C 驱动脚本

   ```
   from tuner.exceptions import ExecutionError
   ```

 版权所有 © 华为云计算技术有限公司

```
# WARN: You need to download the benchmark-sql test tool to the system,
# The program starts the test by running the following command:
path = '/path/to/benchmarksql/run'  # TPC-C测试脚本benchmark-sql 的存放路径
cmd = "./runBenchmark.sh props.gs"  # 自定义一个名为 props.gs 的benchmark-sql测试配置文件


def run(remote_server, local_host):
    # 切换到 TPC-C 脚本目录下，清除历史错误日志，然后运行测试命令。
    # 此处建议等待几秒钟，因为benchmark-sql 测试脚本生成最终测试报告是通过一个shell脚本实现的，整个过程会有延迟，
    # 为了保证能够获取到最终的tpmC数值报告，这里选择等待3秒钟。
    stdout, stderr = remote_server.exec_command_sync(['cd %s' % path, 'rm -rf benchmarksql-error.log', cmd, 'sleep 3'])
    # 如果标准错误流中有数据，则报异常退出。
    if len(stderr) > 0:
        raise ExecutionError(stderr)

    # 寻找最终tpmC结果
    tpmC = None
    split_string = stdout.split()  # 对标准输出流结果进行分词。
    for i, st in enumerate(split_string):
        # 在5.0版本的benchmark-sql中，tpmC最终测试结果数值在 ‘(NewOrders)’关键字的后两位，正常情况下，找到该字段后直接返回即可。
        if "(NewOrders)" in st:
            tpmC = split_string[i + 2]
            break
    stdout, stderr = remote_server.exec_command_sync(
        "cat %s/benchmarksql-error.log" % path)
    nb_err = stdout.count("ERROR:")  # 判断整个benchmark运行过程中，是否有报错，记录报错的错误数
    return float(tpmC) - 10 * nb_err  # 这里将报错的错误数作为一个惩罚项，惩罚系数为10，越高的惩罚系数表示越看中报错的数量.
```

2. TPC-H驱动脚本

```
import time

from tuner.exceptions import ExecutionError

# WARN: You need to import data into the database and SQL statements in the following path will be executed.
# The program automatically collects the total execution duration of these SQL statements.
path = '/path/to/tpch/queries'  # 存放TPC-H测试用的SQL脚本目录
cmd = "gsql -U {user} -W {password} -d {db} -p {port} -f {file}"  # 需要运行TPC-H测试脚本的命令，一般使用'gsql -f 脚本文件' 来运行


def run(remote_server, local_host):
    # 遍历当前目录下所有的测试用例文件名
    find_file_cmd = "find . -type f -name '*.sql'"
    stdout, stderr = remote_server.exec_command_sync(['cd %s' % path, find_file_cmd])
    if len(stderr) > 0:
        raise ExecutionError(stderr)
    files = stdout.strip().split('\n')
    time_start = time.time()
    for file in files:
        # 使用 file 变量替换 {file}，然后执行该命令行。
        perform_cmd = cmd.format(file=file)
        stdout, stderr = remote_server.exec_command_sync(['cd %s' % path, perform_cmd])
        if len(stderr) > 0:
            print(stderr)
    # 代价为全部测试用例的执行总时长
    cost = time.time() - time_start
    # 取相反数，适配run 函数的定义：返回结果越大表示性能越好。
    return - cost
```

2.11.3.1.3 使用示例

X-Tuner支持三种模式，分别是获取参数诊断报告的recommend模式、训练强化学习模型的train模式、以及使用算法进行调优的tune模式。上述三种模式可以通过命令行参数来区别，通过配置文件来指定具体的细节。

 版权所有 © 华为云计算技术有限公司

配置数据库连接信息

三种模式连接数据库的配置项是相同的，有两种方式：一种是直接通过命令行输入详细的连接信息，另一种是通过JSON格式的配置文件输入，下面分别对两种指定数据库连接信息的方法进行说明。

1. 通过命令行指定：

   分别传递 --db-name --db-user --port --host --host-user 参数，可选 --host-ssh-port 参数，例如：

   ```
   gs_dbmind component xtuner recommend --db-name testdb --db-user omm --port 5678 --host 192.168.1.100 --host-user omm
   ```

2. 通过JSON格式的连接信息配置文件指定：

   JSON配置文件的示例如下，并假设文件名为 connection.json：

   ```
   {
     "db_name": "testdb",  # 数据库名
     "db_user": "dba",       # 登录到数据库上的用户名
     "host": "127.0.0.1",    # 数据库宿主机的IP地址
     "host_user": "dba",     # 登录到数据库宿主机的用户名
     "port": 5432,           # 数据库的侦听端口号
     "ssh_port": 22          # 数据库宿主机的SSH侦听端口号
   }
   ```

   则可通过 -f connection.json 传递。

说明

为了防止密码泄露，配置文件和命令行参数中默认都不包含密码信息，用户在输入上述连接信息后，程序会采用交互式的方式要求用户输数据库密码以及操作系统登录用户的密码。

recommend 模式使用示例

对recommend模式生效的配置项为scenario，若为auto，则自动检测workload类型。

执行下述命令，获取诊断结果：

```
gs_dbmind component xtuner recommend -f connection.json
```

则可以生成诊断报告如下：

 版权所有 © 华为云计算技术有限公司

图 2-11 recommend 模式生成的报告示意图

```
Start to recommend knobs. Just a moment, please.
************************* Knob Recommendation Report *************************
INFO:
+---------------------------------------+----------------------+
|                Metric                 |        Value         |
+---------------------------------------+----------------------+
|             workload_type             |          tp          |
|         average_connection_age        |          0           |
|         dirty_background_bytes        |          0           |
|             temp_file_size            |          0           |
|          current_connections          |         0.0          |
|          current_locks_count          |         0.0          |
|      current_prepared_xacts_count     |         0.0          |
|          rollback_commit_ratio        | 0.09168372786632421  |
|                 uptime                |  0.122942879722222   |
| checkpoint_proactive_triggering_ratio |  0.488598416181662   |
|         fetched_returned_ratio        |  0.9915911452033203  |
|             cache_hit_rate            |  0.9979742937232552  |
|            read_write_ratio           |  123.86665549830312  |
|           all_database_size           |  134154882.48046875  |
|          search_modify_ratio          |  187.59523392981777  |
|                ap_index               |  2.3759498376861847  |
|            current_free_mem           |       31161892       |
|              os_mem_total             |       32779460       |
|        checkpoint_avg_sync_time       |   381.359603091308   |
|  checkpoint_dirty_writing_time_window |        450.0         |
|            max_processes              |          46          |
|          track_activity_size          |         46.0         |
|            write_tup_speed            |   6810.36309197048   |
|                used_mem               |     73988850.25      |
|             os_cpu_count              |          8           |
|              block_size               |         8.0          |
|            read_tup_speed             |   845237.440716804   |
|      shared_buffer_toast_hit_rate     |  98.16007359705611   |
|      shared_buffer_tidx_hit_rate      |  99.11667280088332   |
|       shared_buffer_idx_hit_rate      |  99.74473859023848   |
|      shared_buffer_heap_hit_rate      |  99.81099543813004   |
|           enable_autovacuum           |         True         |
|                is_64bit               |         True         |
|                 is_hdd                |         True         |
|             load_average              | [1.89, 3.175, 3.005] |
+---------------------------------------+----------------------+
p.s: The unit of storage is kB.
WARN:
[0]. The number of CPU cores is a little small. Please do not run too high concu
rrency. You are recommended to set max_connections based on the number of CPU co
res. If your job does not consume much CPU, you can also increase it.
[1]. The value of wal_buffers is a bit high. Generally, an excessively large val
ue does not bring better performance. You can also set this parameter to -1. The
 database automatically performs adaptation.
BAD:
[0]. The database runs for a short period of time, and the database description
may not be accumulated. The recommendation result may be inaccurate.
************************* Recommended Knob Settings **************************
+---------------------------+-----------+---------+----------+---------+
|            name           | recommend |   min   |   max    | restart |
+---------------------------+-----------+---------+----------+---------+
|       shared_buffers      |  1638973  |  614614 | 1884818  |   True  |
|      max_connections      |     43    |    24   |   500    |   True  |
|    effective_cache_size   |  1638973  | 1638973 | 24584595 |  False  |
|        wal_buffers        |   51217   |   2048  |  51217   |   True  |
|      random_page_cost     |    3.0    |   2.0   |   3.0    |  False  |
| default_statistics_target |    100    |    10   |   150    |  False  |
+---------------------------+-----------+---------+----------+---------+
```

在上述报告中，推荐了该环境上的数据库参数配置，并进行了风险提示。报告同时生成了当前workload的特征信息，其中有几个特征是比较有参考意义的：

版权所有 © 华为云计算技术有限公司

- temp_file_size：产生的临时文件数量，如果该结果大于0，则表明系统使用了临时文件。使用过多的临时文件会导致性能不佳，如果可能的话，需要提高work_mem参数的配置。
- cache_hit_rate：shared_buffer的缓存命中率，表明当前workload使用缓存的效率。
- read_write_ratio：数据库作业的读写比例。
- search_modify_ratio：数据库作业的查询与修改数据的比例。
- ap_index：表明当前workload的AP指数，取值范围是0到10，该数值越大，表明越偏向于数据分析与检索。
- workload_type：根据数据库统计信息，推测当前负载类型，分为AP、TP以及HTAP三种类型。
- checkpoint_avg_sync_time：数据库在checkpoint时，平均每次同步刷新数据到磁盘的时长，单位是毫秒。
- load_average：平均**每个CPU核心**在1分钟、5分钟以及15分钟内的负载。一般地，该数值在1左右表明当前硬件比较匹配workload、在3左右表明运行当前作业压力比较大，大于5则表示当前硬件环境运行该workload压力过大（此时一般建议减少负载或升级硬件）。

说明

- recommend模式会读取数据库中的pg_stat_database以及pg_stat_bgwriter等系统表中的信息，需要登录到数据库上的用户具有足够的权限（建议为**管理员权限**，可通过alter user username sysadmin；授予username相应的权限）。
- 由于某些系统表会一直记录统计信息，这可能会对负载特征识别造成干扰，因此建议先清空某些系统表的统计信息，运行一段时间的workload后再使用recommend模式进行诊断，以便获得更准确的结果。清除统计信息的方法为：

```
select pg_stat_reset_shared('bgwriter');
select pg_stat_reset();
```

train 模式使用示例

该模式是用来训练深度强化学习模型的，与该模式有关的配置项为：

- rl_algorithm：用于训练强化学习模型的算法，当前支持设置为ddpg.
- rl_model_path：训练后生成的强化学习模型保存路径。
- rl_steps：训练过程的最大迭代步数。
- max_episode_steps：每个回合的最大步数。
- scenario：明确指定的workload类型，如果为auto则为自动判断。在不同模式下，推荐的调优参数列表也不一样。
- tuning_list：**用户指定需要调哪些参数，如果不指定，则根据workload类型自动推荐应该调的参数列表。**如需指定，则tuning_list表示调优列表文件的路径。一个调优列表配置文件的文件内容示例如下：

```
{
  "work_mem": {
    "default": 65536,
    "min": 65536,
    "max": 655360,
    "type": "int",
    "restart": false
  },
  "shared_buffers": {
    "default": 32000,
    "min": 16000,
```

 版权所有 © 华为云计算技术有限公司

```
  "max": 64000,
  "type": "int",
  "restart": true
 },
 "random_page_cost": {
  "default": 4.0,
  "min": 1.0,
  "max": 4.0,
  "type": "float",
  "restart": false
 },
 "enable_nestloop": {
  "default": true,
  "type": "bool",
  "restart": false
 }
}
```

待上述配置项配置完成后，可以通过下述命令启动训练：

```
gs_dbmind component xtuner train -f connection.json
```

训练完成后，会在配置项rl_model_path指定的目录中生成模型文件。

tune 模式使用示例

tune模式支持多种算法，包括基于强化学习（Reinforcement Learning, RL）的DDPG算法、基于全局搜索（Global OPtimization algorithm, GOP）算法的贝叶斯优化算法（Bayesian Optimization）以及粒子群算法（Particle Swarm Optimization, PSO）。

与tune模式相关的配置项为：

- tune_strategy：指定选择哪种算法进行调优，支持rl（使用强化学习模型进行调优）、gop（使用全局搜索算法）以及auto（自动选择）。若该参数设置为rl，则rl相关的配置项生效。除前文提到过的train模式下生效的配置项外，test_episode配置项也生效，该配置项表明调优过程的最大回合数，该参数直接影响了调优过程的执行时间（一般地，数值越大越耗时）。
- gop_algorithm：选择何种全局搜索算法，支持bayes以及pso。
- max_iterations：最大迭代轮次，数值越高搜索时间越长，效果往往越好。
- particle_nums：在PSO算法上生效，表示粒子数。
- scenario与tuning_list见上文train模式中的描述。

待上述配置项配置完成后，可以通过下述命令启动调优：

```
gs_dbmind component xtuner tune -f connection.json
```

⚠ 注意

在使用tune和train模式前，用户需要先导入benchmark所需数据并检查benchmark能否正常跑通。调优过程结束后，调优程序会自动恢复调优前的数据库参数配置。

2.11.3.1.4 获取帮助

启动调优程序之前，可以通过如下命令获取帮助信息：

```
gs_dbmind component xtuner --help
```

输出帮助信息结果如下：

版权所有 © 华为云计算技术有限公司

```
usage:  [-h] [--database DATABASE] [--db-user DB_USER] [--db-port DB_PORT] [--db-host DB_HOST] [--
host-user HOST_USER] [--host-ssh-port HOST_SSH_PORT] [-f DB_CONFIG_FILE] [-x TUNER_CONFIG_FILE]
[-v]
       {train,tune,recommend}

X-Tuner: a self-tuning tool integrated by GaussDB.

positional arguments:
  {train,tune,recommend}
                        Train a reinforcement learning model or tune database by model. And also can recommend
best_knobs according to your workload.

optional arguments:
  -h, --help            show this help message and exit
  -f DB_CONFIG_FILE, --db-config-file DB_CONFIG_FILE
                        You can pass a path of configuration file otherwise you should enter database information
by command arguments manually. Please see the template file share/server.json.template.
  -x TUNER_CONFIG_FILE, --tuner-config-file TUNER_CONFIG_FILE
                        This is the path of the core configuration file of the X-Tuner. You can specify the path of
the new configuration file. The default path is /path/to/config/file. You can modify the configuration file to
control the tuning process.
  -v, --version         show program's version number and exit

Database Connection Information:
  --database DATABASE, --db-name DATABASE
                        The name of database where your workload running on.
  --db-user DB_USER     Use this user to login your database. Note that the user must have sufficient
permissions.
  --db-port DB_PORT, --port DB_PORT
                        Use this port to connect with the database.
  --db-host DB_HOST, --host DB_HOST
                        The IP address of your database installation host.
  --host-user HOST_USER
                        The login user of your database installation host.
  --host-ssh-port HOST_SSH_PORT
                        The SSH port of your database installation host.
```

2.11.3.1.5 命令参考

表 2-24 命令行参数

参数	参数说明	取值范围
mode	指定调优程序运行的模式	train，tune，recommend
--tuner-config-file, -x	X-Tuner的核心参数配置文件路径，默认路径为安装目录下的xtuner.conf	-
--db-config-file, -f	调优程序的用于登录到数据库宿主机上的连接信息配置文件路径，若通过该文件配置数据库连接信息，则下述数据库连接信息可省略	-
--db-name	指定需要调优的数据库名	-
--db-user	指定以何用户身份登录到调优的数据库上	-
--port, --db-port	数据库的侦听端口	1024-65535
--host, --db-host	数据库实例的宿主机IP	-

 版权所有 © 华为云计算技术有限公司

参数	参数说明	取值范围
--host-user	指定以何用户身份登录到数据库实例的宿主机上，要求改用户名的环境变量中可以找到gsql、gs_ctl等数据库运维工具。	-
--host-ssh-port	数据库实例所在宿主机的SSH端口号，可选，默认为22	0-65535
--help, -h	返回帮助信息	-
--version, -v	返回当前工具版本号	-

表 2-25 配置文件中的参数详解

参数名	参数说明	取值范围
logfile	生成的日志存放路径	-
output_tuning_result	可选，调优结果的保存路径	-
verbose	是否打印详情	on, off
recorder_file	调优中间信息的记录日志存放路径	-
tune_strategy	调优模式下采取哪种策略	rl, gop, auto
drop_cache	是否在每一个迭代轮次中进行drop cache，drop cache可以使benchmark跑分结果更加稳定。若启动该参数，则需要将登录的系统用户加入到 /etc/sudoers 列表中，同时为其增加 **NOPASSWD** 权限（由于该权限可能过高，建议临时启用该权限，调优结束后关闭）。	on, off
used_mem_penalty_term	数据库使用总内存的惩罚系数，用于防止通过无限量占用内存而换取的性能表现。该数值越大，惩罚力度越大。	建议0 ~ 1
rl_algorithm	选择何种RL算法	ddpg
rl_model_path	RL模型保存或读取路径，包括保存目录名与文件名前缀。在train 模式下该路径用于保存模型，在tune模式下则用于读取模型文件	-
rl_steps	深度强化学习算法迭代的步数	-
max_episode_steps	每个回合的最大迭代步数	-
test_episode	使用RL算法进行调优模式的回合数	-

版权所有 © 华为云计算技术有限公司

参数名	参数说明	取值范围
gop_algorithm	采取何种全局搜索算法	bayes, pso
max_iterations	全局搜索算法的最大迭代轮次（并非确定数值，可能会根据实际情况多跑若干轮）	-
particle_nums	PSO算法下的粒子数量	-
benchmark_script	使用何种benchmark驱动脚本，该选项指定加载benchmark路径下同名文件，默认支持TPC-C、TPC-H等典型benchmark	tpcc, tpch, tpcds, sysbench ...
benchmark_path	benchmark 脚本的存储路径，若没有配置该选项，则使用benchmark驱动脚本中的配置	-
benchmark_cmd	启动benchmark 脚本的命令，若没有配置该选项，则使用benchmark驱动脚本中的配置	-
benchmark_period	仅对 period benchmark有效，表明整个benchmark的测试周期是多少，单位是秒	-
scenario	用户指定的当前workload所属的类型	tp, ap, htap
tuning_list	准备调优的参数列表文件，可参考 share/knobs.json.template 文件	-

2.11.3.1.6 常见问题处理

- 数据库实例连接失败：请检查数据库实例的情况，是否数据库实例出现了问题或安全权限配置（gs_hba.conf文件中的配置项）不正确。
- 重启失败：请检查数据库实例健康情况，确保数据库实例工作正常。
- 跑TPC-C作业时发现性能越来越慢：TPC-C等高并发场景下的压力测试，往往伴随着大量的数据修改。由于每一次测试并非是幂等的（TPC-C数据库数据量的增加、没有进行vacuum 清理掉失效元组、数据库没有触发checkpoint、没有进行drop cache等），因此一般建议TPC-C等伴随着较多数据写入的benchmark应该每隔一段时间（视具体并发量、执行时长的不同而异）重新导入一次数据。
- TPC-C 跑作业时，TPC-C驱动脚本报异常 “TypeError: float() argument must be a string or a number, not 'NoneType'”（不能将None转换为float类型）：这是因为没有获取到TPC-C的压测返回结果，造成该问题的原因比较多，请首先手动检测是否能够跑通TPC-C并能够获取返回结果。若无上述问题，则建议将 TPC-C 驱动脚本中的命令列表中的 "sleep" 命令延迟时间设得更大一些。

2.11.3.2 Index-advisor

本节介绍索引推荐的功能，共包含三个子功能：单query索引推荐、虚拟索引和workload级别索引推荐。

 版权所有 © 华为云计算技术有限公司

2.11.3.2.1 单 query 索引推荐

单query索引推荐功能支持用户在数据库中直接进行操作，本功能基于查询语句的语义信息和数据库的统计信息，对用户输入的单条查询语句生成推荐的索引。本功能涉及的函数接口如下。

表 2-26 单 query 索引推荐功能的接口

函数名	参数	功能
gs_index_advise	SQL语句字符串	针对单条查询语句生成推荐索引。

说明

- 本功能仅支持单条SELECT、INSERT、DELETE、UPDATE类型的语句，不支持其他类型的SQL语句。
- 本功能使用优化器采样结果，用户需要保证最近analyze过，否则优化器结果不准确。
- 本功能暂不支持段页式表、普通视图、物化视图、全局临时表、二级分区表以及密态数据库。
- 如果对ustore表相关语句进行索引推荐，本功能可能无法保证结果的准确性。

使用方法

使用上述函数，获取针对该query生成的推荐索引，推荐结果由索引的表名和列名组成。

例如：

```
gaussdb=> select "table", "column" from gs_index_advise('SELECT c_discount from bmsql_customer where c_w_id = 10');
     table      |  column
----------------+----------
 bmsql_customer | c_w_id
(1 row)
```

上述结果表明：应当在 bmsql_customer 的 c_w_id 列上创建索引，例如可以通过下述SQL语句创建索引：

```
CREATE INDEX idx on bmsql_customer(c_w_id);
```

某些SQL语句，也可能被推荐创建联合索引，例如：

```
gaussdb=# select "table", "column" from gs_index_advise('select name, age, sex from t1 where age >= 18 and age < 35 and sex = ''f'';');
 table | column
-------+------------
 t1    | age, sex
(1 row)
```

则上述语句表明应该在表 t1 上创建一个联合索引 (age, sex)， 则可以通过下述命令创建：

```
CREATE INDEX idx1 on t1(age, sex);
```

针对分区表可推荐具体索引类型，例如：

```
gaussdb=# select "table", "column", "indextype" from gs_index_advise('select name, age, sex from range_table where age = 20;');
```

 版权所有 © 华为云计算技术有限公司

```
 table | column | indextype
-------+--------+-----------
 t1    | age    | global
(1 row)
```

说明

系统函数gs_index_advise()的参数是文本型，如果参数中存在如单引号（'）等特殊字符，可以使用单引号（'）进行转义，可参考上述示例。

2.11.3.2.2 虚拟索引

虚拟索引功能支持用户在数据库中直接进行操作，本功能将模拟真实索引的建立，避免真实索引创建所需的时间和空间开销，用户基于虚拟索引，可通过优化器评估该索引对指定查询语句的代价影响。

本功能涉及的系统函数接口如下表所示：

表 2-27 虚拟索引功能的接口

函数名	参数	功能
hypopg_create_index	参数1：创建索引语句的字符串 参数2：虚拟索引级别，可选参数，可指定global或session，默认为global	创建虚拟索引。
hypopg_display_index	虚拟索引级别，可选参数，可通过参数指定global或session，默认为global	显示所有创建的虚拟索引信息。
hypopg_drop_index	索引的oid	删除指定的虚拟索引。
hypopg_reset_index	虚拟索引级别，可选参数，可通过参数指定global或session，默认为global	清除所有虚拟索引。
hypopg_estimate_size	索引的oid	估计指定索引创建所需的空间大小。

本功能涉及的GUC参数如下：

表 2-28 虚拟索引功能的 GUC 参数

参数名	功能	默认值
enable_hypo_index	是否开启虚拟索引功能	off

使用步骤

案例一：使用虚拟索引，调优等值查询

 版权所有 © 华为云计算技术有限公司

在此案例中，存在表bmsql_customer，该表是TPC-C benchmark中的一张表，此处演示在该表的c_w_id列上创建一个索引，是否可以提升某个等值查询的性能，如果该索引被使用了，则预估执行代价(cost)是多少。

步骤1 使用函数hypopg_create_index创建虚拟索引。例如：

```
gaussdb=# select * from hypopg_create_index('create index on bmsql_customer(c_w_id)');
 indexrelid |              indexname
------------+---------------------------------------
     329726 | <329726>btree_bmsql_customer_c_w_id
(1 row)
```

步骤2 开启GUC参数enable_hypo_index，该参数控制数据库的优化器进行EXPLAIN时是否考虑创建的虚拟索引。通过对特定的查询语句执行explain，用户可根据优化器给出的执行计划评估该索引是否能够提升该查询语句的执行效率。例如：

```
gaussdb=# set enable_hypo_index = on;
SET
```

开启GUC参数前，执行EXPLAIN + 查询语句：

```
gaussdb=# explain SELECT c_discount from bmsql_customer where c_w_id = 10;
                            QUERY PLAN
------------------------------------------------------------------------
 Seq Scan on bmsql_customer  (cost=0.00..52963.06 rows=31224 width=4)
   Filter: (c_w_id = 10)
(2 rows)
```

开启GUC参数后，执行EXPLAIN + 查询语句：

```
gaussdb=# explain SELECT c_discount from bmsql_customer where c_w_id = 10;
                                          QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
 [Bypass]
 Index Scan using <329726>btree_bmsql_customer_c_w_id on bmsql_customer  (cost=0.00..39678.69
rows=31224 width=4)
   Index Cond: (c_w_id = 10)
(3 rows)
```

通过对比两个执行计划可以观察到，该索引预计会降低指定查询语句的执行代价，用户可考虑创建对应的真实索引。

步骤3 （可选）使用函数hypopg_display_index展示所有创建过的虚拟索引。例如：

```
gaussdb=# select * from hypopg_display_index();
                indexname                 | indexrelid |     table      |      column      |                    indexdef
------------------------------------------+------------+----------------+------------------
+-------------------------------------------------------------
 <329726>btree_bmsql_customer_c_w_id       |     329726 | bmsql_customer | (c_w_id)         |CREATE INDEX
ON bmsql_customer USING btree (c_w_id)
 <329729>btree_bmsql_customer_c_d_id_c_w_id |     329729 | bmsql_customer | (c_d_id, c_w_id) |CREATE
INDEX ON bmsql_customer USING btree (c_d_id, c_w_id)
(2 rows)
```

步骤4 （可选）使用函数hypopg_estimate_size估计虚拟索引创建所需的空间大小（单位：字节）。例如：

```
gaussdb=> select * from hypopg_estimate_size(329729);
 hypopg_estimate_size
----------------------
             15687680
(1 row)
```

步骤5 删除虚拟索引。

使用函数hypopg_drop_index删除指定oid的虚拟索引。例如：

```
gaussdb=> select * from hypopg_drop_index(329726);
 hypopg_drop_index
-------------------
```

 版权所有 © 华为云计算技术有限公司

```
 t
(1 row)
```

使用函数hypopg_reset_index一次性清除所有创建的虚拟索引。例如：

```
gaussdb=> select * from hypopg_reset_index();
 hypopg_reset_index
--------------------

(1 row)
```

----结束

案例二：虚拟索引联合Hint，预测调优效果

Hint可以手动要求数据库优化器使用某种方式生成执行计划，因此，对于某些数据库优化器难以生成最优执行计划的场景，可以手动指定执行计划。例如对某张表中的数据进行扫描操作（Scan），可以采用tablescan, indexscan, indexonlyscan, 其分别对应了表扫描、索引扫描、覆盖索引扫描。对于后两种扫描形式，必须要求先在数据库表上存在索引才可以操作。而虚拟索引则可以实现在不创建索引的情况下，测试某个索引扫描的效果。

步骤1 创建一张表t1，并生成一定量数据，供后续测试。

```
create table t1 (id int, name text);
insert into t1 select generate_series(0, 100000), 'test';
analyze t1;
```

步骤2 测试当前优化器默认的范围检索执行计划，并获取其总代价；由于没有创建索引，该SQL语句使用的是全表扫描（SeqScan）。

```
explain select * from t1 where id > 1;
```

步骤3 在t1表的id列上新建虚拟索引。

```
-- 开启参数，以便后续执行explain时能够采用虚拟索引
set enable_hypo_index = on;
-- 创建session级别虚拟索引，该session退出后，这个虚拟索引信息也会被自动清理掉
select hypopg_create_index('create index on t1(id)','session');
```

步骤4 通过explain语句，查看该SQL语句是否能够采用该索引；由于该列的distinct值很大，且涉及回表，优化器默认不会采用该索引，该语句执行计划与步骤2无变化，仍是全表扫描（SeqScan）。

```
explain select * from t1 where id > 1;
```

步骤5 通过hint操作，手动要求走索引扫描，查看能否成功；由于指定了hint, 且存在该索引（尽管是虚拟的），仍然可以通过explain看到优化器使用了索引扫描 IndexScan。

```
-- 其中<57762>btree_t1_id是自动生成的虚拟索引名，实际操作中以创建虚拟索引时的返回值为准
explain select /*+ indexscan(t1 "<57762>btree_t1_id") */ * from t1 where id > 1;
```

----结束

说明

- 执行EXPLAIN ANALYZE不会涉及虚拟索引功能。
- 开启虚拟索引功能并执行EXPLAIN语句时，可以生成创建虚拟索引之后的执行计划；同时，indexscan/indexonlyscan hint支持虚拟索引。
- 会话级别虚拟索引在各个会话间的设置互不影响，关闭会话后将被清空。
- 与真实索引不同，虚拟索引的相关操作不可回滚。
- 虚拟索引相关函数，不支持dblink远程调用。
- 本功能暂不支持视图、物化视图。

 版权所有 © 华为云计算技术有限公司

2.11.3.2.3 workload 级别索引推荐

对于workload级别的索引推荐，用户可通过运行数据库外的脚本使用此功能，本功能将包含有多条DML语句的workload作为输入，最终生成一批可对整体workload的执行表现进行优化的索引。同时，本功能提供从日志中抽取业务数据SQL流水的功能。

前提条件

- 数据库状态正常、客户端能够正常连接。
- 当前执行用户下安装有gsql工具，该工具路径已被加入到PATH环境变量中。
- 具备Python3.7的环境。
- 若使用本功能提供的业务数据抽取功能，需提前将要收集的节点的GUC参数按如下设置：
  - log_min_duration_statement = 0
  - log_statement= 'all'

  说明

  业务数据抽取完毕建议将上述GUC参数复原，否则容易导致日志文件膨胀。

业务数据抽取脚本使用步骤

步骤1 按前提条件中要求设置相关GUC参数。

步骤2 运行命令如下：

```
gs_dbmind component extract_log [-h] [-d DATABASE] [-U DB_USER] [--start-time START_TIME] [--sql-amount SQL_AMOUNT] [--statement] [--max-reserved-period MAX_RESERVED_PERIOD] [--max-template-num MAX_TEMPLATE_NUM] [--json] log_dir file line_prefix
```

其中的输入参数依次为：

- DATABASE：（可选）数据库名称，不指定默认所有数据库。
- DB_USER：（可选）用户名称，不指定默认所有用户。
- START_TIME：（可选）日志收集的开始时间，不指定默认所有文件。
- SQL_AMOUNT：（可选）收集SQL数量的最大值，不指定默认收集所有sql。
- statement：（可选）表示收集gs_log日志中statement标识开头的SQL，不指定默认不收集。
- MAX_RESERVED_PERIOD：（可选）指定json模式下，增量收集日志中保留的模板的最大的更新时长，不指定默认都保留，单位：天。
- MAX_TEMPLATE_NUM：（可选）指定json模式下保留的最大模板数量，不指定默认都保留。
- json：（可选）指定收集日志的文件存储格式为SQL归一化后的json，不指定默认格式每条SQL占一行。
- log_dir：gs_log的存放目录。
- file：输出SQL流水文件的保存路径，即抽取出的业务数据存放的文件路径。
- line_prefix：指定每条日志信息的前缀格式，可通过show log_line_prefix查询。

使用示例：

```
gs_dbmind component extract_log $GAUSSLOG/gs_log/dn_6001 sql_log.txt '%m %c %d %p %a %x %n %e' -d testdb -U omm --start_time '2021-07-06 00:00:00' --statement
```

 版权所有 © 华为云计算技术有限公司

说明

若指定-d/-U参数，日志打印每条日志信息的前缀格式需包含%d、%u，若需要抽取事务，必须指定%p，详见log_line_prefix参数。max_template_num参数设置建议不超5000条，避免workload索引推荐执行时间过长。

步骤3 将步骤1中设置的GUC参数还原为设置前的值。

----结束

索引推荐脚本使用步骤

步骤1 准备好包含有多条DML语句的文件作为输入的workload，文件中每条语句占据一行。用户可从数据库的离线日志中获得历史的业务语句。

步骤2 运行命令如下：

```
gs_dbmind component index_advisor
[-h] [--db-host DB_HOST] [-U DB_USER] --schema SCHEMA [--max-index-num MAX_INDEX_NUM] [--max-index-storage MAX_INDEX_STORAGE] [--multi-iter-mode] [--max-n-distinct MAX_N_DISTINCT] [--min-improved-rate MIN_IMPROVED_RATE] [--max-index-columns MAX_INDEX_COLUMNS] [--min-reltuples MIN_RELTUPLES] [--multi-node] [--json] [--driver] [--show-detail] [--show-benefits] [--advise_gsi] [multi_thread_num MULTI_THREAD_NUM] db_port database file
```

密码通过管道输入或交互式输入，对于免密用户，任意输入都可通过检验。

表 2-29 命令行参数

参数	参数说明	取值范围
-h	返回帮助信息。	-
--db-host DB_HOST	（可选）连接数据库的主机号。	-
-U DB_USER	（可选）连接数据库的用户名。	-
--schema SCHEMA	模式名称。	-
--max-index-num MAX_INDEX_NUM	（可选）最大的索引推荐数目。	>=1
--max-index-storage MAX_INDEX_STORAGE	（可选）最大的索引集合空间大小。	>=1
--multi-iter-mode	（可选）算法模式，可通过是否设置该参数来切换算法。	-
--max-n-distinct MAX_N_DISTINCT	（1/distinct数）的最大值，默认为0.01。	0-1
--min-improved-rate MIN_IMPROVED_RATE	索引的最大提升幅度，默认为0.1，即提升10%。	0-1
--max-index-columns MAX_INDEX_COLUMNS	联合索引的最大列数（默认为4）。	>=1
--min-reltuples MIN_RELTUPLES	表的最小行数，默认为10000。	>0

版权所有 © 华为云计算技术有限公司

参数	参数说明	取值范围
--multi-node	（可选）指定是否为分布式数据库。	-
--json	（可选）指定workload语句的文件路径格式为SQL归一化后的json。	-
--driver	（可选）指定是否使用python驱动器连接数据库，默认gsql连接。	-
--show-detail	（可选）是否显示当前推荐索引集合的详细优化信息。	-
--show-benefits	（可选） 是否显示收益信息。	-
db_port	连接数据库的端口号。	-
database	连接数据库的名字。	-
file	包含workload语句的文件路径。	-
advise_gsi	（可选）集中式不支持。	-
multi_thread_num	（可选）以多线程运行脚本，指定线程数。	[1,64]

例如：

```
gs_dbmind component index_advisor 6001 testdb tpcds_log.txt --schema public --max_index_num 10
```

结果在屏幕输出，包含候选索引、推荐索引、已创建索引、无用索引（该给定的workload里面没有用到系统中的索引列表）、冗余索引（当前系统中重复创建的索引）以及历史有效索引，如下：

```
################################## Generate candidate indexes
##################################
table:  public.catalog_returns columns:  cr_return_amount
table:  public.catalog_sales columns:  cs_item_sk
table:  public.catalog_sales columns:  cs_sold_date_sk
table:  public.customer_address columns:  ca_city type:  global
table:  public.customer_address columns:  ca_state, ca_county type:  global
table:  public.customer_demographics columns:  cd_demo_sk type:  local
table:  public.date_dim columns:  d_month_seq type:  global
table:  public.date_dim columns:  d_year type:  global
table:  public.date_dim columns:  d_date_sk type:  local
table:  public.date_dim columns:  d_month_seq type:  local
table:  public.date_dim columns:  d_year type:  local
table:  public.item columns:  i_class type:  global
table:  public.item columns:  i_manager_id, i_brand_id type:  global
table:  public.item columns:  i_manager_id, i_category_id type:  global
table:  public.item columns:  i_manufact_id type:  global
table:  public.item columns:  i_product_name type:  global
table:  public.store_returns columns:  sr_cdemo_sk
table:  public.store_returns columns:  sr_reason_sk
table:  public.store_returns columns:  sr_return_amt
table:  public.store_sales columns:  ss_item_sk, ss_sold_date_sk
table:  public.store_sales columns:  ss_store_sk
table:  public.time_dim columns:  t_time_sk type:  local
table:  public.web_returns columns:  wr_return_amt
table:  public.web_sales columns:  ws_item_sk
table:  public.web_sales columns:  ws_web_page_sk, ws_ship_hdemo_sk, ws_sold_time_sk
################################## Determine optimal indexes
##################################
CREATE INDEX idx_catalog_sales_cs_item_sk ON public.catalog_sales(cs_item_sk);
```

 版权所有 © 华为云计算技术有限公司

```
CREATE INDEX idx_catalog_sales_cs_sold_date_sk ON public.catalog_sales(cs_sold_date_sk);
CREATE INDEX idx_customer_demographics_local_cd_demo_sk ON
public.customer_demographics(cd_demo_sk) local;
CREATE INDEX idx_item_global_i_manufact_id ON public.item(i_manufact_id) global;
CREATE INDEX idx_store_returns_sr_cdemo_sk ON public.store_returns(sr_cdemo_sk);
CREATE INDEX idx_store_sales_ss_item_sk_ss_sold_date_sk ON public.store_sales(ss_item_sk,
ss_sold_date_sk);
CREATE INDEX idx_store_sales_ss_store_sk ON public.store_sales(ss_store_sk);
CREATE INDEX idx_web_sales_ws_web_page_sk_ws_ship_hdemo_sk_ws_sold_time_sk ON
public.web_sales(ws_web_page_sk, ws_ship_hdemo_sk, ws_sold_time_sk);
################################## Created indexes
################################
public: CREATE UNIQUE INDEX ship_mode_pkey ON ship_mode USING btree (sm_ship_mode_sk)
LOCAL(PARTITION p_list_15_sm_ship_mode_sk_idx, PARTITION p_list_14_sm_ship_mode_sk_idx, PARTITION
p_list_13_sm_ship_mode_sk_idx, PARTITION p_list_12_sm_ship_mode_sk_idx, PARTITION
p_list_11_sm_ship_mode_sk_idx, PARTITION p_list_10_sm_ship_mode_sk_idx, PARTITION
p_list_9_sm_ship_mode_sk_idx, PARTITION p_list_8_sm_ship_mode_sk_idx, PARTITION
p_list_7_sm_ship_mode_sk_idx, PARTITION p_list_6_sm_ship_mode_sk_idx, PARTITION
p_list_5_sm_ship_mode_sk_idx, PARTITION p_list_4_sm_ship_mode_sk_idx, PARTITION
p_list_3_sm_ship_mode_sk_idx, PARTITION p_list_2_sm_ship_mode_sk_idx, PARTITION
p_list_1_sm_ship_mode_sk_idx)  TABLESPACE pg_default;
public: CREATE INDEX temptable_int2_int3_int4_idx ON temptable USING btree (int2, int3, int4)
TABLESPACE pg_default;
public: CREATE INDEX temptable_int2_int3_idx ON temptable USING btree (int2, int3) TABLESPACE
pg_default;
public: CREATE INDEX temptable_int1_int2_int3_idx ON temptable USING btree (int1, int2, int3)
TABLESPACE pg_default;
public: CREATE INDEX temptable_int1_int2_idx ON temptable USING btree (int1, int2) TABLESPACE
pg_default;
public: CREATE INDEX temptable_int1_idx ON temptable USING btree (int1) TABLESPACE pg_default;
################################## Current workload useless indexes
################################
DROP INDEX temptable_int2_int3_int4_idx;
DROP INDEX temptable_int2_int3_idx;
DROP INDEX temptable_int1_int2_int3_idx;
DROP INDEX temptable_int1_int2_idx;
DROP INDEX temptable_int1_idx;
################################## Redundant indexes
################################
DROP INDEX public.test1_age_idx;(CREATE INDEX test1_age_idx ON test1 USING btree (age) TABLESPACE
pg_default)
Related indexes:
        CREATE INDEX test1_age_id_idx ON test1 USING btree (age, id) TABLESPACE pg_default

DROP INDEX public.test1_id_idx;(CREATE INDEX test1_id_idx ON test1 USING btree (id) TABLESPACE
pg_default)
Related indexes:
        CREATE INDEX test1_id_age_idx ON test1 USING btree (id, age) TABLESPACE pg_default
################################## Historical effective indexes
################################
CREATE INDEX idx_temptable_int2 ON ztt_test.temptable(int2);
CREATE INDEX idx_item_i_manufact_id ON public.item(i_manufact_id);
CREATE INDEX idx_item_i_color ON public.item(i_color);
```

----结束

⚠ 注意

如前文所述，"Current workload useless indexes", "Redundant indexes" 分别表示无用索引和冗余索引，判断依据是给定的workload. 由于workload中的SQL语句可能涉及不全（例如日志报错、没有捕获到等原因导致的），故该结论仅供提示，用户需要根据自己的业务逻辑进行排查，防止错误删除。

版权所有 © 华为云计算技术有限公司

说明

与单query索引推荐相同，本功能暂不支持段页式表、普通视图、物化视图、全局临时表、二级分区表以及密态数据库。

2.11.3.3 Slow Query Diagnosis

2.11.3.3.1 概述

慢SQL一直是数据运维中的痛点问题，如何有效诊断慢SQL根因是当前一大难题，工具结合GaussDB自身特点融合了现网DBA慢SQL诊断经验，支持慢SQL根因分析，能同时按照可能性大小输出多个根因并提供针对性的建议。

2.11.3.3.2 环境部署

- 数据库运行正常。
- 指标采集系统运行正常。

2.11.3.3.3 使用指导

慢SQL根因列表

序号	根因	根因解释	补充说明
1	LOCK_CONTENTION	锁竞争	语句执行期间是否被阻塞，导致单SQL执行较慢
2	MANY_DEAD_TUPLES	死元组数量较大	表中死元组占比超过设定阈值会降低查询效率，导致单SQL执行较慢
3	HEAVY_SCAN_OPERATOR	扫描算子代价较大	执行计划中扫描算子代价较大，导致单SQL执行较慢
4	ABNORMAL_PLAN_TIME	异常执行计划生成时间	SQL生成执行计划时间较长
5	UNUSED_AND_REDUNDANT_INDEX	无用/冗余索引	表中存在无用/冗余索引，影响插入更新语句性能
6	UPDATE_LARGE_DATA	更新大量元组	批量更新大量元组，导致单SQL语句性能较差
7	INSERT_LARGE_DATA	插入大量元组	批量插入大量元组，导致单SQL语句性能较差
8	DELETE_LARGE_DATA	删除大量元组	批量删除大量元组，导致单SQL语句性能较差
9	TOO_MANY_INDEX	表中存在太多索引	表中存在大量索引，影响插入更新语句性能
10	DISK_SPILL	磁盘溢出	由于GUC参数设置不当等原因导致SQL执行期间发生落盘现象

 版权所有 © 华为云计算技术有限公司

序号	根因	根因解释	补充说明
13	WORKLOAD_CONTENTION	数据库负载集中	数据库负载集中导致实例整体执行性能较差
14	CPU_RESOURCE_CONTENTION	系统CPU负载集中	由于外部进程等其他原因导致CPU资源紧张，实例整体SQL执行性能较差
15	IO_RESOURCE_CONTENTION	系统IO资源集中	由于外部进程等其他原因导致IO资源紧张，实例整体SQL执行性能较差
16	MEMORY_RESOURCE_CONTENTION	系统内存资源集中	由于外部进程等其他原因导致内存资源紧张，实例整体SQL执行性能较差
17	ABNORMAL_NETWORK_STATUS	异常网络状态	网络异常，导致SQL执行性能较差
18	OS_RESOURCE_CONTENTION	句柄资源紧张	系统句柄资源紧张影响整体执行性能
19	WAIT_EVENT	等待事件	SQL执行期间的等待事件信息
20	LACK_STATISTIC_INFO	统计信息缺失	没有及时更新表统计信息，可能导致执行计划不优进而影响SQL执行性能
21	MISSING_INDEXES	缺少索引	缺失索引导致单SQL执行性能较差
22	POOR_JOIN_PERFORMANCE	join代价较大	JOIN算子代价较大，影响SQL语句执行性能
23	COMPLEX_BOOLEAN_EXPRESSIONS	复杂的bool表达式	主要针对not in (item1, item2, ...)情况，如果元素数量太多可能会导致执行计划较差，从而影响SQL的性能。
24	STRING_MATCHING	字符串匹配	由于不恰当的使用函数等导致索引失效，进而影响SQL性能
25	COMPLEX_EXECUTION_PLAN	复杂的执行计划	执行计划较复杂，执行时间较长
26	CORRELATED_SUBQUERY	关联子查询	有相关子查询导致执行性能较差
27	POOR_AGGREGATION_PERFORMANCE	聚合代价较大	聚合性能较差进而影响SQL执行性能
31	DATABASE_VIEW	系统视图	当前不对系统视图相关的慢SQL进行诊断，统一返回此根因
32	NO_ROOT_CAUSE_FOUND	没有发现根因	没有发现当前慢SQL的根因

版权所有 © 华为云计算技术有限公司

假设用户已经初始化配置文件目录confpath，则可以通过下述命令实现本特性的功能：

- 仅启动慢SQL诊断功能（慢SQL诊断根因数量由算法运行结果决定，数量不固定），启动命令如下（更多用法参考对service子命令的说明）：
  ```
  gs_dbmind service start -c confpath --only-run slow_sql_diagnosis
  ```
- 用户查询慢SQL诊断历史，命令如下：
  ```
  gs_dbmind component slow_query_diagnosis show -c confpath --instance instance --query SQL --start-time timestamps0 --end-time timestamps1
  ```
- 用户交互式诊断慢SQL，命令如下：
  ```
  gs_dbmind component slow_query_diagnosis diagnosis -c confpath --database dbname --schema schema_name --query SQL
  ```
- 启用慢SQL诊断后台任务，首先将opengauss_exporter下的pg_sql_statement_history开启，具体步骤如下：
  ```
  1、停止opengauss_exporter进程；
  2、进入dbmind/components/opengauss_exporter/yamls/statements.yml中，将该指标的status设置为enable；
  3、重启opengauss_exporter进程；
  4、将slow_query_diagnosis加入到配置文件dbmind.conf下TIMED_TASK的task中，任务之间用逗号隔离，同时该任务运行间隔由配置文件dbmind.conf下TIMED_TASK的slow_query_diagnosis_interval控制，默认120秒，该参数支持用户修改；
  5、运行 gs_dbmind service reload -c confpath 命令，启动慢SQL诊断后台任务；
  ```
- 用户手动清理历史预测结果，命令如下：
  ```
  gs_dbmind component slow_query_diagnosis clean -c confpath --retention-days DAYS
  ```
- 停止已启动的服务，命令如下：
  ```
  gs_dbmind service stop -c confpath
  ```

2.11.3.3.4 获取帮助

模块命令行说明：

```
gs_dbmind component slow_query_diagnosis --help
usage:  [-h] -c DIRECTORY [--instance INSTANCE] [--database DATABASE] [--schema SCHEMA]
       [--query SLOW_QUERY] [--start-time TIMESTAMP_IN_MICROSECONDS]
       [--end-time TIMESTAMP_IN_MICROSECONDS] [--retention-days DAYS]
       {show,clean,diagnosis}

Slow Query Diagnosis: Analyse the root cause of slow query

positional arguments:
  {show,clean,diagnosis}
                        choose a functionality to perform

optional arguments:
  -h, --help            show this help message and exit
  -c DIRECTORY, --conf DIRECTORY
                        set the directory of configuration files
  --instance INSTANCE   Set the instance of slow query. Using in show.
  --database DATABASE   name of database
  --schema SCHEMA       schema of database
  --query SLOW_QUERY    set a slow query you want to retrieve
  --start-time TIMESTAMP_IN_MICROSECONDS
                        set the start time of a slow SQL diagnosis result to
                        be retrieved
  --end-time TIMESTAMP_IN_MICROSECONDS
                        set the end time of a slow SQL diagnosis result to be
                        retrieved
  --retention-days DAYS
                        clear historical diagnosis results and set the maximum
                        number of days to retain data
```

 版权所有 © 华为云计算技术有限公司

2.11.3.3.5 命令参考

表 2-30 gs_dbmind component slow_query_diagnosis 命令行说明

参数	参数说明	取值范围
-h, --help	帮助命令。	-
action	动作参数。	● show：结果展示。 ● clean：清理结果。 ● diagnosis：交互诊断。
-c，--conf	配置目录。	-
--query	慢SQL文本。	*
(可选) --start-time	显示开始时间的时间戳，单位毫秒；或日期时间格式为 %Y-%m-%d %H:%M:%S。（action为show时使用）	正整数或日期时间格式。
（可选）--end-time	显示结束时间的时间戳，单位毫秒；或日期时间格式为 %Y-%m-%d %H:%M:%S。（action为show时使用）	正整数或日期时间格式。
--retention-days	清理天数及结果。（action为clean时使用）	实数。（当其小于等于0时，会将结果全部删除）
--instance	慢SQL所属实例。（action为show时使用）	格式为IP:PORT。样例：127.0.0.1:5432
--database	慢SQL关联的数据库。	-
(可选)--schema	慢SQL关联的schema，默认为public。	-

2.11.3.3.6 注意事项说明

DBMind当前支持三种模式的慢SQL诊断能力，分别是：后台定时任务、命令行交互和API调用，下面对这三种方式的注意事项进行说明。

说明

如果没有特别指出具体使用模式，则表示该项三种模式都适用。

- 当前慢SQL诊断定时任务依赖opengauss_exporter指标采集慢SQL流水，该指标默认关闭，如果该指标采集不启用，则慢SQL诊断功能无法正常运行。启动慢SQL采集的方法是手动先将opengauss_exporter/yamls/statements.yaml中pg_sql_statement_history指标status设置成enable，再重启opengauss_exporter进程。

版权所有 © 华为云计算技术有限公司

- 慢SQL诊断定时任务间隔支持用户配置，用户需要修改配置文件dbmind.conf中TIMED_TASK的slow_query_diagnosis_interval，然后执行reload操作才能生效，命令参考**service子命令**。
- 慢SQL诊断定时任务依赖dbe_perf.statement_history视图，当其数据量较大时可能出现查询超时，此时慢SQL后台诊断任务不会生成新的诊断结果。用户可以根据数据库状况设置该指标的超时时间timeout，以免超时时间太小不能采集到指标，具体修改的方法可以参见**Prometheus Exporter组件**。
- 在慢SQL诊断定时任务中，由于慢SQL流水数据源dbe_perf.statement_history视图中的query字段可能会出现截断导致query不完整，此时如果没有提供执行计划则不能进行诊断。
- 慢SQL诊断定时任务的采集基于opengauss_exporter实现，服务运行时不能保证全量采集，可能会遗漏部分慢SQL数据。
- 使用慢SQL命令行交互诊断功能时，工具基于RPC和数据采集服务获取必要的数据，因此如果RPC和数据采集服务没启动则无法诊断。
- 在使用慢SQL交互诊断功能时，工具会对传入的数据库名和schema进行检测，如果数据库或schema不存在则会进行提示，不能正常进行诊断。
- 在使用慢SQL诊断功能时，工具会对传入的数据库名和schema进行检测，如果其中包含非法字符则会进行提示并拒绝诊断。特殊情况说明：为适配云侧使用场景（将dbe_perf.statement_history作为SQL采集数据源时，其schema可能带'$'字符，例如$user,public），在调用慢SQL诊断API接口时会对该场景进行特殊处理以支持诊断。
- 慢SQL诊断过程需要获取诊断语句的执行计划，在调用慢SQL诊断API接口时建议传递执行计划（对应query_plan参数），如果前端没有传递执行计划，那么工具会主动获取执行计划内容，此时需要确保执行诊断的用户具有获取query执行计划的权限。另外获取执行计划过程中如果SQL属于归一化SQL，那么工具会基于PBE方式获取执行计划，获取过程中如果由于语法结构不支持、SQL截断等原因导致获取失败则会诊断失败，当SQL不属于归一化SQL时如果SQL截断则也不能正常诊断。
- 在使用慢SQL诊断API接口时，如果在用户传入的db_name不存在则无法诊断，另外当工具在传入的schema_name下获取执行计划失败时会自动其他schema下尝试获取执行计划直到成功获取为止，否则诊断失败。
- 当DBMind纳管多个实例时，使用命令行交互式慢SQL诊断功能时会提示用户选择哪一个实例，然后再进行诊断动作。
- 当前慢SQL诊断只支持DML语句。
- 慢SQL诊断每次只能诊断一条语句，如果输入多条则只会对第一条进行诊断。
- 当前不对系统视图根因进行诊断，根因统一为DATABASE_VIEW。
- 慢SQL诊断过程中如果用户不传schema信息，则schema默认为public。
- 由于reprocessing_exporter在采集磁盘占用率时只支持EXT和XFS文件系统，因此如果非上述文件系统磁盘超过阈值时不会出现磁盘空间不足的根因。
- 如果需要诊断LOCK_CONTENTION或WAIT_EVENT根因，需要使用API调用的方式，并传入debug_query_id参数。

2.11.3.4 SQLdiag

SQLdiag是GaussDB中SQL语句执行时长预测工具。现有的预测技术主要基于执行计划的预测方法，但这些预测方案仅适用于分析型场景且可以获取执行计划的任务，对于OLTP或者HTAP这样的快速、简单查询是没有太多使用价值的。与上述方案不同，SQLdiag着眼于数据库的历史SQL语句，通过对历史SQL语句的执行表现进行总结归

版权所有 © 华为云计算技术有限公司

纳，将之再用于推断新的未知业务上。由于短时间内数据库SQL语句执行时长不会有太大的差距，SQLdiag可以从历史数据中检测出与已执行SQL语句相似的语句结果集，并基于SQL向量化技术通过SQL模板化和深度学习这两种方法来预测SQL语句执行时长。本工具有如下优点：

1. 不需要SQL语句的执行计划，对数据库性能不会有任何的影响。
2. 使用场景广泛，目前业内的很多算法局限性比较高，比如只适用于OLTP或者其他场景，而SQLdiag使用场景广泛。
3. 该工具容易理解，只需要简单的操作，就可以训练出自己的预测模型。

本工具的典型应用场景是对一批即将上线的SQL语句进行透视，提前识别风险。

2.11.3.4.1 概述

SQLdiag是一个SQL语句执行时间预测工具，通过模板化方法或者深度学习方法，实现在不获取SQL语句执行计划的前提下，依据语句逻辑相似度与历史执行记录，预测SQL语句的执行时间并以此发现异常SQL。

2.11.3.4.2 使用指导

前提条件

- 需要保证用户提供训练数据。
- 如果用户通过提供的工具收集训练数据，则需要启用WDR功能，涉及到的参数为track_stmt_stat_level和log_min_duration_statement，具体情况见下面小结。
- 为保证预测准确率，用户提供的历史语句日志应尽可能全面并具有代表性。

SQL 流水采集方法

本工具需要用户提前准备数据，训练数据格式如下，每个样本通过换行符分隔：

```
SQL,EXECUTION_TIME
```

预测数据格式如下：

```
SQL
```

其中SQL表示**SQL语句的文本**，EXECUTION_TIME表示**SQL语句的执行时间**，样例数据见sample_data中的train.csv和predict.csv。

用户可以按照要求格式自己收集训练数据，工具也提供了脚本自动采集（load_sql_from_rd），该脚本基于WDR报告获取SQL信息，涉及到的参数有log_min_duration_statement和track_stmt_stat_level：

- 其中log_min_duration_statement表示慢SQL阈值，如果为0则全量收集，时间单位为毫秒；
- track_stmt_stat_level表示信息捕获的级别，建议设置为track_stmt_stat_level='L0,L0'

参数开启后，可能占用一定的系统资源，但一般不大。持续的高并发场景可能产生5%以内的损耗，数据库并发较低的场景，性能损耗可忽略。下述脚本存在于sqldiag根目录（DBMINDPATH/dbmind/components/sqldiag）中。

```
使用脚本获取训练集方式：
load_sql_from_wdr.py [-h] --port PORT --start-time START_TIME
                         --finish-time FINISH_TIME [--save-path SAVE_PATH]
例如：
```

 版权所有 © 华为云计算技术有限公司

```
  python load_sql_from_wdr.py --start-time "2021-04-25 00:00:00" --finish-time "2021-04-26 14:00:00" --port 5432  --save-path ./data.csv
```

操作步骤

步骤1 提供历史日志以供模型训练

步骤2 进行训练与预测操作。

```
基于模板法的训练与预测：
  gs_dbmind component sqldiag [train, predict] -f FILE --model template --model-path template_model_path --config-file config_path
基于DNN的训练与预测：
  gs_dbmind component sqldiag [train, predict] -f FILE --model dnn --model-path dnn_model_path --config-file config_path
```

----结束

使用方法示例

使用提供的测试数据进行模板化训练：

```
gs_dbmind component sqldiag train -f DBMINDPATH/dbmind/components/sqldiag/sample_data/train.csv --model template --model-path ./template --config-file config_path
```

使用提供的测试数据进行模板化预测：

```
gs_dbmind component sqldiag predict -f DBMINDPATH/dbmind/components/sqldiag/sample_data/predict.csv --model template --model-path ./template --predicted-file ./result/t_result --config-file config_path
```

使用提供的测试数据进行模板化模型更新：

```
gs_dbmind component sqldiag finetune -f DBMINDPATH/dbmind/components/sqldiag/sample_data/train.csv --model template --model-path ./template --config-file config_path
```

使用提供的测试数据进行DNN训练：

```
gs_dbmind component sqldiag train -f DBMINDPATH/dbmind/components/sqldiag/sample_data/train.csv --model dnn --model-path ./dnn_model --config-file config_path
```

使用提供的测试数据进行DNN预测：

```
gs_dbmind component sqldiag predict -f DBMINDPATH/dbmind/components/sqldiag/sample_data/predict.csv --model dnn --model-path ./dnn_model --predicted-file --config-file config_path
```

使用提供的测试数据进行DNN模型更新：

```
gs_dbmind component sqldiag finetune -f DBMINDPATH/dbmind/components/sqldiag/sample_data/train.csv --model dnn --model-path ./dnn_model --config-file config_path
```

2.11.3.4.3 获取帮助

使用SQLdiag工具前，您可以通过以下指令获取帮助。

```
gs_dbmind component sqldiag --help
```

显示如下帮助信息：

```
usage:   [-h] [-f CSV_FILE] [--predicted-file PREDICTED_FILE]
              [--model {template,dnn}] [--query QUERY] [--threshold THRESHOLD] --model-file MODEL_FILE
              [--config-file CONFIG_FILE]
              {train,predict,finetune}

SQLdiag integrated by GaussDB.

positional arguments:
```

 版权所有 © 华为云计算技术有限公司

```
 {train,predict,finetune}
                    The training mode is to perform feature extraction and
                    model training based on historical SQL statements. The
                    prediction mode is to predict the execution time of a
                    new SQL statement through the trained model.

optional arguments:
  -h, --help            show this help message and exit
  -f CSV_FILE, --csv-file CSV_FILE
                    The data set for training or prediction. The file
                    format is CSV. If it is two columns, the format is
                    (SQL statement, duration time). If it is three
                    columns, the format is (timestamp of SQL statement
                    execution time, SQL statement, duration time).
  --predicted-file PREDICTED_FILE
                    The file path to save the predicted result.
  --model {template,dnn}
                    Choose the model model to use.
  --query QUERY         Input the queries to predict.
  --threshold THRESHOLD
                    Slow SQL threshold.
  --model-file MODEL_FILE, --model-path MODEL_FILE
                    The storage path of the model file, used to read or
                    save the model file.
  --config-file CONFIG_FILE, --config CONFIG_FILE
```

2.11.3.4.4 命令参考

表 2-31 命令行参数说明

参数	参数说明	取值范围
-f	训练或预测文件位置	-
--predicted-file	预测结果存储位置	-
--model	模型选择	template, dnn
--model-path	训练模型存储位置	-

2.11.3.4.5 常见问题处理

- 训练场景失败：请检查历史日志文件路径是否正确，且文件格式符合上文规定。
- 预测场景失败：请检查模型路径是否正确。确保待预测负载文件格式正确。

2.11.3.5 SQL Rewriter

2.11.3.5.1 概述

SQL Rewriter是一个SQL改写工具，根据预先设定的规则，将查询语句转换为更为高效或更为规范的形式，使得查询效率得以提升。

 版权所有 © 华为云计算技术有限公司

说明

- 本功能不适用包含子查询的语句。
- 本功能只支持SELECT语句和DELETE对整个表格删除的语句。
- 本功能包含12个改写规则，对不符合改写规则的语句，不会进行处理。
- 本功能会对原始查询语句和改写后语句进行屏幕输出，不建议对包含涉敏感信息的SQL语句进行改写。
- union转union all规则避免了去重，从而提升了查询性能，所得结果有可能存在冗余。
- 语句中如包含‘order by’+ 指定列名或‘group by’+ 指定列名，无法适用SelfJoin规则。
- 工具不保证查询语句等价转换，其目的是提升查询语句效率，一些推荐结果，需要结合业务实践进行优化，例如显性要求指定select的字段名。

2.11.3.5.2 使用指导

前提条件

数据库状态正常、连接正常。

使用方法示例

以tpcc数据库为例：

```
gs_dbmind component sql_rewriter 5030 tpcc queries.sql --db-host 127.0.0.1 --db-user myname --schema
public
```

queries.sql为需要改写的SQL，内容如下:

```
delete from bmsql_config;
delete from bmsql_config where cfg_name='1';
```

结果为多个改写后的查询语句，显示在屏幕（无法改写的语句，显示为空），如下：

```
+-------------------------------------------------------------------------------+-----------------------------+
| Raw SQL                                                                       | Rewritten SQL               |
+-------------------------------------------------------------------------------+-----------------------------+
| delete from bmsql_config;                                                     | TRUNCATE TABLE bmsql_config; |
| delete from bmsql_config where cfg_name='1';                                  |                             |
+-------------------------------------------------------------------------------+-----------------------------+
```

2.11.3.5.3 获取帮助

使用SQL Rewriter前，您可以通过以下指令获取帮助。

```
gs_dbmind component sql_rewriter --help
```

显示如下帮助信息：

```
usage:  [-h] [--db-host DB_HOST] [--db-user DB_USER] [--schema SCHEMA]
        [-v] db_port database file

SQL Rewriter

positional arguments:
  db_port            Port for database
  database           Name for database
  file               File containing SQL statements which need to rewrite

optional arguments:
  -h, --help         show this help message and exit
  --db-host DB_HOST  Host for database
  --db-user DB_USER  Username for database log-in
```

 版权所有 © 华为云计算技术有限公司

```
--schema SCHEMA    Schema name for the current business data
-v, --version      show program's version number and exit
```

密码通过管道输入或交互式输入，对于免密用户，任意输入都可通过检验。

2.11.3.5.4 命令参考

表 2-32 命令行参数说明

参数名称	释义
db_port	数据库端口号
database	数据库名称
file	包含多个查询语句的文件路径
db-host	（可选）数据库主机号
db-user	（可选）数据库用户名
schema	（可选，模式为public）模式

2.11.3.5.5 常见问题处理

- SQL无法改写：请查看SQL是否符合改写规则或SQL语法是否正确。

2.11.3.6 Anomaly detection

2.11.3.6.1 概述

Anomaly detection 异常检测模块主要基于统计方法来实现时序数据来发现数据中存在的可能的异常情况。该模块框架解耦，可以实现不同异常检测算法的灵活替换，而且该模块功能可以根据时序数据的不同特征来自动选择算法，支持异常值检测、阈值检测、箱型图检测、梯度检测、增长率检测、波动率检测和状态转换检测。

在异常检测的基础上，DBMind支持对关键指标异常的根因分析功能，其分析模型来源于大量现网场景总结，通过对指标发生异常时其他指标进行关联，输出可能的根因。

2.11.3.6.2 使用指导

假设指标采集系统运行正常，并且用户已经初始化了配置文件目录confpath，则可以通过下述命令实现本特性的功能：

异常检测功能

仅启动异常检测功能：

```
gs_dbmind service start --conf confpath --only-run anomaly_detection
```

对于某一指标，在全部节点上，从timestamps1到timestamps2时间段内的数据进行概览：

```
gs_dbmind component anomaly_detection --conf confpath --action overview --metric metric_name --start-time timestamps1 --end-time timestamps2
```

对于某一指标，在特定节点上，从timestamps1到timestamps2时间段内的数据进行概览：

 版权所有 © 华为云计算技术有限公司

```
gs_dbmind component anomaly_detection --conf confpath --action overview --metric metric_name --start-time timestamps1 --end-time timestamps2 --host ip_address
```

对于某一指标，在全部节点上，从timestamps1到timestamps2时间段内的数据，以特定异常检测方式进行概览：

```
gs_dbmind component anomaly_detection --conf confpath --action overview --metric metric_name --start-time timestamps1 --end-time timestamps2 --anomaly anomaly_type
```

对于某一指标，在特定节点，从timestamps1到timestamps2时间段内的数据，以特定异常检测方式进行概览：

```
gs_dbmind component anomaly_detection --conf confpath --action overview --metric metric_name --start-time timestamps1 --end-time timestamps2 --host ip_address --anomaly anomaly_type
```

对于某一指标，在特定节点，从timestamps1到timestamps2时间段内的数据，以特定异常检测方式进行可视化展示：

```
gs_dbmind component anomaly_detection --conf confpath --action plot --metric metric_name --start-time timestamps1 --end-time timestamps2 --host ip_address --anomaly anomaly_type
```

运行异常诊断后台任务：

参考**Slow Query Diagnosis**中的方法，其对应定时任务为： anomaly_detection

指标异常分析功能

指标异常根因分析接口调用：

```
curl -X 'GET' http://127.0.0.1:8080/v1/api/app/metric-diagnosis/?metric_name=os_cpu_user_usage&metric_filter={"from_instance":"127.0.0.1","from_job":"node_exporter","instance":"127.0.0.1:8181","job":"reprocessing_exporter"}&alarm_cause=["high_cpu_usage"]&start=1691482728000&end=1691482728000 -H 'accept: application/json' -H 'Content-Type: application/json' -H "Authorization: bearer xxx"
```

如果使用HTTPS协议，则查询示例为：

```
curl -X 'GET' 'https://127.0.0.1:8080/v1/api/app/metric-diagnosis/?metric_name=os_cpu_user_usage&metric_filter={"from_instance":"127.0.0.1","from_job":"node_exporter","instance":"127.0.0.1:8181","job":"reprocessing_exporter"}&alarm_cause=["high_cpu_usage"]&start=1691482728000&end=1691482728000' -H 'accept: application/json' -H 'Content-Type: application/json' -H "Authorization: bearer xxx" --cacert xx.crt --key xx.key --cert xx.crt
```

返回结果格式参考：

```
{"data":{[{'reason1': 0.0, 'reason2': 1.0}, 'conclusion', 'advice']},"success":true}
```

停止已启动的服务：

```
gs_dbmind service stop --conf confpath
```

指标异常分析支持的场景详细情况如下：

- **场景1：用户CPU使用率异常**

  异常判断条件：用户CPU使用率10分钟内持续高于80%

  可能的异常根因：

  - 业务压力增大导致

    现象：TPS、网络读写、CPU和内存基本存在一定程度的上涨

    分析：通过与相关指标进行相关性比对

    建议：根据业务量评估CPU、内存等资源是否满足业务需求，是否需要扩容
  - iowait延时高导致

    现象：磁盘的读时延和写时延变长

    建议：增加IO吞吐量，排查可以降低IO的进程
- **场景2：线程池使用率异常**

 版权所有 © 华为云计算技术有限公司

异常判断条件：默认规则线程池使用率10分钟内持续高于80%

可能的异常根因：

- 磁盘读写时延过高导致

  现象：磁盘读写时延增高，导致线程池使用率超过配置的阈值

  分析：查看产生报警的节点的数据盘每次读写时间的相关性

  建议：若发现磁盘读写时延频繁过高或者有明显劣化趋势，则继续定位是否磁盘硬件故障

- 慢SQL导致

  现象：期间有明显的statement_responsetime_percentile_p95与statement_responsetime_percentile_p80增高

  分析：查看产生报警的节点的数据盘每次读写时间的相关性

  建议：如果statement_responsetime_percentile_p95与statement_responsetime_percentile_p80持续高，CPU使用率也一直保持很高，线程池使用率反复超过阈值，没有恢复迹象，则需要联系相关人员进行进一步定位分析

- **场景3：动态内存使用率异常**

  异常判断标准：系统内存超过阈值（默认10分钟连续超过80%），再进行动态内存使用率异常分析

  可能的异常根因：

  - 会话数上涨导致

    现象：在线会话数量指标同时上涨

    分析：查看同一时间段内会话数量和内存上涨之间的关系，通过皮尔逊计算相关系数，如果绝对值超过阈值的指标会被认为是相关异常

    建议：停止变更

  - 动态内存泄露导致

    现象：动态内存持续上涨

    分析：查看内存占用较大的上下文数量，如未发生很大变化则可能是内存泄露

    建议：通过pg_terminate_session终止会话或重启DN进程

- **场景4：共享内存使用率异常**

  异常判断标准：系统内存超过阈值（默认10分钟连续超过80%），再进行共享内存使用率异常分析

  可能的异常根因：

  - 未落盘脏页数过高导致

    现象：INSERT或UPDATE操作比例突然增大

    分析：分析INSERT或UPDATE操作比例突然增大与共享内存的相关性，通过皮尔逊计算相关系数，如果绝对值超过阈值的指标会被认为是相关异常

    建议：考虑降低pagewriter_sleep参数，加速脏页落盘的速度；考虑降低dirty_page_percent_max参数，降低刷页阈值上限

  - 共享内存泄露导致

    现象：共享内存持续上涨

    分析：查看系统内存占用，确认是否有除了gaussdb进程外占用大量内存的进程

    建议：手动清理，ipcrm -m shmid（此命令操作危险，请谨慎操作。）

 版权所有 © 华为云计算技术有限公司

- **场景5：磁盘空间占用高异常**

  异常判断标准：磁盘空间占用超过阈值（默认80%）

  可能的异常根因：

  - 数据库表空间膨胀导致

    现象：数据库磁盘占用快速上升

    分析：分析INSERT或UPDATE操作比例和磁盘IO读写来确定是否脏数据增加过快

    建议：临时情况，无需处理

- **场景6：磁盘IO读取时延异常**

  异常判断标准：磁盘IO使用率超过阈值（默认80%）

  可能的异常根因：

  - 数据磁盘读写IO使用率超阈值导致

    现象：数据磁盘读写IO使用率接近100%

    分析：分析数据磁盘读写IO使用率和时延之间的关系

    建议：降低IO压力，提高磁盘的IO限制

- **场景7：扫描攻击**

  异常判断标准：SQL执行错误率和用户越权率加权得分超过阈值（默认阈值：提示0.2，告警0.6，严重0.8）

  可能的异常根因：

  - SQL执行错误率和用户越权率增高导致

    现象：SQL执行错误率和用户越权率增高

    分析：用户使用自动化工具扫描目标网络或系统的漏洞，利用这些漏洞获取未经授权的访问权限，窃取敏感数据或破坏系统目标

    建议：及时更新数据库软件和安全补丁，以修复已知漏洞，减少攻击面

- **场景8：暴力登录**

  异常判断标准：用户无效登录率和用户锁定率指标加权得分超过阈值（默认阈值：提示0.1，告警0.3，严重0.4）

  可能的异常根因：

  - 用户无效登录率和用户锁定率增高导致

    现象：用户无效登录率和用户锁定率增高

    分析：攻击者猜测用户名和密码进行暴力登录，导致账户锁定及其他拒绝服务问题

    建议：根据告警信息，及时检查登录日志、采取相应措施

- **场景9：违规操作**
- 异常判断标准：用户越权率指标超过阈值（默认阈值：提示0.2，告警0.6，严重0.8）

  可能的异常根因：

  - 用户越权率增高导致

    现象：用户越权率增高

    分析：攻击者使用用户凭证进行违规操作

    建议：对于敏感数据，限制访问权限。

 版权所有 © 华为云计算技术有限公司

说明

其中，场景7~9的约束如下：

- 用户需有Monitor admin和Audit admin权限，如果没有Audit admin权限，会导致审计指标数据全为0，导致诊断结果不可用。
- 需要开启audit_enabled、audit_log_logout、audit_user_locked和audit_user_violation参数。
- 审计总开关GUC参数audit_enabled支持动态加载。在数据库运行期间修改该配置项的值会立即生效，无需重启数据库。默认值为on，表示开启审计功能。
- 审计项audit_login_logout：默认值为7，表示开启用户登录、退出的审计功能。设置为0表示关闭用户登录、退出的审计功能。
- 审计项audit_user_locked：默认值为1，表示开启审计用户锁定和解锁功能。
- 审计项audit_user_violation：默认值为0，表示关闭用户越权操作审计功能。通过命令gs_guc reload -Z datanode -N all -I all -c "audit_user_violation=1"开启
- 如无开启审计相关参数，只能处理扫描攻击场景。

亚健康诊断功能

亚健康状态是系统介于健康状态和故障状态之间的一种状态，系统仍在运行且功能正常但处于降级模式的一种情况，它的存在会造成系统性能严重低于预期。

亚健康诊断支持的场景如下：

- **场景1：潜在慢盘监测**

  DBMind默认初始化"slow_disk_detector"检测器，在每一次触发异常检测定时任务时对潜在慢盘进行监测。

  - 现象：“慢盘”现象普遍存在于存储架构之中，由于硬盘体质或者频繁读写的原因，部分硬盘会出现性能故障，IO负载过高等情况进而导致延时变大，读写变慢的现象。
  - 检测逻辑：在最近的过去7天~30天（收集的数据小于7天不进行检测），其磁盘IO平均读写时间长期在30ms以上并呈现出上升趋势，则认为其发生潜在慢盘。

- **场景2：内存泄漏监测**

  DBMind默认初始化"mem_leak_detector"检测器，在每一次触发异常检测定时任务时对内存泄漏进行监测。

  - 现象：程序中已动态分配的堆内存由于某种原因程序未释放或无法释放，造成系统内存的浪费，导致程序运行速度减慢甚至系统崩溃等严重后果。内存泄漏缺陷具有隐蔽性、积累性的特征，比其他内存非法访问错误更难检测。
  - 检测逻辑：最近的过去7天~30天（收集的数据小于7天不进行检测），其内存占用呈现出上升趋势，则认为其发生内存泄漏。

- **场景3：锁冲突监测**

  DBMind默认初始化"deadlock_detector"检测器，在每一次触发异常检测定时任务时对锁冲突进行监测。

  - 现象：当发生所冲突时，日志中会记录锁冲突的详细信息。
  - 检测逻辑：当内核日志记录到死锁日志时，则认为其发生锁冲突，并对死锁信息进行收集。

 版权所有 © 华为云计算技术有限公司

说明

- 在输入anomaly detection的参数时，start-time设置时间至少要早于end-time设置时间30秒以上。
- 异常检测功能依赖于异常检测器，可以通过异常检测器的查询接口/v1/api/app/anomaly-detection/detectors/{name}查看当前已添加的全部异常检测器。
- 添加检测器或更改检测器参数会将检测器状态变为启用。
- 对于初始化时默认的长期指标检测器（如slow_disk_detector和mem_leak_detector），其检测器的监测时间窗口长度是固定的，不支持修改，对于其duration参数的修改是无效的。
- 对于长期指标检测器，当收集到的数据低于一个星期时，不会进行检测。当数据已经长达一小时没有更新时，不会进行检测。

警告

- 异常检测器的落盘存储依赖于元数据库，请勿在元数据库中对异常检测器进行手动修改。
- 当前版本仅支持，在主备切换、扩容和剔除节点的场景下，同一集群的检测器配置参数会被继承与保留，其他场景均不支持。

2.11.3.6.3 获取帮助

异常检测模块命令行说明：

```
gs_dbmind component anomaly_detection --help
```

显示如下帮助信息：

```
usage: [-h] --action {overview,plot} -c CONF -m METRIC -s START_TIME -e END_TIME [-H HOST] [-a
{level_shift,spike,seasonal,volatility_shift}]

Workload Anomaly detection: Anomaly detection of monitored metric.

optional arguments:
  -h, --help            show this help message and exit
  --action {overview,plot}
                        choose a functionality to perform
  -c CONF, --conf CONF  set the directory of configuration files
  -m METRIC, --metric METRIC
                        set the metric name you want to retrieve
  -s START_TIME, --start-time START_TIME
                        set the start time of for retrieving in ms, supporting UNIX-timestamp with microsecond or
datetime format
  -e END_TIME, --end-time END_TIME
                        set the end time of for retrieving in ms, supporting UNIX-timestamp with microsecond or
datetime format
  -H HOST, --host HOST  set a host of the metric, ip only or ip and port.
  -a {level_shift,spike,seasonal,volatility_shift}, --anomaly {level_shift,spike,seasonal,volatility_shift}
                        set a anomaly detector of the metric from: "level_shift", "spike", "seasonal", "volatility_shift"
```

2.11.3.6.4 命令参考

表 2-33 异常检测命令行参数说明

参数	参数说明	取值范围
-h, --help	帮助命令。	-

版权所有 © 华为云计算技术有限公司

参数	参数说明	取值范围
--action	动作参数。	overview：概览。 plot：可视化。
-c，--conf	配置文件目录。	-
-m，--metric	指定显示指标名。	任一能采集到的指标，例如os_mem_usage、os_disk_usage等。
-H, --host	指定数据来源地址信息，通过地址信息进行过滤。	-ip地址或者ip地址加端口号。
-a, --anomaly	指定异常检测方式，用于过滤。	level_shift、spike、seasonal、volatility_shift。
-s, --start-time	显示开始时间的时间戳，单位毫秒；或日期时间格式为%Y-%m-%d %H:%M:%S。	正整数或日期时间格式。
-e, --end-time	显示结束时间的时间戳，单位毫秒；或日期时间格式为%Y-%m-%d %H:%M:%S。	正整数或日期时间格式。

表 2-34 指标异常分析接口

API	请求方法	参数	参数类型	参数范围	功能描述
/v1/api/app/metric-diagnosis	GET	metric_name：指标名称，必选项	string	os_cpu_user_usage，pg_thread_pool_rate，os_mem_usage，os_disk_usage，os_disk_io_read_delay	执行指标异常检测
		metric_filter：筛选指标	string	不涉及	

版权所有 © 华为云计算技术有限公司

API	请求方法	参数	参数类型	参数范围	功能描述
		alarm_cause：选择分析方法	string	high_cpu_usage, high_thread_pool_rate, high_dynamic_mem_usage, high_shared_mem_usage, high_disk_usage, high_io_delay	
		start_time：分析指标开始时间戳	时间戳（毫秒）	不涉及	
		end_time：分析指标结束时间戳	时间戳（毫秒）	不涉及	
/v1/api/security/scenarios	GET	无	不涉及	不涉及	获取所有安全异常类型
/v1/api/security/scenarios/{name}	GET	name：场景名称	string	scanning_attack, brute_force_login_attack, user_violation_attack	获取指定安全异常场景的校准状态

2.11.3.6.5 常见问题处理

- 概览场景失败：请检查配置文件路径是否正确，且配置文件信息是否完整。检查指标名称是否准确，检查host地址是否正确，检查异常检测类型是否准确，检查起止时间内指标是否存在对应数据。
- 可视化场景失败：请检查配置文件路径是否正确，且配置文件信息是否完整。检查指标名称是否准确，检查host地址是否正确，检查异常检测类型是否准确，检查起止时间内指标是否存在对应数据。

2.11.3.7 Cluster Diagnosis

2.11.3.7.1 概述

在现网业务中需要对发生的故障原因进行快速定位定界，本功能可以通过收集数据库实例中各个组件（如CMS、DN）等的信息和即时状态（如网络连通性），来判断实例环境是否存在故障，以及故障根因。可用于实现实例级别的故障根因诊断。

版权所有 © 华为云计算技术有限公司

DBMind对cmd_exporter进行加强，本版本支持DN、CMS、CMA、ffic、OM_Monitor等日志采集，同时也支持基于节点间网络连通（如ping）状态采集。同时DBMind对现网故障场景进行了梳理，并对数据集进行枚举扩充，最终实现DN故障快速定位。

说明

由于该功能是根据日志来进行诊断的，所以诊断结果中的时间可能因为日志的延迟或者日志的延迟处理，导致诊断结果中的时间晚于故障发生的时间。

表 2-35 现支持诊断的 DN 故障根因列表

DN故障根因
未知原因/Unknown
实例被停止/DN manual stop
磁盘故障/DN disk Damage
网卡故障/DN NIC down
端口冲突/DN port conflict
CM Server仲裁重启DN/DN restarted by cms
进程僵死重启/DN phony dead
CORE/Core
只读/DN read only
主机断网或宕机/DN down/disconnection
主备DN间网络异常/DN Primary disconnected with Standby
DN IP丢失/DN ip lost

说明

当cm_ctl query的集群状态输出结果异常时，一般是发生了调用栈输出，这种情况下难以获取集群状态，无法获取集群的诊断结果，相关状态标记为"abnormal_output_from_cm_ctl_query"，诊断结果为Unknown。

当DN节点处于Offline状态时，不对其进行数据库实例故障诊断，返回状态为Normal，状态码-1。

2.11.3.7.2 使用指导

在DN实例产生异常告警时，一个完整的，用于启动实例故障分析功能的命令是：

```
gs_dbmind component cluster_diagnosis --conf {confpath} --host {ip_address} --role dn --time "2023-04-20 16:00:00" --method tree
```

输入此命令后，系统读取所设定时间前3分钟的日志记录，并对选定的DN实例使用选定方法进行分析，分析的结果示例如图所示。

版权所有 © 华为云计算技术有限公司

```
+---------------------+-----------------------+
|         Item        |         Result        |
+---------------------+-----------------------+
|         ping        |          Bad          |
|      dn_status      |         Normal        |
|    bind_ip_failed   |          Good         |
|   dn_ping_standby   |          Good         |
|     ffic_updated    |          Good         |
|    cms_phonydead    |          Good         |
| cms_restart_pending |          Good         |
|     dn_read_only    |          Good         |
|    dn_manual_stop   |          Good         |
|    dn_disk_damage   |          Good         |
|     dn_nic_down     |          Good         |
|   dn_port_conflict  |          Good         |
|     dn_writable     |          Good         |
|        Output       | DN down/disconnection |
+---------------------+-----------------------+
```

返回结果前半部分的字典给出对日志的解析结果，其中Good表示该项正常，Bad表示该项有异常；最后的Output表示输出结果，详情请参见**表2-35**。

说明

虽然单次诊断读取的是诊断时间点之前三分钟的日志和节点状态，但是由于网络延迟，模型计算时间等因素，实际时间会略短于3分钟，综合各种因素，以150秒内的诊断结果更为准确。

2.11.3.7.3 获取帮助

模块命令行说明：

```
gs_dbmind component cluster_diagnosis --help
```

显示如下帮助信息：

```
usage: [-h] --conf CONF --host HOST --role {cn,dn} [--time TIME] [--method {logical,tree}]
Cluster diagnosis.
optional arguments:
  -h, --help            show this help message and exit
  --conf CONF           set the directory of configuration files
  --host HOST           set the host of the cluster node, ip only.
  --role {cn,dn}        set the role of instance for diagnosis. roles: [cn]
                        are not supported for centralized DB.
  --time TIME           set time for diagnosis in timestamp(ms) or datetime format
  --method {logical,tree}
                        set method for the model: logical: if-else, tree: xgboost.
```


版权所有 © 华为云计算技术有限公司

2.11.3.7.4 命令参考

表 2-36 命令行参数说明

参数	参数说明	取值范围
-h, --help	帮助命令。	-
--conf	连接时序数据库需要的配置文件地址。	-
--host	分析的目标节点的IP地址。	-
--role	分析的目标节点的角色（集中式仅支持DN，使用CN会触发异常提示）。	目前仅支持{dn}。（集中式输入cn会提示异常：cn不在实例状态中）。
--time	分析的异常发生的时间点，默认值是当前时间，日期时间格式为 %Y-%m-%d %H:%M:%S。	日期时间格式或者以毫秒为单位的时间戳。
--method	分析的方法，目前提供故障定位逻辑模型以及决策树模型两种方法。	{logical, tree}。

2.11.3.7.5 常见问题处理

如果尝试启动实例诊断时发现系统不能返回预期的诊断结果，需按照以下顺序逐项排查：

- 检查连接时序数据库需要的配置文件地址是否输入正确，是否包含实例诊断需要读取的日志文件，所用账户是否有权限读写日志，以及日志文件是否损坏，能否正常读写。
- 检查目标节点的IP地址是否有误。
- 检查是否支持对所选定的实例进行诊断，集中式的实例诊断目前仅支持对DN的诊断，后期将会加入对GTM与ETCD的支持，并检查参数是否输入正确。
- 检查所键入的异常发生时间点是否符合日期时间格式，此外，实例诊断仅支持对当前及过去时刻的诊断。
- 检查所选择的诊断方法是否有输入正确，并且是否超出支持范围。
- 对于网卡故障的监测，建议在时序数据库采集数据时将cmd_exporter同时连接在多个网卡上，以便在其中某一网卡故障时，其他网卡仍然能正常将消息发送。

2.11.3.8 智能巡检

2.11.3.8.1 概述

在现网业务中需要定期对数据库实例进行巡检并输出健康报告。DBMind构建了20+个巡检项，包括硬件状态、实例状态、数据库资源、数据库性能与诊断优化等类别。对其中关键资源指标趋势进行风险判断，避免不易发现的潜在问题影响实例健康。同时，支持生成日报、周报、月报供用户查看，并基于当前巡检项情况给出健康评分。支持用户对巡检项自定义设置告警阈值，方便用户根据业务特性进行调整，避免无意义告警。

 版权所有 © 华为云计算技术有限公司

说明

- 月报与周报生成基于日报：如果未构建日报，则无法生成周报与月报；当日报数量不满足连续7天时，无法生成这7天对应的周报；当日报数量不满足连续14天以上时，无法生成对应的月报。此外，周报和月报的生成基于实例，比如出现实例IP和port的变化，会导致周报/月报无法生成；比如节点实例被删除，会导致周报/月报无法生成。采用定时方式生成周报和月报时，建议控制并发量，避免影响实时任务。配置文件中需要将实例的ip映射关系写到ip_map中，否则无法保证巡检结果的实例为管理IP。
- 周报和月报仅支持所有巡检项的巡检，不支持部分巡检项的拼接；因此周报和月报的自定义阈值需要由前端传入，规则与实时巡检等一致，没有传入的巡检项默认不进行告警（除了部分由前端控制的巡检项，见**11.3.8.2-使用指导**）
- 调用巡检接口时需要传递实例，该实例需要为集群主节点实例，否则在节点出现异常时无法返回所有节点的巡检结果。
- 智能巡检中db_size、buffer_hit_rate、db_tmp_file、db_deadlock、db_transaction等巡检项仅展示主节点的数据库结果，如果出现主备切换等主节点切换的情况，可能出现结果跳变（不同节点的结果拼接）。
- 智能巡检中db_size、buffer_hit_rate、db_transaction、db_tmp_file、db_deadlock等巡检项仅展示主节点的数据库结果；user_login_out、db_latency、thread_pool等巡检项展示所有节点的结果；xlog_accumulate巡检项展示所有节点的结果。
- log_error_check和core_dump两个巡检项防止遗漏关键信息，在实时巡检时仅支持45小时内数据；其他巡检项不能超过TSDB数据存储范围。

表 2-37 巡检项与权重设置

巡检类	巡检项	巡检字段	分数权重
系统资源 system_resource	CPU使用率	os_cpu_usage	0.06
	系统磁盘占用率	os_disk_usage	0.06
	内存使用率	os_mem_usage	0.06
	磁盘IO使用率	os_disk_ioutils	0.06
	网络状况	network_packet_loss	0.06
实例状态 instance_status	组件异常	component_error	0.05
数据库资源 database_resource	数据库目录占用率	data_directory	0.04
	数据库日志目录占用率	log_directory	0.04
	数据库大小	db_size	0
数据库性能 database_performance	Buffer命中率	buffer_hit_rate	0.03
	用户登录登出次数	user_login_out	0
	活跃Session率	active_session_rate	0.03
	日志异常检查	log_error_check	0.03
	线程池占用率	thread_pool	0.03
	数据库时延	db_latency	0.08

版权所有 © 华为云计算技术有限公司

巡检类	巡检项	巡检字段	分数权重
	数据库事务	db_transaction	0
	数据库临时文件大小	db_tmp_file	0
	数据库执行语句	db_exec_statement	0
	数据库死锁	db_deadlock	0.03
	数据库TPS性能	db_tps	0
	数据库Top Query	db_top_query	0
	长事务	long_transaction	0.05
	xlog堆积	xlog_accumulate	0.05
	oldestXmin长时间未推进	xmin_stuck	0.05
诊断优化 diagnosis_optimization	Core dump	core_dump	0.04
	动态内存	dynamic_memory	0.04
	程序内存	process_memory	0.04
	其他内存	other_memory	0.03
	GUC参数	guc_params	0.04

各巡检项实现细节与告警设置

- 系统资源
  - CPU使用率：获取TSDB中的“os_cpu_user_usage”和“os_cpu_iowait_usage”两个指标。
    - 对于os_cpu_user_usage，观测是否持续上升，若是则进行告警；同时，观测数值是否大于0.7，若超出阈值，则进行告警；最后，进行趋势预测，判断未来24小时数值是否大于0.7，若超出阈值，则进行告警。
    - 对于os_cpu_iowait_usage，当发现持续上升时会进行告警，当数值大于0.3时，进行告警
  - 系统磁盘占用率：获取TSDB中的“os_disk_usage”指标，观测其中系统盘（mount_point='/'）的磁盘使用率，当数值大于0.8时，进行告警。
  - 内存使用率：获取TSDB中的“os_mem_usage”指标，当发现持续上升时会进行告警；当数值大于0.7时，进行告警；最后，进行趋势预测，如果判断未来24小时数值大于0.8，进行告警。
  - 磁盘IO使用率：获取TSDB中的“os_disk_ioutils”指标，当数值大于0.8时，进行告警。
  - 网络状况：获取TSDB中的“gaussdb_network_packet_loss”指标，得出数据库节点间的网络连接状况，当丢包率大于0.05时，进行告警。

版权所有 © 华为云计算技术有限公司

- 实例状态
  - 组件异常：根据instance从元数据库中获取节点的状态信息，得出时间范围内当前集群中CN和DN的状态，当出现异常状态（即值不为-1）时，进行告警。
- 数据库资源
  - 数据库目录占用率：获取TSDB中的“pg_node_info_uptime”指标，解析出各个数据库实例的数据目录，同时获取“gaussdb_progress_cpu_usage”指标，匹配判断是否存在change_root，存在则组合成正确的目录路径，然后获取“os_disk_usage”指标，匹配判断对应数据目录的占用率，当发现持续上升时，进行告警；当数值大于0.8时，进行告警；最后，进行趋势预测，如果判断未来24小时数值大于0.8，进行告警；判断文件系统是否有误，不匹配则进行告警。
  - 数据库日志目录占用率：获取TSDB中的“pg_node_info_uptime”指标，然后解析出各个数据库实例的日志目录，同时获取“gaussdb_progress_cpu_usage”指标，匹配判断是否存在change_root，存在则组合成正确的目录路径，，然后获取“os_disk_usage”指标，匹配判断对应日志目录的占用率，当发现持续上升时会进行告警，当数值大于0.8时，同样进行告警，再进行趋势预测，如果判断未来24小时数值大于0.8，也会进行告警，最后还会判断文件系统是否有误，不匹配则进行告警
  - 数据库大小：获取TSDB中的“pg_database_size_bytes”指标，获取所有数据库的大小并进行记录，用于前端进行趋势展示。暂无告警。
- 数据库性能
  - Buffer命中率：获取TSDB中的“pg_db_blks_access”指标，当数值小于0.9时，进行告警。
  - 用户登录登出次数：获取TSDB中的“gaussdb_user_login”和“gaussdb_user_logout”两个指标，获取1分钟内平均每秒用户登录/登出次数，用于前端进行趋势展示。暂无告警。
  - 活跃Session率：获取TSDB中的活跃会话数gaussdb_active_connection和总会话数gaussdb_total_connection指标，活跃session率即为gaussdb_active_connection / gaussdb_total_connection，当其比值小于0.8时，进行告警。
  - 日志异常检查：获取TSDB中“gaussdb_log_*”的指标，统计每种日志错误出现的次数并进行相加，如果出现日志异常，则将日志异常的出现次数返回前端，进行告警。
  - 线程池占用率：获取TSDB中的“pg_thread_pool_rate”指标，当发现持续上升时会进行告警，当数值小于0.9时，进行告警。
  - 数据库时延：获取TSDB中的“statement_responsetime_percentile_p95”和“statement_responsetime_percentile_p80”两个指标，当发现持续上升时会进行告警。
  - 数据库事务：获取TSDB中的“pg_db_xact_commit”和“pg_db_xact_rollback”两个指标，获取业务中提交和回滚的次数，用于前端进行趋势展示。暂无告警。
  - 数据库临时文件大小：获取TSDB中的“pg_db_temp_files”指标，获取数据库临时文件的大小，用于前端进行趋势展示。暂无告警。
  - 数据库执行语句：获取TSDB中的“pg_sql_count_select”、“pg_sql_count_update”、“pg_sql_count_insert”和“pg_sql_count_delete”四个指标，获取数据库中select、update、insert、delete的执行次数，用于前端进行趋势展示。暂无告警。

 版权所有 © 华为云计算技术有限公司

  - 数据库死锁：获取TSDB中的“pg_db_deadlocks”指标，获取数据库中出现的死锁情况，当死锁数不为0时，表示数据库出现死锁。数值表示死锁数量。暂无告警。
  - 数据库TPS性能：获取TSDB中的“gaussdb_qps_by_instance”和“qps”两个指标，获取数据库的QPS和TPS信息，用于前端进行趋势展示。暂无告警。
  - 数据库Top Query：通过执行SQL语句从dbe_perf.statement查询出现调用次数最多的TopK个查询，用于前端展示。当前K为10。暂无告警。
  - 长事务：通过执行SQL语句从pg_catalog.pg_stat_activity查询超过12小时未结束的活跃事务，当返回结果列表的长度大于0时，表示存在长事务，进行告警。
  - xlog堆积：获取TSDB中的“gaussdb_xlog_count”指标，获取xlog目录下的文件数量，当数量大于3000时，进行告警。
  - oldestXmin长时间未推进：获取TSDB中的“oldestxmin_increase”指标，当oldestxmin超过12小时未出现变化，则表示长时间为推进，此时巡检返回数据中数值为0，进行告警。
- 诊断优化
  - Core dump：获取TSDB中的“gaussdb_log_ffic”指标，当返回值中count大于0时，表示出现core dump，进行告警。
  - 动态内存：获取TSDB中的“pg_total_memory_detail_mbytes”指标，计算其中“dynamic_used_memory”相对于“max_dynamic_memory”的比例，当数值大于0.8时，进行告警；计算其中“dynamic_used_shrctx”相对于“max_dynamic_memory”的比例，当数值大于0.8时，同样进行告警。
  - 程序内存：获取TSDB中的“pg_total_memory_detail_mbytes”指标，计算其中“process_used_memory”相对于“max_process_memory”的比例，当数值大于0.8时，进行告警。
  - 其他内存：获取TSDB中的“pg_total_memory_detail_mbytes”指标，获取其中“other_used_memory”的值，当发现持续上升时会进行告警，当超过20G时，进行告警。
  - GUC参数：获取TSDB中的“pg_total_memory_detail_mbytes”和“pg_settings_setting”指标，获取其中“max_process_memory”、“shared_buffers”和“work_mem”的值，将当前的值与DBMind根据workload推算出的最优参数进行对比，如果当前参数与最优参数相差大，结果中的warning字段为true，进行告警。

版权所有 © 华为云计算技术有限公司

说明

健康评分根据各巡检项的重要性权重以及异常程度来给出评分，最后汇总成当前实例的健康评分，具体计算公式如下：

$$score=\sum_{i=1}^{n} w_i * S_i$$

上述：

$$\sum_{i=1}^{n} w_i = 100$$

S_i = 0 or 1（其中0代表巡检项异常，1代表正常）。

此外，结论中会提供TOP3列表，TOP3指的是异常情况最严重的三个巡检项，通过统计各个巡检项的异常数量，再结合巡检项的权重进行排序，取前3个作为TOP3列表，如果没有异常，则为空。

支持的巡检自定义阈值

当前巡检项的告警类型与默认设置如下表所示。

- √和数字表示该巡检项已设置默认告警阈值
- -表示该巡检项支持设置对应告警类型
- ×表示该巡检项不支持设置对应告警类型

巡检项	巡检项	数值范围	异常类型				判断标准
			超过阈值	预测超过阈值	持续上升	文件类型有误	
CPU使用率	os_cpu_usage	[0, 1]	>0.7	>0.7	√	×	任意异常出现则告警
磁盘IO使用率	os_disk_ioutils	[0, 1]	>0.8	-	-	×	任意异常出现则告警
系统磁盘占用率	os_disk_usage	[0, 1]	>0.8	-	-	×	任意异常出现则告警
内存使用率	os_mem_usage	[0, 1]	>0.7	>0.8	√	×	任意异常出现则告警
网络状况	network_packet_loss	[0, 1]	>0.05	-	×	×	任意异常出现则告警
组件异常	component_error	×	×	×	×	×	任意异常出现则告警
数据库目录占用率	data_directory	[0, 1]	>0.8	>0.8	-	√	任意异常出现则告警

 版权所有 © 华为云计算技术有限公司

数据库日志目录占用率	log_directory	[0, 1]	>0.8	>0.8	-	√	任意异常出现则告警
数据库大小	db_size	[0, ∞)	-	-	-	×	暂无告警
Buffer命中率	buffer_hit_rate	[0, 1]	<0.9	-	×	×	任意异常出现则告警
用户登录登出次数	user_login_out	[0, ∞)	-	-	-	×	暂无告警
活跃Session率	active_session_rate	[0, 1]	<0.8	-	×	×	任意异常出现则告警
日志异常检查	log_error_check	×	×	×	×	×	存在则告警
线程池占用率	thread_pool	[0, 1]	>0.9	-	√	×	任意异常出现则告警
数据库时延	db_latency	[0, ∞)	-	-	√	×	任意异常出现则告警
数据库事务	db_transaction	[0, ∞)	-	-	-	×	暂无告警
数据库临时文件大小	db_tmp_file	[0, ∞)	-	-	-	×	暂无告警
数据库执行语句	db_exec_statement	[0, ∞)	-	-	×	×	暂无告警
数据库死锁	db_deadlock	[0, ∞)	-	-	-	×	暂无告警
数据库TPS性能	db_tps	[0, ∞)	-	-	-	×	暂无告警
数据库Top Query	db_top_query	×	×	×	×	×	暂无告警
长事务	long_transaction	×	×	×	×	×	存在则告警
oldestXmin长时间未推进	xmin_stuck	[0, ∞)	×	×	×	×	任意异常出现则告警
xlog堆积	xlog_accumulate	[0, ∞)	>3000	-	-	×	任意异常出现则告警
Core dump	core_dump	×	×	×	×	×	存在则告警
动态内存	dynamic_memory	[0, 1]	>0.8	-	-	×	任意异常出现则告警
程序内存	process_memory	[0, 1]	>0.8	-	-	×	任意异常出现则告警

版权所有 © 华为云计算技术有限公司

其他内存	other_memory	[0, ∞)	>20*1024	-	√	×	任意异常出现则告警
GUC参数	guc_params	×	×	×	×	×	warning中存在值则告警

说明

- 持续上升和预测超过阈值两类告警涉及检测算法，会占用DBMind性能，配置时减少开启的数量，避免影响服务性能以及返回时间等。
- 预测超过阈值需要设置预测的时间范围，以分钟为单位，设置范围(0, 48 * 60]。建议设置的时间长度小于数据的时间长度。

2.11.3.8.2 使用指导

通过巡检相关 API 实现功能调用

- 智能巡检接口

  示例：

  ```
  curl -X 'POST' "http://127.0.0.1:8080/v1/api/app/real-time-inspection?inspection_type=real_time_check&start_time=1689210000000&end_time=1689296400000&instance=127.0.0.1:5432" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"system_resource": ["os_mem_usage"], "instance_status": [], "database_resource": [], "database_performance": [], "diagnosis_optimization": []}' -H "Authorization: bearer xxx"
  ```

  如果使用HTTPS协议，则查询示例：

  ```
  curl -X 'POST' "https://127.0.0.1:8080/v1/api/app/real-time-inspection?inspection_type=real_time_check&start_time=1689210000000&end_time=1689296400000&instance=127.0.0.1:5432" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"system_resource": ["os_mem_usage"], "instance_status": [], "database_resource": [], "database_performance": [], "diagnosis_optimization": []}' -H "Authorization: bearer xxx" --cacert xx.crt --key xx.key --cert xx.crt
  ```

  如果使用自定义阈值，查询示例：

  ```
  curl -X 'POST' "https://127.0.0.1:8080/v1/api/app/real-time-inspection?inspection_type=real_time_check&start_time=1689210000000&end_time=1689296400000&instance=127.0.0.1:5432" -H 'accept: application/json' -H 'Content-Type: application/json' -d '{"system_resource": [{"os_mem_usage": {"increase": false, "threshold": [], "forecast": [1440, 0.0, 0.8]}}], "instance_status": [], "database_resource": [], "database_performance": [], "diagnosis_optimization": []}' -H "Authorization: bearer xxx" --cacert xx.crt --key xx.key --cert xx.crt
  ```

  返回结构示例如下：

  ```
  {
   "data": {
    "conclusion": {
     "full_score": 0.06,
     "health_score": 0.06,
     "health_status": "bad",
     "top3": []
    },
    "database_performance": {},
    "database_resource": {},
    "diagnosis_optimization": {},
          "instance_status": {},
          "system_resource": {
              "os_mem_usage": {
                  "127.0.0.1": {
                      "data": [0.31643905373281667],
      "statistic": {
       "avg": 0.311,
       "max": 0.3166,
  ```

```
      "min": 0.3057,
      "the_95th": 0.3153
     },
     "timestamps": [1694674713000],
     "warnings": {
      "increase_warning": true
     }
    }
   }
  }
 },
 "success": true
}
```


- 展示巡检任务接口示例：

```
curl -X 'GET' "http://127.0.0.1:8080/v1/api/app/real-time-inspection/list?instance=127.0.0.1:5432" -H 'accept: application/json' -H "Authorization: bearer xxx"
```

如果使用HTTPS协议，则查询示例：

```
curl -X 'GET' "https://127.0.0.1:8080/v1/api/app/real-time-inspection/list?instance=127.0.0.1:5432" -H 'accept: application/json' -H "Authorization: bearer xxx" --cacert xx.crt --key xx.key --cert xx.crt
```

返回结构如下：


```
{"data":{"header":["instance","start","end","id","state","cost_time","inspection_type"],"rows":[["127.0.0.1:5432",1689210000000,1689296400000,5,"success",0.033701,"real_time_check"]]},"success":true}
```


- 获取指定巡检任务的巡检结果接口示例：

```
curl -X 'GET' "http://127.0.0.1:8080/v1/api/summary/real-time-inspection?spec_id=5&instance=127.0.0.1:5432" -H 'accept: application/json' -H "Authorization: bearer xxx"
```

如果使用HTTPS协议，则查询示例：

```
curl -X 'GET' "https://127.0.0.1:8080/v1/api/summary/real-time-inspection?spec_id=5&instance=127.0.0.1:5432" -H 'accept: application/json' -H "Authorization: bearer xxx" --cacert xx.crt --key xx.key --cert xx.crt
```

返回结构如下，与**智能巡检接口返回结构**一致。

- 删除指定的巡检任务接口示例：

```
curl -X 'DELETE' "http://127.0.0.1:8080/v1/api/app/real-time-inspection?spec_id=5&instance=127.0.0.1:5432" -H 'accept: application/json' -H "Authorization: bearer xxx"
```

如果使用HTTPS协议，则删除示例：

```
curl -X 'DELETE' "https://127.0.0.1:8080/v1/api/app/real-time-inspection?spec_id=5&instance=127.0.0.1:5432" -H 'accept: application/json' -H "Authorization: bearer xxx" --cacert xx.crt --key xx.key --cert xx.crt
```

返回结构如下：

```
{"data":{"success":true},"success":true}
```

自定义阈值参数

- 自定义阈值传参方式，各告警类型对应的key如下表所示：

告警类型	键
持续上升	increase
超过阈值	threshold
预测超过阈值	forecast
文件类型有误	ftype

版权所有 © 华为云计算技术有限公司

- 启用默认告警配置（505.1.0之前的版本）

```
{
"system_resource": [
"os_mem_usage"
],
"instance_status": [],
"database_resource": [],
"database_performance": [],
"diagnosis_optimization": []
}
```

- 启用默认告警配置（505.1.0及之后版本）

```
{
"system_resource": [
{
"os_mem_usage": true
}
],
"instance_status": [],
"database_resource": [],
"database_performance": [],
"diagnosis_optimization": []
}
```

- 不启用告警

```
{
"system_resource": [
{
"os_mem_usage": false
}
],
"instance_status": [],
"database_resource": [],
"database_performance": [],
"diagnosis_optimization": []
}
```

- 启用自定义告警。如下所示，os_mem_usage启用持续上升告警、自定义设置阈值告警和预测阈值告警；os_disk_usage不启用持续上升告警、阈值告警，启用预测阈值告警。

```
{
"system_resource": [
{
"os_mem_usage": {
"increase": true,
"threshold": [0.0, 0.8],
"forecast": [1440, 0.0, 0.8]
},
"os_disk_usage": {
"forecast": [1440, 0.0, 0.8]
}
}
],
"instance_status": [],
"database_resource": [],
"database_performance": [],
"diagnosis_optimization": []
}
```

- 部分巡检项不支持自定义配置阈值：组件异常、日志异常检查、数据库Top Query、长事务、oldestXmin长时间未推进、Core dump、GUC参数，是否告警由云侧前端控制，忽略由前端传入的参数。
- 各个巡检项支持的阈值类型见**11.3.8.1-概述**中自定义阈值表格，前端传入不支持的告警类型会报错。
- 对于部分巡检项，存在子巡检项，列表如下：

版权所有 © 华为云计算技术有限公司

巡检项	子巡检项
os_cpu_usage	cpu_user
	cpu_iowait
user_login_out	login
	logout
db_latency	p95
	p80
db_transaction	commit
	rollback
db_exec_statement	select
	update
	insert
	delete
db_tps	tps
	qps
dynamic_memory	dynamic_used_memory
	dynamic_used_shrctx

当子巡检项的告警配置一致时，可以省略重复值，具体方式如下：

- 重复设置os_cpu_usage阈值自定义

```
{
"system_resource": [
{
"os_cpu_usage": {
"cpu_user": {
"increase": false,
"threshold": [],
"forecast": [1440, 0.0, 0.8]
},
"cpu_iowait": {
"increase": false,
"threshold": [],
"forecast": [1440, 0.0, 0.8]
}
}
}
],
"instance_status": [],
"database_resource": [],
"database_performance": [],
"diagnosis_optimization": []
}
```

- 简化设置os_cpu_usage阈值自定义

```
{
"system_resource": [
```

 版权所有 © 华为云计算技术有限公司

```
{
"os_cpu_usage": {
"increase": false,
"threshold": [],
"forecast": [1440, 0.0, 0.8]
}
}
],
"instance_status": [],
"database_resource": [],
"database_performance": [],
"diagnosis_optimization": []
}
```

巡检项结果返回值示例

- 集群

  以1主DN、2备DN为例，共3个节点：

  主DN：127.0.0.1:19996

  备DN：127.0.0.2:19996, 127.0.0.3:19996

- 返回结构类型
  - ①：以节点为key：{"127.0.0.1": xxx, "127.0.0.2": xxx, "127.0.0.3": xxx}
  - ②：以DN为key：{"127.0.0.1:19996": xxx, "127.0.0.2:19996": xxx, "127.0.0.3:19996": xxx}
  - ③：以DB为key：{"db1": xxx, "db2": xxx, "db3": xxx}
  - ④：返回list: [{xxx}]

巡检项对应的返回结构

巡检项	返回结构	备注
os_cpu_usage	①	两层结构，子巡检项：cpu_user、cpu_iowait
os_disk_ioutils	①	-
os_disk_usage	①	-
os_mem_usage	①	-
network_packet_loss	①	两层结构，展示节点到节点的网络状况
component_error	①	-
data_directory	②	-
log_directory	②	-
db_size	③	-
buffer_hit_rate	③	-
user_login_out	②	两层结构，子巡检项：login、logout

 版权所有 © 华为云计算技术有限公司

active_session_rate	②	-
log_error_check	①	-
thread_pool	②	-
db_latency	②	两层结构，子巡检项：p80、p95
db_transaction	③	两层结构，子巡检项：commit、rollback
db_tmp_file	③	-
db_exec_statement	②	两层结构，子巡检项：select、update、insert、delete
db_deadlock	③	-
db_tps	②	两层结构，子巡检项：qps、tps
db_top_query	④	-
long_transaction	④	-
xmin_stuck	②	-
xlog_accumulate	①	-
core_dump	①	-
dynamic_memory	②	两层结构，子巡检项：dynamic_used_memory、dynamic_used_shrctx
process_memory	②	-
other_memory	②	-
guc_params	②	-

说明

巡检结果会存储到DBMind元数据库中，DBMind会定期清除老数据以避免磁盘膨胀。

版权所有 © 华为云计算技术有限公司

2.11.3.8.3 命令参考

表 2-38 智能巡检接口

API	入参	参数介绍	请求方法	功能描述与预期返回结果
/v1/api/app/real-time-inspection	inspection_type start_time end_time tz instance inspection_items	巡检类型，String，必选 起始时间，String，可选 终止时间，String，可选 时区信息，String，可选 实例IP，String，必选 巡检项，dict，必选	POST	执行智能巡检功能并返回巡检结果
/v1/api/app/real-time-inspection/list	instance	实例IP，String，必选	GET	展示巡检任务的基础信息
/v1/api/summary/real-time-inspection	instance spec_id	实例IP，String，必选 巡检任务ID，String，必选	GET	获取指定巡检任务的巡检结果
/v1/api/app/real-time-inspection	instance spec_id	实例IP，String，必选 巡检任务ID，String，必选	DELETE	删除指定的巡检任务

2.11.3.8.4 常见问题处理

- 月报与周报生成基于日报：如果未构建日报，则无法生成周报与月报；当日报数量不满足连续7天时，无法生成这7天对应的周报；当日报数量不满足连续14天以上时，无法生成对应的月报。此外，周报和月报的生成基于实例，比如出现实例IP和port的变化，会导致周报/月报无法生成；比如节点实例被删除，会导致周报/月报无法生成。采用定时方式生成周报和月报时，建议控制并发量，避免影响实时任务。配置文件中需要将实例的ip映射关系写到ip_map中，否则无法保证巡检结果的实例为管理IP。
- 周报和月报仅支持所有巡检项的巡检，不支持部分巡检项的拼接；因此周报和月报的自定义阈值需要由前端传入，规则与实时巡检等一致，没有传入的巡检项默认不进行告警（除了部分由前端控制的巡检项，见**11.3.8.2-使用指导**）
- 调用巡检接口时需要传递实例，该实例需要为集群主节点实例，否则在节点出现异常时无法返回所有节点的巡检结果。
- 智能巡检中db_size、buffer_hit_rate、db_tmp_file、db_deadlock、db_transaction等巡检项仅展示主节点的数据库结果，如果出现主备切换等主节点切换的情况，可能出现结果跳变（不同节点的结果拼接）。

版权所有 © 华为云计算技术有限公司

- 智能巡检中db_size、buffer_hit_rate、db_transaction、db_tmp_file、db_deadlock等巡检项仅展示主节点的数据库结果；user_login_out、db_latency、thread_pool等巡检项展示所有节点的结果；xlog_accumulate巡检项展示所有节点的结果。
- log_error_check和core_dump两个巡检项防止遗漏关键信息，在实时巡检时仅支持45小时内数据；其他巡检项不能超过TSDB数据存储范围。

2.12 DB4AI: 数据库驱动 AI

DB4AI是指利用数据库的能力驱动AI任务，实现数据存储、技术栈的同构。通过在数据库内集成AI算法，令GaussDB具备数据库原生AI计算引擎、模型管理、AI算子、AI原生执行计划的能力，为用户提供普惠AI技术。不同于传统的AI建模流程，DB4AI“一站式”建模可以解决数据在各平台的反复流转问题，同时简化开发流程，并可通过数据库规划出最优执行路径，让开发者更专注于具体业务和模型的调优上，具备同类产品不具备的易用性与性能优势。

2.12.1 原生 DB4AI 引擎

GaussDB当前版本支持了原生DB4AI能力，通过引入原生AI算子，简化操作流程，充分利用数据库优化器、执行器的优化与执行能力，获得高性能的数据库内模型训练能力。更简化的模型训练与预测流程、更高的性能表现，让开发者在更短时间内能更专注于模型的调优与数据分析上，而避免了碎片化的技术栈与冗余的代码实现。

关键字解析

表 2-39 DB4AI 语法及关键字

名称		描述
语法	CREATE MODEL	创建模型并进行训练，同时保存模型。
	PREDICT BY	利用已有模型进行推断。
	DROP MODEL	删除模型。
关键字	TARGET	训练/推断任务的目标列名。
	FEATURES	训练/推断任务的数据特征列名。
	MODEL	训练任务的模型名称。

使用指导

步骤1 本版本支持的算法概述。

当前版本的DB4AI新增支持算法如下：

 版权所有 © 华为云计算技术有限公司

表 2-40 支持算法

优化算法	算法
GD	logistic_regression
	linear_regression
	svm_classification
	PCA
	multiclass
Kmeans	kmeans
xgboost	xgboost_regression_logistic
	xgboost_binary_logistic
	xgboost_regression_squarederror
	xgboost_regression_gamma

步骤2 模型训练语法说明。

- CREATE MODEL

  使用“CREATE MODEL”语句可以进行模型的创建和训练。模型训练SQL语句，选用公开数据集鸢尾花数据集iris。

- 以multiclass为例，训练一个模型。从tb_iris训练集中指定sepal_length, sepal_width,petal_length,petal_widt为特征列，使用multiclass算法，创建并保存模型iris_classification_model。

  ```
  gaussdb=# CREATE MODEL iris_classification_model USING xgboost_regression_logistic FEATURES sepal_length, sepal_width,petal_length,petal_width TARGET target_type < 2 FROM tb_iris_1 WITH nthread=4, max_depth=8;
  MODEL CREATED. PROCESSED 1
  ```

  上述命令中：

  - “CREATE MODEL”语句用于模型的训练和保存。
  - USING关键字指定算法名称。
  - FEATURES用于指定训练模型的特征，需根据训练数据表的列名添加。
  - TARGET指定模型的训练目标，它可以是训练所需数据表的列名，也可以是一个表达式，例如: price > 10000。
  - WITH用于指定训练模型时的超参数。当超参未被用户进行设置的时候，框架会使用默认数值。

    针对不同的算子，框架支持不同的超参组合：

 版权所有 © 华为云计算技术有限公司

表 2-41 算子支持的超参

算子	超参
GD (logistic_regression、linear_regression、svm_classification)	optimizer(char*); verbose(bool); max_iterations(int); max_seconds(double); batch_size(int); learning_rate(double); decay(double); tolerance(double) 其中，SVM限定超参lambda(double)
Kmeans	max_iterations(int); num_centroids(int); tolerance(double); batch_size(int); num_features(int); distance_function(char*); seeding_function(char*); verbose(int);seed(int)
GD(pca)	batch_size(int);max_iterations(int);max_seconds(int);tolerance(float8);verbose(bool);number_components(int);seed(int)
GD(multiclass)	classifier(char*) 注意：multiclass的其他超参种类取决于选择的分类器中类
xgboost_regression_logistic、xgboost_binary_logistic、xgboost_regression_squarederror、xgboost_regression_gamma	batch_size(int);booster(char*);tree_method(char*);eval_metric(char*);seed(int);nthread(int);max_depth(int);gamma(float8);eta(float8);min_child_weight(int);verbosity(int)

当前各个超参数设置的默认值和取值范围如下：

表 2-42 超参的默认值以及取值范围

算子	超参(默认值)	取值范围	超参描述
GD: logistic_regression、linear_regression、svm_classification、pca	optimizer = gd（梯度下降法）	gd/ngd（自然梯度下降）	优化器
	verbose = false	T/F	日志显示
	max_iterations = 100	(0, 10000]	最大迭代次数
	max_seconds = 0 (不对运行时长设限制)	[0,INT_MAX_VALUE]	运行时长
	batch_size = 1000	(0, 1048575]	一次训练所选取的样本数

版权所有 © 华为云计算技术有限公司

算子	超参(默认值)	取值范围	超参描述
	learning_rate = 0.8	(0, DOUBLE_MAX_VALUE]	学习率
	decay = 0.95	(0, DOUBLE_MAX_VALUE]	权值衰减率
	tolerance = 0.0005	(0, DOUBLE_MAX_VALUE]	公差
	seed = 0（对seed取随机值）	[0, INT_MAX_VALUE]	种子
	just for linear、SVM：kernel = "linear"	linear/ gaussian/ polynomial	核函数
	just for linear、SVM：components = MAX(2*features, 128)	[0, INT_MAX_VALUE]	高维空间维数
	just for linear、SVM：gamma = 0.5	(0, DOUBLE_MAX_VALUE]	gaussian核函数参数
	just for linear、SVM：degree = 2	[2, 9]	polynomial核函数参数
	just for linear、SVM：coef0 = 1.0	[0, DOUBLE_MAX_VALUE]	polynomial核函数的参数
	just for SVM：lambda = 0.01	(0, DOUBLE_MAX_VALUE)	正则化参数
	just for pca：number_components	（0，INT_MAX_VALUE]	降维的目标维度
GD: multiclass	classifier="svm_classification"	svm_classification \logistic_regression	多分类任务的分类器
Kmeans	max_iterations = 10	[1, 10000]	最大迭代次数
	num_centroids = 10	[1, 1000000]	簇的数目
	tolerance = 0.00001	(0,1]	中心点误差

版权所有 © 华为云计算技术有限公司

算子	超参(默认值)	取值范围	超参描述
	batch_size = 10	[1, 1048575]	一次训练所选取的样本数
	num_features = 2	[1, INT_MAX_VALUE]	输入样本特征数
	distance_function = "L2_Squared"	L1\L2\L2_Squared\Linf	正则化方法
	seeding_function = "Random++"	"Random++"\"KMeans||"	初始化种子点方法
	verbose = 0U	{ 0, 1, 2 }	冗长模式
	seed = 0U	[0, INT_MAX_VALUE]	种子
xgboost: xgboost_regression_logistic、 xgboost_binary_logistic、 xgboost_regression_gamma、 xgboost_regression_squarederror	n_iter=10	(0, 10000]	迭代次数
	batch_size=10000	(0, 1048575]	一次训练所选取的样本数
	booster="gbtree"	gbtree\gblinear\dart	booster种类
	tree_method="auto"	auto\exact\approx\hist\gpu_hist 注意：gpu_hist参数需要相应的库是GPU版本，否则DB4AI平台不支持该值。	树构建算法
	eval_metric="rmse"	rmse\rmsle\map\mae\auc\aucpr	验证数据的评估指标
	seed=0	[0, INT_MAX_VALUE]	种子
	nthread=1	[0, 100]	并发量
	max_depth=5	(0, MAX_MEMORY_LIMIT]	树的最大深度，该超参仅对树型booster生效。

版权所有 © 华为云计算技术有限公司

算子	超参(默认值)	取值范围	超参描述
	gamma=0.0	[0, DOUBLE_MAX_VALUE]	叶节点上进行进一步分区所需的最小损失减少
	eta=0.3	[0, 1]	更新中使用的步长收缩，以防止过拟合
	min_child_weight=1	[0, INT_MAX_VALUE]	孩子节点中所需的实例权重的最小总和
	verbosity=1	0 (silent)\1 (warning)\2 (info)\3 (debug)	打印信息的详细程度
MAX_MEMORY_LIMIT = 最大内存加载的元组数量			
GS_MAX_COLS = 数据库单表最大属性数量			

- 模型保存成功，则返回创建成功信息：
  ```
  MODEL CREATED. PROCESSED x
  ```

步骤3 查看模型信息。

当训练完成后模型会被存储到系统表gs_model_warehouse中。系统表gs_model_warehouse可以查看到关于模型本身和训练过程的相关信息。

关于模型的详细描述信息以二进制的形式存储在系统表中，用户可用过使用函数gs_explain_model完成对模型的查看，语句如下：

```
gaussdb=# select gs_explain_model('iris_classification_model');
 DB4AI MODEL
----------------------------------------------------------------
 Name: iris_classification_model
 Algorithm: xgboost_regression_logistic
 Query: CREATE MODEL iris_classification_model
 USING xgboost_regression_logistic
 FEATURES sepal_length, sepal_width,petal_length,petal_width
 TARGET target_type < 2
 FROM tb_iris_1
 WITH nthread=4, max_depth=8;
 Return type: Float64
 Pre-processing time: 0.000000
 Execution time: 0.001443
 Processed tuples: 78
 Discarded tuples: 0
 n_iter: 10
 batch_size: 10000
 max_depth: 8
 min_child_weight: 1
 gamma: 0.0000000000
 eta: 0.3000000000
 nthread: 4
 verbosity: 1
 seed: 0
 booster: gbtree
 tree_method: auto
 eval_metric: rmse
```

 版权所有 © 华为云计算技术有限公司

```
rmse: 0.2648450136
model size: 4613
```

步骤4 利用已存在的模型做推断任务。

使用“SELECT”和“PREDICT BY”关键字利用已有模型完成推断任务。

查询语法：SELECT…PREDICT BY…(FEATURES…)…FROM…;

```
gaussdb=# SELECT id, PREDICT BY iris_classification (FEATURES
sepal_length,sepal_width,petal_length,petal_width) as "PREDICT" FROM tb_iris limit 3;

id  | PREDICT
-----+---------
  84 |       2
  85 |       0
  86 |       0
(3 rows)
```

针对相同的推断任务，同一个模型的结果是大致稳定的。且基于相同的超参数和训练集训练的模型也具有稳定性，同时AI模型训练存在随机成分（每个batch的数据分布、随机梯度下降），所以不同的模型间的计算表现、结果允许存在小的差别。

步骤5 查看执行计划。

使用explain语句可对“CREATE MODEL”和“PREDICT BY”的模型训练或预测过程中的执行计划进行分析。Explain关键字后可直接拼接CREATE MODEL/ PREDICT BY语句（子句），也可接可选的参数，支持的参数如下：

表 2-43 EXPLAIN 支持的参数

参数名	描述
ANALYZE	布尔型变量，追加运行时间、循环次数等描述信息
VERBOSE	布尔型变量，控制训练的运行信息是否输出到客户端
COSTS	布尔型变量
CPU	布尔型变量
DETAIL	布尔型变量，不可用。
NODES	布尔型变量，不可用
NUM_NODES	布尔型变量，不可用
BUFFERS	布尔型变量
TIMING	布尔型变量
PLAN	布尔型变量
FORMAT	可选格式类型：TEXT / XML / JSON / YAML

示例：

 版权所有 © 华为云计算技术有限公司

```
gaussdb=# Explain CREATE MODEL patient_logisitic_regression USING logistic_regression FEATURES
second_attack, treatment TARGET trait_anxiety > 50 FROM patients WITH batch_size=10, learning_rate =
0.05;
                          QUERY PLAN
---------------------------------------------------------------------
 Train Model - logistic_regression  (cost=0.00..0.00 rows=0 width=0)
   ->  Materialize  (cost=0.00..41.08 rows=1776 width=12)
         ->  Seq Scan on patients  (cost=0.00..32.20 rows=1776 width=12)
(3 rows)
```

步骤6 异常场景。

- 训练阶段。
  - 场景一：当超参数的设置超出取值范围，模型训练失败，返回ERROR，并提示错误，例如：
    ```
    gaussdb=# CREATE MODEL patient_linear_regression USING linear_regression FEATURES
    second_attack,treatment TARGET trait_anxiety  FROM patients WITH optimizer='aa';
    ERROR:  Invalid hyperparameter value for optimizer. Valid values are: gd, ngd.
    ```
  - 场景二：当模型名称已存在，模型保存失败，返回ERROR，并提示错误原因，例如：
    ```
    gaussdb=# CREATE MODEL patient_linear_regression USING linear_regression FEATURES
    second_attack,treatment TARGET trait_anxiety  FROM patients;
    ERROR:  The model name "patient_linear_regression" already exists in gs_model_warehouse.
    ```
  - 场景三：FEATURE或者TARGETS列是*，返回ERROR，并提示错误原因，例如：
    ```
    gaussdb=# CREATE MODEL patient_linear_regression USING linear_regression FEATURES *
    TARGET trait_anxiety  FROM patients;
    ERROR:  FEATURES clause cannot be *
    ---------------------------------------------------------------------------------------------------------------
    -------
    gaussdb=# CREATE MODEL patient_linear_regression USING linear_regression FEATURES
    second_attack,treatment TARGET *  FROM patients;
    ERROR:  TARGET clause cannot be *
    ```
  - 场景四：对于无监督学习方法使用TARGET关键字，或者在监督学习方法中不适用TARGET关键字，均会返回ERROR，并提示错误原因，例如：
    ```
    gaussdb=# CREATE MODEL patient_linear_regression USING linear_regression FEATURES
    second_attack,treatment FROM patients;
    ERROR:  Supervised ML algorithms require TARGET clause
    ---------------------------------------------------------------------------------------------------------------
    -------------
    CREATE MODEL patient_linear_regression USING linear_regression TARGET trait_anxiety  FROM
    patients;
    ERROR:  Supervised ML algorithms require FEATURES clause
    ```
  - 场景五：当进行分类任务时TARGET列的分类只有1种情况，会返回ERROR，并提示错误原因，例如：
    ```
    gaussdb=# CREATE MODEL ecoli_svmc USING multiclass FEATURES f1, f2, f3, f4, f5, f6, f7
    TARGET cat FROM (SELECT * FROM db4ai_ecoli WHERE cat='cp');
    ERROR:  At least two categories are needed
    ```
  - 场景六：DB4AI在训练过程中会过滤掉含有空值的数据，当参与训练的模型数据为空的时候，会返回ERROR，并提示错误原因，例如：
    ```
    gaussdb=# create model iris_classification_model using xgboost_regression_logistic features
    message_regular target error_level from error_code;
    ERROR:  Training data is empty, please check the input data.
    ```
  - 场景七：DB4AI的算法对于支持的数据类型是有限制的。当数据类型不在支持白名单中，会返回ERROR，并提示非法的oid，可通过pg_type查看OID确定非法的数据类型，例如：
    ```
    gaussdb=# CREATE MODEL ecoli_svmc USING multiclass FEATURES f1, f2, f3, f4, f5, f6, f7, cat
    TARGET cat FROM db4ai_ecoli ;
    ERROR:  Oid type 1043 not yet supported
    ```
  - 场景八：当GUC参数statement_timeout设置了时长，训练超时执行的语句将被终止：执行CREATE MODEL语句。训练集的大小、训练轮数(iteration)、

 版权所有 © 华为云计算技术有限公司

提前终止条件(tolerance、max_seconds)、并行线程数(nthread)等参数都会影响训练时长。当时长超过数据库限制，语句被终止模型训练失败。

- 模型解析。
  - 场景九：当模型名在系统表中查找不到，数据库会报ERROR，例如：
    ```
    gaussdb=# select gs_explain_model("ecoli_svmc");
    ERROR:  column "ecoli_svmc" does not exist
    ```
- 推断阶段。
  - 场景十：当模型名在系统表中查找不到，数据库会报ERROR，例如：
    ```
    gaussdb=# select id, PREDICT BY patient_logistic_regression (FEATURES
    second_attack,treatment) FROM patients;
    ERROR:  There is no model called "patient_logistic_regression".
    ```
  - 场景十一：当做推断任务FEATURES的数据维度和数据类型与训练集存在不一致，将报ERROR，并提示错误原因，例如：
    ```
    gaussdb=# select id, PREDICT BY patient_linear_regression (FEATURES second_attack) FROM
    patients;
    ERROR:  Invalid number of features for prediction, provided 1, expected 2
    CONTEXT:  referenced column: patient_linear_regression_pred
    ----------------------------------------------------------------------------------------------------------------------
    ----------------------
    gaussdb=# select id, PREDICT BY patient_linear_regression (FEATURES
    1,second_attack,treatment) FROM patients;
    ERROR:  Invalid number of features for prediction, provided 3, expected 2
    CONTEXT:  referenced column: patient_linear_regression_pre
    ```

----结束

说明

DB4AI特性需要读取数据参与计算，不适用于密态数据库等情况。

2.12.2 全流程 AI

传统的AI任务往往具有多个流程，如数据的收集过程包括数据的采集、数据清洗、数据存储等，在算法的训练过程中又包括数据的预处理、训练、模型的保存与管理等。其中，对于模型的训练过程，又包括超参数的调优过程。诸如此类机器学习模型生命周期的全过程，可大量集成于数据库内部。在距离数据存储侧最近处进行模型的训练、管理、优化等流程，在数据库端提供SQL语句式的开箱即用的AI全声明周期管理的功能，称之为全流程AI。

GaussDB实现了部分全流程AI的功能，将在本章节中详细展开。

2.12.2.1 DB4AI-Snapshots 数据版本管理

DB4AI-Snapshots是DB4AI模块用于管理数据集版本的功能。通过DB4AI-Snapshots组件，开发者可以简单、快速地进行特征筛选、类型转换等数据预处理操作，同时还可以像git一样对训练数据集进行版本控制。数据表快照创建成功后可以像视图一样进行使用，但是一经发布后，数据表快照便固化为不可变的静态数据，如需修改该数据表快照的内容，需要创建一个版本号不同的新数据表快照。

DB4AI-Snapshots 的生命周期

DB4AI-Snapshots的状态包括published、archived以及purged。其中，published可以用于标记该DB4AI-Snapshots 已经发布，可以进行使用。archived表示当前 DB4AI-Snapshots 处于“存档期”，一般不进行新模型的训练，而是利用旧数据对新的模型进行验证。purged则是该DB4AI-Snapshots 已经被删除的状态，在数据库系统中无法再检索到。

 版权所有 © 华为云计算技术有限公司

需要注意的是快照管理功能是为了给用户提供统一的训练数据，不同团队成员可以使用给定的训练数据来重新训练机器学习模型，方便用户间协同。为此**三权分立**状态(enableSeparationOfDuty=ON)等涉及不支持用户数据转写等情况将不支持Snapshot特性。

用户可以通过“CREATE SNAPSHOT”语句创建数据表快照，创建好的快照默认即为published状态。可以采用两种模式创建数据表快照，即为MSS以及CSS模式，它们可以通过GUC参数db4ai_snapshot_mode进行配置。对于MSS模式，它是采用物化算法进行实现的，存储了原始数据集的数据实体；CSS则是基于相对计算算法实现的，存储的是数据的增量信息。数据表快照的元信息存储在DB4AI的系统目录中。可以通过 db4ai.snapshot 系统表查看到。

可以通过“ARCHIVE SNAPSHOT”语句将某一个数据表快照标记为archived状态，可以通过“PUBLISH SNAPSHOT”语句将其再度标记为published状态。标记数据表快照的状态，是为了帮助数据科学家进行团队合作使用的。

当一个数据表快照已经丧失存在价值时，可以通过“PURGE SNAPSHOT”语句删除它，以便永久删除其数据并恢复存储空间。

DB4AI-Snapshots 使用指导

步骤1 创建表以及插入表数据。

数据库内存在已有的数据表，可根据该已有的数据表创建对应的数据表快照。为了后续演示，在此处新建一个名为 t1 的数据表，并向其中插入测试数据。

```
create table t1 (id int, name varchar);
insert into t1 values (1, 'zhangsan');
insert into t1 values (2, 'lisi');
insert into t1 values (3, 'wangwu');
insert into t1 values (4, 'lisa');
insert into t1 values (5, 'jack');
```

通过SQL语句，查询搭配数据表内容。

```
SELECT * FROM t1;
id |   name
----+----------
  1 | zhangsan
  2 | lisi
  3 | wangwu
  4 | lisa
  5 | jack
(5 rows)
```

步骤2 使用DB4AI-Snapshots。

- 创建DB4AI-Snapshots
  - 示例1：CREATE SNAPSHOT…AS

    示例如下，其中，默认版本分隔符为 “@”, 默认子版本分割符为 “.”，该分割符可以分别通过GUC参数db4ai_snapshot_version_delimiter以及db4ai_snapshot_version_separator进行设置。

    ```
    create snapshot s1@1.0 comment is 'first version' as select * from t1;
    schema |  name
    --------+--------
     public | s1@1.0
    (1 row)
    ```

    上述结果提示已经创建了数据表 s1的快照，版本号为 1.0。创建好后的数据表快照可以像使用一般视图一样进行查询，但不支持通过“INSERT INTO”语句进行更新。例如下面几种语句都可以查询到数据表快照s1的对应版本 1.0 的内容：

 版权所有 © 华为云计算技术有限公司

```
SELECT * FROM DB4AISHOT(s1@1.0);
SELECT * FROM DB4AISHOT(public.s1@1.0);
SELECT * FROM DB4AISHOT(public . s1 @ 1.0);
```

上述语法查询结果示例如下：

```
id |   name
----+----------
  1 | zhangsan
  2 | lisi
  3 | wangwu
  4 | lisa
  5 | jack
(5 rows)
```

可以通过下列SQL语句修改数据表t1的内容：

```
UPDATE t1 SET name = 'tom' where id = 4;
insert into t1 values (6, 'john');
insert into t1 values (7, 'tim');
```

再检索数据表t1的内容时，发现虽然数据表t1的内容已经发生变化，但是数据表快照 s1@1.0 版本的查询结果并未发生变化。由于数据表t1的数据已经发生了改变，如果将当前数据表的内容作为版本2.0，则可创建快照**s1@2.0**，创建的SQL语句如下：

```
create snapshot s1@2.0 as select * from t1;
```

通过上述例子，可以发现，数据表快照可以固化数据表的内容，避免中途对数据的改动造成机器学习模型训练时的不稳定，同时可以避免多用户同时访问、修改同一个表时造成的锁冲突。

- 示例2：CREATE SNAPSHOT…FROM

  SQL语句可以对一个已经创建好的数据表快照进行继承，利用在此基础上进行的数据修改产生一个新的数据表快照。例如：

  ```
  create snapshot s1@3.0 from @1.0 comment is 'inherits from @1.0' using (INSERT VALUES(6, 'john'), (7, 'tim'); DELETE WHERE id = 1);
  schema |  name
  --------+--------
   public | s1@3.0
  (1 row)
  ```

  其中，“@”为数据表快照的版本分隔符，from子句后加上已存在的数据表快照，用法为“@”+版本号，USING关键字后加入可选的几个操作关键字（INSERT …/UPDATE …/DELETE …/ALTER … ）,其中 “INSERT INTO”以及“DELETE FROM”语句中的“INTO”、“FROM”等与数据表快照名字相关联的子句可以省略，具体可以参考《开发者指南》中“SQL参考 > 函数和操作符 > AI特性函数”章节。

  示例中，基于前述s1@1.0快照，插入2条数据，删除1条新的数据，新生成的快照s1@3.0，检索该s1@3.0：

  ```
  SELECT * FROM DB4AISHOT(s1@3.0);
  id |   name
  ----+----------
    2 | lisi
    3 | wangwu
    4 | lisa
    5 | jack
    6 | john
    7 | tim
  (7 rows)
  ```

- 删除数据表快照SNAPSHOT

  ```
  purge snapshot s1@3.0;
  schema |  name
  --------+--------
   public | s1@3.0
  (1 row)
  ```

 版权所有 © 华为云计算技术有限公司

此时，已经无法再从s1@3.0 中检索到数据了，同时该数据表快照在db4ai.snapshot视图中的记录也会被清除。删除该版本的数据表快照不会影响其他版本的数据表快照。

- 从数据表快照中采样

  示例：从snapshot s1中抽取数据，使用0.5抽样率。

```
sample snapshot s1@2.0 stratify by name as nick at ratio .5;
schema |    name
--------+------------
 public | s1nick@2.0
(1 row)
```

  可以利用该功能创建训练集与测试集，例如：

```
SAMPLE SNAPSHOT s1@2.0  STRATIFY BY name AS _test AT RATIO .2, AS _train AT RATIO .8
COMMENT IS 'training';
schema |      name
--------+----------------
 public | s1_test@2.0
 public | s1_train@2.0
(2 rows)
```

- 发布数据表快照

  采用下述SQL语句将数据表快照 s1@2.0 标记为published 状态：

```
publish snapshot s1@2.0;
schema |  name
--------+--------
 public | s1@2.0
(1 row)
```

- 存档数据表快照

  采用下述语句可以将数据表快照标记为 archived 状态：

```
archive snapshot s1@2.0;
schema |  name
--------+--------
 public | s1@2.0
(1 row)
```

  可以通过db4ai-snapshots提供的视图查看当前数据表快照的状态以及其他信息：

```
select * from db4ai.snapshot;
id | parent_id | matrix_id | root_id | schema |    name    | owner |                 commands                 |
comment | published | archived |          created           | row_count
----+-----------+-----------+---------+--------+------------+--------+------------------------------------------
+---------+-----------+----------+-----------------------------+-----------
  1 |           |           |       1 | public | s1@2.0     | omm | {"select *","from t1 where id > 3",NULL} |
| t         | f        | 2021-04-17 09:24:11.139868 |         2
  2 |         1 |           |       1 | public | s1nick@2.0 | omm | {"SAMPLE nick .5 {name}"}                |         |
f         | f        | 2021-04-17 10:02:31.73923  |         0
```

步骤3 异常场景

- 数据表或db4ai-snapshots不存在时。

```
purge snapshot s1nick@2.0;
publish snapshot s1nick@2.0;
---------
ERROR:  snapshot public."s1nick@2.0" does not exist
CONTEXT:  PL/SQL function db4ai.publish_snapshot(name,name) line 11 at assignment

archive snapshot s1nick@2.0;
----------
ERROR:  snapshot public."s1nick@2.0" does not exist
CONTEXT:  PL/SQL function db4ai.archive_snapshot(name,name) line 11 at assignment
```

- 删除snapshot时，有依赖该快照的其他snapshot，需先确保删除对本快照所依赖的其他快照。

```
purge snapshot s1@1.0;
ERROR:  cannot purge root snapshot 'public."s1@1.0"' having dependent snapshots
```

 版权所有 © 华为云计算技术有限公司

```
HINT:  purge all dependent snapshots first
CONTEXT:  referenced column: purge_snapshot_internal
SQL statement "SELECT db4ai.purge_snapshot_internal(i_schema, i_name)"
PL/SQL function db4ai.purge_snapshot(name,name) line 71 at PERFORM
```

步骤4 相关GUC参数

- db4ai_snapshot_mode：

  Snapshot有2种模式：MSS（物化模式，存储数据实体）和CSS（计算模式，存储增量信息）。Snapshot可在MSS和CSS之间切换快照模式，默认是MSS模式。

- db4ai_snapshot_version_delimiter：

  该参数为数据表快照版本分隔符。“@”为数据表快照的默认版本分隔符。

- db4ai_snapshot_version_separator

  该参数为数据表快照子版本分隔符。“.”为数据表快照的默认版本分隔符。

步骤5 DB4AI Schema下的数据表快照详情db4ai.snapshot。

```
gaussdb=# \d db4ai.snapshot
                    Table "db4ai.snapshot"
  Column   |            Type             |         Modifiers
-----------+-----------------------------+---------------------------
 id        | bigint                      |
 parent_id | bigint                      |
 matrix_id | bigint                      |
 root_id   | bigint                      |
 schema    | name                        | not null
 name      | name                        | not null
 owner     | name                        | not null
 commands  | text[]                      | not null
 comment   | text                        |
 published | boolean                     | not null default false
 archived  | boolean                     | not null default false
 created   | timestamp without time zone | default pg_systimestamp()
 row_count | bigint                      | not null
Indexes:
    "snapshot_pkey" PRIMARY KEY, btree (schema, name) TABLESPACE pg_default
    "snapshot_id_key" UNIQUE CONSTRAINT, btree (id) TABLESPACE pg_default
```

----结束

说明

命名空间DB4AI是本功能的私有域，不支持在DB4AI的命令空间下创建函数索引（functional index）。

DB4AI特性不支持密态数据库场景。

DB4AI.SNAPSHOT_SEQUENCE由于功能需求，普通用户具有写权限。建议不要修改时序，会造成本特性出现数据冲突。

2.13 ABO 优化器

2.13.1 智能基数估计

2.13.1.1 概述

智能基数估计使用库内贝叶斯网络模型对多列数据样本联和分布进行建模，从而能够对多列等值查询提供更加准确的基数估计。更加准确的基数估计能够显著提高优化器对于计划和算子的选择的准确性，从而提高数据库整体吞吐量。

 版权所有 © 华为云计算技术有限公司

2.13.1.2 前置条件

数据库运行正常，GUC参数enable_ai_stats设置为on，multi_stats_type设置为'BAYESNET'或者'ALL'。

2.13.1.3 使用指导

步骤1 设置采样方式为按照采样率采样，即设置GUC参数default_statistics_target为[-100, -1]之间的整数，表示采样百分比。

步骤2 使用ANALYZE(([column_name,])) 进行数据统计和模型创建。

步骤3 输入查询，如果查询涉及到的等值查询列上有统计模型创建，那么会自动使用统计模型进行选择率估计。

步骤4 不再需要智能统计模型的时候，使用ALTER TABLE [table_name] DELETE STATISTICS (([column_name,]))进行统计信息以及模型删除。

----结束

其他使用的方法详见《开发者指南》中“SQL参考 > SQL语法 > ALTER TABLE和ANALYZE | ANALYSE”章节。

2.13.1.4 最佳实践

生成如下数据表：

```
benchmark=# \d part;
id             | integer                  |
p_brand        | character varying(256)   |
p_type         | character varying(256)   |
p_container    | character varying(256)   |
p_mfgr         | character varying(256)   |
```

插入10,000,000行数据：

```
benchmark=# select count(1) from part1;
10000000
```

在数据表上创建四种不同的多列索引：

```
benchmark=# select * from pg_indexes where tablename='part1';
public     | part1     | brand_type_container  |             | CREATE INDEX brand_type_container ON part1
USING btree (p_brand, p_type, p_container) TABLESPACE pg_default
public     | part1     | brand_type_mfgr       |             | CREATE INDEX brand_type_mfgr ON part1 USING
btree (p_brand, p_type, p_mfgr) TABLESPACE pg_default
public     | part1     | brand_container_mfgr  |             | CREATE INDEX brand_container_mfgr ON part1
USING btree (p_brand, p_container, p_mfgr) TABLESPACE pg_default
public     | part1     | type_container_mfgr   |             | CREATE INDEX type_container_mfgr ON part1 USING
btree (p_type, p_container, p_mfgr) TABLESPACE pg_default
```

针对数据表生成一批包含多列等值条件的查询，如下：

```
explain analyze select * from part1 where p_container='LG CASE' AND p_brand='Brand#34' AND
p_mfgr='Manufacturer#2' AND p_type='SMALL BRUSHED COPPER';
```

分别测试不创建多列统计信息和创建ABO统计信息场景下的执行计划：

```
benchmark=# explain analyze select * from part1 where p_container='LG CASE' AND p_brand='Brand#34'
AND p_mfgr='Manufacturer#2' AND p_type='SMALL BRUSHED COPPER';
Bitmap Heap Scan on part1 (cost=5.30..336.06 rows=17 width=56) (actual time=0.953..7.061 rows=103
loops=1)
  Recheck Cond: (((p_brand)::text = 'Brand#34'::text) AND ((p_type)::text = 'SMALL BRUSHED COPPER'::text)
AND ((p_container)::text = 'LG CASE'::text))
```

 版权所有 © 华为云计算技术有限公司

```
  Filter: ((p_mfgr)::text = 'Manufacturer#2'::text)
  Rows Removed by Filter: 773
  Heap Blocks: exact=871
  ->  Bitmap Index Scan on brand_type_container  (cost=0.00..5.30 rows=84 width=0) (actual
time=0.704..0.704 rows=876 loops=1)
      Index Cond: (((p_brand)::text = 'Brand#34'::text) AND ((p_type)::text = 'SMALL BRUSHED COPPER'::text)
AND ((p_container)::text = 'LG CASE'::text))
Total runtime: 7.213 ms
benchmark=# explain analyze select * from part1 where p_container='LG CASE' AND p_brand='Brand#34'
AND p_mfgr='Manufacturer#2' AND p_type='SMALL BRUSHED COPPER';
Bitmap Heap Scan on part1  (cost=10.59..723.97 rows=210 width=56) (actual time=0.112..0.434 rows=103
loops=1)
  Recheck Cond: (((p_type)::text = 'SMALL BRUSHED COPPER'::text) AND ((p_container)::text = 'LG
CASE'::text) AND ((p_mfgr)::text = 'Manufacturer#2'::text))
  Filter: ((p_brand)::text = 'Brand#34'::text)
  Rows Removed by Filter: 64
  Heap Blocks: exact=167
  ->  Bitmap Index Scan on type_container_mfgr  (cost=0.00..10.54 rows=183 width=0) (actual
time=0.081..0.081 rows=167 loops=1)
      Index Cond: (((p_type)::text = 'SMALL BRUSHED COPPER'::text) AND ((p_container)::text = 'LG
CASE'::text) AND ((p_mfgr)::text = 'Manufacturer#2'::text))
Total runtime: 0.533 ms
```

通过以上操作可以看出，在这个场景下，ABO基数估计加速了查询10+倍。

2.13.1.5 常见问题处理

如果遇到异常场景导致模型无法创建，ABO优化器会只创建传统统计信息，请根据对应的告警信息进行相应处理。

2.13.2 自适应计划选择

2.13.2.1 概述

自适应计划选择作用于使用通用缓存计划进行计划执行的场景。通过使用范围线性扩张进行缓存计划探索，通过范围覆盖匹配进行计划选择。自适应计划选择弥补了传统单一缓存计划无法根据查询条件参数进行变化带来的性能问题，并且避免了频繁调用查询优化。

PBE方式执行的情况下，GaussDB中的查询计划可能会有三个来源：

1. 单个缓存计划（generic plan，简称gplan）：优势是避免了解析优化代价，缺点是无法实现感知条件参数，某些场景下计划效率可能降低， 比如：当数据分布不均匀并且查询条件发生变化时计划效率就会降低。
2. 通过解析器优化器硬解析生成的计划（custom plan，简称cplan）：优势是能够根据统计信息和代价预测产生较优计划，缺点是计划生成时间消耗较多cpu资源。
3. 多个缓存计划中选择的计划（adaptive plan，简称aplan）：优势是集合了gplan和cplan的优点于一身，能够高效选择较优执行计划，缺点是部分类型查询（比如子查询，复杂查询等）支持效果不佳。

针对上述三种计划来源的优缺点，GaussDB ABO特性默认开启了自适应选择机制，通过对实际执行时间反馈的统计分析，选择当前最优的计划产生策略，使得整体性能最优。

2.13.2.2 前置条件

数据库运行正常，GUC参数"enable_cachedplan_mgr"为on，启动自适应计划选择功能。

 版权所有 © 华为云计算技术有限公司

2.13.2.3 使用指导

现网环境下，对存在缓存计划问题的query使用hint开启计划自适应管理能力：

```
select /*+ choose_adaptive_gplan */ * from tab where c1 = xxx;
```

JDBC客户端默认会将以上带hint的SQL转换为PBE模型，并建立查询模板。除直接修改SQL外，hint还可通过sqlpatch能力进行添加。

gsql环境下，可以使用手动创建查询模板的模式进行：

```
prepare test_stmt as select /*+ choose_adaptive_gplan */ * from tab where c1 = $1;
```

2.13.2.4 最佳实践

多索引自适应选择支持，举例如下：

```
create table t1(c1 int, c2 int, c3 int, c4 varchar(32), c5 text);
create index t1_idx2 on t1(c1,c2,c3,c4);
create index t1_idx1 on t1(c1,c2,c3);

insert into t1( c1, c2, c3, c4, c5) SELECT (random()*(2*10^9))::integer , (random()*(2*10^9))::integer,
(random()*(2*10^9))::integer, (random()*(2*10^9))::integer,  repeat('abc', i%10) ::text from
generate_series(1,1000000) i;
insert into t1( c1, c2, c3, c4, c5) SELECT (random()*1)::integer, (random()*1)::integer, (random()*1)::integer,
(random()*(2*10^9))::integer, repeat('abc', i%10) ::text from generate_series(1,1000000) i;
```

性能对比：

随机参数：c1~ random(1, 20); c2~ random(1, 20); c3~ random(1, 20); c4 ~ random(2, 10000)

线程数50，客户端50，执行时长60s

方法	语句	tps
gplan	prepare k as select * from t1 where c1=$1 and c2=$2 and c3=$3 and c4=$4;	35126
cplan	prepare k as select /*+ use_cplan */ * from t1 where c1=$1 and c2=$2 and c3=$3 and c4=$4;	75817
aplan	prepare k as select /*+ choose_adaptive_gplan */ * from t1 where c1=$1 and c2=$2 and c3=$3 and c4=$4;	175681

2.13.2.5 常见问题处理

对于过于复杂的慢查询由于特征范围限制，可能无法使用本特性正确进行计划选择，建议直接使用CPLAN进行查询计划生成。

2.13.3 自适应代价估计

2.13.3.1 概述

本特性主要是利用真实查询反馈来纠正已有的代价模型和基数估计模型。

 版权所有 © 华为云计算技术有限公司

- 反馈基数估计：利用UMM模型和哈希匹配建模历史负载和选择率之间的映射关系，用于解决join算子和scan算子由于统计偏差导致的基数估计误差大问题，解决在此场景下查询计划质量提升，降低查询时延。
- 反馈代价矫正：利用查询负载算子反馈及时获得对于算子执行时间感知，发现误差较大时，针对代价参数进行拟合调整，从而获得更准确的代价评估，针对由于代价模型不合理导致的慢SQL，提高查询效率。

2.13.3.2 前置条件

数据库运行正常、基数反馈开启、数据库查询解析正常进入计划优化阶段。

2.13.3.3 使用指导

打开参数enable_adaptive_cost，其他参数使用默认设置即可开启本功能，如要调整其他参数已获得更好效果，详情请参见《管理员指南》中“配置运行参数 -> AI特性”章节。

2.13.3.4 最佳实践

反馈基数估计

步骤1 数据库启动，开启参数enable_adaptive_cost，其他参数保持默认设置。

```
set enable_adaptive_cost=on;
```

步骤2 打开计时功能。

```
\timing on
```

步骤3 执行由于错误的基数估计导致次优计划的语句，如含有多种连接路径和连接类型的查询SQL，观察执行计划在多次执行中是否迭代出更优计划。以下面语句举例：

```
EXPLAIN (ANALYZE, ADAPTCOST) SELECT MIN(mc.note) AS production_note,
MIN(t.title) AS movie_title,
MIN(t.production_year) AS movie_year
FROM company_type AS ct,
info_type AS it,
movie_companies AS mc,
movie_info_idx AS mi_idx,
title AS t
WHERE ct.kind = 'production companies'
AND it.info = 'bottom 10 rank'
AND mc.note NOT LIKE '%(as Metro-Goldwyn-Mayer Pictures)%'
AND t.production_year BETWEEN 2003 AND 2010
AND ct.id = mc.company_type_id
AND t.id = mc.movie_id
AND t.id = mi_idx.movie_id
AND mc.movie_id = mi_idx.movie_id
AND it.id = mi_idx.info_type_id
```

步骤4 执行三遍，第三遍执行计划相较第一遍性能提高1000倍左右。在NORMAL模式下可以用EXPLAIN（ADAPTCOST）关键词执行语句，查看每个算子对应的基数估计模型以及估计的基数。查看系统表gs_abo_model_statistic中对应模型的统计信息情况。

----结束

反馈代价矫正

步骤1 数据库启动，开启参数enable_adaptive_cost。

```
SET enable_adaptive_cost=on;
```

 版权所有 © 华为云计算技术有限公司

步骤2 设置adaptive_costest_strategy为L1，即信任默认的基数估计。

```
SET adaptive_costest_strategy="L1";
```

步骤3 默认收集数据的滑动窗口长度设置为5（即训练集5，测试集5），TPCH执行2轮后可累计收集10以上个算子信息，即可纠正计划，获得1倍左右的性能提升。

```
SET cost_update_window_size=5;
```

步骤4 启动数据库，使用gs_costmodel_calibration_manual()手动触发代价收集和代价模型更新，执行SQL（来源于TPCH Q10，数据规模1GB数据量）。

```
gaussdb=# SELECT gs_costmodel_calibration_manual();
gaussdb=# EXPLAIN (analyze,adaptcost,buffers) SELECT
        c_custkey,
        c_name,
        sum(l_extendedprice * (1 - l_discount)) as revenue,
        c_acctbal,
        n_name,
        c_address,
        c_phone,
        c_comment
FROM
        customer,
        orders,
        lineitem,
        nation
WHERE
        c_custkey = o_custkey
        and l_orderkey = o_orderkey
        and o_orderdate >= date '1994-10-01'
        and o_orderdate < date '1994-10-01' + interval '3' month
        and l_returnflag = 'R'
        and c_nationkey = n_nationkey
GROUP BY
        c_custkey,
        c_name,
        c_acctbal,
        c_phone,
        n_name,
        c_address,
        c_comment
ORDER BY
        revenue desc
LIMIT 100;
```

- 原计划

```
Limit
->  Sort
Sort Key: (sum((lineitem.l_extendedprice * (1::numeric - lineitem.l_discount)))) DESC
Sort Method: top-N heapsort  Memory: 80kB
->  HashAggregate
Group By Key: customer.c_custkey, customer.c_name, customer.c_acctbal, customer.c_phone,
nation.n_name, customer.c_address, customer.c_comment
->  Hash Join
Hash Cond: (customer.c_nationkey = nation.n_nationkey)
->  Hash Join
Hash Cond: (orders.o_custkey = customer.c_custkey)
->  Hash Join
Hash Cond: (lineitem.l_orderkey = orders.o_orderkey)
->  Seq Scan on lineitem
Filter: (l_returnflag = 'R'::bpchar)
Rows Removed by Filter: 4522345
->  Hash
Buckets: 65536  Batches: 1  Memory Usage: 2208kB
->  Seq Scan on orders  (cost=1.20..48704.00 rows=56336 width=8)
Filter: ((o_orderdate >= '1995-01-01 00:00:00'::timestamp(0) without time zone) AND (o_orderdate <
'1995-04-01 00:00:00'::timestamp without time zone))
Rows Removed by Filter: 1443494
->  Hash
Buckets: 262144  Batches: 1  Memory Usage: 26813kB
```

 版权所有 © 华为云计算技术有限公司

```
->  Seq Scan on customer
->  Hash
Buckets: 32768  Batches: 1  Memory Usage: 2kB
->  Seq Scan on nation
Total runtime: 5968.483 ms
```

- 新计划

```
Limit
->  Sort
Sort Key: (sum((lineitem.l_extendedprice * (1::numeric - lineitem.l_discount)))) DESC
Sort Method: top-N heapsort  Memory: 77kB
->  HashAggregate
Group By Key: customer.c_custkey, customer.c_name, customer.c_acctbal, customer.c_phone,
nation.n_name, customer.c_address, customer.c_comment
->  Hash Join
Hash Cond: (customer.c_nationkey = nation.n_nationkey)
->  Hash Join
Hash Cond: (orders.o_custkey = customer.c_custkey)
->  Nested Loop
->  Seq Scan on orders
Filter: ((o_orderdate >= '1994-10-01 00:00:00'::timestamp(0) without time zone) AND (o_orderdate <
'1995-01-01 00:00:00'::timestamp without time zone))
Rows Removed by Filter: 1442781
->  Index Scan using lineitem_pkey on lineitem
Index Cond: (l_orderkey = orders.o_orderkey)
Filter: (l_returnflag = 'R'::bpchar)
Rows Removed by Filter: 114403
->  Hash
Buckets: 262144  Batches: 1  Memory Usage: 26813kB
->  Seq Scan on customer
->  Hash
Buckets: 32768  Batches: 1  Memory Usage: 2kB
->  Seq Scan on nation
Total runtime: 2826.489 ms
```

原计划，总执行时间5968ms；新计划，总执行时间2622ms。

当两表连接的基数在HashJoin和NestLoop的决策边界时，代价矫正可以起到效果，选择更好的连接方式，否则一般不改变计划。

----结束

2.13.3.5 常见问题处理

1. 使用本特性进行计划生成，对于同一种查询前几次的计划不同，属于正常现象，收敛之后会获得更优计划。
2. 使用缓存计划不会触发本特性属于正常现象，无参数优化不会查找任何反馈模型。

2.14 Foreign Data Wrapper

GaussDB的FDW（Foreign Data Wrapper）可以实现各个GaussDB数据库及远程服务器（包括数据库、文件系统）之间的跨库操作。目前支持的外部数据封装器类型包括file_fdw。

2.14.1 file_fdw

file_fdw模块提供了外部数据封装器file_fdw，可以用来在服务器的文件系统中访问数据文件。数据文件必须是COPY FROM可读的格式，具体请参见《开发者指南》中“SQL参考 > SQL语法 > COPY”章节。使用file_fdw访问的数据文件是当前可读的，不支持对该数据文件的写入操作。

 版权所有 © 华为云计算技术有限公司

当前GaussDB会默认编译file_fdw，initdb的时候会在pg_catalog schema中创建该插件。

file_fdw对应的server和外表只允许数据库的初始用户、系统管理员或开启运维模式时的运维管理员创建。

说明

- 为防止对服务端文件任意读，系统管理员及运维模式下运维管理员使用时会受enable_copy_server_files及safe_data_path两个参数的控制。
- 当参数enable_copy_server_files关闭时，只允许初始用户创建file_fdw类型外表，打开时系统管理员及运维模式下运维管理员可以创建file_fdw类型外表，以防止用户越权查看或修改敏感文件。
- enable_copy_server_files打开时，管理员可以通过guc参数safe_data_path设置用户可以导入导出的路径校验，文件必须为safe_data_path路径的子路径，未设置此guc参数时候（默认情况），不对用户使用的路径进行拦截。

使用file_fdw创建的外部表可以有下列选项：

- filename

  指定要读取的文件，必需的参数，且必须是一个绝对路径名。
- format

  远端server的文件格式，支持text/csv/binary三种格式，和COPY语句的FORMAT选项相同。
- header

  指定的文件是否有标题行，与COPY语句的HEADER选项相同。
- delimiter

  指定文件的分隔符，与COPY的DELIMITER选项相同。
- quote

  指定文件的引用字符，与COPY的QUOTE选项相同。
- escape

  指定文件的转义字符，与COPY的ESCAPE选项相同。
- null

  指定文件的null字符串，与COPY的NULL选项相同。
- encoding

  指定文件的编码，与COPY的ENCODING选项相同。
- force_not_null

  这是一个布尔选项。如果为真，则声明字段的值不应该匹配空字符串（也就是，文件级别null选项）。与COPY的 FORCE_NOT_NULL选项里的字段相同。

说明

- file_fdw不支持COPY的OIDS和 FORCE_QUOTE选项。
- 这些选项只能为外部表或外部表的字段声明，不是file_fdw的选项，也不是使用file_fdw的服务器或用户映射的选项。
- 修改表级别的选项需要系统管理员权限。因为安全原因，只有系统管理员能够决定读取的文件。
- 对于一个使用file_fdw的外部表，EXPLAIN可显示要读取的文件名和文件大小（单位：字节）。 指定了COSTS OFF关键字之后，不显示文件大小。

 版权所有 © 华为云计算技术有限公司

使用 file_fdw

- 创建服务器对象：CREATE SERVER
- 创建用户映射：CREATE USER MAPPING
- 删除用户映射：DROP USER MAPPING
- 删除服务器对象：DROP SERVER

注意事项

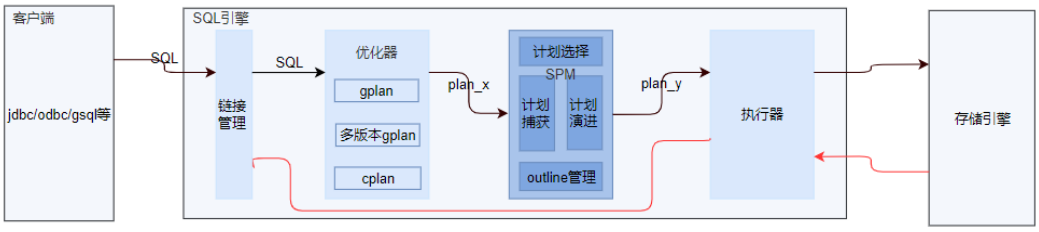
- 使用file_fdw需要指定要读取的文件，请先准备好该文件，并授予数据库对该文件的读取权限。

2.15 SPM 计划管理

SPM 整体流程

业务数据的变化、数据库版本的升级等场景可能导致SQL的执行计划发生变化，且这种变化对SQL的执行可能带来性能的正向收益，同样也可能带来显著的性能劣化。为了将这种不确定的变化变为确定正向的变化，需要增加一个SPM（SQL Plan Management）组件。如**图1 SPM整体流程**所示，SPM处于SQL引擎中，且处于SQL引擎的两大核心组件优化器和执行器之间。

图 2-12 SPM 整体流程



结合上图，对SPM的关键组成介绍如下：

1. Outline管理组件：

   Outline管理是SPM中最基础的组件，主要负责将一个具体的计划转化为一个具体的Hint集合，或者将一个具体的Hint集合转换为一个具体的计划。对于一个具体计划，它的Outline是一组能够完全确定该计划的Hint集合。

2. 计划捕获组件：

   对于一个具体SQL，该组件将优化器给出的计划（如上图中的plan_x）以outline的形式落盘，并将该SQL第一条计划标记为ACC（ACCEPTED的简称）状态，其他计划标记为UNACC（UNACCEPTED的简称）状态，以备后续使用。

3. 计划选择组件：

   计划选择组件用于决策是否将优化器给出的执行计划交给执行器去执行。如果SPM认为该计划存在性能劣化的风险，SPM会在ACC/FIXED状态的计划中再选择一个计划（如上图中的plan_y，plan_y可能等于plan_x，也可能不相等）交给执行器去执行。

4. 计划演进组件

   计划捕获+计划选择组件可以将交给执行器的计划固定下来，计划演进组件是将优化器新产生的计划（已被计划捕获组件固化下来，但为UNACC状态的计划）进行

版权所有 © 华为云计算技术有限公司

优秀程度的判定，如果被判定的计划符合优秀的判定标准，则可以将其调整为ACC状态，以备计划选择使用。

SPM 基本使用

SPM计划管理在不同场景中有一些差异，因此在介绍SPM计划管理基本使用之前，需要说明以下几点：

- GUC参数spm_enable_plan_capture取值范围分别是off、auto、manual、store，其中auto和manual分别对应SPM计划捕获行为的自动计划捕获模式和手动计划捕获模式，这两种计划捕获模式的唯一区别在于：前者仅对重复出现的SQL进行捕获，后者没有这一约束。
- 不同的客户端（例如JDBC和gsql）关于normal sql与pbe sql的SPM计划管理行为是一致的。
- 由于pbe下模板SQL的cplan计划存在无法复用的问题， 因此pbe下的所有cplan计划被捕获后都会被标记为UNACC状态，第一次出现的gplan会被标记为ACC状态。
- SPM计划管理会对pbe下cplan的参数进行捕获。

在不同的场景下，除了上述中的一些差异点外，其他行为在不同的场景中的表现是一致的，下面以在gsql中normal sql的SPM计划管理手动捕获模式的基本使用为例，给出基本操作：

- 初始化数据

  初始化数据的SQL如下：

  ```
  drop table if exists tb_a cascade;
  create table tb_a(id int,c1 int, c2 int, pad text);
  create index tb_a_idx_c1 on tb_a(c1);
  insert into tb_a select id, (random()*200)::int, (random()*10000)::int, 'ss' from (select
  generate_series(1,10000) id) tb_a;
  analyze tb_a;
  ```

- 设置前置GUC参数

  设置如下GUC参数的目的均在注释中已经体现。

  ```
  -- 开启SPM计划捕获
  set spm_enable_plan_capture=manual;
  -- 开启SPM计划选择
  set spm_enable_plan_selection=on;
  -- 当前SPM只支持gplan，确保生成的计划是gplan
  set plan_cache_mode = 'force_generic_plan';
  -- 在pretty模式可以看到baseline的使用情况
  set explain_perf_mode=pretty;
  -- 关闭执行器sql bypass的特殊优化
  -- 这里是off， 只是希望测试的sql执行流程更具有一般性，对SPM流程无任何影响
  set enable_opfusion=off;
  ```

- 计划捕获测试

  关键操作的说明已经在注释中体现。

  ```
  -- 捕获tablescan，确保捕获tablescan计划
  set enable_seqscan=on;
  set enable_indexscan=off;
  set enable_bitmapscan=off;
  -- 执行测试sql，结果下如下，可以看到当前sql并没有使用任何baseline计划。
  prepare spm_query as select * from tb_a where c1 = $1;
  explain(costs off) execute spm_query(1);
   id |      operation
  ----+-----------------------
    1 | ->  Seq Scan on tb_a
  (1 row)
  ```

 版权所有 © 华为云计算技术有限公司

```
 Predicate Information (identified by plan id)
-----------------------------------------------
   1 --Seq Scan on tb_a
         Filter: (c1 = $1)
(2 rows)
-- 查看baseline，可以看到tablescan计划被捕获，且状态为ACC
select sql_hash, plan_hash, outline, status, gplan from gs_spm_sql_baseline where sql_text like '%tb_a
where c1 = $1%';
 sql_hash  | plan_hash  |                outline                 | status | gplan
-----------+------------+----------------------------------------+--------+-------
 982135085 | 4251425169 |  begin_outline_data                   +| ACC    | t
           |            |  TableScan(@"sel$1" public.tb_a@"sel$1")+|        |
           |            |  version("1.0.0")                     +|        |
           |            |  end_outline_data                      |        |
(1 row)
-- 捕获indexscan确保捕获indexscan计划
set enable_indexscan=on;
set enable_seqscan=off;
set enable_bitmapscan=off;
-- 执行测试sql，可以看到计划仍然为tablescan(baseline中ACC的计划)，因为从第二条计划开始计划被标
记为UNACC状态，UNACC状态的计划是不能被使用的。
deallocate spm_query;
prepare spm_query as select * from tb_a where c1 = $1;
explain(costs off) execute spm_query(1);
 id |      operation
----+----------------------
  1 | ->  Seq Scan on tb_a
(1 row)

 Predicate Information (identified by plan id)
-----------------------------------------------
   1 --Seq Scan on tb_a
         Filter: (c1 = $1)
(2 rows)

                ====== Query Others =====
-------------------------------------------------------------------
 use_baseline: Yes, sql_hash: 982135085, plan_hash: 4251425169
(1 row)
-- 查看baseline，可以看到indexscan的计划被捕获，且状态为UNACC
select sql_hash, plan_hash, outline, status, gplan from gs_spm_sql_baseline where sql_text like '%tb_a
where c1 = $1%';
 sql_hash  | plan_hash  | outline                                                | status | gplan
-----------+------------+--------------------------------------------------------+--------+-------
 982135085 | 4251425169 |  begin_outline_data                                   +| ACC    | t
           |            |  TableScan(@"sel$1" public.tb_a@"sel$1")              +|        |
           |            |  version("1.0.0")                                     +|        |
           |            |  end_outline_data                                      |        |
 982135085 |  808368919 |  begin_outline_data                                   +| UNACC  | t
           |            |  IndexScan(@"sel$1" public.tb_a@"sel$1" tb_a_idx_c1)+|        |
           |            |  version("1.0.0")                                     +|        |
           |            |  end_outline_data                                      |        |
(2 rows)
-- 捕获bitmapscan，确保捕获bitmapscan计划
set enable_bitmapscan=on;
set enable_seqscan=off;
set enable_indexscan=off;
-- 执行测试sql
deallocate spm_query;
prepare spm_query as select * from tb_a where c1 = $1;
explain(costs off) execute spm_query(1);
 id |      operation
----+----------------------
  1 | ->  Seq Scan on tb_a
(1 row)

 Predicate Information (identified by plan id)
-----------------------------------------------
```

 版权所有 © 华为云计算技术有限公司

```
  1 --Seq Scan on tb_a
        Filter: (c1 = $1)
(2 rows)

                ====== Query Others =====
-------------------------------------------------------------
 use_baseline: Yes, sql_hash: 982135085, plan_hash: 4251425169
(1 row)
-- 查看baseline，可以看到bitmapscan的计划被捕获，且状态位UNACC
select sql_hash, plan_hash, outline, status, gplan from gs_spm_sql_baseline where sql_text like '%tb_a
where c1 = $1%';
 sql_hash  | plan_hash  |                      outline                      | status | gplan
-----------+------------+---------------------------------------------------+--------+-------
 982135085 | 4251425169 |  begin_outline_data                               +| ACC    | t
           |            |  TableScan(@"sel$1" public.tb_a@"sel$1")          +|        |
           |            |  version("1.0.0")                                 +|        |
           |            |  end_outline_data                                 |        |
 982135085 |  808368919 |  begin_outline_data                               +| UNACC  | t
           |            |  IndexScan(@"sel$1" public.tb_a@"sel$1" tb_a_idx_c1) +|        |
           |            |  version("1.0.0")                                 +|        |
           |            |  end_outline_data                                 |        |
 982135085 |  930064183 |  begin_outline_data                               +| UNACC  | t
           |            |  BitmapScan(@"sel$1" public.tb_a@"sel$1" tb_a_idx_c1)+|        |
           |            |  version("1.0.0")                                 +|        |
           |            |  end_outline_data                                 |        |
(3 rows)
```

- 计划选择测试

  在上面的测试中可以发现，无论如何调整GUC参数对计划的生成进行预，计划最终的选择都是ACC状态的tablescan,这说明计划选择不会使用UNACC状态的计划。

```
-- 测试优先使用FIXED状态的计划
-- 将上方indexscan计划的状态改为ACC，seqscan修改为FIXED状态，并查看baseline，
-- 可以发现状态修改成功。
select * from dbe_sql_util.gs_spm_set_plan_status(982135085, 4251425169, 'FIXED');
select * from dbe_sql_util.gs_spm_set_plan_status(982135085, 808368919, 'ACC');
select sql_hash, plan_hash, outline, status, gplan, cost from gs_spm_sql_baseline where sql_text like
'%tb_a where c1 = $1%' order by creation_time;
 sql_hash  | plan_hash  |                      outline                      | status | gplan |  cost
-----------+------------+---------------------------------------------------+--------+-------+---------
 982135085 | 4251425169 |  begin_outline_data                               +| FIXED  | t     |     167
           |            |  TableScan(@"sel$1" public.tb_a@"sel$1")          +|        |       |
           |            |  version("1.0.0")                                 +|        |       |
           |            |  end_outline_data                                 |        |       |
 982135085 |  808368919 |  begin_outline_data                               +| ACC    | t     | 133.039
           |            |  IndexScan(@"sel$1" public.tb_a@"sel$1" tb_a_idx_c1) +|        |       |
           |            |  version("1.0.0")                                 +|        |       |
           |            |  end_outline_data                                 |        |       |
 982135085 |  930064183 |  begin_outline_data                               +| UNACC  | t     |  49.467
           |            |  BitmapScan(@"sel$1" public.tb_a@"sel$1" tb_a_idx_c1)+|        |       |
           |            |  version("1.0.0")                                 +|        |       |
           |            |  end_outline_data                                 |        |       |
(3 rows)
-- 确保优化器生成的计划是bitmapscan
set enable_bitmapscan=on;
set enable_seqscan=off;
set enable_indexscan=off;
-- 执行SQL发现，优化器仍然选择了FIXED状态的tablescan，而不是选择了代价更小且状态位ACC状态的
indexscan。
deallocate spm_query;
prepare spm_query as select * from tb_a where c1 = $1;
explain(costs off) execute spm_query(1);
 id |      operation
----+----------------------
  1 | ->  Seq Scan on tb_a
(1 row)

 Predicate Information (identified by plan id)
```

 版权所有 © 华为云计算技术有限公司

```
-----------------------------------------------
  1 --Seq Scan on tb_a
        Filter: (c1 = $1)
(2 rows)

                ====== Query Others =====
-----------------------------------------------------------------
 use_baseline: Yes, sql_hash: 982135085, plan_hash: 4251425169
(1 row)
```

- 计划演进测试

```
-- 演进bitmapscan计划，并查看计划评估结果
select * from dbe_sql_util.gs_spm_evolute_plan(982135085, 930064183);
select sql_hash, plan_hash, better, refer_plan, reason from gs_spm_sql_evolution where
sql_hash=982135085;
 sql_hash  | plan_hash | better | refer_plan |                                reason
-----------+-----------+--------+------------
+-------------------------------------------------------------------------------
 982135085 | 930064183 | t      |  808368919 | target plan execution time:0.448333, refer plan
execution time: 0.660667
(1 row)
-- 通过上方演进的结果可以看出bitmapscan（plan_hash）的执行时间远小于indexscan（refer_plan）的执
行时间。并且计划演进给出的结论(better==t)表示bitmapscan是可以被接受的。
-- 根据演进结论修改bitmapscan计划状态为ACC
select * from dbe_sql_util.gs_spm_set_plan_status(982135085, 930064183, 'ACC');
-- 确保优化器生成的计划是bitmapscan
set enable_bitmapscan=on;
set enable_seqscan=off;
set enable_indexscan=off;
-- 执行SQL语句，可以看到，优化器推荐的bitmapscan可以被正常放行使用。
deallocate spm_query;
prepare spm_query as select * from tb_a where c1 = $1;
explain(costs off) execute spm_query(1);
 id |                operation
----+-------------------------------------------
  1 | ->  Bitmap Heap Scan on tb_a
  2 |    ->  Bitmap Index Scan using tb_a_idx_c1
(2 rows)

 Predicate Information (identified by plan id)
-----------------------------------------------
  1 --Bitmap Heap Scan on tb_a
        Recheck Cond: (c1 = $1)
  2 --Bitmap Index Scan using tb_a_idx_c1
        Index Cond: (c1 = $1)
(4 rows)

              ====== Query Others =====
-----------------------------------------------------------------
 use_baseline: Yes, sql_hash: 982135085, plan_hash: 930064183
(1 row)
```

- 清理数据

  清理数据的SQL如下：

```
drop table tb_a;
DROP TABLE
drop index tb_a_idx_c1;
DROP INDEX
```

2.16 敏感数据发现

敏感数据发现功能提供函数gs_sensitive_data_discovery()和gs_sensitive_data_discovery_detail()，通过调用的不同函数，指定扫描对象和敏感数据分类器，得到对应扫描对象不同明细级别的敏感数据信息。

 版权所有 © 华为云计算技术有限公司

使用敏感数据发现函数扫描数据

步骤1 创建扫描对象。

```
gaussdb=# CREATE SCHEMA test_schema1;
gaussdb=# CREATE TABLE test_schema1.normal_table1 (email VARCHAR(50), phonenum text, creditcard
bytea, name BPCHAR(50), encryted_data NVARCHAR2(500));
gaussdb=# INSERT INTO test_schema1.normal_table1 VALUES ('xxx@xxx.com', '13345678901',
'4951-7843-2960-0886', '高斯',  'r1Pj4DKQopSmZfSr/mXwieGcrE+O2iX3G04w1Tda0zqX/pD/
Az90tLuvy0ZnCmUg0/Kl/g89ZqxxAZzta8jQ6bvicRpdXrDCQTHAfsXYSEScB9Idqtwh7kdm1LHq89qs');
```

步骤2 调用敏感数据发现函数扫描对象。

- 调用gs_sensitive_data_discovery()函数，扫描目标数据，返回统计的扫描结果。

```
gaussdb=# SELECT * FROM gs_sensitive_data_discovery('test_schema1.normal_table1',  'all');
           target           | percentage_of_sensitive_data |
classifier_type_distribution
----------------------------+------------------------------
+-----------------------------------------------------------------------------------------
 test_schema1.normal_table1 | 5/5                          |  Email: 1/5  CreditCard: 1/5  PhoneNumber: 1/5
ChineseName: 1/5  EncryptedContent: 1/5
(1 row)
```

- 调用gs_sensitive_data_discovery_detail()函数，扫描目标数据，返回详细的扫描结果。

```
gaussdb=# SELECT * FROM gs_sensitive_data_discovery_detail('test_schema1',  'all');
   schema    |     table     |    column     |    classifier    | laws_and_regulations
-------------+---------------+---------------+------------------+----------------------
 test_schema1 | normal_table1 | email         | Email            | GDPR
 test_schema1 | normal_table1 | phonenum      | PhoneNumber      | GDPR
 test_schema1 | normal_table1 | creditcard    | CreditCard       | GDPR
 test_schema1 | normal_table1 | name          | ChineseName      | GDPR
 test_schema1 | normal_table1 | encryted_data | EncryptedContent | GDPR
(5 rows)
```

步骤3 清理数据。

```
gaussdb=# DROP TABLE test_schema1.normal_table1;
gaussdb=# DROP SCHEMA test_schema1;
```

----结束

2.17 计划内应用无损透明

GNS（GaussDB Notification Service）组件介绍

为实现GaussDB计划内维护对应用程序的透明性，新增GNS服务。当应用程序调用JDBC接口首次向数据库实例中的任意节点建立连接时，JDBC驱动会与GNS服务建立数据库实例状态订阅链路。当GNS检测到数据库实例状态发生变化，通过订阅链路将状态变化事件发送给JDBC驱动，事件处理线程收到任务后，对受到影响的连接进行管理和迁移。

当数据库运维平台下发计划内维护操作（DN主备倒换、重启数据库实例、重启节点、重启DN）时，若开启计划内ALT选项，OM相关接口将调用gns_ctl命令完成该次操作（而不是cm_ctl）；gns_server将通知JDBC客户端数据库实例即将进行变更，JDBC客户端将继续向服务端发送正在运行的事务、而暂存新事务，不再向数据库发送，待正在运行的事务完成后挂起和DN的连接。事务运行结束后，JDBC向GNS发送确认消息，并在维护完成后，JDBC开始恢复与DN的连接。

GNS组件仅用于计划内ALT特性，GNS组件的关键组成如下：

1. gns_server进程：

 版权所有 © 华为云计算技术有限公司

gns_server进程调用CM组件命令，实时获取DN状态变更，并通知所有的已连接JDBC客户端。gns_server资源由CM组件进行管理。cm_agent每间隔1s尝试启动一次gns_server，若五次启动失败后，需要手动启动资源，命令为：cm_ctl start 。间隔和启动次数可用cm_ctl res --edit命令修改，例如：

```
cm_ctl res --edit --res_name=GNS --
res_attr=check_interval=1,time_out=10,restart_delay=1,restart_period=1,restart_times=5
```

详情请参见《工具参考》中“统一数据库管理工具 > cm_ctl工具介绍”章节。

2. gns_ctl工具：

   供OM模块调用，实际执行ALT模式下的计划内维护（当前仅支持DN主备倒换、重启数据库实例、重启节点、重启DN）。

前置配置

GNS组件的配置文件信息如下：

表 2-44 GNS 配置文件

配置项	取值范围	描述
gns_listen_address	合法IPv4地址，默认值为127.0.0.1。	当前节点GNS进程的数据面IP地址。
gns_bind_addresses	合法IPv4地址列表，以分号分割，默认值为127.0.0.1。	gns侦听的ip，和DN配置文件中的listen_addresses一致。
gns_port	可用合法端口值，默认值为CMS的端口号+100，若超出65535，则默认为CMS的端口号-100。GNS的数据目录与CM的数据目录同级。	当前节点GNS进程的数据面端口。
g_log_min_level	DEBUG5，DEBUG1，WARNING，ERROR，LOG，FATAL，默认值为WARNING。	日志级别。
log_file_size	1MB~10MB，默认值为2MB。	单个日志文件大小，单位为MB。
all_gns_addresses	合法IPv4地址列表，以分号分割，默认值为127.0.0.1;127.0.0.1;127.0.0.1。	所有GNS进程的管理面IP地址，用于gns_ctl连接。
all_gns_ports	可用合法端口值列表，以分号分割，默认值为25100;25100;25100。	所有GNS进程的管理面端口，用于gns_ctl连接。
connection_timeout	0-5，默认值为5。	连接请求最长时间，单位为秒。

版权所有 © 华为云计算技术有限公司

配置项	取值范围	描述
alt_max_time	1~10，默认值为10。	JDBC事务排干的最大时间，单位为秒。
jdbc_max_waiting_time	1~120，默认值为120。	计划内命令执行场景下，JDBC的最大等待时间，单位为秒。
max_io_threads	2或4，默认值为2。	消息收发线程数。
max_work_threads	2或4，默认值为2。	消息处理线程数。
dn_check_interval	100-3000，默认值为100。	查询DN状态时间间隔，单位ms。
ssl	on/off，默认值为on。	是否开启SSL。
ssl_ca_file	linux合法文件路径，默认值为cacert.pem。	CA服务器的根证书。此参数可选择配置，需要验证客户端证书的合法性时才需要配置。 请以实际的CA服务器根证书名称为准。
ssl_cert_file	linux合法文件路径，默认值为server.crt。	指定服务器证书文件（采用国密认证则为签名证书），包含服务器端的公钥。服务器证书用以表明服务器身份的合法性，公钥将发送给对端用来对数据进行加密。 请以实际的证书名为准，其相对路径是相对于数据目录的。
ssl_key_file	linux合法文件路径默认值为server.key。	指定服务器私钥文件（采用国密认证则为签名证书私钥），用以对公钥加密的数据进行解密。 请以实际的服务器私钥名称为准，其相对路径是相对于数据目录的。
ssl_crl_file	linux合法文件路径，默认值为空，表示没有吊销列表。	证书吊销列表，如果客户端证书在该列表中，则当前客户端证书被视为无效证书。 请以实际的证书吊销列表名称为准。
ssl_ciphers	默认值：ALL，表示允许对端使用产品支持的所有加密算法，但不包含ADH、LOW、EXP、MD5算法。	SSL通讯使用的加密算法。
ssl_cert_notify_time	SSL服务器证书到期前提醒的天数。	请按照需求配置证书过期前提醒天数。 默认值：90

版权所有 © 华为云计算技术有限公司

配置项	取值范围	描述
gns_data_path	linux合法文件路径，默认值为CmDataDir/../gns_server。	GNS的数据路径，为CM数据路径同级。
gns_hba_path	linux合法文件路径，默认值为gns_data_path。	GNS的白名单文件路径。
log_dir	linux合法文件路径，默认值为$GAUSSLOG/gns。	GNS的日志文件路径。

说明

- 可通过gs_guc命令修改上述参数的值，其中gns_listen_address、gns_bind_addresses、gns_port、all_gns_adresses、all_gns_ports、gns_data_path、gns_hba_path、log_dir参数值修改需要重启gns_server后才能生效。详见《工具参考》中“服务端工具 > gs_guc”章节中参数-Z。
- 由于gns_server为轻量级进程、与客户端交互消息量不大，在连接1024个客户端情况下，1s内处理消息数量在1w以内、为毫秒级处理，故无需配置过多I/O和worker线程。因此为尽量降低影响，CPU核数不大于8时，max_io_threads和max_work_threads的值建议设置为2；若CPU核数大于8时，max_io_threads和max_work_threads的值建议设置为4。但在建连阶段，由于需要到DN鉴权，最多支持100个客户端同时向同一个gns_server发起连接请求，否则可能出现连接超时现象，需要重新尝试连接。

基本使用

使用前请确认数据库实例为可用状态。

本文档使用Benchmark工具模拟业务场景，也可以通过Java程序客户端连接数据库，具体配置方法参见《开发者指南》中“应用程序开发教程 > 基于JDBC开发 > 示例：不同场景下连接数据库配置”章节中“应用无损透明场景”。

- 前置条件

  数据库实例正常运行、前置安装无问题，benchmark工具或Java进程连接url配置正确。

cm_ctl query -Cv显示GNS状态为Online

```
[omm@ac001 dn_01_master]$ cm_ctl query -Cv
[  CMServer State   ]

node     instance state
--------------------------
1  ac001 1        Standby
2  ac002 2        Primary
3  ac003 3        Standby

[    ETCD State     ]

node     instance state
--------------------------------
1  ac001 7001     StateFollower
2  ac002 7002     StateLeader
3  ac003 7003     StateFollower


[ Defined Resource State ]
```

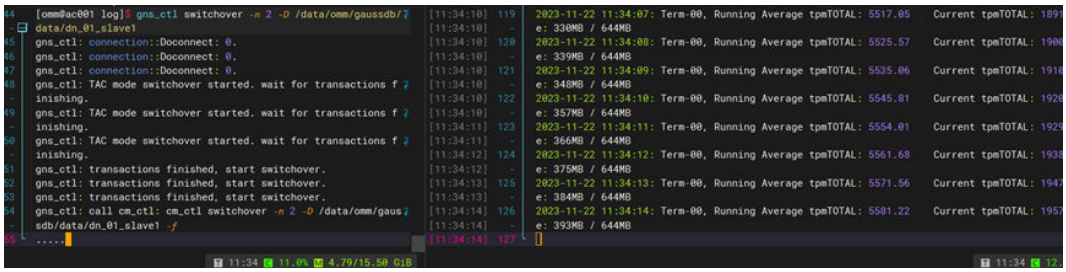

版权所有 © 华为云计算技术有限公司

```
node     res_name instance  state
--------------------------------------
1  ac001 GNS      20001     OnLine
2  ac002 GNS      20002     OnLine
3  ac003 GNS      20003     OnLine

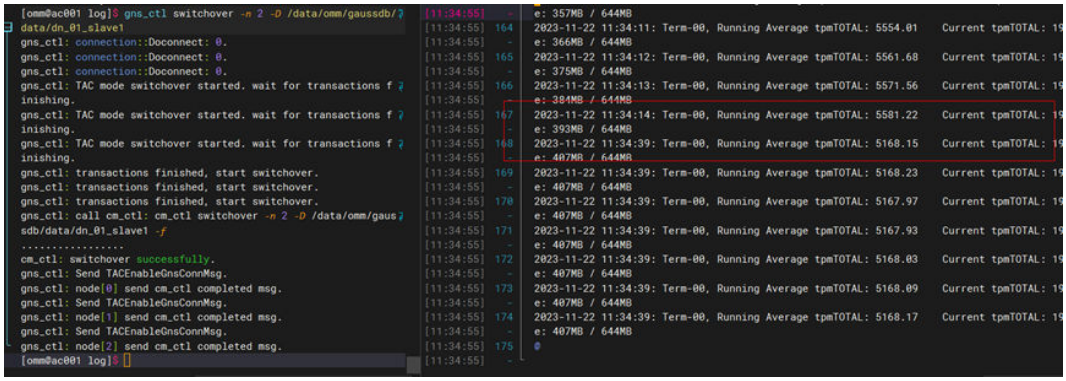
...
```

步骤1 使用benchmark工具运行TPCC，连接数据库。

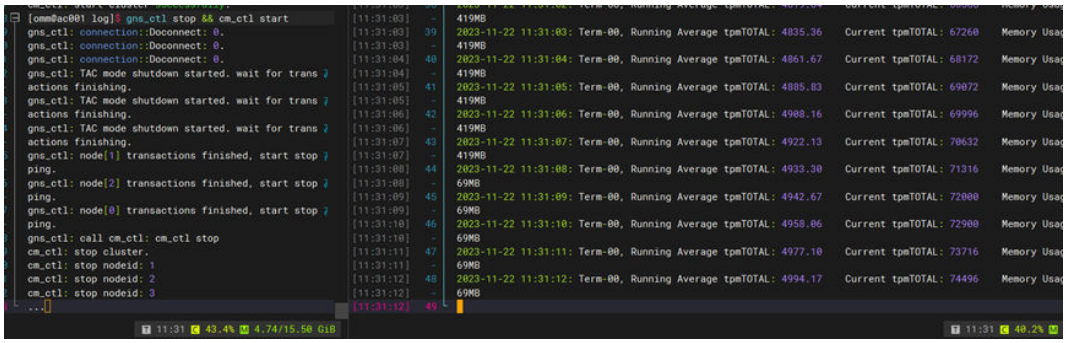
步骤2 使用gns_ctl命令进行主备切换，TPCC停止运行，但无报错（gns_ctl工具使用参考《工具参考》中“系统内部调用的工具 > gns_ctl”章节）。



主备切换后TPCC正常恢复运行。

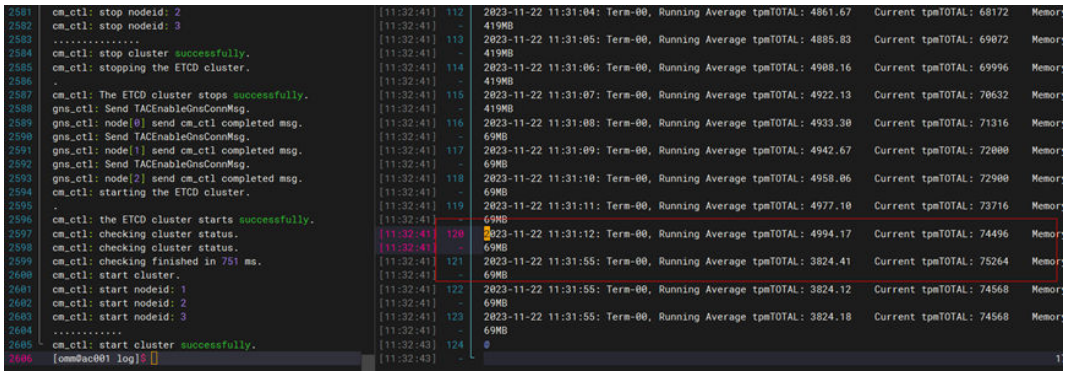


步骤3 使用gns_ctl命令进行数据库实例重启，TPCC停止运行，但无报错。

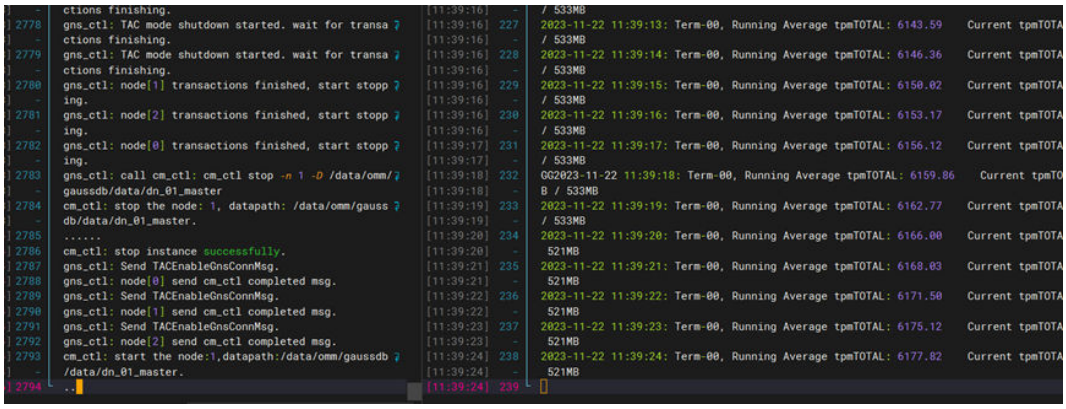


数据库实例重启后TPCC正常恢复运行。

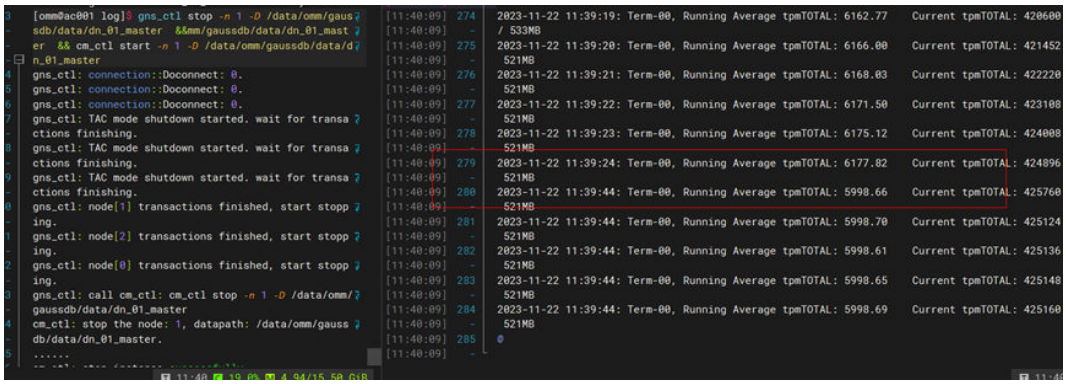
版权所有 © 华为云计算技术有限公司



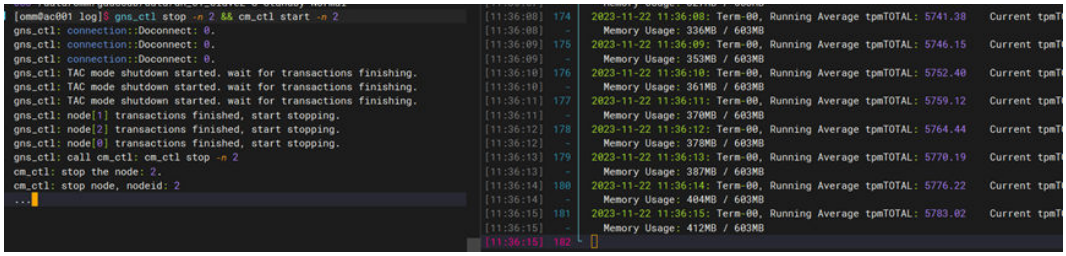
步骤4 使用gns_ctl命令进行重启DN，TPCC停止运行，但无报错。



重启DN后TPCC正常恢复运行。

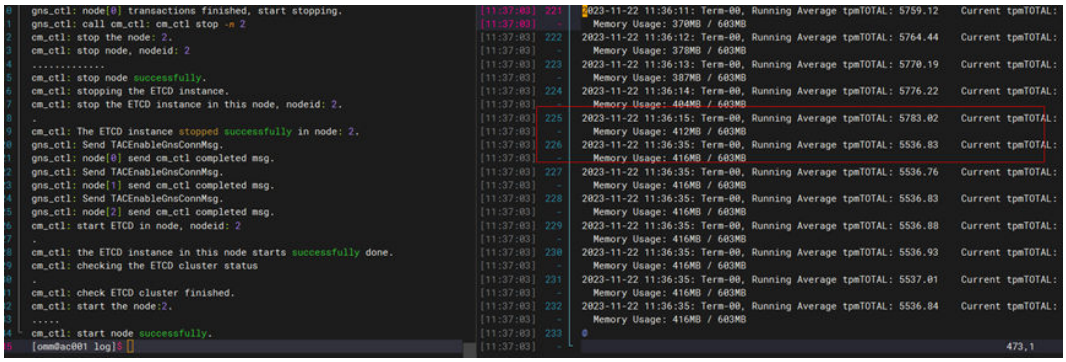


步骤5 使用gns_ctl命令进行重启节点，TPCC停止运行，但无报错。



重启节点后TPCC正常恢复运行。

版权所有 © 华为云计算技术有限公司



----结束

版权所有 © 华为云计算技术有限公司